

《Code: Terraform》 中文译本

术语采用中文（English）形式；代码、API、命令与标识符均保持原样。源文档中的嵌入图示已随页附入。

CODE: TERRAFORM

玩家文档 (PLAYER DOCUMENTATION)

构建版本 (Build) **441ad76**

目录 (Contents)

指南 (GUIDE)

分类	条目	页码
从这里开始 (Start Here)	入门 (Getting Started)	13
从这里开始 (Start Here)	第一步 (First Steps)	14
从这里开始 (Start Here)	命令结果 (Command Results)	15
从这里开始 (Start Here)	长时间运行的脚本 (Long-Running Scripts)	17
教程 (Tutorials)	游戏如何运作 (How This Game Works)	19
教程 (Tutorials)	选择你的第一条信用点路径 (Choosing Your First Credit Path)	20
教程 (Tutorials)	电力与地形改造机器 (Power & Terraforming Machines)	21
教程 (Tutorials)	第一个生物学循环 (First Biology Loop)	22
教程 (Tutorials)	第一条采集路线 (First Harvesting Route)	24
教程 (Tutorials)	合约 (Contracts)	25
教程 (Tutorials)	阅读错误与控制台输出 (Reading Errors and Console Output)	26
教程 (Tutorials)	第一个先锋 (First Pioneer)	27
教程 (Tutorials)	使用计划模式 (Using Plan Mode)	28
教程 (Tutorials)	第一个前哨站 (First Outpost)	29
教程 (Tutorials)	扩展生产 (Expanding Production)	30
编程 (Programming)	变量 (Variables)	31
编程 (Programming)	运算符 (Operators)	32
编程 (Programming)	数字 (Numbers)	34
编程 (Programming)	打印 (Print)	36
编程 (Programming)	self 与组件 (Self & Components)	37
编程 (Programming)	循环与脚本 (Loops & Scripts)	38
编程 (Programming)	匹配 / 分支 (Match / Case)	39
编程 (Programming)	转换函数 (Conversion Functions)	40
编程 (Programming)	字符串与 F 字符串 (Strings & F-strings)	41
编程 (Programming)	正则表达式 (Regular Expressions)	43
编程 (Programming)	列表与元组 (Lists & Tuples)	45
编程 (Programming)	字典 (Dictionaries)	47
编程 (Programming)	内置函数概览 (Built-in Functions Overview)	49
编程 (Programming)	异常 (Exceptions)	52
编程 (Programming)	实用工具辅助函数 (Utility Helpers)	54
编程 (Programming)	导入与库 (Imports & Libraries)	57
编程 (Programming)	文档字符串 (Docstrings)	59
编程 (Programming)	编写类 (Writing Classes)	61
编辑器与工具 (Editor & Tools)	代码编辑器 (Code Editor)	65

分类	条目	页码
编辑器与工具 (Editor & Tools)	Vim 模式 (Vim Mode)	68
编辑器与工具 (Editor & Tools)	脚本命令 (Script Commands)	69
编辑器与工具 (Editor & Tools)	脚本变体 (Script Variants)	71
编辑器与工具 (Editor & Tools)	调试模式 (Debug Mode)	73
编辑器与工具 (Editor & Tools)	控制室 (Control Room)	76
编辑器与工具 (Editor & Tools)	外部编辑器 (External Editor)	79
编辑器与工具 (Editor & Tools)	键盘快捷键 (Keyboard Shortcuts)	81
自动化系统 (Automation Systems)	合约 (Contracts)	82
自动化系统 (Automation Systems)	载具邻近与服务 (Vehicle Proximity & Service)	83
自动化系统 (Automation Systems)	地球订单 (Earth Orders)	85
自动化系统 (Automation Systems)	信号总线 (Signal Bus)	87
自动化系统 (Automation Systems)	数据存档 (Data Archive)	95
自动化系统 (Automation Systems)	地图标记 (Map Markers)	97
自动化系统 (Automation Systems)	无人机 (Drones)	99
生产与物流 (Production & Logistics)	电池与充电 (Batteries & Charging)	101
生产与物流 (Production & Logistics)	精炼与存储 (Refinement & Storage)	103
生产与物流 (Production & Logistics)	输入与输出 (Input & Output)	104
生产与物流 (Production & Logistics)	电池支架 (Battery Holder)	106
生产与物流 (Production & Logistics)	货架 (Cargo Rack)	107
生产与物流 (Production & Logistics)	便携式电池 (Portable Battery)	108
生产与物流 (Production & Logistics)	便携式存储箱 (Portable Storage Bin)	109
世界与基础设施 (World & Infrastructure)	生物圈、生物量层级 (Biosphere, Biomass Tier)	110
世界与基础设施 (World & Infrastructure)	生物圈、植物 (Biosphere, Plants)	112
世界与基础设施 (World & Infrastructure)	植物地形改造器 (Plant Terraformer)	115
世界与基础设施 (World & Infrastructure)	野生动物概览 (Wildlife, Overview)	117
世界与基础设施 (World & Infrastructure)	野生动物供应 (气体与液体) (Wildlife, Supply (Gases & Liquids))	118
世界与基础设施 (World & Infrastructure)	野生动物饲养 (Wildlife, Husbandry)	120
世界与基础设施 (World & Infrastructure)	栖息地开发 (Habitat Development)	123
世界与基础设施 (World & Infrastructure)	野生动物进程 (Wildlife, Progression)	124
世界与基础设施 (World & Infrastructure)	野生动物、生物档案 (Wildlife, Creature Profiles)	125
世界与基础设施 (World & Infrastructure)	天气系统 (Weather System)	126
世界与基础设施 (World & Infrastructure)	流动网络、流体 (Flow Networks, Fluids)	128
世界与基础设施 (World & Infrastructure)	基础设施与管道 (Infrastructure & Pipes)	134
世界与基础设施 (World & Infrastructure)	电力网络 (Power Networks)	137
世界与基础设施 (World & Infrastructure)	热能喷口 (Thermal Vents)	139
世界与基础设施 (World & Infrastructure)	三级进程 (Tier 3 Progression)	140
世界与基础设施 (World & Infrastructure)	水井与油井 (Water & Oil Wells)	143
参考 (Reference)	组件 (Components)	144

命令 (COMMANDS)

分类	条目	页码
	系统 (System)	147
	基础设施 (Infrastructure)	150

内置函数 (BUILT-IN FUNCTIONS)

条目	页码
内置函数 (Built-in Functions)	151

内置模块 (BUILT-IN MODULES)

模块	页码
random	187
functools	189
re	191
dataclasses	196

语言 (LANGUAGE)

分类	条目	页码
基础 (Basics)	变量 (Variables)	198
基础 (Basics)	打印 (Print)	199
基础 (Basics)	类型注解 (Type Annotations)	200
基础 (Basics)	正则表达式 (Regular Expressions)	201
基础 (Basics)	原始字符串 (Raw Strings)	202
基础 (Basics)	位运算与集合运算符 (Bitwise & Set Operators)	203
控制流 (Control Flow)	If / Else	204
控制流 (Control Flow)	While 循环 (While Loop)	205
控制流 (Control Flow)	For 循环 (For Loop)	206
控制流 (Control Flow)	Match / Case	207
数据结构 (Data Structures)	列表与元组 (Lists & Tuples)	208
数据结构 (Data Structures)	字典 (Dictionaries)	209
数据结构 (Data Structures)	TypedDict (记录形状)	210
函数 (Functions)	函数 (Functions)	211
函数 (Functions)	生成器 (yield) (Generators (yield))	212
类 (Classes)	类 (Classes)	213
类 (Classes)	@dataclass	214
类 (Classes)	继承与 super() (Inheritance & super())	215
类 (Classes)	运算符与双下划线方法 (Operators & dunder methods)	216
类 (Classes)	装饰器 (Decorators)	217
类 (Classes)	@property	218

组件 (COMPONENTS)

分类	条目	页码
核心系统 (Core Systems)	时钟 (Clock)	219

分类	条目	页码
核心系统（Core Systems）	指挥官（Commander）	222
核心系统（Core Systems）	控制台（Console）	224
核心系统（Core Systems）	GPS	229
核心系统（Core Systems）	夜行者（Nocturna）	232
核心系统（Core Systems）	研究（Research）	237
核心系统（Core Systems）	商店（Shop）	239
核心系统（Core Systems）	发射器（Transmitter）	243
传感器（Sensors）	生物量传感器（Biomass Sensor）	246
传感器（Sensors）	氧气传感器（Oxygen Sensor）	248
传感器（Sensors）	植物传感器（Plants Sensor）	251
传感器（Sensors）	压力传感器（Pressure Sensor）	253V
传感器（Sensors）	温度计（Thermometer）	255V
传感器（Sensors）	野生动物传感器（Wildlife Sensor）	257V
探索（Exploration）	日志（Journal）	259V
探索（Exploration）	地图标记（Map Markers）	267V
探索（Exploration）	扫描仪（Scanner）	271V
电力（Power）	电池（Battery）	274V
电力（Power）	避雷针（Lightning Rod）	276V
电力（Power）	石油发电机（Oil Generator）	280
电力（Power）	反应堆（Reactor）	283
电力（Power）	太阳能发电机（Solar Generator）	288
电力（Power）	蒸汽涡轮机（Steam Turbine）	291
电力（Power）	载具充电站（Vehicle Charging Station）	295
地形改造（Terraforming）	大气层（Atmosphere）	304
地形改造（Terraforming）	生物施放器（Bio Caster）	307
地形改造（Terraforming）	生物采集器（Bio Collector）	321
地形改造（Terraforming）	生物调节器（Bio Conditioner）	325
地形改造（Terraforming）	生物交换器（Bio Exchange）	334
地形改造（Terraforming）	生物实验室（Bio Lab）	341
地形改造（Terraforming）	生物发光器（Bio Luminizer）	348
地形改造（Terraforming）	DNA 测序仪（DNA Sequencer）	355
地形改造（Terraforming）	饲料制造机（Feed Maker）	361
地形改造（Terraforming）	栖息地（Habitat）	368
地形改造（Terraforming）	热能发生器（Heat Generator）	394
地形改造（Terraforming）	氧气发生器（Oxygen Generator）	400
地形改造（Terraforming）	压力发生器（Pressure Generator）	406
地形改造（Terraforming）	精炼机（Refiner）	412
生产与存储（Production & Storage）	制造机（Fabricator）	421

分类	条目	页码
生产与存储 (Production & Storage)	燃料组装机 (Fuel Assembler)	433
生产与存储 (Production & Storage)	采集器 (Harvester)	439
生产与存储 (Production & Storage)	库存 (Inventory)	459
生产与存储 (Production & Storage)	大型仓库 (Large Warehouse)	466
生产与存储 (Production & Storage)	铅桶 (Lead Cask)	474
生产与存储 (Production & Storage)	采矿钻机 (Mining Drill)	479
生产与存储 (Production & Storage)	冶炼炉 (Smelter)	481
生产与存储 (Production & Storage)	存储箱 (Storage Bin)	487
生产与存储 (Production & Storage)	仓库 (Warehouse)	496
生产与存储 (Production & Storage)	废料处理器 (Waste Processor)	504W
载具与模块 (Vehicles & Modules)	建造模块 (Constructor Module)	511
载具与模块 (Vehicles & Modules)	钻探模块 (Drill Module)	514
载具与模块 (Vehicles & Modules)	无人机 (Drone)	517
载具与模块 (Vehicles & Modules)	机队 (Fleet)	532
载具与模块 (Vehicles & Modules)	导航模块 (Nav Module)	534
载具与模块 (Vehicles & Modules)	先锋 (Pioneer)	539
载具与模块 (Vehicles & Modules)	漫游车 (Rover)	549
载具与模块 (Vehicles & Modules)	声呐模块 (Sonar Module)	558
物流与订单 (Logistics & Orders)	数据存档 (Data Archive)	563
物流与订单 (Logistics & Orders)	无人机基地 (Drone Depot)	569
物流与订单 (Logistics & Orders)	无人机服务站 (Drone Service Station)	573
物流与订单 (Logistics & Orders)	地球订单 (Earth Orders)	584
物流与订单 (Logistics & Orders)	信号总线 (Signal Bus)	587
物流与订单 (Logistics & Orders)	供应码头 (Supply Dock)	599
基础设施与流体 (Infrastructure & Fluids)	建造蓝图 (Construction Blueprint)	606
基础设施与流体 (Infrastructure & Fluids)	奇异气体盖 (Exotic Gas Cap)	616
基础设施与流体 (Infrastructure & Fluids)	奇异泉接头 (Exotic Spring Tap)	621
基础设施与流体 (Infrastructure & Fluids)	气体罐 (Gas Tank)	626
基础设施与流体 (Infrastructure & Fluids)	大型液体罐 (Large Liquid Tank)	631
基础设施与流体 (Infrastructure & Fluids)	液体罐 (Liquid Tank)	636
基础设施与流体 (Infrastructure & Fluids)	油泵 (Oil Pump)	641
基础设施与流体 (Infrastructure & Fluids)	前哨站 (Outpost)	645
基础设施与流体 (Infrastructure & Fluids)	前哨站网络 (Outpost Network)	649
基础设施与流体 (Infrastructure & Fluids)	电力控制 (Power Control)	651
基础设施与流体 (Infrastructure & Fluids)	运行控制 (Run Control)	655
基础设施与流体 (Infrastructure & Fluids)	蒸汽冷凝器 (Steam Condenser)	658
基础设施与流体 (Infrastructure & Fluids)	热能盖 (Thermal Cap)	663
基础设施与流体 (Infrastructure & Fluids)	水泵 (Water Pump)	670

分类	条目	页码
生物圈 (Biosphere)	生物量混合器 (Biomass Mixer)	675T
生物圈 (Biosphere)	作物自动化器 (Crop Automator)	681C
生物圈 (Biosphere)	分配器 (Dispenser)	691C
生物圈 (Biosphere)	精华液化器 (Essence Liquifier)	696C
生物圈 (Biosphere)	生长灯 (Grow Lamp)	700C
生物圈 (Biosphere)	植物地形改造器 (Plant Terraformer)	704C
生物圈 (Biosphere)	种子制造机 (Seed Maker)	712W
生物圈 (Biosphere)	喷灌器 (Sprinkler)	719W
存储与库存 (Storage & Inventory)	物品目录 (Item Catalog)	724W
天气与天空 (Weather & Sky)	气象站 (Weather Station)	726W

数据库 (DATABASE)

分类	条目	页码
生产 (Production)	生产层级 (Production Tiers)	730W
配方 (Recipes)	冶炼配方 (Smelter Recipes)	731
配方 (Recipes)	制造配方 (Fabricator Recipes)	733W
配方 (Recipes)	饲料制造配方 (Feed Maker Recipes)	753W
配方 (Recipes)	精炼配方 (Refiner Recipes)	759
配方 (Recipes)	燃料组装配方 (Fuel Assembler Recipes)	761
设备 (Equipment)	电力设备 (Power Equipment)	762W
设备 (Equipment)	大气设备 (Atmosphere Equipment)	764
设备 (Equipment)	生物学设备 (Biology Equipment)	768S
设备 (Equipment)	生物圈设备 (Biosphere Equipment)	770S
设备 (Equipment)	采矿设备 (Mining Equipment)	774S
设备 (Equipment)	物流设备 (Logistics Equipment)	779S
设备 (Equipment)	生产设备 (Production Equipment)	780S
设备 (Equipment)	模块与载具装备 (Modules & Vehicle Gear)	782S
设备 (Equipment)	流体基础设施 (Fluid Infrastructure)	789S
物品 (Items)	矿物 (Minerals)	792S
物品 (Items)	精炼材料 (Refined Materials)	794S
物品 (Items)	制造组件 (Fabricated Components)	797S
物品 (Items)	农业与饲料 (Agriculture & Feeds)	808S
物品 (Items)	生命形态 (Life Forms)	816S
物品 (Items)	野外资源 (Field Resources)	824S
物品 (Items)	生物学样本 (Biology Samples)	826S
物品 (Items)	实验室试剂 (Lab Reagents)	853S
流体 (Fluids)	流体 (Fluids)	855S
研究 (Research)	研究 (Research)	862S

类型 (TYPES)

分类	类型	页码
核心数据 (Core Data)	边界 (Bounds)	889SF
核心数据 (Core Data)	组件 (Component)	890F
核心数据 (Core Data)	兴趣点 (PointOfInterest)	891F
核心数据 (Core Data)	位置 (Position)	893F
核心数据 (Core Data)	脚本命令 (ScriptCommand)	894F
世界与地点 (World & Sites)	前哨站 (Outpost)	896F
世界与地点 (World & Sites)	前哨站网络 (OutpostNetwork)	899F
世界与地点 (World & Sites)	前哨站引用 (OutpostRef)	900F
世界与地点 (World & Sites)	建筑引用 (BuildingRef)	904F
世界与地点 (World & Sites)	采集机器引用 (HarvestingMachineRef)	906F
世界与地点 (World & Sites)	行星 (Planet)	908F
世界与地点 (World & Sites)	扫描结果 (ScanResult)	909F
世界与地点 (World & Sites)	地点 (Site)	911F
世界与地点 (World & Sites)	奇异矿床 (ExoticDeposit)	914F
世界与地点 (World & Sites)	地质异常 (GeologicalAnomaly)	920F
世界与地点 (World & Sites)	采矿地点 (MiningSite)	922I
世界与地点 (World & Sites)	油井 (OilWell)	925I
世界与地点 (World & Sites)	热能喷口 (ThermalVent)	928I
世界与地点 (World & Sites)	水井 (WaterWell)	932I
存储与库存 (Storage & Inventory)	电池 (Battery)	935I
存储与库存 (Storage & Inventory)	持有器 (Holder)	937I
存储与库存 (Storage & Inventory)	便携式电池 (PortableBattery)	939I
存储与库存 (Storage & Inventory)	货物 (Cargo)	940O
存储与库存 (Storage & Inventory)	货架 (Rack)	943O
存储与库存 (Storage & Inventory)	箱 (Bin)	944O
存储与库存 (Storage & Inventory)	丢弃结果 (DiscardResult)	946O
存储与库存 (Storage & Inventory)	输入槽 (InputSlot)	948O
存储与库存 (Storage & Inventory)	物品信息 (ItemInfo)	955O
存储与库存 (Storage & Inventory)	物品结果 (ItemResult)	957O
存储与库存 (Storage & Inventory)	物品堆 (ItemStack)	958O
存储与库存 (Storage & Inventory)	输出槽 (OutputSlot)	959O
存储与库存 (Storage & Inventory)	拾取输出槽 (PickupOutputSlot)	964P
存储与库存 (Storage & Inventory)	配方 (Recipe)	965C
存储与库存 (Storage & Inventory)	出售结果 (SaleResult)	969C
存储与库存 (Storage & Inventory)	商店物品 (ShopItem)	971C
存储与库存 (Storage & Inventory)	槽位 (Slot)	972C
存储与库存 (Storage & Inventory)	转移结果 (TransferResult)	975

分类	类型	页码
存储与库存 (Storage & Inventory)	载具输入槽 (VehicleInputSlot)	978C
存储与库存 (Storage & Inventory)	仓库槽 (WarehouseSlot)	983C
机队与载具 (Fleet & Vehicles)	采集结果 (CollectResult)	985C
机队与载具 (Fleet & Vehicles)	建造模块 (ConstructorModule)	987C
机队与载具 (Fleet & Vehicles)	钻探模块 (DrillModule)	989C
机队与载具 (Fleet & Vehicles)	无人机电池 (DroneBattery)	991C
机队与载具 (Fleet & Vehicles)	无人机货舱 (DroneCargo)	993C
机队与载具 (Fleet & Vehicles)	无人机油箱 (DroneOilTank)	999C
机队与载具 (Fleet & Vehicles)	机队 (Fleet)	1001C
机队与载具 (Fleet & Vehicles)	无人机引用 (DroneRef)	1002C
机队与载具 (Fleet & Vehicles)	移动单元引用 (MobileUnitRef)	1007C
机队与载具 (Fleet & Vehicles)	载具引用 (VehicleRef)	1010C
机队与载具 (Fleet & Vehicles)	安装槽 (MountSlot)	1014C
机队与载具 (Fleet & Vehicles)	导航模块 (NavModule)	1016C
机队与载具 (Fleet & Vehicles)	声呐模块 (SonarModule)	1019C
机队与载具 (Fleet & Vehicles)	声呐扫描结果 (SonarScanResult)	1022C
机队与载具 (Fleet & Vehicles)	勘测结果 (SurveyResult)	1023C
基础设施与流体 (Infrastructure & Fluids)	建造 (Construction)	1024C
基础设施与流体 (Infrastructure & Fluids)	流体端口 (FluidPort)	1027C
基础设施与流体 (Infrastructure & Fluids)	流体连接 (FluidConnection)	1030C
基础设施与流体 (Infrastructure & Fluids)	管道 (Pipe)	1033C
基础设施与流体 (Infrastructure & Fluids)	电力网 (PowerGrid)	1037C
基础设施与流体 (Infrastructure & Fluids)	电力网成员 (PowerGridMember)	1040C
基础设施与流体 (Infrastructure & Fluids)	电力摘要 (PowerSummary)	1043C
订单与通信 (Orders & Comms)	广播信息 (BroadcastInfo)	1045C
订单与通信 (Orders & Comms)	通信消息 (CommsMessage)	1046C
订单与通信 (Orders & Comms)	对接槽 (DockSlot)	1047C
订单与通信 (Orders & Comms)	订单 (Order)	1048C
订单与通信 (Orders & Comms)	接收结果 (ReceiveResult)	1051C
订单与通信 (Orders & Comms)	发送结果 (SendResult)	1052C
订单与通信 (Orders & Comms)	发射器信息 (TransmitterInfo)	1053B
订单与通信 (Orders & Comms)	等待任一结果 (WaitAnyResult)	1054B
订单与通信 (Orders & Comms)	等待广播结果 (WaitBroadcastResult)	1055B
面板 (Panels)	面板 (Panel)	1056B
合约 (Contracts)	合约 (Contract)	1079B
合约 (Contracts)	击败系统合约 (BeatTheSystemContract)	1082B
合约 (Contracts)	仲裁者 (Arbiter)	1084B
合约 (Contracts)	埋藏五号合约 (BuriedFiveContract)	1087B

分类	类型	页码
合约 (Contracts)	分析器 (Analyzer)	1089B
合约 (Contracts)	冷启动合约 (ColdBootContract)	1090B
合约 (Contracts)	核心样本合约 (CoreSampleContract)	1092B
合约 (Contracts)	核心设备 (CoreDevice)	1094B
合约 (Contracts)	损坏存档合约 (CorruptedArchiveContract)	1096B
合约 (Contracts)	存档 (Archive)	1098B
合约 (Contracts)	串扰合约 (CrosstalkContract)	1099B
合约 (Contracts)	数据平板合约 (DataTabletContract)	1101B
合约 (Contracts)	数据平板 (DataTablet)	1103B
合约 (Contracts)	探针结果 (ProbeResult)	1105B
合约 (Contracts)	漂移信号合约 (DriftingSignalContract)	1106B
合约 (Contracts)	石板设备 (SlabDevice)	1108E
合约 (Contracts)	晶格合约 (LatticeContract)	1109E
合约 (Contracts)	晶格网格 (LatticeGrid)	1111E
合约 (Contracts)	中继黑客合约 (RelayHackContract)	1113I
合约 (Contracts)	中继锁 (RelayLock)	1115S
合约 (Contracts)	密封金库合约 (SealedVaultContract)	1116S
合约 (Contracts)	金库 (Vault)	1118S
合约 (Contracts)	金库位置 (VaultPosition)	1120S
合约 (Contracts)	终端突破合约 (TerminalBreachContract)	1121S
合约 (Contracts)	外星终端 (AlienTerminal)	1123S
合约 (Contracts)	猜测结果 (GuessResult)	1124T
合约 (Contracts)	“ 织机 ” 合约 (TheLoomContract)	1125T
合约 (Contracts)	织机 (Loom)	1127T
合约 (Contracts)	三回声合约 (ThreeEchoesContract)	1128T
合约 (Contracts)	三回声广播 (ThreeEchoesBroadcast)	1130T
合约 (Contracts)	异种遗传学合约 (XenogeneticsContract)	1131T
合约 (Contracts)	合约脚本 (ContractScript)	1133T
合约 (Contracts)	晶格探针结果 (LatticeProbeResult)	1134T
合约 (Contracts)	金库逃脱结果 (VaultEscapeResult)	1135T
内置类型 (Built-in Types)	dict	1136T
内置类型 (Built-in Types)	generator	1140T
内置类型 (Built-in Types)	list	1142W
内置类型 (Built-in Types)	set	1146W
内置类型 (Built-in Types)	slice	1152W
内置类型 (Built-in Types)	str	1153W
内置类型 (Built-in Types)	tuple	1165W
生物圈 (Biosphere)	分析结果 (AnalyzeResult)	1166W

分类	类型	页码
生物圈 (Biosphere)	生物提取结果 (BioExtractionResult)	1167W
生物圈 (Biosphere)	生物扫描结果 (BioScanResult)	1168W
生物圈 (Biosphere)	细胞 (Cell)	1169W
生物圈 (Biosphere)	生命形态扫描结果 (LifeFormScanResult)	1174
生物圈 (Biosphere)	生命形态样本 (LifeFormSample)	1175
生物圈 (Biosphere)	植物需求 (PlantRequirement)	1177
生物圈 (Biosphere)	便携式生物提取器 (PortableBioExtractor)	1178
生物圈 (Biosphere)	便携式生物扫描仪 (PortableBioScanner)	1179
生物圈 (Biosphere)	种子配方 (SeedRecipe)	1180
生物圈 (Biosphere)	种子结果 (SeedResult)	1183
内置模块 (Built-in Modules)	匹配 (Match)	1185
探索 (Exploration)	已编目生物 (CatalogedCreature)	1188
探索 (Exploration)	已编目碎片 (CatalogedFragment)	1192
探索 (Exploration)	标记 (Marker)	1196
基础设施 (Infrastructure)	蓝图计划结果 (BlueprintPlanResult)	1198
系统 (System)	操作结果 (ActionResult)	1200
系统 (System)	命令结果 (CommandResult)	1206
系统 (System)	计数结果 (CountResult)	1207
系统 (System)	作物作业 (CropJob)	1208
系统 (System)	作物作业结果 (CropJobResult)	1210
系统 (System)	作业收据 (JobReceipt)	1212
地形改造 (Terraforming)	分析信息 (AnalyzeInfo)	1214
地形改造 (Terraforming)	生物施放器配方 (BioCasterRecipe)	1218
地形改造 (Terraforming)	生物订单 (BioOrder)	1220
地形改造 (Terraforming)	舱室碎片 (ChamberFragment)	1223
地形改造 (Terraforming)	舱室样本 (ChamberSample)	1225
地形改造 (Terraforming)	碎片位置 (FragmentLocation)	1227
地形改造 (Terraforming)	栖息地加成节点 (HabitatBonusNode)	1231
地形改造 (Terraforming)	栖息地加成树 (HabitatBonusTree)	1234
地形改造 (Terraforming)	栖息地洞见 (HabitatInsight)	1235
地形改造 (Terraforming)	标本 (Specimen)	1237
地形改造 (Terraforming)	废料倾倒结果 (WasteDumpResult)	1242
天气与天空 (Weather & Sky)	信号接收器 (SignalReceiver)	1243
天气与天空 (Weather & Sky)	信号传输 (SignalTransmission)	1244
天气与天空 (Weather & Sky)	风暴 (Storm)	1246
天气与天空 (Weather & Sky)	天气事件预报 (WeatherEventForecast)	1249
天气与天空 (Weather & Sky)	天气报告 (WeatherReport)	1251
天气与天空 (Weather & Sky)	天气信号板 (WeatherSignalBoard)	1253

分类	类型	页码
天气与天空 (Weather & Sky)	天气信号板状态 (WeatherSignalBoardStatus)	1256
天气与天空 (Weather & Sky)	天气打击 (WeatherStrike)	1257
天气与天空 (Weather & Sky)	区域 (Zone)	1259

指南 / 从这里开始 (GUIDE / START HERE)

入门 (Getting Started)

欢迎来到 Code: Terraform。

你已连接到一颗无人居住行星上的远程终端。你的目标是通过编写代码来改造它的地形，使其适合居住。

这颗行星上的一切都由代码控制。系统、传感器和机器都需要通过编程才能上线。

先在编辑器中输入代码，然后点击 ► 运行。

```
print("Hello, planet!")
```

控制台会显示你的输出和所有错误。随着你激活系统，传感器读数会更新。

编码时查找信息 (LOOKING THINGS UP WHILE YOU CODE)

将鼠标悬停在脚本中的任意方法名或函数名上，即可内联查看其签名、描述和返回类型。

在 macOS 上按 **Cmd+click** / 在 Windows 或 Linux 上按 **Ctrl+click**，即可打开 DOCS，并直接跳转到任意方法、函数或组件 ID 对应的条目。你点击的代码块会短暂高亮，便于确认跳转位置。

完整的编辑器快捷键（重命名、自动补全、参数提示、查找/替换、调试单步执行）请参阅“代码编辑器”DOCS 页面。

指南 / 从这里开始 (GUIDE / START HERE)

第一步 (First Steps)

完成“初次接触”以及启动/供电/传感器/上行链路教程链后，你拥有 2,500 点信用点，商店和生物学实验室也已解锁。本页是接下来行动的交接指南。它会一直保留在 DOCS 中；需要复习时随时搜索 First Steps。

后续步骤 (NEXT STEPS)

- 压力传感器和氧气传感器已经部署。修复压力传感器即可解锁采集与合约；修复氧气传感器即可解锁大气层。研究功能已经可用。
- 打开左侧边栏中的商店，购买太阳能发电机、电池和氧气发生器。
- 打开库存并部署这三件设备。
- 为太阳能发电机和氧气发生器编写脚本。氧气达到 1 ppt 后，自动喂料器会解锁，生物学流程也可以自动化。

你可以按任意顺序修复传感器。这个起步配置为当前的手动生物学以及自动喂料器解锁后的自动化生物学预留了足够的信用点和电力。

获取信用点 (EARNING CREDITS)

四种早期方式会教会你不同的技能。采集与合约并列存在：如果你偏好可见的机器，可能会觉得前者更容易；如果你偏好封闭的逻辑谜题，则可能会觉得后者更容易。

手动生物学 (MANUAL BIOLOGY, 最容易, 无需代码)：使用生物学工作台按钮来采集、分析、提取并交付样本。这是最简单的第一笔收益，但反复订单会变成需要大量点击的流程。

采集 (HARVESTING, 第一次自动化, 修复压力传感器后)：编写简单的网格扫描脚本，移动、采集、存储并出售资源。需要注意热量和手持物品槽，但路径查找和路线优化不是必需的。

合约 (CONTRACTS, 封闭式谜题, 修复压力传感器后)：用最少的机器状态和安全的重试来解决隔离的编程问题。这类任务偏好循环、集合和抽象推理。

自动化生物学 (AUTOMATED BIOLOGY, 高级自动化, 自动喂料器解锁后)：通过独立脚本协调生物采集器、生物实验室和生物交换器，同时管理订单、试剂、物品端口、已完成输出以及部分状态。

每一点信用都会回流到商店。机器越多，地形改造进展越快。

指南 / 从这里开始 (GUIDE / START HERE)

命令结果 (Command Results)

基本模式 (THE BASIC PATTERN)

每条会改变世界的命令都会返回一个固定的结果对象。如果脚本不需要作出反应，可以忽略该结果：

```
self.set_power(5)
```

如果结果很重要，就保存它：

```
result = self.extract()
if result.status == "ok":
    print("sample ready")
else:
    print(result.message)
```

每个结果都有：

- `.status`：供程序逻辑使用的稳定、机器可读代码。
- `.message`：用于显示和调试、面向玩家且经过本地化的可读说明。

永远不要根据 `.message` 作分支；其措辞可能改变，也可能被翻译。没有负载的命令返回 `ActionResult`。带负载的命令返回专用结果，其中包含同样的两个字段以及带名称的负载字段。`AnalyzeResult` 添加 `.info`，`SonarScanResult` 添加 `.sites`，`TransferResult` 添加 `.requested` 和 `.moved`。

结果不是布尔值 (RESULTS ARE NOT BOOLEANS)

结果对象是结构化值，而不是成功布尔值。即使其状态表示拒绝，对象本身仍然是真值。不要写：

```
if self.analyze():
    print("success")
```

应保存结果，并比较文档规定的状态。`"ok"` 很常见，但不是成功的通用拼写。根据命令不同，成功或已接受的结果还可能包括 `"started"`、`"queued"`、`"charging"`、`"complete"` 或其他领域专用代码。命令 DOCS 中的结果表才是权威依据。

负载字段 (PAYLOAD FIELDS)

只有在某个状态明确说明可以读取负载之后，才读取负载：


```
analysis = self.analyze()
if analysis.status == "ok":
    recipe = analysis.info.required_recipe
else:
    print(analysis.message)
```

被拒绝的结果仍保持相同的结果类型。它们不会变成 `False`、`None`、空字典或单独的状态字符串。

查询返回自然值（QUERIES RETURN NATURAL VALUES）

只读查询不会执行游戏操作，因此会直接返回其自然值。`self.is_running()` 返回布尔值，`self.cargo.count()` 返回数字，`self.sonar.tier()` 返回字符串。可选查询只有在其 DOCS 条目所说明的那一种“缺失”含义下才会返回 `None`。

等待与后台工作（WAITING AND BACKGROUND WORK）

有些命令会让出执行权：脚本会暂停，直到尝试的操作完成，然后接收其结果。`self.analyze()` 就是一个例子。另一些命令会启动或排队持久工作并立即返回。充电站的 `charge(...)` 结果可能报告 `"charging"` 或 `"queued"`，同时充电站继续在后台工作。

不要假设所有计时系统都以相同方式等待。命令描述会说明它是让出执行权、启动工作，还是排队工作。

忙碌与异常（BUSY AND EXCEPTIONS）

对于定义了该结果的命令，`"busy"` 是普通的临时结果。保存该结果，让游戏时间流逝或阻塞状态发生变化，然后在脚本需要新结果时再次调用命令。不要把每个非成功状态都当作 `busy` 来重试。

错误的参数类型和文档中规定的格式错误值可能引发精确异常，例如 `TypeError` 或 `ValueError`。普通的、会变化的世界状态会使用该命令列出的结果状态。打开命令的 DOCS 条目，查看其确切状态、负载字段、时序和异常。关于如何在游戏加载后让自动化安全地使用这些结果，请参阅“长时间运行的脚本”。

指南 / 从这里开始 (GUIDE / START HERE)

长时间运行的脚本 (Long-Running Scripts)

游戏加载时会发生什么 (WHAT HAPPENS WHEN A GAME LOADS)

游戏加载时，正在运行的脚本会从第一行重新开始。游戏会保存脚本源代码和运行状态，但不会恢复当前行、局部变量、调用栈或待处理的返回值。如果保存时脚本处于暂停状态，加载后仍会处于暂停状态，但只保留暂停状态。执行位置会重置。按 Resume 会从源代码开头开始新的运行；它不会从之前的行继续。

保存的世界状态可能比某次脚本运行存活得更久。当重启后的脚本再次触及机器时，机器中可能仍有货物、标本、准备好的材料、排队的作业、已完成的输出或计时工作。这种状态不是错误，而是下一次决策的起点。

让机器成为事实来源 (MAKE THE MACHINE THE SOURCE OF TRUTH)

将长时间运行的脚本构建为协调循环。每一轮都应：

- 读取机器当前状态。
- 判断仍缺少哪一步。
- 尝试该步骤一次。
- 检查返回的 `.status`。
- 让游戏时间流逝后再重试临时阻塞。

不要依赖上一轮的局部变量来记住机器里有什么。重新读取舱室、货物、端口、队列或当前作业。

生物实验室示例 (BIO LAB EXAMPLE)

这个循环可以从空实验室、已采集标本、正在进行的分析或已分析标本开始。它总会在决定操作前重新读取舱室：

```
collector = get_component("bio_collector_1")

while True:
    specimen = self.specimen

    if specimen is None:
        transfer = self.take_from(collector)
        if transfer.status == "busy":
            sleep(0.2)
            continue
        if transfer.status != "ok":
            print(transfer.message)
            sleep(0.2)
            continue
        continue

    if specimen.stage == "collected":
        analysis = self.analyze()
        if analysis.status == "busy":
            sleep(0.2)
            continue
        if analysis.status != "ok":
            print(analysis.message)
            sleep(0.2)
            continue
        print(analysis.info.required_recipe)
        continue

    print(specimen.recipe)
    break
```

这里负载规则很重要：只有在 `analysis.status == "ok"` 之后，`analysis.info` 才可用。`"busy"` 结果没有分析负载，因此脚本会等待并开始下一轮，而不是读取 `.info`。

让写入可安全重复（MAKE WRITES SAFE TO REPEAT）

在转移、购买、装载或排队某项操作之前，先检查已有内容。计算缺少的数量，而不是再次提交完整计划数量。开始更多工作前先排出已完成的输出，并区分临时的 `"busy"` 与 `"output_full"`、`"insufficient_input"` 或无效参数等阻塞原因。命令的 DOCS 条目会列出哪些结果会随着世界变化而改变。

有意识地持久化知识（PERSIST KNOWLEDGE DELIBERATELY）

机器内容属于世界状态，但普通脚本变量只属于一次运行。当自动化必须跨越脚本停止和游戏加载记住已学习的数据、检查点或决策时，请使用数据存档（Data Archive）。有关通用结果契约，请参阅“命令结果”；有关循环与节奏基础，请参阅“循环与脚本”。

指南 / 教程 (GUIDE / TUTORIALS)

游戏如何运作 (How This Game Works)

Code: Terraform 是一款系统类游戏：进展来自理解机器，然后编写小型脚本，让这些机器发挥作用。

基本循环 (THE BASIC LOOP)

- 选择一台机器或一个问题。
- 阅读该机器的 Info 选项卡。
- 在 DOCS 中查看它提供的方法和返回值。
- 编写一个只做一件明确事情的小脚本。
- 运行脚本，检查返回结果，观察控制台，然后改进脚本。

游戏并不是要求你把所有内容都背下来，而是要求你学会信息在哪里。Info 选项卡解释意图。DOCS 解释 API。命令结果解释确切的程序化结果。控制台显示脚本选择打印的输出、未捕获的错误和专用测试记录；仪表板则显示世界是否发生了变化。

命令与游戏时间 (COMMANDS AND GAME TIME)

游戏命令始终返回结果对象：根据 `result.status` 作分支，读取 `result.message`，然后使用命令专用的负载字段。命令不会自动打印普通结果。有些命令会暂停脚本，直到尝试的工作完成；另一些命令会启动或排队后台工作，并立即返回其当前状态。无论哪种情况，行星都会继续运行。完整模型请参阅“命令结果”。

良好习惯 (GOOD HABITS)

从一台机器开始。保存每一个重要的命令结果。检查 `.status`；学习时打印 `.message`。小脚本运行正常后，让它循环。一个循环运行正常后，再把它连接到另一个系统。

指南 / 教程 (GUIDE / TUTORIALS)

选择你的第一条信用点路径 (Choosing Your First Credit Path)

完成“初次接触”后，信用点会成为扩张的燃料。早期有四种方式，每一种都会教会你不同的技能。手动生物学立即开放。修复压力传感器后会出现采集和合约；起步氧气发生器达到 1 ppt 氧气并解锁自动喂料器后，自动化生物学才真正可行。

手动生物学 - 最容易的第一笔收益 (MANUAL BIOLOGY - EASIEST FIRST PAYOUT)

你开始时拥有的 2,500 点信用足以购买太阳能发电机、电池和氧气发生器，并且还剩下信用点购买起步试剂。使用生物学工作台按钮采集标本、分析标本、装入揭示出的试剂、提取样本并交付。无需代码。这让流程很容易学习，但第一批订单之后，重复操作会变慢。

采集 - 第一次机器自动化 (HARVESTING - FIRST MACHINE AUTOMATION)

基础采集器脚本可以扫描网格、采集现有资源、存储资源，然后从库存中出售。它必须考虑热量和唯一的手持物品槽，但无需路径查找或优化路线也能工作。对于喜欢观察机器行为的玩家，这是最直接的第一个自动化项目。

合约 - 封闭式编程谜题 (CONTRACTS - CONTAINED PROGRAMMING PUZZLES)

合约不是生产机器。它们读取数据、计算答案并将答案传输到地球。它们几乎不涉及机器 API 状态，错误答案可以安全重试；不过相较于采集，其循环、集合和抽象推理可能让玩家感觉更容易，也可能更难。

自动化生物学 - 高级早期项目 (AUTOMATED BIOLOGY - ADVANCED EARLY PROJECT)

自动喂料器解锁后，生物学会从一套手动工作台流程变成三个相互协作的机器脚本。生物采集器、生物实验室和生物交换器分别负责自己的操作。可靠的自动化必须协调订单进度、试剂准备、物品端口、已完成输出，以及脚本停止后仍然存在的标本或试剂状态。

做出明智选择 (CHOOSING WELL)

如果你想获得最简单的第一笔收益，就从手动生物学开始。想要可见的自动化就选择采集；想要专注的逻辑问题就选择合约。如果你想做更高级的多机器项目，就把完整的生物学链自动化。四种方式都仍然是赚取早期信用点的有效途径。

指南 / 教程

能源与地球化机器

地球化机器是基地的核心引擎。热量发生器、氧气发生器和压力发生器把电力转化为行星进度。

电力优先

没有电力的机器无法帮助改造行星。早期太阳能会随昼夜变化，因此中午能够工作的基地，日落后可能停滞。电池通过储存白天的剩余电力来填补这一间隙。

可靠的早期配置是：添加一台发电机，添加一块电池，运行一台地球化机器，然后在添加更多用电设备前先扩充电力。

地球化从三条轨道开始

温度、氧气和压力是最初的三条轨道，每条轨道都有自己的机器和升级。它们都很重要。你可以暂时专注于其中一条，但只有三条都持续推进，行星才会变得适合居住。后续进展会开启更多地球化轨道；现在先让这三套系统可靠运行。

最初的机器速度很慢。增加电力，增加机器，然后扩展到更多前哨站和更高等级，以提高进度。

氧气依靠二氧化碳运行：碳循环

氧气发生器把大气中的二氧化碳（CO₂）一比一转化为氧气。行星起初拥有大量 CO₂ 储备，这些储备足够使用很长时间，但大规模制氧会消耗得更快。燃烧石油（Oil Generator）、焚烧物品（Waste Processor），以及在真正的大规模生产中由野生动物群落实现的活动，都会把 CO₂ 送回大气：已经建立的殖民地会呼出 CO₂。一点点火山补给会让 CO₂ 永远不会完全见底，因此缺料的制氧机群最终总能恢复。留意 `atmosphere.get_co2()`，并记住效率的最佳区间会随它变化。

脚本应当做出反应

优秀的机器脚本会先读取状态再行动。发电机可以根据太阳角度做出反应。电池或电力监视器可以在储能偏低时发出警告。电力紧张时，地球化机器可以降低功率，基地恢复后再继续运行。

如果发电机在夜间停止，或地球化机器因电力不足而暂停，通常不是错误。电力生产、储存和消耗必须彼此匹配。

指南 / 教程

第一条生物学循环

手动生物学是最温和的首条积分路径，因为工作台会引导你完成每个狭窄的机器步骤。自动化同一条流水线是早期的进阶项目，因为其中各台机器的独立脚本必须协调持久状态。

手动与自动化物流

手动生物学工作台在每个前哨站恰好有一个储藏室。在 Nocturna Base，该储藏室就是 Base Inventory。在其他每个前哨站，需在 Manual Stockroom 栏中选择一座已部署的本地 Warehouse；其中所有手动生物学卡片都会从该 Warehouse 取用并归还物品。选择不会移动已有物品。商店购买的物品仍会送到 Base Inventory，因此请用车辆或无人机把补给运到远程 Warehouse。

手动按钮不会创建隐藏的 I/O 连接，也不会通过机器的 `self.input` 和 `self.output` 缓冲区移动物品。它们会把补给直接从手动储藏室装入工作台，并把完成品或回收品直接送回该储藏室。因此，机器输入或输出缓冲区已满，不会阻塞相应的手动操作。

开始编写脚本后会发生什么变化

脚本不会使用手动储藏室选择器。在研究 Auto Feeders

之后，你的代码必须显式管理每台机器的脚本物流：在存在物品端口的地方连接端口，把补给移动到

`self.input`，必要时将其装入机器，并从 `self.output` 排出完成品。每次固定设施之间的转移仍然局限于同一个前哨站。

Bio Collector、Bio Lab 和 Bio Exchange

各自拥有自己的硬件动作。停止或重新启动脚本不会清空标本室、卸下试剂、排出输出，也不会重置 Bio Order 进度。可靠的代码必须检查这些部分状态，再决定下一步做什么。

```
input_link = self.input.connect("Reagent Warehouse")
output_link = self.output.connect("Sample Warehouse")
if input_link.status != "ok":
    print(input_link.message)
```

流水线

- Bio Collector 获取标本。
- Bio Lab 分析标本、准备试剂，并提取样本。
- 生物群系机器转化样本。
- Bio Exchange 将其交付给一项活动中的 Bio Order。

每条游戏玩法命令都会返回一个结果对象。根据 `.status` 进行分支，使用 `.message` 获取确切原因，并读取任何特定于方法的载荷。例如，`analyze()` 会返回 `AnalyzeResult.info`，而物品转移会返回 `TransferResult.moved`。"busy"、"no_input" 和 "output_full" 等状态绝不是裸字符串。

属性属于每个物品堆，并且会在受支持的转移中保留。Bio Order 来自 Bio Exchange，而不是 Earth Supply Dock 的 orders 组件。规划生产时，要同时减去 `.delivered` 和 `.in_transit`。请参阅《长时间运行的脚本》，其中介绍了无人值守自动化所用的可安全重启循环模式。

指南 / 教程

第一条采集路线

采集是进入可见机器自动化最直接的早期路径。它的收益可能高于手动生物学，同时首个可用版本要比可靠的自动化生物学链小得多。

你正在构建什么

一条有用的采集路线通常包含三部分：选择去哪里，收集材料，然后在 Inventory 表示该行动时储存并出售。主要决策不是单个调用，而是决定何时继续前进。

要检查什么

先阅读 Harvester Info 标签页。然后查看 DOCS 中的 Scanner 和 Harvester 条目。Scanner 说明地表上已知的信息。Harvester 说明移动、采集、热量和持有物品槽。

让第一条路线保持简单

你不需要路径规划。沿相邻网格单元进行固定扫掠，就可以收集并储存找到的一切。移动前检查热量，再次采集前处理好手上物品；只有基础循环正常后，才优化路线。

一个好的第一个目标

先完成一次短途行程：抵达已知区域，收集一些东西，将其储存，并让它准备好从 Inventory 出售。然后再延长扫掠路线，或加入更聪明的选择。

指南 / 教程

合约

合约是地球侧的编程工作。它们更像谜题，而不是工厂自动化。

它们有什么不同

机器脚本通常会改变行星。合约脚本读取合约数据，计算答案，再把答案发送回去。奖励来自正确性，而不是让生产循环持续运行。合约状态是封闭的，错误答案可以安全地检查和重试。

如何处理合约

先阅读简报。然后从脚本中检查可用的合约数据。把答案拆成小步骤，在学习数据形状时打印中间值。答案准备好后，使用合约和 DOCS 中描述的发送器流程。

难度

合约和采集考验不同技能，因此没有哪一个在所有情况下都更难。合约不涉及路线、热量、货物和多机器恢复，但要求运用循环和集合进行更抽象的推理。如果你想做一个专注的编码谜题并获得更大的单次报酬，它们是不错的选择。如果某份合约感觉太抽象，就先放下，建造一段时间的基地；它之后仍会在那里。

指南 / 教程

阅读错误与控制台输出

控制台是界面的一部分。它会显示脚本明确打印的输出、未捕获的异常，以及传感器校准测试等专用记录。

打开控制台

点击底部窗口栏中的

Console，以打开或聚焦控制台。在停靠模式下，控制台位于编辑器旁边的右下窗格。使用控制台筛选器，在 THIS SCRIPT、ALL 输出、WARNINGS 和 ERRORS 之间切换。

结果就是程序接口

游戏玩法命令会返回固定的结果对象，不会自动打印普通的成功或拒绝消息。`result.status` 是用于分支的稳定代码，`result.message` 解释确切结果。携带载荷的结果会添加诸如 `.moved`、`.sites` 或 `.info` 这样的命名字段。

```
result = self.extract()
if result.status != "ok":
    print(result.message)
```

脚本决定哪些命令结果应出现在控制台中。程序逻辑中绝不要比较

`.message`，因为它是本地化的显示文本。格式错误的调用会引发 `TypeError` 或 `ValueError` 等类型化异常；`"busy"` 或 `"no_cargo_space"` 等普通游戏状态会保留为结果状态，以便脚本决定如何处理。

忙碌不是失败

`"busy"` 状态表示机器正在完成另一个计时动作，并不表示发生了故障。重试模式请参阅《命令结果》。

调试节奏

保存重要命令的结果，检查 `.status` 和 `.message`，然后只在相应状态提供载荷时读取载荷字段。想查看脚本停在哪里时使用 Trace；需要直接检查变量时使用 Debug。

指南 / 教程

第一台先驱车

先驱车 (Pioneer) 是把远程计划变成已建基础设施的第一种车辆。通常的流程是：购买一台 Pioneer，安装它需要的模块，将它派往地点，然后让其本地脚本完成工作。

它有什么不同

Rover 用于探索和采矿。Pioneer 用于建造。底盘只是载体；真正决定它能做什么的是安装的模块。

早期扩张时，留意以下四类模块：

- Nav Module - 让 Pioneer 在完整的行星坐标平面上移动。
- Battery Holder - 携带 Portable Batteries。必须安装至少一块已充电的 Portable Battery，否则 Pioneer 无法移动。
- Cargo Rack - 容纳 Portable Storage Bins，用来携带套件、管道段、电力线段和其他建造材料。
- Constructor Module - 当正确材料已装载时，执行 Planet Map 蓝图。

第一个好目标

在尝试建立前哨站前，先练习一个小任务：在家附近放置一份简单蓝图，安装一块已充电的 Portable Battery，将所需套件或线段装入 Pioneer 货舱，把它移动到蓝图处，刹车，然后执行蓝图。蓝图可以来自 Plan Mode，也可以来自 `get_component("construction_blueprint")`；两条路径都会创建同一个持久任务。如果 Pioneer 无法离开，请检查它的 Battery Holder 和已安装电池。如果建造被拒绝，请检查 Constructor Module 和货舱。

去哪里查看

Info 标签页会说明可通过 `self` 使用的模块 API。Plan Mode 会在放置前显示材料需求。查询 `get_component("construction_blueprint").pending_constructions()`，获取排队中的 Construction 快照；每个快照都会向脚本公开 `required_item` 和 `required_count`。

指南 / 教程

使用规划模式

Plan Mode 是 Planet Map 上的可视化建造规划层。它不会立即建造，而是创建持久蓝图，之后由配备 Constructor Module 的 Pioneer 执行。

可视化规划流程

- 打开 Planet Map，启用 Plan Mode。
- 选择一个类别，然后选择工具。
- 在地图允许的位置放置蓝图。
- 阅读放置卡片和蓝图悬停卡片中的所需材料。
- 使用配备 Constructor Module 的 Pioneer 装载该材料。
- 将 Pioneer 派往蓝图处，并从 Pioneer 脚本执行建造。

脚本规划

脚本通过 `get_component("construction_blueprint")` 写入同一个队列。对于 Outposts、caps、pumps、taps 和 Mining Drills，使用 `plan_structure(kind, x, y)`；对于公用设施，使用 `plan_pipe`、`plan_power_line` 和 `plan_bridge`。规划调用成功后会立即放置幽灵蓝图，无需先把车辆派到那里。实体建造仍然需要 Pioneer。

```
plans = get_component("construction_blueprint")
result = plans.plan_structure("thermal_cap", vent.x, vent.y)
if result.status == "ok":
    print(result.blueprint_ids[0])
```

被锁定的工具

可视化工具和脚本调用执行相同的研究限制。前哨站有两道门槛：Outpost Construction 解锁套件，Constructor Module 解锁能够建造蓝图的 Pioneer 模块。

规划习惯

在第一份蓝图成功前，让规划保持小规模。一份蓝图加上它准确要求的材料，比整座远程基地更容易调试。熟悉循环后，再扩展到前哨站、管道、电力线、水泵和野外提取器。前哨站内部的机器（包括无人机设施）会直接从 Inventory 部署，绝不会作为地图蓝图出现。

指南 / 教程

第一座前哨站

前哨站 (Outpost) 会把你的建造区域扩展到行星的新区域。购买 Outpost Kit 只是材料步骤；建立前哨站还需要 Pioneer 建造流程。

要求

- 研究 Outpost Construction, 才能购买 Outpost Kit。
- 研究 Constructor Module, 才能建造 Planet Map 蓝图。
- 安装了 Nav Module 和 Constructor Module 的 Pioneer。
- 一个至少安装了已充电 Portable Battery 的 Battery Holder。没有储存的能量, Pioneer 就无法驾驶。
- 一个 Cargo Rack, 其 Portable Storage Bin 空间足以容纳一套 Outpost Kit。
- 一套装入 Pioneer 货舱的 Outpost Kit, 而不是留在 Base Inventory 中。
- 一份通过 Plan Mode 或 `construction_blueprint.plan_structure("outpost", x, y)` 放置的前哨站蓝图。

要做什么

先放置前哨站占地轮廓。Plan Mode 会提供可视化放置反馈；脚本 API 会返回准确的规划状态。两条路径都会创建同一个任务标记。然后装载套件, 把 Pioneer 移到蓝图处, 刹车, 并从 Pioneer 执行建造。如果 Pioneer 无法离开, 请检查其电池容量和电量。如果套件在库存里但没有任何东西建成, 请确认 Constructor Module 已安装, 并且套件确实在 Pioneer 货舱中。

这为什么重要

单个家园基地在后期会变慢。前哨站步骤标志着地球化开始具有地理维度：更多建造区域、更多机器、更多本地电力, 以及最终连接远距离生产的网络。

指南 / 教程

扩大生产

首座前哨站之后，进展取决于扩张和升级两方面。Mk II 机器会提高产出，但不能替代地理扩张。新的前哨站让你可以在更多地点建造更多机器，从而倍增生产。

扩张模式

- 当家园占地拥挤或旅行浪费时间时，添加一座前哨站。
- 先带来电力，再带来该地点所需的地球化机器。
- 让每个地点都易于理解：少量机器、清晰的储存位置，以及明确的脚本职责。
- 研究完成后就升级机器，但要预期用更多前哨站在合理时间内承载下一阶段。

要留意什么

只有一座前哨站时，后期阶段通常会运行得很慢。当旅行时间、占地空间或本地电力成为瓶颈时，添加前哨站。如果仅交换一次机器等级就让进度跳得过远，那是值得报告的经济平衡调校反馈。

指南 / 编程

变量

变量用于存储之后可以使用的值。

```
x = 10
name = "oxygen"
active = True
```

你可以在表达式中使用变量：

```
a = 5
b = 3
result = a + b
print(result)
```

输出：8

指南 / 编程

运算符

运算符用于组合值、比较值和构建条件。

赋值与相等

使用 `=` 存储一个值。使用 `==` 比较两个值。脚本不使用 `===`。

```
target = 10
if value == target:
    print("match")
```

`=` 会改变变量。`==` 会提出一个问题，并返回 `True` 或 `False`。

比较

```
a == b    # equal
a != b    # not equal
a < b     # less than
a <= b    # less than or equal
a > b     # greater than
a >= b    # greater than or equal
```

布尔逻辑

使用 `and`、`or` 和 `not` 组合条件：

```
if powered and battery > 100:
    print("ready")
if not blocked:
    print("clear")
```

数学与除法

```
a + b    # add
a - b    # subtract
a * b    # multiply
a / b    # divide
a // b   # floor division
a % b    # remainder, also called modulo
a ** b   # power
```

当你想要普通除法时使用 `/`。当你想要整数商时使用 `//`。当你想要除法后的余数时使用 `%`。

模运算 (modulo) 适合重复模式和奇偶检查：

```
value % 2 == 0 # even
value % 2 == 1 # odd
```

成员关系与 None

使用 `in` 检查一个值是否位于列表、元组、集合、字符串或字典键中。对特殊的空值使用 `is None`。

```
if target in scanned:
    print("known")
if result is None:
    print("nothing found")
```

指南 / 编程

数字

概览

数字属于同一个家族，不存在分别存储的 `int` 类型和 `float` 类型。`int`、`float` 和 `number` 是同一个值的三种视图，具体标签由值本身决定。

`int`、`float`、`number`

`int`，整数值（没有小数部分）

`float`，带小数部分的值

`number`，以上两者之一

标签取决于当前值，而不是你书写它的方式：

```
type(5)      # int
type(0.6)    # float
type(4.0)    # int    (whole value)
type(10 / 2) # int    (10 / 2 is 5.0)
type(10 / 3) # float
```

这不是 Python 的模型。在 Python 中，`type(4.0)` 是 `float`，因为字面量写成了浮点数。在这里它是 `int`，因为值是整数；数字本身相同，只有标签不同。

检查数字

```
isinstance(x, int)      # True if x is whole
isinstance(x, float)    # True if x has a decimal part
isinstance(x, int | float) # True for either numeric category
isinstance(x, "number") # True for any number
x % 1 == 0              # True if x is whole
```

当你只关心某个值是否为数字时，使用 `isinstance(x, "number")`；使用 `int(x)` 将值截断为整数（`int(3.7)` 是 3）。

精度

十进制数字使用标准浮点算术，因此可能出现微小的舍入误差：

```
0.1 + 0.2      # 0.30000000000000004, not exactly 0.3
```

对于计算得出的十进制值，这是正常现象。当你的意思是“足够接近”而不是逐位相等时，使用 `isclose()`：

```
if isclose(0.1 + 0.2, 0.3):
    print("equal for this calculation")
```

默认的 `rel_tol` 会根据值的大小处理普通舍入。当可接受的差值具有实际游戏含义时，使用 `abs_tol`。例如，下面的代码会把两个坐标轴上都距目标半米以内的 Rover 视为到达目标：

```
target_x = 100
target_y = 40
pos = self.nav.get_position()

if isclose(pos.x, target_x, abs_tol=0.5) and isclose(pos.y, target_y, abs_tol=0.5):
    print("at target")
```

根据值所表示的内容选择容差，例如米、Wh 或 0 到 1 的等级。只知道某个值是浮点数，并不能决定正确的游戏容差。

指南 / 编程

Print

使用 `print()` 将普通文本输出到控制台。

```
print("Hello")
print(42)
```

你可以打印多个值，它们之间会以空格分隔：

```
temp = -63
print("Temperature:", temp, "°C")
```

对于希望在持久控制台中醒目显示、但不显示提示框的监视消息，使用 `warn()`：

```
if self.efficiency() < 100:
    warn("oxygen generator below 100% efficiency")
```

对于只想在调试脚本时查看的嘈杂遥测信息，使用 `debug()`。除非在控制台选项菜单中启用调试输出，否则 Debug 行会从 ALL 中隐藏：

```
debug("target", target_sector, "heat", self.get_heat())
```

控制台可以显示 THIS SCRIPT、ALL 输出、WARNINGS 或 ERRORS。

指南 / 编程

Self 与组件

Code: Terraform 中的大多数脚本都在机器内部运行。Scanner 脚本运行在 Scanner 内；Bio Lab 脚本运行在 Lab 内。在该脚本中，`self` 表示这台机器。

```
scan = self.scan("E14")
analysis = self.analyze()
if analysis.status == "ok":
    info = analysis.info
```

硬件动作是本地的

改变物理状态或消耗游戏时间的动作通常是 仅限 SELF。保存它们的结果，根据 `.status` 进行分支并读取 `.message`；专用结果还会添加载荷字段。

```
move = self.move("E14")
if move.status == "ok":
    pickup = self.collect()
```

读取另一个组件

使用 `get_component(id)` 进行只读协调：

```
scanner = get_component("scanner_1")
scanned = scanner.get_scanned()
```

有时，本地动作会接受另一个组件引用：

```
collector = get_component("bio_collector_1")
taken = self.take_from(collector)
if taken.status != "ok":
    print(taken.message)
```

该动作仍然属于 Bio Lab 的 `self`；它不是对 Collector 的远程控制。使用共享状态或 Signal Bus 消息来协调不同机器的脚本。

对于长时间运行的脚本，像 `"bio_collector_1"` 这样的稳定 ID 最安全。显示名称使用方便，但可能发生变化。

指南 / 编程

循环与脚本

脚本默认只运行一次。要让脚本持续运行，请使用 `while` 循环：

```
while True:
    # this runs every game tick
    print("running")
```

解释器会在循环运行时自动把时间还给游戏。世界系统和其他脚本会继续运行，因此不需要 `sleep()` 就能让循环安全。

只有当你有意让游戏时间在脚本继续前流逝时，才使用 `sleep()`：

```
while True:
    print("every 5 seconds")
    sleep(5)
```

`for` 循环会遍历一个列表：

```
for i in range(5):
    print(i)
```

有关如何设计循环，使其重新读取机器状态，并在游戏加载后正在运行的脚本再次启动时保持安全，请参阅《长时间运行的脚本》。

指南 / 编程

Match / Case

`match` 允许脚本根据值的形状进行分支，而不必编写很长的一串 `if` 检查。

```
packet = ["ore", 12]

match packet:
    case ["ore", amount] if amount > 0:
        print("ore", amount)
    case ["ice", amount]:
        print("ice", amount)
    case _:
        print("unknown packet")
```

模式

- 字面量模式：`case "ok":`、`case 0:`、`case True:`、`case None:`
- 通配符：`case _:`
- 捕获：`case amount:`
- 替代项：`case "busy" | "cooling":`
- 守卫：`case [kind, amount] if amount > 0:`
- 列表/元组模式：`case [x, y]:`、`case [head, *rest]:`
- 字典模式：`case {"status": "ok", "value": value}:`

字典风格的模式也会按字段名匹配结果对象：

```
result = self.input.take("iron_ore", 10)
match result:
    case {"status": "ok", "moved": moved}:
        print("moved", moved)
    case {"status": reason, "message": message}:
        warn(reason, message)
```

像 `case Thing(x)`：这样的类模式不属于该解释器。请改用列表、元组、字典、字面量和字段名模式。

指南 / 编程

转换函数

使用内置函数在类型之间转换。

```
str(42)      # "42"  
int("7")    # 7  
float("3.14") # 3.14
```

字符转换可用于构建区域 ID（A1、B2 等）：

```
chr(65)      # "A"  
chr(66)      # "B"  
ord("A")    # 65
```

示例：生成区域名称：

```
row = 0  
while row < 8:  
    letter = chr(65 + row)  
    sector = letter + str(1)  
    print(sector) # A1, B1, C1...  
    row = row + 1
```

指南 / 编程

字符串与 f 字符串 (Strings & F-strings)

f 字符串让你可以直接在字符串中嵌入表达式：

```
heat = 45
print(f"Heat: {heat}/100")
name = "Crystal Shard"
print(f"Found {name} worth {300} credits")
```

切片可以提取字符串或列表的一部分：

```
pos = "E13"
letter = pos[0]    # "E"
column = pos[1:]  # "13"
first3 = pos[:3]   # "E13"
```

使用 `in` 检查成员关系：

```
if "A1" in scanned_list:
    print("already scanned")

if sector not in visited:
    print("new sector")
```

`in` 可用于列表、字典（检查键）和字符串（检查子字符串）。

字符串方法 (STRING METHODS)

- `.upper()/.lower()`, 大小写转换
- `.strip()/.lstrip()/.rstrip()`, 移除空白
- `.split(sep)`, 拆分为列表
- `.join(list)`, 将列表连接为字符串：", ".`join(["a", "b"])` → "a, b"
- `.find(sub)`, 查找子字符串的索引, 找不到时返回 -1
- `.index(sub)`, 类似 `find`, 但找不到时抛出错误
- `.replace(old, new)`, 替换所有出现的内容
- `.startswith(s)/.endswith(s)`, 检查前缀/后缀
- `.count(sub)`, 统计出现次数
- `.title()/.capitalize()`, 标题大小写 / 首字母大写
- `.isdigit()/.isalpha()/.isalnum()/.isspace()`, 字符检查
- `.zfill(width)`, 用零填充
- `.center(w)/.ljust(w)/.rjust(w)`, 对齐

```
.format(args), "Hello {}".format("world")
```

指南 / 编程

正则表达式 (Regular Expressions)

当普通字符串方法不够用时，使用内置的 `re` 模块：匹配模式、提取分组、替换形状可变的文本，或按多个分隔符拆分。

`re` 是内置模块。它无需共享库研究即可工作，但仍要导入它，这样脚本就能明确表明自己正在使用正则表达式：

```
import re

text = "ore_42 at E13"
match = re.search("ore_(\d+)", text)
if match:
    print(match.group(1))    # 42
```

匹配辅助函数 (MATCH HELPERS)

- `re.search(pattern, string, flags=0)`，在任意位置查找第一个匹配项
- `re.match(pattern, string, flags=0)`，只匹配开头
- `re.fullmatch(pattern, string, flags=0)`，匹配整个字符串

这些函数会返回一个 `Match` 对象，或者在没有匹配项时返回 `None`。

```
import re

m = re.fullmatch("([A-Z])(\d+)", "E13")
if m is not None:
    print(m.group(0))    # E13
    print(m.group(1))    # E
    print(m.group(2))    # 13
    print(m.span())      # (0, 3)
```

`Match.group(0)` 是完整匹配结果。捕获分组从 1 开始。`Match.groups()` 会以元组形式返回所有捕获分组。

列表与替换 (LISTS AND REPLACEMENTS)

```
import re

print(re.findall("\d+", "A12 B7"))
print(re.split("[,;]\s*", "iron, ice; quartz"))
print(re.sub("ore_(\d+)", "ore-\1", "ore_42"))
```

当模式没有分组时，`re.findall()` 返回字符串；有一个分组时返回一个捕获值；有多个分组时返回元组。`re.sub()` 的替换文本支持数字反向引用（numeric backreference），例如 `\\1` 和 `\\g<1>`。

标志（FLAGS）

标志是模块常量。使用 `|` 将它们组合：

```
import re

flags = re.IGNORECASE | re.MULTILINE
print(re.findall("^ore", "Ore\nice", flags))
```

可用标志：

- `re.IGNORECASE` / `re.I`
- `re.MULTILINE` / `re.M`
- `re.DOTALL` / `re.S`

语法说明（SYNTAX NOTE）

Code: Terraform 提供了一个 Python 风格的 `re` API，其背后由线性时间模式引擎支持。常见模式（如 `\d+`、`[A-Z]+`、`.`、`*`、`^`、`$`、分组 `(...)` 以及交替 `a|b`）均受支持。仅回溯功能支持的特性（如环视和模式反向引用）不受支持。`re.sub()` 中的替换反向引用仍受支持。

模式长度上限为 512 个字符。像 `(a+)+` 这样的量词分组可以安全使用，因为匹配不会回溯。

指南 / 编程

列表与元组 (Lists & Tuples)

列表是有序集合，使用方括号创建。元组是固定的有序集合；当你需要稳定的二元组、坐标或小型记录时，使用圆括号创建：

```
items = [1, 2, 3]
empty = []
mixed = ["hello", 42, True]
point = (12, 8)
single = (42,)
```

按索引访问（从 0 开始；负数索引从末尾开始）：

```
first = items[0]    # 1
last = items[-1]   # 3
x = point[0]        # 12
```

使用步长切片：

```
items[1:3]          # [2, 3]
items[::2]           # [1, 3], every other
items[::-1]          # [3, 2, 1], reversed
point[:]             # (12, 8)
```

列表推导式，以简洁方式构建列表：

```
squares = [x * x for x in range(10)]
evens = [x for x in range(20) if x % 2 == 0]
```

元组解包与多重赋值：

```
a, b, c = [1, 2, 3]
first, second = ("hello", "world")

def choose_route():
    return "E14", ["E13", "E14"]

target, route = choose_route()

routes = [(0, "nav_module"), (1, "cargo_rack")]
for slot, item in routes:
    print(slot, item)
```

列表方法 (LIST METHODS)

- `.append(value)`, 添加到末尾
- `.pop()`, 移除并返回最后一项
- `.insert(index, value)`, 插入到指定位置
- `.remove(value)`, 移除第一次出现的值
- `.index(value)`, 查找值的位置
- `.count(value)`, 统计出现次数
- `.sort()`, 就地排序
- `.reverse()`, 就地反转
- `.copy()`, 浅复制
- `.extend(list_or_tuple)`, 添加另一个序列中的所有项
- `.clear()`, 移除所有项
- `.length`, 项数

元组不可变。它们支持索引、切片、`.index(value)`、`.count(value)`、`.length`、`len()` 和 `for` 循环。

指南 / 编程

字典 (Dictionaries)

字典存储键值对。键可以是任何可哈希值：字符串、数字、布尔值、**None**，或仅由可哈希值组成的元组。字符串键最常用于游戏数据：

```
data = {"name": "Crystal", "value": 300}
empty = dict()
```

访问和修改：

```
print(data["name"])      # Crystal
data["quality"] = "high"
coords = {}
coords[(0, 0)] = "base"
```

检查键是否存在：

```
if "name" in data:
    print(data["name"])
```

迭代：

```
for key in data.keys():
    print(key, data[key])

for pair in data.items():
    key = pair[0]
    val = pair[1]
    print(f"{key}: {val}")
```

合并 (MERGE)

使用 `left | right` 创建新的合并字典。使用 `left |= right` 就地更新现有字典。当同一个键同时出现在两边时，右侧的值优先。

```
thing = {"a": 1, "b": 2}
other = {"a": 3, "c": 4}
print(thing | other) # {"a": 3, "b": 2, "c": 4}
```

方法 (METHODS)

- `.keys()`，所有键的列表
- `.values()`，所有值的列表

- `.items()`, `[key, value]` 对的列表
- `.get(key, default)`, 获取值；缺失时返回默认值
- `.has(key)`, 检查键是否存在
- `.pop(key, default)`, 移除键并返回值
- `.popitem()`, 移除并返回最后插入的 `[key, value]` 对
- `.update(dict)`, 合并另一个字典
- `.setdefault(key, default)`, 获取或设置默认值
- `.clear()`, 移除所有条目

指南 / 编程

内置函数概览 (Built-in Functions Overview)

这些函数始终可用。

输出 (OUTPUT)

- `print(values...)`, 向控制台打印普通文本
- `warn(values...)`, 向控制台的 WARNINGS 视图打印一行琥珀色警告
- `debug(values...)`, 打印低优先级遥测信息；除非启用调试输出, 否则对 ALL 隐藏
- `notify(text, level?, duration?)`, 显示提示消息, 并将其归档到 Notifications 中

类型 (TYPES)

- `type(value)`, 返回基于值的类别, 例如整数的 `int`、小数的 `float`、`str` 和 `bool`。数字类别使用当前值, 因此 `type(4.0)` 是 `int`。
- `isinstance(value, type)`, 检查类型/类别: `isinstance(name, str)`、`isinstance(n, int)`、`isinstance(n, int | float)`、`isinstance(x, (int, str))`, 或宽泛的 `isinstance(n, "number")`
- `callable(value)`, 如果值是函数、方法、类, 或具有可调用 `__call__` 的对象, 则为 `True`
- `len(value)`, 字符串、列表、元组、字典或集合的长度
- `bool(value)`, 转换为 `True/False`

转换 (CONVERSION)

- `str(value)`、`int(value, base?)`、`float(value)`
- `list(value)`、`tuple(value)`、`set(value)`、`dict(...)`, 类型构造器
- `chr(code)`、`ord(char)`
- `hex(n)`、`bin(n)`、`oct(n)`, 将整数转换为带进制前缀的字符串
- `repr(value)`, 便于调试的字符串表示

数学 (MATH)

- `abs(n)`、`round(n, digits?)`、`pow(base, exp)`
- `isclose(a, b, rel_tol=0.000000001, abs_tol=0.0)`, 比较浮点结果是否在可接受的差异范围内
- `min(values...)/max(values...)`, 参数、列表或元组中的最小值/最大值
- `sum(sequence, start=0)/prod(sequence, start=1)`, 将数值项相加或相乘

- `random()/rand()`, 从0（含）到1（不含）的随机浮点数
- `randint(min, max)`, 包含边界的随机整数
- `import random`, 模块形式, 包含 `random.random()`、`random.rand()` 和 `random.randint()`
- `floor(n)`、`ceil(n)`、`trunc(n)`、`sign(n)`、`divmod(a, b)`
- `sqrt(n)`、`exp(n)`、`log(n)`、`log2(n)`、`log10(n)`
- `sin(n)`、`cos(n)`、`tan(n)`、`asin`、`acos`、`atan`、`atan2(y, x)`
- `degrees(rad)`、`radians(deg)`
- `inf`, 正无穷; 负无穷使用 `-inf`
- `pi`、`tau`, 常量

序列 (SEQUENCES)

- `range(stop)/range(start, stop, step)`
- `sorted(sequence, key=fn, reverse=False)`, 新的已排序列表。传入 `key=lambda x: x.value` 可按计算字段排序; `reverse=True` 表示降序。
- `reversed(sequence)`, 新的反转序列
- `enumerate(sequence, start=0)`, `(index, value)` 对的列表
- `zip(sequence1, sequence2, ..., strict=False)`, 将序列组合成对; `strict=True` 时, 长度不同时抛出错误
- `map(fn, sequence)/filter(fn, sequence)/reduce(fn, sequence, initializer?)`, 转换、筛选或折叠项
- `pairwise(sequence)`, 相邻的 `(a, b)` 对
- `batched(sequence, size)`, 固定大小的元组批次
- `starmap(fn, sequence)`, 将元组/列表项解包为参数并调用 `fn`
- `flatten(sequence)`, 展开一层嵌套
- `count_by(sequence, key_fn?)`, 将值或计算出的键计数到字典中
- `iter(sequence)/next(iterator)`, 手动迭代
- `chain(seq1, seq2, ...)`, 连接序列
- `accumulate(sequence)`, 累计总和
- `combinations(sequence, r)/permutations(sequence, r)`, 组合与排列
- `product(seq1, seq2, ...)`, 笛卡尔积

逻辑 (LOGIC)

- `all(sequence)`, 如果每一项都为真值则为 `True`
- `any(sequence)`, 如果任一项为真值则为 `True`

模块 (MODULES)

- `import random`, 随机数辅助函数
- `from functools import reduce`, 作为模块导入的归约辅助函数
- `import re`, 正则表达式: `search`、`match`、`fullmatch`、`findall`、`sub` 和 `split`
- `from dataclasses import dataclass, field`, 生成记录类及字段配置

计时 (TIMING)

- `sleep(seconds)`, 有意等待后继续。循环会自动调节节奏；只有需要让游戏时间流逝时才使用它。

其他 (OTHER)

- `hash(value)`, 对字符串、数字、布尔值、`None`、仅含可哈希值的元组或类实例进行哈希（默认使用身份；定义了 `__hash__` 时使用它）

指南 / 编程

异常 (Exceptions)

当调用者提供了不可能的输入，或某个假设已经失效时，异常可以让辅助函数以命名错误停止。将它们用于错误和格式错误的调用。预期的游戏结果（例如货物已满、没有扫描到任何内容或队列为空）应放在命令结果对象中：根据 `.status` 分支，并读取 `.message`。这些对象是普通返回值，而不是异常。

引发错误 (RAISE AN ERROR)

使用零个或一个清晰消息调用异常构造器，然后引发它：

```
def normalize_sector(sector):
    if not isinstance(sector, str):
        raise TypeError(f"normalize_sector: expected str, got {type(sector)}")
    if len(sector) < 2:
        raise ValueError("normalize_sector: expected a sector like A1")
    return sector
```

`type("A1")` 返回 `str`，而不是 `string`。当你需要真正的 `true/false` 类型检查时，优先使用 `isinstance(value, str)`。

API 边界 (API BOUNDARY)

游戏玩法 API 遵循相同的区分。错误的参数类型会引发 `TypeError`；类型正确但值无效时，会引发 `ValueError` 或其他有文档说明的异常。普通世界状态不会引发异常：命令会返回结果对象，因此脚本可以处理 `"busy"`、`"no_cargo_space"`、`"not_found"` 等结果，而无需猜测。

捕获并继续运行 (CATCH AND KEEP RUNNING)

当脚本可以恢复并继续时，使用 `try/except`。as

后的变量是异常对象：打印它会显示其消息；`type(error)`、`isinstance(error, ValueError)`、身份检查以及 `raise error` 会保留其异常类和身份。

```
target = None
try:
    target = normalize_sector(command.args["sector"])
except KeyError as error:
    warn("command missing sector", error)
except (TypeError, ValueError) as error:
    warn(error)

if target is not None:
    print("target", target)
```

捕获有意义的最窄错误范围。`except Exception as error:` 会捕获从 `Exception` 派生的普通游戏脚本异常，这在边界处很有用，但如果到处使用，可能会隐藏错误。它不会捕获直接从 `BaseException` 派生的控制流异常，例如 `GeneratorExit`。处理器可以命名一个受支持的异常、一个带点号的别名（如 `errors.ValueError`），或一个元组（如 `except (KeyError, TypeError):`）。计算出的处理器表达式不属于游戏脚本表面。处理器名称在捕获错误时解析，因此别名和遮蔽行为正常。`as error` 目标只存在于该处理器内部，并会在每次退出时被清除；如果之后仍需要某些细节，请将其复制到另一个变量中。

清理与重新引发（CLEANUP AND RE-RAISING）

`try` 块可以有 `else`，它只在 `try` 主体无异常完成时运行；也可以有 `finally`，它会在控制流通过成功、错误、`return`、`break` 或 `continue` 离开之前始终运行。因此，`finally` 中产生的新错误或控制流退出会替换待处理的错误，所以应让清理操作小而可预测。在活动处理器或其 `finally` 块中使用不带参数的 `raise`，会重新引发同一个异常身份。

`raise RuntimeError("message") from cause` 会计算并验证 `cause`，但游戏控制台有意只显示所引发的异常。链式回溯元数据不属于显示的运行时表面。

内置名称（BUILT-IN NAMES）

常见选择包括：`ValueError` 表示类型正确但内容错误的值，`TypeError` 表示值的类型错误，`KeyError` 表示字典键缺失，`IndexError` 表示列表索引错误，`AttributeError` 表示字段或方法缺失，以及 `RuntimeError` 表示运行时某个假设失败。

其他受支持的名称包括 `NameError`、`UnboundLocalError`、`ImportError`、`ModuleNotFoundError`、`ZeroDivisionError`、`StopIteration`、`GeneratorExit`、`AssertionError`、`NotImplementedError`、`RecursionError`、`IndentationError`、`OverflowError` 和 `SyntaxError`。完成的生成器会引发 `StopIteration`；它的 `.value` 是生成器的返回值。`GeneratorExit` 由 `generator.close()` 使用，通常应放在生成器清理代码中。

你无法在脚本中定义新的异常类。选择最接近的内置名称，并让消息足够具体，以便控制台能告诉你问题来自哪里。

指南 / 编程

实用辅助函数 (Utility Helpers)

小型辅助函数在每个脚本中始终可用。当你需要快速生成随机数、进行边界安全的坐标计算、执行数学运算、转换集合，或输出控制台信息，而又不想翻查完整的内置函数参考时，请使用本页。

随机数 (RANDOM NUMBERS)

当你希望代码读起来像普通 Python 时，使用 Python 风格的模块形式：

```
import random as rng

roll = rng.randint(1, 6)
jitter = rng.random()
print("roll", roll, "jitter", jitter)
```

`random.random()` 返回从 0（含）到 1（不含）的浮点数。`random.rand()` 是相同的辅助函数，只是名称更短。`random.randint(min, max)` 返回介于 `min` 和 `max` 之间的整数，包括两个端点。

你也可以直接全局调用相同的辅助函数：

```
roll = randint(1, 6)
jitter = rand()
```

如果你直接 `import random` 导入模块，那么名称 `random` 在该脚本中指向模块。这没有问题；调用浮点辅助函数时使用 `random.random()`。

随机有效坐标 (RANDOM VALID COORDINATES)

行星边界与 `randint` 是很好的搭档：

```
planet = get_component("nocturna")
b = planet.get_bounds()

x = randint(b.min_x, b.max_x)
y = randint(b.min_y, b.max_y)

if planet.contains(x, y):
    print("valid target", x, y)
```

数值辅助函数 (NUMERIC HELPERS)

常见数学辅助函数是内置的：

- `min(...)/max(...)`, 选择最小值或最大值
- `sum(items, start=0)/prod(items, start=1)`, 将数值相加或相乘
- `round(n, digits?)`、`floor(n)`、`ceil(n)`、`trunc(n)`, 调整数字形态
- `abs(n)`、`sign(n)`、`divmod(a, b)`, 距离、方向和商/余数
- `sqrt(n)`、`sin(n)`、`cos(n)`、`atan2(y, x)`, 几何和转向辅助函数
- `degrees(rad)/radians(deg)`, 转换角度单位

示例：

```
distance = 42.8
whole = ceil(distance)
hours, minutes = divmod(135, 60)
print(whole, hours, minutes)
```

集合辅助函数 (COLLECTION HELPERS)

这些函数可处理列表、元组、字符串、字典和集合：

- `len(value)`, 长度
- `range(...)`, 用于循环的整数序列
- `sorted(sequence, key=fn, reverse=False)`, 排序副本
- `reversed(sequence)`, 反转副本
- `enumerate(sequence)`, (index, value) 对
- `zip(a, b, ..., strict=False)`, 组合序列；`strict=True` 时长度不同时抛出错误
- `map(fn, sequence)/filter(fn, sequence)/reduce(fn, sequence, initializer?)`, 转换、筛选或折叠值
- `pairwise(sequence)`, 相邻对, 适用于路线片段
- `batched(sequence, size)`, 按块处理, 用于分页大小的工作或重复命令
- `starmap(fn, sequence)`, 将元组/列表行解包到函数调用中
- `flatten(sequence)`, 将嵌套路由或货物列表展开一层
- `count_by(sequence, key_fn?)`, 计数到字典中, 也可以按计算出的键计数
- `all(sequence)/any(sequence)`, 布尔检查

示例：

```
items = ["iron_ore", "ice", "quartz"]
for index, item in enumerate(sorted(items)):
    print(index, item)

print(count_by(["ice", "ore", "ice"]))
```


控制台辅助函数 (CONSOLE HELPERS)

使用 `print()` 输出普通内容, 使用 `warn()` 输出持久警告, 使用 `debug()` 输出低优先级遥测信息, 使用 `notify(text, level?)` 显示屏幕提醒。当脚本有意管理嘈杂的显示内容时, 使用 Console 组件。

```
warn("storage nearly full")
debug("loop heartbeat")
notify("Rover battery low", "warn")
```

```
console = get_component("console")
console.clear("alarms")
```

要查看完整列表和精确签名, 请打开 Built-in Functions。

指南 / 编程

导入与库 (Imports & Libraries)

导入有几种明确类型：可执行内置模块、仅用于类型检查的支持模块、编译器指令、显式共享根，以及你自己的 Library 脚本。Library 脚本需要通过 Shared Library 研究解锁；内置和支持命名空间不需要该研究。

可执行内置模块 (EXECUTABLE BUILT-IN MODULES)

使用 `import random`，然后调用 `random.randint(1, 10)`；使用 `from functools import reduce` 执行归约式算法；使用 `import re` 处理正则表达式；或使用 `from dataclasses import dataclass, field` 生成记录类。你也可以直接将 `random()`、`rand()`、`randint(min, max)` 和 `reduce(fn, iterable, initializer?)` 作为全局辅助函数调用。正则表达式辅助函数保留在 `re` 模块中，以便模式匹配代码明确；`dataclass` 和 `field` 同样需要导入其标准模块。

类型检查与编辑器支持 (TYPING AND EDITOR SUPPORT)

`typing`、`types`、`collections.abc` 和 `user_stubs`

为注解和编辑器分析提供名称。脚本运行时，这些类型名称会被擦除；它们不是放置可执行辅助函数的地方。游戏中的 `typing.TYPE_CHECKING` 始终为 `False`。`collections` 是访问 `collections.abc` 所使用的包。将可执行的共享代码放进 `Library`，而不是 `user_stubs.py`。

`from __future__ import annotations` 是编译器指令，而不是普通绑定。将 `future` 指令放在模块开头：可位于可选的模块文档字符串之后，但必须位于普通语句之前。该指令本身不会创建 `annotations` 名称。

`__builtins__` 是共享解释器和游戏根的显式导入视图，例如 `from __builtins__ import len, get_component`。脚本所有者的局部名称（如 `self` 和 `panel`）不属于该共享模块。生成的 `builtins` 和 `code_terraform` 存根只用于外部编辑器的类型检查，无法由正在运行的游戏脚本导入。

玩家库 (PLAYER LIBRARIES)

研究 Ship Computer 和 Shared Library 后，在左侧边栏打开 Ship → Computer → Library，然后点击 + New。Library 脚本是共享代码：编写一次辅助函数，就可以从任意机器脚本中导入它。

创建一个名为 `sensors` 的 Library 脚本：

```
def temp():
    """Read the current thermometer value."""
    return get_component("thermometer").get_value()

def o2():
    """Read the current oxygen sensor value."""
    return get_component("oxygen_sensor").get_value()
```

导入指定函数：

```
from sensors import temp

print(temp())
```

从库中导入所有公开名称：

```
from sensors import *

print(temp())
print(o2())
```

以 `_` 开头的名称保持私有，不会被 `*` 导入。也可以导入整个模块：

```
import sensors

print(sensors.o2())
```

Library 脚本在自己的共享作用域中运行。内置函数和顶层游戏函数可用，但调用者的局部名称不可用。Library 内部没有定义 `self` 和 `panel`，因此当辅助函数需要为导入脚本执行操作时，请将组件、组件 ID 或其他上下文传递给它。

从导入库中得到的函数文档字符串会显示在悬停提示和自动补全中，因此共享辅助函数可以记录自己的参数、返回值、异常和固定结果契约。

指南 / 编程

文档字符串 (Docstrings)

文档字符串是放在函数体顶部的字符串字面量。它们不会改变函数的运行方式；它们用于记录函数的用途。

当你将鼠标悬停在用户定义的函数上，或在自动补全中看到它时，编辑器会显示它的签名、文档字符串，以及（如果存在）返回注解。导入的 Library 函数也会显示文档字符串。当你希望工具提示单独显示返回值一行时，添加如 `-> str | None` 这样的返回注解。

```
def next_needed(order) -> str | None:
    """Pick the next missing item for an order.

    [[helper]]

    ## Returns

    - a fragment id when one is still required
    - `None` when the order already has everything it needs
    """
    for item in order.requires.keys():
        if order.delivered.get(item, 0) < order.requires[item]:
            return item
    return None
```

格式化 (FORMATTING)

文档字符串使用与 DOCS 描述相同的轻量 Markdown 格式：

- 行内代码：`status`、`"ok"`、`self.deliver()`
- 粗体文本：`important`
- 徽章：`HELPER` 或 `SELF ONLY`
- 标题：`## Returns`
- 提示框：以 `>` 开头的行
- 项目符号：`- item` 或 `• item`
- 编号列表：`1. item`
- 使用 `|` 分隔符的表格
- 使用三个反引号围起的围栏代码块

如果希望悬停提示将各部分渲染为独立区块，请在部分之间留空行。

变体 (VARIANTS)

脚本最顶部的三引号字符串是脚本描述，而不是函数文档字符串。Variants

选项卡会使用该文件顶部文档字符串的第一行非空内容作为变体描述。如果没有文件顶部的文档字符串，则使用第一个 `#` 注释。

```
"""Night route: charge first, then collect ice."""
```

```
while True:  
    print("working")
```

对于函数级帮助，将文档字符串直接放在 **def** 下方。对于变体摘要，将它放在文件顶部、所有代码之前。

编程指南

编写类

类把数据（属性）与行为（方法）捆绑成一个可复用的形状。实例是其中一个已填充的副本。

```
class Counter:
    def __init__(self, start):
        self.n = start      # an instance attribute
    def inc(self):
        self.n += 1

c = Counter(10)
c.inc()
print(c.n)                # 11
```

SELF、__INIT__ 与属性

`__init__(self, ...)` 会在你写出 `Counter(10)` 时运行一次，用于设置新实例。每个方法都把 `self`（该实例）作为第一个参数，而 `self.x = ...` 会创建实例属性。在类体中赋值的值（`limit = 5`）是类属性，由所有实例共享，并可通过 `Counter.limit` 或 `c.limit` 读取。

机器脚本中的 SELF

在机器自己的脚本中，顶层的 `self` 就是机器（`self.mine()`）。在类方法内部，`self` 是该对象；两者不会冲突，因为方法的 `self` 只是它的第一个参数。

如果方法需要驱动机器，就把机器传进去：在顶层（此处 `self` 是机器）使用 `Planner(self)`，把它保存起来（`self.bot = bot`），然后在方法内调用 `self.bot.mine()`。

继承与 SUPER()

`class Dog(Animal):` 会继承 `Animal` 的方法和属性。你可以覆盖其中任意成员，并用 `super()` 访问基类版本：

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        return "..."

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)    # run Animal's setup first
        self.breed = breed
    def speak(self):
        return self.name + " says woof"

d = Dog("Rex", "husky")
print(d.speak())    # Rex says woof
print(isinstance(d, Animal), isinstance(Dog, Animal)) # True True
```

多重继承使用 Python 的 C3 方法解析顺序（method-resolution order, MRO），因此协作式 `super()` 可以跨越菱形继承结构工作，链中的每个 `__init__` 都只运行一次。

运算符是双下划线方法（Dunder Methods）的语法糖

运算符和内置函数实际上都是对 `__dunder__` 方法的语法糖。定义这些方法，就能让你的对象表现得像内置类型：

```
class Vec:
    def __init__(self, x):
        self.x = x
    def __add__(self, o):
        return Vec(self.x + o.x)    # powers a + b
    def __eq__(self, o):
        return self.x == o.x        # powers a == b
    def __repr__(self):
        return "Vec(" + str(self.x) + ")"    # how print() shows it
```

`__lt__` 支持 `<`，`__len__` 支持 `len()` 和真值判断，`__getitem__` 支持 `x[k]`，`__contains__` 支持 `in`，`__iter__` 支持 `for`，而 `__call__` 让实例可以被调用。

运算符重载必须是纯函数，不能调用 `sleep()` 或改变游戏状态。普通方法、`__init__` 和 `__call__` 可以这样做。

@PROPERTY 与方法装饰器

`@property` 会把方法转换成无需括号即可读取的属性；添加 `@name.setter` 可允许赋值：

```
class Tank:
    def __init__(self):
        self._level = 0
    @property
    def level(self):
        return self._level
    @level.setter
    def level(self, v):
        self._level = max(0, v)

t = Tank()
t.level = 42          # runs the setter
print(t.level)        # runs the getter
```

`@staticmethod` 定义没有 `self` 的方法；`@classmethod` 接收作为 `cls` 的类。任何函数都可以成为装饰器（`@deco` 会改写成 `name = deco(name)`）。`from functools import total_ordering` 会根据 `__eq__` 加上 `__lt__` / `__le__` / `__gt__` / `__ge__` 中的一个，补齐其余比较方法。

数据类 (Dataclasses)

当一个类主要是对命名字段的声明、但仍需要类行为时，使用内置的 `dataclasses` 模块。该装饰器会生成按声明顺序构造、可读的表示形式、值相等性以及可选排序；`__post_init__` 是在生成的赋值之后执行验证的地方。

```
from dataclasses import dataclass, field

@dataclass(order=True)
class Job:
    priority: int
    label: str = ""
    tags: list[str] = field(default_factory=list, compare=False)

    def __post_init__(self):
        if self.priority < 0:
            raise ValueError("priority must be non-negative")

a = Job(2, "ice run")
b = Job(1, "ore run")
a.tags.append("cold")
print(b < a, a)
```

default_factory

会为每个实例创建独立的可变值。列表、字典、集合等可变或其他不可哈希的直接默认值会被拒绝；应对它们使用 `default_factory`。`dataclass` 支持 `init`、`repr`、`eq`、`order` 和 `kw_only`。`field` 支持 `default`、`default_factory`、`init`、`repr`、`compare` 和 `kw_only`。继承得到的带注解字段以及显式字段覆盖，都会参与生成的构造函数。

数据类仍然是普通的用户类实例。它不会变成字典，也不能跨越 Signal Bus（信号总线）或 notebook API 这类 JSON 形状的游戏边界。对映射形状的载荷和缓存行使用 `TypedDict`；需要方法、验证、生成式构造、值相等性或排序时使用数据类。

可以传入兼容性拼写 `frozen=False`、`unsafe_hash=False`、`slots=False` 和 `weakref_slot=False`，但不支持它们为 `True` 时的行为。不支持 `match_args`、`ClassVar`、`InitVar`、`MISSING`、`KW_ONLY`、`Field`、`FrozenInstanceError`、`is_dataclass`、`asdict`、`astuple`、`replace`、`fields`、`make_dataclass` 以及公开的 `__dataclass_fields__` 内省。其他情况下，注解类型会被擦除，不会强制执行。只有规范的 `ClassVar` 和 `InitVar` 注解会被识别，以便装饰器清楚地拒绝这些不受支持的字段形式；其他所有字段都来自执行过的带注解名称。

何时使用类

- `Class`，需要自定义初始化、继承或运算符重载。
- `dataclass`，需要命名字段加上生成的构造、显示、相等性或排序。
- `TypedDict`，映射形状的记录或 JSON 载荷（编辑器会自动补全其键）。
- `dict`，键动态、由数据驱动。

不支持的内容

这是功能相当完整的 Python 类实现，但有以下刻意排除项：

- 元类、`__slots__`、`abc/@abstractmethod`。
- 通用描述符协议；只开放 `@property`。
- `__new__`、`__del__`。
- 动态属性钩子：`__getattr__`、`__setattr__`、`__delattr__`。
- `for` 会对高级的 `__iter__`/`__next__` 和生成器进行惰性处理，因此对无限迭代器也能使用 `break`。需要完整结果的操作（例如 `list(...)`）仍受集合上限限制。

定义 `__eq__` 但没有定义 `__hash__` 的类不可哈希，与 Python 一致。定义一个返回整数的 `__hash__`，即可把具有相等性语义的实例用作字典/集合键。两者都未定义的类使用基于身份的哈希，也可以作为键。

指南 / 编辑器与工具

代码编辑器

每台带有脚本槽位的机器都有一个代码编辑器窗口。点击机器卡片上的 MANAGE 即可打开。

标签页

- Info, 任务描述, 说明该脚本应该做什么。
- Code, 在此编写代码。
- Variants, 命名版本, 可在匹配的机器上复用 (见下文)。
- Notes, 供你记录笔记的自由文本草稿区。

标题栏控件

- ▶ Run (Shift+Enter), 执行活动变体。
- ⏸ Pause (F6), 暂停或恢复运行中的脚本。
- ■ Stop (Shift+F5), 完全停止脚本, 并重置脚本拥有的控制项。Rover、Pioneer 和 Drone 的移动会立即取消。
- 🔍 Trace, 每个 tick 高亮脚本停留的位置。绿色条标出最近一个 tick 结束时的“停留”行。它会如实显示挂起点 (对于使用 `sleep()` 或 `scan()` 的脚本), 以及步数上限在紧循环中触发的位置。
- ⚙ Debug, 启用断点、单步执行和变量监视面板。完整流程见 Debug 指南。默认关闭。
- 🔍 Find, 在脚本中搜索和替换 (Ctrl+F)。

机器人栏控件

- Duplicate Variant, 将当前代码保存为新的命名变体。
- 🔇 Mute, 静音此脚本的控制台输出。静音时文本会显示删除线。静音脚本不会向控制台打印; 错误仍会显示。适合让当前不在关注的后台脚本保持安静。
- 📌 Dock, 切换到 IDE 风格布局。编辑器固定在左侧, 控制台停靠在右下角, 仪表板占据剩余空间。侧边栏会自动隐藏, 以最大化内容区域。拖动面板之间的分隔线即可调整大小。停靠布局会跨会话记忆。点击 Undock 可返回浮动窗口。

- Auto-min, 启用时（绿色），运行脚本后编辑器会自动最小化。适合在代码执行时观察仪表盘。

从代码跳转到 DOCS

编辑代码时，编辑器可以直接跳转到匹配的 DOCS 条目：

- 将鼠标悬停在任意方法或函数名上，工具提示会显示签名、一行描述和返回类型。支持 `self.X`、`self.X.Y`、顶层 API 调用（`get_component`、`sleep`、`print`）、内置函数（`len`、`range` 等）、通过 `get_component(id).method` 访问的组件方法，以及带文档字符串的用户定义函数。
- 在任意方法、函数、组件 id 或变量上使用 Cmd+点击（macOS）/ Ctrl+点击（Windows/Linux），会打开 DOCS 并直接跳到对应条目；具体方法会高亮一两秒，让你看到确切落点。这类似专业 IDE 中的“转到定义”，但目标是 DOCS 页面，因为游戏 API 没有源文件。
- 输入时会自动补全（按 Ctrl+Space 可强制触发），建议会感知类型。在 `self.` 后只显示机器上可用的方法和子对象；在 `self.battery.` 后只显示 Battery 子对象的 API。字符串参数会在可用时提供已知有效值（组件 id、箱子名称、矿物 id）。
- 在 (...) 内输入参数时会显示提示。例如输入 `self.drill.mine(` 时，提示会显示每个参数的名称、类型和描述。
- 按 F2 重命名符号（变量、用户函数、参数）。它会重命名当前脚本中每个完整单词的出现位置，并跳过字符串与注释。保留名称（关键字、`self`、内置名称）不能重命名。

将鼠标悬停可查看签名。Cmd/Ctrl+点击可打开匹配的 DOCS 条目而不离开编辑器。

COMMANDS 标签页

脚本运行时，其 Commands 标签页允许你通过带 JSON 参数的命名命令向脚本发送命令，脚本通过 `self.next_command()` 读取并决定如何响应。常用命令可以保存为预设。完整模式见 Script Commands 页面。

变体

命名变体使用分层目录。精确类型变体只属于一种具体机器类型；兼容族变体只在具有相同脚本契约的层级之间共享。Main 是该机器自己的后备代码，从不共享。编辑 Code 标签页会保存到当前活动变体。对 Main 的更改只保留在该脚本中；对命名变体的更改会更新其共享目录条目。其他脚本会继续使用已加载的副本，直到你明确更新它们。使用 Duplicate Variant 可把当前代码发布为新的命名变体，并在存在兼容族时选择其范围。点击 Use 可将目录变体加载到这台机器。Apply to all 会根据拥有该变体的部分，将选中的目录版本应用到精确类型或整个兼容族，并且始终要求脚本槽位相同。Main 不能删除。提示：

在文件顶部添加 `"""docstring"""` 或第一行 `#` 注释，它就会显示为变体描述。打开 Docstrings 可查看函数悬停文档和受支持的格式。

保存

脚本每 30 秒自动保存一次。按 **Ctrl+S**

可立即保存并刷新当前源代码。所有编辑器快捷键（包括撤销、重做、注释、搜索和运行）都可以在 Settings → Keybinds 中更改。

如果脚本包含 `while True:` 循环，它会持续运行；否则运行一次后完成。

重新加载行为

保存为运行中的脚本在加载游戏时会从顶部重新开始。不会恢复其局部变量和当前行。暂停的脚本会保持暂停，但加载后恢复时也会从顶部开始；停止的脚本保持停止。持久化的世界状态和机器工作会保留原状，因此在重复操作前请检查当前货物、存储和机器状态。需要在重新加载后仍保存的脚本进度，应使用 Data Archive。

控制台窗口显示所有未静音脚本的输出。使用 CLEAR 清空控制台，使用 AUTO-SCROLL 跟随新输出。

指南 / 编辑器与工具

Vim 模式

概览

Vim 模式为游戏内代码编辑器提供 vim 键绑定。在 Settings → Editor → Keybindings → Vim 中启用。它是选择加入的功能，只影响代码编辑器；游戏菜单、地图和仪表板控件仍使用正常按键。

可用功能

编辑器使用 vim 风格的键位映射，因此日常工具都可用：普通 / 插入 / 可视模式，移动键（h、j、k、l、w、b、e、gg、G、f、%），运算符（d、c、y、p）及计数和文本对象，搜索（/、?、n、N），寄存器、标记以及撤销 / 重做（u、Ctrl-r）。它是 vim 风格而非 100% 功能完整的 vim，大多数肌肉记忆都能沿用，但少数奇特命令和插件不可用。

保存与退出

以下 ex 命令连接到游戏真实的保存和窗口控件：

- `:w`，立即保存。会立刻把脚本刷新到磁盘（脚本本来就会自动保存，所以这只是让保存的习惯真正生效）。
- `:wq` / `:x`，保存后关闭编辑器窗口。
- `:q`，关闭编辑器窗口。你的工作已经保存。

使用你自己的 Vim

更喜欢实际的 vim 配置？请查看 External Editor 页面。你的脚本会作为磁盘上的真实文件存在，可以用任何编辑器（包括完整的 vim）编辑，游戏会实时获取更改。

指南 / 编辑器与工具

脚本命令

概览

每个机器脚本都有一个命令邮箱，这是一个小型的命名命令队列，你可以在脚本运行时手动发送。命令让你无需编辑脚本就能操控正在运行的脚本：脚本保留自己的逻辑，并决定如何处理每个请求。

这是玩家到脚本的通道。脚本之间的协调请使用 Signal Bus（信号总线）。

保存可复用命令

打开脚本的 Commands 标签页。添加命令时填写 Name（按钮标签）、Command id（代码检查的内容），并以简单的键/值字段添加参数，不要使用 JSON。

选择命令的 Availability：

- All machines of this type, 让当前及未来的同一精确类型机器都能使用该定义。
- All compatible family machines, 当机器注册表声明了兼容族时显示，并在该族内共享定义。
- Only this machine, 让实例专用的坐标、目标 id 或其他一次性数据只保留在本地。

之后可以 Send 一次，或将其 Save 为可复用的一键命令。现有本地命令有 Share 操作，可将它们移入精确类型或兼容族集合。共享编辑和删除会影响每台匹配的机器。

保存的定义可以复用，但执行始终是本地的：Send 只会在当前打开 Commands 标签页的机器上排入一条新消息。待处理队列和最近历史记录从不复制或广播，而是按机器保留。

命令定义在标签页中，而不在脚本内，也不属于脚本变体。切换或复制变体不会复制命令定义。每台机器的邮箱最多可容纳 40 个待处理命令；在脚本消耗或清除一些命令前，后续发送会被拒绝。

在脚本中读取命令

每个机器脚本中的 `self` 都提供四个方法：

- `self.next_command()` 消耗最早的命令并返回 `CommandResult`。`.status` 为 "ok" 时，命令位于 `.command`；当邮箱为空时为 "empty"，且 `.command` 为 `None`。
- `self.peek_command()` 读取下一个 `ScriptCommand` 但不消耗它；邮箱为空时返回 `None`。
- `self.command_count()` 报告正在等待的命令数量。

- `self.clear_commands()` 丢弃全部命令并返回 `CountResult`；`.count` 正好是被移除的数量。

被消耗的命令是 `CommandResult.command` 中的 `ScriptCommand`，包含

`.name`（字符串；每个名称代表什么由脚本决定）和 `.args`（字典；对可选参数使用 `.args.get(key, default)`）。

```
while True:
    next_result = self.next_command()
    if next_result.status == "ok":
        cmd = next_result.command
        if cmd.name == "goto":
            self.nav.set_target(cmd.args["x"], cmd.args["y"])
            self.nav.set_throttle(0.6)
        elif cmd.name == "stop":
            self.nav.brake()
```

命令是请求，不是远程控制。永远不调用 `next_command()`

的脚本会忽略自己的邮箱。如何处理命令是脚本设计的一部分；未知名称应忽略，或使用 `print()` 报告。

命令与信号总线的区别

- 命令邮箱：从编辑器中的你（玩家）发给一台机器脚本。最适合手动操控、测试模式或一次性指令。

- Signal

Bus (`get_component("comms")`)：在命名频道上从一个脚本发给另一个脚本。最适合在你离开键盘时协调自动化机器。

指南 / 编辑器与工具

脚本变体

变体是命名的脚本快照，无需手动复制代码即可在一支机器舰队中复用。

MAIN 与命名变体

每个脚本都从 Main 开始。Main 是这台机器自己的后备代码，不共享，也不能重命名或删除。

机器可以公开两个命名目录：

- Exact type，针对该精确机器型号或层级。
- Compatible family，只在公开相同脚本契约的型号之间共享。例如 Small、Medium 和 Large Drones 共享 Drone 族变体；Water Pump 和 Oil Pump 不会仅仅因为都能提取流体就共享。

Variants 标签页始终先列出精确类型变体，再列出兼容族变体。没有明确兼容族的机器只显示精确类型目录。在兼容机器上保存变体时，你要选择哪个目录拥有它。

编辑活动变体

Code 标签页编辑当前活动的变体。对 Main 的更改留在此脚本中；对命名变体的更改会更新其共享目录条目，而其他脚本会继续使用已加载的副本，直到你明确更新它们。较旧的已加载副本会标为 outdated；点击 Update 可加载最新目录版本，或点击 Keep mine 将该副本保存为单独的命名变体。

使用 Duplicate Variant 或 Save Selection as

Variant，可将当前代码发布为新的命名目录条目。可见目录中不允许重复名称，以保持变体选择明确无歧义。

使用并应用到全部

Use 会把目录变体加载进这台机器的运行代码，并使其在此处成为活动变体。

Apply to all 遵从拥有该变体的目录。精确类型变体针对该精确类型；族变体针对拥有相同脚本槽位、且明确兼容的每个层级。如果所选命名变体在此处于活动状态并且你编辑过它，Apply to all 会使用更新后的目录源代码。

应用源代码发生变化时，正在运行的目标会自动用新代码重启。手动暂停的目标保持暂停；因电力暂停的目标会等待电力然后恢复；停止的目标保持停止。已经拥有同一源代码的目标会保留当前求值器，不会重启。

Apply to all 是一次性复制，不是永久共享链接。之后对命名变体的编辑会更新其目录条目，但不会改变其他脚本已加载的副本。当你确实希望这些脚本加载改进版本时，再次使用 Apply to all。对 Main 的更改仍然只对拥有它的脚本私有。

目录变更

重命名或删除命名变体只会改变其所属目录。删除已保存的变体时，正在使用它的脚本保留当前工作代码和生命周期，然后脱离并转到 Main。正在运行的脚本保留其求值器，暂停的脚本保持暂停，停止的脚本保持停止。

文件顶部的 `"""docstring"""` 或第一行 `#` 注释，会成为列表中显示的变体描述。函数 `docstring` 则是分开的：它们出现在悬停和自动补全中，Docstrings 页面会列出受支持的格式。

共享真正的代码

兼容族共享依赖明确的机器元数据，而不是根据名称或外观猜测。如果不同的脚本契约需要共用辅助函数，请把这些辅助函数放进 Library 脚本并导入它们，不要强行把这些机器纳入同一个变体族。

指南 / 编辑器与工具

调试模式

调试模式为任何脚本添加断点、未捕获异常暂停、三种单步模式、条件断点、日志点和实时变量监视面板。默认关闭；点击编辑器标题栏中的  Debug 按钮即可启用。

调试模式提供什么

- 断点槽位，点击任意行即可让执行在该处暂停。
- 右侧的 Watch 面板，显示所有局部变量的当前值。
- 脚本暂停时可用的 Step Over、Step Into、Step Out 和 Continue 控件。
- Call Stack 区域，用于在函数帧之间跳转并检查每一帧的局部变量。
- Watch 面板中的持久断点列表，可在断点之间跳转并编辑其条件/日志消息。

设置断点

点击行号左侧的槽位。该行会出现红点；同一行也会出现在 Watch 面板的 Breakpoints 区域。再次点击槽位可移除断点，或点击 Watch 面板旁边的 .

断点在关闭再开启 Debug 时以及跨游戏会话时都会保留。如果脚本上存在休眠断点但脚本不在调试模式，Debug 按钮会显示一个小 .

条件断点与日志点

点击 Watch 面板中断点旁边的  按钮进行编辑。

- Condition，只读 Python 表达式。如果设置，只有条件为真值时断点才会暂停。可使用变量、属性、索引和算术；函数与方法调用会被阻止，因此调试表达式永远不能改变游戏。示例：`heat > 80`。
- Log message，带有只读 `{expr}` 占位符的模板。如果设置，断点会把消息打印到控制台并继续运行，而不是暂停。这就像无需编辑脚本便散布 `print()`。`scan hit: {site.id}` 会在每次命中时打印当前站点 id。

可以同时设置二者：条件 + 日志消息只会在条件匹配时记录日志。

断点颜色提示：红色 = 普通，黄色 = 条件，强调色 = 日志点。

执行暂停时

断点会在标记行运行前暂停。未捕获异常会在失败的行暂停，并显示错误消息。两种情况下该行都会以橙色高亮。Watch 面板显示：

- Paused at line N 或 Paused on exception at line N 标题，以及调试器控件。
- 所选调用栈帧的局部变量；字典、列表、元组和集合先显示紧凑预览，点击后展开。
- 可点击的 Call Stack 行，用于跳转到相应帧并检查该帧的局部变量。
- 带有跳转、编辑和移除控件的断点列表。

跨过 / 进入 / 跳出

三种单步模式遵循业界标准：

- Over ()，运行下一条语句。如果其中包含函数调用，调用会原子执行（不会进入调用内部）。最常用的单步方式。
- Into ()，进入下一次函数调用，在内部第一条语句处暂停。想调试辅助函数时使用。
- Out ()，运行到当前函数返回。当你进入了无趣的代码并想返回调用者时使用。
- Continue ()，恢复执行，直到下一个断点或完成。从未捕获异常暂停处继续，会让脚本正常失败并清除暂停状态。

世界仍在运行

断点和异常暂停只会暂停正在调试的脚本，其他一切继续运行：

- 时钟继续走动。
- 其他机器上的其他脚本继续运行。
- 大气、电力以及所有系统照常 tick。
- 被暂停脚本控制的机器会在暂停期间保持最后一次命令的状态。如果脚本报错、停止或完成，脚本拥有的控制项会重置；Rover、Pioneer 和 Drone 的移动会取消。

这意味着逐步检查代码所花的时间会推进游戏世界。如果你正在调试太阳追踪器，并花两分钟检查变量，太阳在这段时间里已经移动。对于有严格时序要求的脚本，要把这一点纳入考虑。

检查变量

脚本运行时，Watch 面板每个 tick 实时更新；脚本暂停时，它冻结为当前值。

- 数字、字符串和布尔值直接显示其值。
- 字典、列表、元组和集合显示大小及浅层预览；嵌套值在展开前会缩略。
- 小型对象可能显示简短字段预览；大型对象先显示其类型，点击后展开。
- 面板底部显示当前行号。

固定监视值是可选的

你不需要固定任何内容即可检查当前帧。脚本暂停时，Local variables 和 Call Stack 会自动更新。只有想让自定义值或计算结果跨单步持续可见时，才在 Watch 中使用 + Pin，例如 `co2 / 10`、`self.waste()` 或 `atmo.get_co2()`。

Watch 面板在脚本最近一次挂起点对变量采样。对于每个循环末尾有 `sleep(1)` 的脚本，这就是调用 `sleep()` 瞬间的值。对于没有刻意暂停的紧循环，这是 tick 的步数上限触发时的值。

隔离保证

调试模式是只读加暂停工具。在一个脚本上启用 Debug 不会影响另一个脚本的行为。断点、异常暂停、单步状态和监视检查都按脚本存储；对加热器脚本开启调试，对并行运行的制氧机脚本不会产生任何影响。

指南 / 编辑器与工具

控制室

Control Room（控制室）是一个顶层卡片页面：脚本驱动的面板可以绘制实时仪表板，并通过共享游戏 API 发出命令。打开 Control Room 页面，点击 + New Card，一个新的面板脚本就会绑定到画布。

面板脚本每个游戏 tick 运行一次。模式始终相同：

```
while True:
    panel.clear()
    # ... call panel.draw_X(...) and panel.widget(...) here ...
```

你的脚本控制布局、颜色、显示的数据和更新节奏。你在机器脚本中使用的相同读取 API 在此也可用：`get_component(...)`、`clock.get_elevation()`、`orders.list_orders()`。面板可以读取组件状态；只有通过 `power_control`、`shop`、`comms`、`inventory` 或 `atmosphere` 等共享权限机构，它们才能改变基地。

常见用途

面板可以读取脚本公开的任意组件。常见用途包括：

- 大气历史， O_2 / N_2 / 压力的三个进度条，下方带一条 50-tick 火花线。
- 舰队状态板，每辆载具一行，包括状态点、名称、当前活动和电池条。
- 太阳追踪器监视器，显示太阳高度、面板倾角和太阳能效率的三个仪表。
- 储罐液位，显示基地每个流体缓冲区的条形图，下方显示净流量文字。
- 活动订单简报，承包商简报的换行文本，加上铁锭图标和进度条。
- 存储库存，每个存储箱的图标、名称、填充百分比和数量。

数据来自现有 API；面板提供用于显示数据的画布和控件。

控件集合

十五个用于常见模式的命名控件：

- 布局：`card`、`divider`、`label`
- 指示器：`status_dot`、`toggle`、`pill`、`counter`
- 条形图：`progress_bar`、`vertical_bar`、`bar_chart`
- 曲线：`gauge`、`spark_line`
- 交互式（玩家点击）：`button`、`switch`、`slider`；每个 tick 读取它们，以响应玩家输入。

十个用于覆盖命名控件之外所有需求的原语：

- `draw_text`（带可选 `wrap`）、`draw_icon`
- `draw_rect/fill_rect`、`draw_circle/fill_circle`、`draw_line`
- `clear`、`width`、`height`

所有接受颜色参数的控件都支持主题令牌：`"accent"`、`"success"`、`"warning"`、`"error"`、`"text-bright"`、`"text-secondary"`、`"text-muted"`、`"text-value"`。卡片还可以用表面令牌

`bg-base`、`bg-surface`、`bg-panel`、`border`、`border-dim` 绘制。使用 `card(x, y, w, h, title)` 包装出带边框和标题的框架，用 `fill_rect(x, y, w, h, "bg-surface")` 填充区域；管理样式的一行由 `status_dot`（绿色）+ `draw_text`（名称）+ `button`（播放/管理）组成。每当脚本重绘卡片时，主题令牌都会从活动主题解析。

完整的方法列表以及每个控件的实时预览，请查看左侧边栏的 Panel 条目（API 参考）。

一个完整的工作示例

舰队状态面板：

```
rover = get_component("rover_1")
pioneer = get_component("pioneer_1")

while True:
    panel.clear()
    panel.label(12, 22, "FLEET", "caption")

    panel.status_dot(20, 50, 5, "running")
    panel.draw_text(38, 50, rover.name(), 13)
    panel.progress_bar(360, 44, 110, 10, rover.battery.level(), "success")

    panel.status_dot(20, 80, 5, "running")
    panel.draw_text(38, 80, pioneer.name(), 13)
    panel.progress_bar(360, 74, 110, 10, pioneer.battery.level(), "success")
```

不需要 `sleep()`，解释器会自动控制循环节奏。面板会在每个游戏 tick 重绘。

注意事项……

卡片只能通过共享权限机构发出命令。卡片没有针对机器的

`self`，因此机器自身的动作（`set_throttle`、`set_recipe`、`move_to`、`mine`）不能从卡片调用。但它仍可调用 `power_control`（断路器）、`shop`（买/卖）、`comms`（Signal Bus）、`inventory` 和 `atmosphere` 等共享权限机构。把卡片当作受信任的自动化，因为它们可以改变基地状态。

画布不会自动清除。务必在每次循环迭代的顶部调用 `panel.clear()`，否则旧的绘制内容会堆在新绘制内容下面，面板看起来会被涂抹。

坐标使用逻辑像素。一个 1×1 卡片为 500×200 ；两列卡片宽 1000 像素，两行卡片高 400 像素。CSS 会把视觉内容缩放到屏幕，因此当脚本必须适应卡片跨度时，应使用 `panel.width()/panel.height()`。

限制

- 每次保存最多 50 张卡片。超过后，+ New Card 会变灰。删除一张即可腾出空间。
- 板面密度：使用板面上方的选择器选择 2–6 列。未更改的默认值对普通工作区使用 2，对 4K 级工作区使用 4，对足够宽的超宽屏使用 6。手动选择会持久化。
- 四种卡片大小： 1×1 、 2×1 （宽）、 1×2 （高）和 2×2 （大），直接从卡片大小菜单选择。逻辑画布会随跨度缩放，因此要相对于 `width()/height()` 绘制，卡片会在任意尺寸下重新排布。
- 每张卡片一个脚本。需要分离关注点？创建多张卡片。

指南 / 编辑器与工具

外部编辑器

脚本会作为 `.py` 文件保存在磁盘上。你可以用任意文本编辑器、VS Code、Sublime、Notepad++ 或其他工具编辑它们。

游戏运行时，更改会双向实时同步：

- 在游戏内编辑，文件会在约半秒内更新到磁盘。
- 在外部编辑，游戏会在约 250ms 内获取保存内容，游戏内编辑器的缓冲区也会刷新（会尽力保留光标位置）。

文件存放位置

活动存档的脚本位于平台应用数据目录下名为 `<save_id>_scripts/` 的文件夹中：

- Windows：`%APPDATA%\io.codeterraform.game\<save_id>_scripts\`
- macOS：`~/Library/Application Support/io.codeterraform.game/<save_id>_scripts/`
- Linux：`$XDG_DATA_HOME/io.codeterraform.game/<save_id>_scripts/` 或
`~/.local/share/io.codeterraform.game/<save_id>_scripts/`

`<save_id>` 在第一次创建存档时自动生成（类似 `save_lk2j8x_a1b2c3`）。它与脚本文件夹旁的 `.json` 存档文件相匹配；如果不确定，可按修改时间排序应用数据文件夹，使用刚打开存档对应的最新 `save_..._scripts/` 文件夹。

每个机器或面板脚本使用游戏内编辑器中显示的相同文件名，例如 `o2gen_1.py`、`solar_1.py` 或 `rover_1.py`。库脚本位于 `lib/` 子文件夹中，按普通方式命名；例如名为 `sensors` 的库是 `lib/sensors.py`。`.py` 文件只包含代码；`codeterraform-scripts.json`

是一个小型同步清单，用于跟踪重命名后的安全文件名别名。曾使用旧生成名称（如 `oxygen_gen_1.py`）的文件，在迁移后仍作为安全别名被跟踪；之后请使用以 canonical id 命名的文件。

编辑器存根

脚本文件夹还包含自动生成的 `.pyi` 存根，其中有游戏 API、组件 id、签名、返回类型和文档文本；每次文件夹同步时，它们会刷新以匹配当前构建，并且可以安全删除（游戏会重新创建它们）。

`user_stubs.py` 属于你自己，永远不会被覆盖。它用于仅供外部编辑器使用的类型别名和类型声明；在玩家脚本中显式导入这些名称。可执行辅助函数应放入游戏内 Library。`pyrightconfig.json` 是编辑器导入配置，缺失时会创建；所有已存在的副本都视为玩家拥有，并会保持不变。若仍在使用旧配置，需手动添加新需要的设置。Settings → Editor → External Editor 中有 Freeze stubs 切换项。

Freeze stubs 会停止自动更新生成的存根，使你的手动编辑保持不变；Regenerate Stubs 按钮会强制从当前构建生成一份新副本。

编辑器支持

自动补全、类型检查和导入解析来自 Pyright，即生成存根所针对的类型检查器。

- VS Code，安装官方 Python 扩展。它捆绑了 Pylance（也就是 Pyright），因此会自动工作，无需额外设置。
- 其他编辑器（Neovim、Sublime、Emacs、Helix 等），添加 pyright language server（`pip install pyright` 或 `npm i -g pyright`），然后将编辑器的 LSP 指向它。
- PyCharm，使用自己的检查器，不会读取 `pyrightconfig.json`；右键点击 `lib/` 文件夹 → Mark Directory as → Sources Root，让库导入能够解析。

无论使用哪种编辑器，脚本都在游戏内运行。Pyright 会从生成的存根提供行内帮助。将 `<save_id>_scripts/` 文件夹作为工作区打开，使编辑器找到 `pyrightconfig.json`；正是该文件让 `import sensors` 无需 `lib.` 前缀即可解析到 `lib/sensors.py`。

控制台日志文件

Settings → Editor → External Editor 可以把可见的控制台输出镜像到 `<save_id>_scripts/logs/`。`all.log` 包含合并后的控制台流；像 `rover_1.log` 这样的脚本专用文件只包含该脚本的输出。日志在游戏运行期间追加并按大小轮换，因此嘈杂的 `print` 循环不会无限增长。需要新建外部记录时，可在设置中使用 Clear Logs。

编辑发生冲突时

如果外部编辑与游戏内的脏缓冲区发生冲突，游戏首先会把两份源代码都保存为不同的恢复变体。随后它会要求你在活动源代码中选择 Use External 或 Keep In-Game。任何一方都不会被静默丢弃：没有激活的版本仍可在 Variants 标签页中找到。如果无法创建两个恢复变体，编辑器会保持锁定且不会选择任何版本。需要时先释放一个变体槽位，然后再次保存外部文件重试。干净的游戏内缓冲区会正常接受外部更改。

使用场景

- 使用具有完整 IDE 功能的外部编辑器。
- 使用 git 对脚本进行版本控制。
- 与其他玩家共享脚本。
- 更舒适地编写更长、更复杂的程序。

键盘快捷键

这里列出了游戏使用的所有应用级键盘快捷键。在“[按键绑定](#)”（Keybinds）设置页中，点击任意可自定义的绑定即可修改。对于多行光标，按住 Alt（macOS 上为 Option）并在编辑器中拖动。

指南 / 自动化系统

合同

Earth 会发布合同，即他们需要在星球上完成的工程任务。你编写代码来解决任务，然后传回答案。

每份合同包含：

- 说明 Earth 需要什么的任务简报
- 位于 `self.contract` 中的输入数据
- 以 credits 计的奖励

完成合同：

- 前往 Contracts 页面，使用 View Contract 打开一份合同
- 阅读 Info 页签中的任务简报
- 打印 `self.contract` 以查看可用的输入字段
- 通过 transmitter 传回答案：

```
c = self.contract
print(c)

transmitter = get_component("transmitter")
transmitter.connect("earth")
transmitter.transmit(c.id, answer)
```

如果答案正确，你会立即获得报酬。

Commander 和 Shop 也可以访问：

```
me = get_component("me")
print(me.get_credits())

shop = get_component("shop")
shop.sell("soil_sample")
shop.buy("solar_generator")
```

指南 / 自动化系统

车辆接近与服务

地面车辆使用宽松的服务区域模型。你不需要停在某个精确坐标上，但进行实体操作时，车辆必须处于停放状态。

五个术语

- Near（接近）：车辆位于机器或前哨站的服务区域内。它仍可能正在驶过。
- Stopped（停止）：实时速度为零。电池耗尽可以让车辆停下，但不会取消其路线。
- Parked（停放）：车辆没有实时移动，也没有正向油门。self.nav.brake() 会立即将其停放。
- Docked（停靠）：车辆已停放在前哨站、机器或空间站所要求的服务区域内。
- Connected（连接）：已配置物品端点、管道或电力关系。连接不能证明车辆已停靠，也不会自行移动货物。

简短规则是：先接近，再停放，然后进行实体操作。

服务区域

机器和前哨站的服务区域包含约 2 m 的余量，因此车辆持续移动时不需要精确的浮点坐标。前哨站的完整占地范围就是其常规服务区域。场地机器使用自身的位置和服务区域。靠近前哨站绝不会使场地机器成为该前哨站的一部分。

服务前停车

```
x = 25
y = -10
self.nav.set_target(x, y)
self.nav.set_throttle(0.5)
while self.nav.get_distance_to(x, y) > 2:
    sleep(1)
self.nav.brake()
```

执行 brake() 后，货物转移、充电和硬件服务即可使用相应的服务区域。定时货物交接会让车辆保持停放，直到处理周期结束；在此周期内，移动命令会返回 busy。

fleet.vehicles() 会独立报告当前活动与停靠状态。status 可以是 "moving"、"charging"、"drilling" 或其他活动。is_docked 表示创建该快照时地面车辆是否停放在前哨站；docked_at 给出该前哨站 id。

无人机使用明确的空间站停靠机制。位于场地 Mining Drill 的无人机必须先完成路线，才能装载矿石。

指南 / 自动化系统

Earth 订单

Earth 通过承包商战役订单（contractor campaign orders）和独立的 Weekly Earth Orders（每周 Earth 订单）看板请求材料。两者都使用 Supply Docks（供应坞）和同一个 Order API。订单是供应线；合同是谜题。

承包商战役订单

Helios Orbital、Spire Research 和 Vestibule Logistics

各自会公开其编写队列中第一份尚未完成的订单。这些订单永不过期。orders.list_orders()

只返回当前的三个承包商槽位；完成的战役订单会进入永久的 Completed Orders（已完成订单）台账。Contractor Reputation（承包商声望）可以提高其 credits 付款，有些战役订单还会解锁科技或配方。

每周 Earth 订单

Weekly Orders 页签是一个专用的全宽看板，提供五个渐进式报价。它会等待你发现符合条件的采矿或生产链，因此 orders.list_weekly_orders() 最初可能为空。一旦可用，随着存档发现这些链条，会出现更高的生产层级；不可用的层级会回退到更早的材料。报价是一项机会，不保证每周都能完成：已知链条仍可能需要你尚未建造的机器或基础设施。看板从符合条件的链条出现当天开始计时，并在之后第七天开始时整体刷新。例如，在第 40 天创建的看板会在第 47 天开始时刷新。

每周订单只奖励 credits。它们合计的基础付款不超过 20,000 credits，Contractor Reputation

不会使其倍增；已履行的每周行会一直显示到刷新。它们不会进入永久战役台账。使用 orders.list_weekly_orders() 读取看板；每周对象的 kind == "weekly"，具有 expires_day，且没有 contractor。

刷新时，每份未完成的每周订单以及所有已运送的进度都会永久过期。分配给旧看板的 Supply Docks

会被取消分配并禁用。已经装载但尚未运出的货物仍留在其 dock

内。只有当至少一个单位确实已向未完成的每周订单运出时，游戏才会发布警告 toast；仅分配或装载 dock 而没有运出不会产生提示。该 toast 仍可在 Computer > Notifications 中查看，计划中的刷新不会产生导航徽章。

供应坞 workflow

- 选择订单。读取 orders.list_orders() 或 orders.list_weekly_orders()，然后调用 self.set_order(order.id)。
- 装载订单。遍历 self.current_order().requires，调用 self.input.take(item_id, count)。dock 会拒绝超额装载。

- 启用派送。调用 `self.set_enabled(True)`。多个 dock 可以服务同一订单，并共享已运送进度。
- 响应释放。完成或每周订单过期会清除 `current_order()` 并禁用派送，因此脚本必须重新选择并重新启用。

每个 dock 都会以 `self.dispatch_rate()` 单位/h 持续派送。`self.current_dispatch()` 报告当前正在离开的物品，`self.dispatch_progress()` 报告到下一个单位的进度比例。已装载的单位始终是实体货物。`clear_order()` 会释放分配，但保留 dock 中的这些单位；不同的 `set_order(...)` 会产生 `.status == "cargo_present"` 的 `ActionResult`，直到本地机器或车辆将它们排空。

奖励与身份

最终所需单位落地时，credits 和解锁会自动生效。配方奖励会在其指定机器处解锁相应配方。科技奖励会让指定升级包在 Shop 中可购买；它们不会把免费升级包放入 Inventory。战役对象的 `kind == "campaign"`，包含承包商身份且没有过期时间。每周对象的 `kind == "weekly"`、`contractor_id == None`、`contractor_name == None`，并带有看板过期日。使用 `orders.get_order(id)` 获取当前订单和永久战役完成记录；已过期的每周 id 返回 `None`。

指南 / 自动化系统

信号总线

概览

Signal Bus（信号总线）是供脚本和指挥官使用的显式共享状态。Libraries（库）共享代码；Signal Bus 在命名频道上共享实时值。如果只是手动操控一个正在运行的脚本，请改用它的 `command mailbox`（命令邮箱）（`self.next_command()`）。

Signal Bus 通过 Signal Bus 研究解锁。在此之前，`get_component("comms")` 返回 `None`。

手动操作员

打开 Ship Computer > Signal Bus，即可无需编写临时脚本发送通信。一个接收者应消费消息时选择 Queue；许多读取者都应看到当前值时选择 Broadcast。普通输入会作为文本发送；有效 JSON 会发送结构化对象、数组、字符串、数字、布尔值或 `null`。手动通信会显示你的 commander 名称作为发送者。

使用 Inspect 检查频道上的排队消息。可以删除一条消息，或清除当前显示的队列。清除操作只针对你检查过的确切消息 id，因此之后到达的脚本消息会被保留。删除队列不会移除频道的广播值。

排队消息

当每条消息都应只处理一次时，使用 `send(channel, value)` 和 `receive(channel)`。

```
comms = get_component("comms")
sent = comms.send("jobs.mine", {
    "sector": "E14",
    "resource": "iron_ore",
    "priority": 2,
})
if sent.status != "ok":
    print(sent.message)
```

`send()` 返回一个 `SendResult`，包含 `status`、`message` 和 `message_id`。当状态为 "ok" 时，`message_id` 是刚排队消息的正整数 id。对于 "invalid_channel"、"invalid_value"、"queue_full"、"channel_limit" 和 "id_exhausted"，它为 `None`，因为这些结果不会发送任何内容。


```
received = comms.receive("jobs.mine")
if received.status == "ok":
    msg = received.packet
    job = msg.value
    print("job from", msg.sender, job["sector"])
else:
    print(received.message)
```

receive() 返回 ReceiveResult。只有当 status == "ok" 时，packet 才是已消费的 CommsMessage；对于 "empty"、"not_found" 和 "invalid_channel"，它为 None。省略 id 或传入 None 会取得最早的消息。提供 id 则只取得那条消息。简而言之：receive() 会消费；latest() 不会。

等待工作

对于应在收到请求前保持空闲的 worker，使用 wait(channel)。如果有可用消息，它会立即取得最早的排队消息。否则只有此脚本会等待；游戏和其他脚本继续运行。它不需要 async、await 或检查循环。

```
comms = get_component("comms")
while True:
    received = comms.wait("jobs.mine")
    if received.status != "ok":
        print(received.message)
        break
    job = received.packet
    print("Received", job.id, job.value)
    # Perform the requested work here before waiting again.
```

wait() 返回带有 status、message 和 packet 的 ReceiveResult。有效调用会等待并在收到 "ok" 时返回已消费的 CommsMessage。格式错误的频道会立即返回 "invalid_channel"，并将 packet 设为 None。不存在、为空以及仅广播的频道会让脚本持续等待。没有超时。广播不会唤醒等待工作的 worker。

消息会一直排队，直到活动 worker 取得它。每条消息只能由一个 receiver 取得。如果另一个 receiver 或 Cancel 按钮先取得消息，等待中的 worker 会继续等待下一条。Worker 遵循正常的脚本执行顺序；等待不会预留消息，也不会赋予其优先于 receive() 的权利。

暂停脚本会保留其等待状态而不取得工作。停止或重启脚本会放弃该等待，但不会移除排队消息。保存/加载会保留排队消息并遵循正常脚本启动流程；它不会保存挂起的调用或局部变量。一旦消息被接收，worker 就负责完成工作并报告结果。

如果同一个脚本在没有工作可用时仍需继续检查传感器、命令或 Control Room 按钮，请使用 receive()。使用 wait() 的卡片也会暂停自身的绘制和输入处理。

保留发送回执

当脚本稍后可能需要识别或取消自己的请求时，应保留成功发送的 `message_id`。这与 `pending()` 和 `receive().packet` 显示的 `id` 相同，即使有效载荷相同，收到的 `id` 也仍然不同。

```
sent = comms.send("jobs.water", {"outpost": "A"})
if sent.status == "ok":
    request_id = sent.message_id
    print("My delivery request", request_id)
else:
    print(sent.message)
```

成功发送后，如果不再需要该请求，将保存的 `request_id` 传给 `comms.cancel("jobs.water", request_id)`。回执记录发送了哪条消息；它不保证消息仍在等待，也不保证工作已经完成。将 `id` 与新的 `pending()` 列表匹配，以检查它是否仍在排队。如果 `worker` 已经接收它，取消会返回 `"not_found"`。

检查待处理工作

使用 `pending(channel)` 检查等待中的消息而不消费它们。它返回 `CommsMessage` 条目列表，按从最早到最新排列，并具有与已接收消息相同的 `id`、`sender`、`tick` 和 `value` 字段。

```
comms = get_component("comms")
for msg in comms.pending("jobs.mine"):
    print(msg.id, msg.sender, msg.value)
```

可以在 `Control Room` 卡片中使用此列表显示积压，同时 `worker` 脚本继续接收工作。读取 `pending()` 不会取得或预留任何工作。列表、每条消息及嵌套值都是副本；修改它们不会改变队列。

快照只包含调用时正在等待的消息。它不包含广播，也不包含已经被 `receive()` 或 `wait()` 取得的工作，即使对应 `worker` 仍在忙碌。消息到达或被接收后，需再次读取以刷新。空队列、缺失频道或仅含广播的频道都会返回 `[]`。格式错误的频道 `id` 会引发 `ValueError`。

选择并取得工作

将消息 `id` 传给 `receive(channel, message_id)`，即可从 `pending()` 的结果中准确取得所选工作。`Worker` 可以按优先级、位置或能力进行选择；`Control Room` 卡片可以为特定工作提供 `Run` 按钮。

```
for candidate in comms.pending("jobs.mine"):
    if candidate.value["sector"] == "E14":
        received = comms.receive("jobs.mine", candidate.id)
        if received.status == "ok":
            job = received.packet
            print("Claimed", job.id, job.value)
            break
        elif received.status == "not_found":
            continue
        else:
            print(received.message)
            break
```

检查所选 id 并移除其消息会在一个操作中完成。如果两个 worker 选择同一条消息，只有一个能够接收。另一个会得到 "not_found"，其 packet 设为 None，即使队列中仍有其他工作。它绝不会代替所选消息取得另一项工作。只有状态为 "ok" 时才开始工作，并使用返回的 packet 作为工作数据。

该 id 也可以来自成功的发送回执。它必须是在受支持整数范围内的正整数。错误的频道，或已接收、已取消或已清除的 id，会产生 "not_found"；格式错误的频道会产生 "invalid_channel"。无效的数字 id 会引发文档规定的异常。不带 id 时，空队列会产生 "empty"。

其他排队消息、新到达的消息和广播都会保留。早先的 pending() 快照仍是副本；需再次读取以刷新。接收会立即从队列中移除工作。你的 worker 脚本仍负责完成工作并报告结果。

取消待处理工作

使用 cancel(channel, message_id) 移除一个过时的请求。传入成功发送的 message_id 或 pending() 中消息的 id，而不是它在列表中的位置。Control Room 卡片可以为每个等待中的工作提供单独的 Cancel 按钮。

```
for msg in comms.pending("jobs.mine"):
    if msg.value["sector"] == "E14":
        result = comms.cancel("jobs.mine", msg.id)
        if result.status == "ok":
            print("Canceled", msg.id)
        elif result.status == "not_found":
            print("This job is no longer waiting")
        else:
            print(result.message)
```

cancel() 返回 ActionResult。"ok" 表示那条确切的等待消息已被移除。"not_found" 表示该 id 不在此频道的队列中，包括另一脚本已经接收或取消它的情况。格式错误的频道返回 "invalid_channel"。消息 id 必须是在受支持整数范围内的正整数；格式错误的数字参数会引发文档规定的异常。

其他等待中的消息会保留其 id、有效载荷和顺序。新到达消息和最新广播也会保留。取消不会停止已经接收该工作的 worker。早先的 pending() 快照保持不变；需再次读取以更新面板。

等待多个频道

当一个 worker 处理多种请求时，使用 wait_any(channels)：

```
comms = get_component("comms")
result = comms.wait_any(["urgent_deliveries", "deliveries"])
if result.status == "ok":
    print(result.channel, result.packet.value)
```

列表中第一个拥有排队工作的频道会提供一条消息，在该频道内按最早顺序取得。脚本能够取得工作时（包括唤醒或从暂停恢复后）会检查优先级。如果所有列出的队列都为空，只有此脚本会等待，直到其中一个拥有工作。无关频道和广播不会提供消息。没有超时，也不需要预先创建缺失的频道。

结果是一个带有 status、message、channel 和 packet 的 WaitAnyResult。状态为 "ok" 时，channel 命名被选中的频道，packet 是已消费的 CommsMessage。如果任意频道 id 无效，会立即返回 "invalid_channel"，且 channel 与 packet 都为 None；不会从任何频道取得消息。

传入包含 1-128 个字符串条目的列表。其他容器类型和非字符串条目会引发 TypeError；空列表或超过 128 个条目会引发 ValueError。整个列表会在接收前完成验证。调用时会复制列表；重复名称只按其第一次出现的位置计算。

优先级决定下一个请求；它不会中断已经进行的工作。持续繁忙的较早频道可能让较后的频道继续等待。竞争中的 receiver 仍然至多取得每条消息一次。暂停、停止、保存/加载以及 Control Room 绘制遵循与 wait(channel) 相同的规则。

等待新的广播

使用 wait_broadcast(channel)，让多个脚本对同一个新信号作出反应：

```
comms = get_component("comms")
result = comms.wait_broadcast("factory.phase")
if result.status == "ok":
    print(result.broadcast.value)
    print(result.broadcast.sender)
```

当 worker 已处于等待状态时，另一个脚本或 Computer Signal Bus 操作员可以在 "factory.phase" 上广播 "start"。所有已经等待的脚本都会捕获该发布，并在下一次获得执行机会时恢复。

它不会消费共享广播或任何排队消息。send()、队列编辑、取消和清除频道都不能满足此等待。调用会等待开始之后发布的第一条成功广播。通过 latest() 已经可用的值不能满足新的等待；如果想立即获取当前值，请使用 latest_info()。再次发布相同的值仍然计数，包括 None。失败的发布和其他频道上的广播不会唤醒它。没有超时，等待不会创建频道，也不会使用其队列容量。

固定的 WaitBroadcastResult 具有 status、message 和 broadcast。状态为 "ok" 时，broadcast 是 BroadcastInfo，包含复制的 value、sender 以及脚本恢复时采样的 age_seconds。状态为 "invalid_channel" 时，broadcast 为 None。错误的参数类型或参数数量会引发 TypeError。

如果在脚本恢复前后来又到达广播（即使发生在同一个 tick 内），或频道被清除，则第一条捕获的发布仍会保留。每个脚本都会收到独立副本。暂停等待中的脚本会保留其第一条信号直到恢复；停止或重启会丢弃它。保存/加载会保留总线的最新值，但不会保留活动等待或其捕获的信号；重启后的等待会监听未来的发布。脚本进行其他工作期间发布的信号，不会为它下一次等待缓存。

编辑等待中的请求

使用 update(channel, message_id, value)，在请求仍处于排队状态时替换它。Control Room 工作看板可以让玩家调整数量、目的地或优先级，而不必取消并重新发送工作。

```
sent = comms.send("deliveries", {"item": "iron_ingot", "amount": 50})
if sent.status == "ok":
    updated = comms.update("deliveries", sent.message_id, {
        "item": "iron_ingot",
        "amount": 100,
    })
    print(updated.status, updated.message)
```

update() 返回带有 status 和 message 的 ActionResult。"ok" 表示完整的替换值已应用。"not_found" 表示该 id 不在指定频道中等待，包括已接收、取消或清除之后的情况。"invalid_channel" 拒绝格式错误的频道；"invalid_value" 拒绝违反以下有效载荷规则的替换值。拒绝会使总线保持不变，也不会重新创建缺失的消息。id 遵循与 cancel() 相同的正整数规则。

消息会保留其

id、队列位置、原始发送者和原始发送时间。其他消息、新到达消息及最新广播保持不变。更新不需要新的队列槽位或消息 id，因此即使队列已满也仍然可以编辑。成功的更新不是新的发送。

整个值会被替换；不会合并字典字段。早先的 pending()

快照保持不变。需再次读取以刷新面板。如果多个脚本更新同一条等待中的消息，最后一次成功更新获胜。

更新和接收各自都在一个操作中完成。Worker 接收时得到完整的原始值或完整的替换值。如果 worker 先接收，更新会返回 "not_found"，并且不能改变 worker 的副本或其工作。Worker 应使用 receive() 或 wait() 返回的 packet 作为实际工作数据，因为早先的快照可能早于编辑发生。

广播

当许多脚本应读取同一个当前值时，使用 broadcast(channel, value) 和 latest(channel)。广播不会被消费；在后续广播替换它或频道被清除之前，每个读取者都会看到当前值。

```
published = comms.broadcast("fleet.mode", "return_home")
if published.status != "ok":
    print(published.message)

mode = comms.latest("fleet.mode")
if mode == "return_home":
    self.nav.set_target(0, 0)
```

broadcast() 返回 ActionResult。latest() 是只读查询：它直接返回当前值；没有广播时返回 None。对于格式错误的频道 id，latest() 和 queue_size() 会引发 ValueError，而不是将其伪装成缺失数据。

检查广播新鲜度

当一个存储值只有在发布者持续报告时才有用，请使用 latest_info(channel)。它返回带有 value、sender 和 age_seconds 的 BroadcastInfo 快照。

```
comms = get_component("comms")
info = comms.latest_info("worker.status")
if info is None:
    print("No report yet")
elif info.age_seconds is None:
    print("Report age is unknown")
elif info.age_seconds > 30:
    print("Worker report is outdated")
else:
    print(info.sender, info.value)
```

年龄以 simulation seconds（模拟秒）计，与 sleep(seconds) 和 clock.elapsed_seconds() 使用相同的时间基准。模拟运行时它会增加，暂停时冻结，保存关闭期间经过的时间不计入。选择与你的 worker 广播频率相匹配的阈值。

没有广播时为 None，包括缺失频道或仅含排队消息的频道。广播 None 仍会返回 value 为 None 的快照。对于没有 sender 或有效时间戳信息的旧存档广播，相应的 sender 或 age_seconds 为 None；再次发布会提供新信息。格式错误的频道 id 会引发 ValueError。

值及其嵌套数据都是副本。年龄在调用 API 时捕获，因此需再次读取以刷新面板。排队消息不会改变广播的 sender 或 age，读取也不会消费任何内容。新广播会重置其 age，即使值没有变化。API 报告已过去的时间；由你的脚本决定何时认为报告过时。

监控与清除

```
for channel in comms.channels():
    print(channel, comms.queue_size(channel))

cleared = comms.clear("jobs.mine")
print(cleared.status, cleared.count, cleared.message)
```

clear() 返回 CountResult；count 是被移除的排队/最新条目数量，"invalid_channel" 与有效但无操作的 "no_op" 不同。与 Computer 的仅队列删除不同，脚本的 clear() 会同时移除队列和当前广播。只有当脚本有意拥有该频道时才使用它。

有效载荷规则

值必须是 JSON-safe（JSON 安全的）：None、布尔值、有限数字、字符串、列表/元组，以及键为字符串的字典。函数、组件、集合、类实例和包含非字符串键的字典会产生 "invalid_value"。有效载荷最多支持 8 层嵌套、1,024 个值，以及每个字符串或字典键最多 4,096 个字符。

频道 id 包含 1-64 个字母、数字、_、.、: 或 -。"__proto__" 等保留的对象字段名称会被拒绝。一个队列最多保存 64 条消息，一个存档最多保存 128 个频道。Computer 操作员使用相同限制。始终根据返回的 status 分支；message 是权威的原因文本。

指南 / 自动化系统

数据档案

概览

Data Archive（数据档案）是供脚本使用的持久共享知识。Libraries 共享代码，Signal Bus 共享实时协调，而 Data Archive 会跨越脚本停止、保存和重新加载，存储已学习的数据。

Data Archive 通过 Data Archive 研究解锁。在此之前，`get_component("notebook")` 返回 `None`。

存储与读取

```
notebook = get_component("notebook")
fuel_table = {"slow": 0.8, "cruise": 1.0, "fast": 1.7}
written = notebook.set("rover.fuel_table", fuel_table)
if written.status != "ok":
    print(written.message)

saved = notebook.get("rover.fuel_table", {})
print(saved["cruise"])
```

`set()` 是 gameplay command（游戏命令），返回 `ActionResult`。`get()`、`has()` 和 `keys()` 是只读查询，直接返回各自的自然值。

原子更新

`transaction(key, default, updater)` 会原子地读取、更新并写入一个键。它返回 `ActionResult`；`status == "busy"` 表示另一个 transaction 已经拥有该键。

```
def add_one(count):
    return count + 1

result = notebook.transaction("stats.samples", 0, add_one)
if result.status != "ok":
    print(result.message)
```

Updater 必须是小型纯函数：不能调用 `sleep()`、会产生让步的动作或修改世界的 API 调用。

管理条目

```
deleted = notebook.delete("rover.old_table")
if deleted.status == "not_found":
    print(deleted.message)

cleared = notebook.clear("scratch.")
print(cleared.status, cleared.count)
```

delete() 返回 ActionResult。clear() 返回 CountResult，其 count 是被移除条目的准确数量。

值规则

值必须是 JSON-safe：None、布尔值、有限数字、字符串、列表/元组，以及键为字符串的字典。每个存储值最多可包含 8 层嵌套、16,384 个总节点，以及每个字符串或字典键最多 4,096

个字符。根值、每个包含的值以及每个列表、元组或字典容器都各计为一个节点；字典键不计入。确切的结果状态为 "ok"、"invalid_key"、"entry_limit"、"invalid_value"、"not_found" 和 "busy"；每个适用命令都会返回精确的 message。512 条目限制和 transaction 争用在程序上是不同的。

按子系统命名键，例如 "rover.fuel_table" 或 "plants.recipe_scores"。实时协调使用 Signal Bus；希望保留的知识使用 Data Archive。

指南 / 自动化系统

地图标记

概览

地图标记（Map markers）是你留在 Planet Map

上的注释：一个字形、一种颜色、一个标签，以及固定在坐标上的简短备注。你可以在 Plan Mode 中手动放置，也可以让脚本通过 `get_component("markers")` 放置。两种路径都会写入相同的记录。

标记是备注，不是建造。放置它即时且免费：不需要材料、Pioneer、建造时间，也没有碰撞。标记可以自由重叠建筑、地点和其他标记。

标记通过 Cartography 研究解锁。在此之前，`get_component("markers")` 返回 `None`。

从脚本放置

```
markers = get_component("markers")

result = markers.place("survey.rover_1.empty:120:-40", 120, -40,
                      "No contact", "x", "neutral",
                      "Sonar sweep found nothing here.")

if result.status != "ok":
    print(result.message)
```

重复使用 id 会重写该标记，包括其位置，因此没有单独的移动或重命名调用。这很重要，因为保存并重新加载后，脚本会从第一行重新开始；只有创建的 API 会让地图充满重复标记。

按 id 前缀组织

这里没有标签。Id 携带结构，方式与 Data Archive 键和 Signal Bus 频道相同。

```
for marker in markers.list("build."):
    print(marker.label, marker.x, marker.y)
```

前缀中应包含控制机器，例如使用 `"survey.rover_1."` 而不是 `"survey."`。共享裸前缀的两个脚本会覆盖并清除彼此的标记。

保持一个标记族为最新

```
prefix = "survey.rover_1."
markers.clear(prefix)
for site in get_component("journal").surveyed_sites("nocturna"):
    markers.place(prefix + site.id, site.x, site.y, site.mineral, "resource")
```

先清除会让这个族反映脚本当前的判断。代价是中途出错会使地图只填充一半。不清除而直接调用 place() 可以防止这种情况，但不再真实的结论对应的标记会堆积。按脚本情况选择。

markers.clear("") 会删除星球上的每个标记，包括你手动放置的标记；不会记录谁放置了什么。前缀参数是必需的，因此必须明确写出完整清除操作。

对标记采取行动

标记保存坐标；它们不会建造任何东西。要将标记转化为实际工作，请将其位置传给 construction queue，并让 Pioneer 完成工作。

```
for marker in markers.list("build.outpost"):
    plan = get_component("construction_blueprint").plan_structure("outpost", marker.x, marker.y)
    if plan.status != "ok":
        print(marker.id, plan.message)
```

限制

一颗星球最多容纳 256 个标记。Id 长度为 1-64 个字符，只能使用字母、数字、_、.、: 或 -。标签最多 48 个字符，备注最多 240 个字符。坐标使用世界米并保留小数部分；星球范围之外的任何坐标都会被拒绝。

标记用于地图。任何结构化内容都应使用相同 id 存入 Data Archive。

指南 / 自动化系统

无人机

概览

无人机是空中物流与场地作业单位。从一开始，每架无人机都可以使用 `self.go_to(x, y)`；专用模块决定它抵达后能做什么，而不是决定是否存在坐标飞行。电动无人机使用 Battery Packs（电池包），以速度换取航程。直升机无人机使用 Oil Tanks（油箱），飞行速度快得多。Cargo Pods（货舱）增加单一材料存储空间。

蓝图解锁与部署

Basic Drone Operations（基础无人机操作）会开启该系统。硬件配方来自 Earth Orders。制造一个 Drone Depot Kit，并将其部署到前哨站。将每个制造好的机体保留在 Inventory 中，选择其物品卡，然后使用 Deploy 将它调试到空闲的 Depot 舱位中。无人机停靠后，通过该无人机的服务控制安装推进器、能源存储和角色模块。调试和空机架模块安装是服务订单；不要把机体或模块运送到 Depot 库存堆。

Drone Depots（无人机仓）会调试机体并处理本地货物。Drone Service Stations（无人机服务站）为电动无人机充电，从 oil_in 为直升机无人机加油，每小时将工作中的已停靠无人机去污 10 exposure，并救援停转或受干扰的无人机。

场地作业

Ground Rover/Pioneer Sonar（地面漫游车/Pioneer 声纳）会发现矿物地点、喷口、井和异质矿藏。Portable Bio Scanners and Extractors（便携式生物扫描仪和提取器）处理永久生物坐标。天气过后收集使用相同的坐标飞行和 `collect()` 接口；Weather DOCS 定义程序可获得的信息。

`collect()` 只会在无人机的精确坐标找到仍存在的灾后遗留物。错误、附近、已过期和已耗尽的目标都会返回 `nothing_here`，不会提供方向提示。Storm Glass 是普通货物。Raw Uranium 是高温货物：一次调用最多取得 5 个；没有 Shield Plating 会增加 40 exposure，有 Shield Plating 则增加 0；如果这一批货让无人机在 100 exposure 时受到干扰，它仍会留在货舱中。Plating 还会使 Cargo Pod 容量减半，并使燃料消耗提高到 1.5 倍。

读取机群

```
fleet = get_component("fleet")
for drone in fleet.drones():
    print(drone.id, drone.name, drone.status, drone.battery_level, drone.oil_level)
```

向空间站和救援 API 传入无人机时，使用稳定的 id。在收获或转移之前使用 `self.cargo.space_for(item_id)`，并在燃料、机油或 exposure 变成问题之前经过服务站。

交叉引用

- Weather、programmable signals 和 aftermath collection
- Biosphere / Biomass、Portable Bio Scanner 和 Extractor 工作
- Drone Depot 和 Drone Service Station component API
- Orders，提供无人机蓝图奖励的来源
- Flow Networks and Refinement，直升机无人机的机油供应

指南 / 生产与物流 (GUIDE / PRODUCTION & LOGISTICS)

电池与充电 (Batteries & Charging)

车辆使用电池。停放的车辆不消耗任何电量，只有执行动作时电池才会下降。

什么会消耗电池 (WHAT DRAINS BATTERY)

- 移动，这是主要消耗。Rover 在高速下的效率会大幅变差；Pioneer 更高效，但最高速度仍会增加每米所需的瓦时。
- 钻探，钻头运行时每秒消耗 10-30 W，具体取决于钻头变体 (Basic / Industrial / Heavy)。
- 声呐，每次扫描或勘测消耗少量固定电量（按声呐等级为 0.5-2 Wh）。
- 静止，免费，不消耗电量。

ROVER 的电池 (THE ROVER'S BATTERY)

- 集成式固定容量 100 Wh，无法升级。
- Rover 在全速下会很快耗尽电池。半速时效率高得多。长途行驶应降低油门。
- 满电 Rover 全油门约可持续 5 小时；半油门则接近 20 小时。

PIONEER

- 没有内置电池，需要安装电池模块来提供容量。
- 传动系统效率更高，高油门惩罚比 Rover 温和，但较慢的巡航速度仍能延长续航。
- 每个安装的模块都会增加移动时的功耗。

查询电池 (QUERYING THE BATTERY)

```
level = self.battery.level()      # 0-1
wh = self.battery.wh()            # current Wh stored
cap = self.battery.capacity()     # max Wh
```

长途驾驶前检查 `level`。

充电与救援 (CHARGING & RESCUE)

将车辆停在 Vehicle Charging Station 约 2 m

范围内、且位于其服务区域中的任意位置，即可直接充电。该站会按目标电量为停靠车辆排队：`self.charge(vehicle_id, 0.8)` 会为该车充电，直到达到 80%。`dispatch_rescue(vehicle, target_level)`

也可以派出一架缓慢的现场服务无人机，为野外的 Rover 或 Pioneer 充电。脚本决定阈值和目标：0.2 用于安全补充，0.8 用于长途返程前，1.0 用于远程充满。

`cancel_rescue()` 会召回该站当前的救援无人机；已经输送的电量会保留在车辆上，车辆会在无人机飞回时解除占用。

电池耗尽时，只要脚本仍在运行，当前路线就会暂停；脚本的意图不会被取消。如果车辆在脚本停止前重新充电，它会继续前进。按下 Stop、脚本运行到末尾或发生未捕获错误时，车辆会停止并清除路线。正在接受救援充电的车辆会保持停放，直到无人机完成、脱离或被 `cancel_rescue()` 召回。

提示 (TIPS)

- 根据电池规划行程。往返路程加上钻探耗电必须在当前电量范围内。
- 长途远征时降低油门；较低速度可以延长续航。
- 编写一个舰队管理器，在车辆电量耗尽前自动派出救援。

指南 / 生产与物流 (GUIDE / PRODUCTION & LOGISTICS)

精炼与存储 (Refinement & Storage)

开始采矿后，精炼流水线会把原矿转化为库存，并为制造阶段准备物料。

设置流水线 (SETTING UP A PIPELINE)

"inventory" 仅在基地是货运端点；远程前哨使用本地存储和车辆。

```
input_link = self.input.connect("Iron Ore Bin")
output_link = self.output.connect("Iron Ingot Bin")
recipe = self.set_recipe("smelt_iron_ingot")

if input_link.status == "ok" and output_link.status == "ok" and recipe.status == "ok":
    while True:
        ready = self.get_output_count()
        if ready > 0:
            sent = self.output.send("iron_ingot", ready)
            if sent.status not in ["ok", "partial", "no_op"]:
                print(sent.message)

        room = self.input.capacity() - self.input.count()
        if room > 0:
            taken = self.input.take("iron_ore", min(10, room))
            if taken.status not in ["ok", "partial", "no_op"]:
                print(taken.message)
```

`connect()` 和配方命令返回 `ActionResult`。`take()` 和 `send()` 返回 `TransferResult`，其中包含 `.requested` 和 `.moved`。处理是异步的，因此完成后的输出会在稍后出现。

`self.input.flush()` 返回 `TransferResult`；`.moved`

是被永久丢弃的单位数量。在处理机器上，它还会取消当前进度。请有意地调用它。

指南 / 生产与物流 (GUIDE / PRODUCTION & LOGISTICS)

输入与输出 (Input & Output)

机器处理物品，而 I/O 端口决定这些物品来自哪里以及将要送往哪里。

连接 (CONNECTIONS)

路由机器端口连接到同一前哨中兼容机器或存储设施的显示名称/ID。"inventory" 仅在基地是货运端点。

```
self.input.connect("Raw Materials")
self.output.connect("inventory")
self.input.take("iron_ore", 10)
self.output.send("iron_ingot", self.output.count())
```

连接会持续存在。转移需要 Auto

Feeder，并且耗时与移动的单位数成正比；转移具有事务性：被拒绝或阻塞的转移会使源中的物品保持完全不变。Fast Feeder 在获得 400,000 Terraform Points

后解锁，并将整个物流系统中新启动的定时物品转移时长减半。已经进行中的转移保持其启动时的时长。

Plant Terraformer 的接收 feeder 在 Mk I 每个处理步骤处理 16 个物品，在 Mk II 处理 80 个。无论输入是从源拉取，还是另一台机器或存储设施向它发送，容量都相同。两个物理端点都会在产生的转移时长内保持占用。

Field Mining Drill 和 Water Pump 改用仅取货的 PickupOutputSlot。它提供 count()、capacity() 和 stacks()，但没有连接或 send 方法。物理上在场的载体通过自身货运 API 发起转移。

带属性的物品 (PROPERTY-BEARING ITEMS)

规范值是一个 ItemStack：.id、整数 .count 和不透明的 .properties。属性属于物品，并会在 Inventory、箱子、仓库、端口和货舱之间保留。只有属性深度相等时，相同 ID 才会合并。

stacks() 返回按属性区分的快照：

```
for stack in self.output.stacks():
    print(stack.id, stack.count, stack.properties)
```

在 take 或 send 上使用可选的属性字典，可以选择包含该子集的堆叠：

```
self.output.send(fragment_id, 1, {"forged": True})
```

第四个参数控制匹配方式。"exact" 选择一个完整的属性身份，其中普通无属性物品也包括 **None**。"subset" 选择字典子集，"any" 忽略属性。省略时，字典使用子集匹配，**None** 使用任意变体匹配。

```
self.output.send(fragment_id, 1, stack.properties, "exact")
self.output.send(fragment_id, 1, None, "exact")
```

没有选择器时，转移具有确定性。大多数来源会按存储顺序取出匹配的堆叠。Warehouse 会按存储顺序保留属性变体组，但在选中的精确变体内部，会先排空最小物理堆叠，以便更早释放部分填充的重复槽位。不同的 Warehouse 属性变体仍会占用不同的物理槽位。Storage Bin 会锁定一个物品 ID，但可以容纳该 ID 的多个属性不同的堆叠。

端口 API (PORT API)

- 路由端口上的 `connect(name) / disconnect() / connected_to()`
- 输入端口上的 `take(item_id, count)`，可选属性和匹配参数
- 路由输出上的 `send(item_id, count)`，可选属性和匹配参数
- `count() / capacity()`
- `stacks()`

手动 UI 操作与端口自动化是分开的。在 Nocturna Base，手动 Biology 使用 Inventory。在其他前哨，Manual Stockroom 选择器会把每张卡绑定到一个本地 Warehouse。Habitat 复苏试剂使用自己的本地暂存区。端口自动化和其他机器一样遵循货运本地性。

车辆交接还要求两辆车都停止，并且彼此相距约 2 m 以内。Inventory 货运要求车辆位于基地；静态存储转移要求车辆位于目标服务区域内。如果目标拒绝转移，其结果会说明原因，并且不会有任何物品丢失。

指南 / 生产与物流 (GUIDE / PRODUCTION & LOGISTICS)

电池支架 (Battery Holder)

Battery Holder 是一种安装在模块化车辆 Universal 槽位中的模块，用于容纳 Portable Battery 物品。支架本身不储存能量；它只是一个架子。内部电池负责存储电量。

安装在任意支架中的每节电池都会加入车辆唯一的共享电池池。卸下电池会立即缩小电池池，并将该电池所占的存储电量比例返还给电池物品。重新安装时只会恢复该电池保留的电量，因此移动电池不会产生免费能源。脚本通过 `self.battery` 查询电池池：

```
level = self.battery.level()      # 0-1
wh = self.battery.wh()            # current Wh
cap = self.battery.capacity()     # max Wh = sum of every battery's rated Wh
```

查看每个支架的详细信息：

```
for h in self.battery.holders():
    print(h.size, h.wh, "/", h.capacity)
    for b in h.batteries:
        if b is not None:
            print(" ", b.id, b.wh(), "/", b.capacity())
```

变体 (VARIANTS)

- Small Battery Holder, 1 个槽位
- Medium Battery Holder, 2 个槽位
- Large Battery Holder, 3 个槽位

每个支架占用兼容模块化车辆的 1 个 Universal 槽位。可在基地或任意前哨安装/卸下。

指南 / 生产与物流 (GUIDE / PRODUCTION & LOGISTICS)

货物架 (Cargo Rack)

Cargo Rack 是一种安装在模块化车辆 Universal 槽位中的模块，用于容纳 Portable Storage Bin 物品。货物架本身不存储任何东西；它只是一个框架。内部箱子负责存储材料。

安装在任意货物架中的每个箱子都会加入车辆的货舱容量。每个箱子只能容纳一个材料或物品 ID：装入的第一个单位会锁定该 ID；后续单位必须匹配。该 ID 的不同属性变体在箱子中仍作为独立逻辑堆叠存在，因此转移物品不会抹除其属性。

```
print(self.cargo.count(), "/", self.cargo.capacity())

for r in self.cargo.racks():
    for bin in r.bins:
        if bin is not None:
            print(bin.id, bin.item_id, bin.count, "/", bin.capacity)
            for stack in bin.stacks:
                print(stack.id, stack.count, stack.properties)
```

当 `send()` 将箱子排空至零时，箱子会恢复为未分配状态，下次行程即可装入任意材料或物品。

变体 (VARIANTS)

- Small Cargo Rack, 1 个箱子槽位
- Medium Cargo Rack, 2 个箱子槽位
- Large Cargo Rack, 3 个箱子槽位

每个货物架占用兼容模块化车辆的 1 个 Universal 槽位。可在基地或任意前哨安装/卸下。

指南 / 生产与物流 (GUIDE / PRODUCTION & LOGISTICS)

便携式电池 (Portable Battery)

Portable Battery 是一种可充电电池，可安装到 Battery Holder 的内部槽位。商店购买的电池到手时电量为 100%。安装后，电池会将其中存储的 Wh 转移到车辆的共享电池池中。

所有已安装电池会作为一个电池池共同放电。单个电池的 `.wh()` 追踪的是按该电池额定容量缩放后的电池池充电百分比，而不是独立电量。卸下电池会将电池池中按比例分配的电量转回该电池物品，因此重新安装永远不会产生免费能源。

变体 (VARIANTS)

- Portable Battery, 额定容量 50 Wh
- Heavy Portable Battery, 额定容量 100 Wh

可在基地或任意前哨安装或卸下。

指南 / 生产与物流 (GUIDE / PRODUCTION & LOGISTICS)

便携式存储箱 (Portable Storage Bin)

Portable Storage Bin 是一种单材料容器，可安装到 Cargo Rack

的内部槽位。便携式箱子单独使用时不存储任何东西。安装后，它会将其额定容量加入车辆货舱，并一次接受一个材料或物品 ID。

调用 `self.install(...)` 时，标准箱使用物品 ID `"portable_bin"`，重型箱使用 `"heavy_portable_bin"`。

装入箱子的第一个单位会锁定该 ID；后续单位必须匹配。当 `send()` 将箱子排空至零时，箱子会解除分配，下次行程即可装入任意材料或物品。

变体 (VARIANTS)

- Portable Storage Bin, 容量 25 单位
- Heavy Portable Storage Bin, 容量 50 单位

可在基地或任意前哨安装/卸下。

指南 / 世界与基础设施 (GUIDE / WORLD & INFRASTRUCTURE)

生物圈，生物量等级 (Biosphere, Biomass Tier)

Biosphere 是第二条主要地球化改造轨道。它的第一等级是

Biomass：无人机组发现并采集本土生命，随后前哨机器将其转化为行星生物量。

解锁 (UNLOCK)

Basic Drone Operations 在 180k Terraform Index 解锁，早于

Biosphere。它提供无人机部署和控制能力，但每种机体、推进器、基站、电池和 Cargo Pod 仍需要通过 Earth Orders

获得蓝图并制造。Biosphere 在 210k 解锁，并加入 Portable Bio Scanner、Portable Bio Extractor、Essence Liquifier 和 Biomass Mixer。

永久生物地点 (PERMANENT BIOSITES)

Planet Map 包含 35 个永久生物地点：整个地图中每种生物群系共有 7 个，所有互不相连的同类生物群系斑块共享这些地点。它们最初与其他物理地点一样，显示为问号接触点并阻止建造。每个地点包含 1-3 种本土生命形态；每个生物群系的 6 种生命形态保证会出现在其 7 个地点中的某处。所有游戏中的位置和内容都相同。

生物野外工具只能安装在无人机上。你不需要用肉眼寻找问号：`get_component("nocturna").points_of_interest()` 会返回地图上所有 "?" 接触点（每个都有 `.x / .y /`

`.scanned`），因此扫描无人机可以直接飞往未扫描的点而不是四处游荡。使用 `go_to(x, y)` 飞到接触点，然后调用

`self.bio_scanner.scan()`。问号会变成生物标记，结果会保存在 Journal 中。接触点的 `.kind` 在扫描前保持为 "unknown"，因此要扫描每个未知点才能找到生物地点（矿物、喷口和水井共享同一张地图，生物扫描仪在这些地点会扫描为空）。采集无人机返回生物地点坐标，并调用 `self.bio_extractor.extract()`。

Small electric drone 使用 Thruster 槽安装 Electric Thruster，并在两个 Module 槽中的一个安装 Battery

Pack。这样会留下一个专业槽位：为这次行程安装 Scanner 或 Extractor，停靠时交换角色，或部署两架无人机。Medium 和 Large 机体之后会提供更多空间。

采集器带有一个 25t 的密封舱，一次只能容纳一种生命形态。每个 Cargo Pod 只能容纳一种材料：Small 100、Medium 250、Large 500 单位，因此无人机可携带的材料种类数等于其货舱数量。出发前使用 `self.cargo.space_for(item_id)`

检查特定材料的剩余空间。`get_component("journal").biomass_coords()`

是可安全跨重启使用的已发现路线列表；每个样本公开峰值 `.tons` 和当前 `.remaining_tons`。

枯竭与冷却 (DEPLETION AND COOLDOWN)

一次采集只会移除无人机能够携带的数量。剩余库存会留在地点中供下一次行程使用。只有当该处所有生命形态都枯竭后，冷却才会开始。普通、非普通和稀有地点的冷却时间逐步增加，然后在同一个永久坐标补充。使用 `journal =`

`get_component("journal")`，再通过 `journal.is_ready(x, y)` 和 `journal.next_ready_at(x, y)` 来安排轮换。

使用 `journal = get_component("journal")`，再通过 `journal.is_ready(x, y)` 和 `journal.next_ready_at(x, y)` 来安排轮换。

本地处理 (LOCAL PROCESSING)

Drone Depot 的库存属于一个前哨。无人机在那里停靠卸货，然后将样本送入同一前哨的 Essence Liquifier。样本必须原产于该前哨的生物群系。Liquifier 只会生产该生物群系的精华。

不同前哨之间不共享任何东西。使用无人机移动生命形态，并通过由端口、管道和储罐组成的物理液体网络移动精华。Biomass Mixer 消耗所需的各生物群系精华并产生生物量吨数。后续阶段需要更多不同的精华，因此进度会把本地站和 Liquifier 链扩展到更多生物群系，并将这些液体管线汇聚到 Mixer。

采集器舱室足以让基础采集无人机运行。Cargo Pod 可以减少行程，但不能取代本地前哨链。

Plants 和 Wildlife 是接下来的 Biosphere 支柱。

指南 / 世界与基础设施 (GUIDE / WORLD & INFRASTRUCTURE)

生物圈，植物 (Biosphere, Plants)

Plants 是一条主动农作链。田地由 Wildlife 和工业生产一次性 Forage 收成。Plant Terraformer 舰队处理这些物理库存，以可见批次创建永久 Plants km^2 。田地本身不会自动增加 km^2 。

1. 发现并制作种子 (DISCOVER AND MAKE SEEDS)

达到 500 t Biomass 后，制造并部署 Seed Maker。每次 `combine(...)` 试验会消耗三种不同生命形态各 1 t。成功的混合会在结果槽中放置一个物理物种种子，并将配方记录到 Flora journal。已知的混合可以随时重复，以获得更多种子。

2. 播种并维护田地 (PLANT AND MAINTAIN THE FIELD)

将种子发送到 home Inventory。在本地网格任意位置调用 `load_seed(seed_id)`，将一个物理种子放入 Harvester 的持有槽；移动到空单元格后调用 `plant(seed_id)`。播种在 0.5 小时后完成。每个物种都有可见需求，例如光照、阴影、水、盐、间距、群落、伴生植物或拮抗植物。任何需求缺失时，生长都会安全暂停。

“相邻”是指正交相邻：正上、正下、正左和正右。对角线永远不计入植物关系。关于邻近植物的每条规则都只读取这四个位置，因此群落物种需要一个 2×2 方块，这是所有四个单元格各自都有两个同类相邻位置的最小形状。三株横排会让两端永远停滞。

Spacer 植物要求四个相邻单元格都没有其他植物。机器（包括 Grow Lamp、Sprinkler、Dispenser 和 Crop Automator）可以放在它旁边。例如，只要该植物的其他需求满足，正左侧的 Sprinkler 不会阻止生长。允许对角线植物，因此 Spacer 植物可以组成棋盘格。

Companion 和 antagonist 读取相同的四个单元格。固定的 Grow Lamp、Sprinkler 和 Dispenser 也服务于正交相邻的四个单元格（正上、正下、正左和正右）；对角线不计入。

每种作物的 Forage 会在可见生长阶段中增加，并在准备好时冻结。Harvester 每次到访可以提供 24 小时的早期手动光照、浇水和盐护理。之后由 Grow Lamp、Sprinkler 和 Dispenser 维持作物条件，由 Crop Automator 进行收割、播种和施加处理。将鼠标悬停在田地中的任意这些机器上，即可查看其服务区域。

3. 建立高产多样性 (BUILD PRODUCTIVE DIVERSITY)

每一种受支持的独特物种都会使作物产量相乘。一个高产物种为 $\times 1$ ，五个为 $\times 5$ ，十个为 $\times 10$ ，全部十五个为 $\times 15$ 。更难的物种还具有更高的基础收获值。

4. 升级 Grow Lamp 和 Sprinkler (GROW LAMP AND SPRINKLER UPGRADES)

每个升级包会改进一个已部署的 provider。两种机器都保持相同的四格覆盖范围：正上、正下、正左和正右。等级会增加生长期间累积的 Forage；不会改变生长速度，也不会增加对角线覆盖。

等级	单独使用时的 Forage 倍率	每个 provider 的功率	Sprinkler 用水量
Mk I	×1	5 W	2 t/h
Mk II	×2	25 W	10 t/h
Mk III	×4	100 W	200 t/h
Mk IV	×8	500 W	1,000 t/h

启用期间会产生功耗。Sprinkler 只要获得供应就会消耗 Water，即使旁边没有作物也一样。作物必须需要该 provider 提供的服务：Grow Lamp 增强需要光照的作物，Sprinkler 增强需要水的作物。作物的其他所有需求也必须满足。Provider 必须通电、启用并获得供应。Dispenser 保持 Mk I，并提供盐但没有产量加成。

每种所需条件只计算覆盖范围内最强的 provider。两个 sprinkler 不会叠加。不同所需服务的加成会在 ×1 基线之上相加，合计 provider 倍率上限为 ×12。对于同时需要光和水的作物，Mk II lamp 加 Mk II sprinkler 得到 ×3：一个基线加上两个 +100% 加成。Mk IV lamp 加 Mk IV sprinkler 达到 ×12 上限。

5. 肥料、生长加速剂与产量放大器 (FERTILIZER, GROWTH ACCELERANT, AND YIELD AMPLIFIER)

Fertilizer Mk I、II 和 III 每次施用 8 小时，分别使基础 Forage 产量达到 ×2、×3 和 ×5。继续使用相同等级会延长持续时间；切换等级前要等待当前剂量到期。这些田地加成与 Plant Terraformer 配方使用的 10、30 和 50 potency 值分开计算。

Growth Accelerant 每次施用 8 小时，使生长速度达到 ×2；在作物需求满足时更快达到相同产量。Fertilizer 和 Growth Accelerant 的时间会在作物未成熟时消耗，包括因需求缺失而停滞时；作物成熟后停止消耗。Yield Amplifier 每次施用 24 小时，使整个田地中生长作物的基础 Forage 产量达到 ×3；其计时器持续运行。额外剂量会延长持续时间。

Provider、Fertilizer 和 Yield Amplifier 的加成会在 ×1 基线之上相加。例如，Mk II sprinkler (+100%)、Mk III Fertilizer (+400%) 和 Yield Amplifier (+200%) 得到 ×8，然后再乘以物种多样性。此例要求作物需要水且其他条件全部满足。加成只作用于其生效期间获得的 Forage。已经存入的 Forage 不变，成熟作物在收割并替换前不会继续获得更多。

6. 收割并重新播种 (HARVEST AND REPLANT)

准备好的作物会留在单元格中，直到明确收割。移动 Harvester 工作 0.5 小时。每个 Crop Automator 作业在 100% 前哨效率下工作 0.1 小时。提交作业会立即返回；一个物理执行器总共最多处理 50 个作业，包括活动作业和待处理作业，严格按 FIFO（先进先出）顺序执行，因此最旧的作业先运行。它不会同时运行两个作业，也不会自行跳过被阻塞的队首，并会在作业之间暂停约 1 秒以重置机械臂。脚本可以检查活动和待处理工作，然后使用 `move_job(job_id, position)` 实现另一种调度策略。目标不再匹配的作业在到达执行器时立即失败，不会消耗工作时间。移动 Harvester 会尽量把作物转移到 Base Inventory 能接收的数量，剩余部分仍存放在作物上。Crop Automator 会把能放入本地输出的数量全部移出，丢弃剩余数量，并清空单元格。输出完全装满会阻塞作业，但不会改变作物。多个 automator 仍可在各自服务区域内并行工作。

7. 运行 Plant Terraformer 舰队 (RUN THE PLANT TERRAFORMER FLEET)

达到 2,000 t Biomass 后，在运行中的前哨制造并部署一个或多个 Plant Terraformer。将每台 Terraformer 的标准输入连接到本地 Crop Automator 输出、存储设施或 home Inventory，然后从源调用 `take(...)` 或 `send`。通过 `water_in` 连接 Water，然后调用 `self.set_enabled(True)`。物品转移使用普通 feeder 冷却；Terraformer 在 Mk I 每步处理 16 个物品，在 Mk II 处理 80 个。每台机器提交所装载材料所支持的最大比例批次，工作 3 小时，然后交付 km^2 。Mk I 在 2,250,000 km^2 停止；Mk II 处理最后两个区段。

完整流程是：发现种子、装载并播种、维护条件、增加多样性、收割并清理、重新播种，然后把 Forage 转入 Terraformer。

指南 / 世界与基础设施 (GUIDE / WORLD & INFRASTRUCTURE)

Plant Terraformer

Plant Terraformer 是唯一能增加永久 Plants km^2 的机器。在运行中的前哨尽可能部署多台，并为它们提供足够输入。每台获得供应的机器都会将其等级速率乘以该前哨当前的实时效率。

输入与周期 (INPUT AND CYCLES)

在研究 Auto Feeder 后，将 `self.input` 连接到 home Inventory、本地 Storage Bin 或 Warehouse，或者连接到 Crop Automator 等本地机器的输出。使用 `self.input.take(item_id, count)`，或从源的输出发送。这与其他机器使用的定时物品转移模型相同：两个端点都会在转移时长内保持占用。输入一次只能连接一个来源，因此更换来源时脚本可以重新连接。

Terraformer 的目标 feeder 在 Mk I 每个处理步骤处理 16 个物品，在 Mk II 处理 80 个，无论来源是推送还是 Terraformer 拉取。Fast Feeder 在 400,000 Terraform Points 解锁，并将所有新启动的定时物品转移时长减半。因此，在研究之前，一次完整 Forage 装载在 Mk I 约需 18.8 秒、Mk II 约需 20.6 秒；研究之后，在源空闲时分别约需 9.4 秒和 10.3 秒。Crop Automator 直连 Terraformer 可避免额外的 Warehouse 环节；Warehouse 仍适合作为玩家控制的暂存。

现有支架可以容纳一个完整 Forage 批次所需的 Forage、该批次所需的 Salt，以及 Fertilizer Mk I、Mk II、Mk III 和 Growth Accelerant 各最多 10 个。没有独立的输入缓冲区。周期可以开始时，机器会提交由所有已装载材料共同支持的最大完整 Forage 批次。例如，10 Forage 配合足够 8 次装载的 Water，会装载 8 的批次。如果某个必需材料连 1 个 Forage 都无法支持，机器会等待。

周期开始时，所有选中的物品和匹配的 Water 会恰好消耗一次。批次随后在 100% 前哨效率下工作 3 小时，并一步交付其 km^2 。断电或 `set_enabled(False)` 会使批次原地暂停。周期完成后，下一个可负担的已装载批次会立即开始。

等级与舰队消耗 (TIERS AND FLEET COST)

在 100% 前哨效率下, Mk I 最多装载 1,200 Forage, 平均每小时处理 400 Forage, 启用时消耗 180 W。它处理前三种转化, 并在 2,250,000 km ² 报告 "needs_mk2"。Mk II 最多装载 6,600 Forage, 平均每小时处理 2,200 Forage, 消耗 900 W, 并加入 Fertilizer 和 Growth Accelerant 注入器。每台机器都需要单独制造 Mk II pack。

累计材料阶梯 (CUMULATIVE MATERIAL LADDER)

进度	每份 Forage 的 km ²	区段 Forage	新增支持
----	-----------------------------	-----------	------

0 至 500,000 km ²	20	25,000	无
500,000 至 1,250,000 km ²	5	150,000	0.05 t Water
1,250,000 至 2,250,000 km ²	5 / 3	600,000	0.002 Salt
2,250,000 至 3,500,000 km ²	1	1,250,000	0.004 Fertilizer potency, Mk II
3,500,000 至 5,000,000 km ²	1 / 3	4,500,000	0.0001 Growth Accelerant, Mk II

Fertilizer Mk I 提供 10 potency, Mk II 提供 30, Mk III 提供 50。每个周期会先消耗已存储的 potency。为弥补短缺, 机器会选择总 potency 过剩最少、然后物品数量最少的完整物品组合。未使用的 potency 会留在机器中供之后的批次使用, 并且在保存/加载后保留。一个 Mk III 物品在 27-potency 批次后会剩余 23 potency。卡片会把已存储 potency 计入总量; 物品计数只显示尚未打开的物品。完整 Mk II 批次需要 27 potency。Growth Accelerant 按完整物品计数和消耗; 完整 Mk II 批次需要 1 个。

脚本控制 (SCRIPT CONTROL)

```
self.input.connect("crop_automator_1")
self.water_in.connect("Water Reservoir")
self.set_enabled(True)

while True:
    result = self.input.take("forage", self.batch_requirements()["forage"])
    print(result.status, self.status(), self.batch_size())
    sleep(1)
```

Salt 按完整物品消耗, 每批次每 500 Forage 向上取整消耗 1 个。完整 Mk I 周期消耗 3 Salt; 完整 Mk II 周期消耗 14 Salt。未使用的处理值不会结转。

`batch_requirements()` 将 Fertilizer 报告为 `fertilizer_potency`, 将 Growth Accelerant 报告为物品数量。使用 `fertilizer_potency(item_id)` 检查 Fertilizer 等级。物品转移局限于 Terraformer 所在前哨。Water 使用专用的 `water_in` 关系; 对于远程来源, 还需要兼容且已完成的 Liquid Pipe 路线。

指南 / 世界与基础设施 (GUIDE / WORLD & INFRASTRUCTURE)

Wildlife, 概览 (Wildlife, Overview)

Wildlife 统计当前附着于已部署 Habitat 的已建立物种群落。断电的 Habitat 仍会计入其中容纳的种群，但会暂停增长。

端到端循环 (END-TO-END LOOP)

- 为一种生物编录全部 5 个 fragment，并通过其 Bio Order 解锁饲料配方。
- 使用 `self.set_revival_target(creature_id)` 选择它。这会在复苏前显示该生物的两节点树。
- 准备 Habitat 或 Journal 显示的精确物种饲料和按稀有度缩放的 Biology 试剂。生物加成永远不会改变这一次性试剂配方。
- 调用 `self.revive()`，在饲养期间保持每项活动需求健康，然后随着群落推进调节之后的气体和液体范围。
- 每个完成的繁殖周期通常增加 1 个个体。Brood 加成会逐步累积，直到在繁殖周期中保证额外后代。其他节点可以加速部分种群曲线、奖励不间断的支持、减少饲料需求、扩大流体范围、保留基础流体，或改善初始种群。
- 种群正向变化会自动产生共享 Insight。静态种群和经过的时间不会产生 Insight。使用 `unlock_bonus(node_id)` 进行真正的花费决策；代码之外没有购买控制。
- Wildlife 达到 600,000 后，制造 Habitat Mk II pack。Mk I 在 175,000 停止，这是 Thriving 的上限。Mk II 将上限提高到 350,000，使群落可以进入并完成 Abundant，而不会改变任何繁殖统计。

一个 Habitat 容纳一个物种；每个前哨可容纳多少 Habitat 没有 Wildlife 专属上限。Habitat 遵循普通前哨过度拥挤规则，因此可以集中部署，分散部署则可能提高吞吐量。物种拥有自己的群落进度，围栏可替换。撤下已建立群落的 Habitat 会保留种群、生命阶段、Brood 进度、Insight 历史和加成，同时暂停增长，并将该群落从活跃 Wildlife 和物种广度加成中移除。可以从一个空 Habitat 调用 `self.rehouse(creature_id)`，前提是其当前容量能够容纳整个群落；也可以使用同一调用直接从另一个 Habitat 转移。十五种完整的 Mk II 物种达到 5,250,000 Wildlife，十四种达到 4,900,000。全部 16 种完整 Mk II 物种共生成 112 Insight，而每个节点需要 80 Insight，因此剩余 32。饲料、气体和液体只会为实际产生的个体消耗；达到上限的群落会闲置。

指南 / 世界与基础设施 (GUIDE / WORLD & INFRASTRUCTURE)

Wildlife, 供应 (气体与液体) (Wildlife, Supply (Gases & Liquids))

群落攀升得越高，所需的气体 and 液体就越稀有。奇异流体来自“勘探 → 采集 → 精炼 → 计量”链。普通流体跳过 Refiner；非普通和稀有流体使用原料加 tar。

按稀有度划分的流体 (FLUIDS BY RARITY)

Habitat 需求中会出现四种气体和三种液体：

- Common (可直接抽取并使用)：swamp_gas、ammonia (气体)；brine (液体)。
- Uncommon (原料 → Refiner + tar)：sulfur_gas (气体)；cryofluid (液体)。
- Rare (原料 → Refiner + heavy tar)：chlorine (气体)；quicksilver (液体)。

非普通和稀有奇异物质都有原始沉积物形式 (raw_sulfur_gas、raw_cryofluid、raw_chlorine、raw_quicksilver) 和可供 Habitat 使用的精炼形式。大多数群落永远不需要稀有组合 chlorine + quicksilver，因此对这些沉积物的需求很低。

1. 勘探 (PROSPECT)

奇异沉积物遵循与热喷口相同的模型：

- 使用声呐发现和勘测 (basic = active?, wide = rate, deep = cycle timing)；
- 使用上限收集 (气体喷口) 或抽取 (液体泉)；
- 将输出通过管道送入气体或液体存储。

沉积物在活跃和休眠阶段之间交替。活跃阶段进行储备，然后在休眠期间从储罐向 Habitat 供给。Deep sonar 的计时可以预测休眠阶段，并先确定缓冲区大小。沉积物不会枯竭；多个沉积物可以帮助互相覆盖休眠间隙。

2. 精炼 (仅限非普通 / 稀有) (REFINE (UNCOMMON / RARE ONLY))

原始原料不能直接供 Habitat 使用。Refiner 将原始原料 + tar 按配方转化为精炼奇异物 (通过 Biolab orders 解锁)。使用 `self.set_recipe(...)` 选择奇异物；将原始原料接入 `gas_in` / `liquid_in`，将 tar 接入 `self.input`，并从 `gas_out` / `liquid_out` 输出精炼产品。

Tar, 精炼试剂 (Tar, the refining reagent)

精炼会随原始原料一起消耗 tar。Tar 是石油精炼副产品，否则大多会送入 Waste Processor。Tar 成本随稀有度增加：

- Common, 不需要 tar (直接抽取, 不使用 Refiner)。
- Uncommon, 需要中等 tar。
- Rare (Chlorine / Quicksilver), 需要大量 tar。

Tar 是承载石油的液体管网中的持续成本，而不是硬性门槛。如果短缺，就扩大石油生产，或放慢 Habitat 的摄入速度，直到 Refiner 赶上。

3. 计量 (METER)

精炼后的气体/液体流入 Gas/Liquid Tank。Habitat 脚本通过 `self.set_gas_intake(...)` / `self.set_liquid_intake(...)` 计量输入，以保持范围。参见 Husbandry 页面。

每个阶段何时重要 (WHEN EACH STEP MATTERS)

刚建立的群落只需要饲料。中期群落会增加基础气体/液体。Thriving 群落可能需要精炼后的稀有奇异物，这会推动你进行更远的勘探和更重的精炼。Exotic Husbandry 研究会解锁非普通奇异物的勘测/收集/抽取以及 Refiner。Deep Exotics 在 500,000 Wildlife 解锁，并开放最稀有的采集与精炼。早期的 legendary 群落可能会在转入稀有流体时暂停，直到更大的动物园达到该总量；这是可以恢复的，不会损失种群。阅读 Habitat 的下一阶段确切方法，并在准备稀有供应的同时培育其他物种。

指南 / 世界与基础设施 (GUIDE / WORLD & INFRASTRUCTURE)

Wildlife, 饲养 (Wildlife, Husbandry)

Husbandry 意味着保持每一种活跃生命支持输入随时可用。最差的活跃输入决定 `breeding_efficiency()`，它是生命支持倍率，而不是种群增长率。

已建立的种群永远不会衰减。

复苏设置检查 (REVIVAL SETUP CHECK)

首先调用 `set_revival_target(creature_id)`，并根据其 `ActionResult` 分支。它会检查生物、目录以及每个物种只能有一个群落的规则，然后保存目标而不消耗材料。在专用试剂输入中准备至少 2 个正确的饲料单位和所有按稀有度缩放的 Biology 试剂。然后调用无参数的 `revive()`，并根据 `.status` 分支 ("ok"、"occupied"、"no_target"、"species_exists"、"not_cataloged"、"wrong_feed"、"insufficient_feed" 或 "insufficient_reagents")。任何被拒绝的结果都不会消耗物品，并会保留已选目标。复苏开始后，每项活动输入都必须在 12 个世界小时的饲养期内保持健康。

搬迁已建立的群落 (RELOCATE AN ESTABLISHED COLONY)

群落属于其物种，而不是围栏。撤下已建立群落的 Habitat 后，该群落会等待住房，但其种群、生命阶段、Brood 进度、Insight 历史和加成保持完整。重新安置前，群落对 Wildlife 和物种广度的贡献会暂停。在空 Habitat 中调用 `result = self.rehouse(creature_id)`，并根据返回的 `ActionResult` 分支。同一调用也可以直接从另一个已部署 Habitat 转移群落。目标当前容量必须能够容纳整个群落；"insufficient_capacity" 表示不移动任何东西。

饲料自动处理 (FEED IS AUTOMATIC)

当输入箱持有该生物的精确饲料时，`feed_ok()` 为 true。复苏会消耗一个饲料，并留下一个用于饲养。已建立群落只会在实际出生个体时消耗饲料；达到上限的群落不会消耗饲料。`feed_level()` 报告部分消耗后的可用饲料，因此脚本可以从本地存储补货，而不必猜测。

气体和液体是双向范围 (GAS AND LIQUID ARE TWO-SIDED BANDS)

`gas_band()` 和 `liquid_band()` 在输入启用后返回以吨为单位的 `[low, high]`；在输入开放前返回 `[]`。过少和过多都会降低效率。流体身份也必须匹配：将 `gas_fluid()` 与 `required_gas()` 比较，液体使用对应方法。减少摄入量可以让过满的围栏慢慢降回范围。对于大量过剩，使用 `purge_reserve("gas")` 或 `purge_reserve("liquid")` 明确清空储备，然后正确补充。被丢弃的流体会损失，但群落及其其他供应保持完整。

完整调节模式（Complete Regulator Pattern）

```
def regulate_fluid(band, level, set_intake):
    if len(band) == 0:
        set_intake(0)
    elif level < band[0]:
        set_intake(5)
    elif level > band[1]:
        set_intake(0)
    else:
        set_intake(1)

while True:
    regulate_fluid(self.gas_band(), self.gas_level(), self.set_gas_intake)
    regulate_fluid(self.liquid_band(), self.liquid_level(), self.set_liquid_intake)

    if not self.feed_ok():
        print("Feed missing or wrong; need " + self.required_feed())
    if self.gas_band() and self.gas_fluid() != self.required_gas():
        print("Wrong gas, need " + self.required_gas())
    if self.liquid_band() and self.liquid_fluid() != self.required_liquid():
        print("Wrong liquid, need " + self.required_liquid())

    print("efficiency", self.breeding_efficiency())
    sleep(1)
```

从本地存储连接并补给 Habitat 的饲料输入和专用试剂输入。将气体端口和液体端口连接到装有完全匹配所需异质资源的储罐。流体摄入设定器会立即改变策略，但缓冲量和效率只有在模拟推进且物料到达后才会改变。

在跃迁发生前读取跃迁信息（Read the Transition Before It Happens）

`next_stage_population()` 给出精确阈值，`next_requirement()` 命名变化类型。对于实际供应计划，`next_required_gas()` 和 `next_required_liquid()` 会命名跃迁后立即需要的流体。`next_gas_band()` 和 `next_liquid_band()` 返回未来精确的 [low, high] 窗口。空字符串或空列表表示该输入不会启用。这样就能在跨过阈值前完成勘探、精炼、铺管和调节器调谐。

恢复（Recovery）

一次失败的培育尝试会消耗已准备的材料，但绝不会损坏 Habitat。修正供应或脚本，然后重试 `revive()`。

对于围栏内已经存在的多余气体，停止摄入并明确丢弃该储备：

```
self.set_gas_intake(0)
result = self.purge_reserve("gas")
print(result.status, result.message)
```

然后重新填充所需气体并恢复调节。对于液体，使用 `set_liquid_intake(0)` 和 `purge_reserve("liquid")`。每次清除只会销毁所选围栏储备并清除其流体身份；另一种储备、饲料、殖群进度、入口缓冲区、连接和摄入设置都会保留。`purge_intake("gas_in")` 或 `purge_intake("liquid_in")` 仍是清除装有错误流体的入口的独立工具。

指南 / 世界与基础设施 (Guide / World & Infrastructure)

Habitat 的发展 (Habitat Development)

每个已建立的殖群都会从成功的世代中学习。种群的正向变化会直接增加一个共享的 Wildlife

Insight (野生生物洞察) 余额；经过的时间和静态种群不会增加任何值。`self.get_insight()` 会报告共享余额、此 Habitat 当前的速率以及它的终生贡献。

每种生物两个永久节点 (Two Permanent Nodes per Creature)

每种生物恰好有两个由作者定义的节点。较早的

Adaptation (适应) 只影响该物种，并按照其生命支持或繁殖节奏提供强力的持续减负。较晚的 Breakthrough (突破) 影响所有当前及未来物种。完整集合有意保持有限：它能让得到良好支持的后期殖群的未处理产出提高数倍，而不是提高一个数量级。加成包括生长曲线不同部分的速度提升、保证随时间额外产仔的繁育加成、连续支持满一天的奖励、饲料效率、更宽的流体区间、更大的创始种群，以及更高的后期速率上限。培育始终需要 12 个世界小时。脚本通过 `self.get_bonus_tree()` 检查 `.scope` 和 `.source_species`，并使用 `self.unlock_bonus(node_id)` 购买；代码之外没有购买控制项。

成本与门槛 (Costs and Gates)

仅限物种的 Adaptation 只有一个要求：1 点共享 Insight。调用 `set_revival_target(...)`

后、复苏前，它就会变为可购买状态。面向所有物种的 Breakthrough 恰好有两个要求：4 点 Insight，以及其来源生物达到 10,000 的种群数量。它不要求第一个节点、研究、行星范围的 Wildlife

总量或购买材料。每项检查都是原子的；被拒绝的购买不会消耗任何东西。

为什么更多 Habitat 很重要 (Why More Habitats Matter)

累积 Insight 曲线的单个个体产出会递减。一种生物在种群达到 10,000 时产生 3 点 Insight，而它的两个节点总共需要 5

点，因此没有一种生物能在突破门槛时自给自足地完成自己的整棵树。玩家必须汇集多个物种的成长。所有 16

种生物都达到完整 Mk II 种群时，会产生 112 点 Insight，而拥有全部 32 个节点需要 80 点，剩余 32 点。这相当于终生预算的 1.4 倍；即使只有 15 个完整 Mk II 物种，也会提供 105 点 Insight。完整或停滞的殖群不能永远铸造货币。

指南 / 世界与基础设施 (Guide / World & Infrastructure)

Wildlife 与进度 (Wildlife, Progression)

Wildlife 有三个相互关联的进度尺度：每个殖群的生命阶段、共享 Insight，以及全行星的已建立种群。

殖群阶段 (Colony Stages)

当前阶段	共享种群范围	Habitat 访问权限
已建立 (Founded)	4-249	Mk I 和 Mk II
首次繁殖 (First Breeding)	250-2,499	Mk I 和 Mk II
自给自足 (Self-Sustaining)	2,500-24,999	Mk I 和 Mk II
繁荣 (Thriving)	25,000-175,000	Mk I 和 Mk II
丰盛 (Abundant)	175,001-350,000	仅 Mk II

两种 Habitat 等级都显示同一套五阶段阶梯。Mk I 在繁荣阶段未达到 175,000 的容量。完成 Deep Exotics（深层异质资源）后，Wildlife 达到 600,000 时解锁 Mk II，并将容量提高到 350,000，使殖群能够进入丰盛阶段。升级只改变容量。成熟期功耗在 Mk I 为 40-64 W，在 Mk II 为 144-168 W。

繁殖 (Breeding)

每种生物都会完成繁殖周期。种群提供自然动量：种群为 5,000 时，Common（普通）殖群自然产生约 28 个个体/小时，随后种群贡献会平滑趋向每小时 150 个体。生命支持效率和稀有度会改变该自然速率：Common 为 1.0x、Uncommon（罕见）为 0.75x、Rare（稀有）为 0.50x、Legendary（传奇）为 0.30x。购买的加成会改善这条曲线的选定部分，而硬上限会使完整技能树的产出仍限制在未处理产出的数倍以内。一些繁育加成会在周期之间保存小数进度，并最终保证额外产生一个个体。完整支持的动量需要 24 个世界小时达到最大值，而支持不完善时进度会以两倍速度损失。容量从不会减慢繁殖；只有 Habitat 已满时才会停止增长。`breeding_efficiency()` 只是生命支持乘数，不是种群速率。读取 `breeding_rate()` 获取预期的每小时个体数。

适应与 Insight (Adaptations and Insight)

每个物种都有两个相互独立的永久节点。需要 1 点 Insight 的 Adaptation 只影响该物种。需要 4 点 Insight 的 Breakthrough 影响所有物种，并且还要求其来源殖群达到 10,000 的种群数量。两个节点都不需要研究、Compound（化合物）、行星范围 Wildlife 或另一个节点。Insight 根据种群增量沿递减里程碑累积：达到 10 时为 1.00，100 时为 1.50，1,000 时为 2.25，10,000 时为 3.00，50,000 时为 4.00，175,000 时为 5.50，350,000 时为 7.00。

只有已建立的殖群会计入行星范围阶段。失败的培育不会触碰传感器，而已建立的种群不会减少。

指南 / 世界与基础设施 (Guide / World & Infrastructure)

Wildlife 与生物档案 (Wildlife, Creature Profiles)

每种生物都有饲料配方，并且随着成长需要气体（对于更稀有的生物还需要液体）。每个物种只能存在一个存活殖群。当目的地 Habitat 有足够容量时，殖群可以安置在任意前哨，并且之后移动而不会丢失进度。

Forage, 热量基础 (Forage, the Calorie Base)

所有饲料都以 Forage（觅食物）为基础：收获成熟作物并清空田地格时收集的实体产出。因此，持续供应饲料依赖于持续生产种子、播种、培育、收获和重新种植。配方使用跨生物群系组合；16 种生物合计使用全部 30 种生命形态，所以每个生物群系都值得收获。生物的生物群系标签不会影响放置或要求。

如何读取生物需求 (How to Read a Creature's Needs)

- 饲料：完成生物实验室的订单以解锁饲料配方，然后在 Feed Maker（饲料制造机）中制造；其配方会列出精确的生命形态原料。
- 当前气体 / 液体：`self.required_gas()` / `self.required_liquid()` 命名精确的异质资源，`self.gas_band()` / `self.liquid_band()` 给出目标窗口。
- 下一个气体 / 液体：`self.next_required_gas()` / `self.next_required_liquid()` 以及 `self.next_gas_band()` / `self.next_liquid_band()` 会在殖群跨过下一个阈值前，揭示跃迁后的精确供应。空字符串或空列表表示该输入不会启用。

稀有度与要求 (Rarity and Requirements)

生物能成长到多高由其稀有度决定（从 Biology Lab（生物实验室）目录读取）：

- Common，以饲料为主；一种普通气体；没有液体。
- Uncommon，一种普通气体；在繁荣阶段会开启一种液体。
- Rare，其气体会升级为精炼气体，液体的窗口会收紧。
- Legendary，达到最稀有的异质资源，并使用最严格的窗口。

完整的逐阶段表位于 Wildlife, Progression。两种传奇生物最难保持在窗口内；让其中一种达到丰盛阶段可完成 Apex Husbandry（顶级畜育）成就，而复苏、建立并安置全部 16 种生物可完成 Nocturna Reborn（诺克图娜重生）。

指南 / 世界与基础设施 (Guide / World & Infrastructure)

天气系统 (Weather System)

天气是通过 Weather Station (气象站) 和无人机运行的定时资源系统。活动中的风暴会显示在 Weather (天气) 和 Planet Map (行星地图) 上, 但必须从实时传输中恢复后效坐标。

风暴生命周期 (Storm Lifecycle)

沙尘暴和雷暴会横跨行星移动, 并且可能重叠。Heat (热量) 和 Pressure (压力) 会增加强度、奖励产量和闪电能量; 它们不会让风暴更频繁地发生。

风暴开始时, 其传输会变得可听见, 并在结束后的 4 个世界时钟小时内持续可用。其后效只会在风暴结束时出现:

- 沙尘暴会留下 14-28 单位 Raw Uranium (原始铀), 持续 48 个世界时钟小时。
- 符合条件的雷暴会留下 2-4 单位 Storm Glass (风暴玻璃), 持续 96 个世界时钟小时。没有后效的雷暴不会传输 Storm Glass 消息。

奖励在无人机抵达精确坐标前一直隐藏。靠近但不精确的位置或已过期的位置会返回 `nothing_here`, 也不会给出方向提示。

建立接收覆盖 (Build Reception Coverage)

Weather Program (天气计划) 会在 Nocturna Base 安装一座已完成的 Weather Station。每座通电的气象站消耗 50 W，监测本地 300 m 半径，接收广播流量以及自身生物群系的沙尘频道。额外气象站部署在各前哨。

沙尘移动记录分布在全部五个生物群系频道中，因此完整的沙尘消息需要每个生物群系都有通电接收。雷暴移动记录采用广播方式，所以一座通电气象站就能听到完整的 Storm Glass 消息。

恢复坐标 (Recover the Coordinate)

`signal_receiver.transmissions()` 只返回此刻该气象站可听到的记录。它不保存历史；多个事件可能同时存在，而且损坏的副本可能声明与有效记录相同的数据包编号。玩家代码必须在接收关闭前保留所需内容。

具有相同 `event_id` 的记录属于同一个事件。每条记录都声明其 `number`、预期的 `total`、编码后的 `data` 和 `checksum`。数据格式为 `event_id|number|total|dx|dy`。有效校验和等于 `data` 中每个字符 ASCII 代码的完整总和，不进行模运算缩减。

一组完整且编号有效的记录描述从 $(0, 0)$ 开始的移动。按编号顺序应用其 `dx` 和 `dy`

记录，终点就是精确的后效坐标。`SignalTransmission` 的 DOCS

条目定义了每个字段。玩家代码负责决定如何保留记录、在气象站之间共享记录以及跟踪未完成事件。

收集后效（Collect the Aftermath）

风暴结束后，将携带货舱的无人机发送到恢复出的坐标，并调用 `collect()`。一次成功收集最多转移 5 个单位，因此较大的后效可能需要重复收集或多次飞行。Raw Uranium 会增加暴露，除非无人机装有 Shield Plating（护盾装甲）；Storm Glass 不具有放射性。已发现的存活后效随后会在地图上显示，直到耗尽或过期。

每个界面的作用（What Each Surface Does）

- `Incoming` 会自动总结预计进入通电气象站覆盖范围的风暴。它不需要 `observe()`，也绝不会揭示后效。
- `observe()` 测量本地活动风暴和预报。`last_report()` 读取该气象站最新的报告。
- `signal_receiver` 提供恢复后效所用的原始记录。
- `signal_board`
是可选的逐气象站显示器。它只呈现玩家代码发布的记录和带标签的值；不会验证、合并、存储或解码这些记录。
- `strikes()` 提供有限的本地闪电历史。Lightning Rod（避雷针）捕获机制独立于 Weather Station 脚本运行。

所需流程是接收、保留、验证、组装、恢复坐标和收集。存储、协调、呈现及派遣程序仍由你设计。

指南 / 世界与基础设施 (Guide / World & Infrastructure)

流体网络 (Flow Networks, Fluids)

完整心智模型 (The Complete Mental Model)

气体和液体使用相同的运输模型。Gas Pipe (气体管道) 承载已注册气体；Liquid Pipe (液体管道) 承载已注册液体。始终按顺序回答三个问题：

- 两个位置能互相到达吗？完整的物理拓扑回答这一点。
- 哪些机器打算交换哪一种精确物质？`FluidPort.connect(...)` 关系回答这一点。
- 现在能移动多少？供应、需求、电力、节流、缓冲空间和共享吞吐量共同回答这一点。

一个有用的简写是：兼容连接 + 可达拓扑 + 实时供应 + 实时需求 = 流动。连接和拓扑确立身份。实时条件只改变当前 t/h 速率，因此零流量不会使管道变成中性状态。

`connect()` 的含义 (What `connect()` Means)

玩家把机器端口连接到机器，而不是把端口连接到管道 ID：

```
# Water Pump script
self.water_out.connect("bio_caster_1")

# Equivalent when written in the Bio Caster script
self.water_in.connect("water_pump_1")
```

任一端都可以声明提供者-消费者关系，只需一端声明即可。目标可以是稳定的机器 ID 或显示名称。目标必须暴露兼容的反向端口：`water_out` 匹配 `water_in` 或通用的 `liquid_in`；`steam_in` 匹配 `steam_out` 或通用的 `gas_out`。

每个端口保存一个已声明的目标。再次对该端口调用 `connect()` 会替换之前的目标。`result.status == "ok"` 表示目标存在且有兼容端口；它不保证远程管道已完成、源有供应或流量非零。

`disconnect()` 只清除该端口拥有的声明，并保留已缓冲的流体。`connected_to()` 只报告该端口声明的目标；它不会列出对等端所作的反向声明。如果两端独立地为同一对机器调用 `connect()`，运输会去重，但必须断开两个声明才能完全移除关系。

不存在 `connect_pipe`、`disconnect_pipe` 或 `connected_pipe` API。管道 ID 用于建造和诊断。

本地与远程 (Local Versus Remote)

同一已完成前哨中的机器在调用兼容的 `connect()` 后直接传输。它们不需要本地管道片段，但仍然需要机器连接。

位于不同位置的机器至少需要一个已完成、无冲突且介质正确的物理组件，并且该组件要到达两个位置锚点。Pump 或 Thermal Cap 等地图机器使用自身可见的公用设施占地。内部机器使用其已完成的前哨占地作为锚点。

远程 `connect()` 调用可能在管道存在之前产生 `.status == "ok"`。这会记录持久意图。合适的拓扑完成后才开始流动。切断管道会使路径不可达，但不会擦除连接；恢复拓扑后流动会恢复。

占地暴露可用性，而不是存储 (Footprints Expose Availability, Not Storage)

接触前哨服务占地的管道，会将该组件暴露给那里机器拥有的兼容连接。人员分配不会改变公用设施的可达性。前哨不存储、不混合，也不会自动选择源或储罐。每台本地机器仍需要显式的 `connect()` 关系。

接触同一前哨服务占地的独立物理组件仍保持分离；占地接触是一种可用性关系，绝不是流体接点。这使水、石油和火山精华可以通过独立的液体组件到达同一个前哨，而不会变成一个池子。地图读数列出多种液体，表示这些已识别组件到达该占地；不表示液体已经混合。

一个远程关系最多使用一个物理组件。已有的有效且不冲突的分配会保留。新关系会使用已经承载该精确流体的组件，或一个中性组件。如果恰好一个组件为不同流体关系同时服务，两个连接器声明会在该处冲突。独立组件不会自动合并容量。

管道如何获得精确内容 (How a Pipe Gets Exact Contents)

管道硬件只确立气体或液体介质。一个已完成的物理组件只有在完整的兼容提供者-消费者关系被分配到该组件后，才会获得精确物质。

```
No connection: neutral.  
Provider present but unconnected: neutral.  
Consumer present but unconnected: neutral.  
  
One complete water connection: contents are water .  
Several water connections: compatible, they share supply and throughput.
```

```
Water and oil relationships simultaneously have only one eligible component: both claims
conflict and all flow on that component stops.
Separately designated Water and Oil components are physically joined: conflict, all flow
on the
joined component stops.
A water connection with throttle at zero, no power, no supply, or a full sink: contents
remain
water , live flow is zero.

Remove the final complete connection: the component becomes neutral again.
```

同样的规则适用于气体。只要连接仍存在，休眠的 Steam（蒸汽）源仍会指定其组件。如果新的 Ammonia（氨）关系只有该 Steam 组件可用，重叠的连接声明就会冲突。如果被分别指定为 Steam 和 Ammonia 的组件在物理上连接，连接后的组件也会冲突，即使两个源都处于空闲状态。

如果多个组件到达同一对锚点，每个关系只选择其中一个。第二种精确流体可以指定另一个中性组件，而不会影响第一个。如果恰好只有一个组件同时符合两种精确流体的条件，它们的同步连接声明会在该组件上冲突。独立分配的流体如果之后在物理上连接其管道几何，也会冲突。

物理连接（Physical Joins）

- 只有已完成的管道和桥梁硬件会运输流体。
- 气体和液体硬件永远不会互相连接。
- 同介质地面片段只要真实几何相接，就会连接。
- T 形以及普通地面交叉点是真实接点。
- 接触一个服务占地的组件彼此保持物理独立，除非它们自己的地面几何相接。
- 在地面几何中有意建造真实的普通接点，会把路线连接起来。
- 桥梁沿桥轴连接其匹配的坡道。
- 桥梁中间瓦片下方的垂直地面线仍保持分离。

`Pipe.start()` 和 `Pipe.end()` 只报告建造几何；它们不会强制实时流动方向。端口方向、供应和需求决定实时移动。

储罐是显式存储（Tanks Are Explicit Storage）

空 Gas Tank（气体储罐）接受任意气体；空 Liquid Tank（液体储罐）接受任意液体。将精确源连接到空储罐，会在交付前建立路线身份。第一种实际存入的精确流体会锁定储罐。此后储罐只接受并提供该流体，直到完全排空为零并解除锁定。

储罐不会由前哨自动插入。源到储罐再到消费者的链路需要两个关系：

```
# Water Pump script, fill storage
self.water_out.connect("Water Reserve")
self.set_throttle(1.0)
```

```
# Consumer script, draw from storage
self.water_in.connect("Water Reserve")
```

储罐本身是被动的，不需要控制循环。将它排空到零不会擦除其端口声明。如果之后锁定为另一种液体或气体，旧的不兼容声明无法流动，应重新连接。

两个只彼此连接的空通用储罐仍保持中性，因为任一端都没有识别出精确物质。

一个提供者、多个消费者（One Provider, Several Consumers）

每个消费者都可以将自己的输入连接到同一个提供者：

```
# Bio Caster script
self.water_in.connect("water_pump_1")
```

```
# Sprinkler script
self.water_in.connect("water_pump_1")
```

```
# Reactor script
self.water_in.connect("water_pump_1")
```

三个关系都会参与。可用供应、目的地余量和物理吞吐量会共享，不按列表顺序给予优先级。当某个消费者必须有明确优先级时，使用独立储罐、拓扑、连接选择或节流器。

管道吞吐量上限（Pipe Throughput Ceiling）

单个管道片段本身没有吞吐量容量。每个物理组件从每个匹配的源侧和汇侧连接链接各自拥有 2,000 t/h 的共享吞吐量。一个拥有一个源链接和一个汇链接的常见布局，因此起始为 2,000 t/h。High-Pressure Fluid Transport（高压流体运输）研究会将每个已完成及未来的 Gas Pipe 和 Liquid Pipe 连接链接改造为 6,000 t/h。相同路线中的管道长度和额外片段不会降低或提高该上限。

组件上分配的每个兼容关系都会共享此预算。两侧有多个连接链接可以提高总容量，但单纯增加长度或内部分支不会增加任何容量。一个逻辑连接永远不会合并独立组件。机器产量、节流、供应、消费者需求和缓冲余量仍可能设定更低的实时速率；存储突发流量或多个生产者仍然可以达到该上限。

读取 API (Reading the APIs)

`FluidPort.level()` 是该端口缓冲的吨数。一些直通式输出（包括 Pump）不存储任何内容，因此读数为 0。`capacity()` 是端口的缓冲上限。`flow_rate()` 是当前实时速率，单位为 t/h；0 可能表示空闲、缺料、已满、不可达、冲突，或者只是在等待模拟时序边界。

```
print(self.water_in.connected_to())
print(self.water_in.level(), self.water_in.capacity())
print(self.water_in.flow_rate())
```

管道诊断显示物理侧：

```
for pipe in list_pipes():
    print(pipe.id, pipe.type(), pipe.contents(), pipe.state())
    print("rate", pipe.flow_rate())
    print("connections", pipe.connections())
    print("conflicts", pipe.conflicting_contents())
```

`type()` 报告 "gas" 或 "liquid"。`contents()` 报告已建立的精确物质；中性或冲突时为 `None`。因为这两种情况都返回 `None`，还要检查 `state()` 和 `conflicting_contents()`。`connections()` 列出使用该组件的完整逻辑路线；每行诊断中的代表性管道 ID 不是连接句柄。

故障排查：零流量 (Troubleshooting Zero Flow)

按以下顺序检查：

- 预期端口的 `connected_to()` 是否显示了自己的声明？
- `connect()` 的结果是否为 `.status == "ok"`，而不是 `"not_found"` 或 `"incompatible"`？
- 对于远程机器，所有预期的管道或桥梁片段是否都已完成？
- 正确介质的组件是否到达两个锚点？
- `Pipe.state()` 是否报告 `"conflict"`？
- 源是否通电、有供应、处于活动状态且节流高于零？
- 目的地是否有余量或当前需求？
- 共享吞吐量是否已经被其他路线消耗？

存储流体会在电力之前重新分配，因此发电机可以在该电力 tick 使用储罐交付。通电的 Pump 和 Cap 会在电网检查之后释放。后续处理可能在同一 tick 使用该释放量，但发电机会在下一个电力 tick 使用新泵送或封存的流体。因此出现一个 tick 的延迟可能是正常的。

无效的机器位置 (Invalid Machine Locations)

位置缺失、陈旧或未完成的机器会关闭运行。它不会借用所属前哨的连接性。前哨拆除绝不能留下这种机器；此规则可以防止损坏状态获得隐藏的公用设施访问。

交叉引用 (Cross-References)

- **Infrastructure & Pipes** (基础设施与管道) : 建造、物理连接和桥梁。
- **FluidPort** : 精确的 API 返回值。
- **Pipe** : 物理诊断。
- **Thermal Vents** (热能喷口) : 蒸汽源行为。
- **Wells** (水井) : 水和石油源行为。

指南 / 世界与基础设施 (Guide / World & Infrastructure)

基础设施与管道 (Infrastructure & Pipes)

概览 (Overview)

基础设施是建造在 Planet Map 上的实体硬件。Gas Pipe 承载气体并消耗 `gas_pipe_segment` 物品。Liquid Pipe 承载液体并消耗 `liquid_pipe_segment` 物品。硬件只确立介质；完整的、由玩家编写的提供者-消费者连接才确立精确内容。

建造管道 (Building a Pipe)

Plan Mode (规划模式) 和 `construction_blueprint.plan_pipe(...)` 会创建基于瓦片的 Construction (建造) 作业。携带 Constructor (建造器) 的 Pioneer (先锋) 执行每个作业：

```
bpq = get_component("construction_blueprint")
bpq.plan_pipe("liquid", 0, 0, 80, 0)

for job in bpq.pending_constructions():
    if job.kind == "pipe":
        self.nav.set_target(job.position.x, job.position.y)
        self.nav.set_throttle(1)
        while self.nav.get_distance_to(job.position.x, job.position.y) > 2:
            pass
        self.nav.brake()
        build = self.constructor.execute(job.id)
        print(build.status, build.message)
```

未建造的片段是幽灵，只会运输任何东西。

物理拓扑 (Physical Topology)

相接的几何会连接。同介质片段只要其与瓦片对齐的路径相接，就会连接。普通 T 形和十字形是实际接点。

服务占地接触时仍保持独立。两个流体组件不会因为都接触同一个前哨就合并。前哨占地会将每个组件独立暴露给已连接的本地机器。这允许水、石油和其他路线接触同一个前哨，而不会变成一个物理网络。

连接定义内容。只有提供者或只有消费者存在时都保持中性。每个完整兼容的远程 `connect()` 关系最多选择一个到达两个位置的物理组件，并在那里建立其精确流体，即使组件空闲、已满、断电或节流为零。

冲突会停止流动。如果恰好一个组件同时服务于水和石油关系，两个连接器声明会冲突，该组件停止。即使是分别指定的组件，如果其物理几何连接起来，也会产生同样的冲突。不同气体同理。同种流体的连接可以共享一个组件及其容量。

机器不会连接到管道 ID。脚本连接提供者和消费者。本地机器直接传输；每个远程关系都会保留一个有效、无冲突且已完成的物理组件，该组件到达两个位置，或者先选择同种流体组件，再选择中性组件。独立组件不会自动合并容量。

桥梁跨越而不连接。气体、液体和电力桥都是 3 瓦片高架通道。它们沿自身轴线连接；中间瓦片下方的垂直线仍属于不同网络。

吞吐量 (Throughput)

吞吐量属于组件，而不是片段。单个管道片段没有容量统计。一个拥有一个源链接和一个汇链接的常见布局起始为 2,000 t/h，由分配给该组件的所有兼容关系共享。High-Pressure Fluid Transport 研究会将所有已完成及未来 Gas Pipe 和 Liquid Pipe 的连接链接改造为 6,000 t/h。相同路线中的管道长度和额外片段不会降低或提高上限。两侧有多个连接链接可以提高总容量，而机器速率、节流、供应、需求和余量可能产生更低的实时速率。

检查管道 (Inspecting Pipes)

```
for pipe in list_pipes():
    print(pipe.id, pipe.type(), pipe.contents(), pipe.state())
    print("connections", pipe.connections())
    print("conflicts", pipe.conflicting_contents())
```

这些管道 ID 是检查和建造身份，不是玩家流体端口句柄。在 Planet Map 上，流动网络使用精确的流体颜色，停滞网络为琥珀色，冲突网络为红色。

材料与恢复 (Materials and Recovery)

管道片段成本取决于路线长度，大约每 10 米一个片段。如果建造暂停，查询 `get_component("construction_blueprint").paused_constructions()`，并在范围内再次执行同一个作业。拆分或移除路线会立即重新计算物理可达性，同时保留玩家编写的机器连接意图。

交叉引用 (Cross-References)

- Flow Networks：运输、吞吐量和背压行为。
- Power Networks：电线拓扑和电力桥。

- `Constructor Module`, `self.constructor.execute(...)` : 执行建造模块。
- `Construction Blueprint` : 脚本侧的规划。
- `Thermal Vents and Wells` : 物理生产者。

指南 / 世界与基础设施 (Guide / World & Infrastructure)

电力网络 (Power Networks)

概览 (Overview)

已完成的电线会将行星划分为互相连接的子网。同一个子网中的每个发电机、消费者和电池共享该子网的发电量和储存能量。子网可以依靠发电、储存能量或两者运行；只要电池仍有能量，就不要求本地存在或当前活动的发电机。

物理拓扑 (Physical Topology)

相接的电线会连接。电线片段只要其与瓦片对齐的几何相接就会加入同一网络。T 形接点和普通交叉会形成一个子网。

前哨会暴露电力服务占地。每个接触前哨占地的已完成电力组件都会加入该结构的总线，因此加入同一个子网。属于该前哨的每台有效已完成机器会自动使用同一子网。人员分配不会影响公用设施连接性。

Power Bridge (电力桥) 跨越而不连接。Power Bridge 沿自身轴线连接。穿过中间瓦片下方的垂直地面电线仍是独立子网。

电线没有方向、损耗或每条线路的瓦特上限。发电、存储和需求会在相连子网中汇总。方向由哪些机器发电、哪些机器耗电而产生。

未完成的工作不传输电力。蓝图幽灵和部分建成的片段只属于可见的规划状态。位置缺失或陈旧的前哨机器无效，也不会获得备用电力。

分配与存储 (Allocation and Storage)

发电会首先服务子网完整的实时需求。通电的常规电池形成一个按容量加权的池：它们从盈余充电，并在 Lightning Rod (避雷针) 储备之前放电。Lightning Rod 不会从普通电网盈余充电，也不计入主电池 Stored total (存储总量)。

不存在消费者优先级或部分负载切除。如果发电量加常规电池和 Lightning 储备无法支付当前 tick 的完整实时负载，该子网的所有实时消费者都会在工作运行前暂停。取消的工作不会消耗部分存储。如果可用能量恰好覆盖该 tick，该 tick 会完成；下一个资金不足的 tick 才会暂停电网。显示的 W 和 Wh 值可能经过四舍五入，但内部的小额充放电增量仍会持续累积。

建造与恢复（Building and Recovery）

在 Plan Mode 中规划电线和 Power Bridge，或者使用 `construction_blueprint.plan_power_line()` 和 `construction_blueprint.plan_bridge()`，然后让配备 Constructor 的 Pioneer 驾驶到每个作业的交互范围内，并调用 `self.constructor.execute(job.id)`。如果路线因拆除而断开，剩余片段会立即成为独立子网。停电而暂停的机器会在实时发电量覆盖所有恢复负载时一起恢复，或者在常规电池加 Lightning 储备能够维持剩余缺口一小时内恢复。新存入的 Lightning 能量可以直接恢复黑暗子网。手动关闭的机器不会加入自动恢复。

诊断子网（Diagnosing a Subnet）

使用 Planet Map 跟踪高亮的连接片段，并使用 Power dashboard（电力仪表板）比较发电、需求和电池存储。Lightning Rod 卡片及其类别总量显示独立的深层储备。如果远程前哨断电，请确认所有预期电线片段和桥梁都已完成，地面几何接触两个服务占地，并且子网有足够的当前发电量或存储能量来承担其完整实时负载。

交叉引用（Cross-References）

- Infrastructure & Pipes：建造和桥梁跨越规则。
- Batteries：存储行为。
- Constructor Module：执行 Planet Map 作业。

指南 / 世界与基础设施 (Guide / World & Infrastructure)

Thermal Vents (热能喷口)

概览 (Overview)

热能喷口是行星地壳中的裂缝，内部热量会以蒸汽形式从中泄漏。通过 sonar（声呐）发现喷口，并在其上放置 Thermal Cap（热能封盖）蓝图来收集蒸汽。每个喷口会在活动和休眠阶段之间循环。

发现 (Discovery)

`self.sonar.scan()` 返回 `SonarScanResult`。当 `.status == "ok"` 时，遍历 `.sites`，选择 `kind() == "thermal"` 的接触点；`.message` 会解释被拒绝的扫描。使用 `self.sonar.survey(site)` 调查选中的接触点，并且只有在其 `status` 为 `"ok"` 后才读取 `SurveyResult.site` 载荷。Basic Sonar（基础声呐）揭示阶段，Wide Sonar（广域声呐）增加蒸汽速率，Deep Sonar（深层声呐）增加周期时序。

收集蒸汽 (Harvesting Steam)

在喷口上规划一个 Thermal Cap，将其套件装入 Pioneer，并使用 Constructor 执行排队的蓝图。已完成的封盖会在通电时将蒸汽捕获到自己的腔室中。

捕获后的输送 (Routing After Capture)

只有当封盖脚本调用 `self.steam_out.connect(...)`，连接到兼容消费者的稳定机器 ID 或显示名称，并使用 `self.set_throttle(value)` 打开它（`value` 为 0-1）后，蒸汽才会离开。目标在本地时直接传输。目标在远程时，会自动使用封盖与目标位置之间已完成的气体拓扑；脚本从不选择单独的管道。完整连接会在流动开始前确立蒸汽身份，并在空闲、已满、关闭或节流为零时保持该身份。

释放不足时，腔室液位会上升。达到 100% 时会过压并排出整个腔室，然后从空状态重新填充。读取 `pressure()`、`is_overpressured()` 和 `is_stalled()` 来控制释放、添加存储或恢复物理可达性。

交叉引用 (Cross-References)

- Flow Networks：捕获后发生的事情。
- Infrastructure & Pipes：物理输送。
- Construction Blueprint：排队的建造工作。

满负荷时合计为 13 t/h 的水。

单口标准水井（10 t/h）无法同时供给两者。你有以下选择：

- 接入更丰饶的水井（丰饶 / 纯净产量为 20 / 30 t/h），或增加一口带独立泵的水井。
- 使用液体罐进行缓冲，让短时需求峰值从储存中取水。
- 通过脚本分配，在当前时刻优先满足更重要的 Mk III，让另一个设备暂时降级到 Mk II。
- 在 1,000,000 Plants 后解锁 Steam Condensation 时，把储存的排放蒸汽导入 Steam Condenser，在满功率下再获得 250 t/h 的水。

分配脚本

水分配器会持续运行，并根据 Terraforming（地形改造）进度决定哪个 Mk III 获得新鲜水：

```
o2_tank = get_component("liquid_tank_1")
oxy = get_component("o2gen_1")
pres = get_component("pressure_1")
atmo = get_component("atmosphere")

while True:
    # Prioritize whichever pillar has the larger gap to next phase
    oxy_gap = ... # compute from atmo.oxygen and next phase threshold
    pres_gap = ...
    if oxy_gap > pres_gap:
        # Favor oxygen, route the current water supply to its tank first
        ...
```

选择与你当前地形改造目标相符的分配规则。只有在确实希望分配器降低运行频率时，才添加 `sleep()`。

热量有所不同

Heat Mk III 直接使用蒸汽，而不是水。捕获的蒸汽可以供给三个汇中的任意组合：

- Steam Turbine，用于电网供电。
- Heat Gen Mk III，用于最大热量输出。
- Steam Condenser，用于为 Oxygen、Pressure、Plants 及其他消费者提供洁净水。

当一个排气口的平均输出和储存余量足以覆盖需求时，它可以为多个相连的汇供给蒸汽。可到达的消费者共享可用流量；它们的节流设置、需求和缓冲空间决定了各自接受的流量。Gas Tanks 是储存活动阶段蒸汽并让每条分支渡过休眠期的实用方式。因此，由冷凝器供给的 Oxygen 和 Pressure 会间接地与 Turbine 及 Heat 争夺同一批捕获蒸汽。

平稳回退

降级是正常的运行状态。机器不会停止；在输入缓冲重新充满之前，输出会继续保持在上一档位。脚本可以观察 `is_degraded()`，检查输入端口，并修正供应链。

交叉引用

- Flow Networks：输入系统 Mk III 使用的内容
- Machine Tiers：档位倍率的工作方式
- Thermal Vents：蒸汽捕获、储存、供电和冷凝

指南 / 世界与基础设施

水井与油井

概览

这颗星球拥有确定性的水井和油井。水井产量稳定；油井会在活动阶段和休眠阶段之间脉冲切换。

水井按产量档位产生 10 / 20 / 30 t/h。活动中的油井产生 8 / 16 / 24 t/h，休眠时产量为 0。

在脚本中读取井

```
scan = self.sonar.scan()
if scan.status == "ok":
    for contact in scan.sites:
        if contact.kind() == "water" or contact.kind() == "oil":
            survey = self.sonar.survey(contact)
            if survey.status == "ok":
                site = survey.site
                print(site.id, site.kind(), site.yield_tier(), site.flow_rate())
            else:
                print(survey.message)
    else:
        print(scan.message)
```

`scan()` 返回 `SonarScanResult`；`survey()` 返回 `SurveyResult`。它们的 `payload` 字段只有在对应的 `.status` 下才可用。

已部署的 Pump 会从节流值 0 开始。连接它的输出端口，检查连接

`ActionResult`，然后设置节流值。本地目标会直接传输；远程目标则需要一条已完成的 Liquid Pipe 路由。Pump 通过只读的 `well()` 查询暴露其绑定的水井。

指南 / 参考

组件

这颗星球上的一切都是组件：传感器、发电机、扫描器、载具、储存箱、冶炼炉、星球本身以及你的日志。

有两种访问方式：

self，即你自己的机器。每个脚本都有一个 **self** 变量，指向运行该脚本的机器。你可以读取状态并控制它。

get_component(id)，即共享组件或通过具体实例 id 指定的另一台机器。从任何位置读取共享状态：

```
clock = get_component("clock")
print(clock.get_elevation())
```

对于实例化机器，id 是稳定的程序化标识符，例如 "bio_collector_1"、"rover_1" 或

"storage_bin_2"。卡片标题是可显示名称，而且可以重命名；在长期运行的脚本中应使用 id，这样重命名不会破坏引用：

```
rover_id = "rover_1"
rover = get_component(rover_id)
print(rover.nav.get_position())
```

使用机器卡片中的精确 id。像 "bio_collector" 这样的类型名称描述的是机器种类；它们不等同于已部署机器的 id "bio_collector_1"。

每次调用都是新实例

get_component(id) 是一个工厂，每次调用都会创建一个新的包装器，就像实例化 Python 类一样。对于绑定到实体机器（rover、smelter、generator）的机器，所有包装器都会读写同一份后端数据，因此实际使用时看起来近似单例：从一个包装器设置的值，另一个包装器也能看到。

对于 transmitter 这类纯虚拟组件，每个包装器都是独立的：

```
# Fails, two separate transmitters, only the first is connected:
get_component("transmitter").connect("earth")
get_component("transmitter").transmit("weather", 42)

# Works, one transmitter, used twice:
t = get_component("transmitter")
t.connect("earth")
t.transmit("weather", 42)
```

包装器只是对象。可以创建一个，也可以创建十个；每个都是独立的。

按名称查找储存箱

储存箱很特殊：你会用它们所存放的材料为其命名（“Iron Ore Bin” “Copper Ingot Bin”），这样按标签查询读起来很自然：

```
ore_bin = get_component_by_name("Iron Ore Bin")
if ore_bin.fill_percent() < 0.2:
    print("Ore running low!")
```

`get_component_by_name()` 适用于任何名称唯一的机器或前哨站，但名称可以编辑。Custom Panels 共享重命名命名空间，却不是组件，因此不会在这里返回。对于长期运行的脚本，应尽可能保存并使用稳定的 id。

车队查找

当脚本需要监视所有载具或无人机，而不想硬编码名称时，使用 `fleet`：

```
fleet = get_component("fleet")
for unit in fleet.vehicles():
    print(unit.id, unit.name, unit.battery_level)
```

Fleet 引用是只读快照。Station API 接受 `.id` 或 `.name`；对于不受重命名影响的脚本，id 更安全。需要完整组件时使用 `get_component(id)`。

星球与日志

有两个组件代表你正在探索的世界：星球和你的日志。

`nocturna` 是星球本身，属于固有数据；对于任何观察者，它都相同：

```
planet = get_component("nocturna")
print(planet.get_name())           # "Nocturna"
b = planet.get_bounds()
print(b.min_x, b.max_x, b.min_y, b.max_y)
```

`journal` 是你记录探索内容的地方；`sonar` 会写入其中，你可以读取它：

```
log = get_component("journal")
for site in log.discovered_sites("nocturna"):
    print(site.id, "at", site.x, site.y)

for site in log.surveyed_sites("nocturna"):
    if site.kind() == "mineral":
        print(site.id, site.item_id, site.purity)
```

传入星球 id 可以限定查询范围；未来的星球也会采用相同方式。

读取与控制

你可以从任何脚本读取共享组件，检查时钟、查看库存，或者按实例 id 监视发电机。

但硬件操作必须通过机器自己的脚本并使用 `self`。DOCS 中标记为 **SELF ONLY** 的方法无法通过 `get_component()` 使用；你会得到一条解释原因的错误。

每个组件都有 `.id`（程序化标识符）和 `.name`（显示名称）。下面的 Components 部分列出了所有可用组件及其方法。

命令

系统

boot()

初始化系统并运行诊断。

返回值

返回类型	结果字段	成功载荷
ActionResult	.status、.message	无

结果

状态	类型	含义
"booting"	成功	启动序列已开始。
"already_booted"	成功	系统已经启动。

boot()

activate_power()

在空间站完成启动后打开电网。

返回值

返回类型	结果字段	成功载荷
ActionResult	.status、.message	无

结果

状态	类型	含义
"activating"	成功	激活已开始。
"already_online"	成功	系统已经在线。
"boot_required"	拒绝	必须先完成空间站启动，才能使用此操作。

activate_power()

activate_sensors()

使传感器阵列上线。需要电力。

返回值

返回类型	结果字段	成功载荷
ActionResult	.status、.message	无

结果

状态	类型	含义
"initializing"	成功	初始化已开始。
"already_online"	成功	系统已经在线。
"power_required"	拒绝	此操作需要在线的电力系统。

```
activate_sensors()
```

get_component(name)

按不可变 id 访问玩家拥有的实体（例如 "solar_3"、"outpost_home"）。id 在创建时自动生成，之后永不改变：需要生命周期长于重命名的脚本应使用它。若要按可变的显示名称查找，请使用 get_component_by_name(name)。如果没有实体使用给定 id，则返回 None。

参数

名称	类型	描述
name	string	实体 id（机器或前哨站）

返回值

Component 对象

```
clock = get_component("clock")
time = clock.get_time()
print(time)
```

get_component_by_name(name)

按显示名称查找任何可寻址的玩家拥有组件（机器或前哨站）。名称默认是实体的 id，但可以从 Computer System 选项卡自由重命名；共享重命名命名空间会强制保证唯一性。Custom Panels 共享该命名空间，但不是组件，因此不会在这里返回。该名称可变：对于长期运行的脚本，get_component(id) 才是稳定形式。如果没有组件使用给定名称，则返回 **None**。

参数

名称	类型	描述
name	string	实体的显示名称

返回值

Component（机器或前哨站）；如果不存在使用该名称的组件，则为 **None**

```
ore_bin = get_component_by_name("Iron Stockpile")
print(ore_bin.count("iron_ore"))
```

命令

基础设施

get_pipe(pipe_id)

按 id 查找基础设施管道。返回一个实时的只读 **Pipe** 句柄；如果不存在该 id 的管道，则返回 **None**。

参数

名称	类型	描述
pipe_id	string	管道 id（来自 list_pipes()）

返回值

实时的只读 **Pipe** 句柄，或 **None**

```
pipe = get_pipe("pipe_1")
if pipe != None:
    print(pipe.state())
```

list_pipes()

列出当前铺设的每一条基础设施管道（已完成或进行中），并以实时的只读 **Pipe** 句柄表示。

返回值

实时只读 **Pipe** 句柄的列表

```
for pipe in list_pipes():
    print(pipe.id, pipe.state())
```


内置函数

输出

`print(*values, sep=" ", end="\n")`

向控制台输出文本。多个值会用 `sep` 连接（默认是单个空格）；`end` 会追加在最后一次输出之后（默认是换行）。控制台输出是字符流，其中的换行符负责断行，因此 `end=""` 会让当前行保持打开状态，下一次 `print()` 会继续写入：可以用这种方式通过多次调用构建一行，然后用一个不带参数的 `print()` 结束这一行。每次调用都会立即更新未完成的行。在这一行内，`\r` 会把写入位置移回行首，`\b` 会向后移动一个可见字符。两个控制符本身都不会擦除文本；后续文本会覆盖已有内容。两个控制符都不能进入更早的行，ANSI 转义序列也不会被解释。

返回值

None

`warn(*values, sep=" ", end="\n")`

向持久化控制台输出一行琥珀色警告。参数行为与 `print()` 相同，但会路由到 WARNINGS 过滤器。需要关注但不应显示 toast 的后台监视器可以使用它。玩家需要被打断时，使用 `notify(text, "warn")`。

返回值

None

`debug(*values, sep=" ", end="\n")`

向持久化控制台输出低优先级遥测信息。参数行为与 `print()` 相同，但除非在控制台选项中启用 debug 输出，否则会在 ALL 视图中隐藏。它适合嘈杂的调优数据，避免挤占普通日志。

返回值

None

`notify(text, /, level="info", duration_seconds?, dismissible=True)`

向玩家显示 toast，并将其添加到 Computer → Notifications 存档。请谨慎使用：只用于确实需要操作员注意的事件（电池危急、合约完成、无人机搁浅）；持续的遥测信息应优先使用 `print()`。`level` 为 "info"（默认）、"warn" 或 "error"，决定 toast 的颜色和历史徽标。`duration_seconds` 设置 toast 在自动消失前的显示时长：会限制在 0.5-30s；省略时使用默认值（info/warn 为 5s，error 为 6s）。将持续时间设为 0 可使 toast 固定显示：它不会自动消失，直到玩家点击它；致命错误且必须确认时可使用此方式。`dismissible` 默认为 `True`；将第四个参数设为 `False`，可创建不能提前关闭的强制阅读 toast。固定 toast 始终可以关闭，因此不会永久钉在屏幕上。同一脚本连续发出的相同通知会在 1 秒窗口内合并；已经显示的固定 toast 不会重复，因此紧密循环不会刷屏。

参数

名称	类型	描述
<code>text</code>	<code>string</code>	通知文本
<code>level</code>	<code>string</code>	通知严重级别
<code>duration_seconds</code>	<code>number</code>	以真实秒数计的显示时长；0 表示固定显示
<code>dismissible</code>	<code>boolean</code>	玩家是否可以提前关闭 toast

返回值

`None`

控制

sleep(seconds, /)

等待后继续执行。参数使用真实秒数，而不是世界时钟小时数。昼夜循环会把 24 个世界时钟小时压缩到更短的真实时间窗口中，因此在默认的每天 10 分钟节奏下，sleep(25) 大约相当于 1 个世界时钟小时。对于与星球相关的延迟，可以查询换算值：sleep(get_component("clock").real_seconds_per_hour() * 2) 会等待整整两个世界时钟小时，与节奏设置无关。循环会自动控制节奏；sleep() 用于有意的延迟。

返回值

None

引发的异常

异常	条件
TypeError	sleep(seconds) 需要一个数值型时长。
ValueError	sleep(seconds) 需要一个有限且大于或等于 0 的时长。
OverflowError	sleep(seconds) 无法在模拟刻度范围内表示请求的时长。

工具

len(value, /)

返回列表、元组、字符串、字典或集合的长度。

返回值

number

`range(stop, /) / range(start, stop, /) / range(start, stop, step, /)`

从 `start` 到 `stop`（不包含 `stop`）按 `step` 步长生成有限整数范围并将其具体化。`range(5) → [0, 1, 2, 3, 4]`。端点和步长保留任意大小的精确整数，但生成的列表必须适合解释器的集合上限。所有参数都必须是整数；`step=0` 和小数值会被拒绝。负步长会倒数：`range(5, 0, -1) → [5, 4, 3, 2, 1]`。

返回值

`list<number>`

`slice(stop, /) / slice(start, stop, step=None, /)`

构造可复用的切片对象，用于列表、元组和字符串下标。`slice(None, None, -1)` 是 `[::-1]` 的可复用形式；`seq[s]` 遵循与 `seq[start:stop:step]` 相同的边界和步长规则。

返回值

`slice`

`type(value, /)`

以字符串返回玩家值的基于值的类型类别：用户类实例返回其类名；`"int"` 覆盖整数（包括精确的大整数）；`"float"` 覆盖小数 / 非有限数；内置类型报告 `"str"`、`"bool"`、`"NoneType"`、`"list"`、`"tuple"`、`"dict"`、`"set"` 或 `"slice"`。Game API 值继续保留既有的 `"object"` 类别，以便现有脚本保持不变；需要具体的已注册数据对象名称时，使用 `object_type(value)`。数值类别根据当前值而不是字面写法确定，因此 `type(4.0)` 会返回 `"int"`，因为该值是整数。Game API 仍会把接受整数或浮点数的宽泛数值参数记录为 `number`。

返回值

`string`

object_type(value, /)

返回具体的已注册名称，例如 "MiningSite"、"CatalogedFragment" 或 "Position"，用于表示 Game data object，而不会改变既有的 type(value) 结果。组件引用返回 "Component"；其他值使用与 type(value) 相同的公共类别。

参数

名称	类型	描述
value	any	需要确定具体 Game data-object 名称的值

返回值

string

vars(object, /)

返回对象公共数据字段的脱离对象的浅字典。结构化结果、位置、日志条目和其他 Game data value 会包含文档记录的属性字段，但不包含方法；类实例会包含其自身存储的属性。修改返回的字典不会改变对象，不过其中嵌套的列表和字典是共享的。需要具体的已注册名称时，另行使用 object_type(object)。必须恰好传入一个对象；不支持无参数的局部作用域形式。

参数

名称	类型	描述
object	any	用于生成快照的 Game API 值或类实例

返回值

dict

object()

构造一个仅具身份的基础对象。它不接受参数，也没有可写的游戏脚本属性。用户类会隐式继承 object。

返回值

object

TypedDict(name, fields, /)

为编辑器声明固定键的字典形状。使用函数式形式 (`State = TypedDict("State", {"mode": str})`)，并用该名称标注记录 (`state: State = {...}`)。运行时将其作为无作用的类型标记；编辑器用它实现键自动补全、字段类型流转和拼写错误检查。

参数

名称	类型	描述
name	string	声明的类型名称
fields	any	将字段名称映射到类型表达式的字典

返回值

any

hasattr(value, name, /)

如果 `value.name` 可读，则返回 `True`，否则返回 `False`。属性名称必须是字符串。它适用于接受多个已记录对象形状的 duck-typed helper，例如 `hasattr(value, "required_recipe")`。Gameplay result object 会保留其固定字段，因此应在读取 `analysis.info` 这类有记录的 payload 前，先根据 `.status` 分支。未安装模块时，像 `self.sonar` 这样的缺失挂载子对象会返回 `False`。

返回值

boolean

getattr(value, name, default?, /)

按字符串名称读取属性，行为与 `value.name` 完全相同。不提供 `default` 时，缺失属性会引发 `AttributeError`；提供 `default` 时，缺失属性会返回该回退值。它适用于游戏对象的属性 / 方法以及 `getattr([1], "append")` 这样的内置方法。请与 `hasattr()` 一起用于 duck-typed helper，不要用它替代 gameplay result 的已记录 `.status`、`.message` 和 `payload` 字段。

返回值

any

`sorted(sequence, /, *, key=None, reverse=False)`

使用 `<` 在相互可比较的键之间进行比较，并返回一个新的排序列表。`key=None`（默认值）时，每个值自身就是键。数值键和布尔键可以在同一族中比较，字符串按字典序比较，用户对象可以定义 `<` 所需的精确富比较槽；不支持的组合会引发异常。`key=` 会对每个项目调用一次，而且必须是纯函数：不能暂停或改变游戏状态。`reverse` 会按真值判断；有限输入会在解释器的集合上限内立即处理。

返回值

`list`

`reversed(iterable, /)`

返回任意有限可迭代对象的逆序列表副本。该操作会完整消耗输入，因此无限生成器会超出集合上限；字典（其键）、集合（确定性的插入顺序）、生成器以及自定义迭代器协议类都可以接受。

返回值

`list`

`enumerate(iterable, /, start=0)`

返回 `(index, value)` 对的列表。可选的整数 `start=N` 会移动索引：`enumerate(items, start=1)` 表示从 1 开始。任意大的整数起点仍保持精确。`start` 可以按位置或按关键字传入，但不能两种方式同时使用。

返回值

`list<tuple>`

`zip(*iterables, strict=False)`

将可迭代对象组合成元组。不带参数时返回空列表；带一个参数时返回单元元素组的列表。除非 `strict=True`（此时输入长度不同时可能会引发异常），否则在最短输入处停止。

返回值

`list<tuple>`

`isinstance(value, type_or_tuple, /)`

检查值是否属于给定类型。接受内置类型可调用对象（`object`、`int`、`float`、`str`、`list`、`dict`、`set`、`tuple`、`slice`、`bool`）、字符串类型名称、类型元组，或像 `int | float` 这样的类型联合（匹配任意一种）。`int` 匹配整数和布尔值；`float` 匹配小数 / 非有限数；字符串 `"number"` 仍表示宽泛的数值族，供有意接受整数或浮点数的脚本使用。

返回值

`boolean`

`callable(value, /)`

当值具有调用表面时返回 `True`：包括用户定义的 `def / lambda`、内置函数、类、绑定方法，以及类型定义了 `__call__` 的类实例。与 Python 一样，它检查类型级别的调用槽而不执行其描述符；实际调用如果该槽解析为不可调用值，仍然可能引发异常。

返回值

`boolean`

`dir(value, /)`

按排序顺序列出 `value` 可用的方法和属性名称，用于运行时自省，帮助发现可执行的操作。对于组件或游戏对象（`dir(self)`、`dir(get_component("smelter_1"))`），它会直接从控制台返回该对象的可调用方法和属性。对于内置容器，它会返回该方法：`dir([1, 2]) → "append"、"pop"、……`；`dir("hi") → "upper"、"split"、……`。没有成员的值（数字、布尔值、`None`）会返回空列表。必须恰好传入一个值；不支持列出当前作用域名称的无参数 `dir()`。

返回值

`list`

`hash(value, /)`

将任何可哈希值转换为整数哈希值：字符串、数字、布尔值、`None`、由可哈希值组成的元组、函数、类以及可哈希的用户实例。实例默认按身份进行哈希；定义 `__eq__` 却不定义 `__hash__` 会使实例不可哈希，而自定义 `__hash__` 会控制 `hash(obj)`、字典键和集合成员关系。

返回值

number

`super() / super(type, object, /)`

返回一个代理，从选定的类之后开始搜索接收者的 MRO。在方法内部，使用 `super()` 进行协作式父类调用，例如 `super().__init__(...)`。显式的 `super(type, object)` 形式接受 `type` 的实例或子类；不支持单参数形式。

返回值

any

`issubclass(cls, class_or_tuple, /)`

如果 `cls` 是给定类或其子类（或元组中任一类的子类），则返回 `True`，判断依据是 MRO。`issubclass(Dog, Animal)` 为 `True`；每个类都是 `object` 的子类。

返回值

boolean

property(fget=None, fset=None, fdel=None, doc=None)

构造一个 property 描述符。常见的 getter 形式通常写作 @property，也可以直接调用 property(fget, fset, fdel, doc)；每个参数都是可选的，也可以按名称传入。实例读取会调用 fget，写入会调用 fset，删除会调用 fdel；已经绑定的钩子会保持绑定，并额外接收 property 实例作为参数。getter(fn)、setter(fn) 和 deleter(fn) 会返回克隆后的描述符。省略 doc 或传入 None 时，会通过普通属性查找沿用 fget.__doc__；显式的非 None 文档值会在克隆中保留。

参数

名称	类型	描述
fget	any	可选的实例 getter
fset	any	可选的实例 setter
fdel	any	可选的实例 deleter
doc	any	可选的显式描述符文档值

返回值

property

classmethod(func, /)

将函数包装为类方法。通常写作 @classmethod；被装饰的方法会接收类（惯例上命名为 cls）作为第一个参数，并且可以在类或实例上调用。

参数

名称	类型	描述
func	any	要绑定到类的函数

返回值

any

`staticmethod(func, /)`

将函数包装为静态方法。通常写作 `@staticmethod`；被修饰的方法既不接收实例，也不接收类，而是作为类命名空间下的普通函数运行。

参数 (PARAMETERS)

名称	类型	描述
<code>func</code>	<code>any</code>	要存储且不绑定接收者的函数

返回值 (RETURNS)

`any`

`NotImplemented: NotImplemented`

由运算符方法返回的不可变单例，用于请求 Python 尝试反射分派或普通回退处理。

返回值 (RETURNS)

`NotImplemented`

`__debug__: boolean`

不可变的布尔常量，在游戏脚本中始终为 `True`。可以读取，但不能直接赋值或删除。

返回值 (RETURNS)

`boolean`

转换 (CONVERSION)

`str(value='', /)`

将值转换为字符串。没有参数时返回空字符串 `''`。

返回值 (RETURNS)

`string`

`int(value=0, /, base?)`

转换为整数。没有参数时返回 `0`。数字会向零截断。字符串转换会消耗去除首尾空白后的完整值，并接受 Python 下划线分隔符和 Unicode 十进制数字。可选基数为 `2-36` 或 `0`；`base=0` 会检测 `0x`、`0b`、`0o` 前缀。带前缀和基数的形式会在解释器整数预算范围内保留任意大小的精确整数。

返回值 (RETURNS)

`number`

`float(value=0.0, /)`

转换为浮点数。没有参数时返回 `0.0`。字符串转换会消耗去除首尾空白后的完整值，并接受 Python 小数/指数语法、下划线分隔符、Unicode 十进制数字，以及不区分大小写的 `inf` / `nan`。尾随无效字符会引发错误；布尔值会转换为 `0.0` / `1.0`。

返回值 (RETURNS)

`number`

`chr(code, /)`

根据 Unicode 码点返回字符（例如 `chr(65) → 'A'`）。

返回值 (RETURNS)

`string`

`ord(char, /)`

根据字符返回 Unicode 码点（例如 `ord('A') → 65`）。

返回值 (RETURNS)

`number`

`bool(value?, /)`

转换为布尔值（True/False）。没有参数时返回 False。

返回值（RETURNS）

boolean

`list(iterable?, /)`

将任意可迭代对象转换为列表，或使用 `list()` 创建空列表。方法包括：`.append(x)`、`.pop([i])`、`.insert(i, x)`、`.remove(x)`、`.index(x [, start [, end]])`、`.count(x)`、`.sort()`、`.reverse()`、`.copy()`、`.extend(iter)`、`.clear()`、`.length`（属性）。使用 `lst[i]` 进行下标访问，使用 `lst[start:end:step]` 进行切片。完整的 Python 兼容语义请参见下面的各个条目。

返回值（RETURNS）

list

`dict(kwargs) / dict(source, /, kwargs)`

构建字典。空形式为 `dict()`。从键值对构建：`dict([("a", 1), ("b", 2)])`。浅复制另一个字典：`dict(d)`。关键字形式为 `dict(name="Mars", temp=-63)`。键可以是任意可哈希值，包括由可哈希值组成的元组，以及正确实现可哈希性的用户实例。方法包括：`.keys()`、`.values()`、`.items()`、`.has(k)`、`.get(k, default?)`、`.pop(k, default?)`、`.popitem()`、`.setdefault(k, default?)`、`.update(other)`、`.copy()`、`.clear()`、`.length`（属性）。运算符：`a | b` 返回合并副本，右侧值优先；`a |= b` 就地更新。使用 `d[k]` 进行下标访问；缺失键会引发错误。安全查找请使用 `.get(k)` 或 `.has(k)`。

返回值（RETURNS）

dict

`set(iterable?, /)`

从任意可迭代对象构建唯一成员集合，空集合使用

`set()`。成员可以是任意可哈希值，包括由可哈希值组成的元组，以及正确实现可哈希性的用户实例。字面量使用 `{1, 2, 3}`；空的 `{}` 是字典而不是集合，因此空集合始终使用 `set()`。方法包括：`.add(x)`、`.remove(x)`、`.discard(x)`、`.pop()`、`.clear()`、`.copy()`、`.union(s)`、`.intersection(s)`、`.difference(s)`、`.symmetric_difference(s)`、`.update(s)`、`.issubset(s)`、`.issuperset(s)`、`.isdisjoint(s)`。运算符包括：`in`、`|`（并集）、`&`（交集）、`-`（差集）、`^`（对称差集）、`< <= >= >`（子集/超集）和 `==`。

返回值 (RETURNS)

`set`

`tuple(iterable?, /)`

从任意可迭代对象构建元组，空元组使用 `tuple()`。元组类似列表，但不可变（没有 `append/pop/sort`）；适合存放固定记录并作为可哈希键。方法包括：`.index(x)`、`.count(x)`、`.length`（属性）。下标访问和切片

`t[i]` / `t[a:b]` 与列表相同。

返回值 (RETURNS)

`tuple`

`hex(integer, /)`

将整数渲染为带 `0x` 前缀的 Python 风格十六进制字符串。`hex(255) → '0xff'`、`hex(-16) → '-0x10'`。浮点数会被拒绝。

返回值 (RETURNS)

`string`

`bin(integer, /)`

将整数渲染为带 `0b` 前缀的二进制字符串。`bin(10) → '0b1010'`。浮点数会被拒绝。

返回值 (RETURNS)

`string`

`oct(integer, /)`

将整数渲染为带 `0o` 前缀的八进制字符串。`oct(8) → '0o10'`。浮点数会被拒绝。

返回值 (RETURNS)

string

`repr(value, /)`

返回便于开发者读取的值字符串：字符串带引号，容器使用嵌套的 `repr`。`repr([1, "a"]) → "[1, 'a']"`（注意 `'a'` 两侧的引号）。当你需要查看值的结构而不是显示形式时使用它。也可以通过 f-string 的 `!r` 转换访问：`f"{name!r}"`。

返回值 (RETURNS)

string

函数式工具 (FUNCTIONAL)

`total_ordering(cls, /)`

保留类对象，并根据 `__eq__` 与 `__lt__`、`__le__`、`__gt__` 或 `__ge__` 中的一个缺失方法，补齐其余排序方法的类装饰器。显式定义的方法绝不会被替换。应用该装饰器会改变本地类命名空间。

参数 (PARAMETERS)

名称	类型	描述
<code>cls</code>	any	要补全的类

返回值 (RETURNS)

any

`wraps(wrapped, /)`

返回一个装饰器：它保留被包装可调用对象受支持的名称、限定名称、文档字符串、自定义属性和 `__wrapped__` 链接，同时保持包装器的调用行为。应用返回的装饰器只会改变该本地包装器函数。

参数 (PARAMETERS)

名称	类型	描述
<code>wrapped</code>	<code>any</code>	要复制其元数据的可调用对象

返回值 (RETURNS)

`any`

数学 (MATH)

`abs(number, /)`

绝对值。

返回值 (RETURNS)

`number`

`isclose(a, b, rel_tol=0.000000001, abs_tol=0.0)`

当两个数足够接近、可以视为相等时返回 `True`。`rel_tol` 随被比较的值缩放；`abs_tol` 设置同一单位中的固定可接受差值，适用于接近零的值以及坐标等物理读数。容差必须为非负数。这是 Python `math.isclose()` 的直接可用等价形式。

参数 (PARAMETERS)

名称	类型	描述
<code>a</code>	<code>number</code>	第一个数
<code>b</code>	<code>number</code>	第二个数
<code>rel_tol</code>	<code>number</code>	最大相对差值
<code>abs_tol</code>	<code>number</code>	以值的单位表示的最大绝对差值

返回值 (RETURNS)

`boolean`

`min(a, b, ...) / min(sequence, key=fn, default=v)`

返回在 < 比较下键最小的原始值。接受多个值或一个有限可迭代对象，并在集合限制范围内立即处理。使用 `key=None`（默认值）时，每个值本身就是其键；否则，纯 `key=` 回调会对每个项目运行一次，且不能暂停或改变游戏状态。键必须可以相互比较：数值键与布尔键可在同一族内比较，字符串按字典序比较，用户对象可以提供 < 所需的丰富比较槽。在单可迭代对象形式中，`default=v` 为空输入提供结果；不提供时，空输入会引发错误。

返回值 (RETURNS)

any

`max(a, b, ...) / max(sequence, key=fn, default=v)`

返回在 > 比较下键最大的原始值。接受多个值或一个有限可迭代对象，并在集合限制范围内立即处理。使用 `key=None`（默认值）时，每个值本身就是其键；否则，纯 `key=` 回调会对每个项目运行一次，且不能暂停或改变游戏状态。键必须可以相互比较：数值键与布尔键可在同一族内比较，字符串按字典序比较，用户对象可以提供 > 所需的丰富比较槽。在单可迭代对象形式中，`default=v` 为空输入提供结果；不提供时，空输入会引发错误。

返回值 (RETURNS)

any

`round(number, ndigits=None)`

使用“舍入到偶数”的规则进行舍入。没有 `ndigits`（或为 `None`）时，返回最近的整数值，并拒绝 NaN/无穷大。整数和布尔输入保持精确；负的 `ndigits` 会以十进制舍入精确整数。两个参数都接受关键字形式。

参数 (PARAMETERS)

名称	类型	描述
<code>number</code>	<code>number</code>	要舍入的值
<code>ndigits</code>	<code>any</code>	十进制位数或 <code>None</code>

返回值 (RETURNS)

number

`pow(base, exp, mod=None)`

计算 `base` 的 `exp` 次幂。可选的 `mod` 执行精确模幂运算；当底数存在模逆元时，接受负指数。`base`、`exp` 和 `mod` 都接受位置形式或关键字形式。

参数 (PARAMETERS)

名称	类型	描述
<code>base</code>	<code>number</code>	底数
<code>exp</code>	<code>number</code>	指数
<code>mod</code>	<code>any</code>	可选的整数模数

返回值 (RETURNS)

`number`

`random()`

返回大于等于 0 且小于 1 的随机浮点数。每次成功运行都会开始新的自动序列。导入 `random` 后，也可使用 `random.random()`。

返回值 (RETURNS)

`number`

`rand()`

`random()` 的简短别名：返回大于等于 0 且小于 1 的随机浮点数。每次成功运行都会开始新的自动序列。导入 `random` 后，也可使用 `random.rand()`。

返回值 (RETURNS)

`number`

`randint(min, max, /)`

返回满足 $\text{min} \leq N \leq \text{max}$ 的随机整数 N 。边界包含在内，并且必须是整数。每次成功运行都会开始新的自动序列。可与 `planet.get_bounds()` 配合，用于生成有效的随机坐标。导入 `random` 后，也可使用 `random.randint(min, max)`。

返回值 (RETURNS)

`number`

`sqrt(number, /)`

平方根。负输入会出错。

返回值 (RETURNS)

`number`

`floor(number, /)`

向下舍入到最近的整数。

返回值 (RETURNS)

`number`

`ceil(number, /)`

向上舍入到最近的整数。

返回值 (RETURNS)

`number`

`trunc(number, /)`

丢弃小数部分（向零舍入）。

返回值 (RETURNS)

`number`

`divmod(a, b, /)`

用 **a** 除以 **b** 并返回两个值：向下舍入的商，以及剩余的余数。例如，`divmod(19, 2)` 返回 **(9, 1)**，因为 2 在 19 中可以完整容纳九次，余数为 1。

返回值 (RETURNS)

`tuple<number>`

`sign(number, /)`

对于负数、零和正数输入，分别返回 **-1**、**0** 或 **1**。

返回值 (RETURNS)

`number`

`exp(number, /)`

返回 **e** 的参数次幂。

返回值 (RETURNS)

`number`

`log(number, base?, /)`

返回自然对数，或以给定基数计算的对数。

返回值 (RETURNS)

`number`

`log2(number, /)`

以 **2** 为底的对数。

返回值 (RETURNS)

`number`

`log10(number, /)`

以 10 为底的对数。

返回值 (RETURNS)

`number`

`sin(radians, /)`

返回以弧度表示的角的正弦。

返回值 (RETURNS)

`number`

`cos(radians, /)`

返回以弧度表示的角的余弦。

返回值 (RETURNS)

`number`

`tan(radians, /)`

返回以弧度表示的角的正切。

返回值 (RETURNS)

`number`

`asin(number, /)`

反正弦。输入必须位于 -1 到 1 之间。

返回值 (RETURNS)

`number`

`acos(number, /)`

反余弦。输入必须位于 -1 到 1 之间。

返回值 (RETURNS)

number

`atan(number, /)`

反正切。

返回值 (RETURNS)

number

`atan2(y, x, /)`

计算 y/x 的反正切，并正确选择象限。

返回值 (RETURNS)

number

`degrees(radians, /)`

将弧度转换为角度。

返回值 (RETURNS)

number

`radians(degrees, /)`

将角度转换为弧度。

返回值 (RETURNS)

number

`sum(iterable, /, start=0)`

对可迭代对象中的每个数值项求和。可选的 `start`（数字或列表）：`sum(list_of_lists, [])` 会展平列表。`start` 可以是位置参数或关键字参数，但不能同时使用两者。

返回值 (RETURNS)

`number | list`

`prod(iterable, /, start=1)`

对可迭代对象中的每个数值项求积。空可迭代对象返回 `start`。`start` 可以是位置参数或关键字参数，但不能同时使用两者。

返回值 (RETURNS)

`number`

`inf: number`

正无穷，大于任何有限数。将 `-inf` 用于负无穷，包括在搜索和寻路算法中作为初始最佳值或最差值的场景。

返回值 (RETURNS)

`number`

`pi: number`

数学常量 $\pi \approx 3.14159$ 。

返回值 (RETURNS)

`number`

`tau: number`

数学常量 $\tau = 2\pi$ 。

返回值 (RETURNS)

`number`

逻辑 (LOGIC)

`all(iterable, /)`

当可迭代对象中的每一项都是真值时返回 `True`。遇到第一个假值就停止，因此生成器只会被消耗到所需位置。

返回值 (RETURNS)

`boolean`

`any(iterable, /)`

当可迭代对象中任意一项是真值时返回 `True`。遇到第一个真值就停止，因此生成器只会被消耗到所需位置。

返回值 (RETURNS)

`boolean`

迭代 (ITERATION)

`iter(iterable, /)`

返回一个迭代器。内置可迭代对象使用兼容性的列表形迭代器；协议迭代器和生成器保留自身身份，因此 `iter(iterator)` `is iterator`。`for` 会逐项推进生成器和自定义迭代器，所以可以从无限迭代器中跳出。必须完成消费的操作（例如 `list(...)`）仍然受到边界限制。

返回值 (RETURNS)

`list | generator`


```
next(iterator, default?, /)
```

将迭代器推进一项。由 `iter()` 返回的迭代器和自定义 `__next__` 迭代器会在多次调用之间保留游标。普通列表保持兼容行为，即从前端弹出；其他非迭代器可迭代对象会返回其第一个物化项。耗尽时会引发 `StopIteration`，除非提供了 `default`。

返回值 (RETURNS)

```
any
```

```
pairwise(iterable, /)
```

返回可迭代对象中的相邻成对元素。 `pairwise([1,2,3]) → [(1,2), (2,3)]`。

返回值 (RETURNS)

```
list<tuple>
```

```
batched(iterable, size, /)
```

将可迭代对象拆分为大小为 `size` 的元组批次。最后一个批次可以更短。 `batched([1,2,3,4,5], 2) → [(1,2), (3,4), (5,)]`。

返回值 (RETURNS)

```
list<tuple>
```

```
starmap(fn, iterable, /)
```

将每个元组/列表项解包为参数，并调用纯 `fn`。回调不能暂停脚本或改变游戏状态。 `starmap(pow, [(2,3), (3,2)]) → [8,9]`。空输入不会检查或调用 `fn`。

返回值 (RETURNS)

```
list
```

`flatten(iterable, /)`

将嵌套可迭代对象展平一层为列表。`flatten([[1,2], (3,4)]) → [1,2,3,4]`。

返回值 (RETURNS)

`list`

`count_by(iterable, key_fn?, /)`

将项目计数到字典中。没有 `key_fn`（或其为 `None`）时，对每个项目计数。使用纯 `key_fn` 时，对计算得到的键计数；回调不能暂停脚本或改变游戏状态：`count_by(items, lambda x: x.kind)`。空输入不会检查或调用键函数。

参数 (PARAMETERS)

名称	类型	描述
<code>iterable</code>	<code>any</code>	要计数的值
<code>key_fn</code>	<code>any</code>	将每个项目映射到要计数键的可选函数

返回值 (RETURNS)

`dict<any, number>`

`chain(*iterables)`

按顺序将任意数量的可迭代对象展平为一个列表。`chain([1,2], [3,4]) → [1,2,3,4]`。接受列表、元组、字符串和集合。

返回值 (RETURNS)

`list`

`accumulate(iterable, /)`

对任意有限可迭代对象计算运行前缀和。`accumulate([1,2,3,4]) → [1,3,6,10]`。只接受数值项；布尔值作为整数参与计算，任意大的整数仍保持精确。

返回值 (RETURNS)

`list<number>`

`combinations(iterable, k, /)`

返回可迭代对象中所有包含 `k` 个元素的组合，按输入顺序排列且不重复。返回元组列表。`combinations([1,2,3], 2) → [(1,2), (1,3), (2,3)]`。

返回值 (RETURNS)

`list<tuple>`

`permutations(iterable, k?, /)`

返回可迭代对象的所有长度为 `k` 的有序排列。`k` 默认为完整长度。`permutations([1,2,3]) →` 全部 6 种排列的元组。

返回值 (RETURNS)

`list<tuple>`

`product(*iterables, repeat=1)`

笛卡尔积。`product([0,1], [0,1]) → [(0,0), (0,1), (1,0), (1,1)]`。每个输入都必须是可迭代对象；结果是元组列表。可选关键字 `repeat=N` 会重复输入池，其行为与 `itertools.product([0,1], repeat=2)` 一致。

返回值 (RETURNS)

`list<tuple>`

`map(fn, iter1, iter2?, ..., /, strict=False)`

对每个可迭代对象中对应的项目应用纯 `fn`。回调不能暂停脚本或改变游戏状态。单迭代对象形式调用 `fn(x)`；多迭代对象形式调用 `fn(x, y, ...)`，并在最短输入处停止，除非 `strict=True`，此时输入长度不同时可能会引发错误。如果没有生成任何行，就不会检查或调用 `fn`。

返回值 (RETURNS)

`list`

`filter(fn, iterable, /)`

保留使纯 `fn(item)` 返回真值的项目；回调不能暂停脚本或改变游戏状态。`filter(None, iter)` 不使用回调，保留每个真值项目。空输入不会检查或调用 `fn`。

返回值 (RETURNS)

`list`

`reduce(fn, iterable, initializer?, /)`

通过反复调用纯 `fn(total, item)`，将可迭代对象合并为一个值；回调不能暂停脚本或改变游戏状态。没有初始化值时，第一个项目会成为初始总值；此时空可迭代对象会引发错误。带初始化值的空输入，以及不带初始化值的单项目输入，都不会调用回调。也可使用 `from functools import reduce` 获取。

返回值 (RETURNS)

`any`

异常 (EXCEPTIONS)

`BaseException(message?, /)`

受支持异常层次结构的根类。

参数 (PARAMETERS)

名称	类型	描述
<code>message</code>	<code>any</code>	可选的位置异常消息

返回值 (RETURNS)

`BaseException`

Exception(message?, /)

普通 except Exception: 处理器所匹配的基类。

参数 (PARAMETERS)

名称	类型	描述
message	any	可选的位置异常消息

返回值 (RETURNS)

Exception

ArithmeticError(message?, /)

数值计算失败的基类。

参数 (PARAMETERS)

名称	类型	描述
message	any	可选的位置异常消息

返回值 (RETURNS)

ArithmeticError

ValueError(message?, /)

值的类型正确，但值本身无效。

参数 (PARAMETERS)

名称	类型	描述
message	any	可选的位置异常消息

返回值 (RETURNS)

ValueError

`TypeError(message?, /)`

某个操作接收到类型不合适的值。

参数 (PARAMETERS)

名称	类型	描述
message	any	可选的位置异常消息

返回值 (RETURNS)

`TypeError`

`PermissionError(message?, /)`

某个操作违反了调用方的所有权或访问约定，因此被禁止。

参数 (PARAMETERS)

名称	类型	描述
message	any	可选的位置异常消息

返回值 (RETURNS)

`PermissionError`

`ReferenceError(message?, /)`

某个已捕获的组件或模块句柄不再有效。

参数 (PARAMETERS)

名称	类型	描述
message	any	可选的位置异常消息

返回值 (RETURNS)

`ReferenceError`

LookupError(message?, /)

查找错误（LookupError）。

参数	类型	说明
message	any	可选的位置参数错误消息

返回值

LookupError

KeyError(message?, /)

字典键不存在。

参数	类型	说明
message	any	可选的位置参数错误消息

返回值

KeyError

IndexError(message?, /)

序列索引超出范围。

参数	类型	说明
message	any	可选的位置参数错误消息

返回值

IndexError

AttributeError(message?, /)

属性引用失败。

参数	类型	说明
message	any	可选的位置参数错误消息

返回值

AttributeError

RuntimeError(message?, /)

运行时错误（RuntimeError）无法归入更具体的类别。

参数	类型	说明
message	any	可选的位置参数错误消息

返回值

RuntimeError

RecursionError(message?, /)

解释器的调用深度限制已超出。

参数	类型	说明
message	any	可选的位置参数错误消息

返回值

RecursionError

NameError(message?, /)

无法解析局部、自由或全局名称。

参数	类型	说明
message	any	可选的位置参数错误消息

返回值

NameError

UnboundLocalError(message?, /)

静态局部变量在绑定之前被读取；这是 NameError 的子类。

参数	类型	说明
message	any	可选的位置参数错误消息

返回值

UnboundLocalError

ImportError(message?, /)

导入无法提供所请求的绑定。

参数	类型	说明
message	any	可选的位置参数错误消息

返回值

ImportError

ModuleNotFoundError(message?, /)

找不到已导入的模块；这是 ImportError 的子类。

参数	类型	说明
message	any	可选的位置参数错误消息

返回值

ModuleNotFoundError

OverflowError(message?, /)

数值转换或有界分配超出了支持的范围。

参数	类型	说明
message	any	可选的位置参数错误消息

返回值

OverflowError

ZeroDivisionError(message?, /)

除法或取模使用了零除数。

参数	类型	说明
message	any	可选的位置参数错误消息

返回值

ZeroDivisionError

StopIteration(message?, /)

迭代器已耗尽。其 .value 包含生成器的返回值，如果没有则为 None。

参数	类型	说明
message	any	可选的位置参数错误消息

返回值

StopIteration

StopIteration.value

已完成生成器携带的返回值，如果没有则为 None。

返回值

any

GeneratorExit(message?, /)

当 `close()` 要求生成器停止时，在生成器内部引发。它直接派生自 `BaseException`。

参数	类型	说明
message	any	可选的位置参数错误消息

返回值

GeneratorExit

AssertionError(message?, /)

断言语句失败。

参数	类型	说明
message	any	可选的位置参数错误消息

返回值

AssertionError

SyntaxError(message?, /)

源代码无法编译。

参数	类型	说明
message	any	可选的位置参数错误消息

返回值

SyntaxError

内置模块 (BUILT-IN MODULES)

random

内置的随机数辅助函数。每次成功运行都会获得一个新的自动序列；如果需要可复现的序列，请使用 `random.seed(value)`。无需研究共享库 (Shared Library) 即可工作。

`random.random()`

大于等于 0 且小于 1 的随机浮点数。每次成功运行都会开始一个新的自动序列。导入 `random` 后也可使用 `random.random()`。

返回值

number

`random.rand()`

`random()` 的短别名：大于等于 0 且小于 1 的随机浮点数。每次成功运行都会开始一个新的自动序列。导入 `random` 后也可使用 `random.rand()`。

返回值

number

`random.randint(min, max, /)`

返回满足 `min <= N <= max` 的随机整数 `N`。边界包含在内，且必须是整数。每次成功运行都会开始一个新的自动序列。可与 `planet.get_bounds()` 搭配使用，以生成有效的随机坐标。导入 `random` 后也可使用 `random.randint(min, max)`。

返回值

number

random.seed(a=None)

初始化随机数生成。省略 **a** 或传入 **None**
可选择另一个自动序列。传入数字、布尔值或字符串，可使后续抽样具有可复现性；相同的值会产生相同的序列。

参数	类型	说明
a	any	可选的数字、布尔值、字符串或 None 种子

返回值

None

内置模块 (BUILT-IN MODULES)

functools

内置的函数式编程辅助函数。无需研究共享库 (Shared Library) 即可工作。

`functools.reduce(fn, iterable, initializer?, /)`

通过反复调用纯函数 `fn(total, item)`，将一个可迭代对象合并为一个值；回调不能暂停脚本或改变游戏状态。没有 `initializer` 时，第一个元素会成为初始 `total`；可迭代对象为空时会引发错误。带初始值的空可迭代对象，以及不带初始值的单元素可迭代对象，都不会调用回调。导入 `functools` 后也可使用 `from functools import reduce`。

返回值

any

`functools.total_ordering(cls, /)`

类装饰器 (class decorator)，通过 `__eq__` 加上 `__lt__`、`__le__`、`__gt__` 或 `__ge__` 中的一个，保留类对象并补齐缺少的排序方法。显式定义的方法不会被替换。应用该装饰器会改变局部类命名空间。

参数	类型	说明
<code>cls</code>	any	要补全的类

返回值

any

functools.wraps(wrapped, /)

返回一个装饰器，保留被包装可调用对象所支持的名称、限定名称、文档字符串、自定义属性以及 `__wrapped__` 链接，同时保持包装器的调用行为。应用返回的装饰器只会改变该局部包装器函数。

参数	类型	说明
<code>wrapped</code>	<code>any</code>	要复制其元数据的可调用对象

返回值

`any`

内置模块（BUILT-IN MODULES）

`re`

内置的正则表达式（regular expression）辅助函数。无需研究共享库（Shared Library）即可工作。模式使用安全的线性时间正则表达式子集。

`re.IGNORECASE`: number

不区分大小写的匹配标志。短别名：`re.I`。

返回值

number

`re.MULTILINE`: number

`^` 和 `$` 也会匹配行边界。短别名：`re.M`。

返回值

number

`re.DOTALL`: number

`.` 也会匹配换行字符。短别名：`re.S`。

返回值

number

`re.I`: number

`re.IGNORECASE` 的短别名。

返回值

number

re.M: number

re.MULTILINE 的短别名。

返回值

number

re.S: number

re.DOTALL 的短别名。

返回值

number

re.search(pattern, string, flags=0)

在 string 的任意位置搜索 pattern。返回 Match 对象；如果没有匹配则返回 None。模式使用文档规定的安全正则表达式子集。

参数	类型	说明
pattern	string	正则表达式模式
string	string	要搜索的字符串
flags	number	可选标志，例如 re.IGNORECASE

返回值

Optional[Match]

re.match(pattern, string, flags=0)

在 string 的开头匹配 pattern。返回 Match 对象；如果开头不匹配则返回 None。

参数	类型	说明
pattern	string	正则表达式模式
string	string	要检查的字符串
flags	number	可选标志，例如 re.IGNORECASE

返回值

Optional[Match]

re.fullmatch(pattern, string, flags=0)

将整个字符串与 pattern 进行匹配。如果有任何部分未匹配，则返回 None；否则返回 Match 对象。

参数	类型	说明
pattern	string	正则表达式模式
string	string	要检查的字符串
flags	number	可选标志，例如 re.IGNORECASE

返回值

Optional[Match]

re.findall(pattern, string, flags=0)

返回所有不重叠的匹配项。没有捕获组时，结果是匹配字符串列表；有一个捕获组时，结果是该组；有多个捕获组时，结果是元组列表。

参数	类型	说明
pattern	string	正则表达式模式
string	string	要搜索的字符串
flags	number	可选标志，例如 re.IGNORECASE

返回值

list

re.sub(pattern, repl, string, count=0, flags=0)

将 string 中与 pattern 匹配的内容替换为 repl。count=0 会替换全部匹配项；正数 count 会限制替换次数。替换文本支持 \1 和 \g<1> 这类数字反向引用。

参数	类型	说明
pattern	string	正则表达式模式
repl	string	替换文本
string	string	要转换的字符串
count	number	最大替换次数；0 表示全部替换
flags	number	可选标志，例如 re.IGNORECASE

返回值

string

`re.split(pattern, string, maxsplit=0, flags=0)`

在 `pattern` 匹配的位置拆分字符串。`maxsplit=0` 表示不限制次数。捕获组会包含在输出中，这与 Python 的 `re.split` 行为一致。

参数	类型	说明
<code>pattern</code>	string	正则表达式模式
<code>string</code>	string	要拆分的字符串
<code>maxsplit</code>	number	最大拆分次数；0 表示全部拆分
<code>flags</code>	number	可选标志，例如 <code>re.IGNORECASE</code>

返回值

list

内置模块 (BUILT-IN MODULES)

dataclasses

内置的记录类 (record-class) 辅助函数。无需研究共享库 (Shared Library) 即可工作。生成简洁的用户类值对象；对于 JSON 形状的记录，请使用 TypedDict。

```
dataclasses.dataclass(cls?, /, *, init=True, repr=True, eq=True, order=False,
kw_only=False, unsafe_hash=False, frozen=False, slots=False,
weakref_slot=False)
```

装饰一个类，以生成按声明顺序构造、表示、值相等比较以及可选排序功能。支持 @dataclass 和 @dataclass(...)、继承字段、__post_init__ 以及显式方法保留。生成的行为由 init、repr、eq、order 和 kw_only 控制。兼容性标志 unsafe_hash、frozen、slots 和 weakref_slot 只能传入 False；传入 True 的行为以及所有未列出的标准库选项都会被拒绝，而不是被忽略。数据类实例仍是普通用户对象，不能跨越 JSON 形状的游戏 API 边界。

参数	类型	说明
cls	any	可选类，用于函数式或裸装饰器用法
init	boolean	生成 __init__ (默认 True)
repr	boolean	生成 __repr__ (默认 True)
eq	boolean	生成精确类的 __eq__ (默认 True)
order	boolean	生成排序方法 (默认 False)
kw_only	boolean	使生成的构造函数数字段只能使用关键字 (默认 False)
unsafe_hash	boolean	兼容性标志；仅支持 False
frozen	boolean	兼容性标志；仅支持 False
slots	boolean	兼容性标志；仅支持 False
weakref_slot	boolean	兼容性标志；仅支持 False

返回值

any

```
dataclasses.field(*, default?, default_factory?, init=True, repr=True, compare=True, kw_only?)
```

配置一个带注解的数据类字段。对于不可变或可哈希的共享值，请使用 `default`；对于创建每个实例独立值的零参数工厂，请使用 `default_factory`；两者同时提供会出错。可变或其他不可哈希的直接默认值会被拒绝，必须使用 `default_factory`。`init` 控制是否包含在构造函数中，`repr` 控制生成的显示内容，`compare` 控制相等比较和排序，`kw_only` 控制该字段在构造函数中的位置。字段元数据/自省、`hash` 以及所有未列出的标准库选项均不支持。

参数	类型	说明
<code>default</code>	<code>any</code>	可选的不可变或可哈希共享值；可变或不可哈希值必须使用 <code>default_factory</code>
<code>default_factory</code>	<code>any</code>	可选的零参数工厂
<code>init</code>	<code>boolean</code>	将此字段包含在生成的构造函数中（默认 <code>True</code> ）
<code>repr</code>	<code>boolean</code>	将此字段包含在生成的表示中（默认 <code>True</code> ）
<code>compare</code>	<code>boolean</code>	将此字段包含在生成的相等比较和排序中（默认 <code>True</code> ）
<code>kw_only</code>	<code>boolean</code>	使此构造函数字段只能使用关键字

返回值

`any`

语言 / 基础 (LANGUAGE / BASICS)

Variables (变量)

存储供后续使用的值。不需要类型声明。

```
x = 10  
name = "oxygen"  
active = True
```


语言 / 基础 (LANGUAGE / BASICS)

Print (打印)

将文本输出到控制台。

```
print("Hello")  
print(42)  
print("Temp:", temp, "°C")
```

语言 / 基础 (LANGUAGE / BASICS)

Type Annotations (类型注解)

赋值和函数签名中的可选类型提示，采用标准 Python (PEP 526) 形式。`thermometer: Thermometer = get_component("thermometer")` 会在代码中内联记录变量类型；编辑器会使用该注解提供悬停提示、自动补全和 Cmd+单击导航。函数参数接受相同形式 (`def f(x: int):`)，还可以使用返回值注解 (`-> Site:`)。泛型形状也可使用：`xs: list[Site] = []`、`m: dict[str, int] = {}`、`b: Optional[Battery] = None`。注解可帮助编辑器理解代码。注解类型不会被求值，也不会运行时检查或转换值。结构上的例外是 `@dataclass`：带注解的类属性名称会成为字段，用于生成其构造函数和其他方法。

```
thermometer: Thermometer = get_component("thermometer")
xs: list[int] = []

def pick(sites: list[Site]) -> Site:
    return sites[0]
```

语言 / 基础

正则表达式 (Regular Expressions)

导入内置的 `re` 模块，用于模式匹配、捕获、替换和基于正则表达式的拆分。它无需进行共享库 (Shared Library) 研究即可工作，并且会从 `search`、`match` 和 `fullmatch` 返回 `Match` 对象。将它与原始字符串 (raw strings, `r"..."`) 搭配使用，这样就不必在模式中把每个反斜杠都写成双反斜杠。

```
import re

m = re.search(r"ore_(\d+)", "ore_42")
if m:
    print(m.group(1))
```

语言 / 基础

原始字符串 (Raw Strings)

给字符串加上 `r` 前缀，可以让反斜杠保持字面含义，而不是被当作转义符；这对正则表达式很方便，因为你不必把反斜杠写成双反斜杠。`r"\d+"` 是三个字符：反斜杠、`d`、`+`。可以将它与 `f` 结合成原始 `f` 字符串 (raw `f`-strings, `rf"...`)，此时 `{...}` 插值仍然有效，但反斜杠会保持字面含义。反斜杠仍然可以转义结束引号，因此原始字符串不能以奇数个反斜杠结尾。

```
import re

# r"..." keeps backslashes literal: no need to double them
m = re.search(r"ore_(\d+)", "ore_42")
print(m.group(1))

label = "iron"
print(rf"\d items of {label}")
```

语言 / 基础

位运算与集合运算 (Bitwise & Set Operators)

|、&、^ 运算符具有多态性。用于两个集合时，它们分别表示并集、交集和对称差 ($\{1, 2\} \mid \{2, 3\} \rightarrow \{1, 2, 3\}$)。用于两个数字时，它们分别表示按位 OR、AND、XOR ($0xF0 \& 0x33 \rightarrow 0x30$)。混合类型的操作数会引发错误。<</>> 移位以及一元 ~ 仅适用于整数。位运算使用 Python 风格的任意精度整数，并对失控增长的值设置安全上限；`1 << 61` 会打印 `2305843009213693952`，而不是四舍五入后的 JavaScript 数字。增强赋值形式 (`x |= 1`、`x <= 8`) 对这两类数据形状都有效。

```
flags = (1 << 8) | 0xFF
masked = flags & 0xF0
toggled = flags ^ 0b1010
```

```
a = {1, 2, 3}
b = {2, 3, 4}
print(a | b)    # {1, 2, 3, 4}
print(a & b)    # {2, 3}
```

语言 / 控制流

If / Else

根据条件做出决策。如果需要内联选择一个值，请使用 `x if condition else y`。

```
if temp > 0:
    print("Above freezing")
else:
    print("Below freezing")

status = "warm" if temp > 0 else "cold"
```

语言 / 控制流

While 循环 (While Loop)

当条件为真时重复执行代码。支持 **break**、**continue**，以及可选的 **else:** 代码块；当循环结束且没有执行 **break** 时，该代码块会运行。循环控制属于当前函数/无类循环体；嵌套的辅助函数或类不能跳出或继续调用者的循环。

```
i = 0
while i < 5:
    print(i)
    i = i + 1
```

语言 / 控制流

For 循环 (For Loop)

遍历列表、元组或范围。支持 `break`、`continue`、元组解包目标 (`for k, v in d.items():`)，以及可选的 `else:` 代码块；当循环结束且没有执行 `break` 时，该代码块会运行。循环控制属于当前函数/无类循环体；嵌套的辅助函数或类不能跳出或继续调用者的循环。

```
for i in range(5):  
    print(i)
```


语言 / 控制流

Match / Case

根据值的形状进行分支。支持字面量 `case`、`_` 通配符、变量捕获、`|` 或选项、`if` 守卫、带 `*rest` 的列表/元组模式，以及字典风格的模式。未加守卫的捕获或 `_` 始终匹配，并且必须是最后一个可达的 `case`；带守卫的 `case` 仍然可能继续落入后续分支。映射模式的键必须唯一（`True` 和 `1` 会被视为同一个键）。字典风格的模式也适用于固定结果对象，例如先执行 `result = self.input.take("iron_ore", 10)`，再使用 `case {"status": "ok", "moved": moved}:`。

```
result = self.input.take("iron_ore", 10)
match result:
    case {"status": "ok", "moved": moved}:
        print("moved", moved)
    case {"status": status, "message": message}:
        print(status, message)
```

语言 / 数据结构

列表与元组 (Lists & Tuples)

有序集合和固定的值对。

```
readings = [10, 20, 30]
point = (12, 8)
print(point[0])
readings.append(40)
```

语言 / 数据结构

字典 (Dictionaries)

用于命名数据的键值对。

```
planet = {"name": "Mars", "temp": -63}  
print(planet["name"])
```

语言 / 数据结构

TypedDict（记录形状）

对于小型的记录形状结构，可以使用 `TypedDict("Name", {...})`：例如控制器状态、路由记录、缓存的扫描行，或原本可能会使用类/装饰器来建模的简单数据对象。用它标注变量（`state: Name`）后，编辑器会自动补全键，并让每个字段的类型流经 `state["key"]` 和 `state.key`，还会在运行前标出拼写错误的键。在运行时，对于包含普通字符串键 `"mode"` 的字典，`state.mode` 与 `state["mode"]` 等价；内置字典方法名仍然会解析为方法。TypedDict 增加了稳定的声明形状，因此即使值流经其他代码，编辑器仍能自动补全并进行类型检查。

```
State = TypedDict("State", {
    "mode": str,
    "scans": int,
})

s: State = {"mode": "sonar", "scans": 0}
s.mode          # or s["mode"]: autocompletes; typos flagged
s["scans"] += 1
```

语言 / 函数

函数 (Functions)

定义可复用的代码块。参数可以有默认值 (`def f(x, n=10): ...`)，调用者既可以使用位置参数 (`f(5, 20)`)，也可以使用关键字参数 (`f(5, n=20)`)，与 Python 相同。在调用位置，所有位置参数都必须出现在任何关键字参数之前。可选的类型注解用于表达意图：`def scan(site: MiningSite) -> Site:`。它们可以帮助编辑器提供悬停提示和自动补全，但不会改变脚本的运行方式。

```
def double(x):  
    return x * 2  
  
print(double(5))
```

语言 / 函数

生成器 (Generators, `yield`)

包含 `yield` 的 `def` 是生成器：调用它会返回一个惰性迭代器，每次被请求时产生一个值，而不是预先构建完整列表。可以用 `for x in gen():` 遍历它，用 `next(it)` 取出一个值（耗尽时引发 `StopIteration`，或使用 `next(it, fallback)` 返回默认值），也可以用 `list(gen())` 将其物化。`yield from other()` 会重新发出另一个可迭代对象/生成器的每个值，并求值为该生成器的返回值。高级控制包括：`gen.send(v)` 用 `v` 恢复暂停的 `yield`，`gen.throw(error)` 在该 `yield` 处引发异常，以及 `gen.close()` 停止它（并运行任何 `finally`）。生成器是一次性的：耗尽后会一直为空。注意：裸生成器表达式 `((x for x in xs))` 现在也具有惰性，因此如果需要可复用的列表，请用 `list(...)` 包装它。生成器只存在于正在运行的脚本中，不能通过信号总线（Signal Bus）发送，也不能存储到数据归档（Data Archive）中；请先将其转换为 `list(...)`。

```
def route(points):
    for point in points:
        yield point[0], point[1]

# Values are produced one at a time, on demand:
waypoints = [(10, 4), (14, 9), (20, 12)]
for x, y in route(waypoints):
    print("next", x, y)

# yield from re-emits another iterable's values:
def full_route(near, far):
    yield from route(near)
    yield from route(far)
```

语言 / 类

类 (Classes)

将数据和行为组合在一起。`__init__(self, ...)` 会设置每个实例 (`self.count = 0`)，方法则将 `self` 作为第一个参数。在机器脚本中，顶层的 `self` 是机器；在方法内部，`self` 是对象；如果方法需要驱动机器，请将机器传入。编辑器会自动补全实例成员，并在调用链中传递方法的返回类型。

```
class Counter:
    def __init__(self, start):
        self.n = start
    def inc(self):
        self.n += 1

c = Counter(10)
c.inc()
print(c.n)
```

语言 / 类

@dataclass

从 `dataclasses` 导入 `dataclass` 和 `field`，可将带注解的类属性转换为按声明顺序排列的构造函数字段。`dataclass` 支持 `init`、`repr`、`eq`、`order` 和 `kw_only`；`field` 支持 `default`、`default_factory`、`init`、`repr`、`compare` 和 `kw_only`。可变或其他不可哈希的直接默认值会被拒绝；请使用 `default_factory` 为每个实例创建独立的值。生成的行为包括继承字段、`__init__`、`__repr__`、精确类相等性、可选排序和 `__post_init__`。兼容性拼写 `frozen=False`、`unsafe_hash=False`、`slots=False` 和 `weakref_slot=False` 会被接受，但不支持它们为 `True` 时的行为。不支持 `match_args` 以及 `ClassVar` / `InitVar` 字段语义；规范用法会收到清晰的错误。`MISSING`、`KW_ONLY`、`Field`、`FrozenInstanceError`、`is_dataclass`、`asdict`、`astuple`、`replace`、`fields`、`make_dataclass` 和公开的 `__dataclass_fields__` 自省均不可用；注解类型会保持擦除状态。数据类仍然是普通的用户类实例：当记录需要方法、验证、值相等性或排序时使用它。当值本质上是映射，或必须跨越类似 JSON 的边界（例如信号总线或笔记本 API）时，请使用 `TypedDict`。

```
from dataclasses import dataclass, field

@dataclass(order=True)
class Job:
    priority: int
    label: str = ""
    tags: list[str] = field(default_factory=list, compare=False)

job = Job(2, "ice run")
print(job)
```


语言 / 类

继承与 `super()` (Inheritance & `super()`)

`class Dog(Animal):` 会继承 `Animal` 的方法和属性。可以覆盖其中任意成员, 并使用 `super().method(...)` 调用基类版本。多重继承遵循 Python 的 C3 方法解析顺序 (MRO), 因此协作式 `super()` 可以在菱形继承中工作。`isinstance(x, Animal)` 和 `issubclass(Dog, Animal)` 都会沿继承链检查。

```
class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        return "... "

class Dog(Animal):
    def speak(self):
        return "woof"

print(Dog("Rex").speak())
```

语言 / 类

运算符与双下划线方法 (Operators & dunder methods)

定义运算符和内置函数在你的对象上的行为。__eq__ / __lt__ 支持 == 和 < ; __add__ (以及反向的 __radd__) 支持 + ; __len__、__bool__、__getitem__、__contains__、__iter__、__call__ 以及 __str__ / __repr__ 分别由 len()、真值判断、x[k]、in、for、调用和 print 调度。运算符重载必须是纯的, 不能调用 sleep() 或改变游戏状态。

```
class Vec:
    def __init__(self, x):
        self.x = x
    def __add__(self, o):
        return Vec(self.x + o.x)
    def __eq__(self, o):
        return self.x == o.x
    def __repr__(self):
        return "Vec(" + str(self.x) + ")"
```

```
print(Vec(1) + Vec(2))
print(Vec(3) == Vec(3))
```

语言 / 类

装饰器 (Decorators)

位于 `def` 或 `class` 上方的 `@deco` 会对其进行包装：`name = deco(name)`。可以叠加装饰器，也可以传入参数（`@deco(arg)`）。内置的方法装饰器包括：`@staticmethod`（没有 `self`）、`@classmethod`（接收类作为 `cls`）和 `@property`。标准的 `dataclasses` 模块提供了 `@dataclass`，用于生成记录类行为。

```
def trace(fn):
    def wrapper(*args):
        print("calling", fn)
        return fn(*args)
    return wrapper
```

```
@trace
def step(n):
    return n * 2
```

```
print(step(5))
```

语言 / 类

@property

@property 会将方法转换为无需括号即可读取的计算属性（`tank.level`，而不是 `tank.level()`）。添加 `@name.setter` 后，`tank.level = 42` 会执行验证。使用它可以在保留普通属性语法的同时，公开派生状态或受保护的状态。

```
class Tank:
    def __init__(self):
        self._level = 0
    @property
    def level(self):
        return self._level
    @level.setter
    def level(self, v):
        self._level = max(0, v)

t = Tank()
t.level = 42
print(t.level)
```

组件 / 核心系统

时钟 (Clock)

飞船的时钟。它跟踪一天中的时间、天数和太阳的位置，为脚本提供日夜计时和太阳跟踪所需的一切信息。

类型 基础设施 (Infrastructure)

返回方式：`get_component("clock")`

每个组件都有稳定的 `.id`。对于已部署的机器，请打开其卡片上的 ①，查找确切的 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性 (PROPERTIES)

`.id`

此组件稳定的程序化标识符。将它与 `get_component(id)` 以及要求组件、行星、载具、空间站或订单 ID 的 API 一起使用。

返回值

字符串 (String)

`.name`

面向用户的可读显示名称。对于需要经受重命名的脚本，优先使用 `.id`。

返回值

字符串 (String)

方法 (METHODS)

`.get_time()`

当前时间，返回 24 小时制的三元素列表 `[hours, minutes, seconds]`。使用 `t[0]`、`t[1]`、`t[2]` 进行索引。可用于按一天中的时间进行分支，或等待指定的小时。

返回值

列表 `[hours, minutes, seconds]`

`.get_day()`

当前天数，从 1

开始；当游戏内时钟越过午夜时递增。可用于每日预算逻辑（例如在每天开始时重置计数器），或检测热发生器（Heat Generator）等状态按天变化的机器的日期转换。

返回值

数字（天数）

`.get_time_of_day()`

当前日照阶段，字符串取值为 "dawn"、"day"、"dusk" 或

"night"。这是太阳/昼夜逻辑所使用的模拟阶段；页眉可能为了营造氛围，进一步将 "day" 标注为 Morning（早晨）/Afternoon（下午）/Evening（傍晚）。

返回值 可能的值

字符串：dawn / day / dusk / night "dawn"、"day"、"dusk"、"night"

`.get_elevation()`

太阳高于地平线的高度（0-90 度）。夜间为 0；在太阳正午（赤道处）升至 90 度；黄昏时回到 0。太阳能板在其倾角与当前高度互补时达到峰值。

返回值

数字（0-90 度）

`.tick()`

自保存开始以来的确定性模拟刻。在正常速度下，模拟每一刻都会提供新的脚本步骤预算（每秒 10 刻，因此一刻是 0.1 个模拟秒）。使用刻之间的差值来分析脚本计时，而不要使用墙上时钟毫秒数。

返回值

自保存开始以来的整数模拟刻。具有确定性；正常速度下的一刻对应 100 毫秒模拟时间。

`elapsed_seconds()`

保存开始后经过的模拟秒数。

返回值：

Number：保存开始后经过的模拟秒数。

`elapsed_game_hours()`

保存开始后经过的世界时钟小时数。昼夜循环会把 24 个世界时钟小时压缩到固定的实时窗口中，因此 `sleep(clock.real_seconds_per_hour())` 会等待恰好一个世界时钟小时；乘以 24 则会等待完整的一天。这样可以令速率计算以及遵循昼夜节奏的脚本使用行星感知的延迟，而不必硬编码换算值。

返回值：

Number：保存开始后经过的世界时钟小时数。

`real_seconds_per_hour()`

一世界时钟小时对应的实际秒数。乘以 24 即可等待完整的一天。

返回值：

Number：自保存开始以来经过的模拟秒数。

组件 / 核心系统 (COMPONENTS / CORE SYSTEMS)

Commander

使用 `get_component("me")` 或 `get_component("commander")` 读取玩家名称和当前 credits (信用点)。脚本无法修改这两个值。

返回自：`get_component("commander")`

每个组件都有稳定的 `.id`。对于已部署的机器，请打开其卡片上的 ①，查找确切 ID，然后将该值传递给 `get_component(id)`。ID 区分大小写。

属性 (PROPERTIES)

`.id`

此组件的稳定程序化标识符。将它与 `get_component(id)` 以及要求提供组件、行星、载具、空间站或订单 ID 的 API 一起使用。

返回值 (RETURNS)

String

`.name`

面向用户的可读显示名称。对于需要经受重命名的脚本，优先使用 `.id`。

返回值 (RETURNS)

String

方法 (METHODS)

`.get_name()`

以字符串返回你的指挥官名称。它在初始角色创建时设置 (或使用默认值)。可用于个性化仪表盘消息。

返回值 (RETURNS)

String

`.get_credits()`

返回当前 credits（信用点）余额。调用 `shop.buy()` / `shop.sell()`、生物交换（Bio Exchanges）结算、合约传输成功以及订单完成时，余额会发生变化。可在进行昂贵的 `shop.buy()` 调用前将其作为门槛检查。

返回值（RETURNS）

Number

组件 / 核心系统 (COMPONENTS / CORE SYSTEMS)

Console

将结构化脚本输出写入与 `print()` 使用的同一个 Console (控制台)。通过 `get_component("console")` 访问；无需研究即可使用。消息可以带有严重性、命名频道、颜色和时间戳。

返回自：`get_component("console")`

每个组件都有稳定的 `.id`。对于已部署的机器，请打开其卡片上的 ⓘ，查找确切 ID，然后将该值传递给 `get_component(id)`。ID 区分大小写。

属性 (PROPERTIES)

`.id`

此组件的稳定程序化标识符。将它与 `get_component(id)` 以及要求提供组件、行星、载具、空间站或订单 ID 的 API 一起使用。

返回值 (RETURNS)

String

`.name`

面向用户的可读显示名称。对于需要经受重命名的脚本，优先使用 `.id`。

返回值 (RETURNS)

String

方法 (METHODS)

```
.print(message, level="info", channel="", color="", timestamp=False)
```

打印一行并提供完整控制。`level` 可以是 `info/warn/error/debug` (这些级别会进入 WARNINGS / ERRORS 过滤器), 也可以是任何其他非空字符串, 用作以彩色徽章显示的自定义级别。空的 `level` 按 `info` 处理。`channel` 将该行路由到指定名称的标签页 (空字符串 = 主流)。`color` 可以是主题令牌 (`"warning"`、`"success"`、`"accent"`), 此时会随主题重新着色; 也可以是任意 CSS 颜色: 十六进制 (`"#aabbcc"`)、`"rgb(255,100,0)"`、`"hsl(30,100%,50%)"`, 或诸如 `"orange"` 的名称。将 `timestamp` 设为 `true` 会在前面加上游戏中的时刻。示例: `get_component("console").print("Overheat", "alert", "alarms", "warning", True)`。

参数 (PARAMETERS)

名称 (Name)		类型 (Type)	描述 (Description)
message		any	要打印的值。
level	string	严重性或自定义级别。内置值: <code>info/warn/error/debug</code> 。空字符串按 <code>info</code> 处理; 任何其他非空字符串都会作为带彩色徽章的自定义级别显示。	
channel		string	频道/标签页名称。空字符串 = 主流。
color	string	主题令牌 (会随主题重新着色), 或任意 CSS 颜色: 十六进制 (<code>"#aabbcc"</code>)、 <code>"rgb(255,100,0)"</code> 、 <code>"hsl(...)"</code> , 或诸如 <code>"orange"</code> 的名称。	
timestamp		boolean	在前面加上游戏中的时刻。

返回值 / 结果字段 / 成功载荷

返回值 (RETURNS)	结果字段 (RESULT FIELDS)	成功载荷 (SUCCESS PAYLOAD)
ActionResult	.status、.message	None

结果 (OUTCOMES)

状态 (Status)	类型 (Kind)	含义 (Meaning)
"ok"	success	操作成功完成。

```
.info(message, channel="", color="", timestamp=False)
```

打印一行 info（信息）级别的消息（默认级别）。可选的频道、颜色和时间戳参数行为与 `print` 中的对应参数相同。等价于 `print(message)`。

参数（PARAMETERS）

名称（Name）	类型（Type）	描述（Description）
message	any	要打印的值。
channel	string	频道/标签页名称。空字符串 = 主流。
color	string	主题令牌，或任意 CSS 颜色（hex、rgb()、hsl() 或命名颜色）。
timestamp	boolean	在前面加上游戏中的时刻。

返回值 / 结果字段 / 成功载荷

返回值（RETURNS）	结果字段（RESULT FIELDS）	成功载荷（SUCCESS PAYLOAD）
ActionResult	.status、.message	None

结果（OUTCOMES）

状态（Status）	类型（Kind）	含义（Meaning）
"ok"	success	操作成功完成。

```
.warn(message, channel="", color="", timestamp=False)
```

打印一行警告消息，该消息会出现在 Console（控制台）的 WARNINGS 过滤器中。可选的频道、颜色和时间戳参数控制路由和呈现方式。若要显示会打断操作的弹窗，请改用全局函数 `notify(text, "warn")`。

参数（PARAMETERS）

名称（Name）	类型（Type）	描述（Description）
message	any	要打印的值。
channel	string	频道/标签页名称。空字符串 = 主流。
color	string	主题令牌，或任意 CSS 颜色（hex、rgb()、hsl() 或命名颜色）。
timestamp	boolean	在前面加上游戏中的时刻。

返回值 / 结果字段 / 成功载荷

返回值（RETURNS）	结果字段（RESULT FIELDS）	成功载荷（SUCCESS PAYLOAD）
ActionResult	.status、.message	None

结果（OUTCOMES）

状态（Status）	类型（Kind）	含义（Meaning）
"ok"	success	操作成功完成。

```
.error(message, channel="", color="", timestamp=False)
```

打印一行错误消息，该消息会出现在 Console（控制台）的 ERRORS 过滤器中。这是你主动以错误严重性发送的消息，并非未捕获的异常。

参数（PARAMETERS）

名称（Name）	类型（Type）	描述（Description）
message	any	要打印的值。
channel	string	频道/标签页名称。空字符串 = 主流。
color	string	主题令牌，或任意 CSS 颜色（hex、rgb()、hsl() 或命名颜色）。
timestamp	boolean	在前面加上游戏中的时刻。

返回值 / 结果字段 / 成功载荷

返回值（RETURNS）	结果字段（RESULT FIELDS）	成功载荷（SUCCESS PAYLOAD）
ActionResult	.status、.message	None

结果（OUTCOMES）

状态（Status）	类型（Kind）	含义（Meaning）
"ok"	success	操作成功完成。

```
.debug(message, channel="", color="", timestamp=False)
```

打印一行低优先级调试消息。除非玩家启用调试输出，否则它会在 ALL 视图中隐藏。可选的频道、颜色和时间戳参数控制路由和呈现方式。

参数（PARAMETERS）

名称（Name）	类型（Type）	描述（Description）
message	any	要打印的值。
channel	string	频道/标签页名称。空字符串 = 主流。
color	string	主题令牌，或任意 CSS 颜色（hex、rgb()、hsl() 或命名颜色）。
timestamp	boolean	在前面加上游戏中的时刻。

返回值 / 结果字段 / 成功载荷

返回值（RETURNS）	结果字段（RESULT FIELDS）	成功载荷（SUCCESS PAYLOAD）
ActionResult	.status、.message	None

结果（OUTCOMES）

状态（Status）	类型（Kind）	含义（Meaning）
"ok"	success	操作成功完成。

`.now()`

返回当前游戏中的时刻，格式为 "HH:MM:SS" 字符串；当你希望完全控制格式时，可用它构建自定义行前缀。

返回值 (RETURNS)

String：游戏中的时刻，格式为 "HH:MM:SS"。

`.clear(channel="")`

清除此脚本产生的输出。提供 `channel` 参数时，只清除此脚本在该频道中的行；不提供参数时，清除此脚本的全部输出。其他脚本和系统消息会保留。

参数 (PARAMETERS)

名称 (Name)	类型 (Type)	描述 (Description)
<code>channel</code>	string	可选的此脚本频道；空字符串 = 此脚本的所有频道。

返回值 / 结果字段 / 成功载荷

返回值 (RETURNS)	结果字段 (RESULT FIELDS)	成功载荷 (SUCCESS PAYLOAD)
<code>ActionResult</code>	<code>.status</code> 、 <code>.message</code>	None

结果 (OUTCOMES)

状态 (Status)	类型 (Kind)	含义 (Meaning)
"ok"	success	操作成功完成。

组件 / 核心系统 (COMPONENTS / CORE SYSTEMS)

GPS

一种飞船传感器，用于报告你正在查看的前哨站、其名称和坐标，以及那里部署了多少座建筑。在仪表盘上切换前哨站会重新设定它的目标。

类型 (Type) : Infrastructure (基础设施)

返回自 : `get_component("gps")`

每个组件都有稳定的 `.id`。对于已部署的机器，请打开其卡片上的 ⓘ，查找确切 ID，然后将该值传递给 `get_component(id)`。ID 区分大小写。

属性 (PROPERTIES)

`.id`

此组件的稳定程序化标识符。将它与 `get_component(id)` 以及要求提供组件、行星、载具、空间站或订单 ID 的 API 一起使用。

返回值 (RETURNS)

String

`.name`

面向用户的可读显示名称。对于需要经受重命名的脚本，优先使用 `.id`。

返回值 (RETURNS)

String

方法 (METHODS)

`.planet()`

返回当前的 planet (行星) 组件。调用 Journal API 时, 使用 `gps.planet().id` 获取稳定 ID (例如 "nocturna"), 使用 `gps.planet().get_name()` 获取显示名称 "Nocturna"。

返回值 (RETURNS)

Nocturna planet (行星) 组件: 使用 `.id` 获取稳定的 journal (日志) ID, 使用 `.get_name()` 获取显示名称。

`.site_name()`

返回当前前哨站的显示名称。主基地 (默认) 的名称为 "Nocturna Base" (玩家可以重命名); 玩家建立的前哨站名称为 "Outpost 1"、"Outpost 2"……

返回值 (RETURNS)

String (当前前哨站名称, 例如 "Nocturna Base")

`.coords()`

返回当前前哨站的世界坐标, 格式为包含 2 个元素的列表 `[x, y]`。主基地位于 `[0, 0]`; 建立的前哨站使用玩家在 Plan mode (规划模式) 中选择的位置。

返回值 (RETURNS)

List `[x, y]` (当前前哨站坐标)

`.buildings_used()`

返回当前前哨站部署的建筑数量。传感器、移动单位、结构枢纽和 POI 提取机器不计入其中；只有从商店购买并可部署的建筑才计数。

返回值 (RETURNS)

Number (当前前哨站部署的建筑数量)

`.buildings_capacity()`

返回当前前哨站的软建筑阈值。每一座超过阈值的计数建筑都会降低生产和服务吞吐量。Nocturna Base 有几个额外的初始槽位；建立的前哨站使用标准阈值。

返回值 (RETURNS)

Number (当前前哨站的软建筑阈值)

`.is_full()`

当当前前哨站达到或超过其软建筑阈值时返回 True。阈值本身不会阻止普通部署。

返回值 (RETURNS)

Boolean (如果达到或超过软建筑阈值则为 true)

组件 / 核心系统 (COMPONENTS / CORE SYSTEMS)

Nocturna

通过 `get_component("nocturna")` 提供 Nocturna 的全行星稳定数据，包括地图边界、生物群系、地球化进度和永久地图联系点。隐藏的天气余波会被刻意排除；它们的坐标只存在于你的空间站网络捕获的风暴数据包中。声呐发现和勘测结果会单独存储在 `get_component("journal")` 中。

返回自：`get_component("nocturna")`

每个组件都有稳定的 `.id`。对于已部署的机器，请打开其卡片上的 ⓘ，查找确切 ID，然后将该值传递给 `get_component(id)`。ID 区分大小写。

属性 (PROPERTIES)

`.id`

此组件的稳定程序化标识符。将它与 `get_component(id)` 以及要求提供组件、行星、载具、空间站或订单 ID 的 API 一起使用。

返回值 (RETURNS)

String

`.name`

面向用户的可读显示名称。对于需要经受重命名的脚本，优先使用 `.id`。

返回值 (RETURNS)

String

方法 (METHODS)

`.get_name()`

返回此行星的显示名称 "Nocturna"。可以安全地硬编码，但查询它能让脚本面向未来，适应重命名或多行星扩展。

返回值 (RETURNS)

String (显示名称)

.get_bounds()

返回一个 Bounds（边界）对象，其中包含 `.min_x`、`.max_x`、`.min_y`、`.max_y`，全部以距基地的米数表示。可用它将导航目标限制在作业表面内，或生成有效的随机坐标：`import random; b = planet.get_bounds(); x = random.randint(b.min_x, b.max_x); y = random.randint(b.min_y, b.max_y)`。参见 Bounds。

返回值（RETURNS）

Bounds 对象，包含 `.min_x`、`.max_x`、`.min_y`、`.max_y`（米）

.contains(x, y)

如果点 (x, y) 位于 Nocturna 的表面边界内，则返回 True。在调用 `nav.set_target()` 前可将其作为安全检查：`if nocturna.contains(x, y): self.nav.set_target(x, y)`。目标位于边界之外时会立即返回，不会驱动。

参数（PARAMETERS）

名称（Name）	类型（Type）	描述（Description）
x	number	以米为单位的 X 坐标。
y	number	以米为单位的 Y 坐标。

返回值（RETURNS）

Boolean：如果该点位于行星表面边界内则为 True。

`.biome_at(x, y)`

返回世界坐标 (x, y) 处的 biome id (生物群系 ID)。返回值之一为 "frozen"、"coastal"、"geothermal"、"volcanic"、"deep"。这是纯地理信息；同一坐标永远返回相同结果。可用于在行星任意位置编写感知生物群系的脚本：`if nocturna.biome_at(x, y) == "volcanic": deploy_heat_resistant()`。

参数 (PARAMETERS)

名称 (Name)	类型 (Type)	描述 (Description)
x	number	世界 X 坐标。
y	number	世界 Y 坐标。

返回值 / 可能值

生物群系 ID：以下值之一："frozen"、"coastal"、"geothermal"、"volcanic"、"deep"。

`.life_form_biome(item_id)`

查找生命形态物品的原生生物群系："frozen"、"coastal"、"geothermal"、"volcanic" 或 "deep"。其他物品返回 None。同一物种始终属于同一生物群系，与发现该标本的位置无关。Essence Liquifier (精华液化器) 只接受原生于其前哨站的生命形态，因此无人机可以在卸载前测试 `nocturna.life_form_biome(item) == self.outpost.biome`。

参数 (PARAMETERS)

名称 (Name)	类型 (Type)	描述 (Description)
item_id	string	生命形态物品 ID，例如 "vent_fungus"。

返回值 / 可能值

生命形态的原生生物群系 ID；如果是其他物品类型则为 None。该值按物种固定，并与其 Essence Liquifier 接受的生物群系相匹配。

`.biomes()`

列出行星上的每个生物群系 ID。可用于行星范围的循环：`for biome in nocturna.biomes(): print(biome)`。在游戏过程中，这个集合不会改变。

返回值 (RETURNS)

行星上每个生物群系 ID 的列表。

`.terraform_progress()`

返回 0 – 100% 范围的 Terraform Index（地球化指数）进度：0 表示尚未开始；100 表示达到 1,000,000 TP 且六个支柱全部完成。大气占指数的 70%；生物量、植物和野生动物分别占 10%，彼此不能替代。可用它控制进度逻辑：`if nocturna.terraform_progress() >= 50: ...`。要获取原始 TP 值，请使用 `total_tp()`。

返回值 (RETURNS)

Number：Terraform Index 进度百分比（0 – 100）。

`.total_tp()`

返回以 TP 表示的有限 Terraform Index，范围为 0 – 1,000,000。温度、氧气和压力合计占 700,000 TP；生物量、植物和野生动物分别占不可替代的 100,000 TP。原始指标在达到最终阶段后仍可能增长，但已完成的支柱不会为指数贡献额外值。该值用于控制跨系统研究。

返回值 (RETURNS)

Number：有限的 Terraform Index 点数（0 – 1,000,000），供跨系统研究使用。

`.points_of_interest()`

列出 Planet Map（行星地图）上的每个永久 "?" 联系点，使脚本能够导航到真实地点。每个 PointOfInterest（兴趣点）都有整数坐标、scanned 标记和 kind。扫描器到达联系点前，kind 保持为 "unknown"；扫描后会显示 mineral、biomass、thermal、water、oil、exotic 或 inert。筛选 `not point.scanned`，前往其坐标，然后使用 Rover 或 Pioneer sonar，或无人机 Bio Scanner 进行扫描。参见 PointOfInterest。

返回值 (RETURNS)

PointOfInterest 记录列表：所有实体地图联系点（"?" 标记），每条记录都有 `.x / .y / .scanned`。扫描联系点前，`.kind` 读取为 "unknown"。

组件 / 核心系统 (COMPONENTS / CORE SYSTEMS)

Research

通过 `get_component("research")` 检查全局研究进度。使用 Research（研究）页面上显示的公共 ID，例如 `"research_auto_feeders"`。从一开始即可用，并且是只读的。

返回自：`get_component("research")`

每个组件都有稳定的 `.id`。对于已部署的机器，请打开其卡片上的 ⓘ，查找确切 ID，然后将该值传递给 `get_component(id)`。ID 区分大小写。

属性 (PROPERTIES)

`.id`

此组件的稳定程序化标识符。将它与 `get_component(id)` 以及要求提供组件、行星、载具、空间站或订单 ID 的 API 一起使用。

返回值 (RETURNS)

String

`.name`

面向用户的可读显示名称。对于需要经受重命名的脚本，优先使用 `.id`。

返回值 (RETURNS)

String

方法 (METHODS)

`.is_unlocked(research_id)`

仅当 `research_id` 已知且已解锁时返回 `True`。已知但锁定的研究返回 `False`。未知 ID 也会返回 `False`，且不会打印消息，因此脚本可以自行处理每个查询结果。

参数 (PARAMETERS)

名称 (Name)	类型 (Type)	描述 (Description)
<code>research_id</code>	<code>string</code>	公共研究 ID，例如 <code>research_auto_feeders</code> 。

返回值 (RETURNS)

`Boolean`。仅当已知研究已解锁并且在当前版本中可用时为 `True`。

`.unlocked()`

返回当前版本中已解锁且可用的公共研究 ID 的新列表。列表遵循稳定的 Research 页面注册表顺序，不包含内部能力 ID；修改该列表不会改变游戏状态。

返回值 (RETURNS)

公共研究 ID 列表，按稳定的研究注册表顺序排列，且这些研究已解锁并可用。

组件 / 核心系统 (COMPONENTS / CORE SYSTEMS)

Shop

与 Earth (地球) 进行买卖。使用 `get_component("shop")`, 在达到阈值时自动出售盈余或购买物品。Shop UI (商店界面) 使用相同的目录和价格。

返回自: `get_component("shop")`

每个组件都有稳定的 `.id`。对于已部署的机器, 请打开其卡片上的 ⓘ, 查找确切 ID, 然后将该值传递给 `get_component(id)`。ID 区分大小写。

属性 (PROPERTIES)

`.id`

此组件的稳定程序化标识符。将它与 `get_component(id)` 以及要求提供组件、行星、载具、空间站或订单 ID 的 API 一起使用。

返回值 (RETURNS)

String

`.name`

面向用户的可读显示名称。对于需要经受重命名的脚本, 优先使用 `.id`。

返回值 (RETURNS)

String

方法 (METHODS)

`.sell(item_id, quantity=1)`

出售 `item_id` 的正整数数量，默认为 1。完整数量会在一次事务中从索引最低的匹配 Inventory（库存）槽位移除；如果 Inventory 中的单位少于请求数量，则不会出售任何东西。电池类产品会按其充电百分比退款，最低为 50%；充满电的电池会退还其完整的正常价值。需要选择某个确切的电池实例时，请使用 Inventory 页面。

参数 (PARAMETERS)

名称 (Name)	类型 (Type)	描述 (Description)
<code>item_id</code>	string	要出售的物品 ID。
<code>quantity</code>	number	要出售的正整数单位数量；默认为 1。

返回值 / 结果字段 / 成功载荷

返回值 (RETURNS)	结果字段 (RESULT FIELDS)	成功载荷 (SUCCESS PAYLOAD)
<code>SaleResult</code>	<code>.status</code> 、 <code>.message</code>	<code>.item_id</code> 、 <code>.units</code> 、 <code>.credits</code>

结果 (OUTCOMES)

状态 (Status)	类型 (Kind)	含义 (Meaning)
<code>"ok"</code>	success	出售完成： $.item_id \times .units$ ；收到 <code>.credits</code> cr。
<code>"not_sellable"</code>	rejection	请求的物品不可出售。
<code>"no_stock"</code>	rejection	Inventory 中没有请求数量的物品。

引发异常 (RAISES)

异常 (Exception)	条件 (Condition)
<code>TypeError</code>	<code>quantity</code> 必须是有限的整数。
<code>ValueError</code>	<code>quantity</code> 必须大于零。
<code>OverflowError</code>	<code>quantity</code> 必须处于支持的整数范围内。

`.sell_all(item_id)`

在一次交易中出售库存中当前属于 `item_id` 的所有单位。每单位没有单独的冷却时间。每种电池产品都根据其自身保留的电量计价：最低为 50% 的价值，满电时为完整的正常价值。

参数

名称	类型	说明
<code>item_id</code>	<code>string</code>	要出售其所有堆叠的物品 ID

返回值

返回类型	结果字段	成功载荷
<code>SaleResult</code>	<code>.status</code> 、 <code>.message</code>	<code>.item_id</code> 、 <code>.units</code> 、 <code>.credits</code>

结果

状态	类型	含义
<code>"ok"</code>	成功	出售完成： $.item_id \times .units$ ；收到 <code>.credits</code> cr。
<code>"not_sellable"</code>	拒绝	请求的对象不可出售。
<code>"no_stock"</code>	拒绝	库存中没有请求数量的物品。

`.buy(item_id, quantity=1)`

购买 `item_id` 的正整数数量，默认值为 1。请求的完整数量必须负担得起，并且必须能装入基础库存；否则不会扣款，也不会交付任何物品。购买物品会放入基础库存，而不是直接交付给机器或远程前哨站。

参数

名称	类型	说明
<code>item_id</code>	<code>string</code>	商店物品 ID（机器套件、模块、包、试剂或设备）
<code>quantity</code>	<code>number</code>	要购买的正整数单位数；默认为 1

返回值

返回类型	结果字段	成功载荷
<code>ActionResult</code>	<code>.status</code> 、 <code>.message</code>	<code>None</code>

结果

状态	类型	含义
<code>"ok"</code>	成功	操作成功完成。
<code>"not_found"</code>	拒绝	请求的对象、目标或记录不存在。
<code>"locked"</code>	拒绝	所需的功能、配方或操作尚未解锁。
<code>"insufficient_credits"</code>	拒绝	可用积分低于请求购买的总成本。
<code>"inventory_full"</code>	拒绝	库存没有空间容纳结果。

引发异常

异常	条件
<code>TypeError</code>	<code>quantity</code> 必须是有限的整数。
<code>ValueError</code>	<code>quantity</code> 必须大于零。
<code>OverflowError</code>	<code>quantity</code> 必须处于所支持的整数范围内。

`.get_catalogue()`

返回所有可用的目录条目，形式为 `{id, name, cost}` 对象列表。可用它动态选择目标，或在脚本中显示经过筛选的选择器。地球商店的目录物品永远不会售罄；被科技门禁隐藏的条目不会出现。

返回值

List of `{id, name, cost}`

组件 / 核心系统

发射器 (Transmitter)

向其他行星发送数据。使用它向地球报告传感器读数，或提交合约答案。调用 `connect()` 选择行星，然后调用 `transmit(key, value)` 发送数据；`disconnect()` 会清除连接。连接只在当前脚本运行期间有效，因此每个发送脚本都必须先连接。将 `get_component("transmitter")` 的结果保存到变量中，并在两个调用中重复使用。

返回自：`get_component("transmitter")`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，即可找到确切 ID，然后将该值传递给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件的稳定程序化标识符。将它与 `get_component(id)` 以及要求组件、行星、载具、空间站或订单 ID 的 API 一起使用。

返回值

String

`.name`

面向用户的可读显示名称。对于需要经受重命名仍能工作的脚本，优先使用 `.id`。

返回值

String

方法

`.list_planets()`

返回所有可用的传输目的地，形式为 **Planet** 对象列表（每个对象都有 `.id`、`.name` 等属性）。在脚本开始时调用一次，以查看可用目标；将返回对象的 `.id` 传递给 `connect(id)`。

返回值

List of Planet objects

.connect(planet)

打开到指定 ID 行星的通道：`result = transmitter.connect("earth")`。该 ID 必须是小写（来自 `list_planets()`）。成功后读取 `transmitter.get_info().target`。连接只在当前脚本运行期间有效；如果脚本重新启动，请在 `transmit()` 之前再次调用 `connect()`。

参数

名称	类型	说明
<code>planet</code>	<code>string</code>	要连接的行星 ID

返回值

返回类型	结果字段	成功载荷
<code>ActionResult</code>	<code>.status</code> 、 <code>.message</code>	<code>None</code>

结果

状态	类型	含义
<code>"ok"</code>	成功	操作成功完成。
<code>"not_found"</code>	拒绝	请求的对象、目标或记录不存在。

.disconnect()

关闭当前脚本运行期间的通道。这不会影响合约或任何其他脚本；它只会清除这个发射器对象的活动目标，因此之后调用 `transmit()` 时必须重新连接。

返回值

返回类型	结果字段	成功载荷
<code>ActionResult</code>	<code>.status</code> 、 <code>.message</code>	<code>None</code>

结果

状态	类型	含义
<code>"ok"</code>	成功	发射器当前脚本运行期间的通道已断开。

.get_info()

当前连接状态。返回一个包含 `.connected`（布尔值）和 `.target`（已连接的行星 ID，或 `"none"`）的对象。在 `transmit()` 之前将其作为保护条件使用：如果 `transmitter.get_info().connected`，则调用 `transmitter.transmit(...)`。

返回值

`Object { connected, target }`

.transmit(key, value)

使用 `transmitter.transmit(key, value)` 向已连接的行星发送命名值。只有在完成电力和传感器引导步骤并且上行链路步骤处于活动状态后，才能开启传感器遥测。对于传感器读数，使用地球要求的名称，例如 `"current_temperature"`。对于合约答案，在合约脚本中使用 `self.contract.id`。数据会在同一个 tick 中到达。

参数

名称	类型	说明
key	string	传感器遥测键，例如 <code>"current_temperature"</code> （开场上行链路要求的键），或合约脚本中从 <code>self.contract.id</code> 读取的合约 ID
value	any	要发送的读数或计算结果

返回值

返回类型	结果字段	成功载荷
ActionResult	.status、.message	None

结果

状态	类型	含义
"correct"	成功	提交的值正确。
"incorrect"	拒绝	提交的值不正确。
"already_completed_correct"	成功	合约已经完成；这次提交的值正确。
"already_completed_incorrect"	拒绝	合约已经完成；这次提交的值不正确。
"accepted"	成功	提交的值已被接受。
"rejected"	拒绝	提交的值或请求被拒绝。
"not_connected"	拒绝	组件没有活动连接。
"wrong_planet"	拒绝	操作针对的是另一颗行星。
"wrong_contract"	拒绝	提供的合约不是此操作的活动合约。
"locked"	拒绝	所需的功能、配方或操作尚未解锁。
"unknown_contract"	拒绝	提供的合约标识符不存在。
"key_is_planet"	拒绝	提供的键命名的是行星，而不是遥测键或合约键。
"unknown_key"	拒绝	提供的遥测键或合约键不存在。

组件 / 传感器

生物量传感器 (Biomass Sensor)

报告行星上培育的生物量 (biomass)，单位为吨。随着生物量混合器 (Biomass Mixer) 生产而实时更新。在生物圈研究 (Biosphere research) 抵达后可用；无需校准步骤。

返回自：`get_component("biomass_sensor")`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，即可找到确切 ID，然后将该值传递给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件的稳定程序化标识符。将它与 `get_component(id)` 以及要求组件、行星、载具、空间站或订单 ID 的 API 一起使用。

返回值

String

`.name`

面向用户的可读显示名称。对于需要经受重命名仍能工作的脚本，优先使用 `.id`。

返回值

String

方法

`.get_value()`

以数字形式返回行星上的生物量总吨数。在任何生物量混合器生产之前为 0。

返回值

Number (tons of biomass)


```
component = get_component("biomass_sensor")
value = component.get_value()
print(value)
```

组件 / 传感器

氧气传感器（Oxygen Sensor）

一个损坏后着陆的大气氧气探测器。在脚本计算出校准值并修复它之前，它会报告原始电压；修复后，它会直接读取氧气含量。

类型	传感器
----	-----

返回自：`get_component("oxygen_sensor")`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，即可找到确切 ID，然后将该值传递给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件的稳定程序化标识符。将它与 `get_component(id)` 以及要求组件、行星、载具、空间站或订单 ID 的 API 一起使用。

返回值

String

`.name`

面向用户的可读显示名称。对于需要经受重命名仍能工作的脚本，优先使用 `.id`。

返回值

String

方法

`.get_value()`

修复前，从未校准的探头读取原始电压，结果为一个较小的小数值。这还不是 ppt 读数；将它与已知参考值比较，以计算校准因子。修复后，直接读取当前大气氧气含量，单位为 ppt（parts per thousand，千分比）。

返回值

Number (raw voltage before repair; atmospheric oxygen in ppt after repair)

`.calibrate(value)`

使用处理后的值启动黑盒校准套件（black-box calibration suite）：`result = self.calibrate(raw_value * factor)`。随后，该套件会根据多个读数检查整个脚本。完全正确的套件会修复传感器；失败的测试用例会继续显示在控制台中。

参数

名称	类型	说明
value	number	要测试的校准值

返回值

返回类型	结果字段	成功载荷
ActionResult	.status、.message	None

结果

状态	类型	含义
"started"	成功	操作已启动。
"already_repaired"	成功	目标已修复。
"no_source"	拒绝	没有配置或可用的源。
"already_testing"	暂态	请求的测试已经在运行。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` 仅自身

读取脚本运行期间从编辑器“命令”选项卡发送的命令。所有可编程机器都相同，参见指南：脚本命令。

```
component = get_component("oxygen_sensor")
value = component.get_value()
print(value)
```

组件 / 传感器

植物传感器 (Plants Sensor)

报告植物地球化改造机 (Plant Terraformer) 机群产生的永久植被面积, 单位为平方千米。在生物圈研究 (Biosphere research) 抵达后可用。单纯的野外生长不会改变该数值。

返回自: `get_component("plants_sensor")`

每个组件都有稳定的 `.id`。对于已部署的机器, 打开其卡片上的 ①, 即可找到确切 ID, 然后将该值传递给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件的稳定程序化标识符。将它与 `get_component(id)` 以及要求组件、行星、载具、空间站或订单 ID 的 API 一起使用。

返回值

String

`.name`

面向用户的可读显示名称。对于需要经受重命名仍能工作的脚本, 优先使用 `.id`。

返回值

String

方法

`.get_value()`

以数字形式返回永久植物面积 (平方千米)。植物地球化改造机 (Plant Terraformer) 是唯一的写入者; 反复的作物周期、多样性、供应商、肥料 (Fertilizer) 和产量放大器 (Yield Amplifier) 会增加它们所处理的实际饲料供应量。

返回值

Number (km² planted)

```
component = get_component("plants_sensor")
value = component.get_value()
print(value)
```

压力传感器 (Pressure Sensor)

一个已经损坏的气压探头。其读数在脚本稳定修复信号之前会以乱码形式输出；修复后，它会直接读取气压。

类型：传感器 (Sensors)

返回方式：`get_component("pressure_sensor")`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ① 以找到确切 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性 (PROPERTIES)

`.id`

该组件的稳定程序化标识符。将它与 `get_component(id)` 以及那些要求提供组件、行星、载具、空间站或订单 ID 的 API 配合使用。

返回值 (RETURNS)：String

`.name`

面向用户的可读显示名称。对于需要在重命名后仍能运行的脚本，优先使用 `.id`。

返回值 (RETURNS)：String

方法 (METHODS)

.get_value()

当前不稳定的修复读数，类型为整数。如果该值为奇数，加 1；如果为偶数，则保持不变。修复完成后，此方法会返回真实的大气压力，单位为 kPa。

返回值 (RETURNS) : Number (修复读数不稳定；修复后单位为 kPa)

.stabilize(value)

使用修正后的偶数读数启动黑盒稳定测试套件：`result = self.stabilize(corrected_value)`。随后，测试套件会用多个读数检查整个脚本。测试套件完全正确时会修复传感器；失败的测试用例仍会显示在控制台中。

参数 (PARAMETERS)

名称	类型	描述
value	number	要测试的稳定化数值

返回值 (RETURNS) : ActionResult

结果字段 (RESULT FIELDS) : .status、.message

成功载荷 (SUCCESS PAYLOAD) : None

结果 (OUTCOMES)

状态	类型	含义
"started"	成功	操作已开始。
"already_repaired"	成功	目标已经修复。
"no_source"	拒绝	未配置或无法获得源代码。
"already_testing"	暂态	请求的测试已经在运行。

.peek_command() · .next_command() · .command_count() · .clear_commands() — 仅限自身 (SELF ONLY)

读取脚本运行时从编辑器的 Commands（命令）选项卡发送的命令。所有可编程机器上的行为都相同，参见指南：Script Commands。

```
component = get_component("pressure_sensor")
value = component.get_value()
print(value)
```


组件 / 传感器

温度计 (Thermometer)

始终正常工作的表面温度探头，无需校准。直接读取行星当前表面温度，单位为 °C。

返回方式：`get_component("thermometer")`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ① 以找到确切 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性 (PROPERTIES)

`.id`

该组件的稳定程序化标识符。将它与 `get_component(id)` 以及那些要求提供组件、行星、载具、空间站或订单 ID 的 API 配合使用。

返回值 (RETURNS) : `String`

`.name`

面向用户的可读显示名称。对于需要在重命名后仍能运行的脚本，优先使用 `.id`。

返回值 (RETURNS) : `String`

方法 (METHODS)

`.get_value()`

当前表面温度，单位为 °C，返回数值。可以从任何脚本安全调用；无需修复步骤。这是显示用的 °C。对于研究阈值比较的热量单位进度指标，请读取 `get_component("atmosphere").get_heat()`。

返回值 (RETURNS) : `Number` (°C)

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` — 仅限自身 (SELF ONLY)

读取脚本运行时从编辑器的 Commands（命令）选项卡发送的命令。所有可编程机器上的行为都相同，参见指南：Script Commands。

```
component = get_component("thermometer")
value = component.get_value()
print(value)
```

组件 / 传感器

野生动物传感器 (Wildlife Sensor)

报告所有已建立栖息地殖民地中的动物个体总数。在生物圈 (Biosphere) 研究落地后可用；在第一个野生动物殖民地建立之前会读取为 0。

返回方式：`get_component("wildlife_sensor")`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ① 以找到确切 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性 (PROPERTIES)

`.id`

该组件的稳定程序化标识符。将它与 `get_component(id)` 以及那些要求提供组件、行星、载具、空间站或订单 ID 的 API 配合使用。

返回值 (RETURNS)：`String`

`.name`

面向用户的可读显示名称。对于需要在重命名后仍能运行的脚本，优先使用 `.id`。

返回值 (RETURNS)：`String`

方法 (METHODS)

`.get_value()`

返回当前野生动物种群数量，即从已建立的栖息地殖民地汇总得到的数值。

返回值 (RETURNS)：`Number` (个体数)

```
component = get_component("wildlife_sensor")
value = component.get_value()
print(value)
```

组件 / 探索

日志 (Journal)

存储声呐发现或测量的地点，以及生物实验室编目的碎片。使用 `get_component("journal")` 访问它，无需再次扫描即可规划旅行和生物订单。记录按行星分开保存，并且会跨越脚本重启、载具更换以及保存/加载而保留。

返回方式：`get_component("journal")`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ① 以找到确切 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性 (PROPERTIES)

`.id`

该组件的稳定程序化标识符。将它与 `get_component(id)` 以及那些要求提供组件、行星、载具、空间站或订单 ID 的 API 配合使用。

返回值 (RETURNS) : `String`

`.name`

面向用户的可读显示名称。对于需要在重命名后仍能运行的脚本，优先使用 `.id`。

返回值 (RETURNS) : `String`

方法 (METHODS)

`.discovered_sites(planet_id)`

列出行星 `planet_id` 上所有经声呐分类的地点。对于 Nocturna, 请调用 `journal.discovered_sites("nocturna")`。每个条目根据 `kind()` 可能是 `MiningSite`、`ThermalVent`、`WaterWell`、`OilWell`、`ExoticDeposit` 或 `GeologicalAnomaly`。未测量的可产出地点会将其详细字段保留为 `None`；惰性地层会在扫描时解析。重复扫描不会添加重复条目。尚未发现地点时返回空列表。参见 `Site`。

参数 (PARAMETERS)

名称	类型	描述
<code>planet_id</code>	<code>string</code>	日志中的行星 ID；当前行星通常为 <code>"nocturna"</code> 。

返回值 (RETURNS)：该行星的已分类 `Site` 值列表。每种具体类型都遵循 `kind()`。未测量的可产出地点会将详细字段保留为 `None`；惰性地层会在扫描时解析。再次查询可获得当前的测量、周期和容量数据。

`.surveyed_sites(planet_id)`

以 `list<Site>` 列出行星 `planet_id` 上已完全解析的每个地点；形状与 `discovered_sites()` 相同，但筛选条件为 `surveyed == True`。其中包括声呐无需第二次测量即可解析的惰性 `GeologicalAnomaly` 接触点。根据 `kind()` 分支来访问字段：`MiningSite` 提供 `.item_id`、`.hardness`、`.purity`；`ThermalVent` 提供 `phase / rate / cycle timing`（由声呐等级控制）；`WaterWell / OilWell` 提供 `.yield_tier`、`.flow_rate`。参见 `Site`。

参数 (PARAMETERS)

名称	类型	描述
<code>planet_id</code>	<code>string</code>	日志中的行星 ID；当前行星通常为 <code>"nocturna"</code> 。

返回值 (RETURNS)：完全解析的 `Site` 快照列表，包括通过扫描解析的惰性地层。使用与 `discovered_sites()` 相同的类型，并筛选 `surveyed == True`。再次查询可获得当前周期和容量数据。

`.cataloged_fragments(planet_id)`

列出在行星 `planet_id` 的生物实验室分析过的碎片，按最新在前排列。每个 `CatalogedFragment` 包含稳定的碎片 ID、显示名称、生物群系、坐标和稀有度。将 `entry.fragment_id` 与 `BioOrder.requires` 匹配，并将 `entry.coords` 传给 `bio_collector.collect(...)`。未分析的碎片和生物身份仍然隐藏。五个碎片全部编目后，完成的生物会出现在 `journal.cataloged_creatures(planet_id)` 中。对其他行星返回空列表。

参数 (PARAMETERS)

名称	类型	描述
<code>planet_id</code>	<code>string</code>	日志中的行星 ID；当前行星通常为 <code>"nocturna"</code> 。

返回值 (RETURNS) : `CatalogedFragment` 列表：该行星生物实验室分析过的每个碎片，包括稳定的 `.fragment_id` 和面向玩家的 `.name`。按最近编目时间倒序排列。对于当前行星以外的任何行星返回 `[]`。

`.cataloged_creatures(planet_id)`

列出在行星 `planet_id` 上五个碎片均已分析的生物，最近完成的排在最前。每个 `CatalogedCreature` 提供稳定的生物 ID、其五个碎片 ID、所需的喂食物品和 Feed Maker 配方、最低启动喂食量，以及按确切稀有度缩放的复苏试剂。使用 `.creature_id` 配合 `habitat.set_revival_target(...)`。使用 `.feed_recipe_id` 在 `feed_maker.list_recipes()` 中找到对应的已解锁 Recipe；配方成分仍由该 Recipe 持有。对其他行星返回空列表。

参数 (PARAMETERS)

名称	类型	描述
<code>planet_id</code>	<code>string</code>	日志中的行星 ID；当前行星通常为 "nocturna"。

返回值 (RETURNS)： `CatalogedCreature` 列表：该行星上五个碎片均已分析的每个生物，以及通用 Habitat 复苏脚本所需的 ID 和确切启动补给。按最近完成时间倒序排列。对于当前行星以外的任何行星返回 `[]`。

`.coord_info(x, y)`

读取已发现的永久生物站点坐标的保存的 `LifeFormScanResult`。对于未触及的生物站点以及并非地点的已扫描坐标，返回 `None`。查询会立即返回。

参数 (PARAMETERS)

名称	类型	描述
<code>x</code>	<code>number</code>	整数世界 <code>x</code> 坐标
<code>y</code>	<code>number</code>	整数世界 <code>y</code> 坐标

返回值 (RETURNS)：如果永久生物站点 (`x, y`) 已扫描，则返回其 `LifeFormScanResult`。如果未触及或不是生物站点，则返回 `None`。开销低；不会产生收益。

`.biomass_coords()`

列出每个已发现的永久生物站点，返回 `LifeFormScanResult`。这是收割无人机可安全跨重启使用的路线源：检查 `.coord`、每个样本的 `.remaining_tons`，并在派遣前检查 `is_ready(x, y)`。

返回值 (RETURNS)：所有已发现永久生物站点的 `LifeFormScanResult` 快照列表。使用每项的 `.coord`、`.life_forms` 和 `.remaining_tons` 安排无人机收割路线。

`.has_scanned(x, y)`

整数坐标 (x, y) 被扫描过后为 `True`。当路线跨脚本重启恢复时，可用它跳过已经访问的生物站点。

参数 (PARAMETERS)

名称	类型	描述
x	number	整数世界 x 坐标
y	number	整数世界 y 坐标

返回值 (RETURNS) : `Boolean` : 如果该整数坐标已扫描则为 `True`。

`.is_empty(x, y)`

仅当该整数瓦片已经扫描且未包含生命体时才为 `True`。对于已占用和未触及的瓦片都会返回 `False`，因此要将其与 `has_scanned()` 配合使用。

参数 (PARAMETERS)

名称	类型	描述
x	number	整数世界 x 坐标
y	number	整数世界 y 坐标

返回值 (RETURNS) : `Boolean` : 仅当该整数坐标已扫描且不是生物站点时才为 `True`。

`.is_ready(x, y)`

当已发现的生物站点现在可以提取时为 **True**。在其他无人机正在该处提取、资源耗尽或处于基于稀有度的冷却期间时返回 **False**。查询使用当前地点状态并立即返回。

参数 (PARAMETERS)

名称	类型	描述
x	number	生物站点世界 x 坐标
y	number	生物站点世界 y 坐标

返回值 (RETURNS) : **Boolean** : 如果已发现的生物站点有库存或其冷却已结束，且当前没有正在进行的提取，则为 **True**。

`.next_ready_at(x, y)`

提取冷却结束的绝对小时数。即使该小时已经过去，耗尽地点的时间戳仍会保留，直到提取操作补充资源。如果未记录该生物站点、仍有材料，或从未进行过提取，则返回 **None**。使用 `is_ready(x, y)` 检查现在是否可以开始提取。

参数 (PARAMETERS)

名称	类型	描述
x	number	生物站点世界 x 坐标
y	number	生物站点世界 y 坐标

返回值 (RETURNS) : 耗尽生物站点的提取冷却结束绝对小时数。在提取补充地点资源之前，该截止时间可能已经处于过去。如果地点未记录、仍有材料，或从未提取过，则返回 **None**。

组件 / 探索

地图标记 (Map Markers)

从脚本为行星地图添加注释。制图学解锁后，使用 `get_component("markers")` 获取它，然后调用 `markers.place("survey.rover_1.empty:120:-40", 120, -40, "No contact", "x")`，即可即时在世界任意位置放置标记，无需载具或材料。标记是备注，不是蓝图：若要真正在那里建造，请将坐标传给 `construction_blueprint.plan_structure(...)`。将结构化数据存储在 Data Archive 中，并使用相同的 ID。

返回方式：`get_component("markers")`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ① 以找到确切 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性 (PROPERTIES)

`.id`

该组件的稳定程序化标识符。将它与 `get_component(id)` 以及那些要求提供组件、行星、载具、空间站或订单 ID 的 API 配合使用。

返回值 (RETURNS)：`String`

`.name`

面向用户的可读显示名称。对于需要在重命名后仍能运行的脚本，优先使用 `.id`。

返回值 (RETURNS)：`String`

方法 (METHODS)

```
.place(id, x, y, label="", icon="pin", color="accent", note="")
```

创建或重写一个标记。重新使用某个 ID 会移动并重新设置该标记的样式，而不会添加第二个标记，因此脚本在保存后重启也不会让地图充满重复项。坐标为世界米制坐标，并保留其小数部分。按 ID 前缀组织标记族，并在前缀中包含控制机器，例如 "survey.rover_1."，这样两个脚本就不会互相覆盖。

参数 (PARAMETERS)

名称	类型	描述
id	string	稳定标记 ID，长度为 1 – 64 个字符，只能使用字母、数字、_、.、: 或 -。重新使用 ID 会重写该标记。
x	number	世界 x 坐标，单位为米。小数会原样保留。
y	number	世界 y 坐标，单位为米。小数会原样保留。
label	string	显示文本，最多 48 个字符。为空时显示 ID。
icon	string	标记图 glyph。
color	string	标记颜色。
note	string	悬停和 Markers 列表中显示的较长备注，最多 240 个字符。

返回值 (RETURNS) : ActionResult

结果字段 (RESULT FIELDS) : .status、.message

成功载荷 (SUCCESS PAYLOAD) : None

结果 (OUTCOMES)

状态	类型	含义
"ok"	成功	操作成功完成。
"invalid_key"	拒绝	提供的键无效。
"invalid_coords"	拒绝	提供的坐标无效。
"out_of_bounds"	拒绝	请求位置超出有效世界边界。
"invalid_icon"	拒绝	提供的标记 glyph 不是可用 glyph 之一。
"invalid_color"	拒绝	提供的标记颜色不是可用颜色之一。
"invalid_text"	拒绝	提供的文本长度超过字段限制。
"limit_reached"	拒绝	集合已经包含最大数量的条目。

`.get(id)`

按 ID 读取一个标记。

参数 (PARAMETERS)

名称	类型	描述
id	string	要查找的标记 ID。

返回值 (RETURNS) : 具有该 ID 的 Marker ; 如果没有标记使用该 ID, 则返回 `None`。

引发 (RAISES)

异常	条件
ValueError	标记 ID 必须为 1–64 个字符, 只能使用字母、数字、_、.、: 或 -, 并且不能是保留的对象字段名称。

`.list(prefix="")`

读取标记列表, 并按 ID 排序。传入诸如 `"build."` 的前缀即可读取一个标记族。遍历结果来为载具规划路线: `for m in markers.list("build."): self.nav.set_target(m.x, m.y)`。

参数 (PARAMETERS)

名称	类型	描述
prefix	string	可选的 ID 前缀, 例如 <code>"survey.rover_1."</code> 。

返回值 (RETURNS) : 按 ID 排序的 Marker 值列表, 可选地缩小到一个 ID 前缀。空前缀返回全部标记。

引发 (RAISES)

异常	条件
ValueError	标记前缀必须只使用字母、数字、_、.、: 或 -, 且长度最多为 64 个字符。

`.remove(id)`

按 ID 删除一个标记。

参数 (PARAMETERS)

名称	类型	描述
id	string	要删除的标记 ID。

返回值 (RETURNS) : `ActionResult`

结果字段 (RESULT FIELDS) : `.status`、`.message`

成功载荷 (SUCCESS PAYLOAD) : `None`

结果 (OUTCOMES)

状态	类型	含义
"ok"	成功	操作成功完成。
"not_found"	拒绝	请求的对象、目标或记录不存在。
"invalid_key"	拒绝	提供的键无效。

`.clear(prefix)`

按 ID 前缀删除一个完整的标记族，然后重新放置当前标记，使该标记族与脚本当前认定的状态保持同步。必须提供前缀：`markers.clear("")` 会删除行星上的每个标记，包括你手动放置的标记；系统不会记录每个标记的放置者。

参数 (PARAMETERS)

名称	类型	描述
prefix	string	要删除的 ID 前缀。传入 "" 会删除所有标记，包括你手动放置的标记。

返回值 (RETURNS) : `CountResult`

结果字段 (RESULT FIELDS) : `.status`、`.message`

成功载荷 (SUCCESS PAYLOAD) : `.count`

结果 (OUTCOMES)

状态	类型	含义
"ok"	成功	命令影响了 <code>.count</code> 个条目或单位。
"no_op"	成功	命令没有影响任何条目或单位。
"invalid_key"	拒绝	提供的键无效。

组件 / 探索

扫描器 (Scanner)

揭示基地周围收割网格的扇区，以便 Harvester 知道在哪里收集。它只绘制本地网格；探索更广阔的行星是载具声呐的工作。

类型：收集 (Harvesting)

返回方式：`get_component("scanner_1")`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ① 以找到确切 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性 (PROPERTIES)

`.id`

该组件的稳定程序化标识符。将它与 `get_component(id)` 以及那些要求提供组件、行星、载具、空间站或订单 ID 的 API 配合使用。

返回值 (RETURNS)：`String`

`.name`

面向用户的可读显示名称。对于需要在重命名后仍能运行的脚本，优先使用 `.id`。

返回值 (RETURNS)：`String`

方法 (METHODS)

`.scan(sector)` — 仅限自身 (SELF ONLY)

使用 `self.scan("E14")` 扫描一个本地扇区。扫描需要若干 tick，并会暂停脚本。格式错误或超出边界的扇区 ID 会引发 `ValueError`。这个本地网格扫描器查找的是地表物品，而不是行星 Site 接触点。

参数 (PARAMETERS)

名称	类型	描述
<code>sector</code>	<code>string</code>	本地网格扇区 ID (例如 "E14")。

返回值 (RETURNS) : `ScanResult`

结果字段 (RESULT FIELDS) : `.status`、`.message`

成功载荷 (SUCCESS PAYLOAD) : `.id`、`.name`、`.value`

结果 (OUTCOMES)

状态	类型	含义
<code>"ok"</code>	成功	扇区扫描找到了 <code>item_id</code> 。
<code>"empty"</code>	成功	扇区扫描完成，但没有找到物品。

引发 (RAISES)

异常	条件
<code>ValueError</code>	<code>sector</code> 参数不是有效的本地网格扇区 ID。

`.get_scanned()`

以新的字典 `{sector_id: ScanResult}` 返回此前扫描过的每个扇区。使用 `.keys()` / `.values()` / `.items()` 遍历，或按扇区 ID 索引：`self.get_scanned()["E14"]`。每个扇区只需实际扫描一次，且历史会跨脚本运行保留。每次调用 `get_scanned()` 都会反映这些扇区的当前内容，包括上次调用后收集或丢弃的物品。脚本中已经保存的字典或 `ScanResult` 不会自行更新，因此在选择下一个目标前要再次调用 `get_scanned()`。尚未扫描任何内容时返回空字典。

返回值 (RETURNS) : `Dict` (扇区 ID → `ScanResult`)

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` 仅限 SELF

读取你从编辑器的 Commands

选项卡发送的命令。脚本运行期间，这些命令会在此处显示。所有可脚本化机器的行为都相同，详见《Script Commands》指南。

组件 / 电力

电池（Battery）

基地站的能源储存设备。发电有盈余时会自动充电，电网供电不足时会放电。电量耗尽后，机器会关闭，其脚本也会暂停。

类型	电力
能量	500 Wh

获取方式

- 在商店购买，价格为 300 cr。

返回方式：`get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，即可找到准确 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件稳定的程序化标识符。将它与 `get_component(id)` 以及要求提供组件、星球、载具、空间站或订单 ID 的 API 一起使用。

返回值

String

`.name`

面向用户显示的可读名称。需要脚本能够在重命名后继续运行时，优先使用 `.id`。

返回值

String

方法

.outpost

部署此建筑的前哨站。返回的 **OutpostRef** 包含其稳定 ID、显示名称、生物群系、位置、容量以及 **buildings()** 查询。需要当前值时，请重新读取此属性。

返回值

部署此建筑的前哨站的 **OutpostRef**。

.get_level()

当前储存的能量，单位为瓦时（Wh）。当消耗超过发电量时下降，发电量超过消耗时上升，两者相等时保持不变。接近 0 是危险信号，表示电网即将发生电压骤降。

返回值

Number (Wh)

.get_capacity()

电池的总容量，单位为瓦时（Wh）。应查询此值而不是硬编码，这样未来升级不会破坏脚本。与 **get_level()** 配合使用即可计算充电百分比。

返回值

Number (Wh)

组件 / 电力

避雷针（Lightning Rod）

容量为 4,000 Wh 的应急储备设备，可捕获 600 m 范围内的闪电，并在电池之后向电网放电。耐久度每天下降 0.05；除非运行中的脚本使用 1 个 Storm Glass 修复它，否则捕获能力最终会降为零。

类型	电力
能量	4,000 Wh
输入缓冲	5 units

获取方式

- 通过 Lightning Rods 研究（Temperature 8,000）解锁配方。
- 在 Fabricator 上制造避雷针：1× Machine Frame、4× Battery Cell 和 2× Circuit Panel。
- 从 Inventory 中部署。

返回方式：`self / get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，即可找到准确 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件稳定的程序化标识符。将它与 `get_component(id)` 以及要求提供组件、星球、载具、空间站或订单 ID 的 API 一起使用。

返回值

String

`.name`

面向用户显示的可读名称。需要脚本能够在重命名后继续运行时，优先使用 `.id`。

返回值

String

方法

.outpost

部署此建筑的前哨站。返回的 **OutpostRef** 包含其稳定 ID、显示名称、生物群系、位置、容量以及 **buildings()** 查询。需要当前值时，请重新读取此属性。

返回值

部署此建筑的前哨站的 **OutpostRef**。

.bank()

当前储存的 Wh，范围为 0 至 **capacity()**。只有当闪电在此避雷针的捕获范围内击中时才会增加；当其所在电网短缺且电池为空时会自动下降。它绝不会从电网盈余中充电。

返回值

Number：当前储存的 Wh，范围为 0 至 **capacity()**。只有雷暴闪电在此避雷针的捕获范围内击中时才会填充；当其所在电网短缺且电池为空时会自动放电。不会从电网盈余中充电。

.capacity()

以 Wh 为单位的储能容量，是基础电池的数倍。应查询此值而不是硬编码。

返回值

Number：以 Wh 为单位的储能容量。

`.last_strike()`

最近一次使此避雷针接收能量的闪击时间戳（小时）；若从未接收过则为 -1。当闪击没有增加能量时，例如储能已满或完整度为零，不会更新此记录。将其与时钟的当前时间比较，即可知道距离上次捕获能量已经过了多久。

返回值

Number：最近一次使此避雷针接收能量的闪击时间戳（小时）；若没有则为 -1。当闪击没有增加能量时，例如储能已满或完整度为零，不会更新此记录。

`.input`

避雷针的材料槽。使用普通物流为其中补充 Storm Glass，然后调用 `repair()` 消耗一个并恢复完整度。

返回值

InputSlot：支持

`connect()`、`take()`、`eject()`、`flush()`、`count()`、`capacity()`、`connected_to()`。其中存放供 `repair()` 消耗的 Storm Glass。

`.integrity()`

此避雷针的状态，范围为 0 至 1，同时也是它的捕获效率。持续腐蚀会使其每天降低 0.05；闪击不会造成额外损伤。完整度为 0.5 时，每次捕获的闪击只能储存一半能量；完整度为 0 时不会储存任何能量，但设备仍然存在且可以修复。

返回值

Number (0-1)：此避雷针的状态，同时也是捕获效率。完整度为 0.5 时，每次捕获的闪击只能储存一半；完整度为 0 时不会储存能量，但仍然存在。

`.repair()` 仅限 SELF

将此避雷针恢复为完整状态。如果设备已经磨损，一次调用会从其输入槽消耗 1 个 Storm Glass，并将状态设为 1。状态已满时不会消耗材料。输入槽中没有 Storm Glass 时，状态不会改变。

返回值	结果字段	成功载荷
ActionResult	.status、.message	None

结果

状态	类型	含义
"ok"	成功	操作成功完成。
"no_op"	拒绝	操作本已满足，因此状态未改变，也没有消耗任何物品。
"no_material"	拒绝	机器输入中没有所需数量的材料。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` 仅限 SELF

读取你从编辑器的 Commands 选项卡发送的命令。脚本运行期间，这些命令会在此处显示。所有可脚本化机器的行为都相同，详见《Script Commands》指南。

组件 / 电力

石油发电机（Oil Generator）

将石油燃烧为强劲且带缓冲的桥接电力。油井会在活跃与休眠阶段之间脉冲切换，因此应将其输出储存在 Liquid Tank 中，以实现持续发电。只有脚本运行它时才会工作。

类型	电力
输出功率	+700 W（供电网）
消耗	Oil, 缓冲 10 t

获取方式

- 需要 Oil Generator 研究（Temperature 1,500）。
- 在商店购买，价格为 2,000 cr。

返回方式：`self / get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，即可找到准确 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件稳定的程序化标识符。将它与 `get_component(id)` 以及要求提供组件、星球、载具、空间站或订单 ID 的 API 一起使用。

返回值

String

`.name`

面向用户显示的可读名称。需要脚本能够在重命名后继续运行时，优先使用 `.id`。

返回值

String

方法

`.outpost`

部署此建筑的前哨站。返回的 `OutpostRef` 包含其稳定 ID、显示名称、生物群系、位置、容量以及 `buildings()` 查询。需要当前值时，请重新读取此属性。

返回值

部署此建筑的前哨站的 `OutpostRef`。

`.power_output()`

上一个电力 tick 送入电网的瓦数。当节流降为 0 或 `oil_in` 缓冲耗尽时为 0。发电机按照可用石油量按比例调整输出，因此部分缺油时会产生部分功率。每个电力 tick 更新一次；新的 `set_throttle(...)` 会在下一 tick 生效。

返回值

Number：当前送入电网的瓦数。空闲时（`throttle=0`）或缺油时（缓冲中没有石油）为 0。

`.oil_consumption()`

上一个电力 tick 实际消耗的石油，单位为 t/h。石油供应充足时，需求会随节流从 0 线性增加到 1 时的 8 t/h。部分或完全缺油都会降低实际速率。

返回值

Number：消耗的石油，单位为 t/h。空闲时为 0。

`.throttle()`

当前节流值（0-1）。默认值为 0，发电机在脚本控制前保持空闲。

返回值

Number（0-1）。

`.set_throttle(rate)` 仅限 SELF

设置发电机节流值（0-1）。输出功率与石油消耗会随节流值线性变化。脚本拥有的设定点会在脚本停止、结束或出错时重置为 0，因此发电机应持续运行时必须让控制循环保持运行。

参数

名称	类型	描述
rate	number	节流设置（0-1）。
返回值	结果字段	成功载荷
ActionResult	.status、.message	None

结果

状态	类型	含义
"ok"	成功	操作成功完成。

`.oil_in`

石油输入流端口。使用 `self.oil_in.connect("Liquid Tank 1")`（推荐），或直接连接 Oil Pump。参见 `FluidPort`。

返回值

`FluidPort`：`connect()`、`connected_to()`、`level()`、`capacity()`、`flow_rate()`。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` 仅限 SELF

读取你从编辑器的 Commands 选项卡发送的命令。脚本运行期间，这些命令会在此处显示。所有可脚本化机器的行为都相同，详见《Script Commands》指南。

组件 / 电力

反应堆 (Reactor)

使用燃料棒和冷却水，最高可产生 5,000 W。热量为 1.0 时，一根燃料棒可持续 72 小时；即使核心正在升温或处于高效区间之外，燃料消耗也遵循命令设定的热量。

类型	电力
输出功率	+5,000 W（供给电网）
消耗	Water, 缓冲 3 t
输入缓冲	3 units

获取方式

- 需要 Nuclear Program 研究（Terraform Index 650,000）。
- 在商店购买，价格为 750,000 cr。

返回方式：`self / get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，即可找到准确 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件稳定的程序化标识符。将它与 `get_component(id)` 以及要求提供组件、星球、载具、空间站或订单 ID 的 API 一起使用。

返回值

String

`.name`

面向用户显示的可读名称。需要脚本能够在重命名后继续运行时，优先使用 `.id`。

返回值

String

方法

.outpost

部署此建筑的前哨站。返回的 `OutpostRef` 包含其稳定 ID、显示名称、生物群系、位置、容量以及 `buildings()` 查询。需要当前值时，请重新读取此属性。

返回值

部署此建筑的前哨站的 `OutpostRef`。

.set_heat(value) 仅限 SELF

将反应堆热量设置为 0-1。超出范围的值会被截断。所属脚本停止时，该设置会恢复为 0。

名称	类型	描述
value	number	0-1 范围内的热量设置。
返回值	结果字段	成功载荷
ActionResult	.status、.message	None

结果

状态	类型	含义
"ok"	成功	操作成功完成。

.heat()

当前热量设置，范围为 0-1。

返回值

Number 0-1：当前热量设置。

`.temperature()`

当前温度，单位为 °C。输出从 300 开始，在 900 达到峰值，然后在 900-950 的红色区间内回落为零。达到 950 会触发自动过热关机。

返回值

Number：本 tick 的反应堆温度（°C）。输出从 300 开始，在 900 达到峰值，在 900-950 的红色区间内回落为零；达到 950 会过热。

`.fuel_level()`

当前燃料棒寿命，范围为 0-1。一根完整燃料棒在热量为 1.0 时可持续 72 小时；较低热量会按比例延长寿命。下一根燃料棒会自动从 `input` 中取出。

返回值

Number 0-1：当前燃料棒的剩余寿命。

`.power_output()`

本 tick 送入电网的瓦数。

返回值

Number：本 tick 产生的瓦数（0-5,000）。

`.status()`

当前运行状态：`running`、`overheated`、`no_fuel` 或 `no_coolant`。冷却完成或供应恢复后，关机会自动恢复。

返回值

可能的值：`"running"`、`"overheated"`、`"no_fuel"`、`"no_coolant"`。过热会自动冷却并恢复；供应恢复后会自动继续运行。

`.water_in`

自动冷却水输入。反应堆在升温时每小时消耗 0.5-1t；供应耗尽时会安全暂停。

返回值

`FluidPort`：自动冷却水输入。支持 `connect()`、`level()`、`capacity()`、`flow_rate()` 和 `connected_to()`。

`.input`

普通燃料棒输入。燃料棒从 Lead Cask 到达，需要时反应堆会自动取出下一根。

返回值

`InputSlot`：普通燃料棒供应。支持 `connect()`、`take()`、`eject()`、`flush()`、`count()`、`capacity()` 和 `connected_to()`。需要时反应堆会自动装入燃料棒。燃料棒回收必须指定兼容的本地 Lead Cask。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` 仅限 SELF

读取你从编辑器的 Commands

选项卡发送的命令。脚本运行期间，这些命令会在此处显示。所有可脚本化机器的行为都相同，详见《Script Commands》指南。

组件 / 电力

太阳能发电机（Solar Generator）

将阳光转化为最高 50 W 的电力供给电网，前提是脚本持续追踪太阳。倾角不佳会降低白天的输出，夜间输出为 0 W。

类型	电力
输出功率	+50 W（供给电网）

获取方式

- 在商店购买，价格为 500 cr。

返回方式：`self / get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，即可找到准确 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件稳定的程序化标识符。将它与 `get_component(id)` 以及要求提供组件、星球、载具、空间站或订单 ID 的 API 一起使用。

返回值

String

`.name`

面向用户显示的可读名称。需要脚本能够在重命名后继续运行时，优先使用 `.id`。

返回值

String

方法

.outpost

部署此建筑的前哨站。返回的 `OutpostRef` 包含其稳定 ID、显示名称、生物群系、位置、容量以及 `buildings()` 查询。需要当前值时，请重新读取此属性。

返回值

部署此建筑的前哨站的 `OutpostRef`。

.set_tilt(degrees) 仅限 SELF

设置面板的倾角，单位为度。范围为 0 °（水平）到 90 °（垂直）；超出范围的值会被截断。调校良好的追踪器能够让全天输出保持接近最大值；固定倾角会浪费很大一部分输出。

名称	类型	描述
degrees	number	面板倾角，单位为度
返回值	结果字段	成功载荷
ActionResult	.status、.message	None

结果

状态	类型	含义
"ok"	成功	操作成功完成。

.tilt()

当前面板倾角设定值，单位为度（0-90）。返回脚本最近通过 `self.set_tilt(...)` 写入的值；面板空闲时则返回默认停放角。可用它验证扫描循环，或基于前一个值逐步搜索倾角。

返回值

Number (degrees, 0-90)：当前面板倾角设定值。

`.get_output()`

此特定发电机当前的输出功率，单位为瓦。断电时返回

0。该值根据太阳高度角与面板倾角实时计算；可用它验证追踪器是否保持峰值（将当前输出与面板额定最大值比较）。

返回值

Number (watts)

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` 仅限 SELF

读取你从编辑器的 Commands

选项卡发送的命令。脚本运行期间，这些命令会在此处显示。所有可脚本化机器的行为都相同，详见《Script Commands》指南。

组件 / 电力

蒸汽涡轮机（Steam Turbine）

使用 90 t/h 的蒸汽，最高可产生 108 W。由脚本拥有的节流值会同时调整蒸汽消耗和输出。

类型	电力
输出功率	+108 W（供电电网）
消耗	Steam，最高 90 t/h，缓冲 100 t

获取方式

- 需要 Steam Turbine 研究（Pressure 3）。
- 在商店购买，价格为 7,500 cr。

返回方式：`self / get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，即可找到准确 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件稳定的程序化标识符。将它与 `get_component(id)` 以及要求提供组件、星球、载具、空间站或订单 ID 的 API 一起使用。

返回值

String

`.name`

面向用户显示的可读名称。需要脚本能够在重命名后继续运行时，优先使用 `.id`。

返回值

String

方法

.outpost

部署此建筑的前哨站。返回的 **OutpostRef** 包含其稳定 ID、显示名称、生物群系、位置、容量以及 **buildings()** 查询。需要当前值时，请重新读取此属性。

返回值

部署此建筑的前哨站的 **OutpostRef**。

.power_output()

上一个电力 tick 送入电网的瓦数。该值随 **throttle()** 和实际可用蒸汽量变化。空闲或缺蒸汽时为 0。每个电力 tick 更新一次；新的 **set_throttle(...)** 会在下一 tick 而不是当前 tick 生效。可用它制作实时仪表板，或与 **Oxygen / Heat** 消耗比较，以平衡电力预算。

返回值

Number：当前送入电网的瓦数。

.efficiency()

上一个电力 tick 实际抽取的蒸汽量，占节流目标蒸汽量的比例（0.0-1.0）。1.0 表示涡轮机获得了节流值要求的全部蒸汽；低于 1 表示蒸汽不足（排气口处于休眠状态，或上游供应跟不上）。用它来检测涡轮机是否得到足够供给。

返回值

Number (0-1)：实际抽取的蒸汽量占节流目标量的比例（低于 1 表示蒸汽不足）。

`.is_stalled()`

当节流阀已打开但没有蒸汽到达时返回 **True**（通风口处于休眠状态或蒸汽管线已断开）。检查 `self.steam_in.level()`（上游）以及供给端 Cap 的通风阶段以进行诊断。现在已经没有水侧会再次造成堵塞。

返回值

布尔值：当节流阀已打开但没有蒸汽到达（通风口休眠或已断开）时为 **True**。

`.throttle()`

当前节流阀设置（0.0-1.0）。0 = 关闭，涡轮机不消耗蒸汽也不产生任何东西（默认值；在调用 `set_throttle()` 之前保持空闲）。数值越高，抽取的蒸汽越多、产生的功率越高，最高点为 1.0。只读视图，显示上一次 `set_throttle()` 提交的值。

返回值

数字（0-1）：当前节流阀设置。

`.set_throttle(t)`
仅限自身

设置涡轮机节流阀（0.0-1.0，超出范围时会截断）。0 会关闭涡轮机（不消耗蒸汽、不产生功率）。1.0 会以峰值瓦数抽取全部蒸汽。`self.set_throttle(1.0)` 会让它全速运行；当蒸汽缓冲区耗尽时应降低设置，避免空转，例如：`if self.steam_in.level() < 5: self.set_throttle(0.3)`。此脚本拥有的设定点会在脚本停止、结束或出错时重置为 0。

参数

名称	类型	描述
t	number	0 = 空闲（无蒸汽、无功率），1 = 全蒸汽抽取 / 最大功率。

返回值：`ActionResult`

结果字段：`.status`、`.message`

成功载荷：`None`

结果

状态	类型	含义
"ok"	成功	操作成功完成。

`.steam_in`

接受来自 Thermal Cap 或 Gas Tank 的蒸汽的输入流端口。使用源设备的稳定机器 ID 或显示名称调用 `self.steam_in.connect(...)`。本地源会直接传输；远程源还需要在两处位置之间存在一条已完成且无冲突的 Gas Pipe 路由。`self.steam_in.level()` 显示缓冲区液位：较低表示即将耗尽；已满表示源受到背压。参见 FluidPort。

返回值

`FluidPort`：`connect()`、`connected_to()`、`level()`、`capacity()`、`flow_rate()`。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()`
仅限自身

读取脚本运行期间从编辑器 Commands 选项卡发送的命令。所有可编程机器均相同，参见指南：Script Commands。

Vehicle Charging Station

电网供电的车队充电站：Mk I 提供 1 个车位 / 30 W，Mk II 提供 2 个车位 / 120 W，Mk III 提供 4 个车位 / 240 W。空闲车位会汇集功率为一辆车充电；多辆车共享预算。脚本可以排队充电或调度救援。

类型	Mining
输入功率	可变（从电网获取）
等级	Mk II 和 Mk III

获取方式

- 需要 Vehicle Charging Station 研究（Oxygen 9）。
- 在 Shop 中以 1,200 cr 购买。

返回方式：`self/get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，找到确切 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件稳定的程序化标识符。将其用于 `get_component(id)`，以及所有要求提供组件、星球、车辆、站点或订单 ID 的 API。

返回值：String

`.name`

面向用户的可读显示名称。需要经受重命名的脚本应优先使用 `.id`。

返回值：String

方法

`.outpost`

此建筑所部署的前哨站。返回的 `OutpostRef` 包含其稳定 ID、显示名称、生物群系、位置、容量以及 `buildings()` 查询。需要当前值时请重新读取该属性。

返回值

部署此建筑的前哨站的 `OutpostRef`。

`.get_docked()`

列出停放在此站本地服务平台及其约 2 m 缓冲范围内，或停放在所属 `Outpost` 公共服务区域内的车辆实例 ID。返回的是 ID，而不是车辆对象；通过 `get_component(id)` 解引用每个 ID，即可读取电池状态、货物或其他信息。空列表表示没有停靠车辆。每次迭代都应调用此方法，因为车辆可能在 tick 之间驶入或驶出。

返回值

停靠在此站的车辆实例 ID 列表。通过 `get_component(id)` 读取每辆车的状态。

`.charge(vehicle_id, target_level=1.0)`
仅限自身

将已停靠的漫游车或 Pioneer 加入充电队列，直到电池达到 `target_level`（大于 0 且不超过 1，默认为 1.0）。站点决定立即开始还是排在另一辆车后面：Mk I 一次为一辆车充电，Mk II 为两辆，Mk III 为四辆。活动车辆越少，每辆车充电越快（空闲车位会汇集功率）。例如：`self.charge("pioneer_1", 0.8)` 表示“为这辆车充电，直到达到 80%”。

参数

名称	类型	描述
<code>vehicle_id</code>	string	已停靠漫游车或 pioneer 的显示名称或 ID
<code>target_level</code>	number	可选的电池目标比例，大于 0 且不超过 1。默认为 1.0。

返回值：`ActionResult`；结果字段：`.status`、`.message`；成功载荷：`None`

结果

状态	类型	含义
<code>"charging"</code>	成功	充电任务处于活动状态。
<code>"queued"</code>	成功	操作已加入队列。
<code>"target_reached"</code>	拒绝	车辆已经达到或超过请求的充电量或燃料量。
<code>"not_docked"</code>	拒绝	请求的车辆未停靠在此站。
<code>"station_offline"</code>	临时	站点处于离线状态。
<code>"invalid"</code>	拒绝	一个或多个提供的参数超出可接受范围。

`.stop(vehicle_id)`
仅限自身

从此站的充电队列中移除一辆车。不会移动车辆，也不会改变其当前电池电量。

参数

名称	类型	描述
vehicle_id	string	已排队车辆的显示名称或 ID

返回值：`ActionResult`；结果字段：`.status`、`.message`；成功载荷：`None`

结果

状态	类型	含义
"ok"	成功	操作成功完成。
"not_found"	拒绝	请求的对象、目标或记录不存在。
"invalid"	拒绝	一个或多个提供的参数超出可接受范围。

`.clear_queue()`
仅限自身

清除此站中所有排队的充电任务。救援无人机任务是独立的，不会因该操作而取消。

返回值：`CountResult`；结果字段：`.status`、`.message`；成功载荷：`.count`

结果

状态	类型	含义
"ok"	成功	命令影响了 <code>.count</code> 个条目或单位。
"no_op"	成功	命令没有影响任何条目或单位。

`.get_active()`

列出当前占用活动充电车位的车辆 ID。Mk I 最多返回一个 ID，Mk II 返回两个，Mk III 返回四个。这些车辆在当前 tick 共享站点汇集的预算。

返回值

当前占用充电车位的车辆 ID 列表。

`.get_queue()`

按充电队列顺序列出车辆 ID。前 `get_bay_count()` 个条目是当前可以处于活动状态的车辆，前提是它们仍停靠且电量低于目标。

返回值

按顺序排列的排队车辆 ID 列表；前 `get_bay_count()` 个条目可以处于活动状态。

`.status(vehicle_id)`

返回一辆车的详细状态：包含 `state` ("charging"、"queued"、"docked"、"target_reached"、"not_docked"、"station_offline" 或 "missing")、`target_level`、`battery_wh`、`capacity_wh`、`rate_w`（该车辆实际获得的汇集瓦数；共享车位的车辆越少，该值越高）、`bay_index` 和 `queue_index`。可用它构建仪表盘或队列管理器。

参数

名称	类型	描述
<code>vehicle_id</code>	string	车辆的显示名称或 ID

返回值

包含 `state`、`target_level`、`battery_wh`、`capacity_wh`、`rate_w`、`bay_index`、`queue_index` 的 Dict。

`.tier()`

以整数（1-3）返回永久安装的 Charging Station 等级。Mk II 会增加车位数量和车位速率；Mk III 会再次增加车位数量。

返回值：整数：永久安装的 Charging Station 等级（1-3）。

`.get_bay_count()`

同时进行车辆充电的车位数量。Mk I 为 1，Mk II 为 2，Mk III 为 4。

返回值：同时进行车辆充电的车位数量。

`.get_bay_rate()`

单个车位输出的瓦数（Mk I 为 30 W，Mk II/III 为 60 W）。通过车位功率汇集，单辆车会获得所有空闲车位的功率，因此其实际速率最高可达 `get_bay_count()` × `get_bay_rate()`；使用 `get_charge_rate(id)` 读取特定车辆实际获得的功率。只要有车辆充电，站点从电网的总消耗就是 `get_bay_count()` × `get_bay_rate()`，另加救援任务正在执行时的无人机消耗。

返回值：每个活动车位输出的瓦数。

`.get_charge_rate(vehicle_id)`

当前实际输入指定车辆的瓦数，即站点的总预算（`get_bay_count()` × `get_bay_rate()`）在所有活动车辆之间平均分配的结果。单辆车获得全部预算（Mk III 为 240 W）；充电车辆越多，每辆车分到的份额越低。如果车辆没有占用活动车位，则返回 0。

参数

名称	类型	描述
<code>vehicle_id</code>	string	已停靠车辆的显示名称或 ID

返回值：数字（当前输入该车辆的 W 数）。

`.dispatch_rescue(vehicle_name, target_level=1.0)`
仅限自身

按显示名称或 ID 向漫游车或 Pioneer 派出现场服务无人机。`target_level` 是大于 0 且不超过 1 的电池比例，默认为 1.0。派出后会停止目标车辆，以便无人机抵达。发射时站点必须通电，但无人机可以在之后的断电期间完成任务。如果目标先抵达已通电的 Charging Station，剩余请求会加入该站队列。一次只能执行一个救援；调用 `cancel_rescue()` 可召回无人机。

参数

名称	类型	描述
<code>vehicle_name</code>	string	要服务的漫游车/Pioneer 的显示名称或实例 ID。
<code>target_level</code>	number	可选的电池目标比例，大于 0 且不超过 1。默认为 1.0。

返回值：`ActionResult`；结果字段：`.status`、`.message`；成功载荷：`None`

结果

状态	类型	含义
"ok"	成功	操作成功完成。
"already_dispatched"	拒绝	此站已有活动救援，或请求的目标已分配给某项救援。
"station_offline"	临时	站点处于离线状态。
"not_found"	拒绝	请求的对象、目标或记录不存在。
"invalid"	拒绝	一个或多个提供的参数超出可接受范围。

`.cancel_rescue()`
仅限自身

召回此站活动中的现场服务无人机。如果无人机正在飞出或进行涓流充电，目标车辆会立即解除绑定，并保留无人机已经提供的充电量；无人机会返回站点。

返回值：`ActionResult`；结果字段：`.status`、`.message`；成功载荷：`None`

结果

状态	类型	含义
"ok"	成功	操作成功完成。
"not_found"	拒绝	请求的对象、目标或记录不存在。

`.is_rescuing()`

在救援无人机已部署期间为 `True`（正在飞出、位于目标处或正在返回）。被召回的无人机在抵达站点前仍会计入救援中，但 `cancel_rescue()` 成功后目标车辆会立即解除绑定。调用 `dispatch_rescue()` 前可用它避免 `"already_dispatched"` 拒绝：`if not self.is_rescuing():`
`self.dispatch_rescue(name)`。一次只能有一架无人机；如需更多救援，请手动排队。

返回值：`Boolean`

`.get_rescue_target()`

返回当前正在接受救援的车辆显示名称；如果无人机空闲，则返回空字符串。可用于仪表板（“正在救援 Rover 1”），或决定等待还是派另一辆车前去接替任务。

返回值：`String`（正在接受救援的车辆显示名称，或空字符串）。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()`
仅限自身

读取脚本运行期间从编辑器 Commands 选项卡发送的命令。所有可编程机器均相同，参见指南：Script Commands。

组件 / 地形改造 (COMPONENTS / TERRAFORMING)

Atmosphere

行星大气层：读取气体成分、压力、温度以及行星热单位进度指标 (`get_heat()`)。读取氧气和压力需要先修复相应传感器。

返回方式：`get_component("atmosphere")`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，找到确切 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件稳定的程序化标识符。将其用于 `get_component(id)`，以及所有要求提供组件、星球、车辆、站点或订单 ID 的 API。

返回值：String

`.name`

面向用户的可读显示名称。需要经受重命名的脚本应优先使用 `.id`。

返回值：String

方法

`.get_co2()`

当前二氧化碳含量，单位为千分比 (ppt)。产氧会按 1:1 消耗 CO₂，因此随着氧气升高，该值会下降。燃烧石油 (Oil Generator)、焚烧物品 (Waste Processor) 以及大规模的野生动物呼吸 (已建立的殖民地会呼出 CO₂) 会将 CO₂ 释放回大气。储量较低时，少量火山逸气会补充 CO₂。

返回值：数字 (ppt)。

`.get_o2()`

当前氧气含量，单位为千分比（ppt）。需要先修复氧气传感器。

返回值：数字（ppt）。

`.get_n2()`

当前氮气含量，单位为千分比（ppt）。

返回值：数字（ppt）。

`.get_pressure()`

当前大气压力，单位为 kPa。需要先修复压力传感器。

返回值：数字（kPa）。

`.get_temperature()`

当前地表温度，单位为 °C。这是显示值，是热单位指标的非线性变换。对于地形改造进度或热量阈值，应使用 `get_heat()`，不要使用此值。

返回值：数字（°C）。

`.get_heat()`

当前累计热单位，即温度支柱的进度指标；温度研究和阶段阈值会与其精确值进行比较（Research 页面会显示当前目标）。热量截止条件也应以此为依据，就像这些支柱使用 `get_o2()` / `get_pressure()` 一样。它不同于 `get_temperature()`（地表 °C，是非线性显示值）：热单位的差值就是地形改造进度。初始值为 0。

返回值

数字（热单位）：温度支柱的进度指标，也是研究/阶段阈值比较的对象。

组件 / 地形改造 (COMPONENTS / TERRAFORMING)

Bio Caster

在装入配方材料后，将坩埚保持在目标温度区间内，然后进行浇铸，以锻造火山碎片。蒸汽和水使用独立的 20 t 内部工艺缓冲区。将每个输入端口连接到兼容的源；本地源会直接传输，远程源还需要在两处位置之间存在一条已完成且无冲突的管道路线。在脚本中驱动加热和冷却；在区间外浇铸或使用错误材料会烧毁整份装料。

类型	Atmosphere
输入功率	-15 W（从电网获取）
消耗	蒸汽，缓冲区 20 t
消耗	水，缓冲区 20 t
输出缓冲区	30 单位
储备堆	30 单位（混合）

获取方式

- 需要 Volcanic Forge-Casting 研究（Oxygen 350）。
- 在 Shop 中以 150,000 cr 购买。

返回方式：`self/get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，找到确切 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件稳定的程序化标识符。将其用于 `get_component(id)`，以及所有要求提供组件、星球、车辆、站点或订单 ID 的 API。

返回值：String

`.name`

面向用户的可读显示名称。需要经受重命名的脚本应优先使用 `.id`。

返回值：String

方法

`.outpost`

此建筑所部署的前哨站。返回的 `OutpostRef` 包含其稳定 ID、显示名称、生物群系、位置、容量以及 `buildings()` 查询。需要当前值时请重新读取该属性。

返回值

部署此建筑的前哨站的 `OutpostRef`。

`self.list_recipes()`

列出全部 16 个锻造配方，不改变当前选中的配方：`self.list_recipes()`。每个 `BioCasterRecipe` 都包含碎片 ID、等级、精确的材料采购清单和目标温度范围。该读取操作通过 `get_component(...)` 工作，因此另一台机器无需拥有每种碎片也能规划供应。

返回值

全部 16 个 `BioCasterRecipe` 对象的列表。每个对象都公开 `.fragment_id`、`.tier`、`.materials` 和 `.temperature_range`，用于只读生产规划。

`self.find_recipe(fragment_id)`

按火山碎片 ID 查找一个锻造配方，但不会选中它：`self.find_recipe(fragment_id)`。未知 ID 返回 `None`。

参数

名称	类型	描述
<code>fragment_id</code>	<code>string</code>	来自 <code>catalog()</code> 的火山碎片 ID。

返回值

匹配的 `BioCasterRecipe`，或未知碎片 ID 对应的 `None`。

`self.catalog()`

列出可锻造的 16 个火山碎片 ID：`self.catalog()`。将其中一个传给 `set_recipe(...)`；如果还需要每个配方的材料和温度要求，则使用 `list_recipes()`。硬件固定，只需读取一次。

返回值

16 个可锻造 Volcanic fragment ID 的列表。如果还需要材料和温度要求，请使用 `list_recipes()`。

`self.recipe_tier(fragment_id)`

根据 `catalog()` 为一个配方计算生产等级。未知碎片 ID 返回 `None`。

参数

名称	类型	描述
<code>fragment_id</code>	<code>string</code>	来自 <code>catalog()</code> 的火山碎片 ID。

返回值

此 Bio Caster 配方的计算所得生产等级，或未知碎片 ID 对应的 `None`。

```
self.set_recipe(fragment_id)
```

仅限自身

选择要锻造的火山碎片，例如 `self.set_recipe("sd_tail_barb")`。之后，`required_range()` 和 `required_materials()` 会描述该配方。选择会像 Smelter 配方一样持久保留。

参数

名称	类型	描述
fragment_id	string	来自 <code>catalog()</code> 的火山碎片 ID：你要锻造的碎片。

返回值：`ActionResult`；结果字段：`.status`、`.message`；成功载荷：`None`

结果

状态	类型	含义
"ok"	成功	操作成功完成。
"invalid_recipe"	拒绝	提供的配方无效。
"busy"	临时	组件正在执行另一项操作。

`self.recipe()`

返回你所选的配方，即设定要锻造的火山碎片 ID（也是要装入的原始碎片）；如果未设置配方，则返回 **None**。

返回值

所选配方 = Caster 将锻造的火山碎片 ID（也是要装入的原始碎片）；如果未设置配方，则为 **None**。

可能的值

"ma_chitinous_seta"、"st_beak"、"ms_abdomen_sclerite"、"mh_chitin_node"、"gm_abdominal_sheath"、"vm_tail_barb"、"vc_walking_leg"、"oc_ink_sac"、"bw_hindlimb"、"vd_photophore"、"hs_abdomen_segment"、"hc_beak"、"fs_oral_tegmen"、"ce_great_appendage"、"gw_jaw_fang"、"sd_tail_barb"

`self.required_range()`

返回所选配方的目标温度区间 [low, high]（单位：°C）：`self.required_range().cast()`
必须在温度处于该区间（含边界）时触发。如果未设置配方，则返回 **None**。

返回值

所选配方的目标温度区间 [low, high]（单位：°C）：`cast()`
必须在温度处于该区间（含边界）时触发。如果未设置配方，则返回 **None**。

`self.required_materials()`

以 `{item_id: count}` 的形式列出所选配方所需的加工材料。浇铸前应准确装入这些数量。使用 `.items()` 进行迭代。未选择配方时返回空字典。

返回值

所选配方需要的加工材料字典 `{item_id: count}`：在 `cast()` 前必须准确装入这些材料（不能多也不能少）。未设置配方时为 `{}`。使用 `.items()` 进行迭代。

`self.required_fragment()`

返回配方消耗的原始碎片 ID：`self.required_fragment()`（与 `recipe()` 相同）。未设置配方时返回 `None`。

返回值

所选配方消耗的原始 Volcanic fragment ID（与 `recipe()` 相同）。未设置配方时为 `None`。

可能的值

"ma_chitinous_seta"、"st_beak"、"ms_abdomen_sclerite"、"mh_chitin_node"、"gm_abdominal_sheath"、"vm_tail_barb"、"vc_walking_leg"、"oc_ink_sac"、"bw_hindlimb"、"vd_photophore"、"hs_abdomen_segment"、"hc_beak"、"fs_oral_tegmen"、"ce_great_appendage"、"gw_jaw_fang"、"sd_tail_barb"

`self.temperature()`

当前坩埚温度（°C），`self.temperature()`，范围为 100（冷基线）到 1,000（最高）。在铸造前，使用旋钮将其调至 `required_range()` 要求的范围内。

返回值

当前坩埚温度，单位为 °C（100-1,000）。在铸造前，使用旋钮将其调至 `required_range()` 要求的范围内。

`self.temp_rate()`

当前温度变化速率，单位为 °C/h，`self.temp_rate()`。正值表示升温，负值表示降温。全功率加热为 +2400 °C/h，全功率冷却为 -2400 °C/h；两个旋钮都为 0 时，高于基线且未锁定的坩埚会以 -20 °C/h 自然冷却。在 100 °C 基线处或铸造进行期间返回 0。

返回值

当前净温度速率，单位为 °C/h，带正负号：正值表示加热，负值表示冷却。全功率加热为 +2400 °C/h，全功率冷却为 -2400 °C/h。两个旋钮都为 0 时，高于基线且未锁定的坩埚会被动以 -20 °C/h 冷却；在基线处或铸造期间返回 0。

`self.heat()`

当前加热旋钮设置（0-100%），`self.heat()`。设置为 100% 时温度以 +2400 °C/h 上升（燃烧蒸汽）；接近目标区间时应使用较低设置。

返回值

当前加热旋钮设置，范围为 0-100%。设置为 100% 时温度以 +2400 °C/h 上升（燃烧蒸汽）；接近目标区间时应使用较低设置。

`self.cool()`

当前冷却旋钮设置（0-100%），`self.cool()`。设置为 100% 时温度以 -2400 ° C/h 下降（燃烧水）；接近目标区间时应使用较低设置。

返回值

当前冷却旋钮设置，范围为 0-100%。设置为 100% 时温度以 -2400 ° C/h 下降（燃烧水）；接近目标区间时应使用较低设置。

`self.fragment()`

腔室中加载的原始碎片 ID，`self.fragment()`；腔室为空时为 `None`。

返回值 | 可能的值

当前腔室中的原始 Volcanic 碎片 ID（来自 `load`），腔室为空时为 `None`。|`"ma_chitinous_seta"`、`"st_beak"`、`"ms_abdomen_sclerite"`、`"mh_chitin_node"`、`"gm_abdominal_sheath"`、`"vm_tail_barb"`、`"vc_walking_leg"`、`"oc_ink_sac"`、`"bw_hindlimb"`、`"vd_photophore"`、`"hs_abdomen_segment"`、`"hc_beak"`、`"fs_oral_tegmen"`、`"ce_great_appendage"`、`"gw_jaw_fang"`、`"sd_tail_barb"`

`self.materials()`

当前坩埚中加载的材料, `self.materials()` 返回 `{item_id: count}`。在 `cast()` 前与 `required_materials()` 比较。使用 `.items()` 迭代。

返回值

当前坩埚中已加载的制造材料字典 `{item_id: count}`。在 `cast()` 前与 `required_materials()` 比较。使用 `.items()` 迭代。

`self.set_heat(pct)` 仅限自身

设置加热旋钮（蒸汽使温度升高）；`self.set_heat(100)` 对应 +2400 °C/h，然后在接近目标时使用较低百分比。范围为 0-100，并会被限制在该范围内。这是开环控制：在你将其调回之前，它会持续加热并消耗蒸汽。两个旋钮都为 0 时，坩埚仍会以 20 °C/h 自然冷却，因此应在进入区间后铸造。腔室清空或脚本停止时会重新置为 0。

参数

名称 | 类型 | 描述

`pct` | `number` | 加热等级 0-100（%）。两个旋钮可同时运行；净速率 = `heat` - `cool`。

返回值 | 结果字段 | 成功载荷

`ActionResult` | `.status`、`.message` | `None`

结果

状态 | 类型 | 含义

"ok" | 成功 | 操作成功完成。

"empty" | 拒绝 | 相关源或队列为空。

"busy" | 暂态 | 组件正在执行其他操作。

`self.set_cool(pct)` 仅限自身

设置冷却旋钮（水使温度降低）；`self.set_cool(100)` 对应 -2400 ° C/h, 然后在接近目标时使用较低百分比。范围为 0-100，并会被限制在该范围内。这是开环控制：在你将其调回之前，它会持续冷却并消耗水。两个旋钮可同时运行，但会同时燃烧两种流体却只产生很小的位移。两个旋钮都为 0 时，坩埚仍会以 20 ° C/h 自然冷却。腔室清空或脚本停止时会重新置为 0。

参数

名称 | 类型 | 描述

`pct` | `number` | 冷却等级 0-100（%）。两个旋钮可同时运行；净速率 = heat - cool。

返回值 | 结果字段 | 成功载荷

`ActionResult` | `.status`、`.message` | `None`

结果

状态 | 类型 | 含义

`"ok"` | 成功 | 操作成功完成。

`"empty"` | 拒绝 | 相关源或队列为空。

`"busy"` | 暂态 | 组件正在执行其他操作。

`self.load(fragment_id, properties=None, property_match=None)` 仅限自身

从 `self.input` 中取出 `fragment_id` 指定的原始火山样本并放入腔室，通常使用 `self.load(self.recipe())`。可选的 `properties` 和 `property_match` 使用标准的 `any`、`subset` 或 `exact` 约定来选择特定身份。

参数

名称 | 类型 | 描述

`fragment_id` | `string` | 在 `self.input` 中暂存的原始 Volcanic 碎片 ID。通常使用 `recipe()`。

`properties` | `any` | 可选属性字典，默认按子集匹配。省略或为 `None` 表示匹配任意属性；与 `property_match="exact"` 一起使用 `None` 可只选择无属性项目。

`property_match` | `string` | 可选选择模式：`any`、`subset` 或 `exact`。

返回值 | 结果字段 | 成功载荷

`ActionResult` | `.status`、`.message` | `None`

结果

状态 | 类型 | 含义

"ok" | 成功 | 操作成功完成。

"chamber_occupied" | 拒绝 | 处理腔室已经包含项目。

"not_in_input" | 拒绝 | 请求的项目不在输入中。

"invalid_fragment" | 拒绝 | 提供的碎片标识符无效。

"invalid_properties" | 拒绝 | 提供的项目属性选择器无效。

"invalid_property_match" | 拒绝 | 提供的属性匹配模式无效。

"busy" | 暂态 | 组件正在执行其他操作。

`self.eject()` 仅限自身

将腔室样本和所有已加载材料原样放入

`self.output`，不改变其属性。单材料输出可能需要针对每种项目类型分别执行一次发送/弹出循环。

返回值 | 结果字段 | 成功载荷

`ActionResult` | `.status`、`.message` | `None`

结果

状态 | 类型 | 含义

"ok" | 成功 | 操作成功完成。

"empty" | 拒绝 | 相关源或队列为空。

"busy" | 暂态 | 组件正在执行其他操作。

"output_full" | 拒绝 | 输出端没有足够容量容纳结果。

`self.cast()` 仅限自身

锻造已加载的碎片，`self.cast()`。

返回值 | 结果字段 | 成功载荷

`ActionResult` | `.status`、`.message` | `None`

结果

状态 | 类型 | 含义

"ok" | 成功 | 操作成功完成。

"out_of_range" | 拒绝 | 腔室温度超出所选配方的要求范围，因此已销毁加载的碎片和材料。

"wrong_materials" | 拒绝 | 暂存材料与操作要求不匹配。

"wrong_fragment" | 拒绝 | 已加载碎片与请求的碎片不匹配。

"empty" | 拒绝 | 相关源或队列为空。

"no_recipe" | 拒绝 | 当前未选择配方。

"busy" | 暂态 | 组件正在执行其他操作。

"output_full" | 拒绝 | 输出端没有足够容量容纳结果。

self.input

用于原始火山样本和制造材料的多材料 **InputSlot**（输入槽）。仅在 Nocturna Base 连接 Inventory；在其他前哨站连接同一前哨站的 Storage Bin/Warehouse。对每个所需项目调用 **take(...)**。

返回值

原始 Volcanic 样本和制造材料的 **InputSlot**。Inventory 仅在基地可用；在远程前哨站，通过本地存储转运货物。在 **cast()** 前，可使用 **eject(...)** 取回误放的样本或材料。

self.output

用于锻造样本和非破坏性弹出项目的 **OutputSlot**。项目的确切属性会被保留。

返回值

用于锻造样本和非破坏性弹出项目的 **OutputSlot**。

`self.steam_in`

通过 20 t 蒸汽缓冲区为 Caster 的加热控制供给蒸汽。使用 Thermal Cap 或蒸汽 Gas Tank 的稳定机器 ID 或显示名称调用 `self.steam_in.connect(...)`。本地源会直接传输；远程源还需要在两个地点之间完成无冲突的 Gas Pipe 路由。空闲、已充满或断电时连接仍保持。有关液位、容量和流量查询，请参见 FluidPort。

返回值

Caster 的 20 t 蒸汽缓冲区，类型为 **FluidPort**。使用供应方的稳定机器 ID 或显示名称调用 `connect(...)`；已完成的气体管道网络会在前哨站之间输送蒸汽。

`self.water_in`

通过 20 t 水缓冲区为 Caster 的冷却控制供水。使用 Water Pump、Steam Condenser、water Liquid Tank 或 Large Liquid Tank 的稳定机器 ID 或显示名称调用 `self.water_in.connect(...)`。本地源会直接传输；远程源还需要在两个地点之间完成无冲突的 Liquid Pipe 路由。空闲、已充满或断电时连接仍保持。有关液位、容量和流量查询，请参见 FluidPort。

返回值

Caster 的 20 t 水缓冲区，类型为 **FluidPort**。使用供应方的稳定机器 ID 或显示名称调用 `connect(...)`；已完成的液体管道网络会在前哨站之间输送水。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` 仅限自身

读取脚本运行期间从编辑器 Commands 选项卡发送的命令。所有可脚本化机器均相同，参见指南：Script Commands。

组件 / 地形改造

Bio Collector

自动执行生物学循环中的 Collect 步骤，自行将野外标本取入货舱。在脚本告诉它收集位置前，它不会执行任何操作。

类型 Atmosphere

输入功率 -5 W（从电网汲取）

获取方式

- 在 Shop 中以 3,000 cr 购买。

由 `self/get_component(id)` 返回。

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ① 可找到精确 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件的稳定编程标识符。将其与 `get_component(id)` 以及要求组件、行星、载具、空间站或订单 ID 的 API 搭配使用。

返回值

String

`.name`

面向人类的显示名称。需要经受重命名的脚本应优先使用 `.id`。

返回值

String

方法

`.outpost`

此建筑所部署的前哨站。返回的 `OutpostRef` 包含其稳定 ID、显示名称、生物群系、位置、容量以及 `buildings()` 查询。需要当前值时重新读取该属性。

返回值

部署此建筑的前哨站的 `OutpostRef`。

`self.scan()`

列出此处哨站生物群系中的碎片位置，按距离由近到远排列。每个 `FragmentLocation` 包含 `coords`、距离以及是否已编目。已编目位置还会显示其碎片 ID、名称和稀有度；未知位置将这些详情保留为 `None`。若要识别未知位置，使用 `bio_collector.collect(location.coords)` 收集它，然后在 Bio Lab 中分析。这里不会显示配方和生物身份。已分析的碎片也会出现在 `journal.cataloged_fragments(...)` 中。

返回值

`FragmentLocation` 列表：此处哨站生物群系中的每个碎片，包含坐标、直线距离和编目状态（该碎片分析后为 `True`）。已编目结果公开 `.fragment_id`、`.name` 和 `.rarity`；未知结果将它们保留为 `None`。配方和生物身份仍隐藏。按距离由近到远排序。

`self.collect(coords)` 仅限自身

取回 `scan()` 返回的 `coords` 处的碎片，并将其放入收集器货舱。根据距离不同，每个生物群系的行程约需 0.1-0.3 h；收集完成前脚本会暂停。

参数

名称 | 类型 | 描述

`coords` | `list` | 来自 `scan()` 的世界坐标 `[x, y]`。

返回值 | 结果字段 | 成功载荷

`ActionResult` | `.status`、`.message` | `None`

结果

状态 | 类型 | 含义

"ok" | 成功 | 操作成功完成。

"busy" | 暂态 | 组件正在执行其他操作。

"cargo_occupied" | 拒绝 | 货舱已经包含不兼容的项目或材料。

"invalid_coords" | 拒绝 | 提供的坐标无效。

"no_fragment" | 拒绝 | 没有可供该操作使用的碎片。

`self.discard()` 仅限自身

丢弃当前装在收集器货舱中的标本。当收集器拾取了不想发送到 Bio Lab 的标本时使用。

返回值 | 结果字段 | 成功载荷

`ActionResult` | `.status`、`.message` | `None`

结果

状态 | 类型 | 含义

"ok" | 成功 | 操作成功完成。

"empty" | 拒绝 | 相关源或队列为空。

"busy" | 暂态 | 组件正在执行其他操作。

`self.cargo`

等待转移的已收集 Specimen，或槽位为空时为 `None`。分析前，`cargo.fragment_id/cargo.rarity/cargo.recipe` 为 `None`；`cargo.coords` 和 `cargo.distance` 始终可用。

返回值

已收集的 Specimen，为空时为 `None`。Bio Lab 使用 `self.take_from(collector)` 取走后，该值会被清除。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` 仅限自身

读取脚本运行期间从编辑器 Commands 选项卡发送的命令。所有可脚本化机器均相同，参见指南：Script Commands。

组件 / 地形改造

Bio Conditioner

根据固定规则手册检查深层生物群系碎片，一次向脚本询问一个属性。必须正确判断每个属性才能通过；一次错误调用就会烧毁整个标本。

类型 Atmosphere

输入功率 -25 W（从电网汲取）

输入缓冲区 10 units

输出缓冲区 10 units

获取方式

- 需要 Deep-Sea Conditioning 研究（Temperature 1,200）。
- 在 Shop 中以 225,000 cr 购买。

由 `self/get_component(id)` 返回。

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ① 可找到精确 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件的稳定编程标识符。将其与 `get_component(id)` 以及要求组件、行星、载具、空间站或订单 ID 的 API 搭配使用。

返回值

String

`.name`

面向人类的显示名称。需要经受重命名的脚本应优先使用 `.id`。

返回值

String

方法

`.outpost`

此建筑所部署的前哨站。返回的 `OutpostRef` 包含其稳定 ID、显示名称、生物群系、位置、容量以及 `buildings()` 查询。需要当前值时重新读取该属性。

返回值

部署此建筑的前哨站的 `OutpostRef`。

`self.report()`

已加载碎片的完整状态报告，`self.report()` 返回全部 10 个属性的 `{property: value}`：`glow`、`brightness`、`smell`、`gunk`、`cracks`、`feel`、`twitch`、`bugs`、`weight`、`sound`。应读取完整报告：组合规则需要关联属性（`brightness` 读取 `glow`，`weight` 读取 `gunk`，`sound` 读取 `cracks`）。单词属性为字符串，数字属性为数字。未加载任何内容时为 `{}`。使用 `.items()` 迭代，或索引 `report["gunk"]`。

返回值

已加载碎片的全部十个状态属性的字典；为空时为 `{}`。单词值为字符串，数值为数字。必须读取完整报告，因为 `brightness` 使用 `glow`，`weight` 使用 `gunk`，`sound` 使用 `cracks`。

`self.properties()`

按固定检查顺序列出十个属性 ID，从 "glow" 到 "sound"。所有 Deep 碎片的顺序都相同，适合通用检查循环。

返回值

按固定顺序排列的 10 个属性 ID 列表（glow、brightness、…、sound）。所有 Deep 碎片使用相同的检查规则。

`self.fragment()`

加载在腔室中的原始 Deep 碎片 ID，`self.fragment()`；为空时为 `None`。

返回值 | 可能的值

当前腔室中加载的原始 Deep 碎片 ID（来自 load），为空时为 `None`。| "gw_spinal_vertebra"、"vc_eye_stalk"、"oc_lens_eye"、"bw_tail_spike"、"vd_tendrill"、"mh_stigmatic_disc"、"hs_compound_eye"、"ms_eye_cluster"、"hc_tentacle_crown"、"ma_luminous_ring"、"gm_antennal_whip"、"fs_holdfast_rootlet"、"sd_talon"、"ce_cephalic_photophore"、"st_carapace_neural"、"vm_ventral_photophore"

`self.stage()`

当前质量控制（QC, Quality Control）阶段，`self.stage()` 在运行期间返回 1-5；没有运行时返回 0（未加载任何内容，或运行刚刚结束）。每个阶段会询问一个属性。

返回值

运行期间返回当前 QC 阶段 1-5；没有运行时返回 0（未加载任何内容，或运行刚刚结束）。每个阶段会询问一个属性。

`self.current()`

当前阶段正在询问的属性，`self.current()` 返回 10 个 ID 中的一个（在 `report()` 中查找其值，应用规则，然后调用 `accept()/reject()`）；没有运行时为 `None`。无法预测 10 个属性中的哪 5 个会出现，因此应编码全部规则。

返回值 | 可能的值

当前阶段正在询问的属性 ID（10 个之一）：在 `report()` 中查找其值，应用规则，然后调用 `accept()` 或 `reject()`；没有运行时为 `None`。| "glow"、"brightness"、"smell"、"gunk"、"cracks"、"feel"、"twitch"、"bugs"、"weight"、"sound"

`self.lights()`

截至目前的 5 个阶段结果, `self.lights()` 返回由 "green" (调用正确)、"red" (失误, 运行结束) 和 "pending" (尚未到达) 组成的列表。回答时指示灯会逐个亮起。

返回值

截至目前的 5 个阶段结果列表: "green" (调用正确)、"red" (失误, 运行结束) 或 "pending" (尚未到达)。回答时指示灯会逐个亮起。

`self.is_running()`

5 阶段运行进行期间为 `True` (已加载碎片且仍有剩余阶段), `self.is_running()`。使用 `while cond.is_running(): prop = cond.current(); ...` 通过整个挑战。运行完成、烧毁或未加载任何内容时变为 `False`。

返回值

5 阶段运行进行期间为 `True` (已加载碎片且仍有剩余阶段)。循环写法: `while cond.is_running(): ...`。运行完成、烧毁或未加载任何内容时变为 `False`。

`self.load(fragment_id, properties=None, property_match=None)` 仅限自身

从 `self.input` 中取出一个原始 Deep 样本放入腔室，并开始新的 5 阶段运行。可选的 `properties` 和 `property_match` 使用标准的 `any`、`subset` 或 `exact` 约定来选择特定身份。

参数

名称 | 类型 | 描述

`fragment_id` | `string` | 在 `self.input` 中暂存的原始 Deep 碎片 ID。

`properties` | `any` | 可选属性字典，默认按子集匹配。省略或为 `None` 表示匹配任意属性；与 `property_match="exact"` 一起使用 `None` 可只选择无属性项目。

`property_match` | `string` | 可选选择模式：`any`、`subset` 或 `exact`。

返回值 | 结果字段 | 成功载荷

`ActionResult` | `.status`、`.message` | `None`

结果

状态 | 类型 | 含义

"ok" | 成功 | 操作成功完成。

"chamber_occupied" | 拒绝 | 处理腔室已经包含项目。

"not_in_input" | 拒绝 | 请求的项目不在输入中。

"invalid_fragment" | 拒绝 | 提供的碎片标识符无效。

"invalid_properties" | 拒绝 | 提供的项目属性选择器无效。

"invalid_property_match" | 拒绝 | 提供的属性匹配模式无效。

"busy" | 暂态 | 组件正在执行其他操作。

self.eject() 仅限自身

将未改变的腔室样本放入 `self.output` 并结束运行。

返回值 | 结果字段 | 成功载荷

`ActionResult` | `.status`、`.message` | `None`

结果

状态 | 类型 | 含义

"ok" | 成功 | 操作成功完成。

"empty" | 拒绝 | 相关源或队列为空。

"busy" | 暂态 | 组件正在执行其他操作。

"output_full" | 拒绝 | 输出端没有足够容量容纳结果。

self.accept() 仅限自身

将当前属性标记为通过。

返回值 | 结果字段 | 成功载荷

`ActionResult` | `.status`、`.message` | `None`

结果

状态 | 类型 | 含义

"ok" | 成功 | 操作成功完成。

"conditioned" | 成功 | 标本完成处理。

"burned" | 成功 | 处理判断错误，标本被烧毁且腔室清空。

"no_run" | 拒绝 | 没有活动中的运行。

"busy" | 暂态 | 组件正在执行其他操作。

"output_full" | 拒绝 | 输出端没有足够容量容纳结果。

`self.reject()` 仅限自身

将当前属性标记为损坏。

返回值 | 结果字段 | 成功载荷

`ActionResult` | `.status`、`.message` | `None`

结果

状态 | 类型 | 含义

"ok" | 成功 | 操作成功完成。

"conditioned" | 成功 | 标本完成处理。

"burned" | 成功 | 处理判断错误，标本被烧毁且腔室清空。

"no_run" | 拒绝 | 没有活动中的运行。

"busy" | 暂态 | 组件正在执行其他操作。

"output_full" | 拒绝 | 输出端没有足够容量容纳结果。

`self.input`

用于原始 Deep 样本的 `InputSlot`。Inventory 仅在 Nocturna Base 作为源；远程 Conditioner 使用同一前哨站的 Storage Bin/Warehouse。

返回值

原始 Deep 样本的 `InputSlot`。加载到腔室前，使用 `eject(...)` 将误选的属性变体取回到明确的本地目的地。

`self.output`

用于处理完成或弹出样本的 `OutputSlot`。样本的确切属性会被保留。

返回值

用于处理完成或弹出样本、并保留确切属性的 `OutputSlot`。

组件 / Terraform

生物交换（Bio Exchange）

交付生物样本以完成生物订单（Bio Order），它是供应码头（Supply Dock）在生物系统中的对应设备。脚本会分配订单，并持续交付所需的碎片，直到获得奖励。

类型	大气（Atmosphere）
输入功率	-8 W（从电网取电）
输入缓冲	10 单位
输出缓冲	10 单位

获取方式

- 在商店（Shop）中以 4,000 cr 购买。

返回方式：`self/get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，即可找到确切 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件的稳定程序化标识符。将它与 `get_component(id)` 以及要求组件、行星、载具、空间站或订单 ID 的 API 搭配使用。

返回值

String

`.name`

人类可读的显示名称。需要让脚本经受重命名时，优先使用 `.id`。

返回值

String

方法

`.outpost`

部署此建筑的前哨站。返回的 `OutpostRef` 包含其稳定 ID、显示名称、生物群系、位置、容量以及 `buildings()` 查询。需要当前值时，请重新读取该属性。

返回值

部署此建筑的前哨站的 `OutpostRef`

`self.orders()`

列出所有 `BioOrder`，包括当前还无法完成的订单。每个订单都包含其 ID、生物群系、需求、奖励、状态、已交付样本、已在运输中的样本、完成百分比，以及所需的 `target_glow`。所有服务该订单的生物交换共享进度。使用 `requires - delivered - in_transit`，避免制造已经提交的样本，然后将选定的 `order.id` 传给 `set_order(...)`。从其他脚本调用 `get_component("bio_exchange_1").orders()`。`get_component("orders")` 用于地球订单（Earth Orders）。

返回值

所有 `BioOrder` 的列表。

已交付和在运输中的数量由所有生物交换共享。其他脚本可以查询已部署的交换设备；地球订单则来自 `get_component("orders")`。

self.set_order(order_id) — 仅限 SELF ONLY

选择此交换设备要完成的生物订单。`self.set_order("bio_order_03")`。多个交换设备可以激活同一个生物订单；它们会协作处理同一个共享交付计数，因此大型生物订单可以同时由多个前哨站完成。

参数

名称	类型	描述
order_id	string	来自 <code>orders()</code> 的生物订单 ID（例如 "bio_order_01"）。

返回值

返回类型	结果字段	成功负载
ActionResult	.status、.message	None

结果

状态	类型	含义
"ok"	成功	操作成功完成。
"unknown_order"	拒绝	提供的订单未知，或当前不可分配。
"completed"	成功	请求的订单已完成，无法再次分配。

self.clear_order() — 仅限 SELF ONLY

清除此交换设备当前的生物订单分配。已经交付的进度仍记录在生物订单上；已完成的生物订单仍保持完成状态，并且不会移动任何样本。

返回值

返回类型	结果字段	成功负载
ActionResult	.status、.message	None

结果

状态	类型	含义
"ok"	成功	操作成功完成。
"busy"	暂时性	组件正在执行其他操作。

`self.active_order()`

返回调用时活动 `BioOrder` 对象的快照，包括 `requires`、已完成的 `delivered` 进度、实时的 `in_transit` 承诺、`percent` 和 `target_glow`（海岸注入目标）。如果没有设置订单，则返回 `None`。`in_transit` 包括已暂存在服务交换设备输入端的合格样本，以及正在进行的定时交付。再次调用 `active_order()` 可读取最新进度。

返回值

活动 `BioOrder` 对象的快照；如果未设置订单，则为 `None`。重新查询以读取最新的 `delivered`、已承诺的 `in_transit` 和 `percent` 进度。

`self.matches_order(item_id, properties=None)`

检查一个确切的物品是否符合此交换设备的活动生物订单，但不会移动该物品。传入 `ItemStack` 的 ID 和属性。`True` 表示该变体满足订单的辉光、基因、锻造 (Forged)、调理 (Conditioned) 或普通样本要求。对 `Inventory`、`Storage Bin` 或 `Warehouse` 的 `stacks()` 结果使用它，然后执行精确的 `self.input.take(...)`。如果没有活动订单、订单已完成，或物品不合格，则返回 `False`。

参数

名称	类型	描述
<code>item_id</code>	<code>string</code>	来自 <code>ItemStack</code> 的候选碎片 ID。
<code>properties</code>	<code>any</code>	来自同一 <code>ItemStack</code> 的精确属性字典；无属性样本则为 <code>None</code> 。

返回值

布尔值。当此物品-属性身份满足活动生物订单当前的样本契约时为 `True`。在遍历存储组件的 `stacks()` 时使用它，然后调用 `self.input.take(..., properties, "exact")`。没有活动订单、订单已完成，或变体无效/错误时返回 `False`。

self.deliver() — 仅限 SELF ONLY

将一个匹配样本从 self.input 交付到活动生物订单（一个短时定时操作）。所有服务该生物订单的生物交换共享进度。将端口连接到 Nocturna Base 的 Inventory，或连接到其他位置的同一前哨站的 Storage Bin/warehouse，然后使用 take(...) 和 send(...) 路由样本。

返回值

返回类型	结果字段	成功负载
ActionResult	.status、.message	None

结果

状态	类型	含义
"ok"	成功	操作成功完成。
"complete"	成功	操作成功完成。
"no_input"	拒绝	没有可用的输入材料。
"output_full"	拒绝	输出端没有容量接收结果。
"no_active"	拒绝	没有正在进行的同类操作。
"busy"	暂时性	组件正在执行其他操作。

self.lifetime_credits()

此交换设备在所有已完成生物订单中累计获得的总积分。

返回值

此交换设备在所有已完成生物订单中获得的总积分。

self.input

用于提交给 `deliver()` 的样本的 `InputSlot`。在此处连接一个商店。`Inventory` 仅在基地可用；远程交换设备使用本地 `Storage Bin` 或 `Warehouse`。使用 `self.input.eject(destination, item_id, count, properties, "exact")` 回收未使用的样本；目标必须位于本地，且正在进行的交付会继续保留其材料。

返回值

用于带属性样本的 `InputSlot`。`Inventory` 仅在 `Nocturna Base` 可用；远程交换设备使用本地货运。使用 `eject(destination, item_id, count, properties, "exact")` 回收未使用的精确样本。

self.output

当另一个交换设备先完成共享订单时，安全接收多余在途样本的 `OutputSlot`。使用 `send(...)` 排出；样本的精确属性会保留。

返回值

用于接收多余在途样本的 `OutputSlot`；这些样本是因为另一个交换设备先完成共享订单而退回的。

组件 / Terraform

生物实验室（Bio Lab）

自动执行生物循环中的分析（Analyze）和提取（Extract）步骤，研究标本并从中提取可用样本。除非脚本驱动，否则它会保持空闲。

类型	大气（Atmosphere）
输入功率	-5 W（从电网取电）
输入缓冲	30 单位
储存堆	30 单位（混合）

获取方式

- 在商店（Shop）中以 5,000 cr 购买。

返回方式：`self/get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，即可找到确切 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件的稳定程序化标识符。将它与 `get_component(id)` 以及要求组件、行星、载具、空间站或订单 ID 的 API 搭配使用。

返回值

String

`.name`

人类可读的显示名称。需要让脚本经受重命名时，优先使用 `.id`。

返回值

String

方法

.outpost

部署此建筑的前哨站。返回的 `OutpostRef` 包含其稳定 ID、显示名称、生物群系、位置、容量以及 `buildings()` 查询。需要当前值时，请重新读取该属性。

返回值

部署此建筑的前哨站的 `OutpostRef`

`self.take_from(collector)` — 仅限 SELF ONLY

将标本从生物采集器的货舱拉入此实验室的标本舱。源采集器必须与此实验室位于同一前哨站；传入显式采集器引用：`self.take_from(get_component("bio_collector_1"))`。

参数

名称		类型	描述
collector	any	一个 <code>bio_collector</code> 组件引用，或带有字符串 <code>id</code> 字段的字典/类实例。采集器必须仍然存在于同一前哨站；货物从当前采集器读取。	

返回值

返回类型	结果字段	成功负载
<code>ActionResult</code>	<code>.status</code> 、 <code>.message</code>	<code>None</code>

结果

状态	类型	含义
<code>"ok"</code>	成功	标本已转移到生物实验室的标本舱。
<code>"busy"</code>	暂时性	生物实验室正在分析或提取。
<code>"input_occupied"</code>	拒绝	生物实验室的标本舱已包含标本。
<code>"not_found"</code>	拒绝	提供的组件引用不存在。
<code>"source_empty"</code>	拒绝	生物采集器的货舱中没有标本。
<code>"source_busy"</code>	暂时性	生物采集器仍在完成采集行程。
<code>"wrong_outpost"</code>	拒绝	生物采集器属于另一个前哨站。
<code>"invalid_source"</code>	拒绝	提供的组件不是有效的生物采集器。

self.analyze() — 仅限 SELF ONLY

识别实验室当前的碎片并显示其提取配方。在每个生物群系中，分析耗时约 0.1 小时；脚本会暂停直到分析完成。成功分析后，碎片会加入 `journal.cataloged_fragments(planet_id)`。在五个碎片全部编目之前，生物的身份会保持隐藏；之后生物会出现在 `journal.cataloged_creatures(planet_id)` 中。

返回值

返回类型	结果字段	成功负载
AnalyzeResult	.status、.message	.info

结果

状态	类型	含义
"ok"	成功	标本分析成功完成。
"busy"	暂时性	生物实验室正在完成另一个操作。
"input_empty"	拒绝	生物实验室没有可分析的标本。
"invalid_specimen"	拒绝	已装载标本没有可识别的碎片身份，无法分析。

`self.load(reagent_id, qty, properties=None, property_match=None)` — 仅限 SELF ONLY

从 `self.input` 消耗整数数量的试剂，为下一次 `extract()` 暂存这些试剂。`self.load("alkaline_buffer", 4)`。试剂由商店出售；界面购买和 `shop.buy(reagent_id)` 都会将试剂放入基地 `Inventory`。可选的 `properties` 和 `property_match` 使用标准的 `any`、`subset` 或 `exact` 约定来选择特定物品身份。小数或负数数量会引发参数错误。使用不匹配配方的试剂调用 `extract()` 会销毁已装载的试剂。

参数

名称	类型	描述
<code>reagent_id</code>	<code>string</code>	从已连接输入端装载的试剂物品 ID。
<code>qty</code>	<code>number</code>	要装载的整数试剂单位数。
<code>properties</code>	<code>any</code>	可选属性字典，默认按子集匹配。省略或为 <code>None</code> 时匹配任意属性；与 <code>property_match="exact"</code> 一起使用 <code>None</code> ，只选择无属性物品。
<code>property_match</code>	<code>string</code>	可选选择模式： <code>any</code> 、 <code>subset</code> 或 <code>exact</code> 。

返回值

返回类型	结果字段	成功负载
<code>ActionResult</code>	<code>.status</code> 、 <code>.message</code>	<code>None</code>

结果

状态	类型	含义
<code>"ok"</code>	成功	操作成功完成。
<code>"busy"</code>	暂时性	组件正在执行其他操作。
<code>"invalid_reagent"</code>	拒绝	提供的试剂对此操作无效。
<code>"invalid_qty"</code>	拒绝	提供的数量无效。

状态	类型	含义
"invalid_properties"	拒绝	提供的物品属性选择器无效。
"invalid_property_match"	拒绝	提供的属性匹配模式无效。
"insufficient_input"	拒绝	输入端没有所需数量。

self.unload_reagents() — 仅限 SELF ONLY

将所有已装载试剂暂存到 self.output，不接触标本。错误暂存了配方时使用此方法。

返回值

返回类型	结果字段	成功负载
ActionResult	.status、.message	None

结果

状态	类型	含义
"ok"	成功	所有已装载试剂都已暂存到生物实验室的输出端，标本未改变。
"empty"	拒绝	生物实验室没有已装载的试剂。
"busy"	暂时性	生物实验室正在取用、分析或提取。
"output_full"	拒绝	输出端没有容量接收已装载试剂；试剂仍保持装载状态。

self.extract() — 仅限 SELF ONLY

消耗 loaded_reagents，并将 1 个已分析标本的样本放入 self.output，同时保留其精确属性。在每个生物群系中，提取耗时约 $0.1 + 0.05 \times \text{units}$ 小时；脚本会暂停直到提取完成。

返回值

返回类型	结果字段	成功负载
ActionResult	.status、.message	None

结果

状态	类型	含义
"ok"	成功	已装载试剂已被消耗，并将一个保留分析标本精确属性的样本暂存到输出端。
"output_full"	拒绝	输出端没有样本容量；已完成的提取会被保留。
"recipe_mismatch"	拒绝	已装载试剂与分析标本的配方不匹配；试剂会被销毁，而标本会被保留。
"busy"	暂时性	生物实验室正在取用、分析或提取。
"input_empty"	拒绝	生物实验室没有可提取的标本。
"not_analyzed"	拒绝	已装载标本尚未分析。
"invalid_specimen"	拒绝	已装载标本没有可识别的碎片身份。

self.discard() — 仅限 SELF ONLY

丢弃当前标本，并将所有已装载试剂暂存到 self.output。当 analyze() 显示出你不需要的碎片时使用它。

返回值

返回类型	结果字段	成功负载
ActionResult	.status、.message	None

结果

状态	类型	含义
"ok"	成功	操作成功完成。
"input_empty"	拒绝	操作输入中没有适用的物品或材料。
"busy"	暂时性	组件正在执行其他操作。
"output_full"	拒绝	输出端没有容量接收结果。

self.specimen

实验室标本舱中当前的 Specimen，通过 self.specimen 暴露；也可能为 None。读取 self.specimen.stage，区分 "collected"（已采集）和 "analyzed"（已分析）。分析前，其识别字段会隐藏；分析后，其 fragment_id、rarity 和 recipe 会填充。

返回值

标本舱中的当前 Specimen，其 .stage 为 "collected" 或 "analyzed"；为空时为 None。

self.loaded_reagents

为下一次 extract() 暂存的试剂字典 {reagent_id: qty}。使用 .items() 进行检查。

返回值

为下一次 extract() 暂存的试剂字典 {reagent_id: qty}。

`self.input`

用于脚本化试剂路由的 `InputSlot`。它一次容纳一个试剂物品 ID，并会一直锁定到 `load()` 消耗完剩余单位或 `flush()` 丢弃它们为止。`stacks()` 会列出属性不同的变体，并不表示端口接受多种试剂类型。仅在 Nocturna Base 连接 `Inventory`；在其他前哨站连接同一前哨站的 `Storage Bin/Warehouse`。在调用 `load(...)` 前调用 `take(...)`。

返回值

用于脚本化试剂路由的 `InputSlot`。`Inventory` 仅在 Nocturna Base 可用；远程实验室使用本地 `Storage Bin` 或 `Warehouse`。使用 `eject(...)` 回收不需要的暂存试剂。

`self.output`

用于提取出的带属性样本和卸载试剂的 `OutputSlot`。连接任何符合条件的本地物品存储，并使用 `send(...)` 排空。

返回值

保存提取样本和卸载试剂的 `OutputSlot`，精确属性保持不变。

组件 / Terraform

生物发光器（Bio Luminizer）

使用三个内置彩色灯，将海岸碎片的辉光调成目标颜色。灯光会相互渗透，因此脚本需要计算出能命中精确目标的亮度混合值。

类型	大气（Atmosphere）
输入功率	-12 W（从电网取电）
输入缓冲	10 单位
输出缓冲	10 单位

获取方式

- 需要“生物发光注入”（Bioluminescent Infusion）研究（温度 90）。
- 在商店（Shop）中以 60,000 cr 购买。

返回方式：`self/get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，即可找到确切 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件的稳定程序化标识符。将它与 `get_component(id)` 以及要求组件、行星、载具、空间站或订单 ID 的 API 搭配使用。

返回值

String

`.name`

人类可读的显示名称。需要让脚本经受重命名时，优先使用 `.id`。

返回值

String

方法

`.outpost`

部署此建筑的前哨站。返回的 `OutpostRef` 包含其稳定 ID、显示名称、生物群系、位置、容量以及 `buildings()` 查询。需要当前值时，请重新读取该属性。

返回值

部署此建筑的前哨站的 `OutpostRef`

self.load(fragment_id, properties=None, property_match=None) — 仅限 SELF ONLY

从 self.input 中取出 fragment_id 的原始发光海岸样本，放入腔室：self.load("gw_caudal_fin")。可选的 properties 和 property_match 使用标准的 any、subset 或 exact 约定选择特定身份。使用 self.chamber.glow 读取其起始颜色。

参数

名称	类型	描述
fragment_id	string	暂存在 self.input 中的海岸碎片 ID。
properties	any	可选属性字典，默认按子集匹配。省略或为 None 时匹配任意属性；与 property_match="exact" 一起使用 None，只选择无属性物品。
property_match	string	可选选择模式：any、subset 或 exact。

返回值

返回类型	结果字段	成功负载
ActionResult	.status、.message	None

结果

状态	类型	含义
"ok"	成功	操作成功完成。
"chamber_occupied"	拒绝	处理腔室已经包含物品。
"not_in_input"	拒绝	请求的物品不在输入端。
"invalid_fragment"	拒绝	提供的碎片标识符无效。
"invalid_properties"	拒绝	提供的物品属性选择器无效。
"invalid_property_match"	拒绝	提供的属性匹配模式无效。
"busy"	暂时性	组件正在执行其他操作。

self.chamber

当前装载的 ChamberSample，通过 self.chamber 暴露；也可能为 None。读取 .glow 获取起始颜色，读取 .fragment_id 获取物品 ID。

返回值

腔室中的 ChamberSample，包含 .fragment_id 和 .glow；为空时为 None。

`self.lamp_signature(channel)`

灯每增加一个亮度步长所添加的 RGB 单位值 $[r, g, b]$ ，即其杂色。`self.lamp_signature("red")` 大约为 $[6, 1, 1]$ ：主要是红色，但会稍微渗入绿色和蓝色。读取全部三个通道（"red"、"green"、"blue"）以构建要进行求逆的 3×3 矩阵。硬件固定，因此读取一次后重复使用。未知通道返回 `None`。

参数

名称	类型	描述
<code>channel</code>	<code>string</code>	灯光通道："red"、"green" 或 "blue"。

返回值

该通道灯的 RGB 单位值 $[r, g, b]$ （其杂色）；对于未知通道为 `None`。硬件固定：读取一次并重复使用。

`self.glow()`

根据当前灯光设置返回腔室的当前结果辉光 $[r, g, b]$ 。它会立即反映 `set_lamps(...)`，因此可在提交前用它验证求解结果：如果 `self.glow() == target`，则调用 `self.infuse()`。腔室为空时返回 `None`。

返回值

根据当前灯光设置返回腔室当前的结果辉光 $[r, g, b]$ （会立即反映 `set_lamps`）；腔室为空时为 `None`。在调用 `infuse()` 前，将它与订单的 `target_glow` 比较。

self.set_lamps(r, g, b) — 仅限 SELF ONLY

设置三个灯的亮度：`self.set_lamps(7, 13, 4)`。每个值必须是 0-40 的整数；精确答案始终为整数，因此请对计算结果使用 `round()`。小数或超出范围的值会引发参数错误（不会静默向下取整）。脚本停止时会重新闲置并归零。

参数

名称	类型	描述
r	number	红灯强度。
g	number	绿灯强度。
b	number	蓝灯强度。

返回值

返回类型	结果字段	成功负载
ActionResult	.status、.message	None

结果

状态	类型	含义
"ok"	成功	操作成功完成。
"busy"	暂时性	组件正在执行其他操作。

self.infuse() — 仅限 SELF ONLY

以当前辉光在 `self.output` 中生成一个发光样本，保留所有现有属性并添加调好的辉光。

返回值

返回类型	结果字段	成功负载
ActionResult	.status、.message	None

结果

状态	类型	含义
"ok"	成功	操作成功完成。
"output_full"	拒绝	输出端没有容量接收结果。
"empty"	拒绝	相关源或队列为空。
"busy"	暂时性	组件正在执行其他操作。

`self.discard()` 仅限自身

将未改变的舱室样本暂存到 `self.output`。

返回值	结果字段	成功载荷
ActionResult	.status、.message	None

结果

状态	类型	含义
"ok"	成功	操作已完成。
"empty"	拒绝	相关源或队列为空。
"busy"	暂时性	组件正在执行其他操作。
"output_full"	拒绝	输出端没有容量存放结果。

`self.input`

用于原始海岸样本的 `InputSlot`。在 Nocturna Base，库存仅作为源；远程 Luminizer 使用同一前哨站的 Storage Bin/Warehouse。

返回值

用于原始海岸样本的 `InputSlot`。如果错误的属性变体已被暂存，请先用 `eject(...)` 将其恢复到明确的本地目的地，再将其装入舱室。

`self.output`

用于 Luminous 或弹出样本的 `OutputSlot`。样本的确切属性会被保留。

返回值

用于 Luminous 或弹出样本的 `OutputSlot`，并保留确切属性。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` 仅限自身

读取脚本运行期间从编辑器 Commands 选项卡发送的命令。每台可脚本化机器上的内容都相同，详见指南：Script Commands。

组件 / Terraforming

DNA Sequencer

将基因拼接到地热片段中，使片段携带订单所需的内容。你在基因层面工作：载入片段，读取它已有的基因以及所需基因，将目标集合拼接进去，机器会为你合成 DNA。每个片段只能拼接一次。

类型	Atmosphere
输入功率	-20 W（从电网取电）
输入缓冲	10 个单位
输出缓冲	10 个单位

获取方式

- 需要 Gene Sequencing 研究（Pressure 120）。
- 在 Shop 中以 100,000 cr 购买。

返回方式：`self/get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ① 以找到确切 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

该组件的稳定程序化标识符。将其与 `get_component(id)` 以及要求组件、行星、载具、空间站或订单 ID 的 API 一起使用。

返回值

String

`.name`

面向用户的可读显示名称。对于需要在重命名后仍能工作的脚本，优先使用 `.id`。

返回值

String

方法

.outpost

部署该建筑的前哨站。返回的 `OutpostRef` 包含其稳定 ID、显示名称、生物群系、位置、容量以及 `buildings()` 查询。需要当前值时重新读取此属性。

返回值

部署该建筑的前哨站的 `OutpostRef`。

`self.load(fragment_id, properties=None, property_match=None)` 仅限自身

从 `self.input` 中取出 `fragment_id` 指定的地热样本并装入舱室：

```
self.load("gw_cardiac_node")
```

可选的 `properties` 和 `property_match` 根据标准的 `any`、`subset` 或 `exact` 约定选择特定身份。

参数

名称	类型	说明
<code>fragment_id</code>	<code>string</code>	暂存在 <code>self.input</code> 中的地热片段 ID。
<code>properties</code>	<code>any</code>	可选属性字典，默认按子集匹配。省略或为 <code>None</code> 时匹配任意属性；将其与 <code>property_match="exact"</code> 一起使用时，仅选择无属性项目。
<code>property_match</code>	<code>string</code>	可选选择模式： <code>any</code> 、 <code>subset</code> 或 <code>exact</code> 。

返回值：`ActionResult` 结果字段：`.status`、`.message` 成功载荷：`None`

结果

状态	类型	含义
<code>"ok"</code>	成功	操作已完成。
<code>"chamber_occupied"</code>	拒绝	处理舱已经包含一个项目。
<code>"not_in_input"</code>	拒绝	请求的项目不在输入端。
<code>"invalid_fragment"</code>	拒绝	提供的片段标识符无效。
<code>"invalid_properties"</code>	拒绝	提供的项目属性选择器无效。
<code>"invalid_property_match"</code>	拒绝	提供的属性匹配模式无效。
<code>"busy"</code>	暂时性	组件正在执行其他操作。

`self.chamber`

当前载入的 `ChamberFragment`，以 `self.chamber` 暴露；如果为空则为 `None`。读取 `.genes` 和 `.spliced`；第二次拼接会销毁已经拼接过的样本。

返回值

带有 `.fragment_id`、`.genes` 和 `.spliced` 的 `ChamberFragment`；为空时返回 `None`。

`self.genes()`

列出当前舱内片段携带的基因，例如 `["cold_tolerance", "pressure_tolerance"]`；舱室为空时返回 `None`。
`splice()` 需要时间。完成后，片段会移至输出端，舱室变为空。如果输出端已满，拼接后的片段会留在舱室中，直到有可用空间。

返回值

当前舱内片段携带的基因，例如 `["cold_tolerance", "pressure_tolerance"]`；舱室为空时为 `None`。
`splice()` 需要时间。成功送达后舱室会清空；如果完成的拼接在等待输出空间，其新基因会保留在舱室中。

self.gene_catalog()

机器可以拼接的每个基因 ID。self.gene_catalog() 返回完整列表（例如 ["heat_resistance", "acid_resistance", "cold_tolerance", "pressure_tolerance", "toxin_resistance", "radiation_shield"]），这些就是传给 splice() 的有效值。

返回值

机器可拼接的完整基因 ID 列表（例如 ["heat_resistance", "acid_resistance", "cold_tolerance", ...]）；即 splice() 的有效值。

self.splice(genes) 仅限自身

将舱内片段的基因集合替换为恰好是 genes 的集合，然后在保留所有无关属性的同时将其放入 self.output。

参数

名称	类型	说明
genes	list	工程化片段必须携带的基因 ID 列表（来自 gene_catalog() 或订单的 required_genes[fragment_id]）。

返回值：ActionResult 结果字段：.status、.message 成功载荷：None

结果

状态	类型	含义
"ok"	成功	操作已完成。
"destroyed"	成功	目标已被销毁。
"empty"	拒绝	相关源或队列为空。
"unknown_gene"	拒绝	提供的基因标识符不存在。
"busy"	暂时性	组件正在执行其他操作。
"output_full"	拒绝	输出端没有容量存放结果。

`self.discard()` 仅限自身

在不进行拼接的情况下，将舱室片段暂存到 `self.output`。

返回值：`ActionResult` 结果字段：`.status`、`.message` 成功载荷：`None`

结果

状态	类型	含义
"ok"	成功	操作已完成。
"empty"	拒绝	相关源或队列为空。
"busy"	暂时性	组件正在执行其他操作。
"output_full"	拒绝	输出端没有容量存放结果。

`self.input`

用于地热样本的 `InputSlot`。在 Nocturna Base，库存仅作为源；远程 Sequencer 使用同一前哨站的 Storage Bin/Warehouse。

返回值

用于地热样本的 `InputSlot`。如果错误的属性变体已被暂存，请先用 `eject(...)` 将其恢复到明确的本地目的地，再将其装入舱室。

`self.output`

用于拼接或弹出样本的 `OutputSlot`。样本的确切属性会被保留。

返回值

用于拼接或弹出样本的 `OutputSlot`，并保留每一项属性。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` 仅限自身

读取脚本运行期间从编辑器 Commands 选项卡发送的命令。每台可脚本化机器上的内容都相同，详见指南：Script Commands。

组件 / Terraforming

Feed Maker

制作 Habitat 殖民地所食用的生物饲料，工作方式类似于由你选择配方的 Fabricator。每种生物的配方都通过 Biolab 订单解锁，因此它只能制作你已解锁的内容。

类型	Biosphere
输入功率	可变（从电网取电）
产量	取决于所选配方
输出缓冲	50 个单位
储料堆	200 个单位（混合）
配方	可用 16 个

获取方式

- 需要 Wildlife 研究（Plants 2,250,000）。
- 在 Shop 中以 50,000 cr 购买。

返回方式：`self/get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ① 以找到确切 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

该组件的稳定程序化标识符。将其与 `get_component(id)` 以及要求组件、行星、载具、空间站或订单 ID 的 API 一起使用。

返回值：`String`

`.name`

面向用户的可读显示名称。对于需要在重命名后仍能工作的脚本，优先使用 `.id`。

返回值：`String`

方法

`.outpost`

部署该建筑的前哨站。返回的 `OutpostRef` 包含其稳定 ID、显示名称、生物群系、位置、容量以及 `buildings()` 查询。需要当前值时重新读取此属性。

返回值

部署该建筑的前哨站的 `OutpostRef`。

`.list_recipes()`

列出通过 Bio Lab 订单解锁的饲料配方：每种可制作饲料的生物对应一个 `Recipe`。锁定的配方不会出现。每个配方包含其 `tier`、`id`、名称、输入、输出、持续时间和功耗。使用 `for recipe in self.list_recipes(): print(recipe.tier, recipe.id)` 来发现可用内容。

返回值

已解锁 `Recipe` 对象的列表（每个对象对应一种由 Biolab 订单解锁配方的生物饲料）。每个对象具有 `.tier`、`.id`、`.name`、`.inputs`、`.output_item`、`.output_count`、`.duration_game_hours`、`.power_draw`。

`.find_recipe(recipe_id)`

按 ID 查找一个已解锁的饲料配方，无需遍历 `list_recipes()`。如果 ID 未知、仍锁定或属于另一台机器，则返回其 `Recipe` 对象或 `None`。

参数

名称	类型	说明
<code>recipe_id</code>	<code>string</code>	Feed Maker 配方 ID。

返回值

匹配的已解锁 `Recipe`；如果该 Feed Maker 当前不能运行该 ID，则为 `None`。

`.set_recipe(recipe_or_id)` 仅限自身

按 ID 选择要制作的生物饲料，或传入 `list_recipes()` 返回的 `Recipe`，例如：

```
self.set_recipe("craft_feed_salt_tortoise")
```

设置后，只要储料堆中有输入且输出箱有空间，`ProcessingSystem` 就会自动制作。

参数

名称	类型	说明
<code>recipe_or_id</code>	<code>any</code>	饲料配方 ID、 <code>Recipe</code> 对象，或具有字符串 <code>id</code> 字段的字典或类实例。配方必须属于此机器且已解锁。

返回值：`ActionResult` 结果字段：`.status`、`.message` 成功载荷：`None`

结果

状态	类型	含义
<code>"ok"</code>	成功	操作已完成。
<code>"unknown_recipe"</code>	拒绝	提供的配方未知或不适用于此机器。
<code>"offline"</code>	暂时性	组件处于离线状态。
<code>"recipe_locked"</code>	拒绝	所选配方处于锁定状态。
<code>"busy"</code>	暂时性	组件正在执行其他操作。

状态	类型	含义
"material_mismatch"	拒绝	现有材料与请求的材料不匹配。

`.clear_recipe()` 仅限自身

取消所选饲料配方，使 Feed Maker 保持空闲。输入储料堆会继续载入。

返回值：ActionResult 结果字段：`.status`、`.message` 成功载荷：None

结果

状态	类型	含义
"ok"	成功	操作已完成。
"busy"	暂时性	组件正在执行其他操作。
"material_present"	拒绝	现有材料阻止请求的配置变更。

`.get_recipe()`

返回当前配方 ID；未设置配方时返回 ""（或者所设置的配方已不再解锁）。在重新设置前使用它检查状态：`if self.get_recipe() == "": self.set_recipe(...)`。

返回值

字符串：当前配方 ID；未设置配方（或所设置配方已不再解锁）时为 ""。

可能的值

""、"craft_feed_salt_tortoise"、"craft_feed_magmatic_annelid"、"craft_feed_mycelial_husk"、"craft_feed_mantle_strider"、"craft_feed_glasswing_mantis"、"craft_feed_veil_mantle"、"craft_feed_vault_crab"、"craft_feed_tidal_cephalopod"、"craft_feed_bone_walker"、"craft_feed_vent_drifter"、"craft_feed_hive_sentinel"、"craft_feed_hollow_choir"、"craft_feed_ferric_sea_lily"、"craft_feed_crustal_echo"、"craft_feed_glacial_wyrm"、"craft_feed_spire_drake"。

`.get_recipe_inputs()`

返回字典，将每个输入 `item_id` 映射到当前配方每次制作消耗的单位数（未设置配方时为空字典）。遍历它即可知道要补充什么：`for item, qty in self.get_recipe_inputs().items(): self.input.take(item, qty * 5)`。

返回值

字典（`item_id` → `int`）；未设置配方时为字典。遍历它可为正确的生命体和饲料补充库存。

`.get_stockpile()`

返回字典，将输入储料堆中当前的每个 `item_id` 映射到其单位数。读取它可查看制作前已载入的内容。

返回值

字典（`item_id` → `int`）。

`.get_stockpile_used()`

输入储料堆中所有材料的总单位数。与 `get_stockpile_capacity()` 比较，以避免装得过满。

返回值

数字：输入储料堆中所有材料的总单位数。

`.get_stockpile_capacity()`

输入储料堆中所有材料共享的单位上限。上限由一个共同限制决定，多种材料的总和都会计入其中。

返回值

数字：输入储料堆中所有材料共享的单位上限。

`.is_running()`

当制作正在本 tick 中积极推进时为 `True`（已设置配方、输入存在且输出有空间）。当缺少材料、输出已满、空闲或断电时为 `False`。轮询它可以检测停滞的生产线。

返回值

布尔值：制作正在积极推进时为 `True`。

`.get_progress()`

当前制作进度的 0-1 分数。每次制作完成（一批饲料进入输出箱）时重置为 0；如果仍有输入，则重新开始。

返回值

数字（0-1）：当前制作进度。

`.get_output_count()`

输出箱中等待取走的已完成饲料单位数。在输出箱装满前，用 `self.output.send(...)` 将其继续发送（输出箱满会使制作停滞）。

返回值

数字：输出箱中等待取走的已完成饲料单位数。

`.input`

将饲料和生命体载入 Feed Maker。使用 `self.input.connect("Forage Bin")` 连接源，然后用 `self.input.take("forage", 8)` 取入材料。参见 `InputSlot`。

返回值

`InputSlot`：载入饲料和生命体：先执行 `self.input.connect("Forage Bin")`，再执行 `self.input.take("forage", 8)`。这是一个多材料储料堆；机器空闲时使用 `eject(...)` 回收已暂存的材料。

`.output`

将成品饲料发送到箱子或 Habitat 供应链。使用 `self.output.connect("Feed Bin")` 连接目的地，然后用 `self.output.send("feed_salt_tortoise", 10)` 发送饲料。参见 `OutputSlot`。

返回值

`OutputSlot`：继续推出成品饲料：先执行 `self.output.connect("Feed Bin")`，再执行 `self.output.send("feed_salt_tortoise", 10)`。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` 仅限自身

读取脚本运行期间从编辑器 Commands 选项卡发送的命令。每台可脚本化机器上的内容都相同，详见指南：Script Commands。

组件 / Terraforming

Habitat

从暂存在 Habitat 专用本地输入端的 Biology 试剂中复苏一个物种，然后将其繁育成殖民地。每个物种只允许有一个活体殖民地。已建立的殖民地在 Habitat 之间移动时会保留进度。

类型	Biosphere
输入功率	可变（从电网取电）
输入缓冲	50 个单位
储料堆	25 个单位（混合）
Tier	Mk II

获取方式

- 需要 Wildlife 研究（Plants 2,250,000）。
- 在 Shop 中以 100,000 cr 购买。

返回方式：`self/get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ① 以找到确切 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

该组件的稳定程序化标识符。将其与 `get_component(id)` 以及要求组件、行星、载具、空间站或订单 ID 的 API 一起使用。

返回值：`String`

`.name`

面向用户的可读显示名称。对于需要在重命名后仍能工作的脚本，优先使用 `.id`。

返回值：`String`

方法

.outpost

部署该建筑的前哨站。返回的 `OutpostRef` 包含其稳定 ID、显示名称、生物群系、位置、容量以及 `buildings()` 查询。需要当前值时重新读取此属性。

返回值

部署该建筑的前哨站的 `OutpostRef`。

.set_revival_target(creature_id) 仅限自身

选择该 `Habitat` 要准备的目录生物：`self.set_revival_target("salt_tortoise")`。验证过的目标会在脚本停止后以及设置检查或饲养尝试失败后继续保留，因此在殖民地存在之前，`Habitat` 卡片就能显示生物及其实时准备要求。选择另一个有效生物会替换目标，但不会消耗材料。

参数

名称	类型	说明
<code>creature_id</code>	<code>string</code>	要在此 <code>Habitat</code> 中准备的目录生物 ID（例如 <code>"salt_tortoise"</code> ）。

返回值：`ActionResult` 结果字段：`.status`、`.message` 成功载荷：`None`

结果

状态	类型	含义
<code>"ok"</code>	成功	操作已完成。
<code>"occupied"</code>	拒绝	请求的单元格、区域或 <code>Habitat</code> 已被占用。
<code>"species_exists"</code>	拒绝	该物种已经存在一个活体殖民地。
<code>"unknown_creature"</code>	拒绝	提供的生物标识符不存在。
<code>"not_cataloged"</code>	拒绝	请求的生物身份尚未编入目录。

`.revive()` 仅限自身

使用 `self.revive()` 使该 Habitat 选定的复苏目标恢复生命。先至少存放 2 份其饲料：复苏会消耗 1 份，且必须留有 1 份用于饲养。暂存 Habitat 或 `journal.cataloged_creatures(...)` 显示的、按稀有度缩放的确切 Bio Lab 试剂。生物加成不会改变这份一次性配方。检查失败不会消耗任何东西，并会保留选定目标。

返回值：`ActionResult` 结果字段：`.status`、`.message` 成功载荷：`None`

结果

状态	类型	含义
"ok"	成功	操作已完成。
"occupied"	拒绝	请求的单元格、区域或 Habitat 已被占用。
"no_target"	拒绝	此操作未选择目标。
"species_exists"	拒绝	该物种已经存在一个活体殖民地。
"not_cataloged"	拒绝	请求的生物身份尚未编入目录。
"wrong_feed"	拒绝	Habitat 输入端含有另一种生物的饲料。
"insufficient_feed"	拒绝	Habitat 没有足够的所需饲料来开始复苏。
"insufficient_reagents"	拒绝	暂存的试剂不满足所需配方。

`.rehouse(creature_id)` 仅限自身

使用 `self.rehouse("salt_tortoise")` 将已建立的物种殖民地附加到此 Habitat。目的地必须为空，且当前容量必须容纳整个殖民地。同一调用可以直接从另一个 Habitat 移动殖民地，也可以在其原 Habitat 被撤部署后恢复殖民地。种群、生命阶段、繁育进度、Insight 历史和购买的加成都会保留。殖民地无住所期间，成长和活跃的 Wildlife 贡献会暂停。

参数

名称	类型	说明
<code>creature_id</code>	<code>string</code>	要附加到此 Habitat 的已建立生物殖民地（例如 "salt_tortoise"）。

返回值：`ActionResult` 结果字段：`.status`、`.message` 成功载荷：`None`

结果

状态	类型	含义
"ok"	成功	操作已完成。
"occupied"	拒绝	请求的单元格、区域或 Habitat 已被占用。
"no_colony"	拒绝	请求物种不存在活体殖民地。
"not_established"	拒绝	请求的殖民地尚未完成饲养。
"insufficient_capacity"	拒绝	目的地没有足够容量完成请求的操作。
"unknown_creature"	拒绝	提供的生物标识符不存在。

`.set_gas_intake(rate)` 仅限自身

设置气体流入速率，单位为 t/h。气体从已连接的 Gas Tank 抽取（使用 `self.gas_in.connect("gas_tank_1")` 进行连接）并进入围栏的气体储备。这是调节器执行器：读取 `gas_level()`，将其与 `gas_band()` 比较，在低于区间时提高进气量，在高于区间时降低进气量。空闲时设为 0。会限制为 ≥ 0 。

参数

名称	类型	说明
<code>rate</code>	<code>number</code>	从已连接的 Gas Tank 抽取并进入围栏气体储备的气体流入速率，单位为 t/h。调节此值以保持 <code>gas_band()</code> 。

返回值：`ActionResult` 结果字段：`.status`、`.message` 成功载荷：`None`

结果

状态	类型	含义
<code>"ok"</code>	成功	操作已完成。

`.purge_intake(port=None)` 仅限自身

排放一个进料口中的流体，使其能够接受另一种流体。传入 `"gas_in"` 或 `"liquid_in"` 可排放一个进料口；省略参数则同时排放两个进料口，并且只排放实际被阻塞的进料口，从而保留健康的缓冲。进料口会接受抵达的第一种流体，随后拒绝其他流体，因此如果向围栏供应了错误的气体，围栏最终会持有一种无法使用的气体，而且没有办法取入正确的气体。

清除、重新接线，然后进行下一次正确输送即可替换围栏空气：`self.purge_intake()`，随后 `self.gas_in.connect("Sulfur Refiner")`。被排放的流体会被销毁；饲料和殖民地进度不受影响。

参数

名称	类型	说明
<code>port</code>	<code>string</code>	要排放的进料口： <code>"gas_in"</code> 或 <code>"liquid_in"</code> 。省略则排放两个进料口。

返回值：`ActionResult` 结果字段：`.status`、`.message` 成功载荷：`None`

结果

状态	类型	含义
<code>"ok"</code>	成功	操作已完成。
<code>"empty"</code>	拒绝	相关源或队列为空。

`.purge_reserve(medium)` (仅自身)

立即清空所选外壳储备 (enclosure reserve) : `self.purge_reserve("gas")` 或 `self.purge_reserve("liquid")`。流体
会被销毁。另一个储备、入口缓冲区、连接、摄取设置、饲料以及群落进度保持不变。如果摄取口仍然打开，储备会在后续 tick
中重新填充。

参数 (PARAMETERS)

名称 (Name)	类型 (Type)	描述 (Description)
<code>medium</code>	string	要清空的外壳储备: <code>"gas"</code> 或 <code>"liquid"</code> 。所选流体将被销毁。

返回值 (RETURNS)

返回类型	结果字段 (RESULT FIELDS)	成功载荷 (SUCCESS PAYLOAD)
<code>ActionResult</code>	<code>.status</code> 、 <code>.message</code>	None

结果 (OUTCOMES)

状态 (Status)	类型 (Kind)	含义 (Meaning)
<code>"ok"</code>	success	所选外壳储备已清空，其流体已丢弃。
<code>"empty"</code>	success	所选外壳储备本来就是空的。

异常 (RAISES)

异常 (Exception)	条件 (Condition)
<code>ValueError</code>	<code>medium</code> 必须是 <code>"gas"</code> 或 <code>"liquid"</code> 。

`.set_liquid_intake(rate)` (仅自身)

设置液体流入速率，单位为 t/h：从相连的 Liquid
Tank (`self.liquid_in.connect("liquid_tank_1")`) 抽入外壳的液体储备。将其调节到保持 `liquid_band()`
的范围。闲置时为 0。下限钳制为 `>= 0`。

参数 (PARAMETERS)

名称 (Name)	类型 (Type)	描述 (Description)
<code>rate</code>	number	从相连 Liquid Tank 抽入外壳液体储备的液体流入速率，单位为 t/h。将其调节到保持 <code>liquid_band()</code> 。

返回值 (RETURNS)

返回类型	结果字段 (RESULT FIELDS)	成功载荷 (SUCCESS PAYLOAD)
<code>ActionResult</code>	<code>.status</code> 、 <code>.message</code>	None

结果 (OUTCOMES)

状态 (Status)	类型 (Kind)	含义 (Meaning)
<code>"ok"</code>	success	操作成功完成。

`.unlock_bonus(node_id)` (仅自身)

永久购买该生物树中的一个节点。传入 `get_bonus_tree().nodes` 中的节点 ID。1 点 Insight Adaptation (洞察适应) 仅影响该物种。4 点 Insight Breakthrough (洞察突破) 影响所有物种, 并且还要求该来源群落达到 10,000 的种群数量。被拒绝的购买不会消耗任何资源。

参数 (PARAMETERS)

名称 (Name)	类型 (Type)	描述 (Description)
<code>node_id</code>	string	<code>get_bonus_tree().nodes</code> 中的节点 ID。

返回值 (RETURNS)

返回类型	结果字段 (RESULT FIELDS)	成功载荷 (SUCCESS PAYLOAD)
<code>ActionResult</code>	<code>.status</code> 、 <code>.message</code>	None

结果 (OUTCOMES)

状态 (Status)	类型 (Kind)	含义 (Meaning)
<code>"ok"</code>	success	操作成功完成。
<code>"unknown_node"</code>	rejection	提供的 Habitat bonus 节点标识符不存在于该生物。
<code>"already_purchased"</code>	rejection	该 Habitat bonus 节点已经永久购买。
<code>"population_locked"</code>	rejection	本地群落尚未达到所需种群数量。
<code>"insufficient_insight"</code>	rejection	共享 Wildlife Insight (野生动物洞察) 余额低于节点成本。

`.tier()`

永久安装的 Habitat tier (栖息地等级) 整数 (1-2)。Mk II 只会使承载容量翻倍; 繁殖速度和生物学成本来自适应项。

返回值 (RETURNS)

整数: 永久安装的 Habitat tier (1-2)。

`.population()`

当前群落的头数（个体数量）。空 Habitat 或仍处于 Founded（已建立）育成窗口中的 Habitat 会读数为 0；群落建立后，这里的计数以及 Wildlife sensor（野生动物传感器）上的计数才会生效。群落会繁殖至 `carrying_capacity()`，且不会下降。

返回值（RETURNS）

数字：当前群落头数（个体数量）。空 Habitat 或仍处于 Founded 育成窗口（尚未建立/尚未在传感器上计数）时为 0。

`.species()`

所容纳生物的 ID（例如 `"glacial_wyrm"`），或 Habitat 为空时为 `" "`。使用它查找生物，或按物种分支设置调节器。

返回值（RETURNS）/可能值（POSSIBLE VALUES）

字符串：所容纳生物的 ID，或 Habitat 为空时为 `" "`。

可能值：`" "`、`"salt_tortoise"`、`"magmatic_annelid"`、`"mycelial_husk"`、`"mantle_strider"`、`"glasswing_mantis"`、`"veil_mantle"`、`"vault_crab"`、`"tidal_cephalopod"`、`"bone_walker"`、`"vent_drifter"`、`"hive_sentinel"`、`"hollow_choir"`、`"ferric_sea_lily"`、`"crustal_echo"`、`"glacial_wyrm"`、`"spire_drake"`。

`.revival_target()`

通过 `set_revival_target(...)` 明确选中的生物 ID；选择前以及建立后为 `""`。失败的设置检查和育成尝试会保留该值，而 `species()` 只有在复苏实际开始后才不为空。

返回值 (RETURNS) /可能值 (POSSIBLE VALUES)

字符串：为准备工作明确选择的生物 ID；没有选中目标或群落已建立时为 `""`。设置检查或育成尝试失败时会保留所选目标。

可能值：`""`、`"salt_tortoise"`、`"magmatic_annelid"`、`"mycelial_husk"`、`"mantle_strider"`、`"glasswing_mantis"`、`"veil_mantle"`、`"vault_crab"`、`"tidal_cephalopod"`、`"bone_walker"`、`"vent_drifter"`、`"hive_sentinel"`、`"hollow_choir"`、`"ferric_sea_lily"`、`"crystal_echo"`、`"glacial_worm"`、`"spire_drake"`。

`.life_stage()`

群落阶段字符串：`"empty"`、`"founded"`、`"first_breeding"`、`"self_sustaining"`、`"thriving"` 或 `"abundant"`。当自身种群跨过各阶段阈值时，群落会逐级提升；每次提升都会开启更高难度的要求并提高上限。可据此扩展供应：`if self.life_stage() == "thriving": self.set_liquid_intake(...)`。

返回值 (RETURNS) /可能值 (POSSIBLE VALUES)

字符串：上述六个值之一。群落会在自身种群跨过各阶段阈值时逐级提升。

`.is_established()`

群落清除 Founded 育成窗口并开始在传感器上计数后为 True；育成中或为空时为 False。用它控制繁殖逻辑：`if self.is_established(): regulate()`。

返回值 (RETURNS)

布尔值：群落清除 Founded 育成窗口并在传感器上计数后为 True；育成中或为空时为 False。

`.rearing_progress()`

Founded 育成窗口保持在范围内的比例（0-1）。只有每个有效范围都满足时才会增加；达到 1.0 即完成建立。空或已建立时读数为 0；如果某个范围失守（育成失败），会重置为 0。在新复苏期间观察它，以确认调节器保持正常。

返回值 (RETURNS)

0 到 1 的数字：Founded 育成窗口保持在范围内的比例。达到 1.0 即完成建立。空或已建立时为 0。范围失守时重置为 0（育成失败：参见 `rearing_failed()`）。

`.rearing_failed()`

育成尝试失去其范围并使 Habitat 恢复到准备状态后为 True。基因组和所选目标会保留；修复调节器后再次调用 `revive()`。选择目标或重新开始复苏时会清除该状态。

返回值 (RETURNS)

布尔值：育成尝试失去范围并使 Habitat 恢复到准备状态后为 True。基因组和所选目标会保留；修复调节器后再次调用 `revive()`。选择目标或重新开始复苏时清除。

`.brood_size()`

下一次完成的繁殖周期产生的完整个体数。通常为 1；brood bonus（育雏加成）会逐步累积，保证额外个体，并周期性地产生 2 个。没有已建立的群落或没有承载容量空间时读数为 0。

返回值 (RETURNS)

下一次完成的繁殖周期产生的完整个体数。通常为 1；育雏加成会逐步累积，保证额外个体，并周期性地产生 2 个。没有已建立的群落或没有承载容量空间时为 0。

`.breeding_rate()`

在当前种群、剩余容量（headroom）、生命支持、稀有度、适应效果和可用本地输入下，预期每小时产生的个体数。种群超过 10 个后，新增种群带来的速度提升会越来越小；brood yield（育雏产量）和已购买的速度效果始终受其总体上限约束。这是速率读数；`breeding_efficiency()` 仅表示生命支持因子。

返回值（RETURNS）

当前条件下的预期个体数/小时。在建立前或被阻塞时为 0。

`.feed_level()`

部分消耗后剩余的可用饲料单位。只有个体出生时才会自动消耗饲料。使用 `self.input.take(...)` 补充；库存过多没有问题。

返回值（RETURNS）

数字：部分消耗后剩余的可用饲料单位。只有个体出生时才会自动提取饲料。应保持高于下一次出生的成本；库存过多没有问题。

`.gas_level()`

外壳当前持有的气体量，单位为吨。将其与 `gas_band()` 比较，并使用 `set_gas_intake(...)` 调节，使其保持在窗口内。群落繁殖消耗气体时读数下降，摄取时上升。

返回值 (RETURNS)

数字：外壳当前持有的气体量，单位为吨。与 `gas_band()` 比较；使用 `set_gas_intake(...)` 调节。

`.liquid_level()`

外壳当前持有的液体量，单位为吨。将其与 `liquid_band()` 比较，并使用 `set_liquid_intake(...)` 保持在窗口内。

返回值 (RETURNS)

数字：外壳当前持有的液体量，单位为吨。与 `liquid_band()` 比较；使用 `set_liquid_intake(...)` 保持在范围内。

`.get_insight()`

读取共享 Wildlife Insight（野生动物洞察）余额，以及该群落当前和累计的贡献。洞察会直接由种群数量的正向变化产生；静态种群和经过的时间本身不会产生洞察。

返回值 (RETURNS)

`HabitatInsight`：共享余额、该群落当前的产出速率及其累计贡献。

`.get_bonus_tree()`

读取该生物的物种专属 Adaptation（适应）和全物种 Breakthrough（突破），包括每个节点的 `.scope`、`.source_species`、永久购买状态、当前效果是否激活、洞察成本及尚未满足的要求。选择复苏目标后树会出现；即使节点锁定也会继续显示。

返回值（RETURNS）

HabitatBonusTree：一个物种专属 Adaptation 和一个全物种 Breakthrough，包含范围、来源物种、成本、门槛、激活状态和购买状态。

`.get_active_bonuses()`

读取当前影响此 Habitat 的所有已购买节点，包括从其他物种获得的全局 Breakthrough。每个结果都暴露 `.scope` 和 `.source_species`。条件节点在其本地条件不满足时会消失；使用其来源 Habitat 的 `get_bonus_tree()` 检查永久所有权。

返回值（RETURNS）

当前影响此 Habitat 的已购买 **HabitatBonusNode** 对象列表，包括从其他物种获得的全局 Breakthrough。

`.gas_band()`

当前阶段群落的安全气体库存范围 `[low, high]`，单位为吨。在范围内时，繁殖获得完整气体支持；过少会使群落缺乏气体，过多则有毒。在需要气体之前返回空列表。由于群落会成长，范围会收紧，因此每个循环都要读取。

返回值 (RETURNS)

列表 `[low, high]` (吨)：群落当前生命阶段的安全气体库存窗口。`gas_level()` 位于 `[low, high]` 内时，气体支持为 100%；过低或过高都会造成节流。气体尚未成为有效要求的早期阶段返回空列表 `[]`。

`.liquid_band()`

当前阶段群落的安全液体库存范围 `[low, high]`，单位为吨。在需要液体之前返回空列表。由于群落会成长，范围会收紧，因此每个循环都要读取。

返回值 (RETURNS)

列表 `[low, high]` (吨)：群落当前生命阶段的安全液体库存窗口。液体尚未成为有效要求时返回空列表 `[]`。

`.feed_ok()`

当 `required_feed()` 指定的饲料已备货时为 `True`。个体出生时会自动消耗饲料。`False` 表示箱子为空或其中是错误的饲料。缺少饲料会停止繁殖，但不会减少已建立的种群数量。

返回值 (RETURNS)

布尔值：当 `required_feed()` 指定的饲料已备货时为 `True`。个体出生时会自动消耗饲料。`False`（卡片上显示）表示箱子为空或存放的是错误的饲料。

`.gas_ok()`

当满足气体要求或尚未需要气体时为 `True`。`False` 表示液位超出 `gas_band()`，或外壳中装的是错误的气体。如果液位在范围内，还要将 `gas_fluid()` 与 `required_gas()` 比较。

返回值 (RETURNS)

布尔值：气体在范围内（或尚未需要）时为 `True`。`False` 表示以下任一情况：`gas_level()` 超出 `gas_band()`（调整 `set_gas_intake(...)`），或外壳装的是错误气体（`gas_fluid() != required_gas()`；重新管接正确的 exotic gas）。两项都要检查。

`.liquid_ok()`

当满足液体要求或尚未需要液体时为 `True`。`False` 表示液位超出 `liquid_band()`，或外壳中装的是错误的液体。与 `required_liquid()` 比较 `liquid_fluid()` 以确定具体情况。

返回值 (RETURNS)

布尔值：液体在范围内（或尚未需要）时为 `True`。`False` 表示以下任一情况：`liquid_level()` 超出 `liquid_band()`，或装的是错误液体（`liquid_fluid() != required_liquid()`）。两项都要检查。

`.breeding_efficiency()`

当前生命支持倍率（0-100%），不是种群增长率。最弱的有效输入决定它；100% 表示每项输入都已就绪。增长还会按种群动量、稀有度和加成进行缩放。容量只有在已满时才会停止增长，在此之前不会减慢繁殖。糟糕的条件不会减少种群数量。

返回值（RETURNS）

生命支持倍率（0-100%），不是种群增长率。最弱的有效输入决定它；100% 表示每项输入都已就绪。增长还会按种群动量、稀有度和加成缩放。容量只有在已满时才会停止增长，在此之前不会减慢繁殖。糟糕的条件不会减少种群数量。

`.gas_fluid()`

外壳当前持有的 exotic gas（例如 "chlorine"），为空时为 ""。它必须等于 `required_gas()`，气体范围才会计入。如果 `gas_level()` 在范围内但 `gas_ok()` 为 False，则外壳中装的是错误气体。

返回值（RETURNS）/可能值（POSSIBLE VALUES）

字符串：外壳当前持有的 exotic gas（即通过管道输入的流体），为空时为 ""。只有它等于 `required_gas()` 时范围才会计入；如果 `gas_level()` 在范围内但 `gas_ok()` 为 False，就比较二者以发现错误气体。

可能值：""、"steam"、"ammonia"、"swamp_gas"、"raw_sulfur_gas"、"sulfur_gas"、"raw_chlorine"、"chlorine"。

`.liquid_fluid()`

外壳当前持有的 exotic liquid，空时为 ""。它必须等于 `required_liquid()`，液体范围才会计入。

返回值 (RETURNS) /可能值 (POSSIBLE VALUES)

字符串：外壳当前持有的 exotic liquid，空时为 ""。必须等于 `required_liquid()`，范围才会计入；比较二者可诊断错误液体供应。

可能值：""、"water"、"oil"、"frozen_essence"、"coastal_essence"、"geothermal_essence"、"volcanic_essence"、"deep_essence"、"brine"、"raw_cryofluid"、"cryofluid"、"raw_quicksilver"、"quicksilver"。

`.required_feed()`

所选复苏目标或所容纳生物所需的确切饲料 item ID；两者都不存在时为 ""。准备或诊断饲料时，将其与 `self.input.stacks()` 比较。

返回值 (RETURNS) /可能值 (POSSIBLE VALUES)

字符串：所选复苏目标或所容纳生物所需的确切饲料 item ID；两者都不存在时为 ""。在准备或诊断饲料时，与 `self.input.stacks()` 比较。

可能值：""、"feed_salt_tortoise"、"feed_magmatic_annelid"、"feed_mycelial_husk"、"feed_mantle_strider"、"feed_glasswing_mantis"、"feed_veil_mantle"、"feed_vault_crab"、"feed_tidal_cephalopod"、"feed_bone_walker"、"feed_vent_drifter"、"feed_hive_sentinel"、"feed_hollow_choir"、"feed_ferric_sea_lily"、"feed_crustal_echo"、"feed_glacial_wyrm"、"feed_spire_drake"。

`.required_gas()`

群落当前阶段所需的 exotic gas, 例如 "swamp_gas"; 尚未需要气体时为 ""。后期阶段的要求会更高, 因此群落成长时要再次检查。其他气体不能满足所需范围。

返回值 (RETURNS) /可能值 (POSSIBLE VALUES)

字符串: 该生物在当前阶段所需的 exotic gas (某些物种会在后期切换到更深层的 exotic, 例如 "sulfur_gas" → "chlorine"; 已购买的 Adaptation 可能保留基础气体), 尚未需要气体时为 ""。必须精确输入此流体; `gas_fluid()` 必须与之匹配。

可能值: "", "swamp_gas", "ammonia", "sulfur_gas", "chlorine"。

`.required_liquid()`

该生物在当前阶段所需的 exotic liquid, 例如 "brine"; 后期阶段会升级 ("brine" → "cryofluid" → "quicksilver")。 `liquid_fluid()` 必须匹配; 尚未需要液体时为 ""。

返回值 (RETURNS) /可能值 (POSSIBLE VALUES)

字符串: 该生物在当前阶段所需的 exotic liquid (某些物种会在后期切换到更深层的 exotic, 例如 "cryofluid" → "quicksilver"; 已购买的 Adaptation 可能保留基础液体), 尚未需要液体时为 ""。 `liquid_fluid()` 必须与之匹配。

可能值: "", "brine", "cryofluid", "quicksilver"。

`.next_required_gas()`

下一次生命阶段转换后所需的气体；如果下一阶段不需要气体或不存在下一阶段，则为 ""。在种群达到阈值前，与 `next_gas_band()` 一起读取。

返回值 (RETURNS) /可能值 (POSSIBLE VALUES)

字符串：紧接下一次生命阶段转换后所需的确切气体；下一阶段没有气体要求或不存在下一阶段时为 ""。与 `next_gas_band()` 配合，以便提前准备。

可能值：""、"swamp_gas"、"ammonia"、"sulfur_gas"、"chlorine"。

`.next_required_liquid()`

下一次生命阶段转换后所需的液体；如果下一阶段不需要液体或不存在下一阶段，则为 ""。在种群达到阈值前，与 `next_liquid_band()` 一起读取。

返回值 (RETURNS) /可能值 (POSSIBLE VALUES)

字符串：紧接下一次生命阶段转换后所需的确切液体；下一阶段没有液体要求或不存在下一阶段时为 ""。与 `next_liquid_band()` 配合。

可能值：""、"brine"、"cryofluid"、"quicksilver"。

`.next_gas_band()`

下一生命阶段的精确气体窗口 `[low, high]`，单位为吨。如果该阶段不需要气体或群落已经达到 Abundant，则返回 `[]`。与 `next_required_gas()` 配合，提前准备正确的供应和调节器。

返回值 (RETURNS)

列表 `[low, high]` (吨)：紧接下一次生命阶段转换后的精确气体范围。该阶段没有气体要求或不存在下一阶段时返回空列表 `[]`。

`.next_liquid_band()`

下一生命阶段的精确液体窗口 `[low, high]`，单位为吨。如果该阶段不需要液体或群落已经达到 Abundant，则返回 `[]`。与 `next_required_liquid()` 配合。

返回值 (RETURNS)

列表 `[low, high]` (吨)：紧接下一次生命阶段转换后的精确液体范围。该阶段没有液体要求或不存在下一阶段时返回空列表 `[]`。

`.carrying_capacity()`

群落在当前生命阶段的上限。种群会繁殖到该上限并在那里趋于平台；达到上限的群落不消耗任何东西。生命阶段提升会提高基础上限，Habitat Mk II 会使其翻倍。Adaptation 不会改变容量。为空时读数为 0。

返回值 (RETURNS)

数字：群落在当前生命阶段的上限。阶段提升会提高基础值；Habitat Mk II 会使其翻倍。Adaptation 不会改变容量。为空时为 0。

`.headroom()`

在当前上限之前仍可繁殖的个体数 (`carrying_capacity() - population()`，向下取整为至少 0)。0 表示群落已达到上限并处于闲置状态。在 Abundant 之前，要达到下一阶段；在 Abundant Mk I 时，安装唯一的 Habitat Mk II 套件，或培育另一个物种。

返回值 (RETURNS)

数字：在群落达到当前上限前仍可繁殖的个体数 (`carrying_capacity() - population()`，向下取整为至少 0)。0 表示群落已封顶并处于闲置状态。

`.next_stage_population()`

开启下一生命阶段的精确种群数量：250、2,500、25,000 或 175,001。Habitat 为空或已经达到 Abundant 时返回 0。Mk I 在 175,000 处停止，因此进入 Abundant 需要 Mk II。将结果与 `population()` 比较，以便脚本在转换前准备下一种供应。

返回值 (RETURNS)

开启下一生命阶段的整数种群阈值（250、2,500、25,000 或 175,001）。为空或已经达到 Abundant 时为 0。与 `population()` 比较，规划下一次供应变更。

`.next_requirement()`

群落下一阶段会发生什么变化："capacity"、"gas"、"liquid"、"switch_gas"、"switch_liquid"、"tighter_bands"，或在 Abundant 时为 "none"。与 `next_stage_population()` 配合，了解转换发生的时间及需要准备的内容。

返回值 (RETURNS) /可能值 (POSSIBLE VALUES)

下一阶段的变化：更大的 "capacity"、新的 "gas" 或 "liquid"、"switch_gas"、"switch_liquid"、"tighter_bands"，或 Abundant 时的 "none"。与 `next_stage_population()` 配合，在转换前准备。

可能值："capacity"、"gas"、"liquid"、"switch_gas"、"switch_liquid"、"tighter_bands"、"none"。

`.gas_in`

为 Habitat 提供气体。使用 `self.gas_in.connect("gas_tank_1")` 连接 Gas Tank，然后使用 `set_gas_intake(...)` 将气体从该端口调节进入外壳。有关液位、容量、流量和连接查询，请参阅 FluidPort。

返回值 (RETURNS)

FluidPort : `connect()`、`connected_to()`、`level()`、`capacity()`、`flow_rate()`。连接 Gas Tank : `self.gas_in.connect("gas_tank_1")`。储罐会填充该端口；`set_gas_intake(...)` 将其调节进入外壳。

`.liquid_in`

为 Habitat 提供液体。使用 `self.liquid_in.connect("liquid_tank_1")` 连接 Liquid Tank。有关液位、容量、流量和连接查询，请参阅 FluidPort。

返回值 (RETURNS)

FluidPort : `connect()`、`connected_to()`、`level()`、`capacity()`、`flow_rate()`。连接 Liquid Tank : `self.liquid_in.connect("liquid_tank_1")`。

`.input`

为群落储存饲料。使用 `self.input.connect("Feed Bin")` 连接来源，然后使用 `self.input.take("feed_salt_tortoise", 20)` 拉取饲料。复苏后，将 `self.input.stacks()` 与 `required_feed()` 比较；只有群落自身的饲料 item 才有用。参见 `InputSlot`。

返回值 (RETURNS)

`InputSlot`: `connect()`、`take()`、`eject()`、`count()`、`capacity()`、`stacks()`、`connected_to()`。储备群落饲料：先执行 `self.input.connect("Feed Bin")`，然后执行 `self.input.take("feed_salt_tortoise", 20)`。复苏后，将 `stacks()` 与 `required_feed()` 比较；使用 `eject(...)` 将错误饲料回收到本地货运。

`.reagents`

专用于复苏试剂的本地 `InputSlot`。选择复苏目标，然后在该 `Habitat` 的前哨站连接 `Storage Bin` 或 `Warehouse`，并在调用 `revive()` 前备齐卡片上显示的全部五种 `Bio Lab` 试剂。位于 `Nocturna Base` 的 `Habitat` 可以连接 `Base Inventory`；远程 `Habitat` 不能连接。

返回值 (RETURNS)

专用于 `revive()` 消耗的五种 `Biology` 试剂的本地 `InputSlot`。选择目标后，在该 `Habitat` 的前哨站连接存储设施，并备齐卡片上显示的按稀有度缩放的数量。`Base Inventory` 只有在 `Habitat` 位于 `Nocturna Base` 时才是有效来源。使用 `eject(...)` 回收未使用的试剂。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` 仅自身

读取脚本运行期间你从编辑器的 Commands 选项卡发送的命令。所有可脚本化机器上的内容相同，请参阅脚本命令（Script Commands）指南。

组件 / Terraform

热发生器（Heat Generator）

通过产生热量加热行星表面。最佳功率设置会随当天的天气变化，因此脚本会读取环境条件，并将加热器保持在适当的水平。

类型	大气（Atmosphere）
输入功率	可变（从电网汲取）
产出	峰值效率下最高 0.252 heat / day
输入缓冲区	4 units
等级	Mk II、Mk III 和 Mk IV

获取方式（HOW TO OBTAIN）

- 从商店（Shop）购买，价格为 800 cr。

返回方式：`self / get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，找到确切 ID，然后将该值传递给 `get_component(id)`。ID 区分大小写。

属性（PROPERTIES）

`.id`

此组件的稳定程序化标识符。将其与 `get_component(id)` 以及要求提供组件、行星、载具、空间站或订单 ID 的 API 配合使用。

返回值（RETURNS）

`String`

`.name`

人类可读的显示名称。对于需要经受重命名的脚本，优先使用 `.id`。

返回值（RETURNS）

`String`

方法 (METHODS)

.outpost

部署此建筑的前哨站 (Outpost)。返回的 **OutpostRef** 包含其稳定 ID、显示名称、生物群系、位置、容量和 **buildings()** 查询。需要当前值时请重新读取该属性。

返回值 (RETURNS)

部署此建筑所在前哨站的 **OutpostRef**。

.input

Mk IV 等级的燃料棒 (Fuel Rod) 弹匣。连接一个铅罐 (Lead Cask)，让备用燃料棒在此处待命；发生器一次吞入整根燃料棒，并在内部将其消耗。Mk IV 以下为空置且未使用；Mk IV 没有燃料棒时会停止产出，而不是降级。

返回值 (RETURNS)

用于 Mk IV Fuel Rod 供应的 **InputSlot**

弹匣：**connect()**、**take()**、**eject()**、**count()**、**capacity()**、**connected_to()**。Mk IV 以下为空置且未使用。

.steam_in

由 Mk III 套件安装的蒸汽供应端口。使用兼容的蒸汽供应器调用 `connect(...)`，然后检查 `level()`、`capacity()`、`flow_rate()` 或 `connected_to()`。Mk IV 改用 Fuel Rod，但已安装的端口仍然可用。

返回值 (RETURNS)

由 Mk III 套件安装的蒸汽供应 `FluidPort`。使用兼容的蒸汽供应器调用 `connect(...)`，然后使用 `level()`、`capacity()`、`flow_rate()` 或 `connected_to()`。Mk IV 使用 Fuel Rod，但已安装的端口仍然可用。

.set_power(watts) 仅自身

将基础加热功率设置为 0-10；超出该范围的值会被截断。0 会关闭加热。最佳正值设置取决于当前的 `thermal_state()`，因此应在条件变化时使用 `self.set_power(value)` 更新。更高的 Mk 等级会倍增电网消耗，但不会改变最佳基础设置。设置不佳会浪费能量并降低热量产出。

参数 (PARAMETERS)

名称	类型	描述
watts	number	以瓦特为单位的功率消耗

返回值 (RETURNS) `ActionResult`

结果字段 (RESULT FIELDS) `.status`、`.message`

成功载荷 (SUCCESS PAYLOAD) `None`

结果 (OUTCOMES)

状态	类型	含义
"ok"	success	操作成功完成。

`.thermal_state()`

当前每日加热器热状态 (thermal state) , 取值之一为

"clear"、"dust_storm"、"heat_bleed"、"dust_veil"。每种状态都有自己的最佳正值 `set_power()`。状态在当前日期内保持稳定, 仅在新的一天变化, 因此应在每次迭代开始时读取它, 并在字符串变化时进行分支: `if state == "clear": self.set_power(5)` 等。找出四个最佳值是你的任务。

返回值 (RETURNS) `String (clear, dust_storm, heat_bleed, dust_veil)`

可能的值 (POSSIBLE VALUES) "clear"、"dust_storm"、"heat_bleed"、"dust_veil"

`.efficiency()`

当前加热效率 (0-100%)。只有当 `set_power()` 恰好匹配当前 `thermal_state()` 的最佳值时才达到 100%, 并且在其附近会急剧下降 (不是线性下降): 偏离一步约为 31%, 偏离两步或更多时为 10% 的下限。只有功率为 0 时读数才为 0%。扫描正功率值, 读取到 100% 的设置就是每种状态的最佳值。

返回值 (RETURNS) `Number (0-100%)`

`.output()`

当前设置下每小时的热量单位产出速率。热量会在多日内累积以提高表面温度; 传感器速率显示会预测每日总量。数值反映 `efficiency() × tier multiplier`。未通电或没有脚本运行时读数为 0。每次读取都会实时重新计算, 新的 `set_power(...)` 会立即反映出来。

返回值 (RETURNS)

`Number`: 当前热量产出速率 (heat units/h)。实时派生; 新的 `set_power(...)` 会立即反映出来。

`.tier()`

永久安装的 Mk 等级，整数值为 1-4。升级套件会提高该值；临时的 Mk III 蒸汽供应不足不会改变它。诊断供应中或已降级的加热器时，将其与 `effective_tier()` 比较。

返回值 (RETURNS) **Integer**：永久安装的 Mk 等级。

`.is_degraded()`

当 Mk III 套件缺少所需流体输入，机器在本次 tick 回退到上一等级倍率时为 **True**。应用 Mk III 套件后进行检查；如果为 **True**，说明你的 tier-3 加热器暂时按 Mk II 运行；查看 `self.steam_in.level()` 和上游热容量。

返回值 (RETURNS) **Boolean**：当 Mk III 套件缺少所需流体输入、机器在本次 tick 回退到上一等级倍率时为 **True**。

`.effective_tier()`

本次 tick 实际生效的等级：通常为 `tier()`；当 `is_degraded()` 为 **True** 时为上一等级。重新平衡各加热器之间蒸汽流量的脚本应将 `effective_tier()` 与 `tier()` 比较。

返回值 (RETURNS) **Integer**：本次 tick 实际生效的等级：通常为 `tier()`；当 `is_degraded()` 为 **True** 时为上一等级。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` 仅自身

读取脚本运行期间你从编辑器的 Commands 选项卡发送的命令。所有可脚本化机器上的内容相同，请参阅脚本命令（Script Commands）指南。

组件 / Terraform

氧气发生器（Oxygen Generator）

从大气中抽取二氧化碳（CO2）并将其转化为可呼吸氧气，这是使行星变得宜居的核心步骤之一。只有脚本设置了进气量时它才会运行。

类型	大气（Atmosphere）
输入功率	-8 W（从电网汲取）
产出	峰值效率下最高 0.240 ppt / day
输入缓冲区	4 units
等级	Mk II、Mk III 和 Mk IV

获取方式（HOW TO OBTAIN）

- 从商店（Shop）购买，价格为 1,000 cr。

返回方式：`self` / `get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ⓘ，找到确切 ID，然后将该值传递给 `get_component(id)`。ID 区分大小写。

属性（PROPERTIES）

`.id`

此组件的稳定程序化标识符。将其与 `get_component(id)` 以及要求提供组件、行星、载具、空间站或订单 ID 的 API 配合使用。

返回值（RETURNS）`String`

`.name`

人类可读的显示名称。对于需要经受重命名的脚本，优先使用 `.id`。

返回值（RETURNS）`String`

方法 (METHODS)

.outpost

部署此建筑的前哨站 (Outpost)。返回的 **OutpostRef** 包含其稳定 ID、显示名称、生物群系、位置、容量和 **buildings()** 查询。需要当前值时请重新读取该属性。

返回值 (RETURNS) 部署此建筑所在前哨站的 **OutpostRef**。

.input

Mk IV 等级的 Fuel Rod 弹匣。连接一个 Lead

Cask，让备用燃料棒在此处待命；发生器一次吞入整根燃料棒，并在内部将其消耗。Mk IV 以下为空置且未使用；Mk IV 没有燃料棒时会停止产出，而不是降级。

返回值 (RETURNS) 用于 Mk IV Fuel Rod 供应的 **InputSlot**

弹匣：**connect()**、**take()**、**eject()**、**count()**、**capacity()**、**connected_to()**。Mk IV 以下为空置且未使用。

`.water_in`

由 Mk III 套件安装的供水端口。使用兼容的供水器调用 `connect(...)`，然后检查 `level()`、`capacity()`、`flow_rate()` 或 `connected_to()`。Mk IV 改用 Fuel Rod，但已安装的端口仍然可用。

返回值 (RETURNS) 由 Mk III 套件安装的供水 `FluidPort`。使用兼容的供水器调用 `connect(...)`，然后使用 `level()`、`capacity()`、`flow_rate()` 或 `connected_to()`。Mk IV 使用 Fuel Rod，但已安装的端口仍然可用。

`.set_intake(value)` 仅自身

设置本次 tick 的 CO2 进气速率。每次迭代调用 `self.set_intake(atmosphere.get_co2() / 10)`；腔室的峰值效率甜点位正好是环境 CO2 的 1/10。高于或低于该点都会使效率平滑下降；不存在对精度敏感的临界点。

参数 (PARAMETERS)

名称	类型	描述
<code>value</code>	<code>number</code>	请求的 CO2 进气速率

返回值 (RETURNS) `ActionResult`

结果字段 (RESULT FIELDS) `.status`、`.message`

成功载荷 (SUCCESS PAYLOAD) `None`

结果 (OUTCOMES)

状态	类型	含义
<code>"ok"</code>	<code>success</code>	操作成功完成。

.waste()

当前碳废料水平 (0-100)。低于 60 时生产保持洁净；60-100 时线性下降；达到 100 时停转（产出为零）。每次调用 `dump_waste()` 前都检查此值；在 50-60 之间倾倒不会受到惩罚，在其他范围倾倒会损失效率。

返回值 (RETURNS) **Number (0-100)**

.dump_penalty()

上一次 `dump_waste()` 调用造成的当前效率惩罚 (0-1)。0 表示没有有效的废料清除惩罚；0.25 表示产出降低 25%。不当倾倒造成的状态会一直显示在这里，直到下一次调用 `dump_waste()`。

返回值 (RETURNS) **Number (0-1, active waste-clearing efficiency penalty)**

.dump_waste() 仅自身

清除累积的碳废料。当 `waste()` 处于 50-60 的洁净甜点区间时，调用 `result = self.dump_waste()` 进行无惩罚倾倒。惩罚会持续到下一次倾倒。

返回值 (RETURNS) **WasteDumpResult**

结果字段 (RESULT FIELDS) **.status**、**.message**

成功载荷 (SUCCESS PAYLOAD) **.penalty**

结果 (OUTCOMES)

状态	类型	含义
"ok"	success	废料舱已清空。本次倾倒施加了 <code>.penalty</code> 的效率惩罚。

.efficiency()

当前转化效率（0-100%）。当进气量匹配 CO2/10、有 CO2 可用、废料不超过 60 且没有倾倒惩罚时达到 100%。废料超过 60 会降低效率；废料为 100 时生产停转。50-60 是调用 `dump_waste()` 的洁净窗口，在该范围外倾倒会施加效率惩罚，可通过 `dump_penalty()` 读取。

返回值 (RETURNS) **Number** (0-100%)

.output()

当前设置下的 O2 产出速率，单位为 ppt/h。反映 `efficiency() × tier multiplier`，并受可用 CO2 限制。机器未通电、没有脚本运行、CO2 耗尽或废料导致生产停转时读数为 0。每次读取都会实时重新计算，新的 `set_intake(...)` 会立即反映出来。用它创建实时仪表板或检测循环中途的降级。

返回值 (RETURNS) **Number**：当前 O2 产出速率 (ppt/h)。实时派生；新的 `set_intake(...)` 会立即反映出来。

.tier()

永久安装的 Mk 等级，整数值为 1-4。升级套件会提高该值；临时的 Mk III 流体供应不足不会改变它。诊断供应中或已降级的发生器时，将其与 `effective_tier()` 比较。

返回值 (RETURNS) **Integer**：永久安装的 Mk 等级。

`.is_degraded()`

当 Mk III 套件缺少所需流体输入，机器在本次 tick 回退到上一等级倍率时为 `True`。应用 Mk III 套件后、在相信 `output()` 能达到 Mk III 预测值之前进行检查；如果为 `True`，检查 `self.water_in.level()` 和上游管道。

返回值 (RETURNS) `Boolean`：当 Mk III 套件缺少所需流体输入、机器在本次 tick 回退到上一等级倍率时为 `True`。

`.effective_tier()`

本次 tick 实际生效的等级：通常为 `tier()`；当 `is_degraded()` 为 `True` 时为上一等级。决定是否将更多水引向此发生器的脚本应将 `effective_tier()` 与 `tier()` 比较。

返回值 (RETURNS) `Integer`：本次 tick 实际生效的等级：通常为 `tier()`；当 `is_degraded()` 为 `True` 时为上一等级。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` 仅自身

读取脚本运行期间你从编辑器的 Commands 选项卡发送的命令。所有可脚本化机器上的内容相同，请参阅脚本命令 (Script Commands) 指南。

组件 / Terraform

压力发生器（Pressure Generator）

压缩稀薄大气以提高表面压力。当脚本捕捉到仪表扫描的每个同步窗口时，它运行效果最佳；错过窗口会损失效率。

类型	大气（Atmosphere）
输入功率	-7 W（从电网汲取）
产出	峰值效率下最高 0.012 kPa / day
输入缓冲区	4 units
等级	Mk II、Mk III 和 Mk IV

获取方式（HOW TO OBTAIN）

- 从商店（Shop）购买，价格为 900 cr。

返回方式：`self / get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，找到确切 ID，然后将该值传递给 `get_component(id)`。ID 区分大小写。

属性（PROPERTIES）

`.id`

此组件的稳定程序化标识符。将其与 `get_component(id)` 以及要求提供组件、行星、载具、空间站或订单 ID 的 API 配合使用。

返回值（RETURNS）`String`

.name

人类可读的显示名称。对于需要经受重命名的脚本，优先使用 **.id**。

返回值 (RETURNS) **String**

方法 (METHODS)

.outpost

部署此建筑的前哨站 (Outpost)。返回的 **OutpostRef** 包含其稳定 ID、显示名称、生物群系、位置、容量和 **buildings()** 查询。需要当前值时请重新读取该属性。

返回值 (RETURNS) 部署此建筑所在前哨站的 **OutpostRef**。

.input

Mk IV 等级的 Fuel Rod 弹匣。连接一个 Lead

Cask，让备用燃料棒在此处待命；发生器一次吞入整根燃料棒，并在内部将其消耗。Mk IV 以下为空置且未使用；Mk IV 没有燃料棒时会停止产出，而不是降级。

返回值 (RETURNS) 用于 Mk IV Fuel Rod 供应的 **InputSlot**

弹匣：**connect()**、**take()**、**eject()**、**count()**、**capacity()**、**connected_to()**。Mk IV 以下为空置且未使用。

`.water_in`

由 Mk III 套件安装的供水端口。使用兼容的供水器调用 `connect(...)`，然后检查 `level()`、`capacity()`、`flow_rate()` 或 `connected_to()`。Mk IV 改用 Fuel Rod，但已安装的端口仍然可用。

返回值 (RETURNS) 由 Mk III 套件安装的供水 `FluidPort`。使用兼容的供水器调用 `connect(...)`，然后使用 `level()`、`capacity()`、`flow_rate()` 或 `connected_to()`。Mk IV 使用 Fuel Rod，但已安装的端口仍然可用。

`.gauge()`

共振扫描的当前值 (0-100)。每个 tick 上升，并在 100 处回绕。将其与 `next_window_low()` 和 `next_window_high()` 比较；当仪表位于该范围内时，调用 `sync()`。

返回值 (RETURNS) `Number (0-100)`

`.next_window_low()`

当前扫描同步窗口的下边界。与 `next_window_high()` 和 `gauge()` 配合使用；当仪表位于该范围内时调用 `sync()`。扫描回绕至下一个周期时，该值会变化。

返回值 (RETURNS) `Number (lower bound of this cycle's sync window)`

`.next_window_high()`

当前扫描同步窗口的上边界。与 `next_window_low()` 和 `gauge()` 配合使用；当仪表位于该范围内时调用 `sync()`。扫描回绕至下一个周期时，该值会变化。

返回值 (RETURNS) `Number (upper bound of this cycle's sync window)`

`.sync()` 仅自身

尝试同步此扫描。每次扫描的首次调用会计数；在仪表回绕之前的后续调用不执行任何操作。命中（`gauge` 在窗口内）→ 效率 +25%；未命中（在窗口外，或扫描结束前没有同步）→ -10%。

返回值 (RETURNS) `ActionResult`

结果字段 (RESULT FIELDS) `.status`、`.message`

成功载荷 (SUCCESS PAYLOAD) `None`

结果 (OUTCOMES)

状态	类型	含义
"ok"	success	操作成功完成。

`.efficiency()`

当前压缩效率 (0-100%)。命中时上升，未命中时下降，最低为 0。调优良好的脚本会在每个周期命中窗口，使其保持在 100% 或接近 100%。使用它检测脚本是否与不断变化的窗口失去同步。

返回值 (RETURNS) `Number (0-100%)`

`.output()`

当前效率下的 kPa/h 产出速率。与 `efficiency() × tier multiplier` 成正比。共振仪表推进缓慢，因此最好根据 `efficiency()` 和传感器上的每日预测来判断压力进度，而不是根据单次 `output()` 读取。

返回值 (RETURNS) **Number**：当前压力产出速率 (kPa/h)。由 `efficiency()` 和等级倍率实时派生。

`.tier()`

永久安装的 Mk 等级，整数值为 1-4。升级套件会提高该值；临时的 Mk III 流体供应不足不会改变它。诊断供应中或已降级的发生器时，将其与 `effective_tier()` 比较。

返回值 (RETURNS) **Integer**：永久安装的 Mk 等级。

`.is_degraded()`

当 Mk III 套件缺少所需流体输入，机器在本次 tick 回退到上一等级倍率时为 **True**。应用 Mk III 套件后进行检查；如果为 **True**，套件处于回退状态；调查 `self.water_in.level()` 和上游供应。

返回值 (RETURNS) **Boolean**：当 Mk III 套件缺少所需流体输入、机器在本次 tick 回退到上一等级倍率时为 **True**。

`.effective_tier()`

本次 tick 实际生效的等级：通常为 `tier()`；当 `is_degraded()` 为 `True` 时为上一等级。重新平衡各发生器之间水流量的脚本应将 `effective_tier()` 与 `tier()` 比较。

返回值 (RETURNS) `Integer`：本次 tick 实际生效的等级：通常为 `tier()`；当 `is_degraded()` 为 `True` 时为上一等级。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` 仅自身

读取脚本运行期间你从编辑器的 `Commands` 选项卡发送的命令。所有可脚本化机器上的内容相同，请参阅脚本命令 (Script Commands) 指南。

组件 / Terraform

精炼器 (Refiner)

使用焦油作为试剂，将原始奇异原料精炼为生物圈等级的气体或液体。只有不常见和稀有的奇异原料需要精炼；普通原料可直接使用。配方通过生物实验室订单 (Biolab orders) 解锁。

类型	生物圈 (Biosphere)
输入功率	可变 (从电网汲取)
产出	取决于所选配方
输入缓冲区	50 units
配方	可用 4 个

获取方式 (HOW TO OBTAIN)

- 需要“ 奇异畜牧业 (Exotic Husbandry) ”研究 (Wildlife 1,000) 。
- 从商店 (Shop) 购买，价格为 80,000 cr。

返回方式：`self / get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，找到确切 ID，然后将该值传递给 `get_component(id)`。ID 区分大小写。

属性 (PROPERTIES)

`.id`

此组件的稳定程序化标识符。将其与 `get_component(id)` 以及要求提供组件、行星、载具、空间站或订单 ID 的 API 配合使用。

返回值 (RETURNS) `String`

`.name`

人类可读的显示名称。对于需要经受重命名的脚本，优先使用 `.id`。

返回值 (RETURNS) `String`

方法 (Methods)

`.outpost`

此建筑部署所在的前哨站 (Outpost)。返回的 `OutpostRef` 包含稳定 ID、显示名称、生物群系、位置、容量以及 `buildings()` 查询。需要当前值时, 请再次读取该属性。

返回值

部署此建筑所在前哨站的 `OutpostRef`。

`.list_recipes()`

列出通过生物实验室 (Bio Lab) 订单解锁的精炼配方; 每种异域流体对应一个 `Recipe`。未解锁的配方不会出现。每个配方包含 `.tier`, 通过 `.input_fluid` 指明确切的原料, 通过 `.fluid_inputs` 给出每次运行消耗的端口吨数, 通过 `.inputs` 列出焦油, 并通过 `.output_fluid/.fluid_outputs` 标识精炼产物及其输出端口。使用 `for recipe in self.list_recipes(): print(recipe.tier, recipe.id, recipe.input_fluid, recipe.output_fluid)` 可发现当前可用内容。

返回值

已解锁的 `Recipe` 对象列表 (每个可精炼异域流体对应一个对象, 且其配方已由生物实验室订单解锁)。每个对象包含 `.tier`, 通过 `.input_fluid/.fluid_inputs` 标识原料, 再通过 `.output_fluid/.fluid_outputs` 标识精炼产物及端口。

`.find_recipe(recipe_id)`

按 ID 查找一个已解锁的精炼配方，无需循环调用 `list_recipes()`。如果 ID 未知、配方被锁定或属于其他机器，则返回 `None`。

参数

名称	类型	描述
<code>recipe_id</code>	<code>string</code>	Refiner 配方 ID

返回值

匹配的已解锁 `Recipe`；如果此 Refiner 当前无法运行该 ID，则返回 `None`。

`.set_recipe(recipe_or_id)` (仅自身)

按 ID 或传入 `list_recipes()` 返回的 `Recipe`，选择要精炼的异域流体，例如 `self.set_recipe("refine_chlorine")`。设置后，只要原料与焦油存在且输出端口有空间，精炼器就会自动制作。

参数

名称	类型	描述
<code>recipe_or_id</code>	<code>any</code>	Refiner 配方 ID、 <code>Recipe</code> 对象，或带有字符串 <code>id</code> 字段的字典或类实例。配方必须属于此机器且已解锁。

返回值：`ActionResult` 结果字段：`.status`、`.message` 成功载荷：`None`

结果

状态	类型	含义
<code>"ok"</code>	成功	操作成功完成。
<code>"unknown_recipe"</code>	拒绝	提供的配方未知或不适用于此机器。
<code>"offline"</code>	暂时性	组件处于离线状态。
<code>"recipe_locked"</code>	拒绝	所选配方已锁定。
<code>"busy"</code>	暂时性	组件正在执行其他操作。
<code>"output_busy"</code>	暂时性	输出中仍有不兼容流体，阻止更换配方。

`.clear_recipe()`（仅自身）

取消所选精炼配方，使 Refiner 保持空闲。焦油和原料输入会被保留，因为它们属于已暂存的供应品。

返回值：ActionResult 结果字段：`.status`、`.message` 成功载荷：None

状态	类型	含义
"ok"	成功	操作成功完成。
"busy"	暂时性	组件正在执行其他操作。
"material_present"	拒绝	已有材料，无法进行请求的配置更改。

`.purge_input()`（仅自身）

排出 `gas_in` 和 `liquid_in` 中暂存的所有原料，释放端口以便接收其他流体。原料端口会接收最先到达的流体，之后只接受该流体，因此接错 Cap 的端口会保留配方无法使用的流体。排出后重新接线即可继续：`self.purge_input()`，然后执行 `self.gas_in.connect("Raw Sulfur Cap")`。排出的流体会被销毁，输入箱中的焦油不受影响。

返回值：ActionResult 结果字段：`.status`、`.message` 成功载荷：None

状态	类型	含义
"ok"	成功	操作成功完成。
"empty"	拒绝	相关来源或队列为空。
"busy"	暂时性	组件正在执行其他操作。

`.get_recipe()`

返回当前配方 ID；如果未设置配方（或设置的配方已不再解锁），则返回 ""。在重新设置之前可用它检查状态：`if self.get_recipe() == "": self.set_recipe("refine_sulfur_gas")`。

返回值

字符串：当前配方 ID；若未设置或已不再解锁则为 ""。可能的值包括 ""、"refine_sulfur_gas"、"refine_cryofluid"、"refine_chlorine"、"refine_quicksilver"。

`.get_recipe_inputs()`

返回字典，将每个输入 `item_id` 映射到每次制作消耗的数量。对 Refiner 来说，这是焦油成本，例如稀有异域流体为 `{"tar": 5}`。未设置配方时返回空字典。读取它可以保持焦油箱库存：`for item, qty in self.get_recipe_inputs().items(): self.input.take(item, qty * 5)`。（原料流体的用量由输入端口计量，不在此处列出。）

返回值

字典（`item_id` → 每次制作消耗数）；对 Refiner 来说是焦油成本，例如 `{"tar": 5}`。未设置配方时为空字典。原料流体成本位于输入端口，而不在此处。

`.is_running()`

当精炼制作在当前 tick 中实际推进时返回

`True`（已设置且已解锁配方、原料与焦油齐备、输出端口有空间）。停滞、空闲或断电时返回 `False`。

返回值：布尔值。

`.is_stalled()`

当 Refiner 已通电且设置了配方，但制作无法推进时返回 **True**，例如原料不足（检查 `self.gas_in.level() / self.liquid_in.level()`）、焦油耗尽（补充输入箱）或精炼流体输出端口已满（下游反压，排空输出箱）。断电、运行中或未设置配方时返回 **False**。可轮询此方法诊断卡住的生产线。

返回值：布尔值；当 Refiner 已通电且设置了配方但原料、焦油不足或精炼流体输出端口已满时为 **True**，断电、运行中或未设置配方时为 **False**。

`.get_rate()`

返回此 tick 生产的精炼异域流体，单位为 t/h。停滞或空闲时为 **0**。可用它在平衡原料与需求时确认吞吐量。

返回值：数值；当前 tick 生产的精炼异域流体速率，停滞或空闲时为 **0**。

`.get_progress()`

返回当前精炼制作的进度，范围为 0-1。每次制作完成（精炼流体批次进入输出端口）时重置为 **0**；如果原料和焦油仍然充足，则重新开始。

返回值：数值（当前精炼制作进度，0-1）。

`.gas_in`

接收原始异域气体。将装有原始气体的储罐连接起来：`self.gas_in.connect("Raw Chlorine Tank")`。端口的液位、容量和流量查询见 `FluidPort`。

返回值

原始异域气体原料的 `FluidPort`。从气体储罐接线：`self.gas_in.connect("Raw Chlorine Tank")`，然后使用 `.level()`、`.capacity()`、`.flow_rate()`、`.connected_to()`。

`.liquid_in`

接收原始异域液体。将装有原始液体的储罐连接起来：`self.liquid_in.connect("Raw Cryofluid Tank")`。端口的液位、容量和流量查询见 `FluidPort`。

返回值

原始异域液体原料的 `FluidPort`。从液体储罐接线：`self.liquid_in.connect("Raw Cryofluid Tank")`，然后使用 `.level()`、`.capacity()`、`.flow_rate()`、`.connected_to()`。

`.gas_out`

向下游发送 Sulfur Gas 或 Chlorine 等精炼气体。使用 `self.gas_out.connect("Chlorine Tank")` 连接目标储罐。端口的液位、容量和流量查询见 `FluidPort`。

返回值

精炼异域气体输出的 `FluidPort` (Sulfur Gas / Chlorine)。连接到气体储罐：`self.gas_out.connect("Chlorine Tank")`，然后使用 `.level()`、`.capacity()`、`.flow_rate()`、`.connected_to()`。

`.liquid_out`

向下游发送 Cryofluid 或 Quicksilver 等精炼液体。使用 `self.liquid_out.connect("Cryofluid Tank")` 连接目标储罐。端口的液位、容量和流量查询见 `FluidPort`。

返回值

精炼异域液体输出的 `FluidPort` (Cryofluid / Quicksilver)。连接到液体储罐：`self.liquid_out.connect("Cryofluid Tank")`，然后使用 `.level()`、`.capacity()`、`.flow_rate()`、`.connected_to()`。

`.input`

装载精炼时消耗的焦油。使用 `self.input.connect("Tar Bin")` 连接来源，然后用 `self.input.take("tar", 20)` 提取焦油。此槽位只能容纳焦油。另见 `InputSlot`。

返回值

用于装载焦油试剂的 `InputSlot`：先执行 `self.input.connect("Tar Bin")`，再执行 `self.input.take("tar", 20)`。这是单一材料箱；机器空闲时可使用 `eject(...)` 回收已暂存的焦油。

`.peek_command()` · `.next_command()` · `.command_count()` ·
`.clear_commands()` (仅自身)

读取脚本运行期间从编辑器 `Commands` 选项卡发送的命令。所有可脚本化机器都相同，参见指南：Script Commands。

组件 / 生产与存储（Components / Production & Storage）

Fabricator

Fabricator 可同时使用多种精炼材料组装成品部件。脚本选择配方，将每种原料收集到共享库存中，然后排出成品。

项目	值
类型	Mining
电力输入	可变（从电网汲取）
消耗	Steam，缓冲 10 t
消耗	Water，缓冲 10 t
消耗	Oil，缓冲 10 t
输出缓冲	20 units
副产物缓冲	20 units
库存	200 units（混合）
配方	可用 69 个

获取方式

- 需要 Fabrication 研究（Terraform Index 130,000）。
- 在 Shop 中以 1,400 cr 购买。

由以下方式返回：`self/get_component(id)`。

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ① 可查找准确 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件的稳定程序化标识符。将它与 `get_component(id)` 以及要求组件、行星、车辆、空间站或订单 ID 的 API 一起使用。

返回值：`String`

`.name`

人类可读的显示名称。需要经得起重命名的脚本应优先使用 `.id`。

返回值：`String`

方法

`.outpost`

此建筑部署所在的前哨站。返回的 `OutpostRef` 包含稳定 ID、显示名称、生物群系、位置、容量以及 `buildings()` 查询。需要当前值时，请再次读取该属性。

返回值

部署此建筑所在前哨站的 `OutpostRef`。

`.list_recipes()`

返回此 Fabricator 已获得蓝图的所有配方。返回的 `Recipe` 对象包含 `.tier`、`.id`、`.name`、`.inputs`、`.output_item`、`.output_count`、`.duration_game_hours`、`.power_draw`、`.fluid_inputs`（每次运行消耗的吨数）以及可选的副产物字段。尚未获得蓝图的锁定配方不会出现，因此列表反映玩家当前实际能够制作的内容。`sorted(self.list_recipes(), key=lambda recipe: recipe.tier)` 可将可用队列按从基础到高级的顺序排列。

返回值

已解锁的 `Recipe` 对象列表。每个对象包含 `.tier`、`.id`、`.name`、`.inputs`、`.output_item`、`.output_count`、`.duration_game_hours`、`.power_draw`、`.fluid_inputs` 以及可选副产物字段。

`.find_recipe(recipe_id)`

按 ID 查找一个已解锁配方，无需循环调用 `list_recipes()`。如果 ID 未知、配方被锁定或属于其他机器，则返回 `None`。

参数

名称	类型	描述
<code>recipe_id</code>	<code>string</code>	Fabricator 配方 ID

返回值

匹配的已解锁 `Recipe`；如果此 Fabricator 当前无法运行该 ID，则返回 `None`。

`.set_recipe(recipe_or_id)` (仅自身)

选择要组装的配方：`self.set_recipe("craft_gas_pipe_segment")`，或传入 `list_recipes()` 返回的 `Recipe`。设置配方不会清空库存，因此之前配方留下的材料会一直保留，直到被消耗或由 `self.input.flush()` 丢弃。

参数

名称	类型	描述
<code>recipe_or_id</code>	<code>any</code>	配方 ID、 <code>Recipe</code> 对象，或带有字符串 <code>id</code> 字段的字典或类实例。配方必须属于此机器且已解锁。

返回值：`ActionResult` 结果字段：`.status`、`.message` 成功载荷：`None`

状态	类型	含义
<code>"ok"</code>	成功	操作成功完成。
<code>"unknown_recipe"</code>	拒绝	提供的配方未知或不适用于此机器。
<code>"offline"</code>	暂时性	组件处于离线状态。
<code>"recipe_locked"</code>	拒绝	所选配方已锁定。
<code>"busy"</code>	暂时性	组件正在执行其他操作。
<code>"material_mismatch"</code>	拒绝	现有材料与请求的材料不匹配。

`.clear_recipe()`（仅自身）

取消所选配方，使 Fabricator 保持空闲。输入库存会被保留，因为它是通用暂存材料，并非所选配方专属材料。

返回值：`ActionResult` 结果字段：`.status`、`.message` 成功载荷：`None`

状态	类型	含义
"ok"	成功	操作成功完成。
"busy"	暂时性	组件正在执行其他操作。
"material_present"	拒绝	已有材料，无法进行请求的配置更改。

`.get_recipe()`

返回当前配方 ID 字符串；如果未设置配方，则返回空字符串。可用它控制其他逻辑，或在调用 `set_recipe()` 后确认结果。

返回值

字符串（配方 ID 或空字符串）。可能的值为：

```
""
"craft_gas_pipe_segment"
"craft_liquid_pipe_segment"
"craft_power_line_segment"
"craft_gas_pipe_bridge"
"craft_liquid_pipe_bridge"
"craft_power_line_bridge"
"craft_pressure_valve"
"craft_machine_frame"
"craft_circuit_panel"
"craft_control_unit"
"craft_battery_cell"
"craft_thermal_cap_kit"
"craft_turbine_rotor"
"craft_tank_lining"
"craft_water_pump"
"craft_oil_pump"
"craft_lubricant"
"craft_plastic"
"craft_rubber"
"craft_tar"
"craft_reinforced_biopolymer"
"craft_enrichment_compound"
"craft_drone_station_kit"
"craft_drone_station_kit_medium"
"craft_drone_station_kit_large"
"craft_drone_service_station_kit"
"craft_mining_drill_kit"
"craft_mining_drill_industrial_kit"
"craft_mining_drill_heavy_kit"
"craft_drone_small"
"craft_drone_medium"
```

.get_recipe() 的其他可能值：

```
"craft_drone_large"  
"craft_electric_thruster"  
"craft_heli_thruster"  
"craft_cargo_pod_small"  
"craft_cargo_pod_medium"  
"craft_cargo_pod_large"  
"craft_battery_pack"  
"craft_oil_tank_small"  
"craft_oil_tank_medium"  
"craft_oil_tank_large"  
"craft_coolant_loop"  
"craft_neutron_capacitor"  
"craft_seed_maker_kit"  
"craft_plant_terraformer_kit"  
"craft_grow_lamp_kit"  
"craft_sprinkler_kit"  
"craft_dispenser_kit"  
"craft_garbage_disposal_kit"  
"craft_exotic_gas_cap_kit"  
"craft_exotic_spring_tap_kit"  
"craft_fertilizer"  
"craft_fertilizer_mk2"  
"craft_fertilizer_mk3"  
"craft_growth_accelerant"  
"craft_yield_amplifier"  
"craft_plant_terraformer_pack_mk2"  
"craft_grow_lamp_pack_mk2"  
"craft_grow_lamp_pack_mk3"  
"craft_sprinkler_pack_mk2"  
"craft_sprinkler_pack_mk3"  
"craft_habitat_pack_mk2"  
"craft_lead_plate"  
"craft_oxygen_upgrade_pack_mk4"  
"craft_heat_upgrade_pack_mk4"  
"craft_pressure_upgrade_pack_mk4"  
"craft_lead_cask"  
"craft_shield_plating"  
"craft_lightning_rod_kit"
```

`.get_recipe_inputs()`

返回当前配方的输入需求，格式为 `{item_id: count_per_craft}`。未设置配方时返回空字典。可结合 `.keys()` / `.values()` / `.items()` 驱动循环：`for mat, need in self.get_recipe_inputs().items():`
`self.input.connect(bin_for(mat)); self.input.take(mat, need)`。

返回值：字典（`item_id` → 每次制作数量）；未设置配方时为空字典。

`.get_stockpile()`

返回当前库存内容，格式为 `{item_id: count_currently_stored}`。使用 `.items()` 遍历所有材料；直接索引 `self.get_stockpile()["iron_ingot"]` 可读取某一种材料。它对于判断还需要拉取哪些材料至关重要。

返回值：字典（`item_id` → 当前存储数量）。

`.get_stockpile_used()`

返回库存中所有材料的总单位数。将它与 `get_stockpile_capacity()` 比较，可以检测库存是否已满。库存满时，运行中的制作消耗材料之前，进一步输入会被阻止。

返回值：数值（所有材料的单位总数）。

`.get_stockpile_capacity()`

返回所有材料共用的库存上限（此 Fabricator 固定）。该上限可查询而不应硬编码；容量会独立于脚本进行调整。使用 `used / capacity` 可生成填充百分比仪表。

返回值：数值（所有材料共享的单位上限）。

`.is_running()`

制作进行期间返回 `True`。可在调用 `set_recipe()` 前使用它以避免 "busy"，或用于显示状态。直到制作完成前，跨 tick 始终为 `True`。

返回值：布尔值。

`.get_progress()`

返回当前制作完成进度（0-1）。制作完成时重置为 0。可用于进度条，也可通过观察数值下降来检测完成事件。

返回值：数值（0-1）。

`.get_output_count()`

返回输出缓冲区中等待取出的成品单位数。应通过 `self.output.send(...)` 排出；缓冲区装满时，制作会停滞。

返回值：数值（等待取出的成品单位数）。

.input

向多材料库存供料的输入 I/O 端口。库存仅在家中可用；远程 Fabricator 连接本地存储，并且可以为每种原料在不同存储之间切换。传输需要 Auto Feeders 研究。另见 **InputSlot**。

返回值：**InputSlot**：**connect()**、**take()**、**eject()**、**flush()**、**count()**、**capacity()**、**connected_to()**。

.output

成品的输出 I/O 端口。库存仅在家中可用；远程 Fabricator 连接本地 Storage Bin 或 Warehouse。需要 Auto Feeders 研究。另见 **OutputSlot**。

返回值：**OutputSlot**：**connect()**、**send()**、**count()**、**capacity()**、**connected_to()**。

.byproduct

副产物的第二输出端口。大多数配方只通过 **self.output** 产出；炼油配方（lubricant、plastic、rubber）则在每次制作时将焦油输出到此处。API 与 **self.output** 相同：先执行 **self.byproduct.connect("Tar Bin")**，再通过 **self.byproduct.send("tar", count)** 排出。需要 Auto Feeders 研究。副产物箱必须为下一次制作留出空间，否则配方会停滞；必须持续排空该端口，而不能只排空 **self.output**。另见 **OutputSlot**。

返回值：用于副产物的 **OutputSlot**：**connect()**、**send()**、**count()**、**capacity()**、**connected_to()**。会产生副产物的配方（例如炼油会产生焦油）会将其存入此处。

`.steam_in`

供声明了 `fluid_inputs["steam_in"]` 的配方使用的内部蒸汽工艺缓冲区。使用 Thermal Cap 或蒸汽 Gas Tank 的稳定机器 ID 或显示名称调用 `self.steam_in.connect(...)`。本地来源会直接传输；远程来源还需要连接两个位置且无冲突、已完成的气体管道路径。读取 `self.steam_in.level()/ .flow_rate()` 可检测供料不足。缓冲容量为 10 ton。另见 `FluidPort`。

返回值

工艺缓冲区 `FluidPort`。使用供应方的稳定机器 ID 或显示名称调用 `connect(...)`；本地供应方直接传输，远程供应方则使用连接两个前哨站的、已完成的任意气体网络。带有 `fluid_inputs["steam_in"]` 的配方会在制作完成时从此端口消耗流体。

`.water_in`

供声明了 `fluid_inputs["water_in"]` 的配方使用的内部水工艺缓冲区。使用 Water Pump、Steam Condenser、water Liquid Tank 或 Large Liquid Tank 的稳定机器 ID 或显示名称调用 `self.water_in.connect(...)`。本地来源会直接传输；远程来源还需要连接两个位置且无冲突、已完成的液体管道路径。读取 `self.water_in.level()/.flow_rate()` 可检测供料不足。缓冲容量为 10 ton。另见 `FluidPort`。

返回值

工艺缓冲区 `FluidPort`。使用供应方的稳定机器 ID 或显示名称调用 `connect(...)`；本地供应方直接传输，远程供应方则使用连接两个前哨站的、已完成的任意液体网络。带有 `fluid_inputs["water_in"]` 的配方会在制作完成时从此端口消耗流体。

`.oil_in`

供声明了 `fluid_inputs["oil_in"]` 的配方（炼油配方）使用的内部石油工艺缓冲区。使用 Oil Pump、oil Liquid Tank 或 Large Liquid Tank 的稳定机器 ID 或显示名称调用 `self.oil_in.connect(...)`。本地来源会直接传输；远程来源还需要连接两个位置且无冲突、已完成的液体管道路径。读取 `self.oil_in.level()/flow_rate()` 可检测供料不足。缓冲容量为 10 ton。另见 `FluidPort`。

返回值

工艺缓冲区 `FluidPort`。使用供应方的稳定机器 ID 或显示名称调用 `connect(...)`。完整的石油供应方-消费者连接会在可达的物理组件上占用石油，即使 Fabricator 空闲、已满或关闭；仅有机器存在并不会占用石油。带有 `fluid_inputs["oil_in"]` 的配方会在制作完成时从此端口消耗流体。

`.peek_command()` · `.next_command()` · `.command_count()` ·
`.clear_commands()`（仅自身）

读取脚本运行期间从编辑器 Commands 选项卡发送的命令。所有可脚本化机器都相同，参见指南：Script Commands。

燃料装配机（Fuel Assembler）

将原铀和铅板压制成燃料棒或核电池，工作方式类似制造机（Fabricator）。运行时会大量消耗配方电力，因此最好在闪电储能库充满时分段运行。

类型	采矿
输入电力	可变（从电网汲取）
产出	取决于所选配方
输出缓冲区	5 单位
储料堆	40 单位（混合）
配方	2 个可用

获取方式

- 需要燃料装配机研究（温度 10,000）。
- 在商店购买，价格为 225,000 cr。

返回值：`self / get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，找到确切 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件的稳定程序化标识符。将它与 `get_component(id)` 以及要求提供组件、行星、载具、空间站或订单 ID 的 API 一起使用。

返回值

`String`

.name

人类可读的显示名称。需要让脚本在重命名后仍能运行时，优先使用 **.id**。

返回值

String

方法

.outpost

部署此建筑的前哨站。返回的 **OutpostRef** 包含稳定 ID、显示名称、生物群系、位置、容量以及 **buildings()** 查询。需要当前值时重新读取该属性。

返回值

部署此建筑所在前哨站的 **OutpostRef**。

.list_recipes()

列出此机器可运行的已解锁配方，包括每个配方派生出的 **.tier**。燃料棒配方通过 **Vestibule** 的队列到达；核电池配方通过 **Helios** 的队列到达。

返回值

此机器可运行的已解锁 **Recipe** 条目列表，包括每个条目派生出的 **.tier**。

`.find_recipe(recipe_id)`

按 ID 查找一个已解锁的燃料配方，无需遍历 `list_recipes()`。如果 ID 未知、仍锁定或属于其他机器，则返回 `None`；否则返回其 `Recipe` 对象。

参数

名称	类型	说明
<code>recipe_id</code>	<code>string</code>	燃料装配机配方 ID

返回值

匹配的已解锁 `Recipe`；如果此燃料装配机当前无法运行该 ID，则为 `None`。

`.set_recipe(recipe_or_id)` (仅自身)

按 ID 选择燃料棒或核电池配方，或传入 `list_recipes()` 返回的 `Recipe`。

参数

名称	类型	说明
<code>recipe_or_id</code>	<code>any</code>	燃料配方 ID、 <code>Recipe</code> 对象，或带有字符串 <code>id</code> 字段的字典或类实例。配方必须属于此机器且已解锁。

返回值

返回类型	结果字段	成功载荷
<code>ActionResult</code>	<code>.status</code> 、 <code>.message</code>	<code>None</code>

结果

状态	类别	含义
<code>"ok"</code>	成功	操作已成功完成。
<code>"unknown_recipe"</code>	拒绝	提供的配方未知或不适用于此机器。
<code>"recipe_locked"</code>	拒绝	所选配方仍处于锁定状态。
<code>"offline"</code>	暂时性	组件处于离线状态。
<code>"busy"</code>	暂时性	组件已在执行其他操作。
<code>"material_mismatch"</code>	拒绝	现有材料与请求的材料不匹配。

`.clear_recipe()`（仅自身）

当前制作处于空闲状态且输出缓冲区已排空后，释放该配方。

返回值

返回类型	结果字段	成功载荷
ActionResult	.status、.message	None

结果

状态	类别	含义
"ok"	成功	操作已成功完成。
"busy"	暂时性	组件已在执行其他操作。
"material_present"	拒绝	存在材料，无法更改请求的配置。

`.get_recipe()`

已提交的配方 ID；没有配方时为空。

返回值

当前配方 ID；未设置时为空字符串。可能的值为 ""、"craft_fuel_rod"、"craft_nuclear_battery"。

`.get_recipe_inputs()`

以字典 {item_id: count_per_craft} 的形式返回已提交配方的输入需求。未提交配方时返回空字典。

返回值

字典（item_id → 每次制作消耗的数量）；未设置配方时为空字典。

`.is_running()`

当制作确实在推进、拥有电力和输入且输出空间充足时为 True。

返回值

布尔值：本 tick 是否正在推进制作。

`.get_progress()`

当前制作进度，范围为 0-1。断电后进度会保留并恢复。

返回值

数字 0-1：当前制作进度。

`.get_stockpile()`

按物品 ID 返回已暂存的输入，形式类似 `{"raw_uranium": 12, "lead_plate": 4}`。

返回值

字典：按物品 ID 列出的暂存输入材料。

`.get_output_count()`

返回所选配方的成品数量，即当前等待在小型输出缓冲区中的数量。

返回值

数字：等待在输出缓冲区中、属于所选配方的成品数量。

.input

仅从铅桶（Lead Cask）接收原铀，并从普通兼容本地来源接收铅板。该端口一次只能保持一种来源关系，因此在切换材料时需要重新连接。

返回值

InputSlot：用于暂存原铀和铅板。原铀只能来自铅桶；对于铅板，请将这个单来源输入重新连接到普通本地来源。空闲时，普通物品会被回收到本地货运，热货物则通过 **eject(...)** 回收到兼容的铅桶。

.output

输出端口。燃料棒属于热货物，只有铅桶（或其准确的 Supply Dock 订单）接受它们。核电池是普通制造产品，可以发送至兼容的本地存储或主库存（home Inventory）。

返回值

OutputSlot：将燃料棒发送到独立的铅桶（或其准确的 Supply Dock 订单）。核电池是普通制造产品，可以发送到普通兼容本地存储或主库存。

.peek_command() · **.next_command()** · **.command_count()** · **.clear_commands()**（仅自身）

读取脚本运行期间从编辑器 Commands 选项卡发送的命令。所有可脚本化机器的行为相同，参见指南：Script Commands。

组件 / 生产与存储（COMPONENTS / PRODUCTION & STORAGE）

收割机 (Harvester)

一种缓慢的通用地表载具，用于收集散落物品、种植和照料作物、收获 Forage，以及部署固定场地机器。移动和场地作业都需要时间并会积累热量，因此长距离路线需要安排冷却暂停。

类型

收获

返回值：`self`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，找到确切 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件的稳定程序化标识符。将它与 `get_component(id)` 以及要求提供组件、行星、载具、空间站或订单 ID 的 API 一起使用。

返回值

`String`

`.name`

人类可读的显示名称。需要让脚本在重命名后仍能运行时，优先使用 `.id`。

返回值

`String`

方法

.move(sector) (仅自身)

使用 self.move("E14") 向上、下、左或右移动一个扇区；不允许对角移动。行进耗时 0.5 小时，并会暂停脚本。self.get_position() 会立即显示目的地，但在抵达之前物理操作仍处于锁定状态。进入有物品的扇区会增加 1 点热量；进入空扇区会增加 7 点，因此应先扫描。

参数

名称	类型	说明
sector	string	要移动到的相邻网格扇区 ID（一个正交步长）。

返回值

返回类型	结果字段	成功载荷
ActionResult	.status、.message	None

结果

状态	类别	含义
"ok"	成功	操作已成功完成。
"invalid"	拒绝	一个或多个提供的参数超出可接受范围。
"too_far"	拒绝	请求的目标超出交互距离。
"already_here"	拒绝	载具已经位于请求的目的地。
"overheated"	暂时性	组件温度过高，无法执行请求的操作，或执行操作会超过热量上限。
"moving"	暂时性	载具或组件当前正在移动。
"busy"	暂时性	组件已在执行其他操作。

.collect()（仅自身）

将当前扇区中的物品拾取到收割机唯一的手持槽，而不是库存。收集耗时 0.25 小时并会暂停脚本。空扇区同样需要时间，并增加 9 点热量。

返回值

返回类型	结果字段	成功载荷
ScanResult	.status、.message	.id、.name、.value

结果

状态	类别	含义
"ok"	成功	已将 item_id 收集到收割机的手持槽。
"empty"	成功	当前扇区没有可收集物品。
"holding"	拒绝	收割机的手持槽已被占用。
"overheated"	拒绝	收割机已达到热量上限，或本次收集会使其超过上限。
"moving"	暂时性	收割机仍在完成物理移动。
"busy"	暂时性	收割机正忙于另一项场地操作。
"collecting"	暂时性	之前的收集周期仍在进行。

.store()（仅自身）

将收割机手持的物品移入第一个空闲库存槽，释放手持槽以便拾取下一件物品。库存已满时物品会继续被手持；可以丢弃物品或通过出售腾出空间。

返回值

返回类型	结果字段	成功载荷
ItemResult	.status、.message	.item_id

结果

状态	类别	含义
"ok"	成功	操作影响了物品 .item_id。
"empty"	成功	所选物品来源已经为空。
"inventory_full"	拒绝	库存没有容纳手持物品的空间。

`.drop()`（仅自身）

将当前手持物品释放到收割机所在的当前扇区，该格子会再次变得可收集。可用它为之后的拾取暂存物品，或在不存入库存的情况下清空手持槽。

返回值

返回类型	结果字段	成功载荷
<code>ActionResult</code>	<code>.status</code> 、 <code>.message</code>	<code>None</code>

结果

状态	类别	含义
<code>"dropped"</code>	成功	物品已从其来源中移除。
<code>"empty"</code>	拒绝	相关来源或队列为空。
<code>"occupied"</code>	拒绝	请求的格子、扇区或 Habitat 已被占用。
<code>"moving"</code>	暂时性	载具或组件当前正在移动。
<code>"busy"</code>	暂时性	组件已在执行其他操作。

`.get_held()`

返回收割机当前手持的物品 ID；手持槽为空时返回空字符串。执行 `collect()` 前应调用 `self.get_held()`（非空表示槽位忙），执行 `self.store()` 前也应调用它（为空表示没有可存入的物品）。这是任何收集循环的必要第一行。

返回值

`String`（物品 ID；不持有物品时为空）

`.get_position()`

以字符串形式返回当前扇区 ID（例如 `"E14"`）。用它规划下一个 `move()` 目标。移动严格限制为相邻格子，因此脚本需要知道当前位置，才能计算可前往的位置。

返回值

`String`（扇区）

`.get_heat()`

返回精确的当前热量等级（0-100），包括整数热量成本之间的分数冷却。每次 `move()` 都会增加热量，尝试在空扇区执行 `collect()` 也会增加热量；随着游戏时间流逝会持续被动冷却，包括移动和收集期间。移动前应读取此值；如果接近 100 且计划路线要穿过空格（每格 +7 热量），应停下冷却。中途因 "overheated" 失去路线会浪费数小时。

返回值

Number (0-100)

`.get_max_heat()`

返回最大热量容量，始终为 100。达到该值会暂停所有移动和收集，直到热量降到上限以下。将它作为方法公开，以便脚本在不硬编码该数字的情况下处理阈值。

返回值

Number (100)

`.is_overheated()`

当 `heat >= 100` 时为 `True`，相当于 `self.get_heat() >= self.get_max_heat()`。如果你希望让收割机冷却，可以将它作为提前退出保护：`if self.is_overheated(): sleep(1)`。只要游戏时间推进，收割机就会被动冷却。

返回值

Boolean

`.water_level()`

读取收割机车载水箱中当前的水量，单位为吨，包括小数部分。每次浇水使用 1t。与 `water_capacity()` 比较，以便规划从主前哨站水箱补水的时机。

返回值

数字：车载水量，单位为吨，包括小数部分。

.water_capacity()

读取收割机车载最大供水量，当前为 5t。与 water_level() 比较，以检查水箱有多满。

返回值

数字：车载水容量，单位为吨。

.load_seed(seed)（仅自身）

从主库存（位于网格任意位置）转移一颗物种种子到收割机唯一的手持槽。驾驶到空的田地格后，使用相同的种子 ID 调用 plant(seed)。手持另一件物品时无法装载。

参数

名称	类型	说明
seed	string	来自种子制造机的种子物品 ID。装载会从任意位置的库存中转移一颗种子到收割机唯一的手持槽。

返回值

返回类型	结果字段	成功载荷
ActionResult	.status、.message	None

结果

状态	类别	含义
"ok"	成功	操作已成功完成。
"locked"	拒绝	所需功能、配方或操作仍处于锁定状态。
"invalid_seed"	拒绝	提供的物品不是植物种子。
"no_seed"	拒绝	请求的种子不可用。
"holding"	拒绝	收割机唯一的手持槽已经被占用。
"overheated"	暂时性	组件温度过高，无法执行请求的操作，或执行操作会超过热量上限。
"moving"	暂时性	载具或组件当前正在移动。
"busy"	暂时性	组件已在执行其他操作。

.plant(seed)（仅自身）

将收割机手持槽中的实体种子播入当前空格。参数必须标识该手持种子。基础扇区是仓库地面，拒绝播种。播种耗时 0.5 小时，动作完成后种子和新作物才会出现。

参数

名称	类型	说明
seed	string	与收割机手持实体种子匹配的种子物品 ID。

返回值

返回类型	结果字段	成功载荷
ActionResult	.status、.message	None

结果

状态	类别	含义
"ok"	成功	操作已成功完成。
"locked"	拒绝	所需功能、配方或操作仍处于锁定状态。
"invalid_seed"	拒绝	提供的物品不是植物种子。
"no_seed"	拒绝	请求的种子不可用。
"not_empty"	拒绝	相关格子、槽位或组件不为空。
"base_sector"	拒绝	基础扇区是收割机的仓库停机坪，不能种植。
"overheated"	暂时性	组件温度过高，无法执行请求的操作，或执行操作会超过热量上限。
"moving"	暂时性	载具或组件当前正在移动。
"busy"	暂时性	组件已在执行其他操作。

`.deploy(kit)`（仅自身）

在当前空格放置一个受支持的场地机器套件，并从库存消耗一个套件。安装耗时 0.25 小时并会暂停脚本。调用 `deployables()` 查看可用套件 ID。种子制造机和其他前哨站建筑通过库存部署。

参数

名称	类型	说明
<code>kit</code>	<code>string</code>	由当前研究解锁的固定场地机器套件物品 ID。

返回值

返回类型	结果字段	成功载荷
<code>ActionResult</code>	<code>.status</code> 、 <code>.message</code>	<code>None</code>

结果

状态	类别	含义
<code>"ok"</code>	成功	操作已成功完成。
<code>"locked"</code>	拒绝	所需功能、配方或操作仍处于锁定状态。
<code>"no_kit"</code>	拒绝	所需的可部署套件不可用。
<code>"not_empty"</code>	拒绝	相关格子、槽位或组件不为空。
<code>"invalid_kit"</code>	拒绝	提供的可部署套件不适用于此操作。
<code>"overheated"</code>	暂时性	组件温度过高，无法执行请求的操作，或执行操作会超过热量上限。
<code>"moving"</code>	暂时性	载具或组件当前正在移动。
<code>"busy"</code>	暂时性	组件已在执行其他操作。

`.deployables()`

列出当前研究解锁的固定场地机器套件 ID。这是能力列表，而不是实时部署检查：`deploy(...)` 仍会检查库存存量、当前格子、移动状态、热量以及收割机是否忙碌。

返回值

研究解锁的固定场地机器套件 ID 列表。它不会检查库存存量、放置位置、热量、移动状态或忙碌状态。在 Grow Lamp 研究之前为空。

.light()（仅自身）

使用收割机的工作灯照亮当前可种植格，即使该格已经被照亮或被 Grow Lamp 覆盖。每次应用会在工作完成时将处理效果重置为 24 小时，并替换剩余时间。照明耗时 0.25 小时并会暂停脚本。稍后可使用 Grow Lamp 连续覆盖四个正交相邻格（正上、正下、正左和正右）。

返回值

返回类型	结果字段	成功载荷
ActionResult	.status、.message	None

结果

状态	类别	含义
"ok"	成功	操作已成功完成。
"locked"	拒绝	所需功能、配方或操作仍处于锁定状态。
"not_plantable"	拒绝	当前格无法接受请求的植物处理。
"overheated"	暂时性	组件温度过高，无法执行请求的操作，或执行操作会超过热量上限。
"moving"	暂时性	载具或组件当前正在移动。
"busy"	暂时性	组件已在执行其他操作。

.water()（仅自身）

使用收割机车供水为当前可种植格浇水，即使该格已经浇过水或被 Sprinkler 覆盖。每次浇水使用 1 t，并在工作完成时将处理效果重置为 24 小时，替换剩余时间。浇水耗时 0.25 小时并会暂停脚本。可以在本地网格任意位置使用 refill_water()，从主前哨站水箱取水。

返回值

返回类型	结果字段	成功载荷
ActionResult	.status、.message	None

结果

状态	类别	含义
"ok"	成功	操作已成功完成。
"locked"	拒绝	所需功能、配方或操作仍处于锁定状态。
"insufficient_water"	拒绝	收割机车载水量少于处理所需的 1 t。
"not_plantable"	拒绝	当前格无法接受请求的植物处理。
"overheated"	暂时性	组件温度过高，无法执行请求的操作，或执行操作会超过热量上限。
"moving"	暂时性	载具或组件当前正在移动。
"busy"	暂时性	组件已在执行其他操作。

.refill_water()（仅自身）

从主前哨站的水箱为收割机车载水箱补水，可在网格任意位置执行。补水耗时 0.25 小时并会暂停脚本。

返回值

返回类型	结果字段	成功载荷
ActionResult	.status、.message	None

结果

状态	类别	含义
"ok"	成功	操作已成功完成。
"locked"	拒绝	所需功能、配方或操作仍处于锁定状态。
"moving"	暂时性	载具或组件当前正在移动。
"overheated"	暂时性	组件温度过高，无法执行请求的操作，或执行操作会超过热量上限。
"tank_empty"	拒绝	来源水箱为空。
"already_full"	拒绝	目标已经达到满容量。
"busy"	暂时性	组件已在执行其他操作。

.dispense_salt()（仅自身）

使用库存中的一单位盐处理当前可种植格，即使该格已经撒盐或被 Dispenser 覆盖。每次应用会在工作完成时将处理效果重置为 24 小时，并替换剩余时间。撒盐耗时 0.25 小时并会暂停脚本。Water Pump 会将盐作为副产品产出。

返回值

返回类型	结果字段	成功载荷
ActionResult	.status、.message	None

结果

状态		类别	含义
"ok"		成功	操作已成功完成。
"locked"		拒绝	所需功能、配方或操作仍处于锁定状态。
"no_salt"		拒绝	没有可用于处理的盐。
"not_plantable"		拒绝	当前格无法接受请求的植物处理。
"overheated"	暂时性	组件温度过高，无法执行请求的操作，或执行操作会超过热量上限。	
"moving"		暂时性	载具或组件当前正在移动。
"busy"		暂时性	组件已在执行其他操作。

`.fertilize(item_id="fertilizer")`（仅自身）

使用一单位 Fertilizer Mk I、II 或 III 为当前格中正在生长的植物施肥。每单位会在 8 小时内提升产量。同一等级的额外单位会延长效果；切换等级前应等待当前剂量耗尽。施肥会占用收割机 0.25 小时。

参数

名称	类型	说明
<code>item_id</code>	<code>string</code>	用于选择等级的 Fertilizer 物品 ID；省略时使用 Mk I。

返回值

返回类型	结果字段	成功载荷
<code>ActionResult</code>	<code>.status</code> 、 <code>.message</code>	<code>None</code>

结果

状态	类别	含义
<code>"ok"</code>	成功	操作已成功完成。
<code>"invalid_input"</code>	拒绝	提供的输入值无效。
<code>"locked"</code>	拒绝	所需功能、配方或操作仍处于锁定状态。
<code>"no_plant"</code>	拒绝	当前格没有植物。
<code>"already_mature"</code>	拒绝	植物已经成熟，生长处理无法对其产生影响。
<code>"tier_conflict"</code>	拒绝	当前格已经有不同等级的 Fertilizer 效果处于激活状态。
<code>"no_dose"</code>	拒绝	所需处理物品不可用。
<code>"overheated"</code>	暂时性	组件温度过高，无法执行请求的操作，或执行操作会超过热量上限。
<code>"moving"</code>	暂时性	载具或组件当前正在移动。
<code>"busy"</code>	暂时性	组件已在执行其他操作。

.accelerate()（仅自身）

使用一单位 growth_accelerant 为当前格中正在生长的植物施用处理，使其生长速度在 8 小时内翻倍。施用会占用收割机 0.25 小时。该剂量无法影响已经成熟的作物。

返回值

返回类型	结果字段	成功载荷
ActionResult	.status、.message	None

结果

状态	类别	含义
"ok"	成功	操作已成功完成。
"locked"	拒绝	所需功能、配方或操作仍处于锁定状态。
"no_plant"	拒绝	当前格没有植物。
"already_mature"	拒绝	植物已经成熟，生长处理无法对其产生影响。
"no_dose"	拒绝	所需处理物品不可用。
"overheated"	暂时性	组件温度过高，无法执行请求的操作，或执行操作会超过热量上限。
"moving"	暂时性	载具或组件当前正在移动。
"busy"	暂时性	组件已在执行其他操作。

`.amplify()` (仅 SELF)

应用一个 `yield_amplifier`，这是全田地 Forage 输出增益的最终组件（不需要单元格）。应用该增益会使 Harvester 占用 0.25 小时。剂量会在约一个游戏日内耗尽，因此需要重新调用以维持效果。

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
ActionResult	.status、.message	None

OUTCOMES

Status	Kind	Meaning
"ok"	success	操作已完成。
"locked"	rejection	所需功能、配方或操作尚未解锁。
"no_dose"	rejection	所需处理物品不可用。
"overheated"	transient	组件对于请求的操作过热，或执行操作会超出其热量上限。
"moving"	transient	车辆或组件当前正在移动。
"busy"	transient	组件已经在执行其他操作。

`.amplifier_remaining()`

返回全田地 Yield Amplifier（产量增幅器）效果的剩余小时数。每个应用的单位增加 24 小时。如果田地未被增幅，则返回 0。

RETURNS

全田地 Yield Amplifier 效果的剩余小时数。

.uproot() (仅 SELF)

移除当前单元格中的植物，并将一个匹配的种子放入 Inventory（库存）。拔除耗时 0.5 小时，并会暂停脚本。

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
ActionResult	.status、.message	None

OUTCOMES

Status	Kind	Meaning
"ok"	success	操作已完成。
"no_plant"	rejection	当前单元格中没有植物。
"inventory_full"	rejection	Inventory 没有空间容纳结果。
"locked"	rejection	所需功能、配方或操作尚未解锁。
"overheated"	transient	组件对于请求的操作过热，或执行操作会超出其热量上限。
"moving"	transient	车辆或组件当前正在移动。
"busy"	transient	组件已经在执行其他操作。

.undeploy() (仅 SELF)

移除当前单元格中的固定田地机器，并将其套件放回 Inventory（库存）。这是 Grow Lamps（生长灯）、Sprinklers（洒水器）、Dispensers（分配器）和 Crop Automators（作物自动机）的唯一移除路径。其存储物品必须为空，硬件退款必须有空间容纳，且附加脚本不得正在运行。已编写的脚本仍可在 Computer > Scripts 中使用，但不再绑定主机。未完成的机器工作和仅限机器的结果历史会被丢弃；排队中的 Crop Automator 作业尚未消耗其输入。移除耗时 0.25 小时，并会暂停 Harvester 脚本。

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
ActionResult	.status、.message	None

OUTCOMES

Status	Kind	Meaning
"ok"	success	操作已完成。
"nothing"	rejection	请求位置没有适用的目标。
"not_empty"	rejection	相关单元格、槽位或组件并非空置。
"script_present"	rejection	机器上附加的脚本当前正在运行。
"inventory_full"	rejection	Inventory 没有空间容纳结果。
"overheated"	transient	组件对于请求的操作过热，或执行操作会超出其热量上限。
"moving"	transient	车辆或组件当前正在移动。
"busy"	transient	组件已经在执行其他操作。

.harvest() (仅 SELF)

收获当前单元格中已成熟的作物。操作耗时 0.5 小时，完成前不会转移任何物品。完成后，会将 `cell.forage` 中尽可能多的数量转移到 Inventory：如果作物数量超过可用空间，剩余部分会留存并继续保留植物，因此应先清空 Inventory，再次收获以收集剩余部分。只有在作物被全部收集后，植物才会被移除，并为另一颗实体种子释放单元格。

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
ActionResult	.status、.message	None

OUTCOMES

Status	Kind		Meaning
"ok"	success		操作已完成。
"partial"	partial	操作只移动了请求数量的一部分；剩余部分仍可从源处取得。	
"locked"	rejection		所需功能、配方或操作尚未解锁。
"no_plant"	rejection		当前单元格中没有植物。
"not_mature"	rejection		植物尚未成熟。
"no_forage"	rejection		已成熟作物没有可收集的完整 Forage（饲料）单位。
"inventory_full"	rejection		Inventory 没有空间容纳结果。
"overheated"	transient	组件对于请求的操作过热，或执行操作会超出其热量上限。	
"moving"	transient		车辆或组件当前正在移动。
"busy"	transient		组件已经在执行其他操作。

.cell(sector)

读取一个网格 sector（区块）并返回 **Cell** 快照。未扫描的自然地面在扫描前状态为 "unknown"；depot（仓库）以及玩家创建的植物或 provider（提供器）仍会显示。对于无效区块，返回 **None**。

PARAMETERS

Name	Type	Description
sector	string	网格区块 ID（例如 "E13"）。

RETURNS

请求区块对应的 **Cell**，或者无效或越界 ID 对应的 **None**。自然地面在扫描前为 "unknown"；depot 和玩家创建的植物或 provider 仍保持可见。结果包含植物、生长、环境条件、处理时间、肥料等级和 Forage。

.cells()

读取所有 Harvester 网格区块，并返回 **Cell** 快照列表。未扫描的自然地面报告为 "unknown"；在规划占用情况前请先扫描区块。可使用此列表制定全田地种植、处理路线调度、拔除、撤除部署和收获策略。

RETURNS

棋盘上的每个网格 **Cell** 列表。未扫描的自然地面为 "unknown"；已扫描单元格会公开占用情况。玩家创建的植物/提供器、生长、直接环境条件、精确剩余处理时间、当前肥料等级和累积 Forage 仍保持可见。

`.position()`

当前区块 ID，类型为字符串。其位置来源与 `get_position()` 相同，作为面向网格脚本的属性式读取公开。

RETURNS

字符串：Harvester 当前所在的区块（例如 "E13"）。Plants verbs（植物动词）全部作用于此单元格；使用 `self.move(...)` 可作用于其他位置。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()`（仅 SELF）

读取脚本运行期间从编辑器 Commands 选项卡发送的命令。所有可编程机器上的行为相同。

COMPONENTS / PRODUCTION & STORAGE

Inventory

Inventory 是 Nocturna Base（诺克图纳基地）的实体储藏室。其页面和只读脚本方法在整个星球范围内可见，但普通货物只有在 home outpost（主前哨）才能运送到机器或车辆；远程地点使用本地存储和车辆。购买的物品会送达此处。

Deploy、undeploy、decommission、upgrade 以及 empty-rig 硬件控制是明确的启用或维护订单，而不是货运路线。Manual Biology 在基地使用 Inventory，并在其他地点使用选定的同一 outpost Warehouse；Habitat 试剂在本地预置。

Type	Storage
------	---------

返回方式：`get_component("inventory")`

每个组件都有稳定的 `.id`。对于已部署机器，打开其卡片上的 ⓘ 以查找确切 ID，然后将该值传入 `get_component(id)`。ID 区分大小写。

PROPERTIES

`.id`

此组件的稳定程序化标识符。与 `get_component(id)` 以及要求 component、planet、vehicle、station 或 order ID 的 API 一起使用。

RETURNS

String

`.name`

人类可读的显示名称。对于需要经受重命名的脚本，优先使用 `.id`。

RETURNS

String

METHODS

`.stacks()`

列出每个已占用物品堆，类型为 `ItemStack`，包含 `.id`、`.count` 和精确的 `.properties`。ID 相同但 `properties` 不同的物品会分别显示。Storage Bins 和 Warehouses 使用相同格式，因此路由脚本可以用同一种方式检查三类存储，并将精确 `properties` 传入 `transfer` 方法。

RETURNS

包含属性区分的 `ItemStack` 快照列表。这是 Storage Bins 和 Warehouses 也公开的通用存储视图。

`.get_slots()`

列出当前 Inventory 的每个槽位。Inventory 起始有 36 个槽位，Cargo Expansion 可将其增加到 60 个。可堆叠物品每槽可容纳 10 个单位，研究 Bigger Stacks 后为 20 个。每个 Slot 有一个从 0 到 `get_size() - 1` 的从零开始索引，并包含物品 ID、名称、价值、数量和 `properties`。对于普通物品，`properties` 是精确的身份字典或 `None`。使用它来区分具有相同 ID 的变体，并在转移时选择精确物品。

RETURNS

包含每个堆精确 `.properties` 身份的 Slot 快照列表。

`.count(item_id)`

返回当前跨所有槽位存储的 `item_id` 单位数。如果没有槽位持有该物品，则返回 0。Storage Bins、Warehouses 和 Lead Casks 也公开相同的 `count(item_id)` 查询，因此一个辅助函数即可搜索所有存储。

PARAMETERS

Name	Type	Description
<code>item_id</code>	<code>string</code>	要计数的物品 ID

RETURNS

数字：跨所有槽位的该物品单位数。

`.has_space(item_id=None, properties=None)`

检查 Inventory 是否有空间。不带参数时，当任一槽位空闲，`has_space()` 为 `True`。传入物品 ID 和 `properties` 时，会检查确切变体：只有两者都匹配时，部分堆才计入；空槽位可以接受该物品。检查带 `properties` 的物品时，传入 `slot.properties`。

PARAMETERS

Name	Type	Description
<code>item_id</code>	<code>string</code>	可选物品 ID；提供后会针对该确切变体执行堆感知检查
<code>properties</code>	<code>any</code>	来自 Slot 或 ItemStack 的精确属性字典； <code>None</code> 选择无属性变体

RETURNS

布尔值；提供物品 ID 时使用精确的属性身份。

`.space_for(item_id, properties=None)`

返回当前一个确切物品身份还能容纳的单位数：包括具有相同 ID 和 properties 的部分堆剩余空间，以及空槽位乘以当前堆叠大小。Bigger Stacks 会将可堆叠物品的堆叠大小从 10 提高到 20。对于带 properties 的物品，使用 `inventory.space_for(slot.id, slot.properties)`。省略 properties 时检查普通的无属性变体。

PARAMETERS

Name	Type	Description
<code>item_id</code>	<code>string</code>	要计算剩余空间的物品 ID
<code>properties</code>	<code>any</code>	来自 Slot 或 ItemStack 的精确属性字典； <code>None</code> 选择无属性变体

RETURNS

当前可容纳的该确切属性变体单位数。

`.transfer_to(target, item_id, count, properties=None, property_match=None)`

需要 Auto Feeders 研究。将此存储端点中的 `item_id` 至多按整数 `count` 个单位移动到另一个 Storage Bin、Warehouse、Large Warehouse、Lead Cask 或 Inventory。传入存储建筑的显示名称或实例 ID，或传入 `"inventory"`。Inventory 仅在 Nocturna Base 参与。调用会等待实体存储的 feeder cycle 完成，参与的每个存储建筑在该周期内都保持忙碌。精确物品 `properties` 会被保留；可选的 `properties` 和 `property_match` 用于选择变体。调用脚本可以在任意位置运行，但货物不会通过此方法跨越 outpost 边界。

PARAMETERS

Name	Type	Description
<code>target</code>	<code>string</code>	同一 outpost 中另一个存储端点的显示名称或实例 ID
<code>item_id</code>	<code>string</code>	要移动的物品 ID
<code>count</code>	<code>number</code>	要转移的最大整数单位数
<code>properties</code>	<code>any</code>	可选属性字典，默认按子集匹配。省略或为 <code>None</code> 时匹配任意 <code>properties</code> ；使用 <code>None</code> 配合 <code>property_match="exact"</code> 仅选择无属性物品。
<code>property_match</code>	<code>string</code>	可选选择模式： <code>any</code> 、 <code>subset</code> 或 <code>exact</code>
RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
<code>TransferResult</code>	<code>.status</code> 、 <code>.message</code>	<code>.requested</code> 、 <code>.moved</code>

OUTCOMES

Status	Kind	Meaning
<code>"ok"</code>	<code>success</code>	已移动全部请求的 <code>.moved</code> 单位。
<code>"partial"</code>	<code>partial</code>	已移动请求的 <code>.moved</code> / <code>.requested</code> 单位；源可用量或目标容量限制了转移。
<code>"no_op"</code>	<code>success</code>	未请求任何单位，因此状态未改变。
<code>"research_required"</code>	<code>rejection</code>	所需的物料转移研究尚未解锁。
<code>"busy"</code>	<code>transient</code>	此物料端点正在处理另一项物料操作。
<code>"invalid_properties"</code>	<code>rejection</code>	物品属性选择器的形状或值无效。
<code>"invalid_property_match"</code>	<code>rejection</code>	请求的属性匹配模式无效。
<code>"target_missing"</code>	<code>rejection</code>	配置的目标已不再存在。
<code>"unsupported_target"</code>	<code>rejection</code>	配置的目标无法接受兼容货物。
<code>"same_storage"</code>	<code>rejection</code>	源和目标是同一个存储端点。

OUTCOMES (续)

Status	Kind	Meaning
"target_not_local"	rejection	配置的目标属于另一个 outpost。
"source_under_construction"	transient	配置的源仍在建造中。
"source_wrong_material"	rejection	源中包含与请求不同的物料。
"source_empty"	rejection	源中没有可用的匹配单位。
"source_changed"	transient	源在转移规划与提交之间发生了变化。
"target_under_construction"	transient	配置的目标仍在建造中。
"target_wrong_material"	rejection	目标已锁定为不同物料，或只接受不同物料。
"hot_cargo_requires_cask"	rejection	该高温货物必须通过兼容的 Lead Cask 移动。
"cask_accepts_hot_only"	rejection	Lead Casks 只接受受支持的高温货物。
"slots_full"	rejection	目标有容量，但没有可容纳该物料身份的槽位。
"target_full"	rejection	目标没有可容纳匹配单位的空间。
"target_changed"	transient	目标在转移规划与提交之间发生了变化。

.drop(slot)

根据指定槽位的从零开始索引移除 1 个单位。有效索引范围为 0 到 `get_size() - 1`，包括 Cargo Expansion 增加的槽位。丢弃的物品会被删除，而不是返回世界；如果想获得 credits，请使用 `shop.sell(item_id)`。

PARAMETERS

Name	Type	Description
slot	number	整数 Inventory 槽位索引
RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
ItemResult	.status、.message	.item_id

OUTCOMES

Status	Kind	Meaning
"ok"	success	操作影响了物品 .item_id。
"empty"	success	所选物品源已经为空。
"invalid_slot"	rejection	请求的 Inventory 槽位不存在。

.drop_all(item_id)

从 Inventory 中移除 item_id 的每个单位。若要获得 credits，请使用 `shop.sell_all(item_id)`。

PARAMETERS

Name	Type	Description
item_id	string	要丢弃每个堆的物品 ID
RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
CountResult	.status、.message	.count

OUTCOMES

Status	Kind	Meaning
"ok"	success	命令影响了 .count 个条目或单位。
"no_op"	success	命令未影响任何条目或单位。

COMPONENTS / PRODUCTION & STORAGE

Large Warehouse

高架多物料仓库，拥有 15 个物料锁定槽位，每个可容纳 2,000 个单位（总计 30,000 个）。容量足以预置完整的生物目录。

Type	Mining
Storage	15 slots × 2,000 units

HOW TO OBTAIN

- 需要 Large Warehouse 研究（Biomass 30,000）。
- 从 Shop 购买，价格为 60,000 cr。

返回方式：`get_component(id)`

每个组件都有稳定的 `.id`。对于已部署机器，打开其卡片上的 ⓘ 以查找确切 ID，然后将该值传入 `get_component(id)`。ID 区分大小写。

PROPERTIES

`.id`

此组件的稳定程序化标识符。与 `get_component(id)` 以及要求 component、planet、vehicle、station 或 order ID 的 API 一起使用。

RETURNS

String

`.name`

人类可读的显示名称。对于需要经受重命名的脚本，优先使用 `.id`。

RETURNS

String

METHODS

`.outpost`

该建筑部署所在的 outpost。返回的 `OutpostRef` 包含其稳定 ID、显示名称、生物群系、位置、容量和 `buildings()` 查询。需要当前值时请重新读取该属性。

RETURNS

该建筑所部署 outpost 的 `OutpostRef`。

`.count(item_id)`

跨所有槽位返回所持有的 `item_id` 单位数。如果没有槽位持有该物品，则返回 0。Inventory、Storage Bins 和 Lead Casks 公开相同查询。`wh.count("iron_ore")`。

PARAMETERS

Name	Type	Description
<code>item_id</code>	<code>string</code>	要计数的物品 ID

RETURNS

数字：跨所有槽位的该物品单位数。

`.total()`

跨所有槽位返回总单位数（合并所有物料）。要查询一种物料，请使用 `count(item_id)`。

RETURNS

数字：所有槽位的总单位数。

`.capacity()`

返回所有实体槽位的总容量。Warehouse 返回 10,000 ($5 \times 2,000$) , Large Warehouse 返回 30,000 ($15 \times 2,000$) 。应查询此值, 而不是对等级进行硬编码。

RETURNS

数字: 所有槽位的总容量。

`.fill_percent()`

返回整个仓库的满载比例, 即 `total() / capacity()`, 范围为 0-1。

RETURNS

数字 (0-1) 。

`.is_empty()`

当每个槽位都为空时返回 `True`。

RETURNS

布尔值。

`.materials()`

列出当前存储的物品 ID (每种在槽位中有单位的物料列出一项)。可迭代使用: `for m in wh.materials():`。

RETURNS

当前存储的物品 ID 列表。

`.stacks()`

将所有实体槽位中存储的物品变体列为 **ItemStack** 值。ID 相同但 **properties** 不同的物品会占用不同槽位。需要当前内容时再次调用。

RETURNS

跨所有实体槽位的属性区分 **ItemStack** 快照列表。

`.space_for(item_id, properties=None)`

返回当前一个确切物品变体还能容纳的单位数，包括匹配身份槽位的剩余空间和每个空槽位。普通无属性物品可省略 **properties**，或传入 **stacks()** 返回的完整 **.properties** 字典。此方法绝不会计算属于不同属性变体的空间。

PARAMETERS

Name	Type	Description
item_id	string	要计算剩余空间的物品 ID
properties	any	该变体的完整属性字典，或无属性物品使用 None

RETURNS

数字：当前可容纳的该确切物品变体单位数。

`.has_space(item_id, amount, properties=None)`

如果该确切物品变体至少还能容纳整数 `amount` 个单位，则返回 `True`。无属性物品可省略 `properties`，或传入完整属性字典。转移前使用此方法可避免部分移动。

PARAMETERS

Name	Type	Description
<code>item_id</code>	<code>string</code>	物品 ID
<code>amount</code>	<code>number</code>	所需的整数单位数
<code>properties</code>	<code>any</code>	该变体的完整属性字典，或无属性物品使用 <code>None</code>

RETURNS

布尔值。

`.slots()`

返回每个实体槽位，记录类型为 `WarehouseSlot`，包含 `.index`、`.item`、`.count`、`.capacity` 和 `.properties`。属性不同的变体使用不同槽位。

RETURNS

包含 `.index`、`.item`、`.count`、`.capacity` 和精确 `.properties` 身份的 `WarehouseSlot` 记录列表。

.compact()

需要 Auto Feeders 研究。将每个确切物品变体整合到能够容纳它的最少 Warehouse 槽位中。最小的冗余堆会移入更大的兼容堆，从而减少实体处理；物品 ID 相同但 properties 不同的物品始终保持分离。调用会等待与重新定位单位数量成比例的时间，并在该周期内将此 Warehouse 锁定为物料端点。Port transfers 和使用此 Warehouse 的 Manual Biology 操作都会等待该周期结束。

RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
TransferResult	.status、.message	.requested、.moved

OUTCOMES

Status	Kind	Meaning
"ok"	success	已将 .moved 个单位压缩到更少的 Warehouse 槽位中。
"already_compact"	success	每个确切物品变体已经占用可能的最少 Warehouse 槽位。
"research_required"	rejection	所需的物料转移研究尚未解锁。
"busy"	transient	此物料端点正在处理另一项物料操作。
"source_under_construction"	transient	配置的源仍在建造中。
"source_changed"	transient	源在转移规划与提交之间发生了变化。
"slots_full"	rejection	目标有容量，但没有可容纳该物料身份的槽位。
"target_full"	rejection	目标没有可容纳匹配单位的空间。

`.transfer_to(target, item_id, count, properties=None, property_match=None)`

需要 Auto Feeders 研究。将此存储端点中的 `item_id` 至多按整数 `count` 个单位移动到另一个 Storage Bin、Warehouse、Large Warehouse、Lead Cask 或 Inventory。传入存储建筑的显示名称或实例 ID，或传入 `"inventory"`。Inventory 仅在 Nocturna Base 参与。调用会等待实体存储的 feeder cycle 完成，参与的每个存储建筑在该周期内都保持忙碌。精确物品 `properties` 会被保留；可选的 `properties` 和 `property_match` 用于选择变体。调用脚本可以在任意位置运行，但货物不会通过此方法跨越 outpost 边界。

PARAMETERS

Name	Type	Description
<code>target</code>	<code>string</code>	同一 outpost 中另一个存储端点的显示名称或实例 ID
<code>item_id</code>	<code>string</code>	要移动的物品 ID
<code>count</code>	<code>number</code>	要转移的最大整数单位数
<code>properties</code>	<code>any</code>	可选属性字典，默认按子集匹配。省略或为 <code>None</code> 时匹配任意 <code>properties</code> ；使用 <code>None</code> 配合 <code>property_match="exact"</code> 仅选择无属性物品。
<code>property_match</code>	<code>string</code>	可选选择模式： <code>any</code> 、 <code>subset</code> 或 <code>exact</code>
RETURNS	RESULT FIELDS	SUCCESS PAYLOAD
<code>TransferResult</code>	<code>.status</code> 、 <code>.message</code>	<code>.requested</code> 、 <code>.moved</code>

OUTCOMES

Status	Kind	Meaning
<code>"ok"</code>	<code>success</code>	已移动全部请求的 <code>.moved</code> 单位。
<code>"partial"</code>	<code>partial</code>	已移动请求的 <code>.moved</code> / <code>.requested</code> 单位；源可用量或目标容量限制了转移。
<code>"no_op"</code>	<code>success</code>	未请求任何单位，因此状态未改变。
<code>"research_required"</code>	<code>rejection</code>	所需的物料转移研究尚未解锁。
<code>"busy"</code>	<code>transient</code>	此物料端点正在处理另一项物料操作。
<code>"invalid_properties"</code>	<code>rejection</code>	物品属性选择器的形状或值无效。
<code>"invalid_property_match"</code>	<code>rejection</code>	请求的属性匹配模式无效。
<code>"target_missing"</code>	<code>rejection</code>	配置的目标已不再存在。
<code>"unsupported_target"</code>	<code>rejection</code>	配置的目标无法接受兼容货物。
<code>"same_storage"</code>	<code>rejection</code>	源和目标是同一个存储端点。

状态 (Status)	类型 (Kind)	含义 (Meaning)
"target_not_local"	拒绝 (rejection)	配置的目标属于另一座前哨站。
"source_under_construction"	暂态 (transient)	配置的来源仍在建造中。
"source_wrong_material"	拒绝 (rejection)	来源包含的材料与请求的材料不同。
"source_empty"	拒绝 (rejection)	来源没有可用的匹配单位。
"source_changed"	暂态 (transient)	来源在传输规划与提交之间发生了变化。
"target_under_construction"	暂态 (transient)	配置的目标仍在建造中。
"target_wrong_material"	拒绝 (rejection)	目标已锁定到不同材料，或只接受不同材料。
"hot_cargo_requires_cask"	拒绝 (rejection)	这种高温货物必须经过兼容的 Lead Cask。
"cask_accepts_hot_only"	拒绝 (rejection)	Lead Cask 只接受受支持的高温货物。
"slots_full"	拒绝 (rejection)	目标仍有容量，但没有容纳这种材料身份的槽位。
"target_full"	拒绝 (rejection)	目标没有可容纳匹配单位的容量。
"target_changed"	暂态 (transient)	目标在传输规划与提交之间发生了变化。

组件（Components） / 生产与存储（Production & Storage）

Lead Cask

高温放射性货物唯一安全的固定存放处。无人机将 Raw Uranium 放入其中，Fuel Assembler 从中取料并放回完成的 Fuel Rod，Reactor 则从中获取燃料。

类型（Type）	Mining
存储（Storage）	100 units

获取方式（HOW TO OBTAIN）

- 完成 Helios, Plate Order 后解锁配方。
- 需要 Shielded Logistics 研究（Oxygen 3,000）。
- 在 Fabricator 上制造 Lead Cask：3× Lead Plate 和 1× Machine Frame。
- 从 Inventory 中部署它。

由以下对象返回：`get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，找到确切的 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性（PROPERTIES）

`.id`

此组件稳定的程序化标识符。将其与 `get_component(id)` 以及要求组件、行星、车辆、空间站或订单 ID 的 API 一起使用。

返回值（RETURNS）

String

`.name`

人类可读的显示名称。对于需要经受重命名的脚本，优先使用 `.id`。

返回值（RETURNS）

String

方法 (METHODS)

`.outpost`

部署此建筑的前哨站。返回的 `OutpostRef` 包含其稳定 ID、显示名称、生物群系、位置、容量以及 `buildings()` 查询。需要当前值时重新读取该属性。

返回值 (RETURNS)

部署此建筑的前哨站的 `OutpostRef`。

`.count(item_id)`

当前装入此容器的 `item_id` 单位数。当容器为空，或已锁定到另一种高温物品时返回 0。Inventory、Storage Bin 和 Warehouse 也提供相同的 `count(item_id)` 查询。

参数 (PARAMETERS)

名称 (Name)	类型 (Type)	描述 (Description)
<code>item_id</code>	<code>string</code>	要计数的高温物品 ID

返回值 (RETURNS)

当前装入的该高温物品单位数 (`Number`)。

`.fill_percent()`

填充比例 0-1，用于监视战略储备。

返回值 (RETURNS)

`Number` (0-1)。

`.capacity()`

最大高温单位数 (100)。

返回值 (RETURNS)

最大容量 (100) 的 `Number`。

`.material()`

此容器锁定的材料："raw_uranium"、"fuel_rod" 或空值。与每个库存箱一样，一个容器只能存放一种材料。

返回值 (RETURNS) | 可能的值 (POSSIBLE VALUES)

String："raw_uranium"、"fuel_rod"；未分配时为空白、"raw_uranium"、"fuel_rod"

Cask 只接受高温物品；其他任何物品都会被拒绝。

`.transfer_to(target, item_id, count, properties=None, property_match=None)`

需要 Auto Feeders 研究。将 `item_id` 的至多整数 `count` 个单位，从此存储端点移动到另一个 Storage Bin、Warehouse、Large Warehouse、Lead Cask 或 Inventory。传入存储建筑的显示名称或实例 ID，或传入 `"inventory"`。Inventory 只有在 Nocturna Base 参与此操作。调用会等待实体存储的送料器循环完成，参与其中的每座存储建筑在该循环期间都会保持忙碌。物品的精确属性会保留；可选的 `properties` 和 `property_match` 用于选择变体。调用脚本可以在任意位置运行，但货物不会通过此方法跨越前哨站边界。

参数 (PARAMETERS)

名称 (Name)	类型 (Type)	描述 (Description)
<code>target</code>	<code>string</code>	同一前哨站中另一存储端点的显示名称或实例 ID
<code>item_id</code>	<code>string</code>	要移动的物品 ID
<code>count</code>	<code>number</code>	要传输的整数最大单位数
<code>properties</code>	<code>any</code>	可选属性字典，默认按子集匹配。省略或为 <code>None</code> 时匹配任意属性；与 <code>property_match="exact"</code> 一起使用 <code>None</code> ，可只选择无属性物品。
<code>property_match</code>	<code>string</code>	可选选择模式： <code>any</code> 、 <code>subset</code> 或 <code>exact</code>

返回值 (RETURNS) | 结果字段 (RESULT FIELDS) | 成功载荷 (SUCCESS PAYLOAD)

`TransferResult` | `.status`、`.message` | `.requested`、`.moved`

结果 (OUTCOMES)

状态 (Status)	类型 (Kind)	含义 (Meaning)
<code>"ok"</code>	成功 (success)	请求的所有单位均已移动； <code>.moved</code> 为移动数量。
<code>"partial"</code>	部分成功 (partial)	已移动 <code>.requested</code> 中的 <code>.moved</code> 个单位；来源可用量或目标容量限制了传输。
<code>"no_op"</code>	成功 (success)	未请求任何单位，因此状态没有改变。
<code>"research_required"</code>	拒绝 (rejection)	所需的材料传输研究尚未解锁。
<code>"busy"</code>	暂态 (transient)	此材料端点正在处理另一个材料操作。
<code>"invalid_properties"</code>	拒绝 (rejection)	物品属性选择器的形状或值无效。
<code>"invalid_property_match"</code>	拒绝 (rejection)	请求的属性匹配模式无效。
<code>"target_missing"</code>	拒绝 (rejection)	配置的目标已不存在。
<code>"unsupported_target"</code>	拒绝 (rejection)	配置的目标无法接受兼容货物。
<code>"same_storage"</code>	拒绝 (rejection)	来源与目标是同一存储端点。

状态 (Status)	类型 (Kind)	含义 (Meaning)
"target_not_local"	拒绝 (rejection)	配置的目标属于另一座前哨站。
"source_under_construction"	暂态 (transient)	配置的来源仍在建造中。
"source_wrong_material"	拒绝 (rejection)	来源包含的材料与请求的材料不同。
"source_empty"	拒绝 (rejection)	来源没有可用的匹配单位。
"source_changed"	暂态 (transient)	来源在传输规划与提交之间发生了变化。
"target_under_construction"	暂态 (transient)	配置的目标仍在建造中。
"target_wrong_material"	拒绝 (rejection)	目标已锁定到不同材料，或只接受不同材料。
"hot_cargo_requires_cask"	拒绝 (rejection)	这种高温货物必须经过兼容的 Lead Cask。
"cask_accepts_hot_only"	拒绝 (rejection)	Lead Cask 只接受受支持的高温货物。
"slots_full"	拒绝 (rejection)	目标仍有容量，但没有容纳这种材料身份的槽位。
"target_full"	拒绝 (rejection)	目标没有可容纳匹配单位的容量。
"target_changed"	暂态 (transient)	目标在传输规划与提交之间发生了变化。

组件（Components） / 生产与存储（Production & Storage）

Mining Drill

用于硬度为 1 的矿藏的 Mk I 固定式钻机：在标准纯度下每小时 25 t，开采时耗电 10 W。

类型（Type）	Mining
建造于（Built on）	Mineral sites
输入功率（Power in）	-10 W（从电网取电）
储料堆（Stockpile）	2,000 units（混合）

获取方式（HOW TO OBTAIN）

- 完成 Helios, Cargo Pod Run 后解锁配方。
- 需要 Basic Drone Operations 研究（Terraform Index 180,000）。
- 在 Fabricator 上制造 Mining Drill Kit：3× Machine Frame、1× Control Unit、1× Circuit Panel 和 2 t Water。
- 使用 Pioneer's Constructor 在矿物地点建造。

由以下对象返回：`self` / `get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，找到确切的 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性（PROPERTIES）

`.id`

此组件稳定的程序化标识符。将其与 `get_component(id)` 以及要求组件、行星、车辆、空间站或订单 ID 的 API 一起使用。

返回值（RETURNS）

`String`

`.name`

人类可读的显示名称。对于需要经受重命名的脚本，优先使用 `.id`。

返回值（RETURNS）

`String`

方法 (METHODS)

`.drill_rate()`

当前矿物提取速率，单位为 t/h：钻机钻进时为完整速率；钻机断电、下方没有矿藏、无法切削该矿藏硬度或储料堆已满时为 0。该值已按地点纯度和钻机等级调整。

返回值 (RETURNS)

当前正在提取的 t/h (**Number**)，未钻进时为 0；该值已计入地点纯度和钻机等级。

`.output`

PickupOutputSlot，通过 `count()`、`capacity()` 和 `stacks()` 暴露 Drill 的储料堆。实体存在的 Rover 或 Pioneer 通过自身输入端取料；无人机使用 `go_to_drill()` 飞到钻机处，再使用 `cargo.load()` 装载。Drill 没有直接的物品路由方法。

返回值 (RETURNS)

暴露储料堆读取功能的 **PickupOutputSlot**。实体存在的 Rover 或 Pioneer 通过其输入端取料；无人机使用 `go_to_drill()` 和 `cargo.load()`。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()`

仅限自身 (SELF ONLY)

读取脚本运行期间从编辑器的 Commands 选项卡发送的命令。所有可脚本化机器都相同，参见指南：Script Commands。

组件（Components） / 生产与存储（Production & Storage）

Smelter

按照所选配方一次精炼一个单位，将原矿精炼成金属库存。脚本设置配方，从库存箱供入矿石，再将成品金属排到另一个库存箱。

类型（Type）	Mining
输入功率（Power in）	可变（从电网取电）
产出（Produces）	取决于所选配方
输入缓冲（Input buffer）	50 units
输出缓冲（Output buffer）	50 units
配方（Recipes）	7 available

获取方式（HOW TO OBTAIN）

- 需要 Ore Refinement 研究（Oxygen 5）。
- 在 Shop 中以 650 cr 购买。

由以下对象返回：`self` / `get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，找到确切的 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性（PROPERTIES）

`.id`

此组件稳定的程序化标识符。将其与 `get_component(id)` 以及要求组件、行星、车辆、空间站或订单 ID 的 API 一起使用。

返回值（RETURNS）

`String`

`.name`

人类可读的显示名称。对于需要经受重命名的脚本，优先使用 `.id`。

返回值 (RETURNS)

String

方法 (METHODS)

`.outpost`

部署此建筑的前哨站。返回的 `OutpostRef` 包含其稳定 ID、显示名称、生物群系、位置、容量以及 `buildings()` 查询。需要当前值时重新读取该属性。

返回值 (RETURNS)

部署此建筑的前哨站的 `OutpostRef`。

`.list_recipes()`

此 `Smelter` 已获得蓝图的所有配方。返回带有

`.tier`、`.id`、`.name`、`.inputs`、`.output_item`、`.output_count`、`.duration_game_hours` 和 `.power_draw` 的 `Recipe` 对象。尚无蓝图的锁定配方不会出现；列表反映玩家今天实际可以运行的内容。第 1 天开始时只有 `"smelt_iron_ingot"`；随着蓝图解锁会增加更多配方。

返回值 (RETURNS)

已解锁 `Recipe` 对象的列表。每个对象都有

`.tier`、`.id`、`.name`、`.inputs`、`.output_item`、`.output_count`、`.duration_game_hours` 和 `.power_draw`。

`.find_recipe(recipe_id)`

按 ID 查找一个已解锁的配方，无需循环遍历 `list_recipes()`。返回其 `Recipe` 对象；如果该 ID 未知、已锁定或属于另一种机器，则返回 `None`。

参数 (PARAMETERS)

名称 (Name)	类型 (Type)	描述 (Description)
<code>recipe_id</code>	<code>string</code>	Smelter 配方 ID

返回值 (RETURNS)

匹配的已解锁 `Recipe`；如果此 Smelter 当前无法运行该 ID，则为 `None`。

`.set_recipe(recipe_or_id)` 仅限自身 (SELF ONLY)

选择 Smelter 应运行的配方。调用 `self.set_recipe("smelt_iron_ingot")`，或传入 `list_recipes()` 返回的 `Recipe`。

参数 (PARAMETERS)

名称 (Name)	类型 (Type)	描述 (Description)
<code>recipe_or_id</code>	<code>any</code>	配方 ID、 <code>Recipe</code> 对象，或带有字符串 <code>id</code> 字段的字典或类实例。配方必须属于此机器且已解锁。

返回值 (RETURNS) | 结果字段 (RESULT FIELDS) | 成功载荷 (SUCCESS PAYLOAD)

`ActionResult` | `.status`、`.message` | `None`

结果 (OUTCOMES)

状态 (Status)	类型 (Kind)	含义 (Meaning)
<code>"ok"</code>	成功 (success)	操作成功完成。
<code>"unknown_recipe"</code>	拒绝 (rejection)	提供的配方未知，或不适用于此机器。
<code>"offline"</code>	暂态 (transient)	组件处于离线状态。
<code>"recipe_locked"</code>	拒绝 (rejection)	所选配方已锁定。
<code>"busy"</code>	暂态 (transient)	组件正在执行另一个操作。
<code>"material_mismatch"</code>	拒绝 (rejection)	已有材料与请求的材料不匹配。

`.clear_recipe()` 仅限自身 (SELF ONLY)

取消当前配方，使 Smelter 保持闲置。空的锁定缓冲区会清除回无材料状态。

返回值 (RETURNS) | 结果字段 (RESULT FIELDS) | 成功载荷 (SUCCESS PAYLOAD)

`ActionResult` | `.status`、`.message` | `None`

结果 (OUTCOMES)

状态 (Status)	类型 (Kind)	含义 (Meaning)
"ok"	成功 (success)	操作成功完成。
"busy"	暂态 (transient)	组件正在执行另一个操作。
"material_present"	拒绝 (rejection)	已有材料阻止请求的配置更改。

`.get_recipe()`

当前配方 ID (字符串) ; 如果未设置配方，则为空字符串。设置后可用它确认，或用它控制其他逻辑 (if `self.get_recipe() == ""`: ...) 。

返回值 (RETURNS) | 可能的值 (POSSIBLE VALUES)

`String` (配方 ID ; 未设置时为空字符串) | `""`、`"smelt_iron_ingot"`、`"smelt_glass"`、`"smelt_titanium_ingot"`、`"smelt_cobalt_ingot"`、`"smelt_rare_earth_core"`、`"smelt_neutronium_bar"`、`"smelt_lead_ingot"`

`.get_recipe_inputs()`

当前配方的输入需求，形式为 `{item_id: count_per_craft}` 字典。如果未设置配方，则返回空字典。

返回值 (RETURNS)

`Dict (item_id → count per craft)` ; 未设置配方时为空字典。

`.is_running()`

Smelter 正在积极处理一个单位时为 `True`。在 `set_recipe()` 前使用它以避免 "busy" 拒绝：`if not self.is_running(): self.set_recipe(new_id)`。在该单位完成前的多个 tick 中会保持为 `True`。

返回值 (RETURNS)

Boolean

`.get_progress()`

向下一个已完成单位推进的进度 (0-1)。一个单位完成并开始新单位时重置为 0。适用于进度条，以及通过观察数值下降来检测完成事件的脚本。

返回值 (RETURNS)

Number (0-1)

`.get_input_count()`

当前在输入缓冲区中等待熔炼的单位数。在 `self.input.take(...)` 前检查，以避免装得过满，或决定是否继续拉取。

返回值 (RETURNS)

输入缓冲区中的单位数 (Number)。

`.get_output_count()`

当前在输出缓冲区中等待排出的单位数。在 `self.output.send(...)` 前检查；如果数值较高，先卸载下游，如果较低，则让处理追上来。

返回值 (RETURNS)

输出缓冲区中的单位数 (Number)。

.input

向 Smelter 装入矿石。在基地处可以使用 Inventory；远程 Smelter 必须连接本地 Storage Bin 或 Warehouse。使用 `self.input.take(item_id, count)` 拉取材料。失败的传输不会改变来源货物。需要 Auto Feeders 研究。参见 `InputSlot`。

返回值 (RETURNS)

`InputSlot` : `connect()`、`take()`、`eject()`、`flush()`、`count()`、`capacity()`、`connected_to()`

.output

将精炼材料从 Smelter 发送出去。在基地处可以使用 Inventory；远程 Smelter 必须连接本地 Storage Bin 或 Warehouse。使用 `self.output.send(item_id, count)` 发送材料。失败的传输不会改变输出。需要 Auto Feeders 研究。参见 `OutputSlot`。

返回值 (RETURNS)

`OutputSlot` : `connect()`、`send()`、`count()`、`capacity()`、`connected_to()`

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()`

仅限自身 (SELF ONLY)

读取脚本运行期间从编辑器的 Commands 选项卡发送的命令。所有可脚本化机器都相同，参见指南：Script Commands。

组件（Components） / 生产与存储（Production & Storage）

Storage Bin

一次只容纳一种材料的被动式基地容器。第一次存入会设定其存储材料，只有排空后锁定才会解除。其他脚本可以读取并移动其中的内容。

类型（Type）	Mining
存储（Storage）	500 units

获取方式（HOW TO OBTAIN）

- 需要 Storage Bins 研究（Temperature 5）。
- 在 Shop 中以 120 cr 购买。

由以下对象返回：`get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，找到确切的 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性（PROPERTIES）

`.id`

此组件稳定的程序化标识符。将其与 `get_component(id)` 以及要求组件、行星、车辆、空间站或订单 ID 的 API 一起使用。

返回值（RETURNS）

String

`.name`

人类可读的显示名称。对于需要经受重命名的脚本，优先使用 `.id`。

返回值（RETURNS）

String

方法 (METHODS)

`.outpost`

部署此建筑的前哨站。返回的 `OutpostRef` 包含其稳定 ID、显示名称、生物群系、位置、容量以及 `buildings()` 查询。需要当前值时重新读取该属性。

返回值 (RETURNS)

部署此建筑的前哨站的 `OutpostRef`。

`.count(item_id)`

当前存储的 `item_id` 单位数。当库存箱为空，或已锁定到另一种物品时返回 0。Inventory、Warehouse 和 Lead Cask 也提供相同的 `count(item_id)` 查询。

参数 (PARAMETERS)

名称 (Name)	类型 (Type)	描述 (Description)
<code>item_id</code>	<code>string</code>	要计数的物品 ID

返回值 (RETURNS)

当前存储的该物品单位数 (Number)。

`.get_capacity()`

库存箱可容纳的最大单位数，默认 500。该值可查询而非硬编码，因此重新调校不会破坏脚本。需要当前填充比例时使用 `fill_percent()`。

返回值 (RETURNS)

最大单位数 (Number)。

`.get_material()`

当前锁定的材料

ID；如果库存箱为空，则为空字符串（因此下一次存入时接受任意材料）。在路由传输前使用它检查库存箱的材料：`if bin.get_material() in ("", "iron_ore"): # safe to deposit iron。`

返回值 (RETURNS)

`String`（物品 ID；库存箱为空时为空字符串）。

`.stacks()`

将此库存箱中存储的物品变体列为 `ItemStack` 值。ID 相同但属性不同的物品仍保持分离。需要当前内容时再次调用。

返回值 (RETURNS)

当前存储的、按属性区分的 `ItemStack` 快照列表。

`.is_empty()`

库存箱不含任何物品时为 `True`。空库存箱没有材料锁定，下一次存入时任意材料都可以占用该槽位。这不同于 `has_space(0)`，后者始终为 `True`。

返回值 (RETURNS)

`Boolean`

`.has_space(amount)`

如果库存箱还有空间容纳整数 `amount` 个单位，则为 `True`。传输前使用它来避免部分移动。

参数 (PARAMETERS)

名称 (Name)	类型 (Type)	描述 (Description)
<code>amount</code>	<code>number</code>	所需的整数空闲容量单位数

返回值 (RETURNS)

`Boolean`

`.space()`

剩余可用容量单位数。一次调用即可确定传输大小：`n = bin.space()`，然后最多移动 `n` 个单位。

返回值 (RETURNS)

剩余空闲单位数 (`Number`)。

`.fill_percent()`

0-1 范围内的填充比例。重新平衡脚本的常用阈值：`if bin.fill_percent() < 0.2: # route more here.`

返回值 (RETURNS)

`Number (0-1)`

`.transfer_from_inventory(item_id, count, properties=None, property_match=None)`

需要 Auto Feeders 研究，并且家园前哨站必须有 Storage Bin。将 `item_id` 的至多整数 `count` 个单位从 Inventory 移入库存箱，同时保留精确属性。调用会等待送料器循环完成，在该循环期间库存箱不能开始另一次传输。属性字典默认选择一个子集。将 `property_match` 传为 "exact" 可匹配完整身份，其中包括属性为 `None` 的无属性物品。

参数 (PARAMETERS)

名称 (Name)	类型 (Type)	描述 (Description)
<code>item_id</code>	<code>string</code>	要从 Inventory 拉取的物品 ID
<code>count</code>	<code>number</code>	要传输的整数最大单位数
<code>properties</code>	<code>any</code>	可选属性字典，默认按子集匹配。省略或为 <code>None</code> 时匹配任意属性；与 <code>property_match="exact"</code> 一起使用 <code>None</code> ，可只选择无属性物品。
<code>property_match</code>	<code>string</code>	可选选择模式： <code>any</code> 、 <code>subset</code> 或 <code>exact</code>

返回值 (RETURNS) | 结果字段 (RESULT FIELDS) | 成功载荷 (SUCCESS PAYLOAD)

`TransferResult` | `.status`、`.message` | `.requested`、`.moved`

结果 (OUTCOMES)

状态 (Status)	类型 (Kind)	含义 (Meaning)
<code>"ok"</code>	成功 (success)	请求的所有单位均已移动； <code>.moved</code> 为移动数量。
<code>"partial"</code>	部分成功 (partial)	已移动 <code>.requested</code> 中的 <code>.moved</code> 个单位；来源可用量或目标容量限制了传输。
<code>"no_op"</code>	成功 (success)	未请求任何单位，因此状态没有改变。
<code>"research_required"</code>	拒绝 (rejection)	所需的材料传输研究尚未解锁。
<code>"busy"</code>	暂态 (transient)	此材料端点正在处理另一个材料操作。
<code>"invalid_properties"</code>	拒绝 (rejection)	物品属性选择器的形状或值无效。
<code>"invalid_property_match"</code>	拒绝 (rejection)	请求的属性匹配模式无效。
<code>"inventory_not_local"</code>	拒绝 (rejection)	此端点无法从当前前哨站访问家园货运 Inventory。
<code>"source_empty"</code>	拒绝 (rejection)	来源没有可用的匹配单位。
<code>"source_changed"</code>	暂态 (transient)	来源在传输规划与提交之间发生了变化。
<code>"target_wrong_material"</code>	拒绝 (rejection)	目标已锁定到不同材料，或只接受不同材料。
<code>"hot_cargo_requires_cask"</code>	拒绝 (rejection)	这种高温货物必须经过兼容的 Lead Cask。

状态 (Status)	类型 (Kind)	含义 (Meaning)
"target_full"	拒绝 (rejection)	目标没有可容纳匹配单位的容量。
"target_changed"	暂态 (transient)	目标在传输规划与提交之间发生了变化。

`.transfer_to_inventory(count, properties=None, property_match=None)`

需要 Auto Feeders 研究以及位于主基地的 Storage Bin。将不超过给定整数 `count` 的单位数移回 Inventory，并保留精确属性。调用会等待 feeder 周期完成；在该周期内，储存箱不能开始另一项转移。默认情况下，属性字典用于选择子集。将 `property_match` 设为 "exact" 可进行完整身份匹配，包括属性为 `None` 的无属性物品。若储存箱被完全排空，其 `item-id` 锁存会清除。

参数

名称	类型	描述
count	number	要转移的最大整数单位数
properties	any	可选属性字典，默认按子集匹配。省略或为 <code>None</code> 时匹配任意属性；与 <code>property_match="exact"</code> 一起使用时，仅选择无属性物品。
property_match	string	可选选择模式： <code>any</code> 、 <code>subset</code> 或 <code>exact</code>

返回值

返回类型	结果字段	成功载荷
TransferResult	<code>.status</code> 、 <code>.message</code>	<code>.requested</code> 、 <code>.moved</code>

结果

状态	类型	含义
"ok"	success	已移动全部 <code>.moved</code> 个请求单位。
"partial"	partial	已移动请求数 <code>.requested</code> 中的 <code>.moved</code> 个；来源可用量或目标容量限制了转移。
"no_op"	success	未请求任何单位，因此状态未改变。
"research_required"	rejection	尚未解锁所需的材料转移研究。
"busy"	transient	此材料端点正在处理另一项材料操作。
"invalid_properties"	rejection	物品属性选择器的形状或值无效。
"invalid_property_match"	rejection	请求的属性匹配模式无效。
"source_empty"	rejection	来源没有可用的匹配单位。
"source_changed"	transient	来源在转移规划与提交之间发生了变化。
"inventory_not_local"	rejection	此端点从当前前哨无法访问主基地的货运 Inventory。
"target_full"	rejection	目标没有匹配单位的容量。
"target_changed"	transient	目标在转移规划与提交之间发生了变化。

`.transfer_to(target, item_id, count, properties=None, property_match=None)`

需要 Auto Feeders 研究。将不超过给定整数 `count` 的 `item_id` 单位，从此储存端点移至另一个 Storage Bin、Warehouse、Large Warehouse、Lead Cask 或 Inventory。可传入储存建筑的显示名称或实例 ID，也可传入 `"inventory"`。Inventory 仅在 Nocturna Base 参与。调用会等待实体储存设施的 feeder 周期完成，所有参与的储存建筑在该周期内都会保持忙碌。物品的精确属性会保留；可选的 `properties` 与 `property_match` 用于选择变体。调用脚本可在任意位置运行，但货物不会通过此方法跨越前哨边界。

参数

名称	类型	描述
<code>target</code>	string	同一前哨中另一储存端点的显示名称或实例 ID
<code>item_id</code>	string	要移动的物品 ID
<code>count</code>	number	要转移的最大整数单位数
<code>properties</code>	any	可选属性字典，默认按子集匹配。省略或为 <code>None</code> 时匹配任意属性；与 <code>property_match="exact"</code> 一起使用时，仅选择无属性物品。
<code>property_match</code>	string	可选选择模式： <code>any</code> 、 <code>subset</code> 或 <code>exact</code>

返回值

返回类型	结果字段	成功载荷
<code>TransferResult</code>	<code>.status</code> 、 <code>.message</code>	<code>.requested</code> 、 <code>.moved</code>

结果

状态	类型	含义
<code>"ok"</code>	success	已移动全部 <code>.moved</code> 个请求单位。
<code>"partial"</code>	partial	已移动请求数 <code>.requested</code> 中的 <code>.moved</code> 个；来源可用量或目标容量限制了转移。
<code>"no_op"</code>	success	未请求任何单位，因此状态未改变。
<code>"research_required"</code>	rejection	尚未解锁所需的材料转移研究。
<code>"busy"</code>	transient	此材料端点正在处理另一项材料操作。
<code>"invalid_properties"</code>	rejection	物品属性选择器的形状或值无效。
<code>"invalid_property_match"</code>	rejection	请求的属性匹配模式无效。
<code>"target_missing"</code>	rejection	配置的目标已不存在。
<code>"unsupported_target"</code>	rejection	配置的目标无法接收兼容货物。
<code>"same_storage"</code>	rejection	来源与目标是同一储存端点。

状态	类型	含义
"target_not_local"	rejection	配置的目标属于另一前哨。
"source_under_construction"	transient	配置的来源仍在建造中。
"source_wrong_material"	rejection	来源包含的材料与请求材料不同。
"source_empty"	rejection	来源没有可用的匹配单位。
"source_changed"	transient	来源在转移规划与提交之间发生了变化。
"target_under_construction"	transient	配置的目标仍在建造中。
"target_wrong_material"	rejection	目标已锁定或接受另一种材料。
"hot_cargo_requires_cask"	rejection	此热货物必须通过兼容的 Lead Cask 移动。
"cask_accepts_hot_only"	rejection	Lead Cask 仅接受受支持的热货物。
"slots_full"	rejection	目标有容量，但没有适用于该材料身份的槽位。
"target_full"	rejection	目标没有匹配单位的容量。
"target_changed"	transient	目标在转移规划与提交之间发生了变化。

组件 / 生产与储存

Warehouse

多材料散装仓库，包含 5 个材料锁定槽位，每个槽位 2,000 单位（总计 10,000）。用于吸收无人机规模的运输量。

类型	Mining
储存	5 个槽位 × 2,000 单位

获取方式

- 需要 Warehouse 研究（Oxygen 150）。
- 在 Shop 中以 3,000 cr 购买。

返回方式：`get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ⓘ，找到准确 ID，再将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件的稳定程序化标识符。将其与 `get_component(id)` 以及要求组件、星球、载具、空间站或订单 ID 的 API 搭配使用。

返回：`String`

`.name`

人类可读的显示名称。对于需要在重命名后仍能工作的脚本，优先使用 `.id`。

返回：`String`

方法

`.outpost`

部署此建筑的前哨。返回的 `OutpostRef` 包含其稳定 ID、显示名称、生物群系、位置、容量以及 `buildings()` 查询。需要当前值时，请重新读取此属性。

返回：部署此建筑的前哨的 `OutpostRef`

`.count(item_id)`

跨所有槽位统计 `item_id` 的单位数。如果没有槽位存放该物品，则返回 0。Inventory、Storage Bin 和 Lead Cask 也提供相同查询。`wh.count("iron_ore")`。

参数：`item_id` (string)，要统计的物品 ID。

返回：Number，所有槽位中的该物品单位数。

`.total()`

统计所有槽位的总单位数（合并所有材料）。如需统计一种材料，请使用 `count(item_id)`。

返回：Number，所有槽位中的总单位数。

.capacity()

所有实体槽位的总容量。Warehouse 返回 10,000 ($5 \times 2,000$) ； Large Warehouse 返回 30,000 ($15 \times 2,000$) 。请查询此值，而不要硬编码等级。

返回：Number，所有槽位的总容量。

.fill_percent()

整个仓库的填充比例，即 `total() / capacity()`，范围为 0-1。

返回：Number (0-1) 。

.is_empty()

当每个槽位都为空时返回 True。

返回：Boolean。

.materials()

列出当前存放的物品 ID（每种在槽位中有单位数的材料占一项）。可迭代使用：`for m in wh.materials():`。

返回：当前存放的物品 ID 列表。

`.stacks()`

将所有实体槽位中存放的物品变体列为 `ItemStack` 值。ID 相同但属性不同的物品会占用独立槽位。需要当前内容时请再次调用。

返回：跨所有实体槽位、按属性区分的 `ItemStack` 快照列表。

`.space_for(item_id, properties=None)`

计算某个精确物品变体当前还能容纳多少单位：使用身份匹配槽位的剩余空间以及所有空槽位。普通无属性物品可省略 `properties`；也可传入 `stacks()` 返回的完整 `.properties` 字典。此方法绝不会计算属于其他属性变体的空间。

参数	类型	描述
<code>item_id</code>	<code>string</code>	要计算剩余空间的物品 ID
<code>properties</code>	<code>any</code>	该变体的完整属性字典，或无属性物品使用 <code>None</code>

返回：Number，该精确物品变体当前可容纳的单位数。

`.has_space(item_id, amount, properties=None)`

当该精确物品变体至少还能容纳整数 `amount` 个单位时返回 `True`。无属性物品可省略 `properties`，或传入完整属性字典。转移前可用此方法避免部分移动。

参数	类型	描述
<code>item_id</code>	string	物品 ID
<code>amount</code>	number	所需的整数单位数
<code>properties</code>	any	该变体的完整属性字典，或无属性物品使用 <code>None</code>

返回：Boolean。

`.slots()`

列出每个实体槽位，返回 `WarehouseSlot` 记录，包含 `.index`、`.item`、`.count`、`.capacity` 和 `.properties`。按属性区分的变体使用独立槽位。

返回： `WarehouseSlot` 记录列表，包含 `.index`、`.item`、`.count`、`.capacity` 以及精确的 `.properties` 身份。

.compact()

需要 Auto Feeders 研究。将每个精确物品变体整合到能够容纳它的最少 Warehouse 槽位中。较小的冗余堆会移入更大的兼容堆，从而减少实体处理；ID 相同但属性不同的物品始终保持分离。调用会等待与重新定位单位数成比例的时间，并在该周期内将此 Warehouse 锁定为材料端点。港口转移以及使用此 Warehouse 的手动 Biology 操作都会等待该周期结束。

返回类型	结果字段	成功载荷
TransferResult	.status、.message	.requested、.moved

结果

状态	类型	含义
"ok"	success	已将 .moved 个单位压缩到更少的 Warehouse 槽位中。
"already_compact"	success	每个精确物品变体已经占用可能的最少 Warehouse 槽位。
"research_required"	rejection	尚未解锁所需的材料转移研究。
"busy"	transient	此材料端点正在处理另一项材料操作。
"source_under_construction"	transient	配置的来源仍在建造中。
"source_changed"	transient	来源在转移规划与提交之间发生了变化。
"slots_full"	rejection	目标有容量，但没有适用于该材料身份的槽位。
"target_full"	rejection	目标没有匹配单位的容量。

`.transfer_to(target, item_id, count, properties=None, property_match=None)`

需要 Auto Feeders 研究。将不超过给定整数 `count` 的 `item_id` 单位，从此储存端点移至另一个 Storage Bin、Warehouse、Large Warehouse、Lead Cask 或 Inventory。可传入储存建筑的显示名称或实例 ID，也可传入 `"inventory"`。Inventory 仅在 Nocturna Base 参与。调用会等待实体储存设施的 feeder 周期完成，所有参与的储存建筑在该周期内都会保持忙碌。物品的精确属性会保留；可选的 `properties` 与 `property_match` 用于选择变体。调用脚本可在任意位置运行，但货物不会通过此方法跨越前哨边界。

名称	类型	描述
<code>target</code>	string	同一前哨中另一储存端点的显示名称或实例 ID
<code>item_id</code>	string	要移动的物品 ID
<code>count</code>	number	要转移的最大整数单位数
<code>properties</code>	any	可选属性字典，默认按子集匹配。省略或为 <code>None</code> 时匹配任意属性；与 <code>property_match="exact"</code> 一起使用时，仅选择无属性物品。
<code>property_match</code>	string	可选选择模式： <code>any</code> 、 <code>subset</code> 或 <code>exact</code>
返回类型	结果字段	成功载荷
<code>TransferResult</code>	<code>.status</code> 、 <code>.message</code>	<code>.requested</code> 、 <code>.moved</code>

结果

状态	类型	含义
<code>"ok"</code>	success	已移动全部 <code>.moved</code> 个请求单位。
<code>"partial"</code>	partial	已移动请求数 <code>.requested</code> 中的 <code>.moved</code> 个；来源可用量或目标容量限制了转移。
<code>"no_op"</code>	success	未请求任何单位，因此状态未改变。
<code>"research_required"</code>	rejection	尚未解锁所需的材料转移研究。
<code>"busy"</code>	transient	此材料端点正在处理另一项材料操作。
<code>"invalid_properties"</code>	rejection	物品属性选择器的形状或值无效。
<code>"invalid_property_match"</code>	rejection	请求的属性匹配模式无效。
<code>"target_missing"</code>	rejection	配置的目标已不存在。
<code>"unsupported_target"</code>	rejection	配置的目标无法接收兼容货物。
<code>"same_storage"</code>	rejection	来源与目标是同一储存端点。

状态	类型	含义
"target_not_local"	rejection	配置的目标属于另一前哨。
"source_under_construction"	transient	配置的来源仍在建造中。
"source_wrong_material"	rejection	来源包含的材料与请求材料不同。
"source_empty"	rejection	来源没有可用的匹配单位。
"source_changed"	transient	来源在转移规划与提交之间发生了变化。
"target_under_construction"	transient	配置的目标仍在建造中。
"target_wrong_material"	rejection	目标已锁定或接受另一种材料。
"hot_cargo_requires_cask"	rejection	此热货物必须通过兼容的 Lead Cask 移动。
"cask_accepts_hot_only"	rejection	Lead Cask 仅接受受支持的热货物。
"slots_full"	rejection	目标有容量，但没有适用于该材料身份的槽位。
"target_full"	rejection	目标没有匹配单位的容量。
"target_changed"	transient	目标在转移规划与提交之间发生了变化。

组件 / 生产与储存

Waste Processor

永久销毁由脚本选择的一种废物流：物品、液体或气体。对不需要的库存使用物品输入，对多余的水或其他液体使用 `liquid_in`，对气体使用 `gas_in`。它没有输出，也不会回收任何价值。

类型	Mining
输入功率	可变（从电网取用）
储料	200 单位（混合）

获取方式

- Waste Processing 研究解锁配方（Oxygen 1,400）。
- 在 Fabricator 上制造 Waste Processor Kit：2× Machine Frame、4× Iron Ingot 和 1× Circuit Panel。
- 从 Inventory 部署。

返回方式：`self` / `get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，找到准确 ID，再将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件的稳定程序化标识符。将其与 `get_component(id)` 以及要求组件、星球、载具、空间站或订单 ID 的 API 搭配使用。

返回：`String`

`.name`

人类可读的显示名称。对于需要在重命名后仍能工作的脚本，优先使用 `.id`。

返回：`String`

方法

.outpost

部署此建筑的前哨。返回的 **OutpostRef** 包含其稳定 ID、显示名称、生物群系、位置、容量以及 **buildings()** 查询。需要当前值时，请重新读取此属性。

返回：部署此建筑的前哨的 **OutpostRef**

.set_enabled(enabled) (仅 SELF)

在所选模式下启动或暂停销毁。停止所属脚本会将此设定点重置为 off。已暂存的物品和流体保持不变。

参数		类型	描述
enabled		boolean	True 表示在所选模式下启动销毁；False 表示停止销毁，并在保留所有暂存缓冲区的同时拒绝新的流体。
返回类型		结果字段	成功载荷
ActionResult		.status、.message	None

结果

状态	类型	含义
"ok"	success	操作成功完成。

.is_enabled()

读取所属脚本是否已启动处理器。

返回：boolean

`.set_mode(mode)` (仅 SELF)

准确选择一种销毁流：`"items"`、`"liquid"` 或 `"gas"`。切换模式会暂停其他缓冲区，但不会删除其中内容。所选流体端口仅在启用状态下接受流量。

参数			类型	描述
mode	string	选择的废物流： <code>"items"</code> 、 <code>"liquid"</code> 或 <code>"gas"</code> 。只有该流会被销毁。所选流体端口仅在启用时接受流量；显式物品转移可在任意模式下暂存物品。其他缓冲区会暂停，但内容不会丢失。		
返回类型			结果字段	成功载荷
ActionResult			.status、.message	None

结果

状态	类型	含义
"ok"	success	操作成功完成。

引发异常

异常	条件
ValueError	mode 必须为 <code>"items"</code> 、 <code>"liquid"</code> 或 <code>"gas"</code> 。

`.mode()`

读取所选废物流。

返回：当前选定的流：`"items"`、`"liquid"` 或 `"gas"`。

`.status()`

读取精确的实时状态：`"disabled"`、`"no_power"`、`"idle"` 或 `"processing"`。

返回：当前运行状态：`"disabled"`、`"no_power"`、`"idle"` 或 `"processing"`。

`.throughput()`

读取所选模式最近一次的销毁速率。物品模式报告单位/h；液体和气体模式报告 t/h。

返回：Number，所选模式在最近处理 tick 中的销毁速率。物品模式报告单位/h；液体和气体模式报告 t/h。未销毁任何内容时为 0。

`.item_throughput()`

读取最近一次的物品销毁速率，单位为单位/h。

返回：Number，最近处理 tick 中销毁的整数物品数，单位为单位/h。

`.liquid_throughput()`

读取最近一次的液体销毁速率，单位为 t/h。在前哨效率为 100% 时，最大值为 120 t/h。

返回：Number，最近处理 tick 中销毁的液体量，单位为 t/h。在前哨效率为 100% 时，最大值为 120 t/h。

`.gas_throughput()`

读取最近一次的气体销毁速率，单位为 t/h。在前哨效率为 100% 时，最大值为 120 t/h。

返回：Number，最近处理 tick 中销毁的气体量，单位为 t/h。在前哨效率为 100% 时，最大值为 120 t/h。

`.is_running()`

读取所选模式是否在最近一次处理 tick 中销毁了材料。

返回：Boolean；当所选模式在最近一次处理 tick 中销毁了材料时为 True。纯销毁没有输出端口，也不会回收任何价值。

`.input`

暂存要永久销毁的物品。使用 `self.input.connect(name)` 连接储存箱或机器缓冲区，然后使用 `self.input.take(item_id, count)` 拉取物品。物品转移是显式的，因此在选择另一模式时也可以暂存；只有启用的 "items" 模式会销毁它们。参见 `InputSlot`。

返回：`InputSlot`，包含 `connect()`、`take()`、`eject()`、`flush()`、`count()`、`capacity()` 和 `connected_to()`。可从储存箱或机器副产物箱将任意物品导入此处。输入端在所有模式下都接受任意物品；只有启用的 "items" 模式会销毁它们。恢复错误装载前，请先禁用销毁，再使用 `eject(...)`。

`.liquid_in`

通用的 120t 液体输入缓冲区。连接液体来源，选择 "liquid"，然后启用处理器。非空缓冲区只接受其当前锁存的液体，直到缓冲区排空。参见 `FluidPort`。

返回：适用于任意液体的通用 120t `FluidPort` 输入。仅当处理器在 "liquid" 模式下启用时才接受流量。非空缓冲区会锁定到一种精确液体，直到排空。

`.gas_in`

通用的 120t 气体输入缓冲区。连接气体来源，选择

"gas"，然后启用处理器。非空缓冲区只接受其当前锁存的气体，直到缓冲区排空。参见 `FluidPort`。

返回：适用于任意气体的通用 120t `FluidPort` 输入。仅当处理器在 "gas"

模式下启用时才接受流量。非空缓冲区会锁定到一种精确气体，直到排空。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` (仅 SELF)

读取脚本运行期间从编辑器 `Commands` 选项卡发送的命令。所有可编程机器上的行为相同，参见指南：Script Commands。

组件 / 载具与模块

Constructor Module（构造模块） MODULE

专供 Pioneer 使用的模块，用于 Plan Mode 或脚本创建的建造与拆除蓝图：气体/液体管道、实用桥、电线、前哨、泵、端盖和采矿钻机。它适配通用槽位。将所需套件、分段或桥梁物品装入 Pioneer 的货舱，驾驶至蓝图位置的交互范围内，然后调用 `execute(blueprint_id)`。拆除会将被拆解的套件或分段回收至 Pioneer 货舱。前哨内部机器（包括无人机设施）直接从 Inventory 部署，不属于 Constructor 作业。

返回方式：`self.constructor`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，找到准确 ID，再将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件的稳定程序化标识符。将其与 `get_component(id)` 以及要求组件、星球、载具、空间站或订单 ID 的 API 搭配使用。

返回：`String`

`.name`

人类可读的显示名称。对于需要在重命名后仍能工作的脚本，优先使用 `.id`。

返回：`String`

方法

.execute(blueprint_id) (仅 SELF)

从共享规划队列中取出一个 Construction 作业并执行建造或拆除。Plan Mode 和 `get_component("construction_blueprint")` 会创建等价作业。首先将 Pioneer 驾驶至蓝图位置的交互范围内，并使用 `get_component("construction_blueprint").pending_constructions()` 查看哪些作业已准备好。Pioneer 一次只能执行一个现场操作。停止、断电、离开现场、救援或移除 Constructor Module 都会暂停已付费作业，但不会丢失其进度或材料。使用 `get_component("construction_blueprint").paused_constructions()` 中的同一 ID 恢复。

名称			类型	描述
blueprint_id	string	来自 <code>get_component("construction_blueprint").pending_constructions()</code> 、 <code>.active_constructions()</code> 或 <code>.paused_constructions()</code> 的蓝图 ID。		
返回类型			结果字段	成功载荷
ActionResult			.status、.message	None

结果

状态	类型	含义
"ok"	success	操作成功完成。
"not_mounted"	rejection	所需模块未安装。
"not_found"	rejection	请求的对象、目标或记录不存在。
"locked"	rejection	所需功能、配方或操作处于锁定状态。
"already_active"	transient	请求的操作已经处于活动状态。
"busy"	transient	组件正在执行另一项操作。
"wrong_position"	rejection	工作单元不在该操作要求的位置。
"insufficient_materials"	rejection	Pioneer 货舱中没有足够的所需建造材料。
"paused_no_power"	transient	操作因无法供电而暂停。
"cargo_present"	rejection	现有货物阻止了请求的配置变更。
"blocked"	rejection	当前世界状态阻止了该操作。
"paused"	transient	操作已暂停。
"canceled"	rejection	操作已取消。

组件 / 载具与模块 (COMPONENTS / VEHICLES & MODULES)

钻探模块 (Drill Module) —— 模块 (MODULE)

通过自身钻头提取矿物。基础钻头可处理硬度 1，以 $1.0\times$ 速度运行并消耗 10 W；工业钻头可处理硬度 3，以 $0.75\times$ 运行并消耗 20 W；重型钻头可处理硬度 4，以 $0.6\times$ 运行并消耗 30 W。未安装钻探模块时，载具无法采矿。

返回自：`self.drill`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ① 即可找到确切 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性 (PROPERTIES)

`.id`

此组件的稳定程序标识符。将其与 `get_component(id)` 以及要求组件、星球、载具、空间站或订单 ID 的 API 配合使用。

返回值 (RETURNS)

String

`.name`

面向用户显示的可读名称。需要经受重命名的脚本应优先使用 `.id`。

返回值 (RETURNS)

String

方法 (METHODS)

`.mine()` (仅自身 SELF ONLY)

将当前地点的 1 单位矿物提取到载具货舱。采矿耗时为 `mineral_base_minutes × drill.speed_multiplier() / site_purity` 游戏时间，脚本会暂停直到操作完成。

返回值 (RETURNS)	结果字段 (RESULT FIELDS)	成功载荷 (SUCCESS PAYLOAD)
ActionResult	.status, .message	无 (None)

结果 (OUTCOMES)

状态 (Status)	类型 (Kind)	含义 (Meaning)
"ok"	成功 (success)	操作成功完成。
"not_mounted"	拒绝 (rejection)	未安装所需模块。
"not_at_site"	拒绝 (rejection)	载具未位于兼容地点。
"not_surveyed"	拒绝 (rejection)	尚未勘测该矿物地点。
"too_hard"	拒绝 (rejection)	目标超出已安装工具的硬度上限。
"no_cargo_space"	拒绝 (rejection)	货舱没有可容纳结果的兼容空间。
"no_power"	暂态 (transient)	组件没有可用电力。
"not_enough_power"	拒绝 (rejection)	可用能量低于操作要求。
"busy"	暂态 (transient)	组件正在执行其他操作。

`.hardness_limit()`

此钻头可提取的最大矿物硬度。

返回值 (RETURNS)

Number：基础型为 1，工业型为 3，重型为 4。过期的已捕获模块引用会引发 **ReferenceError**。

引发异常 (RAISES)

异常 (Exception)	条件 (Condition)
ReferenceError	此 DrillModule 引用已过期，因为其模块不再安装。安装钻头后重新读取 self.drill 。

.speed_multiplier()

每单位时间乘数（越低越快）。

返回值 (RETURNS)

Number：基础型为 1.0，工业型为 0.75，重型为 0.6。过期的已捕获模块引用会引发 **ReferenceError**。

引发异常 (RAISES)

异常 (Exception)	条件 (Condition)
ReferenceError	此 DrillModule 引用已过期，因为其模块不再安装。安装钻头后重新读取 self.drill 。

组件 / 载具与模块 (COMPONENTS / VEHICLES & MODULES)

无人机 (Drone)

小型空中货运无人机，配备 1 个推进器和 2 个模块。

类型 (Type) : 采矿 (Mining)

获取方式 (HOW TO OBTAIN)

- 完成 Helios、Rotor Run 后解锁配方。
- 需要基础无人机操作研究 (Terraform Index 180,000)。
- 在制造机上制造一架无人机 (小型) : 1× 稀土核心 (Rare Earth Core)、1× 钛锭 (Titanium Ingot)、1× 控制单元 (Control Unit) 和 2t 水 (Water)。
- 从库存中部署它。

返回自 : `self / get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ① 即可找到确切 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性 (PROPERTIES)

`.id`

此组件的稳定程序标识符。将其与 `get_component(id)` 以及要求组件、星球、载具、空间站或订单 ID 的 API 配合使用。

返回值 (RETURNS) : `String`

`.name`

面向用户显示的可读名称。需要经受重命名的脚本应优先使用 `.id`。

返回值 (RETURNS) : `String`

方法 (METHODS)

`.go_to_station(name)` (仅自身 SELF ONLY)

将前往指定名称的无人机仓库 (Drone Depot) 或无人机服务站 (Drone Service Station) 的路线排入队列, 并立即返回, 不等待停靠。已接受且有电力的路线会立即报告 `"traveling"`; 位置和停靠状态会在模拟推进后更新。停止、完成或出错会取消飞行并清除路线。将 `current_station()` 与目的地的稳定 ID 比较, 以确认已到达。在同一前哨站内的无人机建筑之间移动属于本地转移, 不消耗飞行燃料。无人机服务站即使未通电也接受停放到达。已满的无人机仓库会让无人机保持未停靠的 `"waiting_bay"` 状态, 直到实体泊位空出。

参数 (PARAMETERS)

名称 (Name)	类型 (Type)	描述 (Description)
<code>name</code>	<code>string</code>	目的地无人机仓库或无人机服务站的显示名称或 ID。

返回值 (RETURNS) : `ActionResult`; 结果字段 `.status`, `.message`; 成功载荷: 无 (None)。

结果 (OUTCOMES)

状态	类型	含义
<code>"ok"</code>	成功	操作成功完成。
<code>"station_not_found"</code>	拒绝	请求的空间站不存在。
<code>"out_of_range"</code>	拒绝	请求目标超出操作范围。
<code>"busy"</code>	暂态	组件正在执行其他操作。
<code>"scrambled"</code>	暂态	辐射扰频阻止了该操作。

`.undock()`（仅自身 SELF ONLY）

释放空间站泊位，但不飞往任何地方。无人机会保留其精确世界位置、货物、模块、燃料和暴露值，清除任何休眠路线，将油门重置为 0，并进入空闲状态。活动中的救援任务，或活动中/排队中的无人机服务站充电/加油任务，会在该站所属工作结束前继续保持控制。需要再次占用泊位时使用 `go_to_station(...)`。仅自身。

返回值（RETURNS）：`ActionResult`；结果字段 `.status`, `.message`；成功载荷：无（None）。

状态	类型	含义
"ok"	成功	操作成功完成。
"not_docked"	拒绝	请求的载具未停靠在此空间站。
"busy"	暂态	组件正在执行其他操作。

`.go_to_drill(name)` (仅自身 SELF ONLY)

飞往指定名称的野外采矿钻机 (Mining Drill) 进行矿石提取。运输不需要野外模块。已接受且有电力的路线会立即报告 "traveling", 随后无人机沿直线飞行并在到达时悬停。将 `current_drill()` 与目的地稳定 ID 比较, 以确认货物装载已可用。停止、完成脚本、出错或调用 `go_to_station()` 会取消该路线。

参数 (PARAMETERS)

名称	类型	描述
name	string	已部署采矿钻机 (任意等级) 的显示名称或 ID。

返回值 (RETURNS) : `ActionResult` ; 结果字段 `.status`, `.message` ; 成功载荷 : 无 (None) 。

状态	类型	含义
"ok"	成功	操作成功完成。
"drill_not_found"	拒绝	在星球边界内没有可用的、匹配请求的已完成采矿钻机。
"out_of_range"	拒绝	请求目标超出操作范围。
"busy"	暂态	组件正在执行其他操作。
"scrambled"	暂态	辐射扰频阻止了该操作。

`.current_station()`

无人机当前实际停靠的稳定空间站 ID。在飞往坐标、在空间站之间行驶或在已满的无人机仓库外等待时, 返回空字符串。将其与目的地 ID 做相等比较, 是权威的空间站到达检查, 即使 `go_to_station()` 使用的是显示名称。

返回值 (RETURNS) : 空间站 ID 字符串 ; 运输中则为空字符串。

`.current_drill()`

无人机当前可以装载货物的稳定采矿钻机 ID；没有钻机可用时空字符串。无人机必须位于钻机装载区域内且没有活动路线；仅飞越钻机上空或持有零油门路线都不算到达。将此值与目的地 ID 比较，是权威的钻机到达检查，即使 `go_to_drill()` 使用的是显示名称。

返回值 (RETURNS)：可装载货物的采矿钻机 ID 字符串，否则为空字符串。

`.position()`

世界坐标 (`.x`, `.y`)。运输期间由 `DroneSystem` 每个 tick 进行插值，停靠时吸附到空间站坐标。

返回值 (RETURNS)：`Position (.x, .y)`，即当前世界坐标。

`.get_distance_to(x, y)`

从无人机当前位置到给定世界坐标的直线距离（米）。可用于比较可能的目的地、检查剩余路线距离，或在派遣前与 `range_remaining()` 配合。它只测量几何距离，不选择目的地，也不考虑可用燃料。

参数 (PARAMETERS)

名称	类型	描述
<code>x</code>	<code>number</code>	目标 X 坐标，单位为米。
<code>y</code>	<code>number</code>	目标 Y 坐标，单位为米。

返回值 (RETURNS)：从无人机当前位置到给定点的直线距离（米），类型为 `Number`。

.battery

读取电动无人机的电力。使用 `self.battery.level()` 获取当前电量 (Wh)，使用 `capacity()` 获取电池组总容量，使用 `percent()` 获取 0-1 的电量比例。在直升机无人机上调用这些方法会引发 `ReferenceError`。在混合机队中，先检查 `fleet.drones()` 的 `DroneRef.engine`。

返回值 (RETURNS)：电动无人机上的 `DroneBattery`，包含 `level()`、`capacity()` 和 `percent()`。在没有电动动力系统的无人机上，其方法会引发 `ReferenceError`；检查 `fleet.drones()`，并根据 `DroneRef.engine` 在混合机队中分支处理。

.oil_tank

读取直升机无人机的燃料。使用 `self.oil_tank.level()` 获取当前油量 (吨)，并使用 `capacity()` 与 `percent()`。在电动无人机上调用这些方法会引发 `ReferenceError`。在混合机队中，先检查 `fleet.drones()` 的 `DroneRef.engine`。

返回值 (RETURNS)：直升机无人机上的 `DroneOilTank`，包含 `level()`、`capacity()` 和 `percent()`。在没有直升机动力系统的无人机上，其方法会引发 `ReferenceError`；检查 `fleet.drones()`，并根据 `DroneRef.engine` 在混合机队中分支处理。

.cargo

管理已安装的货舱吊舱（Cargo

Pods）和生物提取器的密封舱。每个吊舱容纳一种物品类型，清空后会解锁；提取器舱容纳一种生命体类型，并优先装满。使用 `self.cargo.count()`、`contents()`、`capacity()` 和 `space_for(item_id)` 规划装载。停靠在无人机仓库时，`load()` 与 `unload()` 均可用；位于野外采矿钻机或铅罐（Lead Cask）处时，`load()` 也可用。物品的精确属性会保留。参见 `DroneCargo`。

返回值（RETURNS）：带已安装货舱吊舱的 `DroneCargo`，以及生物提取器的类型化 25 t 舱。由于生物空间按物种区分，请使用 `space_for(item_id)`。

.go_to(x, y) (仅自身 SELF ONLY)

飞往任意世界坐标的基础无人机能力；不需要野外模块。已接受且有电力的路线会立即报告 "traveling", 随后无人机沿直线飞行并在到达时悬停。停止、完成脚本、出错或调用 go_to_station() 会取消该路线。对于 Weather, 请传入根据校验和有效的风暴数据包组装出的精确 x、y。

参数 (PARAMETERS)

名称	类型	描述
x	number	世界 x, 单位为 m。
y	number	世界 y, 单位为 m。

返回值 (RETURNS) : ActionResult ; 结果字段 .status, .message ; 成功载荷 : 无 (None) 。

状态	类型	含义
"ok"	成功	操作成功完成。
"invalid_target"	拒绝	提供的目标对该操作无效。
"out_of_bounds"	拒绝	请求位置超出有效世界边界。
"out_of_range"	拒绝	请求目标超出操作范围。
"busy"	暂态	组件正在执行其他操作。
"scrambled"	暂态	辐射扰频阻止了该操作。

.bio_scanner

便携式生物扫描器 (Portable Bio Scanner) 。

返回值 (RETURNS) : PortableBioScanner。scan() 需要该模块、已完成的路线和正常工作的电子设备, 并通过 BioScanResult.scan 暴露已完成的生物读数。

.bio_extractor

便携式生物提取器 (Portable Bio Extractor) 。

返回值 (RETURNS) : PortableBioExtractor。extract() 需要无人机处于悬停且未受扰频, 并使用其集成的 25 t 单一生命体舱以及任意货舱吊舱。

.collect() (仅自身 SELF ONLY)

在无人机当前的精确坐标收集一批天气余波。成功批次最多转移 5 个风暴玻璃（Storm Glass）或原铀（Raw Uranium）。没有护盾镀层时，收集原铀会增加 40 点暴露值；有护盾镀层时增加 0 点；即使达到扰频阈值，也会保留该批次。

返回值 (RETURNS) : CollectResult ; 结果字段 .status, .message ; 成功载荷 .item_id, .collected。

状态	类型	含义
"ok"	成功	无人机货舱收到 : .item_id × .collected。
"moving"	暂态	无人机仍在移动。
"busy"	暂态	无人机正忙于生物提取。
"nothing_here"	拒绝	无人机当前位置没有存活天气余波。
"no_cargo_space"	拒绝	无人机没有足够货舱容量容纳该地点的材料。
"scrambled"	暂态	无人机电子设备受到辐射扰频，无法收集。

`.exposure()`

当前提取暴露值，范围为 0 至 `exposure_capacity()`。只有收集原铀或接受服务站维护时它才会变化；飞越隐藏余波不会产生影响。停靠在通电无人机服务站的正常工作无人机每小时清除 10 点。达到容量时无人会进入扰频状态，需要无人机服务站救援。

返回值 (RETURNS)：Number，辐射暴露值，范围 0 至容量。未镀层无人机收集原铀增加 40；护盾镀层增加 0。通电无人机服务站以 10/h 清除未达到终端状态的暴露值。达到容量时无人机进入扰频并需要救援。

`.exposure_capacity()`

100 点暴露的扰频阈值。未镀层无人机可以安全收集两批 5 单位材料；第三批会被保留，随后使其进入扰频。

返回值 (RETURNS)：Number，扰频阈值。

`.is_plated()`

安装护盾镀层 (Shield Plating) 时为 `True`。镀层将原铀提取暴露值降为零，使每个货舱吊舱容量减半，并由于铅材较重使燃料消耗提高 $1.5\times$ 。

返回值 (RETURNS)：Boolean，是否安装护盾镀层（原铀提取暴露为零，并且从铅罐装载高温货物时必需；货舱吊舱容量减半，消耗提高）。

`.range_remaining()`

根据当前能量和油门估算飞行距离（米）。电动推进在全油门时消耗 5 Wh/h；直升机在全油门时消耗 5 t/h 的油。两者都按油门的平方缩放，因此较慢的路线会延长航程。

返回值（RETURNS）：**Number**，按当前电量和当前油门估算的飞行米数。油门为 0 或电池/油量为空时为 0。

`.throttle()`

当前油门（0-1）。电动推进全油门消耗 5 Wh/h，直升机全油门消耗 5 t/h 的油；两者都按油门平方缩放。停止、完成或出错后读数为 0。

返回值（RETURNS）：**Number**（0-1），当前油门设置。

`.set_throttle(rate)`（仅自身 SELF ONLY）

设置油门（0-1）。全油门时，电动无人机以 300 m/h 飞行并消耗 5 Wh/h；直升机无人机以 900 m/h 飞行并消耗 5 t/h 的油。降低油门会使消耗按平方下降。停止、完成或出错会将油门重置为 0。仅自身。

参数（PARAMETERS）

名称	类型	描述
<code>rate</code>	<code>number</code>	油门设置（0-1）。

返回值（RETURNS）：**ActionResult**；结果字段 `.status`, `.message`；成功载荷：无（None）。

状态	类型	含义
<code>"ok"</code>	成功	操作成功完成。

`.couple(slot_index, module_id)` (仅自身 SELF ONLY)

请求一项硬件服务订单，将库存中的模块安装到明确的整数槽位：推进器槽使用 `self.couple(0, "electric_thruster")`，模块槽使用 `self.couple(1, "battery_pack")`。无人机必须停靠在正在运行的无人机仓库。此专用硬件例外不会暴露该前哨站普通库存货运。仅自身。

参数 (PARAMETERS)

名称	类型	描述
<code>slot_index</code>	<code>number</code>	无人机槽位索引，必须为整数，从 0 开始。
<code>module_id</code>	<code>string</code>	无人机模块的商店 ID。

返回值 (RETURNS) : `ActionResult` ; 结果字段 `.status, .message` ; 成功载荷：无 (None) 。

状态	类型	含义
<code>"ok"</code>	成功	操作成功完成。
<code>"not_at_station"</code>	拒绝	载具未停靠在兼容空间站。
<code>"item_not_in_inventory"</code>	拒绝	库存不含请求物品。
<code>"unknown_module"</code>	拒绝	提供的模块标识符不存在。
<code>"slot_not_compatible"</code>	拒绝	选定槽位与模块不兼容。
<code>"slot_occupied"</code>	拒绝	选定槽位已被占用。
<code>"invalid_slot"</code>	拒绝	提供的槽位不存在。
<code>"locked"</code>	拒绝	所需功能、配方或操作已锁定。
<code>"wrong_engine_for_module"</code>	拒绝	模块与载具发动机类型不兼容。
<code>"cargo_capacity_exceeded"</code>	拒绝	当前货物超过请求装备的容量。

.uncouple(slot_index) (仅自身 SELF ONLY)

请求一项硬件服务订单，将明确整数槽位中的模块返回库存：`self.uncouple(1)`。无人机必须停靠在正在运行的无人机仓库。拆除前货舱吊舱必须为空；载有原铀或燃料棒（Fuel Rod）货物时不能拆除护盾镀层。含燃料的模块会保留其中的内容物。仅自身。

参数 (PARAMETERS)

名称	类型	描述
slot_index	number	无人机槽位索引，必须为整数，从 0 开始。

返回值 (RETURNS) : `ActionResult` ; 结果字段 `.status`, `.message` ; 成功载荷：无 (None) 。

状态	类型	含义
"ok"	成功	操作成功完成。
"invalid_slot"	拒绝	提供的槽位不存在。
"module_not_mounted"	拒绝	选定模块槽没有安装模块。
"not_at_station"	拒绝	载具未停靠在兼容空间站。
"cargo_capacity_exceeded"	拒绝	当前货物超过请求装备的容量。
"container_not_empty"	拒绝	模块的货物容器不为空。
"hot_cargo_requires_plating"	拒绝	仍载有原铀或燃料棒货物，需要护盾镀层。
"inventory_full"	拒绝	库存没有容纳结果的容量。

`.status()`

当前运行活动，用于进度和阻塞处理，而不是到达测试。"idle"

可能表示已停靠、在野外坐标悬停，或以零油门保持排队路线；停靠后可立即开始充电或加油。"waiting_bay" 表示无人机到达已满的仓库但尚未停靠；"holding_weather" 是直升机的临时保持状态；stalled 或 scrambled 状态需要干预。使用 `current_station()` 或 `current_drill()` 确认是否到达交互端点。

返回值 (RETURNS)：字符串 (String)，当前运行活动。使用 `current_station()` 进行单独的实际到达检查。

可能值 (POSSIBLE VALUES)："idle"、"traveling"、"charging"、"refueling"、"waiting_service"、"waiting_oil"、"waiting_bay"、"being_rescued"、"holding_weather"、"scrambled"、"stalled_no_battery"、"stalled_no_oil"、"stalled_no_route"。

`.is_being_rescued()`

当无人机服务站正在主动维护或运送该无人机时为 `True`。救援车辆控制期间，使用此值暂停路线脚本。

返回值 (RETURNS)：`Boolean`

`.rescue_status()`

此无人机当前的恢复任务阶段："none"、"outbound"、"charging"、"carrying" 或 "returning"。"returning" 表示恢复车辆正返回基地，且无人机不再受救援控制。

返回值 (RETURNS)：状态字符串："none" / "outbound" / "charging" / "carrying" / "returning"。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` (仅自身
SELF ONLY)

读取脚本运行期间从编辑器“命令 (Commands)”选项卡发送的命令。所有可编程机器上的行为都相同，参见指南：Script
Commands。

组件 / 载具与模块 (COMPONENTS / VEHICLES & MODULES)

机队 (Fleet)

所有已拥有移动单位的只读索引：地面载具和无人机。使用它为仪表板、充电脚本、救援阈值和调度决策提供数据，无需硬编码名称。机队引用是快照；控制仍通过单位自身的脚本或相关空间站 API 进行。

返回自：`get_component("fleet")`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ① 即可找到确切 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性 (PROPERTIES)

`.id`

此组件的稳定程序标识符。将其与 `get_component(id)` 以及要求组件、星球、载具、空间站或订单 ID 的 API 配合使用。

返回值 (RETURNS)：`String`

`.name`

面向用户显示的可读名称。需要经受重命名的脚本应优先使用 `.id`。

返回值 (RETURNS)：`String`

方法 (Methods)

`.vehicles()`

所有已拥有的地面载具，作为 `VehicleRef` 快照返回。每个引用包含

`.id`、`.name`、`.kind`、`.x`、`.y`、`.battery_level`、`.is_docked`、`.is_being_rescued` 和 `.rescue_status`。

返回值 (Returns)

`VehicleRef` 快照列表。要获取最新位置/状态，请重新查询；或者使用 `get_component(ref.id)` 获取实时载具 API。

`.drones()`

所有已拥有的无人机，作为 `DroneRef` 快照返回。每个引用包含

`.id`、`.name`、`.kind`、`.engine`、`.current_station`、`.battery_level` 或 `.oil_level`、`.is_docked`、`.is_being_rescued` 和 `.rescue_status`。

返回值 (Returns)

`DroneRef` 快照列表。要获取最新位置/状态，请重新查询；或者使用 `get_component(ref.id)` 获取实时无人机 API。

`.mobile_units()`

将所有已拥有的地面载具和无人机合并为一个列表。使用 `.category` 在 "vehicle" 和 "drone" 之间进行分支判断。

返回值 (Returns)

`MobileUnitRef` 快照列表。要获取最新位置/状态，请重新查询。

组件 / 载具与模块 (Components / Vehicles & Modules)

导航模块 (Nav Module)

车辆通过 `self.nav` 行驶。基础模块提供 $1.0\times$ 的最高速度。同一设备上安装一个运动导航模块 (Sport Nav) 时，最高速度为 $2\times$ ，移动功耗为 $2.6\times$ ；在全油门下，每米耗电约为 $1.3\times$ 。继续安装运动导航模块时，每个模块都会额外增加 $1.0\times$ 的基础最高速度，并使功耗更快上升。它适用于导航槽或通用槽。请保持脚本运行直到抵达。若脚本停止、结束或出错，车辆会停止并清除路线。

由 `self.nav` 返回。

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ① 以查找确切 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性 (Properties)

`.id`

此组件的稳定程序化标识符。将其与 `get_component(id)` 以及要求提供组件、星球、载具、空间站或订单 ID 的 API 一起使用。

返回值 (Returns)

String

`.name`

人类可读的显示名称。对于需要在重命名后仍能正常工作的脚本，优先使用 `.id`。

返回值 (Returns)

String

方法 (Methods)

`.set_target(x, y)` — 仅自身 (SELF ONLY)

设置要驶向的目标坐标。立即返回；只要脚本保持活动状态，车辆就会在后续 tick 中异步行驶。轮询 `get_distance_to(x, y)` 或 `get_position()`，并在等待循环中检测接近程度。请使用容差，通常写成 `while self.nav.get_distance_to(x, y) > 2:`，而不是等待精确的零值；然后在调用 `drill.mine()` 或其他需要静止的动作之前调用 `self.nav.brake()`。对于需要在建筑物处执行的动作，应将目标设为该建筑物的 `BuildingRef.position`，而不是前哨站占地范围的锚点。若脚本停止、结束或出错，车辆会停止并清除路线。

参数 (Parameters)

名称	类型	说明
<code>x</code>	<code>number</code>	以米为单位的目标 X 坐标
<code>y</code>	<code>number</code>	以米为单位的目标 Y 坐标

返回值 / 结果字段 / 成功载荷 (Returns / Result Fields / Success Payload)

返回值	结果字段	成功载荷
<code>ActionResult</code>	<code>.status, .message</code>	<code>None</code>

结果 (Outcomes)

状态	类型	含义
<code>"ok"</code>	<code>success</code>	操作成功完成。
<code>"not_mounted"</code>	<code>rejection</code>	所需模块未安装。
<code>"invalid"</code>	<code>rejection</code>	一个或多个提供的参数超出可接受范围。
<code>"out_of_bounds"</code>	<code>rejection</code>	请求的位置位于有效世界边界之外。

.set_throttle(power) — 仅自身 (SELF ONLY)

设置油门 (0.0-1.0, 会被限制在此范围内)。self.nav.set_throttle(0.5) 用于巡航; 1.0 用于冲刺, 但每米消耗更多电量。长距离行驶时可用它在速度与航程之间进行权衡。停止、完成、出错以及 brake() 都会将油门重置为 0。

参数 (Parameters)

名称	类型	说明
power	number	0 到 1 范围内的油门比例

返回值 / 结果字段 / 成功载荷 (Returns / Result Fields / Success Payload)

返回值	结果字段	成功载荷
ActionResult	.status, .message	None

结果 (Outcomes)

状态	类型	含义
"ok"	success	操作成功完成。
"not_mounted"	rejection	所需模块未安装。

.throttle()

当前油门设定值 (0.0-1.0)。返回脚本上次通过 self.nav.set_throttle(...) 写入的值; 停止、完成、出错或 brake() 后返回 0。它不同于 get_speed(): throttle() 表示你的意图, 而 get_speed() 表示车辆在上一个 tick 中实际移动的速度。

返回值 (Returns)

Number (0.0-1.0): 当前油门设定值。

.brake() — 仅自身 (SELF ONLY)

立即停止车辆，将油门和速度设为 0，并将当前目标清除为车辆当前位置。脚本需要中止路线中途的行驶（例如重新规划路线，驶向更近的地点）时使用。与驶向目的地相比，这样更节省电池。

返回值 / 结果字段 / 成功载荷 (Returns / Result Fields / Success Payload)

返回值	结果字段	成功载荷
ActionResult	.status, .message	None

结果 (Outcomes)

状态	类型	含义
"ok"	success	操作成功完成。
"not_mounted"	rejection	所需模块未安装。

.get_position()

返回当前位置，形式为 Position 对象，其中 .x 和 .y 表示相对于基地的米数。在行驶循环的每次迭代中读取它，以检测抵达、规划下一跳或记录路径。参见 Position。

返回值 (Returns)

相对于基地的米数：Position 对象 {x, y}。

.get_speed()

返回上一个行驶 tick 记录的速度，单位为米/小时。空闲或刹车时为 0；油门为 1.0 时最高可达到导航模块的最高速度。用于确认车辆确实在移动（如果你预期它行驶但返回 0，则可能存在电力或目标问题）。新调用的 set_throttle(...) 要到下一个行驶 tick 才会在此处体现。

返回值 (Returns)

Number (m/h)。

`.get_distance_to(x, y)`

返回车辆当前位置到给定点的欧几里得距离，单位为米。应在有意设置的等待循环中使用它检测接近程度，并使用类似 `> 2` 的抵达容差，绝不要等待精确的零值。达到该容差不会使车辆停止；在站点处执行静止动作之前调用 `brake()`。若要在建筑物处执行动作，应将路线设为该建筑物的 `BuildingRef.position`。车辆行驶时，`set_target()` 不会阻塞；若脚本停止、完成或出错，行驶会被取消。这是纯直线距离，不考虑障碍物。

参数 (Parameters)

名称	类型	说明
<code>x</code>	<code>number</code>	目标 X 坐标 (米)
<code>y</code>	<code>number</code>	目标 Y 坐标 (米)

返回值 (Returns)

`Number (meters)`。

`.speed_multiplier()`

当前最高速度倍率：基础导航模块且没有运动导航模块时为 `1.0`；Pioneer 上每安装一个运动导航模块，就在此基础上增加 `1`。可用它规划旅行时间和侦察配置。它只影响速度；航程仍取决于电池、油门、货物载荷和移动功耗。

返回值 (Returns)

`Number`：基础值为 `1.0`，或 `1 + Pioneer 上的运动导航模块数量`。仅表示速度；航程取决于电池、油门、货物载荷和移动功耗。

组件 / 载具与模块 (Components / Vehicles & Modules)

Pioneer

用于在野外行驶、扫描、采矿和建造的模块化远程载具。裸底盘本身没有任何功能；所有功能都来自安装在其八个槽位中的模块、电池和货物。

类型	采矿 (Mining)
储备堆 (Stockpile)	0 units (mixed)

获取方式 (How to Obtain)

- 需要 Pioneer 底盘研究 (Terraform Index 100,000)。
- 在商店购买，价格为 5,000 cr。

由 `self / get_component(id)` 返回。

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ① 以查找确切 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性 (Properties)

`.id`

此组件的稳定程序化标识符。将其与 `get_component(id)` 以及要求提供组件、星球、载具、空间站或订单 ID 的 API 一起使用。

返回值 (Returns)

String

`.name`

人类可读的显示名称。对于需要在重命名后仍能正常工作的脚本，优先使用 `.id`。

返回值 (Returns)

String

方法 (Methods)

`.status()`

读取 Pioneer 当前的物理活动。每次调用都会读取最新状态，包括通过 `get_component(...)` 进行调用时也是如此。处于空闲状态的 Pioneer 仍可能正在运行脚本或已分配作业。

返回值 / 可能值 (Returns / Possible Values)

返回当前载具活动的 `String`，在调用时计算。可能值：`"idle"`、`"moving"`、`"stranded"`、`"scanning"`、`"surveying"`、`"drilling"`、`"discarding"`、`"constructing"`、`"transferring"`、`"charging"`、`"queued"`、`"being_rescued"`。

`.battery`

汇总所有已安装电池座 (Battery Holder) 中每个便携式电池 (Portable Battery) 的电池池。`self.battery.level()` 返回 0-1 的比例；`self.battery.wh()` 返回原始瓦时；`self.battery.capacity()` 返回合计最大容量。若要查看每个电池座的详细信息，`self.battery.holders()` 会返回每个 Holder 及其 `.batteries` 列表。汇总视图足以满足大多数脚本；检查内部信息适用于高级再平衡逻辑。如果未安装任何便携式电池，Pioneer 无法移动。参见 Battery。

返回值 (Returns)

Battery：汇总所有已安装电池座中的每个电池。

.cargo

汇总 Pioneer 货物架（Cargo Rack）中的便携式货箱（Portable Bin）。使用 `count()`、`capacity()` 和 `full()` 获取总量，或使用 `racks()` 检查物理布局。每个货箱容纳一种物品类型。`take()` 和 `send()` 分别处理各个货箱。`compact()` 会将匹配的货物合并到更少的货箱中，但不会改变 `send()` 顺序。`discard(rack_index)` 会永久清空一个货架；有货物时需要 1 小时，空货架会立即完成。参见 Cargo。

返回值（Returns）

Cargo：汇总所有已安装货物架中的每个货箱。

.nav

驾驶 Pioneer。使用 `self.nav.set_target(x, y)` 设置相对于基地的米数目标。调用会立即返回；只要脚本运行，Pioneer 就会继续行驶。距离容差表示 Pioneer 已足够接近，并不代表已停止，因此在采矿、建造或转移货物之前调用 `self.nav.brake()`。若脚本停止、结束或出错，Pioneer 会停止并清除路线。需要将导航模块安装在通用槽中。有关油门、制动和位置的信息，参见 NavModule。

返回值（Returns）

安装导航模块时返回 NavModule。

.sonar

查找 Pioneer 附近的有用地点。使用 `self.sonar.scan()` 查找附近地点，然后使用 `self.sonar.survey(site)` 显示矿物硬度、纯度或喷口产出等详细信息。扫描和新调查需要时间并消耗电池，因此声呐工作期间脚本会暂停。需要安装声呐模块。有关范围、能力和电池消耗，参见 SonarModule。

返回值（Returns）

安装声呐模块时返回 SonarModule。

.drill

使用 `self.drill.mine()` 从 Pioneer 下方已调查的地点提取一个矿物单位。车辆必须保持静止，并且拥有足够的货物空间和电力。需要安装钻机模块。有关速度和硬度限制，参见 `DrillModule`。

返回值 (Returns)

安装钻机模块时返回 `DrillModule`。

.constructor

从建造模式 (Plan Mode) 或脚本中建造或移除行星地图蓝图。通过 `get_component("construction_blueprint")` 读取待处理工作，驶近 `construction.position`，然后调用 `self.constructor.execute(construction.id)`。建造会从货物中消耗所需套件或段件；移除会将回收部件放回货物，因此请预留空间。需要安装建造器模块。参见 `ConstructorModule`。

返回值 (Returns)

安装建造器模块时返回 `ConstructorModule`，并暴露 `execute()`。

.input

从基地的 `Inventory`、前哨站的本地存储、野外提取器储备堆或附近已停止的货运车辆装载货物。野外和车辆转移需要处于服务范围内；两辆车辆都必须停止。它绝不会销毁货物；请使用 `self.cargo.discard(rack_index)` 抛弃一个货架。需要自动送料器 (Auto Feeders)。参见 `VehicleInputSlot`。

返回值 (Returns)

`VehicleInputSlot`: `connect()`、`disconnect()`、`take()`、`count()`、`capacity()`、`stacks()`、`connected_to()`、`connected_id()`。

.output

将货物卸载到 Inventory、储物箱（Storage Bin）、仓库（Warehouse）或附近已停止的货运车辆。只有当 Pioneer 停在基地时，Inventory 货运才可在物理上取得。远程前哨站使用其本地存储。车辆之间交接时，两辆车辆必须在约 ~2 m 内停止。需要自动送料器研究。参见 OutputSlot。

返回值（Returns）

`OutputSlot:connect()`、`send()`、`count()`、`capacity()`、`connected_to()`。

.is_being_rescued()

当车辆充电站（Vehicle Charging Station）的救援无人机正在主动回收 Pioneer 时为 `True`。即使电池电量已开始升至零以上，也应使用它暂停移动、采矿或建造脚本。

返回值（Returns）

`Boolean`

.rescue_status()

返回 Pioneer 当前救援任务阶段：`"none"`、`"outbound"`、`"charging"` 或 `"returning"`。`"returning"` 表示救援无人机正在返航，Pioneer 已重新获得自由。

返回值 / 可能值（Returns / Possible Values）

状态字符串：`"none"` / `"outbound"` / `"charging"` / `"returning"`。

`.modules()`

检查底盘上的每个槽位。对于 Pioneer，返回包含八个条目的 `MountSlot` 列表。每个条目都有 `.index`（传给 `mount / unmount`）、`.type`（Pioneer 上始终为 "universal"）、`.module_id`（已安装模块；空槽为 `None`）、`.internal_count`（电池座 / 货物架中为非零值）和 `.internal_items`（已安装便携物品 ID 列表）。调用 `self.mount(...)` 或 `self.install(...)` 之前先调用它，以查找空闲目标。参见 `MountSlot`。

返回值 (Returns)

`MountSlot` 对象列表：底盘上的每个安装点一个；每个对象包含 `.index`、`.type`、`.module_id`、`.internal_count`、`.internal_items`。

.mount(slot_index, item_id) — 仅自身 (SELF ONLY)

向 Inventory 请求硬件服务订单，将模块安装到整数 slot_index : self.mount(0, "nav_module")。生物野外模块仅限无人机使用。Pioneer 必须位于已建立前哨站的服务区域内。此专用硬件例外并不意味着 Inventory 在那里可作为货运端点。

参数 (Parameters)

名称	类型	说明
slot_index	number	底盘上从零开始的整数槽位位置。
item_id	string	要安装模块的商店 ID。

返回值 / 结果字段 / 成功载荷 (Returns / Result Fields / Success Payload)

返回值	结果字段	成功载荷
ActionResult	.status, .message	None

结果 (Outcomes)

状态	类型	含义
"ok"	success	操作成功完成。
"not_at_service_point"	rejection	车辆没有停在所需服务区域。
"locked"	rejection	所需功能、配方或操作尚未解锁。
"item_not_in_inventory"	rejection	Inventory 不包含请求的物品。
"unknown_module"	rejection	提供的模块标识符不存在。
"slot_not_compatible"	rejection	所选槽位与该模块不兼容。
"slot_occupied"	rejection	所选槽位已被占用。
"invalid_slot"	rejection	提供的槽位不存在。
"capability_already_mounted"	rejection	车辆已经拥有提供此能力的模块。

.unmount(slot_index) — 仅自身 (SELF ONLY)

请求硬件服务订单，将整数 slot_index 处的模块返还到 Inventory：self.unmount(0)。Pioneer 必须位于已建立前哨站的服务区域内。移除电池座或货物架之前，先清空其中的每个内部舱位。

参数 (Parameters)

名称	类型	说明
slot_index	number	底盘上从零开始的整数槽位位置。

返回值 / 结果字段 / 成功载荷 (Returns / Result Fields / Success Payload)

返回值	结果字段	成功载荷
ActionResult	.status, .message	None

结果 (Outcomes)

状态	类型	含义
"ok"	success	操作成功完成。
"not_at_service_point"	rejection	车辆没有停在所需服务区域。
"slot_empty"	rejection	所选槽位为空。
"invalid_slot"	rejection	提供的槽位不存在。
"holder_not_empty"	rejection	设备座仍包含物品。
"inventory_full"	rejection	Inventory 没有足够空间容纳结果。

`.install(slot_index, internal_index, item_id)` — 仅自身 (SELF ONLY)

请求服务订单，将便携式电池或空的便携式储物箱从 Inventory 安装到容器的整数内部槽位中。`self.install(0, 1, "portable_battery")` 使用底盘槽位 0 处电池座的舱位 1。`self.install(4, 0, "portable_bin")` 使用底盘槽位 4 处货物架的舱位 0。Pioneer 必须位于已建立前哨站的服务区域内。

参数 (Parameters)

名称	类型	说明
<code>slot_index</code>	<code>number</code>	容器（电池座 / 货物架）安装所在的槽位。
<code>internal_index</code>	<code>number</code>	容器内部从零开始的整数槽位索引。
<code>item_id</code>	<code>string</code>	要安装便携物品的商店 ID。

返回值 / 结果字段 / 成功载荷 (Returns / Result Fields / Success Payload)

返回值	结果字段	成功载荷
<code>ActionResult</code>	<code>.status, .message</code>	<code>None</code>

结果 (Outcomes)

状态	类型	含义
<code>"ok"</code>	<code>success</code>	操作成功完成。
<code>"not_at_service_point"</code>	<code>rejection</code>	车辆没有停在所需服务区域。
<code>"locked"</code>	<code>rejection</code>	所需功能、配方或操作尚未解锁。
<code>"invalid_slot"</code>	<code>rejection</code>	提供的槽位不存在。
<code>"invalid_internal_slot"</code>	<code>rejection</code>	提供的内部槽位不存在。
<code>"slot_empty"</code>	<code>rejection</code>	所选槽位为空。
<code>"not_container"</code>	<code>rejection</code>	提供的目标不是兼容容器。
<code>"item_not_accepted"</code>	<code>rejection</code>	目标不接受提供的物品。
<code>"internal_slot_occupied"</code>	<code>rejection</code>	所选内部槽位已被占用。
<code>"item_not_in_inventory"</code>	<code>rejection</code>	Inventory 不包含请求的物品。

`.uninstall(slot_index, internal_index)` — 仅自身 (SELF ONLY)

请求服务订单，将容器整数内部槽位中的便携物品返还到 Inventory：`self.uninstall(0, 1)`。Pioneer 必须位于已建立前哨站的服务区域内，并且便携式储物箱必须为空。

参数 (Parameters)

名称	类型	说明
<code>slot_index</code>	<code>number</code>	容器安装所在的槽位。
<code>internal_index</code>	<code>number</code>	容器内部从零开始的整数槽位索引。

返回值 / 结果字段 / 成功载荷 (Returns / Result Fields / Success Payload)

返回值	结果字段	成功载荷
<code>ActionResult</code>	<code>.status, .message</code>	<code>None</code>

结果 (Outcomes)

状态	类型	含义
<code>"ok"</code>	<code>success</code>	操作成功完成。
<code>"not_at_service_point"</code>	<code>rejection</code>	车辆没有停在所需服务区域。
<code>"invalid_slot"</code>	<code>rejection</code>	提供的槽位不存在。
<code>"invalid_internal_slot"</code>	<code>rejection</code>	提供的内部槽位不存在。
<code>"slot_empty"</code>	<code>rejection</code>	所选槽位为空。
<code>"not_container"</code>	<code>rejection</code>	提供的目标不是兼容容器。
<code>"internal_slot_empty"</code>	<code>rejection</code>	所选内部槽位为空。
<code>"inventory_full"</code>	<code>rejection</code>	Inventory 没有足够空间容纳结果。
<code>"container_not_empty"</code>	<code>rejection</code>	模块的货物容器不为空。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` — 仅自身 (SELF ONLY)

读取脚本运行期间从编辑器 Commands 选项卡发送的命令。所有可编程机器均相同，参见指南：Script Commands。

组件 / 载具与模块 (Components / Vehicles & Modules)

Rover

用于行驶、扫描和采矿的初始探险车。裸底盘本身没有任何功能；每项能力都来自安装在其三个固定槽位中的模块。

类型	采矿 (Mining)
能量 (Energy)	100 Wh
储备堆 (Stockpile)	10 units (mixed)

获取方式 (How to Obtain)

- 需要 Rover 底盘研究 (Pressure 0.11)。
- 在商店购买，价格为 2,000 cr。

由 self / get_component(id) 返回。

每个组件都有稳定的 .id。对于已部署的机器，打开其卡片上的 ① 以查找确切 ID，然后将该值传给 get_component(id)。ID 区分大小写。

属性 (Properties)

.id

此组件的稳定程序化标识符。将其与 get_component(id) 以及要求提供组件、星球、载具、空间站或订单 ID 的 API 一起使用。

返回值 (Returns)

String

.name

人类可读的显示名称。对于需要在重命名后仍能正常工作的脚本，优先使用 .id。

返回值 (Returns)

String

方法 (Methods)

`.status()`

读取 Rover 当前的物理活动。每次调用都会读取最新状态，包括通过 `get_component(...)` 进行调用时也是如此。处于空闲状态的 Rover 仍可能正在运行脚本或已分配作业。

返回值 / 可能值 (Returns / Possible Values)

返回当前载具活动的 String，在调用时计算。可能值："idle"、"moving"、"stranded"、"scanning"、"surveying"、"drilling"、"discarding"、"constructing"、"transferring"、"charging"、"queued"、"being_rescued"。

`.battery`

Rover 剩余的电量。`self.battery.level()` 返回 0-1 的比例，`self.battery.wh()` 返回原始瓦时，`self.battery.capacity()` 返回最大值 100 Wh。低于 0.1 时即将搁浅；`charging_station.dispatch_rescue()` 可以前来救援，但速度较慢。`.holders()` 返回空列表（密封式设备）。参见 Battery。

返回值 (Returns)

Battery：level()、wh()、capacity()、holders()。

`.cargo`

Rover 集成货舱中携带的全部物品。`self.cargo.count()` 返回所有物品和属性变体的总单位数；`self.cargo.capacity()` 为 10；达到容量时 `self.cargo.full()` 为 True，装载或采矿前应检查它。由于货舱是集成式的，`.racks()` 为空。货物存在时，`self.cargo.discard(0)` 会永久抛弃整个货舱并耗时 1 小时；空货舱会立即完成。参见 Cargo。

返回值 (Returns)

Cargo：count()、capacity()、full()、racks()、discard()。

.nav

驾驶 Rover。使用 `self.nav.set_target(x, y)` 设置相对于基地的米数目标。调用会立即返回；只要脚本运行，Rover 就会继续行驶。距离容差表示 Rover 已足够接近，并不代表已停止，因此在采矿、扫描、调查或转移货物之前调用 `self.nav.brake()`。若脚本停止、结束或出错，Rover 会停止并清除路线。需要安装导航模块。有关油门、制动和速度的信息，参见 NavModule。

返回值 (Returns)

安装导航模块时返回 NavModule，否则不可用。

.sonar

查找 Rover 附近的有用地点。使用 `self.sonar.scan()` 查找附近地点，然后使用 `self.sonar.survey(site)` 显示矿物硬度、纯度或喷口产出等详细信息。扫描和新调查需要时间，因此声呐工作期间脚本会暂停。需要安装声呐模块。有关范围、能力和电池消耗，参见 SonarModule。

返回值 (Returns)

安装声呐模块时返回 SonarModule，否则不可用。

.drill

使用 `self.drill.mine()` 从 Rover 下方已调查的地点提取一个矿物单位。车辆必须保持静止，并且拥有足够的货物空间和电力。需要安装钻机模块。有关速度和硬度限制，参见 DrillModule。

返回值 (Returns)

安装钻机模块时返回 DrillModule，否则不可用。

.input

从基地的 Inventory、前哨站的本地存储、野外提取器储备堆或附近已停止的货运车辆装载货物。野外和车辆转移需要处于服务范围内；两辆车辆都必须停止。它绝不会销毁货物；请使用 `self.cargo.discard(0)` 抛弃货舱。需要自动送料器。参见 `VehicleInputSlot`。

返回值 (Returns)

`VehicleInputSlot` : `connect()`、`disconnect()`、`take()`、`count()`、`capacity()`、`stacks()`、`connected_to()`、`connected_id()`。

.output

将货物卸载到 Inventory、储物箱 (Storage Bin)、仓库 (Warehouse) 或附近已停止的货运车辆。只有当 Rover 停在基地时，Inventory 货运才可在物理上取得。远程前哨站使用其本地存储。车辆之间交接时，两辆车辆必须在约 ~2 m 内停止。需要自动送料器研究。参见 `OutputSlot`。

返回值 (Returns)

`OutputSlot` : `connect()`、`send()`、`count()`、`capacity()`、`connected_to()`。

.is_being_rescued()

当车辆充电站 (Vehicle Charging Station) 的救援无人机正在主动回收 Rover 时为 `True`。即使电池电量已开始升至零以上，也应使用它暂停移动或采矿脚本。

返回值 (Returns)

Boolean

`.rescue_status()`

当前此 Rover 的救援任务阶段："none"、"outbound"、"charging" 或 "returning"。"returning" 表示救援无人机正在返航，Rover 已重新可用。

返回值

返回值	可能值
字符串状态："none" / "outbound" / "charging" / "returning"	"none"、"outbound"、"charging"、"returning"

`.modules()`

检查已安装的模块。返回一个 MountSlot 列表，每个对象对应一个底盘插槽。每个对象包含 `.index`（传给 `mount / unmount`）、`.type`（适配的模块类型）、`.module_id`（已安装的模块 ID，或 None）以及 `.internal_items`（Rover 的功能插槽为空）。调用 `self.mount(...)` 前先调用此方法，以找到空插槽并确认插槽类型接受目标模块。参见 MountSlot。

返回值

每个安装点对应一个 MountSlot 对象的列表，每个对象含 `.index`、`.type`、`.module_id`、`.internal_count`、`.internal_items`。

`.mount(slot_index, item_id)` 仅 SELF

从 Inventory 请求硬件服务订单，将物品安装到整数插槽 `slot_index : self.mount(0, "nav_module")`。生物野外模块仅限无人机使用。Rover 必须位于已建立前哨的服务区域内。这个专用硬件例外并不会使 Inventory 在该处变成货运端点。

参数

名称	类型	描述
slot_index	number	底盘上的整数、从零开始的插槽位置。
item_id	string	要安装的模块的商店 ID。

返回值

返回值	结果字段	成功载荷
ActionResult	.status、.message	无

结果

状态	类型	含义
"ok"	成功	操作成功完成。
"not_at_service_point"	拒绝	载具未停在所需服务区域。
"locked"	拒绝	所需功能、配方或操作尚未解锁。
"item_not_in_inventory"	拒绝	Inventory 不包含所请求的物品。
"unknown_module"	拒绝	提供的模块标识符不存在。
"slot_not_compatible"	拒绝	所选插槽与该模块不兼容。
"slot_occupied"	拒绝	所选插槽已被占用。
"invalid_slot"	拒绝	提供的插槽不存在。
"capability_already_mounted"	拒绝	载具已经安装了提供此能力的模块。

`.unmount(slot_index)` 仅 SELF

请求硬件服务订单，将整数 `slot_index` 处的模块退回 Inventory：`self.unmount(0)`。Rover 必须位于已建立前哨的服务区域内，并且容器模块必须为空。

参数

名称	类型	描述
<code>slot_index</code>	<code>number</code>	底盘上的整数、从零开始的插槽位置。

返回值

返回值	结果字段	成功载荷
<code>ActionResult</code>	<code>.status</code> 、 <code>.message</code>	无

结果

状态	类型	含义
<code>"ok"</code>	成功	操作成功完成。
<code>"not_at_service_point"</code>	拒绝	载具未停在所需服务区域。
<code>"slot_empty"</code>	拒绝	所选插槽为空。
<code>"invalid_slot"</code>	拒绝	提供的插槽不存在。
<code>"holder_not_empty"</code>	拒绝	设备容器中仍有物品。
<code>"inventory_full"</code>	拒绝	Inventory 没有容纳结果的空间。

`.install(slot_index, internal_index, item_id)` 仅 SELF

将便携物品（Portable Battery / Portable Storage Bin）安装到容器模块的内部插槽中。Rover 不使用此方法；其固定插槽只接受 Nav、Sonar 和 Drill 功能模块，从不接受容器。模块化版本参见 Pioneer。

参数

名称	类型	描述
slot_index	number	安装容器（Battery Holder / Cargo Rack）的整数插槽。
internal_index	number	容器内部插槽中从零开始的整数索引。
item_id	string	要安装的便携物品的商店 ID。

返回值

返回值	结果字段	成功载荷
ActionResult	.status、.message	无

结果

状态	类型	含义
"ok"	成功	操作成功完成。
"not_at_service_point"	拒绝	载具未停在所需服务区域。
"locked"	拒绝	所需功能、配方或操作尚未解锁。
"invalid_slot"	拒绝	提供的插槽不存在。
"invalid_internal_slot"	拒绝	提供的内部插槽不存在。
"slot_empty"	拒绝	所选插槽为空。
"not_container"	拒绝	提供的目标不是兼容的容器。
"item_not_accepted"	拒绝	目标不接受提供的物品。
"internal_slot_occupied"	拒绝	所选内部插槽已被占用。
"item_not_in_inventory"	拒绝	Inventory 不包含所请求的物品。

`.uninstall(slot_index, internal_index)` 仅 SELF

从容器模块的内部插槽卸下便携物品。Rover 不使用此方法，原因与 `install` 相同。模块化版本参见 Pioneer。

参数

名称	类型	描述
slot_index	number	安装容器的插槽。
internal_index	number	容器内部插槽中从零开始的整数索引。

返回值

返回值	结果字段	成功载荷
ActionResult	.status、.message	无

结果

状态	类型	含义
"ok"	成功	操作成功完成。
"not_at_service_point"	拒绝	载具未停在所需服务区域。
"invalid_slot"	拒绝	提供的插槽不存在。
"invalid_internal_slot"	拒绝	提供的内部插槽不存在。
"slot_empty"	拒绝	所选插槽为空。
"not_container"	拒绝	提供的目标不是兼容的容器。
"internal_slot_empty"	拒绝	所选内部插槽为空。
"inventory_full"	拒绝	Inventory 没有容纳结果的空间。
"container_not_empty"	拒绝	模块的货物容器并非空置。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` 仅 SELF

读取脚本运行期间从编辑器 Commands 标签页发送的命令。每台可编程机器都相同，参见指南：Script Commands。

组件 / 载具与模块

Sonar Module（声纳模块） 模块

通过 `self.sonar` 查找并勘测世界中的地点。扫描会检查载具周围区域；仅驾驶不会触发扫描。使用 `get_component("nocturna").points_of_interest()` 查找未扫描的 "?" 标记，移动到其中一个附近，然后调用 `scan()` 和 `survey(site)`。Basic 声纳的范围为 50 m，可探测硬度最高为 1 的矿物；Wide 范围为 180 m、硬度 3；Deep 范围为 280 m、硬度 4。研究会解锁热液喷口、水井和奇异矿床。结果会保存到 Journal。本地采收机区域、生物地点和辐射场使用不同的扫描器。

由 `self.sonar` 返回。

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ① 以查找准确 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件稳定的程序化标识符。将它与 `get_component(id)` 以及要求组件、行星、载具、空间站或订单 ID 的 API 一起使用。

返回值：`String`

`.name`

面向用户的可读显示名称。对于需要经受重命名的脚本，优先使用 `.id`。

返回值：`String`

方法

`.scan()` 仅 SELF

扫描载具当前位置范围内兼容的 **Site**（地点）。接触类型包括 **MiningSite**、**ThermalVent**、**WaterWell**、**OilWell**、**ExoticDeposit** 和 **GeologicalAnomaly**。完成扫描后，Journal 会更新新分类的接触点。

返回值	结果字段	成功载荷
<code>SonarScanResult</code>	<code>.status</code> 、 <code>.message</code>	<code>.sites</code>

结果

状态	类型		含义
"ok"	成功	声纳扫描完成。 <code>result.sites</code> 包含找到的所有兼容接触点；空列表表示范围内没有兼容接触点。	
"too_hard"	部分完成	已完成的扫描检测到附近有尚未解析且硬度超过已安装声纳上限的接触点。	
"tier_too_low"	部分完成	已完成的扫描检测到附近有尚未解析且已安装声纳层级无法分类的接触点。	
"research_required"	部分完成	已完成的扫描检测到附近有尚未解析且其分类研究尚未解锁的接触点。	
"wrong_scanner"	部分完成	已完成的扫描检测到附近有载具声纳无法分类的接触点。	
"busy"	暂时性	另一项野外操作当前占用载具。	
"no_power"	拒绝	载具电池没有足够能量完成声纳扫描。	

抛出异常

异常	条件
<code>ReferenceError</code>	此 <code>SonarModule</code> 引用已失效，因为其模块已不再安装。安装声纳后重新读取 <code>self.sonar</code> 。

`.survey(site)` 仅 SELF

显示生产性 **Site**（地点）可用的详细信息。传入其字符串 ID，或传入 `scan()` 返回的 **Site**。重复勘测免费且即时，除非更高声纳层级能揭示更多信息。无活动的 **GeologicalAnomaly** 接触点在扫描时已完成解析。

参数

名称		类型	描述
site	any	地点 ID（字符串）、来自 <code>scan().sites</code> 的 Site ，或带有字符串 <code>id</code> 字段的字典或类实例。该地点必须仍处于已发现状态、在范围内，并且在此声纳的能力范围内。	

返回值

返回值	结果字段	成功载荷
<code>SurveyResult</code>	<code>.status</code> 、 <code>.message</code>	<code>.site</code>

结果

状态	类型	含义
"ok"	成功	已成功勘测 <code>site_id</code> 。
"busy"	暂时性	另一项野外操作当前占用载具。
"not_discovered"	拒绝	地点 <code>site_id</code> 不在此行星的已发现地点 Journal 中。
"research_required"	拒绝	地点 <code>site_id</code> 所需研究尚未解锁。
"tier_too_low"	拒绝	已安装声纳层级无法勘测地点 <code>site_id</code> 。
"out_of_range"	拒绝	地点 <code>site_id</code> 超出已安装声纳的当前范围。
"too_hard"	拒绝	地点 <code>site_id</code> 超出已安装声纳的矿物硬度上限。
"no_power"	拒绝	载具电池没有足够能量勘测地点 <code>site_id</code> 。

抛出异常

异常	条件
<code>ValueError</code>	<code>survey()</code> 需要非空地点 ID 或 Site 对象。
<code>ReferenceError</code>	此 <code>SonarModule</code> 引用已失效，因为其模块已不再安装。安装声纳后重新读取 <code>self.sonar</code> 。

`.range()`

当前声纳的范围，单位为米。

返回值：数值（米）：Basic 为 50，Wide 为 180，Deep 为 280。已捕获但已失效的模块引用会抛出 `ReferenceError`。

抛出异常

异常	条件
<code>ReferenceError</code>	此 <code>SonarModule</code> 引用已失效，因为其模块已不再安装。安装声纳后重新读取 <code>self.sonar</code> 。

`.hardness_limit()`

此声纳可识别的最大矿物硬度。

返回值：数值：Basic 为 1，Wide 为 3，Deep 为 4。已捕获但已失效的模块引用会抛出 `ReferenceError`。

抛出异常

异常	条件
<code>ReferenceError</code>	此 <code>SonarModule</code> 引用已失效，因为其模块已不再安装。安装声纳后重新读取 <code>self.sonar</code> 。

`.tier()`

此声纳提供的勘测深度层级：`"basic" / "wide" / "deep"`。它控制 `survey()` 能揭示的热液喷口或奇异矿床信息量；发现油井也要求使用 `"deep"`。

返回值

返回值	可能值
此声纳提供的勘测深度层级： <code>"basic" / "wide" / "deep"</code> 。控制热液喷口/奇异矿床细节；发现油井要求 <code>"deep"</code> 。已捕获但已失效的模块引用会抛出 <code>ReferenceError</code> 。	<code>"basic"</code> 、 <code>"wide"</code> 、 <code>"deep"</code>

抛出异常

异常	条件
<code>ReferenceError</code>	此 <code>SonarModule</code> 引用已失效，因为其模块已不再安装。安装声纳后重新读取 <code>self.sonar</code> 。

组件 / 物流与订单

Data Archive（数据存档）

存储可安全表示为 JSON 的数据，这些数据可跨脚本重启以及保存/加载而保留。研究解锁后，通过 `get_component("notebook")` 访问 Data Archive。使用 Libraries 共享代码，使用 Signal Bus 共享临时实时状态。

由 `get_component("notebook")` 返回。

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ① 以查找准确 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件稳定的程序化标识符。将它与 `get_component(id)` 以及要求组件、行星、载具、空间站或订单 ID 的 API 一起使用。

返回值：`String`

`.name`

面向用户的可读显示名称。对于需要经受重命名的脚本，优先使用 `.id`。

返回值：`String`

方法

`.set(key, value)`

在命名键下存储可安全表示为 JSON 的值。存档最多容纳 512 个条目；每个值支持 8 层嵌套，所有值和容器合计最多 16,384 个节点，每个字符串或字典键最多 4,096 个字符。

参数

名称	类型	描述
key	string	存档键，长度 1 – 96 个字符，只能使用字母、数字、_、.、: 或 -。
value	any	可安全表示为 JSON 的值，最多 8 层嵌套、所有值和容器合计 16,384 个节点，每个字符串或字典键最多 4,096 个字符。

返回值

返回值	结果字段	成功载荷
ActionResult	.status、.message	无

结果

状态	类型	含义
"ok"	成功	操作成功完成。
"invalid_key"	拒绝	提供的键无效。
"entry_limit"	拒绝	存档已达到条目上限。
"invalid_value"	拒绝	提供的值无效。

`.transaction(key, default, updater)`

在相同的存档值限制内，以原子方式转换一个已存储值。`updater` 可以是任意纯可调用对象；它接收最新值或提供的默认值，不能休眠、让出执行权或改变世界状态。

参数

名称	类型	描述
key	string	存档键，长度 1 – 96 个字符，只能使用字母、数字、_、.、: 或 -。
default	any	存档值限制内的 JSON 安全值；键缺失时使用。
updater	any	纯可调用对象，接收当前值并返回存档值限制内的下一个值。

返回值

返回值	结果字段	成功载荷
ActionResult	.status、.message	无

结果

状态	类型	含义
"ok"	成功	操作成功完成。
"invalid_key"	拒绝	提供的键无效。
"entry_limit"	拒绝	存档已达到条目上限。
"invalid_value"	拒绝	提供的值无效。
"busy"	暂时性	组件正在执行另一项操作。

`.get(key, default=None)`

按键读取已存储的值。如果键缺失，则返回可选的默认参数；如果未提供默认值，则返回 **None**。读取不会消耗或修改条目。

参数

名称	类型	描述
key	string	存档键，长度 1–96 个字符，只能使用字母、数字、_、.、: 或 -。
default	any	键缺失时返回的值。

返回值

存储的值、可选的默认值或 **None**。

抛出异常

异常	条件
ValueError	存档键必须由 1–96 个字母、数字、_、.、: 或 - 组成，且不能是保留的对象字段名称。

`.has(key)`

当存档包含该键时返回 **True**，否则返回 **False**。

参数

名称	类型	描述
key	string	存档键，长度 1–96 个字符，只能使用字母、数字、_、.、: 或 -。

返回值

布尔值：存档包含该键时为 **True**。

抛出异常

异常	条件
ValueError	存档键必须由 1–96 个字母、数字、_、.、: 或 - 组成，且不能是保留的对象字段名称。

`.delete(key)`

删除一个键。

参数

名称	类型	描述
key	string	存档键，长度 1 – 96 个字符，只能使用字母、数字、_、.、: 或 -。

返回值

返回值	结果字段	成功载荷
ActionResult	.status、.message	无

结果

状态	类型	含义
"ok"	成功	操作成功完成。
"not_found"	拒绝	请求的对象、目标或记录不存在。
"invalid_key"	拒绝	提供的键无效。

`.keys(prefix="")`

以排序列表形式返回存档键。传入如 "rover." 的前缀可只列出匹配的键。

参数

名称	类型	描述
prefix	string	可选键前缀，使用存档键字符集。

返回值

已排序的存档键列表，可选择按前缀筛选。

抛出异常

异常	条件
ValueError	存档前缀必须只使用字母、数字、_、.、: 或 -，且长度最多为 96 个字符。

```
.clear(prefix="")
```

移除存档条目。不带前缀时清除整个存档；带前缀时清除匹配的键。

参数

名称	类型	描述
prefix	string	可选键前缀，使用存档键字符集。

返回值

返回值	结果字段	成功载荷
CountResult	.status、.message	.count

结果

状态	类型	含义
"ok"	成功	命令影响了 .count 个条目或单位。
"no_op"	成功	命令未影响任何条目或单位。
"invalid_key"	拒绝	提供的键无效。

组件 / 物流与订单

Drone Depot（无人机仓库）

前哨的单舱物流端点。货物输入/输出。

项目	值
类型	Mining
输入功率	-2 W（从电网取电）
储备量	50 单位（混合）

获取方式

- 完成 Helios、Frame Order 后解锁配方。
- 需要 Basic Drone Operations 研究（Terraform Index 180,000）。
- 在 Fabricator 上制造 Drone Depot Kit：2× Machine Frame、1× Control Unit、2× Gas Pipe Segment、2× Liquid Pipe Segment，以及 3 t Water。
- 从 Inventory 部署。

由 `self/get_component(id)` 返回。

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ① 以查找准确 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件稳定的程序化标识符。将它与 `get_component(id)` 以及要求组件、行星、载具、空间站或订单 ID 的 API 一起使用。

返回值：`String`

`.name`

面向用户的可读显示名称。对于需要经受重命名的脚本，优先使用 `.id`。

返回值：`String`

方法

`.outpost`

此建筑所部署的前哨。返回的 `OutpostRef` 包含其稳定 ID、显示名称、生物群系、位置、容量和 `buildings()` 查询。需要当前值时重新读取此属性。

返回值：此建筑所在前哨的 `OutpostRef`。

`.input`

此站点本地储备的 `InputSlot`。在同一前哨连接 "inventory"、机器或存储源，然后调用 `self.input.take(item_id, count)`。`self.input.flush()` 会丢弃站点储备。在另一座 Drone Depot 的库存对这里不可见。

返回值：`InputSlot`：`connect()`、`take()`、`eject()`、`flush()`、`count()`、`capacity()`、`connected_to()`。

`.output`

此站点本地储备的 `OutputSlot`。在同一前哨连接 "inventory"、机器或存储目标，然后调用 `self.output.send(item_id, count)`。使用无人机在前哨之间移动货物；各站点不会共享库存。

返回值：`OutputSlot`。

`.get_docked()`

按稳定 ID 顺序返回当前停靠在此 Depot 的无人机 ID 列表。使用 `get_component(id)` 读取每架无人机的状态。

返回值：当前停靠在此 Depot 的无人机实例 ID 列表，按稳定 ID 排序。

`.bay_count()`

此站点的舱位总数：1 (basic) / 2 (medium) / 4 (large)。

返回值：数值：此站点的舱位总数（取决于层级，为 1/2/4）。

`.bays_occupied()`

当前已被停靠无人机占用的舱位数。当它等于 `bay_count` 时，抵达的无人机会在空域中排队。

返回值：当前被停靠无人机占用的舱位数。

`.slots_used()`

储备当前持有的不同材料数量。无论存储多少单位，一种材料都只占用一个插槽，因此同一种材料的 50 单位只占一个插槽。

返回值：当前持有的不同材料数。每种材料无论单位数量多少都占一个插槽。

`.slot_capacity()`

此仓库一次可容纳的不同材料数量。仓库是转运代理而不是仓库：当所有插槽都被占用时，即使仍有空余单位容量，对新材料执行 `cargo.unload()` 也会移动 0 单位并报告没有空闲插槽。将某种材料排出即可释放其插槽。

返回值：此仓库一次可容纳的不同材料数量（取决于层级，为 3/4/6）。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` 仅 SELF

读取脚本运行期间从编辑器 Commands 标签页发送的命令。每台可编程机器都相同，参见指南：Script Commands。

组件 / 物流与订单

无人机服务站（Drone Service Station）

为野外的电动无人机充电，并回收等待加油的直升机无人机。连接电网。

类型	采矿
输入电力	可变（从电网取电）
消耗	石油，缓冲容量 100 t

获取方式

- 完成“尖塔，无人机电力试炼”（Spire, Drone Power Trial）后解锁配方。
- 需要“基础无人机操作”（Basic Drone Operations）研究（Terraform Index 180,000）。
- 在制造机（Fabricator）上制作无人机服务站套件（Drone Service Station Kit）：2×机器框架（Machine Frame）、1×控制单元（Control Unit）、2×电路面板（Circuit Panel）、1×电池单元（Battery Cell）和 1×液体管道段（Liquid Pipe Segment）。
- 从 Inventory 部署。

返回方式：self / get_component(id)

每个组件都有稳定的 .id。对于已部署的机器，打开其卡片上的 ① 查看确切 ID，然后将该值传给 get_component(id)。ID 区分大小写。

属性

.id

此组件稳定的程序标识符。将它与 get_component(id) 以及要求提供组件、星球、载具、站点或订单 ID 的 API 一起使用。

返回值

String

.name

人类可读的显示名称。对于需要在重命名后仍能工作的脚本，优先使用 .id。

返回值

String

方法

`.outpost`

部署此建筑的前哨站（Outpost）。返回的 OutpostRef 包含其稳定 ID、显示名称、生物群系、位置、容量以及 `buildings()` 查询。需要当前值时重新读取该属性。

返回值

部署此建筑的前哨站的 OutpostRef。

`.get_docked()`

列出当前停放在此站的所有无人机 ID（电动和直升机）。通过 `get_component(id)` 读取每架无人机的状态。

返回值

当前停靠在此站的所有无人机 ID（电动和直升机）列表。

`.charge(drone_id, target_level=1.0)` 仅限 SELF

将停放的电动无人机加入充电队列，直到电池达到 `target_level`（大于 0 且不超过 1，默认为 1.0）。电动充电和直升机加油共用一个先进先出（FIFO）队列及相同的服务舱位。因缺油而受阻的直升机仍保留队列位置，但不占用舱位，因此就绪的电动任务可以越过它。示例：`self.charge("drone_small_1", 0.8)`。仅限 self。

参数

名称	类型	描述
<code>drone_id</code>	string	已停靠电动无人机的显示名称或 ID
<code>target_level</code>	number	可选电池目标比例，大于 0 且不超过 1。默认为 1.0。

返回值

返回类型	结果字段	成功载荷
ActionResult	<code>.status</code> 、 <code>.message</code>	None

结果

状态	类型	含义
"charging"	success	充电任务处于活动状态。
"queued"	success	操作已加入队列。
"target_reached"	rejection	载具已经达到或超过请求的充电或燃料水平。
"not_docked"	rejection	请求的载具未停靠在此站。
"station_offline"	transient	站点处于离线状态。
"invalid"	rejection	一个或多个提供的参数超出可接受范围。

`.refuel(drone_id, target_level=1.0)` 仅限 SELF

将停放的直升机无人机加入加油队列，直到其油箱达到 `target_level`（大于 0 且不超过 1，默认为 1.0）。电动充电和直升机加油共用一个先进先出（FIFO）队列及相同的服务舱位。油料从 `self.oil_in` 取用。仅限 self。

参数

名称	类型	描述
<code>drone_id</code>	string	已停靠直升机的显示名称或 ID
<code>target_level</code>	number	可选油料目标比例，大于 0 且不超过 1。默认为 1.0。

返回值

返回类型	结果字段	成功载荷
ActionResult	<code>.status</code> 、 <code>.message</code>	None

结果

状态	类型	含义
"refueling"	success	加油任务处于活动状态。
"queued"	success	操作已加入队列。
"target_reached"	rejection	载具已经达到或超过请求的充电或燃料水平。
"not_docked"	rejection	请求的载具未停靠在此站。
"station_offline"	transient	站点处于离线状态。
"no_oil"	rejection	操作没有可用石油。
"invalid"	rejection	一个或多个提供的参数超出可接受范围。

`.dispatch_rescue(drone_name, target_level=1.0)` 仅限 SELF

将回收载具派往脚本选定的野外无人机；不存在隐藏的燃料阈值。电动无人机会在野外充电至 `target_level`，然后继续其路线。直升机无人机会被带回基地，再加入常规加油队列。失控的无人机也会被带回基地，以便通过停靠重置其电子设备。`target_level` 必须大于 0 且不超过 1，默认为 1.0。发射需要站点供电，但任务可以在后续断电期间完成。仅限 self。

参数

名称	类型	描述
<code>drone_name</code>	string	要服务的受困无人机的显示名称或 ID。
<code>target_level</code>	number	可选电池/油料目标，大于 0 且不超过 1。电动无人机在野外充至该水平；直升机回收后提交给站点加油 FIFO。默认为 1.0。

返回值

返回类型	结果字段	成功载荷
ActionResult	<code>.status</code> 、 <code>.message</code>	None

结果

状态	类型	含义
"ok"	success	操作成功完成。
"already_dispatched"	rejection	此站已有活动中的救援任务，或请求目标已分配给某项救援。
"station_offline"	transient	站点处于离线状态。
"not_found"	rejection	请求的对象、目标或记录不存在。
"not_stranded"	rejection	目标不被视为受困。
"invalid"	rejection	一个或多个提供的参数超出可接受范围。

`.cancel_rescue()` 仅限 SELF

中止正在执行的救援。无人机将在当前位置被释放（保留已获得的充电量），服务载具飞回基地。仅限 self。

返回值

返回类型	结果字段	成功载荷
ActionResult	<code>.status</code> 、 <code>.message</code>	None

结果

状态	类型	含义
"ok"	success	操作成功完成。
"no_rescue"	rejection	没有活动中的救援任务。

`.is_rescuing()`

服务载具执行救援任务期间为 True。

返回值

Boolean；服务载具正在执行任务期间为 True。

`.get_rescue_target()`

服务载具正在出发、服务、运输或返程时，返回任务目标的显示名称；空闲时返回空字符串。

返回值

String；服务载具正在出发、服务、运输或返程时为任务目标的显示名称，空闲时为空字符串。

`.stop(drone_id)` 仅限 SELF

取消此站上的一个活动中或排队中的任务，无论是充电还是加油。无人机会保留已经获得的电量或油量。

参数

名称	类型	描述
<code>drone_id</code>	string	排队中无人机的显示名称或 ID

返回值

返回类型	结果字段	成功载荷
ActionResult	<code>.status</code> 、 <code>.message</code>	None

结果

状态	类型	含义
"ok"	success	操作成功完成。
"not_found"	rejection	请求的对象、目标或记录不存在。
"invalid"	rejection	一个或多个提供的参数超出可接受范围。

`.clear_queue()` 仅限 SELF

清除此站所有活动中或排队中的充电和加油任务。停靠的无人机保持停放状态，并保留当前电池电量和油量。

返回值

返回类型	结果字段	成功载荷
CountResult	<code>.status</code> 、 <code>.message</code>	<code>.count</code>

结果

状态	类型	含义
"ok"	success	命令影响了 <code>.count</code> 个条目或单位。
"no_op"	success	命令没有影响任何条目或单位。

`.get_active()`

列出当前占用活动服务舱位的无人机 ID（充电或加油）。因缺油而受阻的直升机处于等待状态，而非活动状态。

返回值

当前占用活动服务舱位的无人机 ID 列表（充电或加油）。因缺油而受阻的直升机任务处于等待状态，而非活动状态。

`.get_queue()`

返回一份涵盖电动充电和直升机加油的无人机 ID

先进先出列表。因缺油而受阻的直升机保持顺序，同时后面已就绪的任务可以使用原本空闲的舱位。

返回值

涵盖电动充电和直升机加油的排队无人机 ID

先进先出列表。因缺油而受阻的直升机保持此顺序，同时后面已就绪的任务可以使用原本空闲的舱位。

`.status(drone_id)`

返回一架无人机服务任务的详细状态。电动无人机返回一个字典，包含 `state`、`target_level`、`battery_wh`、`capacity_wh`、`rate_w`、`bay_index`、`queue_index`。直升机无人机返回 `state`、`target_level`、`oil_tons`、`capacity_tons`、`rate_tons_per_hour`、`bay_index`、`queue_index`。

参数

名称	类型	描述
<code>drone_id</code>	<code>string</code>	无人机的显示名称或 ID

返回值

此无人机活动中/排队中服务任务的 Dict。电动无人机：`state`、`target_level`、`battery_wh`、`capacity_wh`、`rate_w`、`bay_index`、`queue_index`。直升机无人机：`state`、`target_level`、`oil_tons`、`capacity_tons`、`rate_tons_per_hour`、`bay_index`、`queue_index`。

`.get_bay_count()`

同时可用的服务舱位数量。

返回值

同时可用的服务舱位数量。

`.get_charge_rate(drone_id)`

返回当前注入指定电动无人机的 Wh/h；如果它不在活动舱位中，则返回 0。

参数

名称	类型	描述
drone_id	string	已停靠电动无人机的显示名称或 ID

返回值

Number；当前注入该电动无人机的 Wh/h（如果未充电则为 0）。

`.get_refuel_rate(drone_id)`

返回当前注入指定直升机无人机的油料 t/h；如果它不在活动舱位中或站点没有石油，则返回 0。

参数

名称	类型	描述
drone_id	string	已停靠直升机无人机的显示名称或 ID

返回值

Number；当前注入该直升机无人机的油料 t/h（如果未加油则为 0）。

`.oil_in`

用于直升机加油的 Oil FluidPort。调用 `self.oil_in.connect(...)`，并传入油泵或油罐的稳定机器 ID 或显示名称。远程来源还需要在两个位置之间建立已完成且无冲突的液体管道路线。

返回值

用于直升机加油供应的 FluidPort。调用 `connect(...)` 并传入供应方的稳定机器 ID 或显示名称；本地供应方会直接传输，远程供应方则使用已完成的液体管道网络。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` 仅限 SELF

读取脚本运行期间从编辑器 Commands 选项卡发送的命令。所有可编程机器上的行为均相同，参见指南：Script Commands。

组件 / 物流与订单

地球订单 (Earth Orders)

通过 `get_component("orders")` 列出地球当前的供应链需求。Supply Dock 脚本可以使用 Campaign 和 Weekly Orders 决定要运输的货物，仪表板也可以显示进度。脚本不能创建或取消 Earth Orders。Bio Orders 来自 Bio Exchange。

返回方式：`get_component("orders")`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ⓘ 查看确切 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件稳定的程序标识符。将它与 `get_component(id)` 以及要求提供组件、星球、载具、站点或订单 ID 的 API 一起使用。

返回值

String

`.name`

人类可读的显示名称。对于需要在重命名后仍能工作的脚本，优先使用 `.id`。

返回值

String

方法

`.list_orders()`

将当前承包商 Campaign Orders 作为稳定的 Order 对象列表返回。这些订单永不过期，完全运输后会从此列表中消失。空列表表示当前没有可用的承包商运输任务。参见 Order。

返回值

当前承包商 Campaign Order 对象列表。

`.list_weekly_orders()`

返回当前五个 Weekly Earth Orders，包括本周期内已经完成的报价。如果尚无符合条件的生产链，则返回空列表。Weekly 对象的 `.kind == "weekly"`、`.expires_day`，没有 `contractor`，仅提供 `credits` 奖励，`.status` 为 `"active"` 或 `"completed"`。整个列表每七天替换一次。

返回值

当前五个 Weekly Earth Order 对象，包括本周期内已经完成的报价；在有符合条件的生产链之前为空列表。

.get_order(order_id)

按 ID 查找特定的 Earth Order。返回当前的 Campaign 或 Weekly Earth Order，或已完成的 Campaign Order。过期的 weekly ID 返回 None；weekly 完成记录只会保留到订单板刷新。参见 Order。

参数

名称	类型	描述
order_id	string	来自 list_orders()、list_weekly_orders() 或 completed_orders() 的 Earth Order ID

返回值

匹配当前承包商或 Weekly Earth Order 的 Earth Order，或已完成的承包商订单。过期的 weekly ID 返回 None。Bio Order ID 从 Bio Exchange 读取。

.completed_orders()

永久保存的承包商 Campaign 历史记录，按完成时间排序（最早的在前）。Weekly 完成记录保留在当前 Weekly 订单板中，并有意排除在此账本之外。

返回值

按完成顺序排列的、已经交付的 Earth Order 对象列表。

组件 / 物流与订单

信号总线 (Signal Bus)

通过共享的 JSON-safe 值协调脚本。研究解锁后，通过 `get_component("comms")` 访问 Signal Bus。使用 `send()` 和 `receive()` 处理应只执行一次的工作；使用 `broadcast()` 和 `latest()` 获取最新的共享值。

返回方式：`get_component("comms")`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ① 查看确切 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件稳定的程序标识符。将它与 `get_component(id)` 以及要求提供组件、星球、载具、站点或订单 ID 的 API 一起使用。

返回值

String

`.name`

人类可读的显示名称。对于需要在重命名后仍能工作的脚本，优先使用 `.id`。

返回值

String

方法

`.send(channel, value)`

将 JSON-safe 值添加到指定名称的频道队列。保留发送回执，以便稍后识别或取消该确切请求，即使多个请求包含相同值也不受影响。频道 ID 可以包含字母、数字、`_`、`.`、`:` 和 `-`。对于应只处理一次的工作项，请使用队列。

参数

名称	类型	描述
<code>channel</code>	<code>string</code>	频道 ID，长度为 1-64 个字符，只能使用字母、数字、 <code>_</code> 、 <code>.</code> 、 <code>:</code> 或 <code>-</code>
<code>value</code>	<code>any</code>	JSON-safe 值，最多嵌套 8 层、总计 1,024 个值，每个字符串最多 4,096 个字符

返回值

返回类型	结果字段	成功载荷
<code>SendResult</code>	<code>.status</code> 、 <code>.message</code>	<code>.message_id</code>

结果

状态	类型	含义
<code>"ok"</code>	<code>success</code>	一条消息已追加到频道队列，其 ID 可在 <code>message_id</code> 中取得。
<code>"invalid_channel"</code>	<code>rejection</code>	提供的频道 ID 不符合 Signal Bus 命名规则。没有排入消息。
<code>"channel_limit"</code>	<code>rejection</code>	频道不存在，且 Signal Bus 没有容量创建另一个频道。没有排入消息。
<code>"queue_full"</code>	<code>rejection</code>	频道队列已达到消息上限。没有排入消息。
<code>"id_exhausted"</code>	<code>rejection</code>	没有可用的稳定消息 ID。没有排入消息。
<code>"invalid_value"</code>	<code>rejection</code>	提供的值违反 Signal Bus 值规则。没有排入消息。

`.receive(channel, message_id=None)`

取出一条排队消息。省略 `message_id` 或传入 `None` 时取出最旧消息；提供消息 ID 时精确取出该任务。选择和移除会同时发生，因此只有一个竞争中的接收方可以取出它。其他排队消息保持顺序，广播消息会保留。使用 `pending()` 可以在接收前按优先级、位置或能力选择工作。

参数

名称	类型	描述
<code>channel</code>	string	频道 ID，长度为 1-64 个字符，只能使用字母、数字、 <code>_</code> 、 <code>.</code> 、 <code>:</code> 或 <code>-</code>
<code>message_id</code>	number	来自 <code>pending()</code> 或成功 <code>send()</code> 回执的正整数消息 ID。省略或传入 <code>None</code> 时接收最旧消息；ID 会选择该确切消息，而不是队列中的位置。

返回值

返回类型	结果字段	成功载荷
<code>ReceiveResult</code>	<code>.status</code> 、 <code>.message</code>	<code>.packet</code>

结果

状态	类型	含义
<code>"ok"</code>	success	一条消息已从队列移除并返回。没有消息 ID 时取出最旧消息；有消息 ID 时取出指定消息。
<code>"empty"</code>	success	未请求消息 ID，且频道没有排队消息。没有消耗任何消息。
<code>"not_found"</code>	rejection	提供的消息 ID 不在此频道等待。没有消耗其他消息。
<code>"invalid_channel"</code>	rejection	提供的频道 ID 不符合 Signal Bus 命名规则。

引发异常

异常	条件
<code>TypeError</code>	<code>message_id</code> 必须是支持的整数范围内的有限整数。
<code>ValueError</code>	<code>message_id</code> 必须大于零。
<code>OverflowError</code>	<code>message_id</code> 必须适合支持的整数范围。

`.wait(channel)`

等待并取出最旧的排队消息。如果队列为空，只有此脚本暂停，游戏和其他脚本继续运行。已经排队的消息会立即取出。广播不能满足此等待。暂停会保留等待状态，停止脚本会放弃等待但不会消耗消息。

参数

名称	类型	描述
<code>channel</code>	string	频道 ID，长度为 1-64 个字符，只能使用字母、数字、_、.、: 或 -。有效但不存在的频道会等待其第一条排队消息。

返回值

返回类型	结果字段	成功载荷
ReceiveResult	<code>.status</code> 、 <code>.message</code>	<code>.packet</code>

结果

状态	类型	含义
<code>"ok"</code>	success	一条消息已从队列移除并返回。没有消息 ID 时取出最旧消息；有消息 ID 时取出指定消息。
<code>"invalid_channel"</code>	rejection	提供的频道 ID 不符合 Signal Bus 命名规则。

.wait_any(channels)

等待并从列出的任一频道取出一条排队消息。列表靠前的频道在选择工作时拥有优先级；每个频道内部仍按最旧消息优先。所有队列都为空时，只有此脚本等待。广播不能满足此等待。调用时会复制频道列表，重复名称只按其第一次出现的位置计算。暂停会保留等待状态；停止脚本会放弃等待而不取走工作。

参数

名称	类型		描述
channels	list	由 1-128 个频道 ID 字符串组成的列表，按优先级排列。每个 ID 长度为 1-64 个字符，只能使用字母、数字、_、.、: 或 -。不存在的频道之后可能收到工作。重复名称保留其第一次出现的位置。	

返回值

返回类型	结果字段	成功载荷
WaitAnyResult	.status、.message	.channel、.packet

结果

状态		类型	含义
"ok"	success	从列表中第一个有等待任务的频道消耗一条排队消息。结果包含该频道及其复制的消息。	
"invalid_channel"		rejection	至少一个列出的频道 ID 无效。没有消耗消息；channel 和 packet 为 None。

引发异常

异常	条件
TypeError	channels 必须是只包含频道 ID 字符串的列表。
ValueError	channels 必须包含 1-128 个条目。

.wait_broadcast(channel)

等待频道上的下一次广播。所有已经等待的脚本都会捕获该发布，包括重复值或 None。已有广播不能满足新的等待。只有此脚本暂停；排队消息保持不变。即使随后又发生广播或频道被清除，也会保留第一次发布。暂停会保留该信号以便恢复；停止脚本会放弃等待。

参数

名称		类型	描述
channel	string	频道 ID，长度为 1-64 个字符，只能使用字母、数字、_、.、: 或 -。有效但不存在的频道也可以被监听。只有在调用后于此频道成功广播的消息才能满足等待。	

返回值

返回类型	结果字段	成功载荷
WaitBroadcastResult	.status、.message	.broadcast

结果

状态		类型	含义
"ok"	success	调用后发布的第一条成功广播已被捕获，且没有消耗共享状态。每个已经等待的脚本都会收到自己的副本。	
"invalid_channel"		rejection	提供的频道 ID 不符合 Signal Bus 命名规则。

`.pending(channel)`

列出某个通道中当前等待处理的所有消息，并按接收顺序返回。每条消息及其嵌套值都会被复制；编辑返回的列表或消息不会改变信号总线。广播消息不会出现在列表中。

参数

名称	类型	描述
<code>channel</code>	<code>string</code>	通道 ID，长度为 1-64 个字符，只能使用字母、数字、 <code>_</code> 、 <code>.</code> 、 <code>:</code> 或 <code>-</code> 。

返回值

等待中的 `CommsMessage` 条目列表，按进入队列的顺序排列。列表为空时表示没有等待中的消息。

异常

异常	条件
<code>ValueError</code>	<code>channel</code> 不符合信号总线命名规则，或属于保留的对象字段名。

`.cancel(channel, message_id)`

取消一条等待中的消息。可以使用发送回执中的 ID，或使用 `pending(channel)` 返回的 ID，来从脚本或 Control Room 卡片中取消投递。其他消息会保留原有 ID、队列位置、原始发送者和发送时间。取消只会移除队列中尚未被接收的工作；已经被接收的消息无法取消。

参数

名称	类型	描述
channel	string	通道 ID，长度为 1-64 个字符，只能使用字母、数字、_、.、: 或 -。
message_id	number	等待中消息的正整数 ID，来自成功的 <code>send()</code> 回执或 <code>pending(channel)</code> ；它标识消息，而不是队列位置。

返回值

`ActionResult`，结果字段为 `.status`、`.message`，成功负载为无。

结果

状态	类型	含义
"ok"	成功	指定消息已从通道队列中移除。其他消息和最新广播保持不变。
"not_found"	拒绝	该通道中没有指定 ID 的排队消息，未移除任何内容。
"invalid_channel"	拒绝	通道 ID 不符合信号总线命名规则。

异常

异常	条件
<code>TypeError</code>	<code>message_id</code> 必须是支持范围内的有限整数。
<code>ValueError</code>	<code>message_id</code> 必须大于零。
<code>OverflowError</code>	<code>message_id</code> 必须适合支持的整数范围。

`.update(channel, message_id, value)`

替换一条等待中消息的完整值。使用发送回执或 `pending()` 返回的 ID，可以从脚本或 Control Room 卡片中编辑投递内容、优先级或目的地。ID、队列位置、原始发送者和发送时间保持不变。替换会在一次操作中完成，即使队列已满也有效。已经被接收的消息无法编辑。重新调用 `pending()` 可刷新之前的快照。

参数

名称	类型	描述
channel	string	通道 ID，长度为 1-64 个字符，只能使用字母、数字、_、.、: 或 -。
message_id	number	等待中消息的正整数 ID，来自成功的 <code>send()</code> 回执或 <code>pending(channel)</code> ；它标识消息，而不是队列位置。
value	any	完整的替换 JSON 安全值，最多嵌套 8 层、总计 1,024 个值，每个字符串或字典键最多 4,096 个字符。字典字段会被替换，而不是合并。

返回值

`ActionResult`，结果字段为 `.status`、`.message`，成功负载为无。

结果

状态	类型	含义
"ok"	成功	排队消息现在包含替换后的值。其 ID、队列位置、原始发送者和发送时间均保留。
"not_found"	拒绝	该通道中没有指定 ID 的排队消息，值未改变。
"invalid_channel"	拒绝	通道 ID 不符合信号总线命名规则。
"invalid_value"	拒绝	替换值违反信号总线值规则，排队消息未改变。

异常

异常	条件
<code>TypeError</code>	<code>message_id</code> 必须是支持范围内的有限整数。
<code>ValueError</code>	<code>message_id</code> 必须大于零。
<code>OverflowError</code>	<code>message_id</code> 必须适合支持的整数范围。

`.broadcast(channel, value)`

保存通道最新的 JSON 安全值，不占用队列槽位。广播适合共享舰队模式、目标区域或当前优先级等遥测数据。

参数

名称	类型	描述
channel	string	通道 ID，长度为 1-64 个字符，只能使用字母、数字、_、.、: 或 -。
value	any	JSON 安全值，最多嵌套 8 层、总计 1,024 个值，每个字符串最多 4,096 个字符。

返回值

`ActionResult`，结果字段为 `.status`、`.message`，成功负载为无。

结果

状态	类型	含义
"ok"	成功	操作成功完成。
"invalid_channel"	拒绝	提供的通道标识符无效。
"channel_limit"	拒绝	已达到通信通道数量上限。
"invalid_value"	拒绝	提供的值无效。

`.latest(channel)`

返回通道上最近广播的值；如果通道没有最新值，则返回 `None`。读取不会消耗该值。

参数：`channel` 为通道 ID，长度为 1-64 个字符，只能使用字母、数字、_、.、: 或 -。

返回值：通道上的最新广播值；如果尚未广播，则为 `None`。

异常：`ValueError`：通道 ID 必须符合信号总线命名规则，且不能是保留的对象字段名。

`.latest_info(channel)`

检查最新广播、发布者以及上次更新距今多久。使用该快照可以检测过时的工作者报告，或在 Control Room 卡片中显示新鲜度。读取不会消耗消息，也不会改变通道。重新读取可刷新值和时间。

参数：`channel` 为通道 ID，长度为 1-64 个字符，只能使用字母、数字、`_`、`.`、`:` 或 `-`。

返回值：包含 `value`、`sender` 和 `age_seconds` 的 `BroadcastInfo` 快照；如果通道没有广播则返回 `None`。如果保存的发送者或时间戳信息缺失，对应字段为 `None`。

异常：`ValueError`：通道 ID 必须符合信号总线命名规则，且不能是保留的对象字段名。

`.queue_size(channel)`

返回通道上等待中的排队消息数量。

参数：`channel` 为通道 ID，长度为 1-64 个字符，只能使用字母、数字、`_`、`.`、`:` 或 `-`。

返回值：通道上的排队消息数量。

异常：`ValueError`：通道 ID 必须符合信号总线命名规则，且不能是保留的对象字段名。

`.channels()`

返回当前存在排队消息或最新广播值的所有通道 ID。

返回值：带有排队状态或广播状态的通道 ID 列表。

`.clear(channel)`

移除某个通道的排队消息和最新广播。

参数：`channel` 为通道 ID，长度为 1-64 个字符，只能使用字母、数字、`_`、`.`、`:` 或 `-`。

返回值：`CountResult`，结果字段为 `.status`、`.message`，成功负载为 `.count`。

状态	类型	含义
"ok"	成功	命令影响了 <code>.count</code> 个条目或单位。
"no_op"	成功	命令未影响任何条目或单位。
"invalid_channel"	拒绝	通道 ID 不符合信号总线命名规则。

组件 / 物流与订单

Supply Dock

在吞吐量研究之前，以 25 units/h 的速度将成品运往 Earth。脚本会分配承包商或 Weekly Earth Order，装载所需物资并启用派送；订单完成或过期后，码头会停止运行，直到重新分配。

类型：Logistics

输入功率：-15 W（从电网汲取）

如何获得

- 需要 Supply Logistics 研究（Terraform Index 110,000）。
- 从 Shop 购买。起始价格为 5,000 cr，拥有的数量越多，价格越高。

由以下对象返回：`self` / `get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，找到精确 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

该组件的稳定程序化标识符。将它与 `get_component(id)` 以及要求组件、行星、车辆、空间站或订单 ID 的 API 一起使用。

返回值：String

`.name`

面向用户的显示名称。需要经受重命名的脚本应优先使用 `.id`。

返回值：String

方法

`.outpost`

部署该建筑的 outpost。返回的 **OutpostRef** 包含其稳定 ID、显示名称、生物群系、位置、容量以及 **buildings()** 查询。需要当前值时重新读取该属性。

返回值：部署该建筑的 outpost 的 **OutpostRef**。

`.capacity()`

当前 Order 中所有物品尚未满足的总单位数（即码头剩余需求）。未分配 Order 时返回 0。可将其作为仍需装载和运输数量的上限。

返回值：活动 Order 中所有物品尚未满足的总单位数；闲置时为 0。

`.total()`

汇总所有槽位中当前已装载的单位数。与 **capacity()** 比较可确定码头还需要吸收多少；**total() == 0** 表示所有槽位为空。

返回值：所有槽位当前单位数之和。

`.count(item_id)`

返回码头所有槽位中当前持有的 `item_id` 单位数。如果码头没有该物品，则返回 0。在继续装载前使用它可避免重复调用 `take()`：

```
if self.count("iron_ore") < 20: self.input.take("iron_ore", 20)
```

参数

名称	类型	描述
<code>item_id</code>	string	要统计的物品 ID。

返回值：当前所有槽位中的 `item_id` 单位数。

`.slots()`

将码头的实体槽位作为 `DockSlot` 对象列表返回（`.index`、`.item_id`、`.count`）。始终包含 5 个条目，索引为 0-4；当前 Order 未打开的槽位具有 `.item_id == None` 且 `.count == 0`。参见 `DockSlot`。

返回值：`DockSlot` 对象列表，每个实体槽位一个（索引 0-4）。

`.current_order()`

返回码头当前的 Earth Order；未分配 Earth Order 时返回 `None`。Earth Order 完成后会自动变为 `None`。在决定装载什么之前，可使用它读取 `.requires` 和 `.shipped`。

返回值：分配给该码头的 Order，或 `None`。

`.set_order(order_id)` 仅限 SELF

将 Earth Order 分配给该码头。通过 `orders.list_orders()` 或 `orders.list_weekly_orders()` 查找 ID，然后将其中一个传给 `self.set_order(id)`。多个码头可以服务同一订单并共享已运输进度。货物是实体的：切换订单前，必须通过本地机器或车辆清空该码头。

参数：`order_id` 为来自 `orders.list_orders()` 或 `orders.list_weekly_orders()` 的 Earth Order ID。

返回值：`ActionResult`，结果字段为 `.status`、`.message`，成功负载为无。

状态	类型	含义
"ok"	成功	操作成功完成。
"unknown_order"	拒绝	提供的订单未知，或当前不可分配。
"completed"	成功	请求的订单已完成，无法再次分配。
"cargo_present"	拒绝	现有货物阻止了请求的配置变更。

`.clear_order()` 仅限 SELF

释放该码头的分配并停止派送。已装载货物仍留在码头内。可直接使用 `self.input.eject(destination, item_id, count)` 回收，或将本地机器或车辆的输入连接到该 Supply Dock。

返回值：`ActionResult`，结果字段为 `.status`、`.message`，成功负载为无。

`.set_enabled(on)` 仅限 SELF

切换连续派送器。`True` 恢复运输；`False` 暂停运输。无论哪种状态，装载都不受影响，输入端口仍会接受物料。分配的订单完成时会自动关闭；下一次调用 `set_order` 后，脚本必须重新启用它。

参数：`on` 为 boolean，`True` 恢复派送，`False` 暂停派送。

返回值：`ActionResult`，结果字段为 `.status`、`.message`，成功负载为无。

状态	类型	含义
"ok"	成功	操作成功完成。

`.is_enabled()`

派送器处于活动状态时为 `True`。调用 `set_enabled(False)` 或分配的 Order 完成后会变为 `False`。新码头默认启用，因此在有货物装载时分配 Order 会立即开始运输。

返回值：派送器活动时为 `True`；调用 `set_enabled(False)` 后为 `False`。新部署的码头默认启用。

`.dispatch_rate()`

返回派送器当前的有效吞吐量，单位为 units/h，已包含吞吐量研究和 outpost 过度拥挤惩罚。基础速率为 25（每 2.4 分钟一个单位）；Bulk Logistics II 将其乘以 4，Bulk Logistics III 将其乘以 16。将返回值乘以经过的小时数，即可预测码头将运输多少单位。

返回值：当前派送器的吞吐量，单位为 units/h。

`.current_dispatch()`

返回派送器当前正在输出的 `item_id`；闲置时返回 `None`（无电力、无 Order、通过 `set_enabled(False)` 暂停，或没有装载可运输单位）。需要知道当前流动物料脚本可以使用它。

返回值：当前正在派送的物品 ID；码头闲置时返回 `None`。

`.dispatch_progress()`

返回当前单位累积器距离输出的 0-1 分数。码头没有装载可运输货物时保持为 0；可运输单位到达后开始计量。它驱动码头卡片上的周边时钟动画；脚本可以用它估算“X 小时后下一次发射”。

返回值：0-1 之间的数字，表示当前单位累积进度；达到 1.0 时输出一个单位并重置计数器。

`.input`

通过 `connect(...)` 和 `take(...)` 只装载活动 Order 仍需的数量，或从停放的货运车辆推入。多余物资留在来源处，活动 Order 的货物会保持保留。调用 `clear_order()` 或订单完成后，使用 `eject(destination, item_id, count)` 将剩余物资回收 Nocturna Base 的 Inventory，或转移到本地其他货运设施。`flush()` 会销毁已装载货物。需要 Auto Feeders 研究。参见 `InputSlot`。

返回值：面向 Order 的 `InputSlot`，只接受活动 Order 的剩余需求。Order 清除或完成后，`eject(...)` 会将剩余货物回收明确指定的本地货运设施；`flush()` 会销毁它们。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` 仅限 SELF

读取脚本运行期间从编辑器 Commands 选项卡发送的命令。所有可编程机器上的行为都相同，参见指南：Script Commands。

组件 / 基础设施与流体

Construction Blueprint

管理计划中的建造和拆除工作。Plan Mode 与脚本共享同一队列。脚本可以放置结构、管道、电力线和桥梁，也可以将现有结构标记为拆除。规划会立即创建地图标记，无需车辆在现场。不过，装有 Constructor Module 的 Pioneer 仍必须前往每个工作点，并调用 `self.constructor.execute(construction.id)` 执行工作。

由以下对象返回：`get_component("construction_blueprint")`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，找到精确 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

该组件的稳定程序化标识符。将它与 `get_component(id)` 以及要求组件、行星、车辆、空间站或订单 ID 的 API 一起使用。

返回值：String

`.name`

面向用户的显示名称。需要经受重命名的脚本应优先使用 `.id`。

返回值：String

方法

```
.plan_structure(kind, x, y, rotation=0)
```

根据脚本坐标创建一个点结构的建造幽灵。支持的 kind 包括 "outpost"、"thermal_cap"、"water_pump"、"oil_pump"、"exotic_gas_cap"、"exotic_spring_tap"、"mining_drill"、"mining_drill_industrial" 和 "mining_drill_heavy"。钻机类型需要匹配的 Earth Order kit 配方。坐标会吸附到地图网格。提取结构会精确吸附到匹配的已勘测特征；Outpost 使用吸附后的占地锚点。可选的顺时针旋转角度必须为 0、90、180 或 270。幽灵会立即进入共享队列；之后仍由 Pioneer 建造。

参数

名称	类型	描述
kind	string	点蓝图类型。
x	number	世界 x 坐标。
y	number	世界 y 坐标。
rotation	number	可选的顺时针旋转角度：0、90、180 或 270 度。

返回值：BlueprintPlanResult，结果字段为 .status、.message，成功负载为 .blueprint_ids。

状态	类型	含义
"ok"	成功	已创建建造蓝图：count。
"locked"	拒绝	尚未解锁所需的建造技术或 kit 配方。
"invalid_kind"	拒绝	请求的地图结构类型不受支持。
"invalid_rotation"	拒绝	结构旋转必须为 0、90、180 或 270 度。
"out_of_bounds"	拒绝	至少一个请求的建造地块位于行星边界之外。
"wrong_target"	拒绝	请求位置不包含该结构所需的地图特征。
"unsurveyed_target"	拒绝	所需地图特征尚未勘测。
"too_hard"	拒绝	选定的 Mining Drill 无法切割该矿床的矿物硬度。
"target_claimed"	拒绝	所需地图特征已经有已建造或计划中的提取器。
"occupied"	拒绝	请求的结构占地与现有或计划中的建造，或实体异常重叠。

状态	类型	含义
"clearance"	拒绝	请求的结构占地穿过 Outpost 或实体异常的净空边界。
"blocked"	拒绝	请求的建造或拆除被现有或计划中的基础设施阻挡。

`.plan_pipe(medium, x1, y1, x2, y2)`

根据脚本坐标创建管道建造工作。需要 Constructor Module 研究。`medium` 可以是 "gas"、"liquid"，或已注册的流体 ID，例如 "steam"、"water" 或 "oil"。坐标会吸附到地块中心通道。现场结构使用其站点坐标。内部 outpost 机器使用所属 outpost 作为公用设施锚点，因此应将路径连接到 `building.outpost` 的占地内一点，例如 [`building.outpost.x`, `building.outpost.y`]，而不是 `building.position` 中的车辆停靠点。已有的匹配部件会自动复用。

参数

名称	类型	描述
<code>medium</code>	string	"gas" / "liquid"，或任何已注册的流体 ID。
<code>x1</code>	number	路径起点的世界 x 坐标。对于内部 outpost 机器，应使用所属 outpost 占地内的一点，例如 <code>building.outpost.x</code> 。
<code>y1</code>	number	路径起点的世界 y 坐标。对于内部 outpost 机器，应使用所属 outpost 占地内的一点，例如 <code>building.outpost.y</code> 。
<code>x2</code>	number	路径终点的世界 x 坐标。对于内部 outpost 机器，应使用所属 outpost 占地内的一点，例如 <code>building.outpost.x</code> 。
<code>y2</code>	number	路径终点的世界 y 坐标。对于内部 outpost 机器，应使用所属 outpost 占地内的一点，例如 <code>building.outpost.y</code> 。

返回值：`BlueprintPlanResult`，结果字段为 `.status`、`.message`，成功负载为 `.blueprint_ids`。

状态	类型	含义
"ok"	成功	已创建建造蓝图：count。
"locked"	拒绝	尚未解锁所需的建造技术或 kit 配方。
"invalid_medium"	拒绝	请求的基础设施介质不受支持。
"out_of_bounds"	拒绝	至少一个请求的建造地块位于行星边界之外。
"invalid_route"	拒绝	请求的端点未定义可用的建造路径。
"blocked"	拒绝	请求的建造或拆除被现有或计划中的基础设施阻挡。
"already_exists"	成功	匹配的基础设施路径已经存在，因此未创建蓝图。

`.plan_power_line(x1, y1, x2, y2)`

根据脚本坐标创建电力线建造工作。需要 Constructor Module 研究。坐标会吸附到地块中心通道；偏轴路径会选择新增建造最少的有效 L 形转角。现场结构使用其站点坐标。内部 outpost 机器使用所属 outpost 作为公用设施锚点，因此应将路径连接到 `building.outpost` 的占地内一点，例如 `[building.outpost.x, building.outpost.y]`，而不是 `building.position` 中的车辆停靠点。已有的匹配部件会自动复用。

参数

名称	类型	描述
x1	number	线路起点的世界 x 坐标。对于内部 outpost 机器，应使用所属 outpost 占地内的一点，例如 <code>building.outpost.x</code> 。
y1	number	线路起点的世界 y 坐标。对于内部 outpost 机器，应使用所属 outpost 占地内的一点，例如 <code>building.outpost.y</code> 。
x2	number	线路终点的世界 x 坐标。对于内部 outpost 机器，应使用所属 outpost 占地内的一点，例如 <code>building.outpost.x</code> 。
y2	number	线路终点的世界 y 坐标。对于内部 outpost 机器，应使用所属 outpost 占地内的一点，例如 <code>building.outpost.y</code> 。

返回值：`BlueprintPlanResult`，结果字段为 `.status`、`.message`，成功负载为 `.blueprint_ids`。

状态	类型	含义
"ok"	成功	已创建建造蓝图：count。
"locked"	拒绝	尚未解锁所需的建造技术或 kit 配方。
"out_of_bounds"	拒绝	至少一个请求的建造地块位于行星边界之外。
"invalid_route"	拒绝	请求的端点未定义可用的建造路径。
"blocked"	拒绝	请求的建造或拆除被现有或计划中的基础设施阻挡。
"already_exists"	成功	匹配的基础设施路径已经存在，因此未创建蓝图。

`.plan_bridge(medium, x, y, axis)`

根据脚本坐标创建一个公用设施桥梁工作。需要 Constructor Module 研究。`medium` 可以是 "gas"、"liquid"、"power" 或已注册的流体 ID。`x / y` 是桥梁中心地块，`axis` 为 "horizontal" 或 "vertical"。

参数

名称	类型	描述
medium	string	"gas"、"liquid"、"power" 或任何已注册的流体 ID。
x	number	桥梁地块的世界 x 坐标。
y	number	桥梁地块的世界 y 坐标。
axis	string	"horizontal" 或 "vertical"。

返回值：`BlueprintPlanResult`，结果字段为 `.status`、`.message`，成功负载为 `.blueprint_ids`。

状态	类型	含义
"ok"	成功	已创建建造蓝图：count。
"locked"	拒绝	尚未解锁所需的建造技术或 kit 配方。
"invalid_medium"	拒绝	请求的基础设施介质不受支持。
"invalid_axis"	拒绝	桥梁轴向必须为 "horizontal" 或 "vertical"。
"out_of_bounds"	拒绝	至少一个请求的建造地块位于行星边界之外。
"blocked"	拒绝	请求的建造或拆除被现有或计划中的基础设施阻挡。
"already_exists"	成功	匹配的基础设施路径已经存在，因此未创建蓝图。

```
.mark_deconstruct(x, y, layer="auto", target_id="")
```

将坐标处已建造的基础设施或普通地图建筑标记为拆除。Base 和 Outpost 受到保护。当多个独立地图图层重叠时，选择 "building"、"gas"、"liquid" 或 "power" 图层；默认的 "auto" 要求目标明确无歧义。在同一图层的连接点处，提供可选的精确目标 ID。Pioneer 仍会在工作报告的位置使用 `self.constructor.execute(id)` 执行工作。

参数

名称	类型	描述
x	number	目标地块的世界 x 坐标。
y	number	目标地块的世界 y 坐标。
layer	string	可选目标图层。当多个图层重叠时使用 "building"、"gas"、"liquid" 或 "power"；"auto" 会在无歧义地块上选择第一个目标。
target_id	string	可选的精确机器或基础设施 ID，用于解析同一图层上的连接点。

返回值：BlueprintPlanResult，结果字段为 .status、.message，成功负载为 .blueprint_ids。

状态	类型	含义
"ok"	成功	已创建建造蓝图：count。
"locked"	拒绝	尚未解锁所需的建造技术或 kit 配方。
"nothing_here"	拒绝	该位置没有已完成且可拆除的目标。
"already_queued"	成功	匹配的拆除蓝图已经排队。
"ambiguous_target"	拒绝	该位置和图层匹配多个可拆除目标。请选择更具体的图层或其他地块。
"out_of_bounds"	拒绝	至少一个请求的建造地块位于行星边界之外。
"invalid_layer"	拒绝	拆除图层必须是 "auto"、"building"、"gas"、"liquid" 或 "power"。
"blocked"	拒绝	请求的建造或拆除被现有或计划中的基础设施阻挡。

`.cancel(blueprint_id)`

按 ID 取消排队中、活动中或已暂停的建造蓝图。未支付的工作会立即取消。已支付的工作会将材料留在建造地点：派一台 Pioneer 到那里即可将材料回收到货舱。回收失败时，工作和材料都会保持不变。

参数

名称	类型	描述
blueprint_id	string	来自 <code>.pending_constructions()</code> 、 <code>.active_constructions()</code> 或 <code>.paused_constructions()</code> 的蓝图 ID。

返回值

返回类型	结果字段	成功载荷
ActionResult	.status、.message	无

结果

状态	类型	含义
"ok"	成功	操作已成功完成。
"not_found"	拒绝	请求的对象、目标或记录不存在。
"worker_not_present"	拒绝	建造地点没有 Pioneer 来接收退款。
"no_cargo_space"	拒绝	货舱没有可容纳结果的兼容空间。
"construction_dependency"	拒绝	另一个建造工作依赖于此工作。

`.pending_constructions()`

返回等待工人的蓝图列表

`list<Construction>`。绘制出的管道/电力路径以及标记的拆除目标会拆分为独立工作，通常按放置顺序排列。将每个 `c.id` 传给 `self.constructor.execute(c.id)`，即可建造或拆除它。执行前读取 `c.required_item` 和 `c.required_count`，装载确切的货物；不要从 `c.kind` 推断材料。按 `c.kind` 过滤可专门化 Pioneer 的角色，再使用 `c.medium` 区分 "gas"、"liquid" 和 "power" 公用设施工作。点结构的 `c.medium == None`；完整的种类列表请参阅 `Construction.kind`。

返回值

等待工人的 `Construction` 快照列表。每项包含 `.id`、`.kind`、`.medium`、`.position`、`.progress`、`.required_item` 和 `.required_count`。遍历列表并将 `c.id` 传给 `self.constructor.execute(c.id)`，以建造或拆除结构。

`.active_constructions()`

返回当前正在建造的蓝图（Pioneer 正在工作）。适合用于监控脚本；读取 `c.progress` 可了解完成进度。

返回值

当前正在建造或移除的 `Construction` 快照列表（Pioneer 正在工作）。重新查询以获取最新的 `.progress`。

`.paused_constructions()`

返回已开始后又被放弃的蓝图（工人死亡、燃料耗尽或脚本停止）。任何 Pioneer 都可以前往 `c.position`，然后调用 `self.constructor.execute(c.id)` 继续工作。

返回值

已开始后被放弃的 `Construction` 快照列表（工人死亡或离开）。任何 Pioneer 都可以通过 `self.constructor.execute(c.id)` 恢复工作。重新查询以获取最新的 `.progress`。

组件 / 基础设施与流体

Exotic Gas Cap

在活跃阶段从周期性奇异矿藏捕获气体。将 `self.gas_out` 连接到消费者，构建一条从矿田 Cap 通往该目的地的已完成 Gas Pipe 路线，然后使用 `self.set_throttle(value)` 设置 0 到 1 的释放速率。缓冲区已满时会暂停收集，但不会损失气体。

项目	内容
类型	流体
建造于	奇异矿藏
输入功率	-30 W（从电网取电）

获取方式

- 通过 Exotic Husbandry 研究（Wildlife 1,000）解锁配方。
- 在 Fabricator 上制造 Exotic Gas Cap Kit：2× Titanium Ingot 和 2× Gas Pipe Segment。
- 使用 Pioneer 的 Constructor 将其建造在已勘测的奇异矿藏上。

返回者：`self` / `get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ⓘ，找到确切 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件的稳定程序化标识符。将它与 `get_component(id)` 以及要求组件、行星、载具、空间站或订单 ID 的 API 一起使用。

返回值：`String`

`.name`

面向用户显示的可读名称。需要经受重命名的脚本应优先使用 `.id`。

返回值：`String`

方法

`.deposit()`

此 Cap 所固定的 `ExoticDeposit`，包括 `.id`、`position()`、`fluid()`、`current_phase()` 和周期计时信息。字段可用性由上次勘测该矿藏的声纳等级决定：基础等级只显示阶段，广域等级增加速率，深度等级增加周期计时。如果 Cap 不在矿藏上，则返回 `None`。使用 `deposit.current_phase()` 检查来源是否活跃。参阅 `ExoticDeposit`。

返回值：连接到此 Cap 的 `ExoticDeposit`，或 `None`。其周期、勘测和采集器方法每次调用都会读取实时状态；应保留该对象并调用其方法以获取最新读数。`.surveyed` 属性仍是创建时的快照。

`.capture_rate()`

上一次流体 tick 从矿藏捕获的奇异气体，单位为 t/h。在矿藏休眠阶段，或缓冲区已满并保持不动时（参见 `is_venting()`），返回 0。该值已经考虑当前阶段和缓冲区可用空间；应直接读取它，不要根据矿藏速率自行计算。每个流体 tick 更新一次。

返回值：数值：本 tick 从矿藏捕获的奇异气体，单位为 t/h。在矿藏休眠阶段，或缓冲区已满并保持不动时（参见 `is_venting()`），为 0。

`.is_venting()`

如果上一次流体 tick 中，活跃气体产量超过捕获缓冲区的可用空间，则为 **True**。多出的气体会在上游保留，不会损失。即使缓冲区已满，休眠时也返回 **False**；断电时同样返回 **False**。这是捕获空间限制，不同于报告释放受阻的 `is_stalled()`。将 `self.set_throttle(...)` 调向有空间的罐体。

返回值：布尔值：如果活跃气体产量超过上一次流体 tick 中捕获缓冲区的可用空间，则为 **True**。多出的气体保留在上游而不会损失。休眠期间（即使缓冲区已满）或断电时为 **False**。将 `set_throttle(...)` 调向有空间的罐体。

`.is_stalled()`

如果在上一次流体 tick 中，有电力的 Cap 打开了节流阀，且缓冲区有可释放的气体，但无法通过其连接的路线传输任何气体，则为 **True**。缓冲区为空、节流阀关闭或断电时不会报告停滞。

返回值：布尔值：如果在上一次流体 tick 中，有电力的 Cap 满足 `throttle > 0` 且缓冲区有可释放气体，但无法通过任何连接路线传输，则为 **True**。断电、节流阀关闭或缓冲区没有可释放气体时为 **False**。

`.throttle()`

当前释放阀设置，范围为 **0.0**（保持）到 **1.0**（完全打开）。调用 `set_throttle(...)` 后读取此值以确认设置。

返回值：数值（0-1）：当前释放阀设置。

`.set_throttle(t)` 仅限自身

将 Cap 的释放阀打开到 0.0 至 1.0 范围内的值（超出范围会被钳制）。**0** 保持缓冲区；**1.0** 会在缓冲区供给、可用空间和吞吐量允许的情况下，尽快将气体释放到所有可到达的连接目的地。此由脚本拥有的设定点会在脚本停止、结束或出错时重置为 0。[仅限自身]

参数

名称	类型	描述
t	number	0 到 1 范围内的释放阀比例

返回值

返回类型	结果字段	成功载荷
ActionResult	.status、.message	无

结果

状态	类型	含义
"ok"	成功	操作已成功完成。

`.gas_out`

缓冲气体的流体输出端口。此端口可以通过 `self.gas_out.connect("Chlorine Tank")` 声明一个目的地；其他消费者也可以将其兼容的输入端口连接到此 Cap。由于 Cap 是矿田结构，每个目的地都需要一条兼容的、已完成的 Gas Pipe 组件路线连接 Cap 与目的地。`connected_to()` 只报告此端口自身的声明；`flow_rate()` 报告实时释放总量。参阅 `FluidPort`。

返回值：矿藏气体的 `FluidPort`。此端口可以通过 `self.gas_out.connect("Chlorine Tank")` 声明一个目的地；其他消费者也可以将兼容输入连接到此 Cap。已完成的 gas-pipe 网络会在前哨之间输送气体。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` 仅限自身

读取脚本运行期间从编辑器 Commands 选项卡发送的命令。所有可编程机器上的行为都相同，参阅指南：Script Commands。

组件 / 基础设施与流体

Exotic Spring Tap

在活跃阶段从周期性奇异泉捕获液体。将 `self.liquid_out` 连接到消费者，构建一条从矿田 Tap 通往该目的地的已完成 Liquid Pipe 路线，然后使用 `self.set_throttle(value)` 设置 0 到 1 的释放速率。缓冲区已满时会暂停收集，但不会损失液体。

项目	内容
类型	流体
建造于	奇异矿藏
输入功率	-35 W（从电网取电）

获取方式

- 通过 Exotic Husbandry 研究（Wildlife 1,000）解锁配方。
- 在 Fabricator 上制造 Exotic Spring Tap Kit：2× Titanium Ingot、2× Liquid Pipe Segment 和 1× Pressure Valve。
- 使用 Pioneer 的 Constructor 将其建造在已勘测的奇异矿藏上。

返回者：`self / get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，找到确切 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件的稳定程序化标识符。将它与 `get_component(id)` 以及要求组件、行星、载具、空间站或订单 ID 的 API 一起使用。

返回值：`String`

`.name`

面向用户显示的可读名称。需要经受重命名的脚本应优先使用 `.id`。

返回值：`String`

方法

`.deposit()`

此 Tap 所固定的 `ExoticDeposit`，包括 `.id`、`position()`、`fluid()`、`current_phase()` 和周期计时信息。字段可用性由上次勘测该矿藏的声纳等级决定（基础/广域/深度）。如果 Tap 不在矿藏上，则返回 `None`。使用 `deposit.current_phase()` 检查来源是否活跃。参阅 `ExoticDeposit`。

返回值：连接到此 Tap 的 `ExoticDeposit`，或 `None`。其周期、勘测和采集器方法每次调用都会读取实时状态；应保留该对象并调用其方法以获取最新读数。`.surveyed` 属性仍是创建时的快照。

`.capture_rate()`

上一次流体 tick 从泉中捕获的奇异液体，单位为 t/h。在矿藏休眠阶段，或缓冲区已满并保持不动时（参见 `is_venting()`），返回 0。该值已经考虑当前阶段和缓冲区可用空间。每个流体 tick 更新一次。

返回值：数值：本 tick 从泉中捕获的奇异液体，单位为 t/h。在矿藏休眠阶段，或缓冲区已满并保持不动时（参见 `is_venting()`），为 0。

`.is_venting()`

如果上一次流体 tick 中，活跃液体产量超过捕获缓冲区的可用空间，则为 **True**。多出的液体会在上游保留，不会损失。即使缓冲区已满，休眠时也返回 **False**；断电时同样返回 **False**。这是捕获空间限制，不同于报告释放受阻的 `is_stalled()`。将 `self.set_throttle(...)` 调向有空间的罐体。

返回值：布尔值：如果活跃液体产量超过上一次流体 tick 中捕获缓冲区的可用空间，则为 **True**。多出的液体保留在上游而不会损失。休眠期间（即使缓冲区已满）或断电时为 **False**。将 `set_throttle(...)` 调向有空间的罐体。

`.is_stalled()`

如果在上一次流体 tick 中，有电力的 Tap 打开了节流阀，且缓冲区有可释放的液体，但无法通过其连接的路线传输任何液体，则为 **True**。缓冲区为空、节流阀关闭或断电时不会报告停滞。

返回值：布尔值：如果在上一次流体 tick 中，有电力的 Tap 满足 `throttle > 0` 且缓冲区有可释放液体，但无法通过任何连接路线传输，则为 **True**。断电、节流阀关闭或缓冲区没有可释放液体时为 **False**。

`.throttle()`

当前释放阀设置，范围为 **0.0**（保持）到 **1.0**（完全打开）。调用 `set_throttle(...)` 后读取此值以确认设置。

返回值：数值（0-1）：当前释放阀设置。

`.set_throttle(t)` 仅限自身

将 Tap 的释放阀打开到 0.0 至 1.0 范围内的值（超出范围会被钳制）。**0** 保持缓冲区；**1.0** 会在缓冲区供给、可用空间和吞吐量允许的情况下，尽快将液体释放到所有可到达的连接目的地。此由脚本拥有的设定点会在脚本停止、结束或出错时重置为 0。[仅限自身]

参数

名称	类型	描述
t	number	0 到 1 范围内的释放阀比例

返回值

返回类型	结果字段	成功载荷
ActionResult	.status、.message	无

结果

状态	类型	含义
"ok"	成功	操作已成功完成。

`.liquid_out`

缓冲液体的流体输出端口。此端口可以通过 `self.liquid_out.connect("Cryofluid Tank")` 声明一个目的地；其他消费者也可以将其兼容的输入端口连接到此 Tap。由于 Tap 是矿田结构，每个目的地都需要一条兼容的、已完成的 Liquid Pipe 组件路线连接 Tap 与目的地。`connected_to()` 只报告此端口自身的声明；`flow_rate()` 报告实时释放总量。参阅 `FluidPort`。

返回值：矿藏液体的 `FluidPort`。此端口可以通过 `self.liquid_out.connect("Cryofluid Tank")` 声明一个目的地；其他消费者也可以将兼容输入连接到此 Tap。已完成的 liquid-pipe 网络会在前哨之间输送液体。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` 仅限自身

读取脚本运行期间从编辑器 Commands 选项卡发送的命令。所有可编程机器上的行为都相同，参阅指南：Script Commands。

组件 / 基础设施与流体

Gas Tank

一种无源缓冲器，会锁定首次通过管道输入的气体（蒸汽、氨气、沼气），并且在排空前只容纳该气体。它位于来源与消费者之间，可以平滑供应中的间隙。

项目	内容
类型	流体
存储	任意气体, 5,000 t

获取方式

- 需要 Gas Tank 研究（Pressure 2.75）。
- 在 Shop 购买，价格为 1,200 cr。

返回者：`get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，找到确切 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件的稳定程序化标识符。将它与 `get_component(id)` 以及要求组件、行星、载具、空间站或订单 ID 的 API 一起使用。

返回值：`String`

`.name`

面向用户显示的可读名称。需要经受重命名的脚本应优先使用 `.id`。

返回值：`String`

方法

`.outpost`

部署此建筑的前哨。返回的 `OutpostRef` 包含其稳定 ID、显示名称、生物群系、位置、容量和 `buildings()` 查询。需要当前值时重新读取此属性。

返回值：部署此建筑的前哨的 `OutpostRef`。

`.fluid()`

已锁定的气体 ID（例如 `"steam"`、`"ammonia"`），为空时为 `""`。罐体会锁定接收的第一种气体，并在完全排空前只容纳该气体；排空至 0 后重新锁定。

返回值：字符串：已锁定的气体 ID（例如 `"steam"`、`"ammonia"`），为空/未锁定时为 `""`。

可能值：

`""`、`"steam"`、`"ammonia"`、`"swamp_gas"`、`"raw_sulfur_gas"`、`"sulfur_gas"`、`"raw_chlorine"`、`"chlorine"`

`.level()`

当前存储的气体量，单位为吨，范围从 0 到 `capacity()`。每次迭代读取它以评估缓冲区：接近 0 表示下游即将断供；接近容量表示上游受到反压。在 0 时罐体解除锁定，下一次可以接收不同气体。

返回值：数值：当前存储的气体量，单位为吨。

`.capacity()`

此罐体可容纳的最大气体吨数。该值可查询而非硬编码，因此罐体调校不会破坏脚本。与 `level()` 一起用于填充百分比指示器，或直接使用 `fill_pct()`。

返回值：数值：最大气体容量，单位为吨。

`.fill_pct()`

填充比例（0.0-1.0），是 `level() / capacity()` 的快捷方式。用于阈值检查：`if self.fill_pct() < 0.2: # boost throttle upstream`。

返回值：数值（0-1）。

`.inflow_rate()`

流入气体的速率，单位为 t/h。0 表示没有上游流量，例如来源处于休眠、关系不可用、远程路线未完成或罐体已满。与 `outflow_rate()` 比较可判断罐体是在填充还是排空。

返回值：数值：流入气体速率，单位为 t/h。

`.outflow_rate()`

流出气体的速率，单位为 t/h。0 表示下游消费者已饱和或管道已断开。

返回值：数值：流出气体速率，单位为 t/h。

`.is_full()`

当 `level() == capacity()` 时为 `True`；此时上游反压开始生效，Cap 可能开始向大气排放。通过检查它可以检测连接的目的地无法吸收当前产量的情况。

返回值：布尔值。

`.is_empty()`

当 `level() == 0` 时为 `True`；罐体已解除锁定，无法向下游发送任何内容。如果来源仍然活跃而罐体保持为空，应检查来源及其管道连接。

返回值：布尔值。

`.gas_in`

已锁定的气体 ID（例如 `"steam"`、`"ammonia"`），为空时为 `""`。罐体会锁定接收的第一种气体，并在完全排空前只容纳该气体；排空至 0 后重新锁定。

返回值：空罐时为中性的通用 `FluidPort` 输入。使用提供方的稳定机器 ID 或显示名称调用 `connect(...)`；首次准确输送的气体会锁定罐体。

`.gas_out`

已锁定的气体 ID（例如 "steam"、"ammonia"），为空时为 ""。罐体会锁定接收的第一种气体，并在完全排空前只容纳该气体；排空至 0 后重新锁定。

返回值：通用 `FluidPort` 输出。为空时保持中性，并向连接的消费者提供罐体已锁定的确切气体。

`.steam_in`

已锁定的气体 ID（例如 "steam"、"ammonia"），为空时为 ""。罐体会锁定接收的第一种气体，并在完全排空前只容纳该气体；排空至 0 后重新锁定。

返回值：罐体已锁定气体的 `FluidPort` 输入。此属性仅在 `fluid()` 为 "steam" 时存在。

`.steam_out`

已锁定的气体 ID（例如 "steam"、"ammonia"），为空时为 ""。罐体会锁定接收的第一种气体，并在完全排空前只容纳该气体；排空至 0 后重新锁定。

返回值：罐体已锁定气体的 `FluidPort` 输出。此属性仅在 `fluid()` 为 "steam" 时存在。

组件 / 基础设施与流体

Large Liquid Tank

一种大型无源罐体，可容纳 1,000 t 的一种液体。与 Liquid Tank 类似，它会锁定第一种通过管道输入的流体，并且只有完全排空后才会解除锁定。

项目	内容
类型	流体
存储	任意液体，1,000 t

获取方式

- 需要 Large Liquid Tank 研究（Plants 900,000）。
- 在 Shop 购买，价格为 15,000 cr。

返回者：`get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，找到确切 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件的稳定程序化标识符。将它与 `get_component(id)` 以及要求组件、行星、载具、空间站或订单 ID 的 API 一起使用。

返回值：`String`

`.name`

面向用户显示的可读名称。需要经受重命名的脚本应优先使用 `.id`。

返回值：`String`

方法

`.outpost`

部署此建筑的前哨。返回的 `OutpostRef` 包含其稳定 ID、显示名称、生物群系、位置、容量和 `buildings()` 查询。需要当前值时重新读取此属性。

返回值：部署此建筑的前哨的 `OutpostRef`。

`.fluid()`

已锁定的液体 ID（例如 `"water"`、`"oil"`、`"frozen_essence"`），为空时为 `""`。罐体会锁定接收的第一种液体，并在完全排空前只容纳该液体；排空至 0 后重新锁定。

返回值：字符串：已锁定的液体 ID；为空/未锁定时为 `""`。

可能值：`""`、`"water"`、`"oil"`、`"frozen_essence"`、`"coastal_essence"`、`"geothermal_essence"`、`"volcanic_essence"`、`"deep_essence"`、`"brine"`、`"raw_cryofluid"`、`"cryofluid"`、`"raw_quicksilver"`、`"quicksilver"`

`.level()`

当前存储的液体量，单位为吨，范围从 0 到 `capacity()`。在 0 时罐体解除锁定，下一次可以接收不同液体。

返回值：数值：当前存储的液体量，单位为吨。

`.capacity()`

该储罐可容纳的最大吨数。此值可查询，不应硬编码。可与 `level()` 或 `fill_pct()` 一起用于阈值检查。

返回值

Number：最大容量，单位为吨。

`.fill_pct()`

填充比例（0.0-1.0），即 `level()` / `capacity()` 的快捷方式。常用于供应控制脚本中的阈值判断。

返回值

Number（0-1）。

`.inflow_rate()`

流入液体的速率，单位为 t/h。上游没有流量时为 0。

返回值

Number：流入液体速率，单位为 t/h。

`.outflow_rate()`

流出液体的速率，单位为 t/h。下游没有抽取时为 0。

返回值

Number：流出液体速率，单位为 t/h。

`.is_full()`

当 `level() == capacity()` 时为 True；此时上游源受到背压。

返回值

Boolean。

`.is_empty()`

当 `level() == 0` 时为 `True`；储罐已解除液体锁定，下游消费者会缺料。

返回值

Boolean。

`.liquid_in`

已锁定的液体 id（例如 `"water"`、`"oil"`、`"frozen_essence"`）；为空时为 `""`。储罐会锁定收到的第一种液体，在液位降至 0 前只容纳该液体，随后重新锁定。

返回值

储罐为空时为中性的通用 `FluidPort` 输入。使用供应方的稳定机器 id 或显示名称调用 `connect(...)`；首次送达的精确液体会锁定储罐。

`.liquid_out`

已锁定的液体 id（例如 `"water"`、`"oil"`、`"frozen_essence"`）；为空时为 `""`。储罐会锁定收到的第一种液体，在液位降至 0 前只容纳该液体，随后重新锁定。

返回值

为储罐锁定液体提供输出的通用 `FluidPort`。

`.water_in`

已锁定的液体 id（例如 "water"、"oil"、"frozen_essence"）；为空时为 ""。储罐会锁定收到的第一种液体，在液位降至 0 前只容纳该液体，随后重新锁定。

返回值

储罐精确锁定液体的 FluidPort 输入。仅当 `fluid()` 为 "water" 时，此属性才存在。

`.water_out`

已锁定的液体 id（例如 "water"、"oil"、"frozen_essence"）；为空时为 ""。储罐会锁定收到的第一种液体，在液位降至 0 前只容纳该液体，随后重新锁定。

返回值

储罐精确锁定液体的 FluidPort 输出。仅当 `fluid()` 为 "water" 时，此属性才存在。

组件 / 基础设施与流体（COMPONENTS / INFRASTRUCTURE & FLUIDS）

液体储罐（Liquid Tank）

可容纳任何一种液体（水、油或生物群系精华）的被动缓冲装置。它会锁定通过管道输入的第一种液体。

类型 Fluids

储存 任意液体，100 t

获取方式（HOW TO OBTAIN）

- 需要完成 Liquid Tank 研究（Oxygen 400）。
- 从 Shop 购买，价格为 900 cr。

返回方式：`get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，查找准确 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性（PROPERTIES）

`.id`

此组件的稳定程序化标识符。将其用于 `get_component(id)`，以及要求提供组件、行星、载具、空间站或订单 ID 的 API。

返回值

String

`.name`

面向用户的可读显示名称。需要经得起重命名的脚本应优先使用 `.id`。

返回值

String

方法 (METHODS)

`.outpost`

部署此建筑的前哨站。返回的 `OutpostRef` 包含其稳定 id、显示名称、生物群系、位置、容量和 `buildings()` 查询。需要当前值时请重新读取该属性。

返回值

部署此建筑的前哨站的 `OutpostRef`。

`.fluid()`

已锁定的液体 id（例如 `"water"`、`"oil"`、`"frozen_essence"`）；为空时为 `""`。储罐会锁定收到的第一种液体，在液位降至 0 前只容纳该液体，随后重新锁定。

返回值 可能的值 (POSSIBLE VALUES)

String：已锁定的液体 id（例如 `"water"`、`"oil"`、`"frozen_essence"`），未锁定或为空时为 `""`。`""`、`"water"`、`"oil"`、`"frozen_essence"`、`"coastal_essence"`、`"geothermal_essence"`、`"volcanic_essence"`、`"deep_essence"`、`"brine"`、`"raw_cryofluid"`、`"cryofluid"`、`"raw_quicksilver"`、`"quicksilver"`

`.level()`

当前储存的液体量，单位为吨，范围为 0 至 `capacity()`。液位为 0 时储罐解除锁定，可以接受另一种液体。

返回值

Number：当前储存的液体量，单位为吨。

`.capacity()`

该储罐可容纳的最大吨数。此值可查询，不应硬编码。可与 `level()` 或 `fill_pct()` 一起用于阈值检查。

返回值

Number：最大容量，单位为吨。

`.fill_pct()`

填充比例（0.0-1.0），即 `level()` / `capacity()` 的快捷方式。常用于供应控制脚本中的阈值判断。

返回值

Number（0-1）。

`.inflow_rate()`

流入液体的速率，单位为 t/h。上游没有流量时为 0。

返回值

Number：流入液体速率，单位为 t/h。

`.outflow_rate()`

流出液体的速率，单位为 t/h。下游没有抽取时为 0。

返回值

Number：流出液体速率，单位为 t/h。

`.is_full()`

当 `level() == capacity()` 时为 True；此时上游源受到背压。

返回值

Boolean。

`.is_empty()`

当 `level() == 0` 时为 `True`；储罐已解除液体锁定，下游消费者会缺料。

返回值

Boolean。

`.liquid_in`

已锁定的液体 id（例如 `"water"`、`"oil"`、`"frozen_essence"`）；为空时为 `""`。储罐会锁定收到的第一种液体，在液位降至 0 前只容纳该液体，随后重新锁定。

返回值

储罐为空时为中性的通用 `FluidPort` 输入。使用供应方的稳定机器 id 或显示名称调用 `connect(...)`；首次送达的精确液体会锁定储罐。

`.liquid_out`

已锁定的液体 id（例如 `"water"`、`"oil"`、`"frozen_essence"`）；为空时为 `""`。储罐会锁定收到的第一种液体，在液位降至 0 前只容纳该液体，随后重新锁定。

返回值

通用 `FluidPort` 输出。储罐为空时仍保持中性，并向已连接的消费者提供储罐锁定的精确液体。

`.water_in`

已锁定的液体 id（例如 "water"、"oil"、"frozen_essence"）；为空时为 ""。储罐会锁定收到的第一种液体，在液位降至 0 前只容纳该液体，随后重新锁定。

返回值

储罐精确锁定液体的 FluidPort 输入。仅当 `fluid()` 为 "water" 时，此属性才存在。

`.water_out`

已锁定的液体 id（例如 "water"、"oil"、"frozen_essence"）；为空时为 ""。储罐会锁定收到的第一种液体，在液位降至 0 前只容纳该液体，随后重新锁定。

返回值

储罐精确锁定液体的 FluidPort 输出。仅当 `fluid()` 为 "water" 时，此属性才存在。

组件 / 基础设施与流体（COMPONENTS / INFRASTRUCTURE & FLUIDS）

石油泵（Oil Pump）

以脚本设定的节流率从已勘测的油井中抽取石油。油井会在活跃期和休眠期之间切换，因此应将输出缓冲到 Liquid Tank 中，以度过无产出的时段。

类型 Fluids

建造于 Oil wells

输入功率 -5 W（从电网取电）

产出 Oil, 缓冲 10 t

获取方式（HOW TO OBTAIN）

- Petroleum Survey 研究解锁配方（Oxygen 1,500）。
- 在 Fabricator 上制造 Oil Pump：2× Iron Ingot、1× Titanium Ingot、1× Pressure Valve 和 1× Circuit Panel。
- 使用 Pioneer's Constructor 将其建造在已勘测的油井上。

返回方式：`self` / `get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，查找准确 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性（PROPERTIES）

`.id`

此组件的稳定程序化标识符。将其用于 `get_component(id)`，以及要求提供组件、行星、载具、空间站或订单 ID 的 API。

返回值

String

`.name`

面向用户的可读显示名称。需要经得起重命名的脚本应优先使用 `.id`。

返回值

String

方法 (METHODS)

`.well()`

此泵所固定连接的 `OilWell`。其结构与 `WaterWell` 相同，并包含产量等级 ($1\times/2\times/3\times$) 和基础流速。

返回值

此泵所在油井的 `OilWell` 快照：`.id`、`position()`、`yield_tier()`、`flow_rate()`。

`.pump_rate()`

本 tick 通过所有可达的已连接目的地输送的石油总量，单位为 t/h。当节流率为 0、油井处于休眠期，或没有目的地能够接收流量时为 0。使用 `is_stalled()` 区分路由阻塞与油井休眠。

返回值

Number：本 tick 通过所有可达目的地输送的石油总量，单位为 t/h。当节流率为 0、油井休眠或没有目的地能接收流量时为 0。

`.well_active()`

读取油井的脉冲。`True` 表示下方油井处于活跃期并以全速输送；`False` 表示休眠期：无论节流率如何都没有石油，通常会持续数小时。在下游 Liquid Tank 中储存石油，并在间歇期间降低节流率以节省瓦数。

返回值

Boolean：油井处于活跃期时为 `True`。休眠油井在任何节流率下都不会输送；应在下游 Liquid Tank 中储油以度过间歇期。

`.is_stalled()`

如果在上一次流量 tick 中，通电的泵已打开节流、活跃油井有石油可供输送，但在已连接的路由上无法传输任何石油，则为 **True**。休眠、关闭节流或缺电都不会报告停滞。使用 `self.oil_out.connect(...)` 声明目的地，或让消费者将自己的 `oil_in` 端口连接到此 Pump。

返回值

若上一次流量 tick 中泵已通电、节流率 > 0、活跃油井有石油但无法通过已连接路由传输，则为 **True**。未通电、节流关闭或油井不供油（包括休眠期间）时为 **False**。

`.throttle()`

当前节流设置（0-1）。默认为 0。

返回值

Number（0-1）。

`.set_throttle(rate)`

SELF ONLY

设置泵通过可达的已连接目的地输送的总速率（0-1）。0 会使泵空转；1 允许活跃油井以全输出运行，但仍受余量和吞吐量限制。脚本停止、结束或出错时，此脚本拥有的设定点会重置为 0，因此只要 Pump 应继续运行，就必须保持控制循环运行。

参数（PARAMETERS）

Name	Type	Description
rate	number	节流设置（0-1）。

返回值（RETURNS） 结果字段（RESULT FIELDS） 成功载荷（SUCCESS PAYLOAD）

`ActionResult .status`、`.message` None

结果（OUTCOMES）

Status	Kind	Meaning
"ok"	success	操作成功完成。

`.oil_out`

石油的流体输出端口。此端口可以通过 `self.oil_out.connect("Oil Reserve")` 声明一个目的地；其他消费者也可以将自己的 `oil_in` 端口连接到此 Pump。由于 Pump 是场地结构，每个目的地都需要一条连接 Pump 与目的地、已完成且兼容的 Liquid Pipe 组件。`connected_to()` 只报告此输出端口的声明；`flow_rate()` 报告实时输送总量。Pump 不储存任何东西，因此 `level()` 读数为 0。参见 `FluidPort`。

返回值

将石油路由到已连接目标的 `FluidPort`。使用目标的稳定机器 id 或显示名称调用 `connect(...)`，然后打开 `set_throttle(...)`。已完成的液体管道网络会在前哨站之间输送石油。

组件 / 基础设施与流体 (COMPONENTS / INFRASTRUCTURE & FLUIDS)

前哨站 (Outpost)

表示一个主基地或玩家建立的前哨站。使用稳定 id 调用 `get_component("outpost_1")` 查找，或使用显示名称调用 `get_component_by_name("Mining Camp")` 查找。玩家可以从 Computer System 标签页重命名前哨站，因此需要经得起重命名的脚本应使用 id。

返回方式：`get_component("outpost")`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，查找准确 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性 (PROPERTIES)

`.id`

不可变的实例 id（例如 `"outpost_home"`、`"outpost_1"`）。需要长期存在且不受重命名影响的脚本应使用它。

返回值

String（不可变实例 id，例如 `"outpost_home"`、`"outpost_1"`）

`.name()`

显示名称。主基地初始名称为 `"Nocturna Base"`；玩家建立的前哨站初始名称为 `"Outpost N"`。可从 Computer System 标签页自由重命名。名称可变，因此稳定引用应优先使用 id。

返回值

String（前哨站显示名称，可自由重命名）

`.coords()`

世界坐标，格式为 `[x, y]`。主基地位于 `[0, 0]`。

返回值

List `[x, y]`（前哨站锚点的世界坐标）

`.buildings_used()`

此处已部署的建筑数量。传感器、移动单位、结构枢纽和 POI 采集设施不计入。

返回值

Number（已部署建筑数量）

`.buildings_capacity()`

此处的软建筑阈值。超过该阈值的每栋计数建筑都会降低生产和服务吞吐量。Nocturna Base 有少量额外的初始槽位；玩家建立的前哨站使用标准阈值。

返回值

Number（此处的软建筑阈值）

`.is_full()`

当前哨站达到或超过其软建筑阈值时返回 `True`。阈值本身不会阻止普通部署。

返回值

Boolean（软建筑阈值达到或超过时为 `true`）

`.is_home()`

当这是主基地时返回 `True`。

返回值

Boolean（这是主基地时为 `true`）

`.buildings(type_id?)`

以 `BuildingRef` 快照列出此处部署的建筑。调用 `self.outpost.buildings()` 获取全部建筑，或传入建筑类型 `type_id` 进行筛选。每个引用包含 `.id`、`.name`、`.type_id`、`.outpost`、`.powered` 和 `.position`；使用 `get_component(ref.id)` 获取特定类型的实时读数。列表覆盖计入建筑容量的机器，不包括传感器、移动单位、POI 采集设施或固定 `Harvesting-field` 机器。对于这些场地机器，请使用 `harvesting_machines()`。

参数（PARAMETERS）

Name	Type	Description
<code>type_id</code>	string	可选的建筑类型 id，用于筛选返回列表。

返回值（RETURNS）

包含此处每栋已部署建筑的 `BuildingRef` 列表。传入 `type_id`（例如 `"storage_bin"`）进行筛选，或省略以返回全部建筑。没有匹配项时返回空列表。

`.harvesting_machines(type_id?)`

以 `HarvestingMachineRef` 快照列出此处 Harvesting field 上部署的固定机器。不带参数调用时返回其中全部 Grow Lamp、Sprinkler、Dispenser 和 Crop Automator；也可传入这些机器之一的 `type_id` 进行筛选。它们占用场地格，不使用建筑容量，也不会出现在 `buildings()` 中。不包括移动 Harvester、作物、散落物品、Water Wells 和其他 POI 采集器。当前只有 Nocturna Base 拥有 Harvesting field，因此已建立的前哨站返回空列表。使用 `get_component(ref.id)` 获取特定类型的实时读数。

参数 (PARAMETERS)

Name	Type	Description
<code>type_id</code>	string	可选的固定 Harvesting-field 机器类型 id，用于筛选返回列表。

返回值 (RETURNS)

此 Harvesting field 上固定机器的 `HarvestingMachineRef` 快照列表。传入 `type_id` 进行筛选，或省略以获取 Grow Lamp、Sprinkler、Dispenser 和 Crop Automator。只有 Nocturna Base 拥有此场地；已建立的前哨站返回空列表。

组件 / 基础设施与流体 (COMPONENTS / INFRASTRUCTURE & FLUIDS)

前哨站网络 (Outpost Network)

所有权属前哨站（包括主基地）的只读索引。可用于路由、部署规划、容量面板和最近服务决策，无需硬编码 `outpost_1`、`outpost_2` 等。前哨站的 `.x` 和 `.y` 标识其占地锚点。对于建筑上的动作，应选择该前哨站的 `BuildingRef` 并路由到其 `.position`。建造规划属于 Plan Mode 和共享的 `construction_blueprint` 组件；实际施工属于 Constructor 脚本。

返回方式：`get_component("outpost_network")`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，查找准确 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性 (PROPERTIES)

`.id`

此组件的稳定程序化标识符。将其用于 `get_component(id)`，以及要求提供组件、行星、载具、空间站或订单 ID 的 API。

返回值

String

`.name`

面向用户的可读显示名称。需要经得起重命名的脚本应优先使用 `.id`。

返回值

String

方法 (METHODS)

`.outposts()`

将所有所有权属前哨站作为新鲜的 `OutpostRef` 快照返回。每个引用包含 `.id`、`.name`、`.x`、`.y`、`.is_home`、`.buildings_used`、`.buildings_capacity` 和 `.is_full`。坐标是占地左上角锚点，而非某栋建筑的停靠点。

返回值

`OutpostRef` 快照列表。需要最新名称或建筑数量时重新查询。

`.home()`

返回主基地的 `OutpostRef`。

返回值

`OutpostRef` 快照。

`.nearest(x, y)`

返回距离给定世界坐标最近的所有权属前哨站。在将载具路由回基地充电或选择 `Constructor` 应在哪处待命时很有用。

参数 (PARAMETERS)

Name	Type	Description
<code>x</code>	number	世界 X 坐标。
<code>y</code>	number	世界 Y 坐标。

返回值

最近的 `OutpostRef` 快照。

组件 / 基础设施与流体 (COMPONENTS / INFRASTRUCTURE & FLUIDS)

电力控制 (Power Control)

发现每个独立电网，检查已连接的前哨站、建筑和场地电力结构，读取发电量、消耗量、电池电量和 Lightning 储备，或通过一个共享控制器操作机器断路器。电网对象是最近一次完成电力分配的快照。更改断路器或重新布线基础设施后，在下次循环迭代中重新查询，以获取更新后的电网总量和成员信息。

返回方式：`get_component("power_control")`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，查找准确 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性 (PROPERTIES)

`.id`

此组件的稳定程序化标识符。将其用于 `get_component(id)`，以及要求提供组件、行星、载具、空间站或订单 ID 的 API。

返回值

String

`.name`

面向用户的可读显示名称。需要经得起重命名的脚本应优先使用 `.id`。

返回值

String

方法 (METHODS)

`.grids()`

将行星上的每个独立电网作为最新的 **PowerGrid** 快照列表返回。彼此隔离的已完成前哨站和场地结构会各自显示为独立电网，因此脚本无需猜测或硬编码电网 id。

返回值

行星上每个独立 **PowerGrid** 快照的最新列表。

`.grid(target_id)`

查找包含 **target_id** 的电网；**target_id** 可以是前哨站、建筑或场地电力结构的 id。传入返回电网的 **.anchor_id** 可以再次查找该电网。对于未知、移动中、施工中、不属于电网，或当前尚未映射的目标，返回 **None**。

参数 (PARAMETERS)

Name	Type	Description
target_id	string	前哨站、建筑或场地电力结构的实例 id。

返回值 (RETURNS) **None** 的含义 (NONE MEANS)

Optional[PowerGrid] **None** 表示目标未知、处于移动中、施工中、不是建筑或场地电力结构，或不属于已完成的电网。

结果 (OUTCOMES)

Status	Meaning
None	目标未知、处于移动中、施工中、不是建筑或场地电力结构，或不属于已完成的电网。

`.total()`

返回覆盖每个独立电网的全行星电力汇总（PowerSummary）。常规电池储能与闪电储备保持分离，因此自动化脚本可以判断自己依赖的是哪一种供给。

返回值

覆盖每个独立电网的全行星电力汇总（PowerSummary）快照。

`.is_powered(machine_id)`

当指定机器当前已开启时返回 `True`。未知的机器 ID 返回 `False`，因此在决定是否发送断路器命令前可以安全调用此方法。

参数

名称	类型	描述
<code>machine_id</code>	<code>string</code>	机器实例 ID

返回值

布尔值：指定机器已通电时为 `True`。

`.can_power_off(machine_id)`

当指定机器存在且具有可见的断路器开关时返回 `True`。地形改造机器和许多生产机器通常可以切换；电池、被动储罐、移动单元和船舶设备通常不能切换。

参数

名称	类型	描述
<code>machine_id</code>	<code>string</code>	机器实例 ID

返回值

布尔值：指定机器暴露出断路器/电源开关时为 `True`。

.set_powered(machine_id, on)

发送与点击机器卡片开关相同的断路器命令。`power.set_powered("o2gen_1", False)` 会关闭机器；传入 `True` 则重新开启。关闭电源会暂停附着在该机器上的脚本，并保留其设定值；重新开启后，只会恢复那些因断电而暂停的脚本。

参数

名称	类型	描述
machine_id	string	机器实例 ID
on	boolean	<code>True</code> 开机； <code>False</code> 关机

返回值

返回类型	结果字段	成功载荷
ActionResult	.status、.message	None

结果

状态	类别	含义
"ok"	成功	操作成功完成。
"not_found"	拒绝	请求的对象、目标或记录不存在。
"not_toggleable"	拒绝	目标未暴露脚本电源开关。
"under_construction"	暂时性	目标仍在建造中。
"not_connected"	拒绝	组件没有活动连接。
"not_enough_power"	拒绝	可用能量低于操作要求。

```
power = get_component("power_control")
for grid in power.grids():
    print(grid.anchor_id, grid.generated, grid.consumed, grid.net)
    for member in grid.members:
        print("    ", member.id, member.roles, member.generated, member.consumed)
```


组件 / 基础设施与流体

运行控制（Run Control）

用于机器脚本的共享启动/停止控制器，是机器卡片上“运行/停止”按钮的远程等效接口。可以用它构建监督器：让一个脚本监视基地，在检测到故障时关闭另一台机器，而不必让那台机器永久停留在睡眠循环中。运行/停止轴与电源控制（power_control，断路器）相互独立：停止会结束脚本并锁定为关闭状态，而电源切换只会暂停脚本并自动恢复。

由 `get_component("run_control")` 返回。

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ① 查看精确 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件的稳定程序化标识符。将它用于 `get_component(id)`，以及要求组件、行星、载具、空间站或订单 ID 的 API。

返回值：String

`.name`

面向用户的可读显示名称。需要经受重命名的脚本应优先使用 `.id`。

返回值：String

方法

`.is_running(machine_id)`

当指定机器的脚本当前处于活动调度状态时返回 `True`，包括脚本正处于睡眠或动作中途时。暂停、停止、完成或出错的脚本返回 `False`，未知机器 ID 也一样。调用 `start / stop` 前可先用它避免重复命令。

参数：`machine_id` (string, 机器实例 ID)

返回值：布尔值：指定机器的脚本处于活动调度状态（运行中，包括睡眠/动作中途）时为 `True`；暂停、停止、完成或出错时为 `False`。

`.stop(machine_id)`

与卡片“停止”按钮相同的方式停止指定机器的脚本：`run.stop("o2gen_1")` 会结束脚本，将设定值重置为空闲，并将实时读数归零。结构状态（配方、进行中的进度、已装载材料）会保留。停止是锁定的；与断电不同，脚本不会自动恢复，需用 `start` 重新启动。

参数：`machine_id` (string, 机器实例 ID)

返回值

返回类型	结果字段	成功载荷
ActionResult	<code>.status</code> 、 <code>.message</code>	None

结果

状态	类别	含义
"ok"	成功	操作成功完成。
"not_found"	拒绝	请求的对象、目标或记录不存在。
"no_script"	拒绝	此调用找不到可启动或停止的附加脚本。

.start(machine_id)

从头运行指定机器的脚本，效果与卡片上的“运行”按钮相同：`run.start("o2gen_1")`。新的运行会从第一行重新开始，不会从脚本中途恢复。机器必须已通电且完全建成。

参数：`machine_id` (string, 机器实例 ID)

返回值

返回类型	结果字段	成功载荷
ActionResult	.status、.message	None

结果

状态	类别	含义
"ok"	成功	操作成功完成。
"not_found"	拒绝	请求的对象、目标或记录不存在。
"no_script"	拒绝	此调用找不到可启动或停止的附加脚本。
"already_running"	暂时性	操作已经在运行。
"not_powered"	拒绝	组件未通电。
"under_construction"	暂时性	目标仍在建造中。

```
run = get_component("run_control")

if run.is_running("o2gen_1"):
    run.stop("o2gen_1")          # latched off , won't auto-resume on power
else:
    run.start("o2gen_1")         # re-run the machine's script from the top
```

组件 / 基础设施与流体

蒸汽冷凝器（Steam Condenser）

以 1:1 的质量比将输入蒸汽转换为洁净水。脚本化节流控制器的峰值处理量为 250 t/h，耗电 150 W。

项目	数值
类型	Fluids
输入电力	可变（从电网汲取）
消耗	蒸汽，最高每小时 250 t，缓冲区 250 t
产出	水，最高每小时 250 t，缓冲区 250 t

获取方式

- 需要“蒸汽冷凝”（Steam Condensation）研究（Plants 1,000,000）。
- 在商店购买，价格为 50,000 cr。

由 `self` / `get_component(id)` 返回。

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ⓘ 查看精确 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件的稳定程序化标识符。将它用于 `get_component(id)`，以及要求组件、行星、载具、空间站或订单 ID 的 API。

返回值：String

`.name`

面向用户的可读显示名称。需要经受重命名的脚本应优先使用 `.id`。

返回值：String

方法

`.outpost`

此建筑部署所在的前哨站。返回的 `OutpostRef` 包含稳定 ID、显示名称、生物群系、位置、容量和 `buildings()` 查询。需要当前值时请重新读取此属性。

返回值：此建筑所在前哨站的 `OutpostRef`。

`.condensation_rate()`

上一个模拟 tick 产生的洁净水量，单位为 t/h。当蒸汽输入有供给、水输出有空间且宿主前哨站未超员时，满节流可达到 250 t/h。

返回值：上一个 tick 产生的洁净水量，单位为 t/h。

`.efficiency()`

上一个 tick 实际完成的冷凝量，占请求冷凝量的比例（0.0-1.0）。数值偏低表示蒸汽输入不足，或水输出在 tick 完成前已满。

返回值：Number（0-1），已完成请求冷凝量的比例。

`.is_stalled()`

当节流大于零，且当前流体状态阻止冷凝（因为 `steam_in` 为空或 `water_out` 已满）时返回 `True`。该状态直接由两个端口推导；使用 `status()` 区分具体阻塞原因。

返回值：当节流开启而蒸汽为空或水缓冲区已满时为 `True`。

`.status()`

当前可操作状态：`"idle"`、`"no_power"`、`"no_steam"`、`"output_full"` 或 `"running"`。该状态根据节流、电力和两个流体缓冲区实时推导。节流为 0 时报告 `"idle"`；检查端口液位后再决定何时重新打开节流。

返回值：上述值之一。

`.throttle()`

当前冷凝设定值（0.0-1.0）。它会线性缩放蒸汽消耗、水输出和耗电量。

返回值：Number（0-1），当前冷凝设定值。

.set_throttle(t) 仅限自身 (SELF ONLY)

将冷凝量设置为 0.0-1.0（超出范围会截断）。0 表示空闲，不进行转换且无可变耗电；1.0 请求 250 t/h，并消耗 150 W。即使蒸汽为空或输出已满，耗电也会跟随节流，因此受阻时应设置为 0 以节省电力。此脚本拥有的设定值会在脚本停止、结束或出错时重置为 0。请从此冷凝器自身的脚本调用。

参数

名称	类型	描述
t	number	0-1 范围内的冷凝速率比例。

返回值：ActionResult；结果字段 **.status**、**.message**；成功载荷 **None**。

结果："ok"（成功）表示操作成功完成。

.steam_in

蒸汽输入端口。连接 Thermal Cap（热能帽）或锁定为蒸汽模式的 Gas Tank（气体罐）。其 250 t 内部缓冲区会在电网为机器供电后被冷凝消耗。

返回值：接受来自 Thermal Cap 或 Gas Tank 的 **FluidPort**。

.water_out

洁净水输出端口。连接 Liquid Tank、Large Liquid Tank、Plant Terraformer、Sprinkler 或其他用水设备。其 250 t 内部缓冲区在已满时会对冷凝形成背压。

返回值：供应冷凝洁净水的 **FluidPort**。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()`
仅限自身 (SELF ONLY)

读取脚本运行期间从编辑器 “ Commands ” 选项卡发送的命令。所有可编程机器的行为相同，参见指南 “ 脚本命令 (Script Commands) ”。

组件 / 基础设施与流体

Thermal Cap（热能帽）

从热能喷口捕获蒸汽。如果腔室达到 100%，储存的每一吨蒸汽都会喷入大气；脚本必须释放、导流或泄压。

项目	数值
类型	Fluids
建造于	热能喷口
输入电力	-3 W（从电网汲取）
产出	蒸汽，缓冲区 1,000 t

获取方式

- 配方随 Thermal Cap 研究解锁（Pressure 2.5）。
- 在 Fabricator 上制造 Thermal Cap Kit：2× Titanium Ingot 和 2× Gas Pipe Segment。
- 使用 Pioneer's Constructor 将其建造在已勘测的热能喷口上。

由 `self / get_component(id)` 返回。

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ① 查看精确 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

.id

此组件的稳定程序化标识符。将它用于 `get_component(id)`，以及要求组件、行星、载具、空间站或订单 ID 的 API。

返回值：String

.name

面向用户的可读显示名称。需要经受重命名的脚本应优先使用 `.id`。

返回值：String

方法

`.vent()`

此热能帽所在的 `ThermalVent`（热能喷口）。字段可用性取决于最近一次勘测该喷口所用的声纳等级：基础等级显示阶段，宽域等级增加蒸汽速率，深度等级增加周期时间。使用 `self.vent().current_phase()`（或热能帽自身的 `phase()`）来判断何时有蒸汽流入。参见 `ThermalVent`。

返回值：此热能帽所在喷口的 `ThermalVent`。可从中读取 `current_phase()` / `next_phase_in()` / 速率，也可以使用热能帽自身的 `phase()` / `next_phase_in()`。

`.phase()`

喷口产出蒸汽（腔室填充）时为 `"active"`，休息（腔室只排空）时为 `"dormant"`。在喷口完成勘测前为 `None`。应据此驱动释放循环：处于 `active` 时打开节流，处于 `dormant` 时降低节流，避免下游干涸。

返回值：`"active"`、`"dormant"`；未勘测时为 `None`。

`.next_phase_in()`

喷口在 `active` 与 `dormant` 之间切换前剩余的游戏内分钟数，因此可以在涌流前提前打开，或在干涸期前降低节流。除非喷口经过 `Deep` 勘测，否则返回 `None`；深度声纳的价值就在于此预测循环。

返回值：喷口切换阶段前剩余的游戏内分钟数；未经过 `Deep` 勘测时为 `None`。

`.pressure()`

腔室填充量，范围为 0.0 到 1.0。热能帽从喷口捕获蒸汽时填充量上升，通过 `steam_out` 释放时下降。达到 1.0 后热能帽会过压：整个腔室向大气排放，并从空腔重新建立。目标是避免触顶，因此每个 tick 轮询压力并在压力上升时打开节流。

返回值：0.0 到 1.0 的腔室填充量；达到 1.0 时过压并向大气排放，随后从空腔重新填充。

`.capture_rate()`

上一个 tick 从喷口捕获的蒸汽量，单位为 t/h。dormant 阶段为 0，active 阶段最高可达喷口当前输出。它已经纳入喷口阶段因素，因此应直接读取，而不是根据喷口速率自行计算。每个流体 tick 更新一次。

返回值：本 tick 从喷口捕获的蒸汽量，单位为 t/h；喷口 dormant 时为 0。

`.is_overpressured()`

当腔室在本 tick 达到顶端并将全部内容吹入大气时为 **True**。此后腔室为空，必须从喷口重新填充才能再次获得蒸汽，因此之前储存的全部蒸汽都会丢失。如果看到此状态，说明释放得太慢，应更早打开节流。

返回值：本 tick 腔室达到顶端并向大气排放时为 **True**；之后从空腔重新建立，之前储存的蒸汽全部损失。

`.is_stalled()`

如果在上一个流体 tick 中，已通电的热能帽打开了节流，腔室有可释放的蒸汽，但无法通过任何已连接路径传输，则返回 `True`。腔室为空、节流关闭或没有电力时不会报告 stalled。可用 `self.steam_out.connect(...)` 声明目的地，或让消费者将自己的 `steam_in` 端口连接到此热能帽。

返回值：已通电、节流 > 0 且腔室有蒸汽可释放但无法通过已连接路径传输时为 `True`；未通电、节流关闭或腔室没有可释放蒸汽时为 `False`。

`.throttle()`

当前释放阀设定值，范围为 0.0（密封）到 1.0（完全打开）。在调用 `set_throttle(...)` 后读取该值。

返回值：0.0 到 1.0 范围内的当前释放阀设定值。

`.set_throttle(t)` 仅限自身 (SELF ONLY)

将热能帽释放阀打开到 0.0-1.0 范围（超出范围会截断）。0 会密封腔室，使其填充；1.0 会在腔室供给、目的地余量和吞吐量允许的范围内，尽快向所有可到达的已连接目的地释放蒸汽。这是防止过压的主要旋钮，应根据 `pressure()` 每个 tick 调用。当没有目的地能够接收足够蒸汽且 `pressure()` 仍在上升时，使用 `set_relief(...)` 排出多余蒸汽。此脚本拥有的设定值会在脚本停止、结束或出错时重置为 0。

参数：`t` (number, 0-1 范围内的释放阀比例)

返回值：`ActionResult`；结果字段 `.status`、`.message`；成功载荷 `None`。

结果：`"ok"` (成功) 表示操作成功完成。

`.relief()`

当前泄压阀设定值，范围为 0.0 (关闭) 到 1.0 (完全打开)。在调用 `set_relief(...)` 后读取该值。

返回值：0.0 到 1.0 范围内的当前泄压阀设定值，表示向大气溢流的开度。

`.relief_rate()`

本 tick 通过泄压阀排入大气而浪费的蒸汽量，单位为 t/h。泄压阀关闭时为 0。可用它观察正在排出的多余蒸汽量。

返回值：本 tick 通过泄压阀排入大气的蒸汽量，单位为 t/h；阀门关闭时为 0。

`.set_relief(t)` 仅限自身 (SELF ONLY)

将泄压阀从 0-1 打开，把多余的腔室蒸汽排入大气。当已连接的消费者跟不上且 `pressure()` 仍在上升时使用。0 表示保留全部蒸汽供消费者使用。超出范围的值会被截断。只能从 Thermal Cap 自身的脚本调用。

参数：`t` (number, 0-1 范围内的泄压阀比例)

返回值：ActionResult；结果字段 `.status`、`.message`；成功载荷 None。

结果："ok" (成功) 表示操作成功完成。

`.steam_out`

腔室蒸汽的流体输出端口。该端口可以使用 `self.steam_out.connect("bio_caster_1")` 声明一个目的地；其他消费者可以将自己的 `steam_in` 端口连接到此热能帽。由于热能帽是场结构，每个目的地都需要有一条兼容且已完成的 Gas Pipe 组件连接到热能帽和目的地。`connected_to()` 只报告此输出端口自身的声明；`flow_rate()` 报告实时总释放量。参见 FluidPort。

返回值：腔室蒸汽的 FluidPort。该端口可以使用 `connect("bio_caster_1")` 声明一个目的地；其他消费者可以将自身的 `steam_in` 端口连接到此热能帽。已完成的气体管道网络会在前哨站之间输送蒸汽。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()`
仅限自身 (SELF ONLY)

读取脚本运行期间从编辑器 “ Commands ” 选项卡发送的命令。所有可编程机器的行为相同，参见指南 “ 脚本命令 (Script Commands) ”。

组件 / 基础设施与流体

水泵 (Water Pump)

以脚本设置的节流速率从已勘测的水井抽取水。其行星地图蓝图可以在计划模式中放置，也可以由脚本放置；但仍需由 Pioneer 在水井上建造。

项目	数值
类型	Fluids
建造于	水井
输入电力	-4 W (从电网汲取)
产出	水, 缓冲区 10 t
输出缓冲区	20 units

获取方式

- 配方随 “ 水文勘测 ” (Hydrology Survey) 研究解锁 (Pressure 30)。
- 在 Fabricator 上制造 Water Pump：2× Iron Ingot、2× Glass 和 4× Liquid Pipe Segment。
- 使用 Pioneer's Constructor 将其建造在已勘测的水井上。

由 `self` / `get_component(id)` 返回。

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ① 查看精确 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性

`.id`

此组件的稳定程序化标识符。将它用于 `get_component(id)`，以及要求组件、行星、载具、空间站或订单 ID 的 API。

返回值：String

`.name`

面向用户的可读显示名称。需要经受重命名的脚本应优先使用 `.id`。

返回值：String

方法

`.well()`

此水泵固定安装所在的 `WaterWell`（水井）。读取 `.yield_tier()` 可查看水井是 "standard"/"rich"/"pure"（ $1\times/2\times/3\times$ 倍率），读取 `.flow_rate()` 可查看水井的每小时产出。适用于比较整个泵群产量并进行优先级排序的脚本。

返回值：此泵所在水井的 `WaterWell` 快照：`.id`、`position()`、`yield_tier()`、`flow_rate()`。

`.pump_rate()`

本 tick 输送到已连接目的地的总水量，单位为 t/h。当节流为 0 或没有目的地可以接收流量时读数为 0。如果高产水井仍报告 0，请检查连接、已完成的管道路径、冲突、电力和目的地容量。

返回值：本 tick 跨所有可到达已连接目的地输送的总水量，单位为 t/h；节流为 0 或没有目的地能接收流量时为 0。

`.is_stalled()`

如果在上一个流体 tick 中，已通电的水泵打开了节流，水井有可用水，但无法通过任何已连接路径传输，则返回 **True**。没有可用水、节流关闭或没有电力时不会报告 stalled。可用 `self.water_out.connect(...)` 声明目的地，或让消费者将自己的 `water_in` 端口连接到此水泵。

返回值：已通电、节流 > 0 且水井有水但无法通过已连接路径传输时为 **True**；未通电、节流关闭或没有可抽取的水时为 **False**。

`.throttle()`

当前节流设定值（0-1）。默认值为 0；在脚本调用 `set_throttle()` 前，水泵保持空闲。

返回值：Number（0-1），当前节流设定值。

`.set_throttle(rate)` 仅 SELF

设置泵（pump）在所有可到达的已连接地上的总输出速率（0-1）。0 使泵处于空闲状态（不抽取、不消耗）；1 允许在受水头余量和吞吐量限制的情况下输出井的全部产量。此脚本拥有的设定值会在脚本停止、结束或出错时重置为 0，因此泵运行期间请保持控制循环持续运行。

参数（PARAMETERS）

名称	类型	描述
rate	number	节流设定（0-1）。默认值为 0：泵在脚本控制前保持空闲。

返回值（RETURNS） `ActionResult`

结果字段（RESULT FIELDS） `.status`、`.message`

成功载荷（SUCCESS PAYLOAD） 无

结果（OUTCOMES）

状态（Status）	类型（Kind）	含义（Meaning）
"ok"	成功（success）	操作成功完成。

`.output`

保存抽水副产物盐的拾取输出槽 (`PickupOutputSlot`)。它与通过液体管道输送水的 `water_out` 分开。可读取 `count()`、`capacity()` 或 `stacks()` 进行检查。实体存在的漫游车 (`Rover`) 或先驱者 (`Pioneer`) 会连接自己的输入并抽取盐；泵没有直接的物品路由方法。

返回值 (RETURNS)

为实体存在的 `Rover` 或 `Pioneer` 保存离散盐副产物的 `PickupOutputSlot`。

`.water_out`

水的流体输出端口 (fluid output)。此端口可以通过 `self.water_out.connect("bio_caster_1")` 声明一个目的地；其他消费者也可以将自己的 `water_in` 端口连接到此泵。由于泵是场地结构，每个目的地都需要一条已完成、通达泵和目的地的液体管道路线。`connected_to()` 只报告此输出端口的声明；`flow_rate()` 报告当前实时总输送量。泵不储存任何东西，因此 `level()` 读取为 0。参见 `FluidPort`。

返回值 (RETURNS)

将水路由至已连接目标的 `FluidPort`。调用 `connect(...)` 时传入目标的稳定机器 ID 或显示名称，然后开启 `set_throttle(...)`。已完成的液体管道网络会在前哨之间输送水。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` 仅 SELF

读取脚本运行期间你从编辑器 Commands (命令) 选项卡发送的命令。所有可脚本化机器的行为都相同，参见指南：Script Commands。

组件（COMPONENTS） / 生物圈（BIOSPHERE）

生物质混合器（Biomass Mixer）

将生物群系精华（biome essences）混合为生物质。全局生物质阶段（Biomass phase）设定最低多样性；每一种额外且平衡良好的精华都可以提高产量。Mk II 使产量提高 4.5×，而精华需求仅提高 2.4×，功耗为 5 倍。

类型（Type）	生物圈（Biosphere）
输入功率（Power in）	可变（从电网汲取）
消耗（Consumes）	冻结精华（Frozen Essence），缓冲区 30 t
消耗（Consumes）	海岸精华（Coastal Essence），缓冲区 30 t
消耗（Consumes）	地热精华（Geothermal Essence），缓冲区 30 t
消耗（Consumes）	火山精华（Volcanic Essence），缓冲区 30 t
消耗（Consumes）	深层精华（Deep Essence），缓冲区 30 t
等级（Tiers）	Mk II

获取方式（HOW TO OBTAIN）

- 需要生物圈研究（Biosphere research）（Terraform Index 210,000）。
- 从商店购买，价格为 60,000 cr。

由 `self` / `get_component(id)` 返回。

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，找到确切 ID，然后将该值传递给 `get_component(id)`。ID 区分大小写。

属性（PROPERTIES）

`.id`

此组件稳定的程序化标识符。将其与 `get_component(id)` 以及要求提供组件、行星、载具、空间站或订单 ID 的 API 一起使用。

返回值（RETURNS） `String`

.name

人类可读的显示名称。需要在重命名后仍能工作的脚本应优先使用 **.id**。

返回值 (RETURNS) **String**

方法 (METHODS)

.outpost

部署此建筑的前哨。返回的 **OutpostRef** 包含其稳定 ID、显示名称、生物群系、位置、容量以及 **buildings()** 查询。需要当前值时请再次读取该属性。

返回值 (RETURNS) 部署此建筑的前哨的 **OutpostRef**。

.biomass_rate()

上一个 tick 生产的生物质吨数，单位为 t/h。将行星上所有 Biomass Mixer 的值相加，即为生物质总生产速率。

返回值 (RETURNS) 上一个 tick 生产的生物质数量，单位为 t/h 的 **Number**。

.active_essences()

当前携带供应的精华输入缓冲区数量 (0-5)。当前阶段通过 **required_essences()** 决定最低数量。额外且平衡的精华类型可以提高产量。

返回值 (RETURNS) 当前有供应的精华输入缓冲区数量 (0-5) 的 **Number**。

`.mixing_essences()`

上一个 tick 中，为获得最强平衡混合而选中的已供应精华类型数量（0-5）。混合器会评估从阶段最低值开始的每一个多样性等级，因此较弱的额外供给绝不会降低产量。

返回值（RETURNS） 上一个 tick 中选中的已供应精华类型数量（0-5）的 **Number**。

`.tier()`

已安装的混合器等级：Mk I 为 1，Mk II 为 2。Mk II 产生 4.5× 的生物质，同时每种选定精华的消耗量仅为 2.4×，功耗为 5×，因此每吨精华的生物质产量高出 87.5%。

返回值（RETURNS） 已安装混合器等级的 **Number**：1（Mk I）或 2（Mk II）。

`.is_stalled()`

如果混合器已通电，但含有可用精华的输入缓冲区少于当前生物质阶段所需数量，则为 **True**。即使没有新的流入，缓冲的精华也会计入。未通电或可用精华类型足够时为 **False**。将每个缺失的精华输入连接到兼容来源，并在需要时完成远程液体管道路线；生物质阶段只会前进，不会下降。

返回值（RETURNS） 布尔值：混合器已通电且可用精华类型不足时为 **True**。

`.phase()`

全局生物阶段（1-6），由累计生物吨数推导，使用与 Sensors 阶段徽章相同的阈值。阶段设置精华多样性的最低要求，但不会直接乘算产量。

返回值（RETURNS）由累计生物吨数推导出的全局生物阶段 **Number**（1-6）；它设置所需的最低精华多样性。

`.required_essences()`

当前阶段必须供应的不同生物群系精华数量（1-5，等于阶段值但上限为 5）。低于此数量时混合器会停滞。供应更多精华在混合足够平衡时可以提高产量。

返回值（RETURNS）当前阶段所需的最少不同生物群系精华数量 **Number**（1-5）。额外且平衡的精华可以提高产量。

`.frozen_essence_in`

冻结精华输入端口。使用 `self.frozen_essence_in.connect(...)` 进行连接。参见 **FluidPort**。

返回值（RETURNS）冻结精华输入的 **FluidPort**。

`.coastal_essence_in`

海岸精华输入端口。使用 `self.coastal_essence_in.connect(...)` 进行连接。参见 `FluidPort`。

返回值 (RETURNS) 海岸精华输入的 `FluidPort`。

`.geothermal_essence_in`

地热精华输入端口。使用 `self.geothermal_essence_in.connect(...)` 进行连接。参见 `FluidPort`。

返回值 (RETURNS) 地热精华输入的 `FluidPort`。

`.volcanic_essence_in`

火山精华输入端口。使用 `self.volcanic_essence_in.connect(...)` 进行连接。参见 `FluidPort`。

返回值 (RETURNS) 火山精华输入的 `FluidPort`。

`.deep_essence_in`

深层精华输入端口。使用 `self.deep_essence_in.connect(...)` 进行连接。参见 `FluidPort`。

返回值 (RETURNS) 深层精华输入的 `FluidPort`。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` 仅 SELF

读取脚本运行期间你从编辑器 Commands 选项卡发送的命令。所有可脚本化机器的行为都相同，参见指南：Script Commands。

组件（COMPONENTS） / 生物圈（BIOSPHERE）

作物自动化器（Crop Automator）

在以自身为中心的 5×5 服务区域内，跨最多另外 24 个地块单元格排队收获、种植和处理作业，然后一次执行一项作业，作业之间有短暂停顿。

类型（Type）	生物圈（Biosphere）
输入功率（Power in）	-60 W（从电网汲取）
输出缓冲区（Output buffer）	50,000 units
储备堆（Stockpile）	400 units（混合）

获取方式（HOW TO OBTAIN）

- 需要 Field Automation 研究（Plants 620,000）。
- 从商店购买，价格为 30,000 cr。
- 使用 Harvester 的 `deploy()` 将套件部署在空的田地单元格上。

由 `self` 返回。

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，找到确切 ID，然后将该值传递给 `get_component(id)`。ID 区分大小写。

属性（PROPERTIES）

`.id`

此组件稳定的程序化标识符。将其与 `get_component(id)` 以及要求提供组件、行星、载具、空间站或订单 ID 的 API 一起使用。

返回值（RETURNS）String

`.name`

人类可读的显示名称。需要在重命名后仍能工作的脚本应优先使用 `.id`。

返回值（RETURNS）String

方法 (METHODS)

`.harvest(sector)` 仅 SELF

为一个受覆盖的地块提交收获作业。提交立即完成；有效的田地作业稍后需要 0.1 小时。输出空间不足会暂停这个 FIFO 队首，而不会绕过它。目标不匹配属于终止性结果，不消耗作业时间，并使队列前进。

参数 (PARAMETERS)

名称	类型	描述
<code>sector</code>	<code>string</code>	自动化器服务区域内的一个单元格。

返回值 (RETURNS) `JobReceipt`

结果字段 (RESULT FIELDS) `.status`、`.message`

成功载荷 (SUCCESS PAYLOAD) `.job_id`、`.queue_position`

结果 (OUTCOMES)

状态	类型	含义
<code>"queued"</code>	成功	已将作业 <code>.job_id</code> 排入执行位置 <code>.queue_position</code> 。
<code>"queue_full"</code>	暂时性	Crop Automator 的 50 作业队列已满。
<code>"not_placed"</code>	拒绝	Crop Automator 尚未部署在田地上。
<code>"out_of_range"</code>	拒绝	请求的地块位于 Crop Automator 服务区域之外。

引发 (RAISES)

异常	条件
<code>ValueError</code>	<code>sector</code> 参数不是有效的田地地块 ID。

`.plant(sector, seed_id)` 仅 SELF

提交一个使用指定物种种子的种植作业。提交立即完成，不要求种子已装载。串行执行器会在 FIFO 队首暂停，直到种子到位；随后耗时 0.1 小时，并在提交前再次检查目标。

参数 (PARAMETERS)

名称	类型	描述
<code>sector</code>	<code>string</code>	自动化器服务区域内的一个单元格。
<code>seed_id</code>	<code>string</code>	物种专用种子物品 ID。

返回值 (RETURNS) `JobReceipt`

结果字段 (RESULT FIELDS) `.status`、`.message`

成功载荷 (SUCCESS PAYLOAD) `.job_id`、`.queue_position`

结果 (OUTCOMES)

状态	类型	含义
<code>"queued"</code>	成功	已将作业 <code>.job_id</code> 排入执行位置 <code>.queue_position</code> 。
<code>"queue_full"</code>	暂时性	Crop Automator 的 50 作业队列已满。
<code>"not_placed"</code>	拒绝	Crop Automator 尚未部署在田地上。
<code>"out_of_range"</code>	拒绝	请求的地块位于 Crop Automator 服务区域之外。
<code>"invalid_seed"</code>	拒绝	请求的物品不是物种种子。

引发 (RAISES)

异常	条件
<code>ValueError</code>	<code>sector</code> 参数不是有效的田地地块 ID。

`.apply(sector, item_id)` 仅 SELF

提交一个 Fertilizer Mk I/II/III（肥料 Mk I/II/III）或 Growth Accelerant（生长促进剂）作业。提交立即完成，不要求材料已装载。串行执行器会在 FIFO 队首暂停，直到材料到位；随后耗时 0.1 小时，并在提交前再次检查目标。

参数 (PARAMETERS)

名称	类型	描述
sector	string	自动化器服务区域内的一个单元格。
item_id	string	Fertilizer Mk I/II/III 或 Growth Accelerant。

返回值 (RETURNS) `JobReceipt`

结果字段 (RESULT FIELDS) `.status`、`.message`

成功载荷 (SUCCESS PAYLOAD) `.job_id`、`.queue_position`

结果 (OUTCOMES)

状态	类型	含义
"queued"	成功	已将作业 <code>.job_id</code> 排入执行位置 <code>.queue_position</code> 。
"queue_full"	暂时性	Crop Automator 的 50 作业队列已满。
"not_placed"	拒绝	Crop Automator 尚未部署在田地上。
"out_of_range"	拒绝	请求的地块位于 Crop Automator 服务区域之外。
"invalid_material"	拒绝	请求的物品不是受支持的作物处理材料。

引发 (RAISES)

异常	条件
<code>ValueError</code>	<code>sector</code> 参数不是有效的田地地块 ID。

`.position()`

此自动化器占据的网格地块。作业最多可以指向其以自身为中心的 5×5 服务区域内另外 24 个单元格。

返回值 (RETURNS) 自动化器自身的田地地块。作业最多可以指向其以自身为中心的 5×5 服务区域内另外 24 个单元格。

`.cell(sector)`

将此自动化器服务区域内的一个地块读取为 `Cell` 快照，其中包含植物、状态、生长、条件和剩余处理小时数。可寻址集合正是 `harvest()`、`plant()` 和 `apply()` 接受的集合；因此对于返回 `None` 的地块，任何作业都无法定位到那里。

参数 (PARAMETERS)

名称	类型	描述
<code>sector</code>	<code>string</code>	自动化器服务区域内的一个单元格。

返回值 (RETURNS) `Optional[Cell]`

`None` 的含义 (NONE MEANS) `None` 表示此自动化器无法寻址该地块：它位于服务区域之外，或不是有效的田地地块。

结果 (OUTCOMES)

状态	含义
<code>None</code>	此自动化器无法寻址该地块：它位于服务区域之外，或不是有效的田地地块。

`.cells()`

读取此自动化器服务的所有地块，返回 `Cell` 快照列表，用于一次遍历整个服务区域。未扫描的自然地面会报告状态 `"unknown"`。

返回值 (RETURNS) 此自动化器服务的 `Cell` 快照列表，每个可寻址地块一个。自动化器未放置在田地上时为空。

`.status()`

精确的执行器状态：`"not_placed"`、`"no_power"`、`"working"`、`"no_seed"`、`"no_material"`、`"output_full"`、`"results_full"` 或 `"idle"`。

返回值 (RETURNS) `string`

可能的值 (POSSIBLE VALUES) `"not_placed"`、`"no_power"`、`"working"`、`"no_seed"`、`"no_material"`、`"output_full"`、`"results_full"`、`"idle"`

`.current_job()`

当前的 FIFO 队首，作为 `CropJob` 返回，其中包括动作、目标、进度和阻塞原因。空闲时返回 `None`。

返回值 (RETURNS) `Optional[CropJob]`

`None` 的含义 (NONE MEANS) `None` 表示 Crop Automator 没有活动作业。

结果 (OUTCOMES)

状态	含义
<code>None</code>	Crop Automator 没有活动作业。

`.get_queue()`

按精确 FIFO（先进先出）顺序快照读取待处理的 `CropJob` 值。活动作业由 `current_job()` 单独报告。

返回值 (RETURNS) `list<CropJob>`

`.queue_count()`

未完成作业的总数，包括活动作业和所有待处理作业。最大值为 50。

返回值 (RETURNS) `number`

`.result_count()`

等待处理的已完成终止性结果数量。达到 50 条时，执行会暂停，直到结果被消费。

返回值 (RETURNS) `number`

`.next_result()` 仅 SELF

消费最旧的终止性结果。空的收件箱会被报告，但不会改变机器状态。

返回值 (RETURNS) `CropJobResult`

结果字段 (RESULT FIELDS) `.status`、`.message`

成功载荷 (SUCCESS PAYLOAD) `.job_id`、`.action`、`.sector`、`.item_id`、`.collected`、`.discarded`

结果 (OUTCOMES)

状态	类型	含义
"ok"	成功	作业 <code>.job_id</code> 在 <code>.sector</code> 完成；世界状态变更已成功提交。
"partial"	部分成功	作业 <code>.job_id</code> 收获了 <code>.sector</code> ，但输出箱只有空间容纳 <code>.collected</code> Forage； <code>.discarded</code> 被丢弃，且地块已清空。
"empty"	成功	没有正在等待的已完成 Crop Automator 结果。
"out_of_range"	拒绝	执行前目标离开了此 Crop Automator 的服务区域。
"no_plant"	拒绝	作业到达执行器时，目标地块没有植物。
"not_mature"	拒绝	收获作业到达执行器时，目标植物尚未成熟。
"no_forage"	拒绝	准备好的作物不含完整的 Forage。
"not_empty"	拒绝	种植作业到达执行器时，目标地块并非空地。
"base_sector"	拒绝	Harvester 基础地块不能种植。
"already_mature"	拒绝	处理作业到达执行器时，目标植物已经成熟。
"tier_conflict"	拒绝	请求的 Fertilizer 等级与目标植物上的活动剂量冲突。
"invalid_seed"	拒绝	种植作业引用了无效的物种种子。
"invalid_material"	拒绝	处理作业引用了不受支持的材料。

`.cancel_job(job_id)` 仅 SELF

按 ID 取消一个活动或待处理作业。取消活动作业只会丢弃其进度；输入和田地状态保持不变。

参数 (PARAMETERS)

名称	类型	描述
job_id	number	由 <code>harvest()</code> 、 <code>plant()</code> 或 <code>apply()</code> 返回的作业 ID。

返回值 (RETURNS) `ActionResult`

结果字段 (RESULT FIELDS) `.status`、`.message`

成功载荷 (SUCCESS PAYLOAD) 无

结果 (OUTCOMES)

状态	类型	含义
"ok"	成功	操作成功完成。
"not_found"	拒绝	请求的对象、目标或记录不存在。

`.move_job(job_id, position)` 仅 SELF

将一个未完成作业移动到从 1 开始的执行位置，其中活动作业计为第一个，随后是待处理作业。重新排序仅待处理作业时会保留活动进度。作业处于活动状态时，改变哪个作业排在第一位会抢占执行臂：被移出的作业保留其 ID 和请求，丢失进度，并且不会创建终止性结果。位置 1 为第一位；`queue_count()` 为最后位置。

参数 (PARAMETERS)

名称	类型	描述
<code>job_id</code>	<code>number</code>	活动或待处理作业的稳定 ID。
<code>position</code>	<code>number</code>	跨活动和待处理作业的一基执行位置。

返回值 (RETURNS) `ActionResult`

结果字段 (RESULT FIELDS) `.status`、`.message`

成功载荷 (SUCCESS PAYLOAD) 无

结果 (OUTCOMES)

状态	类型	含义
<code>"ok"</code>	成功	未完成作业现在占据请求的执行位置。
<code>"not_found"</code>	拒绝	没有活动或待处理作业拥有提供的 ID。
<code>"invalid"</code>	拒绝	提供的位置超出从 1 到 <code>queue_count()</code> 的一基范围。

`.clear_queue()` 仅 SELF

取消活动作业和所有待处理作业。已完成的结果仍可通过 `next_result()` 获取。

返回值 (RETURNS) `CountResult`

结果字段 (RESULT FIELDS) `.status`、`.message`

成功载荷 (SUCCESS PAYLOAD) `.count`

结果 (OUTCOMES)

状态	类型	含义
<code>"ok"</code>	成功	命令影响了 <code>.count</code> 个条目或单位。
<code>"no_op"</code>	成功	命令没有影响任何条目或单位。

`.input`

接受物种种子、Fertilizer Mk I/II/III 和 Growth Accelerant 的标准多材料 **InputSlot**。

返回值 (RETURNS) **InputSlot**

`.output`

包含收集到的 Forage 的标准 **OutputSlot**。通过此槽发送到本地目的地，或让本地 Plant Terraformer 从其标准输入中抽取。每条路线都使用正常的定时传输。

返回值 (RETURNS) **OutputSlot**

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` 仅 SELF

读取脚本运行期间你从编辑器 Commands 选项卡发送的命令。所有可脚本化机器的行为都相同，参见指南：Script Commands。

组件（COMPONENTS） / 生物圈（BIOSPHERE）

撒盐器（Dispenser）

在通电、有供应且启用时，为正交相邻的四个田地单元格（正上、正下、正左和正右）撒盐。

类型（Type）	生物圈（Biosphere）
输入功率（Power in）	可变（从电网汲取）
输入缓冲区（Input buffer）	50 units

获取方式 (HOW TO OBTAIN)

- 配方随 Dispenser 研究 (Plants 300,000) 解锁。
- 在 Fabricator 上制造 Dispenser Kit : 1× Machine Frame、1× Control Unit 和 1× Circuit Panel。
- 使用 Harvester 的 `deploy()` 将套件部署在空的田地单元格上。

由 `self / get_component(id)` 返回。

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ①，找到确切 ID，然后将该值传递给 `get_component(id)`。ID 区分大小写。

属性 (PROPERTIES)

`.id`

此组件稳定的程序化标识符。将其与 `get_component(id)` 以及要求提供组件、行星、载具、空间站或订单 ID 的 API 一起使用。

返回值 (RETURNS) String

`.name`

人类可读的显示名称。需要在重命名后仍能工作的脚本应优先使用 `.id`。

返回值 (RETURNS) String

方法 (METHODS)

`.set_enabled(enabled)` 仅 SELF

开启或关闭撒盐。断电会暂停脚本，但保留此设定值；停止机器脚本会将其重置为 `False`。

参数 (PARAMETERS)

名称	类型	描述
<code>enabled</code>	<code>boolean</code>	此脚本是否命令撒盐。

返回值 (RETURNS) `ActionResult`

结果字段 (RESULT FIELDS) `.status`、`.message`

成功载荷 (SUCCESS PAYLOAD) 无

结果 (OUTCOMES)

状态	类型	含义
<code>"ok"</code>	成功	操作成功完成。

`.is_enabled()`

当正在运行的脚本已命令开启撒盐时为 `True`。

返回值 (RETURNS) `boolean`

`.is_active()`

当已命令开启、拥有电力且有盐可用时为 `True`。

返回值 (RETURNS) `boolean`

`.is_supplied()`

当 Dispenser（分配器）已被命令开启、已通电且其输入缓冲区中有盐时返回 **True**。若其被禁用、断电或为空，则覆盖的单元会失去 **salted** 状态。

返回值

Boolean：当分配器已部署、被命令开启、已通电且输入缓冲区中有盐时返回 **True**。当未部署、被禁用、断电或为空时返回 **False**；此时覆盖的单元会失去 **salted** 状态，其植物会暂停。

`.status()`

精确运行状态："not_placed"、"disabled"、"no_power"、"no_salt" 或 "active"。

返回值：精确运行状态。

可能的值："not_placed"、"disabled"、"no_power"、"no_salt"、"active"

`.buffer()`

当前板载盐缓冲区的填充比例（0-1）。分配单元时会下降，并通过 **self.input** 补充。

返回值

Number（0-1）：当前板载盐缓冲区的填充比例。分配器为单元施盐时会下降；向 **self.input** 输入盐时会补充。0 表示为空（覆盖的单元会失去 **salted** 状态）。

`.tier()`

始终为 1。Dispenser（分配器）以 Mk I 出厂且没有升级包；盐供应设备不分级，因此每个已部署的分配器都返回 1。

返回值

Number：始终为 1。分配器以 Mk I 出厂且没有升级包；盐供应设备不分级。每个已部署的分配器均返回 1。

`.position()`

该分配器占据的网格扇区，例如 "E14"。

返回值

String：该分配器占据的网格扇区（例如 "E14"）。

`.input`

从储存箱或其他来源装载盐。使用 `self.input.connect(name)` 连接来源，然后使用 `self.input.take("salt", count)` 提取盐。参见 InputSlot（输入槽）。

返回值

InputSlot：`connect()`、`take()`、`eject()`、`flush()`、`count()`、`capacity()`、`connected_to()`。从储存箱输入盐：`self.input.connect("Salt Bin")`，然后执行 `self.input.take("salt", 50)`。盐是 Water Pump（水泵）的副产品。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` 仅限自身

读取脚本运行期间从编辑器 Commands（命令）选项卡发送的命令。所有可脚本化机器均相同；参见指南：Script Commands（脚本命令）。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` 仅限自身

读取脚本运行期间从编辑器 Commands（命令）选项卡发送的命令。所有可脚本化机器均相同；参见指南：Script Commands（脚本命令）。

组件 / 生物圈（BIOSPHERE）

Essence Liquifier（精华液化器）

将本地生命体样本还原为其生物群系对应的精华液。它只接受来自所属前哨站生物群系的生命体，并产出该生物群系的精华。

项目	内容
类型	Biosphere（生物圈）
输入功率	可变（从电网取电）
产出	一个与生物群系匹配的端口：Frozen Essence（冰冻精华）、Coastal Essence（海岸精华）、Geothermal Essence（地热精华）、Volcanic Essence（火山精华）或 Deep Essence（深渊精华）；缓冲区 50 t
输入缓冲区	50 单位

获取方式

- 需要 Biosphere（生物圈）研究（Terraform Index（地形改造指数）210,000）。
- 在商店购买，价格为 35,000 cr。

返回者：`self/get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ① 以查找确切 ID，然后将该值传递给 `get_component(id)`。ID 区分大小写。

属性

`.id`

该组件稳定的程序化标识符。将其用于 `get_component(id)`，以及要求组件、星球、载具、空间站或订单 ID 的 API。

返回值：`String`

`.name`

面向用户的可读显示名称。需要脚本在重命名后仍能工作的场景中，优先使用 `.id`。

返回值：`String`

方法

`.outpost`

部署该建筑的前哨站。返回的 `OutpostRef`（前哨站引用）包含其稳定 ID、显示名称、生物群系、位置、容量以及 `buildings()` 查询。需要当前值时再次读取此属性。

返回值：部署该建筑的前哨站的 `OutpostRef`。

`.input`

用于输入该前哨站生物群系本地生命体样本的输入槽。机器和储存来源必须属于同一前哨站。库存仅在本地有效；远程 `Liquifier`（液化器）从本地储存接收样本。错误生物群系的样本会被拒绝且不会移动。

返回值：用于输入该前哨站生物群系本地生命体样本的 `InputSlot`。错误生物群系的样本会被拒绝且不会移动；`eject(...)` 可将暂存样本恢复到明确指定的本地目的地。

`.essence_rate()`

当前生物群系精华产出，单位为 t/h。在前哨站效率为 100% 时，`Essence Liquifier`（精华液化器）的生命体基础输入速率为 1 t/h。该读数同时应用前哨站效率和已装载生命体的稀有度产率（普通 $\times 5$ 、不常见 $\times 10$ 、稀有 $\times 25$ ）；当机器无法运行或无法为下一次完整产出腾出缓冲空间时，读数为 0。

返回值：`Number`，当前生物群系精华产出，单位为 t/h。

`.yield_multiplier()`

当前输入箱中生命体每吨产生的未缩放精华吨数：普通生命体为 5，不常见生命体为 10，稀有生命体为 25。输入为空时返回 0。在前哨站效率为 100% 时，只要机器已通电、有输入且能为下一次完整产出腾出缓冲空间，`essence_rate()` 就等于此乘数；过度拥挤可能降低实际产出速率。

返回值：Number：当前装载生命体每吨产生的精华吨数（普通 5 / 不常见 10 / 稀有 25）；输入为空时返回 0。

`.is_stalled()`

只要 `stall_reason()` 不为 "ok" 就返回

True：没有有效的宿主生物群系、没有输入，或下一次按稀有度缩放的完整产出没有足够空间。使用 `stall_reason()` 获取具体原因。

返回值：Boolean。只要 `stall_reason()` 不为 "ok" 就返回 True。

`.stall_reason()`

以可用于分支判断的字符串说明 Liquifier（液化器）为何空闲："no_biome"（机器没有有效的宿主前哨站）、"no_input"（输入箱为空，请输入本地生物群系的生命体）、"output_full"（下一次完整的按稀有度缩放产出无法装入，且当前配置的输出无法排出）、"unconnected"（下一次完整产出无法装入，且精华输出没有有效的对等连接关系），或 "ok"（正在运行、已准备就绪，或能够缓冲下一次完整产出）。它比 `is_stalled()` 提供更具体的信息。

返回值 / 可能的值：String，取值为

"ok"、"no_biome"、"no_input"、"output_full"、"unconnected"；表示液化器空闲的具体原因。

`.biome()`

返回宿主前哨站所在的生物群系："frozen"、"coastal"、"geothermal"、"volcanic" 或 "deep"。机器没有有效前哨站时返回 `None`。这决定 Essence Liquefier（精华液化器）接受哪类生命体物品；错误生物群系的物品会被输入端口拒绝。

返回值 / 可能的值：`String`，前哨站所在的生物群系："frozen"、"coastal"、"geothermal"、"volcanic" 或 "deep"。机器没有有效前哨站时为 `None`。这决定液化器接受的生命体。

`.frozen_essence_out`

该机器恰好有一个以所属前哨站生物群系命名的精华输出：`frozen_essence_out`、`coastal_essence_out`、`geothermal_essence_out`、`volcanic_essence_out` 或 `deep_essence_out`。它是 `FluidPort`（流体端口）。将其连接到本地 `Liquid Tank`（液体罐）或匹配的 `Mixer`（混合器）输入。远程对等连接还需要一条已完成且无冲突的 `Liquid Pipe`（液体管道）路径，连接两个位置。例如：`self.frozen_essence_out.connect("liquid_tank_1")`。

返回值：`Frozen essence`（冰冻精华）的 `FluidPort`。液化器只暴露与所属前哨站生物群系匹配的端口。调用 `self.biome()` 确认端口，然后像连接任何流体源一样连接：`self.frozen_essence_out.connect("liquid_tank_1")`。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` 仅限自身

读取脚本运行期间从编辑器 `Commands`（命令）选项卡发送的命令。所有可脚本化机器均相同；参见指南：`Script Commands`（脚本命令）。

组件 / 生物圈（BIOSPHERE）

Grow Lamp（生长灯）

在通电且启用时，为四个正交相邻的田地单元（正上方、正下方、左侧和右侧）提供照明。

项目	内容
类型	Biosphere（生物圈）
输入功率	可变（从电网取电）
等级	Mk II、Mk III 和 Mk IV

获取方式

- Grow Lamp（生长灯）研究解锁配方（Plants（植物）500,000）。
- 在 Fabricator（制造机）上制造 Grow Lamp Kit（生长灯套件）：1× Machine Frame（机器框架）、2× Circuit Panel（电路面板）和 2× Glass（玻璃）。
- 使用 Harvester（收割机）的 `deploy()` 将套件部署到空的田地单元。

返回者：`self / get_component(id)`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ① 以查找确切 ID，然后将该值传递给 `get_component(id)`。ID 区分大小写。

属性

`.id`

该组件稳定的程序化标识符。将其用于 `get_component(id)`，以及要求组件、星球、载具、空间站或订单 ID 的 API。

返回值：`String`

`.name`

面向用户的可读显示名称。需要脚本在重命名后仍能工作的场景中，优先使用 `.id`。

返回值：`String`

方法

`.set_enabled(enabled)` 仅限自身

命令生长灯开启或关闭。断电会暂停脚本，但保留此设定值；停止机器脚本会将其重置为 `False`。

参数

名称	类型	描述
<code>enabled</code>	<code>boolean</code>	此脚本是否命令生长灯照亮其单元。

返回值：`ActionResult`（操作结果）

结果字段：`.status`、`.message`

成功载荷：`None`

结果

状态	类型	含义
<code>"ok"</code>	<code>success</code> （成功）	操作成功完成。

`.is_enabled()`

当运行中的脚本已命令此生长灯开启时返回 `True`。

返回值：`boolean`

`.is_active()`

当生长灯已被命令开启且有电时返回 `True`。

返回值：`boolean`

`.is_supplied()`

当生长灯已被命令开启、已通电且正主动照亮其覆盖的单元时返回 **True**。若被禁用或断电，覆盖的单元会失去 **lit** 状态，其植物会暂停。

返回值

Boolean：当生长灯已部署、被命令开启、已通电且正主动照亮其覆盖的单元时返回 **True**。当未部署、被禁用或断电时返回 **False**；此时覆盖的单元会失去 **lit** 状态，其植物会暂停。

`.status()`

精确运行状态："not_placed"、"disabled"、"no_power" 或 "active"。

返回值 / 可能的值：部署恢复后为 "not_placed"；脚本命令关闭时为 "disabled"；已命令开启但无电时为 "no_power"；照明时为 "active"。

可能的值："not_placed"、"disabled"、"no_power"、"active"。

`.tier()`

已部署等级（1-4）。Mk I/II/III/IV 分别提供植物产出的 1×/2×/4×/8×，在活动时消耗 5/25/100/500 W。

返回值：Number（1-4），已部署等级。等级越高，覆盖植物的产出提升越多，耗电也显著增加；通过制造并应用 Grow Lamp upgrade pack（生长灯升级包）来升级。

`.position()`

该生长灯占据的网格扇区，例如 "E14"。

返回值：String：该生长灯占据的网格扇区（例如 "E14"）。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` 仅限自身

读取脚本运行期间从编辑器 Commands（命令）选项卡发送的命令。所有可脚本化机器均相同；参见指南：Script Commands（脚本命令）。

组件 / 生物圈（BIOSPHERE）

Plant Terraformer（植物地形改造器）

唯一能将收获的实体 Forage（饲料）转换为永久 Plants（植物）面积（km²）的转换器。通过普通的定时物品传输装载输入；其大容量进料器在 Mk I 时每步处理 16 件物品，在 Mk II 时每步处理 80 件。每个阶段依次加入 Water（水）、Salt（盐）、Fertilizer（肥料），然后加入 Growth Accelerant（生长促进剂）。Mk I 在 Fields（田地）阈值处停止；Mk II 承担最后两个阶段。

项目	内容
类型	Biosphere（生物圈）
输入功率	可变（从电网取电）
消耗	Water（水），缓冲区 60 t
储备	1,243 单位（混合）
等级	Mk II

获取方式

- Plant Terraformer（植物地形改造器）研究解锁配方（Biomass（生物质）2,000）。
- 在 Fabricator（制造机）上制造 Plant Terraformer Kit（植物地形改造器套件）：5× Machine Frame（机器框架）、3× Control Unit（控制单元）、5× Circuit Panel（电路面板）、4× Liquid Pipe Segment（液体管道段）和 10 t Water（水）。
- 从 Inventory（库存）中部署。

返回者：`self`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ① 以查找确切 ID，然后将该值传递给 `get_component(id)`。ID 区分大小写。

属性

`.id`

该组件稳定的程序化标识符。将其用于 `get_component(id)`，以及要求组件、星球、载具、空间站或订单 ID 的 API。

返回值：`String`

.name

面向用户的可读显示名称。需要脚本在重命名后仍能工作的场景中， 优先使用 .id。

返回值：String

方法

.outpost

部署该建筑的前哨站。返回的 OutpostRef 包含其稳定 ID、显示名称、生物群系、位置、容量以及 buildings() 查询。需要当前值时再次读取此属性。

返回值：部署该建筑的前哨站的 OutpostRef。

.set_enabled(enabled) 仅限自身

启用或暂停转换。True 会在板载物品容器和 Water（水）能够供应至少一个按比例的对age（饲料）单位时启动一个周期。在前哨站效率为 100% 时， 一个周期运行 3 小时。停止该机器的脚本会将设定值重置为 False；正在进行的批次仍保持装载状态。

参数

名称	类型	描述
enabled	boolean	True：将已通过 input 装载的材料转换；False：暂停机器。

返回值：ActionResult

结果字段：.status、.message

成功载荷：None

结果

状态	类型	含义
"ok"	success（成功）	操作成功完成。

`.is_enabled()`

当当前机器脚本已命令转换开启时返回 `True`。

返回值：`boolean`

`.tier()`

永久安装的 Plant Terraformer（植物地形改造器）等级，为整数（1-2）。Mk II 提高批次吞吐量，并启用后期 Plants（植物）配方。

返回值：`Integer`：永久安装的植物地形改造器等级（1-2）。

`.status()`

精确实时状态：`"complete"`、`"disabled"`、`"no_power"`、`"needs_mk2"`、`"no_forage"`、`"no_water"`、`"no_salt"`、`"no_fertilizer"`、`"no_accelerant"` 或 `"running"`。

返回值/可能的值：`"complete"`、`"disabled"`、`"no_power"`、`"needs_mk2"`、`"no_forage"`、`"no_water"`、`"no_salt"`、`"no_fertilizer"`、`"no_accelerant"`、`"running"`。

`.is_running()`

转换周期运行期间返回 `True`。未通电或被禁用的机器会保留正在进行的批次，但在恢复前返回 `False`。

返回值：`Boolean`

`.get_progress()`

当前转换周期的完成度（0-1）。在前哨站效率为 100% 时，一个完整周期需要工作 3 小时；过度拥挤会按比例减慢进度。未装载批次时始终返回 0。

返回值：Number（0-1）

`.batch_size()`

正在运行周期中的 Forage（饲料）完整物品数，或当前可以装载的批次大小。Mk I 的完整批次为 1,200，Mk II 为 6,600。材料有限或接近阶段边界时，装载量会更少。

返回值：已提交到运行周期的 Forage（饲料）完整物品数，或下一周期将装载的批次大小。在停滞、禁用或完成时返回 0。

`.km2_rate()`

该机器当前的永久 Plants（植物）产出，单位为 km^2/h 。它结合已装载批次、名义 3 小时工作周期、固定的阶段交换速率（早期为每个 Forage（饲料） 20 km^2 ，后期为每 3 个 Forage（饲料） 1 km^2 ），以及该前哨站的过度拥挤效率。受阻、禁用或完成时返回 0。

返回值：以 km^2/h 为单位的当前转换速率，即正在运行批次在其周期内的平均产出。停滞、禁用或完成时返回 0。

`.phase()`

当前全局 Plants（植物）阶段编号，范围为 1-6。

返回值：当前全局植物阶段索引，范围为 1 到 6。

`.recipe_tier()`

当前累计 Plants（植物）转换配方的派生产出等级。完成 Continental（大陆）阶段后返回 **None**。

返回值：当前植物转换的派生产出等级；完成后为 **None**。

`.next_threshold()`

进入下一阶段所需的永久 Plants（植物）面积。完成时返回 5,000,000 km² 的上限。

返回值：下一阶段所需的植物面积；完成后为已完成的 5,000,000 km² 上限。

`.remaining()`

下一阶段仍需的永久 Plants（植物）面积。Continental（大陆）阶段完成时返回 0。

返回值：距离下一阶段还需的植物面积；完成时返回 0。

`.required_inputs()`

当前累计材料 ID。起始为 `forage`，随后在五次转换中依次加入 `water`、`salt`、肥料类别和 `growth_accelerant`。

返回值：当前累计阶段配方所需材料 ID 的列表。

`.batch_requirements()`

当前等级和阶段允许的最大下一批次的精确数量。字典按需使用 `forage`、`water`、`salt`、`fertilizer_potency` 和 `growth_accelerant`。`salt` 和 `Growth`

`Accelerant`（生长促进剂）按每批次向上取整为完整物品数量。肥料效力为整数；板载储备不足时，该数值不会缩小。

返回值：当前等级和阶段允许的最大下一批次的材料数量字典。`Salt`（盐）和 `Growth`

`Accelerant`（生长促进剂）为完整物品数量，并按批次向上取整。肥料按整数 `fertilizer_potency` 报告。

`.fertilizer_potency(item_id)`

返回一个 Fertilizer（肥料）物品的整数效力：Mk I 为 **10**，Mk II 为 **30**，Mk III 为 **50**。任何其他物品 ID 都会引发 **ValueError**。

参数

名称	类型	描述
<code>item_id</code>	<code>string</code>	一个 Fertilizer（肥料）Mk I、Mk II 或 Mk III 物品 ID。

返回值：一个肥料物品的效力：Mk I 为 **10**，Mk II 为 **30**，Mk III 为 **50**。

引发异常

异常	条件
<code>ValueError</code>	物品 ID 必须标识 Fertilizer（肥料）Mk I、Mk II 或 Mk III。

`.input`

用于 Forage（饲料）、Salt（盐）、Fertilizer（肥料）和 Growth Accelerant（生长促进剂）的标准定时 **InputSlot**。连接 Inventory（库存）、本地 Storage Bin（储存箱）或 Warehouse（仓库），也可连接本地机器输出，例如完成 Auto Feeders（自动进料器）研究后的 Crop Automator（作物自动机）。目标进料器在 Mk I 时每个步骤处理 16 件物品，在 Mk II 时处理 80 件，两个传输方向均适用。其容器可容纳一个完整 Forage（饲料）批次、Salt（盐）以及每种辅助物品最多 10 件。

返回值：用于将物品定时传输到地形改造器单周期材料容器中的 **InputSlot**。Mk I 每个普通进料器量子处理 16 件物品；Mk II 处理 80 件。

`.water_in`

容量足以容纳一个完整周期的 Water FluidPort（水流体端口）：Mk I 为 60 t，Mk II 为 330

t。将其连接到水源；远程水源还需要一条已完成且无冲突的 Liquid Pipe（液体管道）路径，连接两个位置。从 Seedlings（幼苗）阶段开始就需要水。

返回值：用于 Water（水）的 **FluidPort**。从 Seedlings（幼苗）阶段开始，水成为必需品。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` 仅限自身

读取脚本运行期间从编辑器 Commands（命令）选项卡发送的命令。所有可脚本化机器均相同；参见指南：Script Commands（脚本命令）。

组件 / 生物圈 (BIOSPHERE)

Seed Maker (种子制造机)

一种前哨站加工建筑，将三个生命体样本组合成可存活的种子。大多数混合会失败；有效配方因星球而异，需要通过试验发现；发现后可以再次运行以获得更多种子。

项目	内容
类型	Biosphere (生物圈)
输入功率	可变 (从电网取电)
输出缓冲区	1 单位
储备	3 单位 (混合)

获取方式

- Seed Maker (种子制造机) 研究解锁配方 (Biomass (生物质) 500)。
- 在 Fabricator (制造机) 上制造 Seed Maker Kit (种子制造机套件)：2× Machine Frame (机器框架)、1× Control Unit (控制单元)、2× Circuit Panel (电路面板) 和 3 t Water (水)。
- 从 Inventory (库存) 中部署。

返回者：`self`

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ① 以查找确切 ID，然后将该值传递给 `get_component(id)`。ID 区分大小写。

属性

`.id`

该组件稳定的程序化标识符。将其用于 `get_component(id)`，以及要求组件、星球、载具、空间站或订单 ID 的 API。

返回值：`String`

.name

人类可读的显示名称。对于需要经受重命名的脚本，优先使用 **.id**。

返回值

String

方法

.outpost

部署此建筑的前哨站（Outpost）。返回的 **OutpostRef** 包含其稳定 ID、显示名称、生物群系、位置、容量以及 **buildings()** 查询结果。需要当前值时，请重新读取该属性。

返回值

部署此建筑的前哨站的 **OutpostRef**。

.combine(blend) — 仅自身 (SELF ONLY)

运行反应室中装载的、恰好三个不同生命形态 ID 的组合。每次被接受的试验都会消耗每种生命形态各 1 t。每次试验开始前，单种子结果舱必须为空。

参数 (PARAMETERS)

名称	类型	描述
blend	any	恰好包含 3 个不同生命形态物品 ID 的列表，例如 ["ice_algae", "sea_algae", "vent_algae"]。顺序无关；重复项或任何其他长度都会引发 ValueError。

返回值 (RETURNS)

SeedResult；结果字段 (RESULT FIELDS)：.status、.message；成功载荷 (SUCCESS PAYLOAD)：.seed_id、.species。

结果 (OUTCOMES)

状态	类型	含义
"seed_found"	成功	发现 .species，并将 .seed_id 存入制种机 (Seed Maker) 输出端。
"sludge"	成功	被接受的三种生命形态组合未产生种子。
"locked"	拒绝	制种机研究尚未解锁。
"busy"	临时状态	制种机组合试验已经在进行中。
"missing_life_forms"	拒绝	制种机反应室没有恰好包含所请求的三份、各 1 t 的生命形态样本。
"output_full"	拒绝	制种机结果舱已经包含其唯一的实体种子。

引发异常 (RAISES)

异常	条件
ValueError	组合必须恰好包含三个不同且已知的生命形态 ID。

`.life_forms()`

列出 30 个已接受的生命形态物品 ID。对于来自 `combinations(self.life_forms(), 3)` 的每个组合，使用 `self.input.take(item_id, 1)` 装载三个物品，然后调用 `self.combine(blend)`。枚举不会移动材料。继续下一次试验前，将任何生成的实体种子通过 `self.output` 发送出去。

返回值 (RETURNS)

包含 30 个已接受生命形态物品 ID 的列表。对于来自 `combinations(self.life_forms(), 3)` 的每个组合，使用 `self.input.take(item_id, 1)` 装载三个物品，然后调用 `self.combine(blend)`。枚举不会移动材料；下一次试验前，使用 `self.output.send(...)` 发送生成的实体种子。

`.is_running()`

组合试验正在执行时为 `True`。

返回值

布尔值：组合试验正在执行时为 `True`。

`.get_progress()`

返回当前组合试验的进度，范围为 0-1；空闲时返回 0。

返回值

数值 (0-1)：当前试验的进度；空闲时为 0。

715 / 1,261

`.get_output_count()`

单个结果舱中等待的实体种子数量：0 或 1。

返回值

数值（0 或 1）：表示结果舱中是否有实体种子等待。

`.recipes()`

列出截至目前发现的每个 **SeedRecipe**，与 Flora / Seed Recipes 选项卡显示的“发现一次、永久保留”日志相同。每项包含 `.tier`、实体 `.seed_id`、不带修饰的 `.species`、`.blend`、`.requirements`、`.requirement` 和 `.growth_time`。`.requirements` 是可编程形式：每个 **PlantRequirement** 都有 `.kind` 以及可选的 `.species`，因此 companion（伴生）和 antagonist（拮抗）条目会标识确切的相关植物。`.requirement` 仍是紧凑的字符串摘要。首次命中前为空；使用 `self.combine(...)` 重新运行已知的 `.blend`，即可复现该种子，无需再次遍历。

返回值

所有已发现的 **SeedRecipe** 列表；首次发现前为空。每个配方包含其物种、三物品组合、培育要求和生长时间。使用 `self.combine(recipe.blend)` 复现其种子。

.input

用于装载制种机三样本反应室的 **InputSlot**。自然来源是本地无人机仓（Drone Depot），生物无人机会在此卸货；本地仓库（Warehouse）、储物箱（Storage Bin）或家庭库存（home Inventory）同样有效。反应室恰好接受三种不同生命形态各 1t，不能囤积多余材料。

返回值

三吨反应室的 **InputSlot**。它接受三种不同生命形态各一个单位，并拒绝重复项或第四种样本。**combine()** 运行时会锁定这组三样本。空闲时，可使用 **eject(...)** 取回误装的物品，或使用 **flush()** 将其销毁。

.output

包含成功试验生成的唯一实体种子的 **OutputSlot**。将其发送到家庭库存，然后在本地网格任意位置调用 Harvester（收割机）的 **load_seed()**，或将其传送到 Crop Automator（作物自动化机）。在结果舱为空前，制种机无法开始下一次试验。**sludge** 不会生成物品。

返回值

存放唯一实体结果种子的 **OutputSlot**。将其发送到家庭库存，再使用 **load_seed()** 转移到收割机；其他本地存储位置也有效。成功试验会在此留下恰好 1 个特定物种的种子；在 **send(...)** 将其移除前，不能开始进一步试验。**sludge** 结果不会产生输出物品。

717 / 1,261

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` —
仅自身 (SELF ONLY)

读取脚本运行期间从编辑器 Commands 选项卡发送的命令。所有可编程机器上的行为相同，参见指南：Script Commands。

718 / 1,261

组件 / 生物圈 (COMPONENTS / BIOSPHERE)

喷水器 (Sprinkler)

在通电、有供应且启用时，为上下左右四个正交相邻的田地格浇水。

类型	生物圈 (Biosphere)
输入电力	可变 (从电网获取)
消耗	水, 缓冲容量 10 t
等级	Mk II、Mk III 和 Mk IV

获取方式 (HOW TO OBTAIN)

- 通过喷水器研究 (Plants 100,000) 解锁配方。
- 在 Fabricator (制造机) 上制造 Sprinkler Kit : 1× Machine Frame、2× Liquid Pipe Segment、1× Pressure Valve 和 2 t Water。
- 使用 Harvester 的 `deploy()` 将套件部署在空田地格上。

返回方式：`self` / `get_component(id)`。

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ① 以找到确切 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性 (PROPERTIES)

`.id`

此组件的稳定程序化标识符。将其用于 `get_component(id)`，以及所有要求组件、行星、载具、空间站或订单 ID 的 API。

返回值：`String`

`.name`

人类可读的显示名称。对于需要经受重命名的脚本，优先使用 `.id`。

返回值：`String`

方法 (METHODS)

`.set_enabled(enabled)` — 仅自身 (SELF ONLY)

命令喷水开启或关闭。断电会暂停脚本，但保留此设定值；停止机器脚本会将其重置为 `False`。

参数 (PARAMETERS)

名称	类型	描述
<code>enabled</code>	<code>boolean</code>	此脚本是否命令浇水。

返回值 (RETURNS) : `ActionResult` ; 结果字段 : `.status`、`.message` ; 成功载荷 : 无。

结果 (OUTCOMES)

状态	类型	含义
<code>"ok"</code>	成功	操作成功完成。

`.is_enabled()`

运行中的脚本已命令开启浇水时为 `True`。

返回值 : `boolean`

`.is_active()`

已命令开启且有电、有水时为 `True`。

返回值 : `boolean`

`.is_supplied()`

当喷水器已被命令开启、已通电且 `water_in` 缓冲区中有水时为 `True`。如果禁用、断电或缺水，被覆盖的格子会失去 `watered` 状态。

返回值

布尔值：喷水器已放置、已命令开启、已通电且 `water_in` 缓冲区有水时为 `True`。未放置、禁用、断电或干涸时为 `False`；被覆盖的格子随后会失去 `watered` 状态，格中的植物会暂停。

`.status()`

精确运行状态：`"not_placed"`、`"disabled"`、`"no_power"`、`"no_water"` 或 `"active"`。

返回值：精确运行状态，可能值为上述五项。

`.buffer()`

当前机载水缓冲区的填充比例（0-1）。浇水时下降，并从连接的 `water_in` 来源补充。

返回值

数值（0-1）：当前机载水缓冲区的填充比例。喷水器浇水时下降，由连接的 `water_in` 流源补充。0 表示干涸（被覆盖格子会失去 `watered`）。

`.tier()`

已部署等级（1-4）。Mk I/II/III/IV 分别提供 $1 \times 2 \times 4 \times 8 \times$ 的受支持植物产出，消耗 5/25/100/500 W，并在激活时每小时消耗 2/10/200/1,000 t Water。通过制造并应用喷水器升级包来提升等级。

返回值

数值（1-4），即已部署等级。更高等级会提升其覆盖植物的产出，同时消耗更多水和电力；制造并应用 Sprinkler upgrade pack（喷水器升级包）即可升级。

`.position()`

此喷水器占据的网格扇区，例如 "E14"。

返回值：字符串：此喷水器占据的网格扇区（例如 "E14"）。

`.water_in`

从连接的来源供应水。使用来源的稳定机器 ID 或显示名称调用 `self.water_in.connect(...)`。远程来源还需要在两个位置之间存在已完成且无冲突的 Liquid Pipe（液体管道）路线。有关液位、容量、流量和连接查询，请参见 `FluidPort`。

返回值

水流体端口缓冲区。使用提供方的稳定机器 ID 或显示名称调用 `connect(...)`；已完成的液体管道网络会在前哨站之间输送水。与管道或水箱共享前哨站并不会自动连接。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` —
仅自身 (SELF ONLY)

读取脚本运行期间从编辑器 Commands 选项卡发送的命令。所有可编程机器上的行为相同，参见指南：Script Commands。

723 / 1,261

组件 / 存储与库存 (COMPONENTS / STORAGE & INVENTORY)

物品目录 (Item Catalog)

查询任何已知物品 ID 的静态身份元数据。当脚本需要对物品分类、又不想维护自己的数据档案时，使用 `get_component("item_catalog")`。

返回方式：`get_component("item_catalog")`。

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ① 以找到确切 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性 (PROPERTIES)

`.id`

此组件的稳定程序化标识符。将其用于 `get_component(id)`，以及所有要求组件、行星、载具、空间站或订单 ID 的 API。

返回值：`String`

`.name`

人类可读的显示名称。对于需要经受重命名的脚本，优先使用 `.id`。

返回值：`String`

方法 (METHODS)

`.lookup(item_id)`

返回包含 `.id`、`.name`、`.category`、`.stackable`、`.biome`、`.rarity` 和 `.production_tier` 的 `ItemInfo`。分类包括 "mineral" (矿物)、"refined" (精炼品)、"crafted" (制造品)、"agriculture" (农业)、"life_form" (生命形态)、"field_resource" (田地资源)、"biology_sample" (生物样本)、"reagent" (试剂)、"equipment" (设备)、"module" (模块)、"portable" (便携物)、"upgrade_pack" (升级包) 和 "construction_kit" (建造套件)。来源物品的生产等级为 `None`；不适用时，生物群系和稀有度为 `None`。未知物品 ID 返回 `None`。

参数 (PARAMETERS)

名称	类型	描述
<code>item_id</code>	<code>string</code>	要识别的物品 ID

返回值 (RETURNS)

带有身份元数据的 `ItemInfo`；未知物品 ID 返回 `None`。

```
catalog = get_component("item_catalog")
item = catalog.lookup("water_pump")
if item is not None:
    print(item.category, item.production_tier)
```

组件 / 天气与天空 (COMPONENTS / WEATHER & SKY)

气象站 (Weather Station)

可编程的本地风暴站和实时接收器。它测量不可变的本地报告，接收广播频道和生物群系频道上的事件传输，并可向信号板 (Signal Board) 发布内容。

类型	传感器 (Sensors)
输入电力	-50 W (从电网获取)

获取方式 (HOW TO OBTAIN)

- 需要 Weather Program (天气程序) 研究 (Terraform Index 330,000)。
- 从商店购买，价格为 60,000 cr。

返回方式：`self / get_component(id)`。

每个组件都有稳定的 `.id`。对于已部署的机器，打开其卡片上的 ① 以找到确切 ID，然后将该值传给 `get_component(id)`。ID 区分大小写。

属性 (PROPERTIES)

`.id`

此组件的稳定程序化标识符。将其用于 `get_component(id)`，以及所有要求组件、行星、载具、空间站或订单 ID 的 API。

返回值：`String`

`.name`

人类可读的显示名称。对于需要经受重命名的脚本，优先使用 `.id`。

返回值：`String`

方法 (METHODS)

.outpost

部署此建筑的前哨站。返回的 `OutpostRef` 包含其稳定 ID、显示名称、生物群系、位置、容量以及 `buildings()` 查询结果。需要当前值时，请重新读取该属性。

返回值：部署此建筑的前哨站的 `OutpostRef`。

.observe() — 仅自身 (SELF ONLY)

测量一份不可变的本地报告。气象站每个世界时钟小时最多刷新一次；更快的调用会返回相同的报告 ID。报告包括本地覆盖范围、活动风暴快照，以及未来 8 个世界时钟小时的本地预报；研究 Weather Forecasting（天气预报）后可延伸至 24 小时。报告绝不包含事后坐标。

返回值

来自此站本地覆盖范围的不可变 `WeatherReport`。在同一观测小时内重复调用会返回相同报告。若气象站尚未完工或未通电，则引发 `RuntimeError`。

引发异常 (RAISES)

异常	条件
<code>RuntimeError</code>	气象站尚未完工或未通电，因此无法记录观测。

`.last_report()`

读取最近测量的报告，不会进行新的观测。这对启动恢复和处理过期数据很有用；**None** 表示此站从未完成 `observe()`。

返回值

最近的本地 **WeatherReport**；首次成功观测前为 **None**。即使数据过期或设备断电，快照仍可读取。

`.signal_board`

此站可编程的 Weather 显示板。`reveal()` 将传输发布到编号槽位，`reject()` 统计被拒绝的副本，`resolve()` 发布带标签的值。显示板只显示脚本提供的内容，不对其进行解释。

返回值

此站可选的玩家自定义显示板。它显示所提供的传输和带标签的值，不对其进行解释。

`.signal_receiver`

实时接收器。`transmissions()`

只返回当前此通电站可听见的原始副本，不保存历史。尘埃接收使用本地生物群系频道；雷声接收使用广播频道。

返回值

此站的实时接收器。`transmissions()`

只公开当前可听见的内容，不保存原始历史。尘埃事件频道按生物群系区分；所有雷声数据包均广播。

`.strikes()`

返回此站有限的雷击历史。每个 `WeatherStrike` 包含事件 ID、观测时间、能量，以及是否由 Lightning Rod（避雷针）存储。不会包含确切的雷击位置和 Storm Glass（风暴玻璃）资格。只返回此站实际观测到的雷击。

返回值

此站观测到的雷击，按从旧到新的顺序排列。

`.peek_command()` · `.next_command()` · `.command_count()` · `.clear_commands()` — 仅自身 (SELF ONLY)

读取脚本运行期间从编辑器 Commands 选项卡发送的命令。所有可编程机器上的行为相同，参见指南：Script Commands。

729 / 1,261

数据库 / 生产 (DATABASE / PRODUCTION)

生产等级 (Production Tiers)

按依赖深度排列的所有制造和转化配方。冶炼炉基础配方从等级 0 开始。其他所有配方至少为等级 1，并位于其最深制造输入的上一等级；因此，只消耗来源材料的配方会进入等级 1。已发现的制种机和生物实验室配方会加入图谱，但不会暴露尚未发现的组合或样本。

730 / 1,261

数据库 / 配方 (DATABASE / RECIPES)

冶炼炉配方 (Smelter Recipes)

冶炼炉将原矿精炼为库存材料：输入一份矿石，输出一份精炼单位。使用 `smelter.set_recipe` 选择蓝图，并保持输入缓冲区有料。此处从第一天起列出每张蓝图，无论是否已解锁。

Iron Ore → Iron Ingot `smelt_iron_ingot` — 已解锁 (UNLOCKED)

1× Iron Ore → 1× Iron Ingot

从游戏开始即可用。

- 制造时间：0.08 h
- 电力：20 W
- 生产等级：T0

Silicon → Glass `smelt_glass` — 尚未解锁 (NOT YET UNLOCKED)

1× Silicon → 1× Glass

完成订单 Spire, Silicon Bootstrap 后解锁蓝图。

- 制造时间：0.08 h
- 电力：20 W
- 生产等级：T0

Titanium → Titanium Ingot `smelt_titanium_ingot` — 尚未解锁 (NOT YET UNLOCKED)

1× Titanium → 1× Titanium Ingot

完成订单 Helios, Iron Bootstrap 后解锁蓝图。

- 制造时间：0.1 h
- 电力：28 W
- 生产等级：T0

731 / 1,261

Cobalt → Cobalt Ingot `smelt_cobalt_ingot` — 尚未解锁 (NOT YET UNLOCKED)

1× Cobalt → 1× Cobalt Ingot

完成订单 Spire, Avionics Run 后解锁蓝图。

- 制造时间：0.1 h
- 电力：28 W
- 生产等级：T0

Rare Earth → Rare Earth Core `smelt_rare_earth_core` — 尚未解锁 (NOT YET UNLOCKED)

1× Rare Earth → 1× Rare Earth Core

完成订单 Spire, Control Run 后解锁蓝图。

- 制造时间：0.14 h
- 电力：35 W
- 生产等级：T0

Neutronium → Neutronium Bar `smelt_neutronium_bar` — 尚未解锁 (NOT YET UNLOCKED)

1× Neutronium → 1× Neutronium Bar

完成订单 Spire, Polymer Optics Run 后解锁蓝图。

- 制造时间：0.2 h
- 电力：45 W
- 生产等级：T0

Lead Ore → Lead Ingot `smelt_lead_ingot` — 尚未解锁 (NOT YET UNLOCKED)

1× Lead Ore → 1× Lead Ingot

完成订单 Helios, Lead Survey Stock 后解锁蓝图。

- 制造时间：0.1 h
- 电力：22 W
- 生产等级：T0

数据库 / 配方

制造器配方 (Fabricator Recipes)

制造器 (Fabricator) 使用精炼库存组装组件；每个蓝图需要若干输入，有些配方还会产生副产品。使用 `fabricator.set_recipe` 选择配方。大多数蓝图通过承包商订单奖励获得；下面每张卡片都会标明对应订单。

铁 (Iron) → 燃气管段 (Gas Pipe Segment) `craft_gas_pipe_segment` - 尚未解锁 (NOT YET UNLOCKED)

2× 铁锭 (Iron Ingot) → 1× 燃气管段 (Gas Pipe Segment)

完成订单 Helios, Iron Production 后解锁蓝图。

- 制作时间 (Craft time) : 0.1 h
- 功率 (Power) : 25 W
- 生产等级 (Production tier) : T1

铁 (Iron) + 硅 (Silicon) → 液体管段 (Liquid Pipe Segment) `craft_liquid_pipe_segment` - 尚未解锁 (NOT YET UNLOCKED)

1× 铁锭 (Iron Ingot) 和 1× 硅 (Silicon) → 1× 液体管段 (Liquid Pipe Segment)

完成订单 Spire, Silicon Stock 后解锁蓝图。

- 制作时间 : 0.12 h
- 功率 : 25 W
- 生产等级 : T1

铁 (Iron) + 钛 (Titanium) → 电力线段 (Power Line Segment) `craft_power_line_segment` - 尚未解锁 (NOT YET UNLOCKED)

1× 铁锭 (Iron Ingot) 和 1× 钛锭 (Titanium Ingot) → 1× 电力线段 (Power Line Segment)

完成订单 Vestibule, Utility Conduit Stock 后解锁蓝图。

- 制作时间 : 0.14 h
- 功率 : 25 W
- 生产等级 : T1

燃气管段 (Gas Segments) + 阀门 (Valve) → 燃气桥 (Gas Bridge) `craft_gas_pipe_bridge` - 尚未解锁 (NOT YET UNLOCKED)

2× 燃气管段 (Gas Pipe Segment) 和 1× 压力阀 (Pressure Valve) → 1× 燃气管桥 (Gas Pipe Bridge)

完成订单 Helios, Iron Production 后解锁蓝图。

- 制作时间：0.18 h
- 功率：25 W
- 生产等级：T2

液体管段 (Liquid Segments) + 阀门 (Valve) → 液体桥 (Liquid Bridge) `craft_liquid_pipe_bridge` - 尚未解锁 (NOT YET UNLOCKED)

2× 液体管段 (Liquid Pipe Segment) 和 1× 压力阀 (Pressure Valve) → 1× 液体管桥 (Liquid Pipe Bridge)

完成订单 Spire, Silicon Stock 后解锁蓝图。

- 制作时间：0.2 h
- 功率：25 W
- 生产等级：T2

电力线段 (Power Segments) + 电路 (Circuit) → 电力桥 (Power Bridge) `craft_power_line_bridge` - 尚未解锁 (NOT YET UNLOCKED)

2× 电力线段 (Power Line Segment) 和 1× 电路面板 (Circuit Panel) → 1× 电力线桥 (Power Line Bridge)

完成订单 Vestibule, Utility Conduit Stock 后解锁蓝图。

- 制作时间：0.22 h
- 功率：25 W
- 生产等级：T2

铁 (Iron) + 玻璃 (Glass) → 压力阀 (Pressure Valve) `craft_pressure_valve` - 尚未解锁 (NOT YET UNLOCKED)

1× 铁锭 (Iron Ingot) 和 1× 玻璃 (Glass) → 1× 压力阀 (Pressure Valve)

完成订单 Spire, Cobalt Run 后解锁蓝图。

- 制作时间：0.12 h
- 功率：25 W
- 生产等级：T1

铁 (Iron) + 钛 (Titanium) → 机械框架 (Machine Frame) `craft_machine_frame` - 尚未解锁 (NOT YET UNLOCKED)

3× 铁锭 (Iron Ingot)、1× 钛锭 (Titanium Ingot) 和 2 t 水 (Water) → 1× 机械框架 (Machine Frame)

完成订单 Helios, Battery Order 后解锁蓝图。

- 制作时间：0.22 h
- 功率：35 W
- 生产等级：T1

铁 (Iron) + 玻璃 (Glass) → 电路面板 (Circuit Panel) `craft_circuit_panel` - 尚未解锁 (NOT YET UNLOCKED)

1× 铁锭 (Iron Ingot)、2× 玻璃 (Glass) 和 1 t 水 (Water) → 1× 电路面板 (Circuit Panel)

完成订单 Spire, Optical Glass 后解锁蓝图。

- 制作时间：0.18 h
- 功率：30 W
- 生产等级：T1

面板 (Panel) + 钛 (Titanium) + 玻璃 (Glass) → 控制单元 (Control Unit) `craft_control_unit` - 尚未解锁 (NOT YET UNLOCKED)

1× 电路面板 (Circuit Panel)、1× 钛锭 (Titanium Ingot)、1× 玻璃 (Glass) 和 2 t 水 (Water) → 1× 控制单元 (Control Unit)

完成订单 Spire, Circuit Order 后解锁蓝图。

- 制作时间：0.3 h
- 功率：40 W
- 生产等级：T2

钴 (Cobalt) + 铁 (Iron) + 玻璃 (Glass) → 电池单元 (Battery Cell) `craft_battery_cell` - 尚未解锁 (NOT YET UNLOCKED)

1× 钴锭 (Cobalt Ingot)、1× 铁锭 (Iron Ingot)、1× 玻璃 (Glass) 和 1 t 水 (Water) → 1× 电池单元 (Battery Cell)

完成订单 Helios, Titanium Run 后解锁蓝图。

- 制作时间：0.22 h
- 功率：35 W
- 生产等级：T1

钛 (Titanium) + 管道 (Pipes) → 热帽套件 (Thermal Cap Kit) `craft_thermal_cap_kit` - 尚未解锁 (NOT YET UNLOCKED)

2× 钛锭 (Titanium Ingot) 和 2× 燃气管段 (Gas Pipe Segment) → 1× 热帽套件 (Thermal Cap Kit)

完成热帽研究 (Thermal Cap research) 后解锁蓝图。

- 制作时间：0.28 h
- 功率：38 W
- 生产等级：T2

钛 (Titanium) + 钴 (Cobalt) + 稀土 (Rare Earth) → 涡轮转子 (Turbine Rotor) `craft_turbine_rotor` - 尚未解锁 (NOT YET UNLOCKED)

2× 钛锭 (Titanium Ingot)、1× 钴锭 (Cobalt Ingot)、1× 稀土核心 (Rare Earth Core) 和 4t 蒸汽 (Steam) → 1× 涡轮转子 (Turbine Rotor)

完成订单 Helios, Cap Kit Order 后解锁蓝图。

- 制作时间：0.34 h
- 功率：42 W
- 生产等级：T1

铁 (Iron) + 钛 (Titanium) + 玻璃 (Glass) → 储罐内衬 (Tank Lining) `craft_tank_lining` - 尚未解锁 (NOT YET UNLOCKED)

2× 铁锭 (Iron Ingot)、1× 钛锭 (Titanium Ingot)、1× 玻璃 (Glass) 和 2t 水 (Water) → 1× 储罐内衬 (Tank Lining)

完成订单 Helios, Bulk Iron Run 后解锁蓝图。

- 制作时间：0.22 h
- 功率：38 W
- 生产等级：T1

铁 (Iron) + 玻璃 (Glass) + 液体管道 (Liquid Pipe) → 水泵 (Water Pump) `craft_water_pump` - 尚未解锁 (NOT YET UNLOCKED)

2× 铁锭 (Iron Ingot)、2× 玻璃 (Glass) 和 4× 液体管段 (Liquid Pipe Segment) → 1× 水泵 (Water Pump)

完成水文调查研究 (Hydrology Survey research) 后解锁蓝图。

- 制作时间：0.3 h
- 功率：35 W
- 生产等级：T2

铁 (Iron) + 钛 (Titanium) + 阀门 (Valve) + 面板 (Panel) → 油泵 (Oil Pump) **craft_oil_pump** - 尚未解锁 (NOT YET UNLOCKED)

2× 铁锭 (Iron Ingot)、1× 钛锭 (Titanium Ingot)、1× 压力阀 (Pressure Valve) 和 1× 电路面板 (Circuit Panel) → 1× 油泵 (Oil Pump)

完成石油调查研究 (Petroleum Survey research) 后解锁蓝图。

- 制作时间：0.38 h
- 功率：40 W
- 生产等级：T2

铁 (Iron) + 石油 (Oil) → 润滑剂 (Lubricant) + 焦油 (Tar) **craft_lubricant** - 尚未解锁 (NOT YET UNLOCKED)

1× 铁锭 (Iron Ingot) 和 3 t 石油 (Oil) → 1× 润滑剂 (Lubricant)，并额外产生副产品 1× 焦油 (Tar)

完成订单 Vestibule, Liquid Pipe Order 后解锁蓝图。

- 制作时间：0.12 h
- 功率：30 W
- 生产等级：T1

玻璃 (Glass) + 石油 (Oil) → 塑料 (Plastic) + 焦油 (Tar) **craft_plastic** - 尚未解锁 (NOT YET UNLOCKED)

1× 玻璃 (Glass) 和 4 t 石油 (Oil) → 1× 塑料 (Plastic)，并额外产生副产品 1× 焦油 (Tar)

完成订单 Spire, Magnetic Stator Build 后解锁蓝图。

- 制作时间：0.14 h
- 功率：32 W
- 生产等级：T1

钴 (Cobalt) + 石油 (Oil) → 橡胶 (Rubber) + 焦油 (Tar) **craft_rubber** - 尚未解锁 (NOT YET UNLOCKED)

1× 钴锭 (Cobalt Ingot) 和 2 t 石油 (Oil) → 1× 橡胶 (Rubber)，并额外产生副产品 1× 焦油 (Tar)

完成订单 Vestibule, Pipe Network Run 后解锁蓝图。

- 制作时间：0.12 h
- 功率：28 W
- 生产等级：T1

重油裂解 (Heavy Oil Cracking) : 石油 (Oil) → 焦油 (Tar) **craft_tar** - 尚未解锁 (NOT YET UNLOCKED)

5 t 石油 (Oil) → 2 × 焦油 (Tar)

完成订单 Vestibule, Polymer Stockpile 后解锁蓝图。

- 制作时间：0.15 h
- 功率：32 W
- 生产等级：T1

塑料 (Plastic) + 植物饲料 (Forage) + 水 (Water) → 强化生物聚合物 (Reinforced Biopolymer)
craft_reinforced_biopolymer - 尚未解锁 (NOT YET UNLOCKED)

2 × 塑料 (Plastic)、6 × 植物饲料 (Plant Forage) 和 2 t 水 (Water) → 4 × 强化生物聚合物 (Reinforced Biopolymer)

完成高级生物聚合物研究 (Advanced Biopolymers research) 后解锁蓝图。

- 制作时间：0.28 h
- 功率：40 W
- 生产等级：T2

生物聚合物 (Biopolymer) + 植物饲料 (Forage) + 水 (Water) → 富集化合物 (Enrichment Compound)
craft_enrichment_compound - 尚未解锁 (NOT YET UNLOCKED)

1 × 强化生物聚合物 (Reinforced Biopolymer)、6 × 植物饲料 (Plant Forage) 和 2 t 水 (Water) → 4 × 富集化合物 (Enrichment Compound)

完成富集化学研究 (Enrichment Chemistry research) 后解锁蓝图。

- 制作时间：0.32 h
- 功率：45 W
- 生产等级：T3

框架 (Frame) + 控制单元 (Control) + 燃气管道 (Gas Pipes) + 液体管道 (Liquid Pipes) → 无人机基地套件 (Drone Depot Kit) **craft_drone_station_kit** - 尚未解锁 (NOT YET UNLOCKED)

2× 机械框架 (Machine Frame)、1× 控制单元 (Control Unit)、2× 燃气管段 (Gas Pipe Segment)、2× 液体管段 (Liquid Pipe Segment) 和 3 t 水 (Water) → 1× 无人机基地套件 (Drone Depot Kit)

完成订单 Helios, Frame Order 后解锁蓝图。

- 制作时间：0.5 h
- 功率：45 W
- 生产等级：T3

框架 (Frame) + 控制单元 (Control) + 面板 (Panel) → 中型无人机基地套件 (Drone Depot Kit (Medium)) **craft_drone_station_kit_medium** - 尚未解锁 (NOT YET UNLOCKED)

4× 机械框架 (Machine Frame)、2× 控制单元 (Control Unit)、2× 电路面板 (Circuit Panel) 和 4 t 水 (Water) → 1× 中型无人机基地套件 (Drone Depot Kit (Medium))

完成订单 Helios, Mid-Cargo Order 后解锁蓝图。

- 制作时间：0.65 h
- 功率：50 W
- 生产等级：T3

框架 (Frame) + 控制单元 (Control) + 面板 (Panel) → 大型无人机基地套件 (Drone Depot Kit (Large)) **craft_drone_station_kit_large** - 尚未解锁 (NOT YET UNLOCKED)

8× 机械框架 (Machine Frame)、3× 控制单元 (Control Unit)、4× 电路面板 (Circuit Panel) 和 6 t 水 (Water) → 1× 大型无人机基地套件 (Drone Depot Kit (Large))

完成订单 Helios, Drone Hub Stockpile 后解锁蓝图。

- 制作时间：0.85 h
- 功率：55 W
- 生产等级：T3

框架 (Frame) + 控制单元 (Control) + 面板 (Panel) + 电池单元 (Cell) + 液体管道 (Liquid Pipe) → 无人机服务站套件 (Drone Service Station Kit) **craft_drone_service_station_kit** - 尚未解锁 (NOT YET UNLOCKED)

2× 机械框架 (Machine Frame)、1× 控制单元 (Control Unit)、2× 电路面板 (Circuit Panel)、1× 电池单元 (Battery Cell) 和 1× 液体管段 (Liquid Pipe Segment) → 1× 无人机服务站套件 (Drone Service Station Kit)

完成订单 Spire, Drone Power Trial 后解锁蓝图。

- 制作时间：0.5 h
- 功率：45 W
- 生产等级：T3

框架 (Frame) + 控制单元 (Control) + 面板 (Panel) → 采矿钻机套件 (Mining Drill Kit) **craft_mining_drill_kit** - 尚未解锁 (NOT YET UNLOCKED)

3× 机械框架 (Machine Frame)、1× 控制单元 (Control Unit)、1× 电路面板 (Circuit Panel) 和 2 t 水 (Water) → 1× 采矿钻机套件 (Mining Drill Kit)

完成订单 Helios, Cargo Pod Run 后解锁蓝图。

- 制作时间：0.45 h
- 功率：45 W
- 生产等级：T3

框架 (Frame) + 控制单元 (Control) + 面板 (Panel) + 转子 (Rotor) → 工业采矿钻机套件 (Industrial Mining Drill Kit) **craft_mining_drill_industrial_kit** - 尚未解锁 (NOT YET UNLOCKED)

5× 机械框架 (Machine Frame)、2× 控制单元 (Control Unit)、2× 电路面板 (Circuit Panel)、1× 涡轮转子 (Turbine Rotor) 和 3 t 水 (Water) → 1× 工业采矿钻机套件 (Industrial Mining Drill Kit)

完成订单 Helios, Heavy Drill Build 后解锁蓝图。

- 制作时间：0.65 h
- 功率：52 W
- 生产等级：T3

框架 (Frame) + 控制单元 (Control) + 面板 (Panel) + 转子 (Rotors) → 重型采矿钻机套件 (Heavy Mining Drill Kit) **craft_mining_drill_heavy_kit** - 尚未解锁 (NOT YET UNLOCKED)

8× 机械框架 (Machine Frame)、3× 控制单元 (Control Unit)、4× 电路面板 (Circuit Panel)、2× 涡轮转子 (Turbine Rotor) 和 4 t 水 (Water) → 1× 重型采矿钻机套件 (Heavy Mining Drill Kit)

完成订单 Helios, Tether Project 后解锁蓝图。

- 制作时间：0.85 h
- 功率：58 W
- 生产等级：T3

稀土 (Rare Earth) + 钛 (Titanium) + 控制单元 (Control) → 小型无人机 (Drone (Small)) **craft_drone_small** - 尚未解锁 (NOT YET UNLOCKED)

1× 稀土核心 (Rare Earth Core)、1× 钛锭 (Titanium Ingot)、1× 控制单元 (Control Unit) 和 2 t 水 (Water) → 1× 小型无人机 (Drone (Small))

完成订单 Helios, Rotor Run 后解锁蓝图。

- 制作时间：0.55 h
- 功率：45 W
- 生产等级：T3

稀土 (Rare Earth) + 钛 (Titanium) + 控制单元 (Control) → 中型无人机 (Drone (Medium)) **craft_drone_medium** - 尚未解锁 (NOT YET UNLOCKED)

2× 稀土核心 (Rare Earth Core)、2× 钛锭 (Titanium Ingot)、1× 控制单元 (Control Unit) 和 3 t 水 (Water) → 1× 中型无人机 (Drone (Medium))

完成订单 Helios, Mid-Drone Build 后解锁蓝图。

- 制作时间：0.75 h
- 功率：50 W
- 生产等级：T3

稀土 (Rare Earth) + 钛 (Titanium) + 控制单元 (Control) → 大型无人机 (Drone (Large))
craft_drone_large - 尚未解锁 (NOT YET UNLOCKED)

3× 稀土核心 (Rare Earth Core)、3× 钛锭 (Titanium Ingot)、2× 控制单元 (Control Unit) 和 4 t 水 (Water) → 1× 大型无人机 (Drone (Large))

完成订单 Helios, Heavy Drone Build 后解锁蓝图。

- 制作时间：1h
- 功率：55 W
- 生产等级：T3

稀土 (Rare Earth) + 转子 (Rotor) → 电推进器 (Electric Thruster) **craft_electric_thruster** - 尚未解锁 (NOT YET UNLOCKED)

2× 稀土核心 (Rare Earth Core)、1× 涡轮转子 (Turbine Rotor) 和 2 t 蒸汽 (Steam) → 1× 电推进器 (Electric Thruster)

完成订单 Spire, Rare Earth Order 后解锁蓝图。

- 制作时间：0.55 h
- 功率：45 W
- 生产等级：T2

稀土 (Rare Earth) + 控制单元 (Control) + 润滑剂 (Lubricant) + 橡胶 (Rubber) → 直升机推进器 (Heli Thruster) **craft_heli_thruster** - 尚未解锁 (NOT YET UNLOCKED)

2× 稀土核心 (Rare Earth Core)、1× 控制单元 (Control Unit)、2× 润滑剂 (Lubricant)、1× 橡胶 (Rubber) 和 2 t 水 (Water) → 1× 直升机推进器 (Heli Thruster)

完成订单 Vestibule, Refueling Pad Order 后解锁蓝图。

- 制作时间：0.65 h
- 功率：50 W
- 生产等级：T3

钛 (Titanium) + 玻璃 (Glass) → 小型货舱 (Cargo Pod (Small)) **craft_cargo_pod_small** - 尚未解锁 (NOT YET UNLOCKED)

1× 钛锭 (Titanium Ingot)、1× 玻璃 (Glass) 和 1t 蒸汽 (Steam) → 1× 小型货舱 (Cargo Pod (Small))

完成订单 Helios, Drone Fleet Order 后解锁蓝图。

- 制作时间：0.2 h
- 功率：35 W
- 生产等级：T1

钛 (Titanium) + 玻璃 (Glass) + 面板 (Panel) → 中型货舱 (Cargo Pod (Medium)) **craft_cargo_pod_medium** - 尚未解锁 (NOT YET UNLOCKED)

2× 钛锭 (Titanium Ingot)、1× 玻璃 (Glass)、1× 电路面板 (Circuit Panel) 和 2t 蒸汽 (Steam) → 1× 中型货舱 (Cargo Pod (Medium))

完成订单 Helios, Mid-Cargo Build 后解锁蓝图。

- 制作时间：0.3 h
- 功率：40 W
- 生产等级：T2

钛 (Titanium) + 玻璃 (Glass) + 面板 (Panel) → 大型货舱 (Cargo Pod (Large)) **craft_cargo_pod_large** - 尚未解锁 (NOT YET UNLOCKED)

3× 钛锭 (Titanium Ingot)、2× 玻璃 (Glass)、1× 电路面板 (Circuit Panel) 和 3t 蒸汽 (Steam) → 1× 大型货舱 (Cargo Pod (Large))

完成订单 Helios, Cargo Megaorder 后解锁蓝图。

- 制作时间：0.42 h
- 功率：45 W
- 生产等级：T2

钛 (Titanium) + 玻璃 (Glass) + 电池单元 (Cell) → 电池组 (Battery Pack) **craft_battery_pack** - 尚未解锁 (NOT YET UNLOCKED)

1× 钛锭 (Titanium Ingot)、1× 玻璃 (Glass) 和 1× 电池单元 (Battery Cell) → 1× 电池组 (Battery Pack)

完成订单 Spire, Pressure Hardware 后解锁蓝图。

- 制作时间：0.18 h
- 功率：32 W
- 生产等级：T2

钛 (Titanium) + 润滑剂 (Lubricant) + 橡胶 (Rubber) → 小型油罐 (Oil Tank (Small))

craft_oil_tank_small - 尚未解锁 (NOT YET UNLOCKED)

1× 钛锭 (Titanium Ingot)、1× 润滑剂 (Lubricant)、1× 橡胶 (Rubber) 和 1 t 水 (Water) → 1× 小型油罐 (Oil Tank (Small))

完成订单 Vestibule, Heli Thruster Order 后解锁蓝图。

- 制作时间：0.22 h
- 功率：35 W
- 生产等级：T2

钛 (Titanium) + 润滑剂 (Lubricant) + 橡胶 (Rubber) + 内衬 (Lining) → 中型油罐 (Oil Tank (Medium))

craft_oil_tank_medium - 尚未解锁 (NOT YET UNLOCKED)

2× 钛锭 (Titanium Ingot)、1× 润滑剂 (Lubricant)、1× 橡胶 (Rubber)、1× 储罐内衬 (Tank Lining) 和 2 t 水 (Water) → 1× 中型油罐 (Oil Tank (Medium))

完成订单 Vestibule, Oil Tank Order 后解锁蓝图。

- 制作时间：0.32 h
- 功率：40 W
- 生产等级：T2

钛 (Titanium) + 润滑剂 (Lubricant) + 橡胶 (Rubber) + 内衬 (Lining) → 大型油罐 (Oil Tank (Large))

craft_oil_tank_large - 尚未解锁 (NOT YET UNLOCKED)

3× 钛锭 (Titanium Ingot)、2× 润滑剂 (Lubricant)、1× 橡胶 (Rubber)、2× 储罐内衬 (Tank Lining) 和 3 t 水 (Water) → 1× 大型油罐 (Oil Tank (Large))

完成订单 Vestibule, Mid Tank Order 后解锁蓝图。

- 制作时间：0.45 h
- 功率：45 W
- 生产等级：T2

管道 (Pipes) + 阀门 (Valves) + 稀土 (Rare Earth) + 钛 (Titanium) + 焦油 (Tar) → 冷却剂回路 (Coolant Loop) **craft_coolant_loop** - 尚未解锁 (NOT YET UNLOCKED)

4× 液体管段 (Liquid Pipe Segment)、2× 压力阀 (Pressure Valve)、1× 稀土核心 (Rare Earth Core)、2× 钛锭 (Titanium Ingot)、2× 焦油 (Tar) 和 4t 水 (Water) → 1× 冷却剂回路 (Coolant Loop)

完成订单 Spire, Neutronium Order 后解锁蓝图。

- 制作时间：0.75 h
- 功率：50 W
- 生产等级：T2

中子材料 (Neutronium) + 电池单元 (Cells) + 控制单元 (Controls) + 稀土 (Rare Earth) + 焦油 (Tar) → 中子电容器 (Neutron Capacitor) **craft_neutron_capacitor** - 尚未解锁 (NOT YET UNLOCKED)

1× 中子材料棒 (Neutronium Bar)、2× 电池单元 (Battery Cell)、2× 控制单元 (Control Unit)、1× 稀土核心 (Rare Earth Core)、1× 焦油 (Tar)、4t 水 (Water) 和 4t 蒸汽 (Steam) → 1× 中子电容器 (Neutron Capacitor)

完成订单 Spire, Cobalt Stockpile 后解锁蓝图。

- 制作时间：1.2 h
- 功率：70 W
- 生产等级：T3

框架 (Frame) + 控制单元 (Control) + 面板 (Panel) → 制种机套件 (Seed Maker Kit) **craft_seed_maker_kit** - 尚未解锁 (NOT YET UNLOCKED)

2× 机械框架 (Machine Frame)、1× 控制单元 (Control Unit)、2× 电路面板 (Circuit Panel) 和 3t 水 (Water) → 1× 制种机套件 (Seed Maker Kit)

完成制种机研究 (Seed Maker research) 后解锁蓝图。

- 制作时间：0.5 h
- 功率：45 W
- 生产等级：T3

框架 (Frames) + 控制单元 (Controls) + 面板 (Panels) + 液体管道 (Liquid Pipes) → 植物地球化改造器套件 (Plant Terraformer Kit) **craft_plant_terraformer_kit** - 尚未解锁 (NOT YET UNLOCKED)

5× 机械框架 (Machine Frame)、3× 控制单元 (Control Unit)、5× 电路面板 (Circuit Panel)、4× 液体管段 (Liquid Pipe Segment) 和 10 t 水 (Water) → 1× 植物地球化改造器套件 (Plant Terraformer Kit)

完成植物地球化改造器研究 (Plant Terraformer research) 后解锁蓝图。

- 制作时间：1.25 h
- 功率：80 W
- 生产等级：T3

框架 (Frame) + 面板 (Panel) + 玻璃 (Glass) → 生长灯套件 (Grow Lamp Kit) **craft_grow_lamp_kit** - 尚未解锁 (NOT YET UNLOCKED)

1× 机械框架 (Machine Frame)、2× 电路面板 (Circuit Panel) 和 2× 玻璃 (Glass) → 1× 生长灯套件 (Grow Lamp Kit)

完成生长灯研究 (Grow Lamp research) 后解锁蓝图。

- 制作时间：0.35 h
- 功率：35 W
- 生产等级：T2

框架 (Frame) + 管道 (Pipes) + 阀门 (Valve) → 喷灌器套件 (Sprinkler Kit) **craft_sprinkler_kit** - 尚未解锁 (NOT YET UNLOCKED)

1× 机械框架 (Machine Frame)、2× 液体管段 (Liquid Pipe Segment)、1× 压力阀 (Pressure Valve) 和 2 t 水 (Water) → 1× 喷灌器套件 (Sprinkler Kit)

完成喷灌器研究 (Sprinkler research) 后解锁蓝图。

- 制作时间：0.38 h
- 功率：38 W
- 生产等级：T2

框架 (Frame) + 控制单元 (Control) + 面板 (Panel) → 分配器套件 (Dispenser Kit)

craft_dispenser_kit - 尚未解锁 (NOT YET UNLOCKED)

1× 机械框架 (Machine Frame)、1× 控制单元 (Control Unit) 和 1× 电路面板 (Circuit Panel) → 1× 分配器套件 (Dispenser Kit)

完成分配器研究 (Dispenser research) 后解锁蓝图。

- 制作时间：0.4 h
- 功率：40 W
- 生产等级：T3

框架 (Frame) + 铁 (Iron) + 面板 (Panel) → 废料处理器套件 (Waste Processor Kit)

craft_garbage_disposal_kit - 尚未解锁 (NOT YET UNLOCKED)

2× 机械框架 (Machine Frame)、4× 铁锭 (Iron Ingot) 和 1× 电路面板 (Circuit Panel) → 1× 废料处理器套件 (Waste Processor Kit)

完成废物处理研究 (Waste Processing research) 后解锁蓝图。

- 制作时间：0.4 h
- 功率：40 W
- 生产等级：T2

钛 (Titanium) + 燃气管道 (Gas Pipes) → 异星气体帽套件 (Exotic Gas Cap Kit)

craft_exotic_gas_cap_kit - 尚未解锁 (NOT YET UNLOCKED)

2× 钛锭 (Titanium Ingot) 和 2× 燃气管段 (Gas Pipe Segment) → 1× 异星气体帽套件 (Exotic Gas Cap Kit)

完成异星畜牧研究 (Exotic Husbandry research) 后解锁蓝图。

- 制作时间：0.32 h
- 功率：40 W
- 生产等级：T2

钛 (Titanium) + 液体管道 (Liquid Pipes) + 阀门 (Valve) → 异星弹簧水龙头套件 (Exotic Spring Tap Kit) `craft_exotic_spring_tap_kit` - 尚未解锁 (NOT YET UNLOCKED)

2× 钛锭 (Titanium Ingot)、2× 液体管段 (Liquid Pipe Segment) 和 1× 压力阀 (Pressure Valve) → 1× 异星弹簧水龙头套件 (Exotic Spring Tap Kit)

完成异星畜牧研究 (Exotic Husbandry research) 后解锁蓝图。

- 制作时间：0.34 h
- 功率：42 W
- 生产等级：T2

焦油 (Tar) + 玻璃 (Glass) → 肥料 (Fertilizer) `craft_fertilizer` - 尚未解锁 (NOT YET UNLOCKED)

1× 焦油 (Tar)、1× 玻璃 (Glass) 和 2 t 水 (Water) → 2× 肥料 (Fertilizer)

完成生物订单 Soil Enrichment Assay 后解锁蓝图。

- 制作时间：0.18 h
- 功率：32 W
- 生产等级：T2

焦油 (Tar) + 玻璃 (Glass) + 面板 (Panel) → 肥料 Mk II (Fertilizer Mk II) `craft_fertilizer_mk2` - 尚未解锁 (NOT YET UNLOCKED)

2× 焦油 (Tar)、2× 玻璃 (Glass)、1× 电路面板 (Circuit Panel) 和 3 t 水 (Water) → 2× 肥料 Mk II (Fertilizer Mk II)

完成生物订单 Hydrothermal Fragment Manifest 后解锁蓝图。

- 制作时间：0.3 h
- 功率：40 W
- 生产等级：T2

焦油 (Tar) + 玻璃 (Glass) + 电容器 (Capacitor) + 稀土 (Rare Earth) → 肥料 Mk III (Fertilizer Mk III)
craft_fertilizer_mk3 - 尚未解锁 (NOT YET UNLOCKED)

3× 焦油 (Tar)、3× 玻璃 (Glass)、1× 中子电容器 (Neutron Capacitor)、2× 稀土核心 (Rare Earth Core) 和 4 t 水 (Water) → 2× 肥料 Mk III (Fertilizer Mk III)

完成生物订单 Deep-Trench Capstone 后解锁蓝图。

- 制作时间：0.55 h
- 功率：52 W
- 生产等级：T4

塑料 (Plastic) + 稀土 (Rare Earth) → 生长加速剂 (Growth Accelerant) **craft_growth_accelerant** - 尚未解锁 (NOT YET UNLOCKED)

1× 塑料 (Plastic)、1× 稀土核心 (Rare Earth Core) 和 2 t 水 (Water) → 2× 生长加速剂 (Growth Accelerant)

完成生物订单 Volcanic Grand Compendium 后解锁蓝图。

- 制作时间：0.28 h
- 功率：38 W
- 生产等级：T2

电容器 (Capacitor) + 控制单元 (Controls) + 稀土 (Rare Earth) + 冷却剂 (Coolant) → 产量放大器 (Yield Amplifier) **craft_yield_amplifier** - 尚未解锁 (NOT YET UNLOCKED)

1× 中子电容器 (Neutron Capacitor)、2× 控制单元 (Control Unit)、2× 稀土核心 (Rare Earth Core)、1× 冷却剂回路 (Coolant Loop) 和 4 t 水 (Water) → 1× 产量放大器 (Yield Amplifier)

完成订单 Spire, Neutron Capacitor Order 后解锁蓝图。

- 制作时间：1h
- 功率：65 W
- 生产等级：T4

控制单元 (Controls) + 面板 (Panels) + 稀土 (Rare Earth) + 阀门 (Valves) → 植物地球化改造器包 Mk II (Plant Terraformer Pack Mk II) **craft_plant_terraformer_pack_mk2** - 尚未解锁 (NOT YET UNLOCKED)

4× 控制单元 (Control Unit)、6× 电路面板 (Circuit Panel)、3× 稀土核心 (Rare Earth Core)、4× 压力阀 (Pressure Valve) 和 6 t 水 (Water) → 1× 植物地球化改造器 Mk II 升级包 (Plant Terraformer Mk II Upgrade Pack)

完成植物地球化改造器 Mk II 研究 (Plant Terraformer Mk II research) 后解锁蓝图。

- 制作时间：1.5 h
- 功率：90 W
- 生产等级：T3

面板 (Panel) + 玻璃 (Glass) + 稀土 (Rare Earth) → 生长灯包 Mk II (Grow Lamp Pack Mk II) **craft_grow_lamp_pack_mk2** - 尚未解锁 (NOT YET UNLOCKED)

3× 电路面板 (Circuit Panel)、4× 玻璃 (Glass) 和 1× 稀土核心 (Rare Earth Core) → 1× 生长灯 Mk II 升级包 (Grow Lamp Mk II Upgrade Pack)

完成订单 Spire, Optics Stockpile 后解锁蓝图。

- 制作时间：0.45 h
- 功率：42 W
- 生产等级：T2

电容器 (Capacitor) + 稀土 (Rare Earth) + 控制单元 (Controls) → 生长灯包 Mk III (Grow Lamp Pack Mk III) **craft_grow_lamp_pack_mk3** - 尚未解锁 (NOT YET UNLOCKED)

1× 中子电容器 (Neutron Capacitor)、3× 稀土核心 (Rare Earth Core)、2× 控制单元 (Control Unit) 和 3 t 水 (Water) → 1× 生长灯 Mk III 升级包 (Grow Lamp Mk III Upgrade Pack)

完成订单 Spire, Capacitor Bulk Order 后解锁蓝图。

- 制作时间：0.75 h
- 功率：55 W
- 生产等级：T4

塑料 (Plastic) + 冷却剂 (Coolant) + 阀门 (Valves) → 喷灌器包 Mk II (Sprinkler Pack Mk II)
craft_sprinkler_pack_mk2 - 尚未解锁 (NOT YET UNLOCKED)

4× 塑料 (Plastic)、1× 冷却剂回路 (Coolant Loop) 和 2× 压力阀 (Pressure Valve) → 1× 喷灌器 Mk II 升级包 (Sprinkler Mk II Upgrade Pack)

完成订单 Spire, Polymer Megastock 后解锁蓝图。

- 制作时间：0.5 h
- 功率：44 W
- 生产等级：T3

控制单元 (Controls) + 面板 (Panels) + 冷却剂 (Coolant) → 喷灌器包 Mk III (Sprinkler Pack Mk III)
craft_sprinkler_pack_mk3 - 尚未解锁 (NOT YET UNLOCKED)

2× 控制单元 (Control Unit)、4× 电路面板 (Circuit Panel)、2× 冷却剂回路 (Coolant Loop) 和 4 t 水 (Water) → 1× 喷灌器 Mk III 升级包 (Sprinkler Mk III Upgrade Pack)

完成订单 Spire, Avionics Megastock 后解锁蓝图。

- 制作时间：0.85 h
- 功率：58 W
- 生产等级：T3

电容器 (Capacitors) + 稀土 (Rare Earth) + 控制单元 (Controls) + 冷却剂 (Coolant) → 栖息地包 Mk II (Habitat Pack Mk II) **craft_habitat_pack_mk2** - 尚未解锁 (NOT YET UNLOCKED)

2× 中子电容器 (Neutron Capacitor)、3× 稀土核心 (Rare Earth Core)、4× 控制单元 (Control Unit)、3× 冷却剂回路 (Coolant Loop) 和 6 t 水 (Water) → 1× 栖息地 Mk II 升级包 (Habitat Mk II Upgrade Pack)

完成栖息地工程 Mk II 研究 (Habitat Engineering Mk II research) 后解锁蓝图。

- 制作时间：1.4 h
- 功率：110 W
- 生产等级：T4

铅板 (Lead Plate) `craft_lead_plate` - 尚未解锁 (NOT YET UNLOCKED)

2× 铅锭 (Lead Ingot) → 1× 铅板 (Lead Plate)

完成订单 Helios, Lead Consignment 后解锁蓝图。

- 制作时间：0.2 h
- 功率：30 W
- 生产等级：T1

氧气升级包 Mk IV (Oxygen Upgrade Pack Mk IV) `craft_oxygen_upgrade_pack_mk4` - 尚未解锁 (NOT YET UNLOCKED)

4× 铅板 (Lead Plate)、2× 机械框架 (Machine Frame)、3× 电路面板 (Circuit Panel) 和 2× 钴锭 (Cobalt Ingot) → 1× 氧气升级包 Mk IV (Oxygen Upgrade Pack Mk IV)

完成氧气发生器 Mk IV 研究 (Oxygen Generator Mk IV research) 后解锁蓝图。

- 制作时间：2h
- 功率：220 W
- 生产等级：T2

热力升级包 Mk IV (Heat Upgrade Pack Mk IV) `craft_heat_upgrade_pack_mk4` - 尚未解锁 (NOT YET UNLOCKED)

4× 铅板 (Lead Plate)、2× 机械框架 (Machine Frame)、3× 电路面板 (Circuit Panel) 和 2× 钴锭 (Cobalt Ingot) → 1× 热力升级包 Mk IV (Heat Upgrade Pack Mk IV)

完成热力发生器 Mk IV 研究 (Heat Generator Mk IV research) 后解锁蓝图。

- 制作时间：2h
- 功率：220 W
- 生产等级：T2

数据库 / 配方 (Database / Recipes)

饲料制造机配方 (Feed Maker Recipes)

饲料制造机 (Feed Maker) 将饲料原料和生命体压制成生物饲料。每种饲料的蓝图都可通过在生物交易所 (Bio Exchange) 完成对应生物订单 (Bio Order) 来解锁。

盐陆龟饲料 (Salt Tortoise Feed)

Forage + Sea Algae + Snow Moss → Salt Tortoise Feed
craft_feed_salt_tortoise

尚未解锁

100× 植物饲料 (Plant Forage)、1× 海藻 (Sea Algae) 和 1× 雪苔 (Snow Moss) → 20× 盐陆龟饲料 (Salt Tortoise Feed)
蓝图可通过完成生物订单“礁石碎片勘测” (Reef Fragment Survey) 解锁。

制作时间	功率	生产等级
0.3 h	30 W	T1

岩浆环节虫饲料 (Magmatic Annelid Feed)

Forage + Lava Algae + Cave Moss → Magmatic Annelid Feed
craft_feed_magmatic_annelid

尚未解锁

100× 植物饲料 (Plant Forage)、1× 熔岩藻 (Lava Algae) 和 1× 洞穴苔 (Cave Moss) → 20× 岩浆环节虫饲料 (Magmatic Annelid Feed)
蓝图可通过完成生物订单“玄武岩组织样本” (Basalt Tissue Sample) 解锁。

制作时间	功率	生产等级
0.3 h	30 W	T1

菌丝外壳饲料 (Mycelial Husk Feed)

Forage + Cave/Frost/Coral Fungus → Mycelial Husk Feed
craft_feed_mycelial_husk

尚未解锁

100× 植物饲料 (Plant Forage)、1× 洞穴真菌 (Cave Fungus)、1× 冻霜真菌 (Frost Fungus) 和 1× 珊瑚真菌 (Coral Fungus) → 20× 菌丝外壳饲料 (Mycelial Husk Feed)
蓝图可通过完成生物订单“海沟标本板” (Trench Specimen Panel) 解锁。

制作时间	功率	生产等级
0.34 h	32 W	T1

地幔行者饲料（Mantle Strider Feed）

Forage + Magma Crust + Stone Mat → Mantle Strider Feed
craft_feed_mantle_strider

尚未解锁

100× 植物饲料（Plant Forage）、1× 岩浆地壳（Magma Crust）和 1× 岩石垫（Stone Mat）→ 20× 地幔行者饲料（Mantle Strider Feed）

蓝图可通过完成生物订单“黑曜石组织批次”（Obsidian Tissue Lot）解锁。

制作时间	功率	生产等级
0.34 h	32 W	T1

玻璃翼螳螂饲料（Glasswing Mantis Feed）

Forage + Heat Lichen + Frost Lichen → Glasswing Mantis Feed
craft_feed_glasswing_mantis

尚未解锁

100× 植物饲料（Plant Forage）、1× 热地衣（Heat Lichen）和 1× 冻霜地衣（Frost Lichen）→ 20× 玻璃翼螳螂饲料（Glasswing Mantis Feed）

蓝图可通过完成生物订单“热力生物纲要”（Thermal Bio Compendium）解锁。

制作时间	功率	生产等级
0.34 h	32 W	T1

幕纱披风饲料（Veil Mantle Feed）

Forage + Cold Spores + Hot Spores → Veil Mantle Feed
craft_feed_veil_mantle

尚未解锁

100× 植物饲料（Plant Forage）、1× 冷孢子（Cold Spores）和 1× 热孢子（Hot Spores）→ 20× 幕纱披风饲料（Veil Mantle Feed）

蓝图可通过完成生物订单“幕纱披风饮食勘测”（Veil Mantle Diet Survey）解锁。

制作时间	功率	生产等级
0.34 h	32 W	T1

穹窟蟹饲料 (Vault Crab Feed)

Forage + Stone Lichen + Salt Crust → Vault Crab Feed
craft_feed_vault_crab

尚未解锁

100× 植物饲料 (Plant Forage)、1× 岩石地衣 (Stone Lichen) 和 1× 盐壳 (Salt Crust) → 20× 穹窟蟹饲料 (Vault Crab Feed)
蓝图可通过完成生物订单“冷泉化验”(Cold-Seep Assay) 解锁。

制作时间	功率	生产等级
0.4 h	35 W	T1

潮汐头足类饲料 (Tidal Cephalopod Feed)

Forage + Brine Plankton + Deep Algae + Tide Moss → Tidal Cephalopod Feed
craft_feed_tidal_cephalopod

尚未解锁

100× 植物饲料 (Plant Forage)、1× 盐水浮游生物 (Brine Plankton)、1× 深海藻 (Deep Algae) 和 1× 潮汐苔 (Tide Moss) → 20× 潮汐头足类饲料 (Tidal Cephalopod Feed)
蓝图可通过完成生物订单“潟湖标本研究”(Lagoon Specimen Study) 解锁。

制作时间	功率	生产等级
0.4 h	35 W	T1

骨行者饲料 (Bone Walker Feed)

Forage + Ice Crust + Ash Spores → Bone Walker Feed
craft_feed_bone_walker

尚未解锁

100× 植物饲料 (Plant Forage)、1× 冰壳 (Ice Crust) 和 1× 灰烬孢子 (Ash Spores) → 20× 骨行者饲料 (Bone Walker Feed)
蓝图可通过完成生物订单“骨行者饲料索引”(Bone Walker Forage Index) 解锁。

制作时间	功率	生产等级
0.4 h	35 W	T1

喷口漂行者饲料（Vent Drifter Feed）

Forage + Vent Algae + Steam Moss + Sea Algae → Vent Drifter Feed
craft_feed_vent_drifter

尚未解锁

100× 植物饲料（Plant Forage）、1× 喷口藻（Vent Algae）、1× 蒸汽苔（Steam Moss）和 1× 海藻（Sea Algae）→ 20× 喷口漂行者饲料（Vent Drifter Feed）

蓝图可通过完成生物订单“火山口边缘目录”（Caldera-Rim Catalog）解锁。

制作时间	功率	生产等级
0.4 h	35 W	T1

蜂巢哨兵饲料（Hive Sentinel Feed）

Forage + Hot Spores + Crystal Spores → Hive Sentinel Feed
craft_feed_hive_sentinel

尚未解锁

100× 植物饲料（Plant Forage）、1× 热孢子（Hot Spores）和 1× 晶体孢子（Crystal Spores）→ 20× 蜂巢哨兵饲料（Hive Sentinel Feed）

蓝图可通过完成生物订单“泥壶碎片批次”（Mud-Pot Fragment Batch）解锁。

制作时间	功率	生产等级
0.4 h	35 W	T1

空腔合唱团饲料（Hollow Choir Feed）

Forage + Cave Moss + Heat Crust → Hollow Choir Feed
craft_feed_hollow_choir

尚未解锁

100× 植物饲料（Plant Forage）、1× 洞穴苔（Cave Moss）和 1× 热壳（Heat Crust）→ 20× 空腔合唱团饲料（Hollow Choir Feed）

蓝图可通过完成生物订单“俯冲带标本库”（Subduction Specimen Cache）解锁。

制作时间	功率	生产等级
0.4 h	35 W	T1

铁质海百合饲料（Ferric Sea-Lily Feed）

Forage + Shore Lichen + Crystal Spores + Vent Fungus → Ferric Sea-Lily Feed
craft_feed_ferric_sea_lily

尚未解锁

100× 植物饲料（Plant Forage）、1× 岸地衣（Shore Lichen）、1× 晶体孢子（Crystal Spores）和 1× 喷口真菌（Vent Fungus）→ 20× 铁质海百合饲料（Ferric Sea-Lily Feed）

蓝图可通过完成生物订单“浅滩标本库”（Shoal Specimen Cache）解锁。

制作时间	功率	生产等级
0.4 h	35 W	T1

地壳回声饲料（Crustal Echo Feed）

Forage + Stone Mat + Cinder Lichen → Crustal Echo Feed
craft_feed_crustal_echo

尚未解锁

100× 植物饲料（Plant Forage）、1× 岩石垫（Stone Mat）和 1× 炉渣地衣（Cinder Lichen）→ 20× 地壳回声饲料（Crustal Echo Feed）

蓝图可通过完成生物订单“远洋生物勘测”（Pelagic Bio Survey）解锁。

制作时间	功率	生产等级
0.4 h	35 W	T1

冰川巨龙饲料（Glacial Wyrms Feed）

Forage + Ice Algae + Frost Lichen + Vent Algae + Black Fungus → Glacial Wyrms Feed
craft_feed_glacial_wyrms

尚未解锁

100× 植物饲料（Plant Forage）、1× 冰藻（Ice Algae）、1× 冻霜地衣（Frost Lichen）、1× 喷口藻（Vent Algae）和 1× 黑色真菌（Black Fungus）→ 20× 冰川巨龙饲料（Glacial Wyrms Feed）

蓝图可通过完成生物订单“冰川巨龙饲养档案”（Glacial Wyrms Husbandry Dossier）解锁。

制作时间	功率	生产等级
0.5 h	42 W	T1

尖塔飞龙饲料 (Spire Drake Feed)

Forage + Sulfur Moss + Cinder Lichen + Cold Spores + Stone Lichen → Spire Drake Feed
craft_feed_spire_drake

尚未解锁

100× 植物饲料 (Plant Forage)、1× 硫磺苔 (Sulfur Moss)、1× 炉渣地衣 (Cinder Lichen)、1× 冷孢子 (Cold Spores) 和 1× 岩石地衣 (Stone Lichen) → 20× 尖塔飞龙饲料 (Spire Drake Feed)

蓝图可通过完成生物订单“火山碎片清单”(Volcanic Fragment Manifest) 解锁。

制作时间	功率	生产等级
0.5 h	42 W	T1

数据库 / 配方 (Database / Recipes)

精炼机配方 (Refiner Recipes)

精炼机 (Refiner) 将原始异质流体净化为生物可用形态, 并以焦油 (Tar) 作为精炼试剂燃烧: 输入原始流体, 输出精炼流体。蓝图可通过生物订单 (Bio Order) 解锁。

硫磺气体 (Sulfur Gas)

Raw Sulfur Gas + Tar → Sulfur Gas
refine_sulfur_gas
尚未解锁

2× 焦油 (Tar) 和 4 t 原始硫磺气体 (Raw Sulfur Gas) → 4 t 硫磺气体 (Sulfur Gas)

蓝图可通过完成生物订单“喷发区顶点任务” (Eruption-Zone Capstone) 解锁。

制作时间	功率	生产等级
0.3 h	30 W	T2

冷冻流体 (Cryofluid)

Raw Cryofluid + Tar → Cryofluid
refine_cryofluid
尚未解锁

2× 焦油 (Tar) 和 4 t 原始冷冻流体 (Raw Cryofluid) → 4 t 冷冻流体 (Cryofluid)

蓝图可通过完成生物订单“冷冻流体精炼协议” (Cryofluid Refinement Protocol) 解锁。

制作时间	功率	生产等级
0.3 h	30 W	T2

氯气 (Chlorine)

Raw Chlorine + Tar → Chlorine
refine_chlorine
尚未解锁

5× 焦油 (Tar) 和 4 t 原始氯气 (Raw Chlorine) → 4 t 氯气 (Chlorine)

蓝图可通过完成生物订单“地热总纲” (Geothermal Grand Compendium) 解锁。

制作时间	功率	生产等级
0.4 h	48 W	T2

水银（Quicksilver）

Raw Quicksilver + Tar → Quicksilver
refine_quicksilver
尚未解锁

5× 焦油（Tar）和 4t 原始水银（Raw Quicksilver）→ 4t 水银（Quicksilver）

蓝图可通过完成生物订单“深渊总纲”（Abyssal Grand Compendium）解锁。

制作时间	功率	生产等级
0.4 h	48 W	T2

数据库 / 配方 (Database / Recipes)

燃料组装机配方 (Fuel Assembler Recipes)

燃料组装机 (Fuel Assembler) 使用原始铀 (Raw Uranium) 和铅板 (Lead Plate) 制造燃料棒 (Fuel Rod) 与核电池 (Nuclear Battery)。铀从铅罐 (Lead Cask) 输入；燃料棒通过热通道输出，而核电池使用普通产品存储。

燃料棒 (Fuel Rod)

`craft_fuel_rod`

尚未解锁

4× 原始铀 (Raw Uranium) 和 2× 铅板 (Lead Plate) → 1× 燃料棒 (Fuel Rod)

蓝图可通过完成订单“前厅，热货运证明”(Vestibule, Hot Freight Proof) 解锁。

制作时间	功率	生产等级
6h	1,800 W	T2

核电池 (Nuclear Battery)

`craft_nuclear_battery`

尚未解锁

4× 原始铀 (Raw Uranium) 和 3× 铅板 (Lead Plate) → 1× 核电池 (Nuclear Battery)

蓝图可通过完成订单“赫利俄斯，反应堆容器链”(Helios, Reactor Vessel Chain) 解锁。

制作时间	功率	生产等级
4h	1,200 W	T2

数据库 / 设备 (Database / Equipment)

电力设备 (Power Equipment)

来自商店 (Shop) 的发电、储能、充电和电网硬件。研究门控设备会在出现在商店之前先记录在此。

太阳能发电机 (Solar Generator) `solar_generator`

由脚本跟踪的光伏面板 (photovoltaic panel)，在阳光下最多产生 50 W，夜间产生 0 W。倾角不佳会降低日间输出。

出售价	商店价格
500 cr	500 cr

小型电池 (Small Battery) `battery`

基地能源储存。

出售价	商店价格
300 cr	300 cr

大型电池 (Large Battery) `battery_large`

高容量基地能源储存。

出售价	商店价格	组件文档
15,000 cr	15,000 cr	电池 (Battery)

油发电机（Oil Generator） `oil_generator`

燃烧石油发电。

出售价	商店价格	组件文档
2,000 cr	2,000 cr	油发电机（Oil Generator）

反应堆（Reactor） `reactor`

裂变电站（fission plant），使用燃料棒（Fuel Rod）和冷却水最多产生 5,000 W。在热量 1.0 时，一根燃料棒可持续 72 小时；即使核心尚未达到高效温度，燃料消耗也会遵循设定的热量。

出售价	商店价格
750,000 cr	750,000 cr

核电池（Nuclear Battery） `nuclear_battery`

铀核心储能单元，其电量是基础电池的三十倍。

生产者	请求者
核电池（Nuclear Battery）	赫利俄斯，电池合约（Helios, Battery Contract）（王冠）

避雷针（Lightning Rod） `lightning_rod`

部署一个 4,000 Wh、600 m 的雷电储备。状态每天下降 0.05；除非脚本使用风暴玻璃（Storm Glass）修复，否则捕获能力会降为零。

生产者：避雷针（Lightning Rod）

数据库 / 设备（Database / Equipment）

大气设备（Atmosphere Equipment）

大气生成器及其升级包。产品名称与商店和库存中的名称一致，因此精确搜索会回到同一硬件。

氧气生成器（Oxygen Generator） `oxygen_generator`

将大气中的 CO₂ 转换为可呼吸氧气。

出售价	商店价格
1,000 cr	1,000 cr

加热生成器（Heat Generator） `temp_heater`

辐射热量以温暖行星表面。

出售价	商店价格
800 cr	800 cr

压力生成器（Pressure Generator） `pressure_generator`

压缩大气气体以提高表面压力。

出售价	商店价格
900 cr	900 cr

氧气生成器 Mk II 升级包（Oxygen Generator Mk II Upgrade Pack） `oxygen_upgrade_pack_mk2`

将已部署的氧气生成器（Oxygen Generator）升级为 Mk II：输出提高至 5 倍，功耗提高至 5 倍。

出售价	商店价格	组件文档
12,000 cr	12,000 cr	氧气生成器（Oxygen Generator）

加热生成器 Mk II 升级包（Heat Generator Mk II Upgrade Pack） `heat_upgrade_pack_mk2`

将已部署的加热生成器（Heat Generator）升级为 Mk II：输出提高至 4.7 倍，功耗提高至 5 倍。

出售价	商店价格	组件文档
12,000 cr	12,000 cr	加热生成器（Heat Generator）

压力生成器 Mk II 升级包（Pressure Generator Mk II Upgrade Pack） `pressure_upgrade_pack_mk2`

将已部署的压力生成器（Pressure Generator）升级为 Mk II：输出提高至 5 倍，功耗提高至 5 倍。

出售价	商店价格	组件文档
12,000 cr	12,000 cr	压力生成器（Pressure Generator）

氧气生成器 Mk III 升级包（Oxygen Generator Mk III Upgrade Pack） `oxygen_upgrade_pack_mk3`

将已部署的氧气生成器升级为 Mk III：输出为 Mk I 的 200 倍，功率为 Mk I 的 20 倍，并消耗 8 t/h 的水（Water）。没有水时会退回 Mk II 输出（5 倍）。

出售价	商店价格	组件文档
90,000 cr	90,000 cr	氧气生成器（Oxygen Generator）

加热生成器 Mk III 升级包（Heat Generator Mk III Upgrade Pack） `heat_upgrade_pack_mk3`

将已部署的加热生成器升级为 Mk III：输出为 Mk I 的 208 倍，功率为 Mk I 的 20 倍，并消耗 12 t/h 的蒸汽（Steam）。没有蒸汽时会退回 Mk II 输出（4.7 倍）。

出售价	商店价格	组件文档
90,000 cr	90,000 cr	加热生成器（Heat Generator）

压力生成器 Mk III 升级包（Pressure Generator Mk III Upgrade Pack） `pressure_upgrade_pack_mk3`

将已部署的压力生成器升级为 Mk III：输出为 Mk I 的 200 倍，功率为 Mk I 的 20 倍，并消耗 5 t/h 的水（Water）。没有水时会退回 Mk II 输出（5 倍）。

出售价	商店价格	组件文档
90,000 cr	90,000 cr	压力生成器（Pressure Generator）

氧气升级包 Mk IV（Oxygen Upgrade Pack Mk IV） `oxygen_upgrade_pack_mk4`

在相同运行条件下，将一台氧气生成器升级为 Mk IV：输出为 Mk I 的 500 倍，功耗为 Mk I 的 100 倍。用可容纳 4 根燃料棒（Fuel Rod）的弹匣取代水；弹匣由相连的铅罐（Lead Cask）供应。每根燃料棒可运行约 240 小时。装载的燃料棒耗尽且弹匣为空时，生产停止。

出售价	生产者	组件文档
180,000 cr	氧气升级包 Mk IV（Oxygen Upgrade Pack Mk IV）	氧气生成器（Oxygen Generator）

加热升级包 Mk IV（Heat Upgrade Pack Mk IV） `heat_upgrade_pack_mk4`

在相同运行条件下，将一台加热生成器升级为 Mk IV：输出为 Mk I 的 515 倍，功耗为 Mk I 的 100 倍。用可容纳 4 根燃料棒的弹匣取代蒸汽；弹匣由相连的铅罐供应。每根燃料棒可运行约 240 小时。装载的燃料棒耗尽且弹匣为空时，生产停止。

出售价	生产者	组件文档
180,000 cr	加热升级包 Mk IV（Heat Upgrade Pack Mk IV）	加热生成器（Heat Generator）

压力升级包 Mk IV（Pressure Upgrade Pack Mk IV） `pressure_upgrade_pack_mk4`

在相同运行条件下，将一台压力生成器升级为 Mk IV：输出为 Mk I 的 500 倍，功耗为 Mk I 的 100 倍。用可容纳 4 根燃料棒的弹匣取代水；弹匣由相连的铅罐供应。每根燃料棒可运行约 240 小时。装载的燃料棒耗尽且弹匣为空时，生产停止。

出售价	生产者	组件文档
180,000 cr	压力升级包 Mk IV（Pressure Upgrade Pack Mk IV）	压力生成器（Pressure Generator）

气象站（Weather Station） `weather_station`

可编程的 50 W 本地气象站，带实时接收功能和可选的信号板（Signal Board）。

出售价	商店价格
60,000 cr	60,000 cr

数据库 / 设备 (Database / Equipment)

生物设备 (Biology Equipment)

实验室机器、采集器、便携式生物工具和生物订单 (Bio Order) 基础设施。

生物采集器 (Bio Collector) `bio_collector`

自动化采集器，从野外收集生物标本。

出售价	商店价格	组件文档
3,000 cr	3,000 cr	生物采集器 (Bio Collector)

生物实验室 (Bio Lab) `bio_lab`

分析采集的标本并提取生物样本。

出售价	商店价格	组件文档
5,000 cr	5,000 cr	生物实验室 (Bio Lab)

生物发光器 (Bio Luminizer) `bio_luminizer`

调整海岸碎片，使其呈现订单要求的精确发光颜色。

出售价	商店价格	组件文档
60,000 cr	60,000 cr	生物发光器 (Bio Luminizer)

DNA 测序仪（DNA Sequencer） **dna_sequencer**

将基因拼接进地热碎片，使其携带订单所需的基因。

出售价	商店价格	组件文档
100,000 cr	100,000 cr	DNA 测序仪（DNA Sequencer）

生物浇铸器（Bio Caster） **bio_caster**

使用制造材料、蒸汽和水，对生物订单所需的火山碎片进行热浇铸。

出售价	商店价格	组件文档
150,000 cr	150,000 cr	生物浇铸器（Bio Caster）

生物调理器（Bio Conditioner） **bio_conditioner**

通过五阶段质量检查，为生物订单调理深层碎片。

出售价	商店价格	组件文档
225,000 cr	225,000 cr	生物调理器（Bio Conditioner）

生物交换所（Bio Exchange） **bio_exchange**

使用收集的碎片交付生物订单，换取信用点。

出售价	商店价格	组件文档
4,000 cr	4,000 cr	生物交换所（Bio Exchange）

数据库 / 设备（Database / Equipment）

生物圈设备（Biosphere Equipment）

植物、栖息地、饲养和生物群系处理硬件，包括部署套件和升级包。

植物地球化改造器套件（Plant Terraformer Kit） `plant_terraformer_kit`

一台植物地球化改造器（`plant_terraformer`）的制造部署套件。可从库存向运行中的前哨站部署任意数量；每台提供的机器都会将其等级转换率乘以前哨站效率，并计入共享的植物（Plants）支柱。

生产方式：`Frames + Controls + Panels + Liquid Pipes` → `Plant Terraformer Kit`

部署：植物地球化改造器（Plant Terraformer）

作物自动化机套件（Crop Automator Kit） `crop_automator_kit`

部署一台作物自动化机（Crop Automator），可在以自身为中心的 5×5 区域内收获、播种并处理最多 24 个其他格子。

出售价	商店价格	部署
30,000 cr	30,000 cr	作物自动化机（Crop Automator）

便携式生物扫描仪（Portable Bio Scanner） `portable_bio_scanner`

无人机模块，扫描当前地表格的生命体并将结果记录到日志（Journal）。

出售价	商店价格
10,000 cr	10,000 cr

便携式生物提取器（Portable Bio Extractor） `portable_bio_extractor`

无人机模块，用于采集已扫描的生命体。包含一个可容纳单一生命体类型的 25 t 密封舱；货舱吊舱（Cargo Pods）可增加通用容量。

出售价	商店价格
14,000 cr	14,000 cr

精华液化器（Essence Liquifier） `essence_liquifier`

将生命体样本转化为生物群系精华（biome essence）。部署到指定前哨站。

出售价	商店价格
35,000 cr	35,000 cr

生物量混合器（Biomass Mixer） `biomass_mixer`

将生物群系精华混合成生物量；混合物越均衡，产量越高。部署到指定前哨站。

出售价	商店价格
60,000 cr	60,000 cr

生物量混合器 Mk II 升级包（Biomass Mixer Mk II Upgrade Pack） `biomass_mixer_upgrade_pack_mk2`

将已部署的生物量混合器（Biomass Mixer）升级为 Mk II：输出提高至 4.5 倍，精华消耗提高至 2.4 倍，功耗提高至 5 倍。

出售价	商店价格	组件文档
90,000 cr	90,000 cr	生物量混合器（Biomass Mixer）

栖息地（Habitat） **habitat**

一次复苏并容纳一个生物物种。当目的地有足够容量时，已建立的群落可以迁移而不会丢失进度。

出售价	商店价格	组件文档
100,000 cr	100,000 cr	栖息地（Habitat）

饲料制造机（Feed Maker） **feed_maker**

使用植物饲料和采集的生命体制造生物饲料。部署到指定前哨站。

出售价	商店价格	组件文档
50,000 cr	50,000 cr	饲料制造机（Feed Maker）

精炼机（Refiner） **refiner**

将原始异质原料和焦油转化为生物可用的气体或液体。部署到指定前哨站。

出售价	商店价格
80,000 cr	80,000 cr

栖息地 Mk II 升级包（Habitat Mk II Upgrade Pack） **habitat_upgrade_pack_mk2**

将已部署的栖息地升级为 Mk II：承载容量提高至 2 倍，功耗为 144-168 W。繁殖速度和生物成本仍由生物适应性决定。每个栖息地都需要自己的升级包。

生产方式：Capacitors + Rare Earth + Controls + Coolant → Habitat Pack Mk II

组件文档
栖息地（Habitat）

植物改造器 Mk II 升级包 (Plant Terraformer Mk II Upgrade Pack)

plant_terraformer_upgrade_pack_mk2

将已部署的植物改造器 (Plant Terraformer) 升级到 Mk II：转换吞吐量为 2,200 Forage/h，输入送料器每步可处理 80 个物品（而非 16 个），启用时耗电 900 W，并配备肥料 (Fertilizer) 和生长促进剂 (Growth Accelerant) 注入器。

生产配方 控制器 (Controls) + 面板 (Panels) + 稀土 (Rare Earth) + 阀门 (Valves) → 植物改造器 Mk II 升级包 (Plant Terraformer Pack Mk II)

组件文档 植物改造器 (Plant Terraformer)

生长灯 Mk IV 升级包 (Grow Lamp Upgrade Pack Mk IV) **grow_lamp_upgrade_pack_mk4**

将一个生长灯 (Grow Lamp) 升级到 Mk IV：自身基础 Forage 产量变为 8 倍 (+700%)，启用时耗电 500 W。灯通电且启用时，该加成作用于生长期间四个正交相邻格中需要光照的作物。覆盖范围和生长速度保持不变。只有覆盖范围内加成最强的生长灯会计入；其加成可与其他产量加成叠加。

售价 400,000 cr

商店价格 400,000 cr

组件文档 生长灯 (Grow Lamp)

喷灌器 Mk IV 升级包 (Sprinkler Upgrade Pack Mk IV) **sprinkler_upgrade_pack_mk4**

将一个喷灌器 (Sprinkler) 升级到 Mk IV：自身基础 Forage 产量变为 8 倍 (+700%)，启用时耗电 500 W，供给时用水量为 1,000 t/h。喷灌器通电、启用且有供给时，该加成作用于生长期间四个正交相邻格中需要水的作物。覆盖范围和生长速度保持不变。只有覆盖范围内加成最强的喷灌器会计入；其加成可与其他产量加成叠加。

售价 400,000 cr

商店价格 400,000 cr

组件文档 喷灌器 (Sprinkler)

数据库 / 设备 (DATABASE / EQUIPMENT)

采矿设备 (Mining Equipment)

用于勘探车辆、开采硬件、场地设备，以及建立远程采矿作业所需的产品。

车辆充电站 Mk II 升级包 (Vehicle Charging Station Mk II Upgrade Pack) **charging_station_upgrade_pack_mk2**

将已部署的车辆充电站 (Vehicle Charging Station) 升级到 Mk II：拥有 2 个充电位，总充电功率预算为 120 W。一辆车会汇集两个空闲充电位的功率。

售价 8,000 cr

商店价格 8,000 cr

组件文档 车辆充电站 (Vehicle Charging Station)

车辆充电站 Mk III 升级包 (Vehicle Charging Station Mk III Upgrade Pack) **charging_station_upgrade_pack_mk3**

将 Mk II 车辆充电站升级到 Mk III：拥有 4 个充电位，总充电功率预算为 240 W。一辆车会汇集所有空闲充电位的功率。

售价 45,000 cr

商店价格 45,000 cr

组件文档 车辆充电站 (Vehicle Charging Station)

采矿漫游车 (Mining Rover) **rover**

配备集成式 100 Wh 电池和 10 单位货舱的初始远征底盘。安装导航 (Nav)、声呐 (Sonar) 和钻头 (Drill) 模块来配置其用途。

售价 2,000 cr

商店价格 2,000 cr

车辆充电站 (Vehicle Charging Station) `charging_station`

由电网供电的车辆充电器，拥有 1 个充电位和 30 W 充电功率预算。升级包会增加充电位并提高共享充电速度。

售价 1,200 cr

商店价格 1,200 cr

先锋号 (Pioneer) `pioneer`

拥有 8 个插槽的模块化远征底盘，不内置电池、货舱或导航。未安装额外硬件的裸先锋号无法移动。

售价 5,000 cr

商店价格 5,000 cr

前哨站套件 (Outpost Kit) `outpost_kit`

建立新的前哨站，用于容纳机器并连接公用设施网络。额外套件价格更高；商店和库存会显示实时价格。

售价 15,000 cr

商店价格 15,000 cr

无人机基地套件 (Drone Depot Kit) `drone_station_kit`

在前哨站部署一个单充电位无人机基地 (Drone Depot)，并配备 50 单位转运储备。

生产配方 框架 (Frame) + 控制器 (Control) + 气体管道 (Gas Pipes) + 液体管道 (Liquid Pipes) → 无人机基地套件 (Drone Depot Kit)

请求方 Helios、前哨站订单 (Outpost Order)

部署对象 无人机基地 (Drone Depot)

无人机基地套件（中型）（Drone Depot Kit (Medium)） `drone_station_kit_medium`

在前哨站部署一个双充电位无人机基地，并配备 100 单位转运储备。

生产配方 框架（Frame）+ 控制器（Control）+ 面板（Panel）→ 无人机基地套件（中型）（Drone Depot Kit (Medium)）

请求方 Helios、无人机中心储备（Drone Hub Stockpile）

部署对象 无人机基地（Drone Depot）

组件文档 无人机基地（Drone Depot）

无人机基地套件（大型）（Drone Depot Kit (Large)） `drone_station_kit_large`

在前哨站部署一个四充电位无人机基地，并配备 200 单位转运储备。

生产配方 框架（Frame）+ 控制器（Control）+ 面板（Panel）→ 无人机基地套件（大型）（Drone Depot Kit (Large)）

部署对象 无人机基地（Drone Depot）

组件文档 无人机基地（Drone Depot）

无人机服务站套件（Drone Service Station Kit） `drone_service_station_kit`

部署一个可为电动无人机充电、为直升机无人机加油并救援受困无人机的站点。

生产配方 框架（Frame）+ 控制器（Control）+ 面板（Panel）+ 电池单元（Cell）+ 液体管道（Liquid Pipe）→ 无人机服务站套件（Drone Service Station Kit）

请求方 Vestibule、加油台订单（Refueling Pad Order）；Vestibule、加油网络扩建（Refueling Network Expansion）；Vestibule、大陆管道建设（Continental Pipe Build）

部署对象 无人机服务站（Drone Service Station）

采矿钻机套件（Mining Drill Kit） `mining_drill_kit`

部署一台硬度 1 的采矿钻机（Mining Drill），以标准纯度生产 25 t/h，开采时耗电 10 W。

生产配方 框架（Frame）+ 控制器（Control）+ 面板（Panel）→ 采矿钻机套件（Mining Drill Kit）

请求方 Helios、钻机部署订单（Drill Deployment Order）和 Helios、重型钻机建设（Heavy Drill Build）

部署对象 采矿钻机（Mining Drill）

工业采矿钻机套件 (Industrial Mining Drill Kit) `mining_drill_industrial_kit`

部署一台硬度 3 的工业采矿钻机，以标准纯度生产 75 t/h，开采时耗电 35 W。

生产配方 框架 (Frame) + 控制器 (Control) + 面板 (Panel) + 转子 (Rotor) → 工业采矿钻机套件 (Industrial Mining Drill Kit)

请求方 Helios、系缚项目 (Tether Project)

部署对象 采矿钻机 (Mining Drill)

组件文档 采矿钻机 (Mining Drill)

重型采矿钻机套件 (Heavy Mining Drill Kit) `mining_drill_heavy_kit`

部署一台硬度 4 的重型采矿钻机，以标准纯度生产 200 t/h，开采时耗电 100 W。

生产配方 框架 (Frame) + 控制器 (Control) + 面板 (Panel) + 转子 (Rotors) → 重型采矿钻机套件 (Heavy Mining Drill Kit)

部署对象 采矿钻机 (Mining Drill)

组件文档 采矿钻机 (Mining Drill)

无人机 (小型) (Drone (Small)) `drone_small`

小型空中货运无人机底盘。

生产配方 稀土 (Rare Earth) + 钛 (Titanium) + 控制器 (Control) → 无人机 (小型) (Drone (Small))

请求方 Helios、无人机队订单 (Drone Fleet Order) ； Helios、中型无人机建设 (Mid-Drone Build) ； Helios、无人机队订单 (Drone Fleet Order)

部署对象 无人机 (Drone)

无人机 (中型) (Drone (Medium)) `drone_medium`

中型空中货运无人机底盘。

生产配方 稀土 (Rare Earth) + 钛 (Titanium) + 控制器 (Control) → 无人机 (中型) (Drone (Medium))

请求方 Helios、重型无人机建设 (Heavy Drone Build) 和 Helios、无人机队订单 (Drone Fleet Order)

部署对象 无人机 (Drone)

无人机（大型）（Drone (Large)） **drone_large**

重型工业货运无人机底盘。

生产配方 稀土（Rare Earth）+ 钛（Titanium）+ 控制器（Control）→ 无人机（大型）（Drone (Large)）

部署对象 无人机（Drone）

铅制货罐（Lead Cask） **lead_cask**

带屏蔽的储存罐。唯一能存放原铀（Raw Uranium）和燃料棒（Fuel Rods）的固定式容器。

生产配方 铅制货罐（Lead Cask）

雷暴玻璃（Storm Glass） **storm_glass**

雷电击中裸露地面时熔融形成的玻璃。地球愿意高价收购的稀有贸易品。

售价 350 cr

请求方 Spire、雷暴玻璃采购（Storm Glass Acquisition）和 Spire、雷暴玻璃合同（王冠）（Storm Glass Contract (Crown)）

数据库 / 设备 (DATABASE / EQUIPMENT)

物流设备 (Logistics Equipment)

来自商店的订单履行与调度基础设施。

供应码头 (Supply Dock) `supply_dock`

以 25 units/h 的基础速率运送地球订单。额外码头价格更高；商店和库存会显示实时价格。

售价 5,000 cr

商店价格 5,000 cr

数据库 / 设备 (DATABASE / EQUIPMENT)

生产设备 (Production Equipment)

用于存储、制造、物流和核能产品，以构建并移动生产链。

储料箱 (Storage Bin) `storage_bin`

用于存放单一材料的前哨站储存容器。

售价 120 cr

商店价格 120 cr

冶炼炉 (Smelter) `smelter`

工业熔炉。将原矿精炼为金属库存。

售价 650 cr

商店价格 650 cr

制造机 (Fabricator) `fabricator`

制造压机。用精炼库存组装设备。

售价 1,400 cr

商店价格 1,400 cr

仓库 (Warehouse) `warehouse`

可同时存放多种材料的批量储存库。

售价 3,000 cr

商店价格 3,000 cr

组件文档 仓库 (Warehouse)

大型仓库 (Large Warehouse) **large_warehouse**

高层仓库，拥有 15 个材料锁定槽位，总储存量 30,000 单位。

售价 60,000 cr

商店价格 60,000 cr

组件文档 大型仓库 (Large Warehouse)

燃料组装机 (Fuel Assembler) **fuel_assembler**

核工台，将原铀 (Raw Uranium) 和铅板压制成燃料棒 (Fuel Rods) 或核电池 (Nuclear Batteries)。

售价 225,000 cr

商店价格 225,000 cr

数据库 / 设备 (DATABASE / EQUIPMENT)

模块与车辆装备 (Modules & Vehicle Gear)

可安装模块、支架、机架、推进器、货舱、电池、储罐及专用车辆设备。

导航模块 (Nav Module) `nav_module`

用于地面车辆的基础移动传动系统。

售价 500 cr

商店价格 500 cr

声呐模块 (Sonar Module) `sonar_module`

基础声呐：50 m 点扫描、硬度 1 矿物、基础勘测细节。

售价 800 cr

商店价格 800 cr

钻头模块 (Drill Module) `drill_module`

基础钻头。

售价 1,000 cr

商店价格 1,000 cr

运动导航 (Sport Nav) `nav_module_sport`

用于先锋号通用插槽的高性能传动系统。同一装备中，一个运动导航相较基础导航可提供 2× 最高速度，但移动功耗为 2.6×；全油门时每米电池消耗约为 1.3×。

售价 1,200 cr

商店价格 1,200 cr

组件文档 导航模块 (Nav Module)

广域声呐 (Wide Sonar) `sonar_module_wide`

先锋号通用插槽声呐：180 m 点扫描、硬度 3 矿物、更宽的勘测细节。

售价 2,000 cr

商店价格 2,000 cr

组件文档 声呐模块 (Sonar Module)

深层声呐 (Deep Sonar) `sonar_module_deep`

先锋号通用插槽声呐：280 m 点扫描、硬度 4 矿物、最深层的勘测细节。

售价 4,500 cr

商店价格 4,500 cr

组件文档 声呐模块 (Sonar Module)

工业钻头 (Industrial Drill) `drill_module_industrial`

用于先锋号通用插槽的重型钻头。

售价 2,500 cr

商店价格 2,500 cr

组件文档 钻头模块 (Drill Module)

重型钻头 (Heavy Drill) `drill_module_heavy`

用于先锋号通用插槽、适合最致密矿物的顶级钻头。

售价 7,000 cr

商店价格 7,000 cr

组件文档 钻头模块 (Drill Module)

小型电池支架 (Small Battery Holder) **battery_holder_small**

先锋号用的空置单电池位电池外壳。不含便携式电池 (Portable Battery) , 必须另行购买并安装。

售价 500 cr

商店价格 500 cr

中型电池支架 (Medium Battery Holder) **battery_holder_medium**

先锋号用的空置双电池位电池外壳。便携式电池必须另行购买并安装。

售价 1,000 cr

商店价格 1,000 cr

大型电池支架 (Large Battery Holder) **battery_holder_large**

先锋号用的空置三电池位电池外壳。便携式电池必须另行购买并安装。

售价 2,000 cr

商店价格 2,000 cr

小型货架 (Small Cargo Rack) **cargo_rack_small**

先锋号用的空置单箱货架。不含便携式储物箱 (Portable Storage Bin) , 必须另行购买并安装。

售价 500 cr

商店价格 500 cr

中型货架（Medium Cargo Rack） **cargo_rack_medium**

先锋号用的空置双箱货架。便携式储物箱必须另行购买并安装。

售价 1,000 cr

商店价格 1,000 cr

大型货架（Large Cargo Rack） **cargo_rack_large**

先锋号用的空置三箱货架。便携式储物箱必须另行购买并安装。

售价 2,000 cr

商店价格 2,000 cr

便携式电池（Portable Battery） **portable_battery**

适用于任何电池支架的 50 Wh 可充电电池，交付时已充满电。

售价 150 cr

商店价格 150 cr

重型便携式电池（Heavy Portable Battery） **heavy_portable_battery**

适用于任何电池支架的 100 Wh 高密度电池，交付时已充满电。

售价 350 cr

商店价格 350 cr

便携式储物箱（Portable Storage Bin） **portable_bin**

可容纳 25 单位的便携式单矿物容器。装入第一种矿物后会锁定为该矿物。

售价 150 cr

商店价格 150 cr

重型便携式储物箱（Heavy Portable Storage Bin） **heavy_portable_bin**

加固的单矿物容器，可容纳 50 单位，是标准储物箱容量的两倍。

售价 350 cr

商店价格 350 cr

建造模块（Constructor Module） **constructor_module**

先锋号模块，用于建造规划中的地面结构和公用设施网络。

售价 2,500 cr

商店价格 2,500 cr

组件文档 建造模块（Constructor Module）

电动推进器（Electric Thruster） **electric_thruster**

电动无人机推进器：全油门时速度为 300 m/h，耗电 5 Wh/h。消耗量随油门的平方增加，因此低速飞行可以延长航程。

生产配方 稀土（Rare Earth）+ 转子（Rotor）→ 电动推进器（Electric Thruster）

请求方 Spire、推进器订单（Thruster Order）和 Spire、推进控制运行（Propulsion Control Run）

直升机推进器 (Heli Thruster) `heli_thruster`

燃油驱动的无人机推进器：全油门时速度为 900 m/h，耗油 5 t/h。消耗量随油门的平方增加，因此低速飞行可以延长航程。

生产配方 稀土 (Rare Earth) + 控制器 (Control) + 润滑剂 (Lubricant) + 橡胶 (Rubber) → 直升机推进器 (Heli Thruster)

请求方 Vestibule、直升机推进器订单 (Heli Thruster Order) ； Vestibule、重型储罐订单 (Heavy Tank Order) ； Vestibule、直升机机队订单 (Heli Fleet Order) ； Vestibule、直升机推进器大储备 (Heli Thruster Megastock)

货舱 (小型) (Cargo Pod (Small)) `cargo_pod_small`

可装载 100 单位货物的无人机货舱。适用于小型底盘的轻型货运选项。

生产配方 钛 (Titanium) + 玻璃 (Glass) → 货舱 (小型) (Cargo Pod (Small))

请求方 Helios、货舱运行 (Cargo Pod Run)

货舱 (中型) (Cargo Pod (Medium)) `cargo_pod_medium`

可装载 250 单位货物的无人机货舱，适合持续执行前哨站间运输。

生产配方 钛 (Titanium) + 玻璃 (Glass) + 面板 (Panel) → 货舱 (中型) (Cargo Pod (Medium))

请求方 Helios、中型货运订单 (Mid-Cargo Order) ； Helios、中型无人机建设 (Mid-Drone Build) ； Helios、货运大订单 (Cargo Megaorder)

货舱 (大型) (Cargo Pod (Large)) `cargo_pod_large`

可装载 500 单位货物的无人机货舱。重型运输，最适合有多余插槽的底盘。

生产配方 钛 (Titanium) + 玻璃 (Glass) + 面板 (Panel) → 货舱 (大型) (Cargo Pod (Large))

请求方 Helios、重型无人机建设 (Heavy Drone Build)

电池组 (Battery Pack) `battery_pack`

用于远程电动无人机的 150 Wh 可充电电池。新电池组交付时已充满电。

生产配方 钛 (Titanium) + 玻璃 (Glass) + 电池单元 (Cell) → 电池组 (Battery Pack)

请求方 Spire、无人机电力试验 (Drone Power Trial) 和 Vestibule、电力储备订单 (Power Reserve Order)

油箱 (小型) (Oil Tank (Small)) `oil_tank_small`

可容纳 30 t 石油的直升机无人机燃油箱。适合在加油站附近执行短程作业。

生产配方 钛 (Titanium) + 润滑剂 (Lubricant) + 橡胶 (Rubber) → 油箱 (小型) (Oil Tank (Small))

请求方 Vestibule、小型油箱订单 (Oil Tank Order)

油箱 (中型) (Oil Tank (Medium)) `oil_tank_medium`

可容纳 75 t 石油的直升机无人机燃油箱，适合离开前哨站群的航线。

生产配方 钛 (Titanium) + 润滑剂 (Lubricant) + 橡胶 (Rubber) + 内衬 (Lining) → 油箱 (中型) (Oil Tank (Medium))

请求方 Vestibule、中型储罐订单 (Mid Tank Order)

油箱 (大型) (Oil Tank (Large)) `oil_tank_large`

可容纳 150 t 石油的直升机无人机燃油箱。以占用一个模块插槽为代价，提供远程运输航程。

生产配方 钛 (Titanium) + 润滑剂 (Lubricant) + 橡胶 (Rubber) + 内衬 (Lining) → 油箱 (大型) (Oil Tank (Large))

请求方 Vestibule、重型储罐订单 (Heavy Tank Order)；Vestibule、加油网络扩建 (Refueling Network Expansion)；Vestibule、直升机机队订单 (Heli Fleet Order)；Vestibule、系缚储罐运行 (Tether Tank Run)；Vestibule、电力储备订单 (Power Reserve Order)；Vestibule、系缚大运输 (Tether Megahaul)

屏蔽装甲 (Shield Plating) `shield_plating`

用于无暴露收集原铀 (Raw Uranium) 以及从铅制货罐 (Lead Cask) 装载高温货物的无人机铅装甲。重型版本会使每个货舱容量减半，并将燃油消耗提高至 1.5 倍。

生产配方 屏蔽装甲 (Shield Plating)

数据库 / 设备 (DATABASE / EQUIPMENT)

流体基础设施 (Fluid Infrastructure)

用于采集、缓冲、输送或消耗流体的泵、储罐、涡轮机、管道硬件及其他产品。

水泵 (Water Pump) `water_pump`

从已勘测的水井中抽取水。

生产配方 铁 (Iron) + 玻璃 (Glass) + 液体管道 (Liquid Pipe) → 水泵 (Water Pump)

部署对象 水泵 (Water Pump)

油泵 (Oil Pump) `oil_pump`

从已勘测的油井中抽取石油。

生产配方 铁 (Iron) + 钛 (Titanium) + 阀门 (Valve) + 面板 (Panel) → 油泵 (Oil Pump)

部署对象 油泵 (Oil Pump)

蒸汽涡轮机 (Steam Turbine) `steam_turbine`

全油门时从 90 t/h 蒸汽产生最高 108 W。其油门由脚本控制。

售价 7,500 cr

商店价格 7,500 cr

组件文档 蒸汽涡轮机 (Steam Turbine)

蒸汽冷凝器 (Steam Condenser) `steam_condenser`

以最高 250 t/h 的速率将蒸汽冷凝为洁净水。

售价 50,000 cr

商店价格 50,000 cr

组件文档 蒸汽冷凝器 (Steam Condenser)

气体罐 (Gas Tank) `gas_tank`

缓冲罐，可储存一种气体：蒸汽、氨、沼气，或它首次接收的任意气体。

售价 1,200 cr

商店价格 1,200 cr

组件文档 气体罐 (Gas Tank)

液体罐 (Liquid Tank) `liquid_tank`

缓冲罐，可储存一种液体：水、石油或精华液。

售价 900 cr

商店价格 900 cr

组件文档 液体罐 (Liquid Tank)

大型液体罐 (Large Liquid Tank) `bulk_liquid_reservoir`

大型被动式储罐，可容纳 1,000 t 的一种液体，并会锁定为首次接收的液体。

售价 15,000 cr

商店价格 15,000 cr

组件文档 大型液体罐 (Large Liquid Tank)

异种气体封盖套件 (Exotic Gas Cap Kit) `exotic_gas_cap_kit`

部署一个从异种气体喷口收集气体的异种气体封盖 (Exotic Gas Cap)。

售价 1,200 cr

生产配方 钛 (Titanium) + 气体管道 (Gas Pipes) → 异种气体封盖套件 (Exotic Gas Cap Kit)

部署对象 异种气体封盖 (Exotic Gas Cap)

异种泉眼接头套件 (Exotic Spring Tap Kit) `exotic_spring_tap_kit`

部署一个从异种泉眼收集液体的异种泉眼接头 (Exotic Spring Tap) 。

售价 1,200 cr

生产配方 钛 (Titanium) + 液体管道 (Liquid Pipes) + 阀门 (Valve) → 异种泉眼接头套件 (Exotic Spring Tap Kit)

部署对象 异种泉眼接头 (Exotic Spring Tap)

数据库 / 物品 (DATABASE / ITEMS)

矿物 (Minerals)

从已勘测矿物点开采的原矿。硬度 (Hardness, H1 至 H4) 决定哪种声呐可以探测矿点, 以及哪种钻机能够开采; 每种矿石在冶炼炉中都以 1:1 比例熔炼成对应的精炼形态。

铁矿石 (Iron Ore) **iron_ore**

来自内圈矿点的常见结构矿石。早期工业的骨干: 几乎每个框架、结构段和机器零件都从铁开始。

硬度 H1

挖掘时间 每单位 15 min

用途 铁矿石 (Iron Ore) → 铁锭 (Iron Ingot)

请求方 Spire、原铁接收 (Raw Iron Intake)

硅 (Silicon) **silicon**

内圈广泛分布的制玻璃矿石。熔炼为玻璃, 用于面板、外壳和电子设备。

硬度 H1

挖掘时间 每单位 15 min

用途 硅 (Silicon) → 玻璃 (Glass) ; 铁 (Iron) + 硅 (Silicon) → 液体管道段 (Liquid Pipe Segment)

请求方 Spire、硅储备 (Silicon Stock) ; Spire、硅启动储备 (Silicon Bootstrap) ; Spire、硅大运输 (Silicon Megahaul)

钛 (Titanium) **titanium**

中圈的轻质高强度矿石。熔炼为 **titanium_ingot**, 用于转子、推进器和高应力部件。

硬度 H2

挖掘时间 每单位 20 min

用途 钛 (Titanium) → 钛锭 (Titanium Ingot)

请求方 Spire、钛寄售 (Titanium Consignment)

钴（Cobalt） `cobalt`

来自中部环带的电池化学矿石。冶炼为 `cobalt_ingot`，是每个电池单元的核心。

- 硬度：H2
- 挖掘时间：每单位 20 min
- 用于：`Cobalt` → `Cobalt Ingot`

稀土（Rare Earth） `rare_earth`

来自外部环带的稀有磁性矿石。冶炼为 `rare_earth_core`，用于控制电子设备和精密驱动器。

- 硬度：H3
- 挖掘时间：每单位 25 min
- 用于：`Rare Earth` → `Rare Earth Core`

中子矿（Neutronium） `neutronium`

行星上已知最坚硬的矿石，产于远离基地的区域。冶炼为 `neutronium_bar`，用于电容器和深层级硬件。

- 硬度：H4
- 挖掘时间：每单位 30 min
- 用于：`Neutronium` → `Neutronium Bar`

铅矿（Lead Ore） `lead_ore`

来自矿物地点的致密软矿石。冶炼为 `lead_ingot`，是所有辐射屏蔽的基础。

- 硬度：H2
- 挖掘时间：每单位 18 min
- 用于：`Lead Ore` → `Lead Ingot`

数据库 / 物品 (DATABASE / ITEMS)

精炼材料 (Refined Materials)

冶炼炉产物：每个制造机蓝图都会用到的标准材料。

铁锭 (Iron Ingot) `iron_ingot`

冶炼后的铁材。通用结构输入：框架、分段、阀门，以及制造机目录中一半的配方都从这里开始。

- 产自：`Iron Ore` → `Iron Ingot`
- 用于：`Iron` → `Gas Pipe Segment`、`Iron + Silicon` → `Liquid Pipe Segment`、`Iron + Titanium` → `Power Line Segment`、`Iron + Glass` → `Pressure Valve`、`Iron + Titanium` → `Machine Frame`、`Iron + Glass` → `Circuit Panel`、`Cobalt + Iron + Glass` → `Battery Cell`、`Iron + Titanium + Glass` → `Tank Lining`、`Iron + Glass + Liquid Pipe` → `Water Pump`、`Iron + Titanium + Valve + Panel` → `Oil Pump`、`Iron + Oil` → `Lubricant + Tar`，以及 `Frame + Iron + Panel` → `Waste Processor Kit`
- 请求方：`Helios, Iron Bootstrap`；`Helios, Iron Production`；`Helios, Mixed Alloy Order`；`Helios, Battery Order`；`Helios, Bulk Iron Run`；`Helios, Bulk Frame Run`；`Vestibule, Utility Conduit Stock`；`Helios, Lead Survey Stock`

玻璃 (Glass) `glass`

用于面板、外壳和光学器件的熔炼硅片材料。

- 产自：`Silicon` → `Glass`
- 用于：`Iron + Glass` → `Pressure Valve`、`Iron + Glass` → `Circuit Panel`、`Panel + Titanium + Glass` → `Control Unit`、`Cobalt + Iron + Glass` → `Battery Cell`、`Iron + Titanium + Glass` → `Tank Lining`、`Iron + Glass + Liquid Pipe` → `Water Pump`、`Glass + Oil` → `Plastic + Tar`、`Titanium + Glass` → `Cargo Pod (Small)`、`Titanium + Glass + Panel` → `Cargo Pod (Medium)`、`Titanium + Glass + Panel` → `Cargo Pod (Large)`、`Titanium + Glass + Cell` → `Battery Pack`、`Frame + Panel + Glass` → `Grow Lamp Kit`、`Tar + Glass` → `Fertilizer`、`Tar + Glass + Panel` → `Fertilizer Mk II`、`Tar + Glass + Capacitor + Rare Earth` → `Fertilizer Mk III`，以及 `Panel + Glass + Rare Earth` → `Grow Lamp Pack Mk II`
- 请求方：`Spire, Optical Glass`；`Spire, Pressure Hardware`；`Spire, Optics Stockpile`；`Spire, Silicon Megahaul`

钛锭 (Titanium Ingot) titanium_ingot

用于转子、推进器和压力硬件的高强度轻质材料。

- 产自: Titanium → Titanium Ingot
- 用于: Iron + Titanium → Power Line Segment、Iron + Titanium → Machine Frame、Panel + Titanium + Glass → Control Unit、Titanium + Pipes → Thermal Cap Kit、Titanium + Cobalt + Rare Earth → Turbine Rotor、Iron + Titanium + Glass → Tank Lining、Iron + Titanium + Valve + Panel → Oil Pump、Rare Earth + Titanium + Control → Drone (Small)、Rare Earth + Titanium + Control → Drone (Medium)、Rare Earth + Titanium + Control → Drone (Large)、Titanium + Glass → Cargo Pod (Small)、Titanium + Glass + Panel → Cargo Pod (Medium)、Titanium + Glass + Panel → Cargo Pod (Large)、Titanium + Glass + Cell → Battery Pack、Titanium + Lubricant + Rubber → Oil Tank (Small)、Titanium + Lubricant + Rubber + Lining → Oil Tank (Medium)、Titanium + Lubricant + Rubber + Lining → Oil Tank (Large)、Pipes + Valves + Rare Earth + Titanium + Tar → Coolant Loop、Titanium + Gas Pipes → Exotic Gas Cap Kit, 以及 Titanium + Liquid Pipes + Valve → Exotic Spring Tap Kit
- 请求方: Helios, Titanium Run ; Helios, Mixed Alloy Order ; Helios, Lead Survey Stock

钴锭 (Cobalt Ingot) cobalt_ingot

电池级材料。每个 battery_cell 都以它为核心构建。

- 产自: Cobalt → Cobalt Ingot
- 用于: Cobalt + Iron + Glass → Battery Cell、Titanium + Cobalt + Rare Earth → Turbine Rotor、Cobalt + Oil → Rubber + Tar、Oxygen Upgrade Pack Mk IV、Heat Upgrade Pack Mk IV, 以及 Pressure Upgrade Pack Mk IV
- 请求方: Spire, Cobalt Run ; Spire, Magnetic Stator Build ; Spire, Cobalt Stockpile

稀土核心 (Rare Earth Core) `rare_earth_core`

用于控制单元和高级电子设备的精密磁性核心材料。

- 产自: `Rare Earth` → `Rare Earth Core`
- 用于: `Titanium + Cobalt + Rare Earth` → `Turbine Rotor`、`Rare Earth + Titanium + Control` → `Drone (Small)`、`Rare Earth + Titanium + Control` → `Drone (Medium)`、`Rare Earth + Titanium + Control` → `Drone (Large)`、`Rare Earth + Rotor` → `Electric Thruster`、`Rare Earth + Control + Lubricant + Rubber` → `Heli Thruster`、`Pipes + Valves + Rare Earth + Titanium + Tar` → `Coolant Loop`、`Neutronium + Cells + Controls + Rare Earth + Tar` → `Neutron Capacitor`、`Tar + Glass + Capacitor + Rare Earth` → `Fertilizer Mk III`、`Plastic + Rare Earth` → `Growth Accelerant`、`Capacitor + Controls + Rare Earth + Coolant` → `Yield Amplifier`、`Controls + Panels + Rare Earth + Valves` → `Plant Terraformer Pack Mk II`、`Panel + Glass + Rare Earth` → `Grow Lamp Pack Mk II`、`Capacitor + Rare Earth + Controls` → `Grow Lamp Pack Mk III`，以及 `Capacitors + Rare Earth + Controls + Coolant` → `Habitat Pack Mk II`
- 请求方: `Spire, Rare Earth Order` ; `Spire, Drone Power Trial` ; `Spire, Capacitor Bulk Order`

中子矿锭 (Neutronium Bar) `neutronium_bar`

用于中子电容器和高级硬件的超高密度棒材。

- 产自: `Neutronium` → `Neutronium Bar`
- 用于: `Neutronium + Cells + Controls + Rare Earth + Tar` → `Neutron Capacitor`
- 请求方: `Spire, Neutronium Order` ; `Spire, Neutronium Megastock`

铅锭 (Lead Ingot) `lead_ingot`

柔软而致密的屏蔽材料。轧制成 `lead_plate`，用于容器和反应堆硬件。

- 产自: `Lead Ore` → `Lead Ingot`
- 用于: `Lead Plate`
- 请求方: `Helios, Lead Consignment`

数据库 / 物品 (DATABASE / ITEMS)

制造部件 (Fabricated Components)

制造机产物：机加工零件、基础设施分段、部署套件和石油衍生聚合物。地球承包商会订购其中许多物品，请查看每张卡片的“请求方”行。

燃气管分段 (Gas Pipe Segment) `gas_pipe_segment`

一段 10 m 长的燃气管。先锋的 Constructor 在铺设燃气管线时，每个规划分段消耗一个。

- 售价：20 cr
- 产自：Iron → Gas Pipe Segment
- 用于：Gas Segments + Valve → Gas Bridge、Titanium + Pipes → Thermal Cap Kit、Frame + Control + Gas Pipes + Liquid Pipes → Drone Depot Kit，以及 Titanium + Gas Pipes → Exotic Gas Cap Kit
- 请求方：Vestibule, Gas Pipe Order；Vestibule, Pipe Network Run；Vestibule, Pipe Order

液体管分段 (Liquid Pipe Segment) `liquid_pipe_segment`

一段 10 m 长的液体管。先锋的 Constructor 在铺设液体管线时，每个规划分段消耗一个。

- 售价：20 cr
- 产自：Iron + Silicon → Liquid Pipe Segment
- 用于：Liquid Segments + Valve → Liquid Bridge、Iron + Glass + Liquid Pipe → Water Pump、Frame + Control + Gas Pipes + Liquid Pipes → Drone Depot Kit、Frame + Control + Panel + Cell + Liquid Pipe → Drone Service Station Kit、Pipes + Valves + Rare Earth + Titanium + Tar → Coolant Loop、Frames + Controls + Panels + Liquid Pipes → Plant Terraformer Kit、Frame + Pipes + Valve → Sprinkler Kit，以及 Titanium + Liquid Pipes + Valve → Exotic Spring Tap Kit
- 请求方：Vestibule, Liquid Pipe Order；Vestibule, Pressure Network Trial；Vestibule, Pipe Network Run；Vestibule, Pipe Order；Vestibule, Pipe & Lubricant Order；Vestibule, Refueling Hardware Build；Vestibule, Continental Pipe Build

电力线分段 (Power Line Segment) `power_line_segment`

一段 10 m 长的电力线。先锋的 Constructor 在为前哨站和机器布线时，每个规划分段消耗一个。

- 售价：20 cr
- 产自：Iron + Titanium → Power Line Segment
- 用于：Power Segments + Circuit → Power Bridge

燃气管桥 (Gas Pipe Bridge) `gas_pipe_bridge`

跨越另一条管线而不连接网络的 3 格燃气高架桥。

- 产自：Gas Segments + Valve → Gas Bridge

液体管桥 (Liquid Pipe Bridge) `liquid_pipe_bridge`

跨越另一条管线而不连接网络的 3 格液体高架桥。

- 产自：Liquid Segments + Valve → Liquid Bridge

电力线桥 (Power Line Bridge) `power_line_bridge`

跨越另一条线路而不连接网络的 3 格电力高架桥。

- 产自：Power Segments + Circuit → Power Bridge

压力阀（Pressure Valve） `pressure_valve`

用于泵、储罐和大气设备的机加工流量控制阀。

- 产自：Iron + Glass → Pressure Valve
- 用于：Gas Segments + Valve → Gas Bridge、Liquid Segments + Valve → Liquid Bridge、Iron + Titanium + Valve + Panel → Oil Pump、Pipes + Valves + Rare Earth + Titanium + Tar → Coolant Loop、Frame + Pipes + Valve → Sprinkler Kit、Titanium + Liquid Pipes + Valve → Exotic Spring Tap Kit、Controls + Panels + Rare Earth + Valves → Plant Terraformer Pack Mk II, 以及 Plastic + Coolant + Valves → Sprinkler Pack Mk II
- 请求方：Spire, Pressure Hardware；Vestibule, Pressure Network Trial

机器框架（Machine Frame） `machine_frame`

大多数中阶机器都在其上组装的标准结构底盘。

- 产自：Iron + Titanium → Machine Frame
- 用于：Frame + Control + Gas Pipes + Liquid Pipes → Drone Depot Kit、Frame + Control + Panel → Drone Depot Kit (Medium)、Frame + Control + Panel → Drone Depot Kit (Large)、Frame + Control + Panel + Cell + Liquid Pipe → Drone Service Station Kit、Frame + Control + Panel → Mining Drill Kit、Frame + Control + Panel + Rotor → Industrial Mining Drill Kit、Frame + Control + Panel + Rotors → Heavy Mining Drill Kit、Frame + Control + Panel → Seed Maker Kit、Frames + Controls + Panels + Liquid Pipes → Plant Terraformer Kit、Frame + Panel + Glass → Grow Lamp Kit、Frame + Pipes + Valve → Sprinkler Kit、Frame + Control + Panel → Dispenser Kit、Frame + Iron + Panel → Waste Processor Kit、Oxygen Upgrade Pack Mk IV、Heat Upgrade Pack Mk IV、Pressure Upgrade Pack Mk IV、Lead Cask、Shield Plating, 以及 Lightning Rod
- 请求方：Helios, Frame Order；Helios, Bulk Iron Run；Helios, Reservoir Build；Helios, Cargo Pod Run；Helios, Bulk Frame Run；Helios, Reactor Vessel Chain

电路面板 (Circuit Panel) `circuit_panel`

用于机器逻辑和仪表的印刷控制电路。

- 产自 : Iron + Glass → Circuit Panel
- 用于 : Power Segments + Circuit → Power Bridge、Panel + Titanium + Glass → Control Unit、Iron + Titanium + Valve + Panel → Oil Pump、Frame + Control + Panel → Drone Depot Kit (Medium)、Frame + Control + Panel → Drone Depot Kit (Large)、Frame + Control + Panel + Cell + Liquid Pipe → Drone Service Station Kit、Frame + Control + Panel → Mining Drill Kit、Frame + Control + Panel + Rotor → Industrial Mining Drill Kit、Frame + Control + Panel + Rotors → Heavy Mining Drill Kit、Titanium + Glass + Panel → Cargo Pod (Medium)、Titanium + Glass + Panel → Cargo Pod (Large)、Frame + Control + Panel → Seed Maker Kit、Frames + Controls + Panels + Liquid Pipes → Plant Terraformer Kit、Frame + Panel + Glass → Grow Lamp Kit、Frame + Control + Panel → Dispenser Kit、Frame + Iron + Panel → Waste Processor Kit、Tar + Glass + Panel → Fertilizer Mk II、Controls + Panels + Rare Earth + Valves → Plant Terraformer Pack Mk II、Panel + Glass + Rare Earth → Grow Lamp Pack Mk II、Controls + Panels + Coolant → Sprinkler Pack Mk III、Oxygen Upgrade Pack Mk IV、Heat Upgrade Pack Mk IV、Pressure Upgrade Pack Mk IV, 以及 Lightning Rod
- 请求方 : Spire, Circuit Order ; Spire, Avionics Run ; Spire, Avionics Megabuild ; Spire, Polymer Optics Run ; Spire, Optics Stockpile ; Spire, Avionics Megastock

控制单元 (Control Unit) `control_unit`

驱动高级机器和载具的密封处理模块。

- 产自 : Panel + Titanium + Glass → Control Unit
- 用于 : Frame + Control + Gas Pipes + Liquid Pipes → Drone Depot Kit、Frame + Control + Panel → Drone Depot Kit (Medium)、Frame + Control + Panel → Drone Depot Kit (Large)、Frame + Control + Panel + Cell + Liquid Pipe → Drone Service Station Kit、Frame + Control + Panel → Mining Drill Kit、Frame + Control + Panel + Rotor → Industrial Mining Drill Kit、Frame + Control + Panel + Rotors → Heavy Mining Drill Kit、Rare Earth + Titanium + Control → Drone (Small)、Rare Earth + Titanium + Control → Drone (Medium)、Rare Earth + Titanium + Control → Drone (Large)、Rare Earth + Control + Lubricant + Rubber → Heli Thruster、Neutronium + Cells + Controls + Rare Earth + Tar → Neutron Capacitor、Frame + Control + Panel → Seed Maker Kit、Frames + Controls + Panels + Liquid Pipes → Plant Terraformer Kit、Frame + Control + Panel → Dispenser Kit、Capacitor + Controls + Rare Earth + Coolant → Yield Amplifier、Controls + Panels + Rare Earth + Valves → Plant Terraformer Pack Mk II、Capacitor + Rare Earth + Controls → Grow Lamp Pack Mk III、Controls + Panels + Coolant → Sprinkler Pack Mk III, 以及 Capacitors + Rare Earth + Controls + Coolant → Habitat Pack Mk II
- 请求方 : Spire, Avionics Run ; Spire, Control Run ; Spire, Avionics Megabuild ; Spire, Magnetic Stator Build ; Spire, Propulsion Control Run ; Spire, Neutron Capacitor Order ; Spire, Avionics Megastock

电池单元 (Battery Cell) `battery_cell`

钴化学储能单元，是电池组和电网储能的构建基础。

- 产自：`Cobalt + Iron + Glass → Battery Cell`
- 用于：`Frame + Control + Panel + Cell + Liquid Pipe → Drone Service Station Kit`、`Titanium + Glass + Cell → Battery Pack`、`Neutronium + Cells + Controls + Rare Earth + Tar → Neutron Capacitor`，以及 `Lightning Rod`
- 请求方：`Helios, Battery Order`；`Helios, Drone Chassis Build`；`Helios, Mid-Cargo Build`

热帽套件 (Thermal Cap Kit) `thermal_cap_kit`

由 Constructor 部署的套件，用热帽封住勘测到的热通风口并捕集 Steam。若其腔室达到 100%，会把储存的 Steam 全部排入大气；脚本必须释放、导流或缓解压力。

- 售价：800 cr
- 产自：`Titanium + Pipes → Thermal Cap Kit`
- 请求方：`Helios, Cap Kit Order`
- 部署：`Thermal Cap`

涡轮转子 (Turbine Rotor) `turbine_rotor`

用于蒸汽涡轮机的精密平衡钛转子。

- 产自：`Titanium + Cobalt + Rare Earth → Turbine Rotor`
- 用于：`Frame + Control + Panel + Rotor → Industrial Mining Drill Kit`、`Frame + Control + Panel + Rotors → Heavy Mining Drill Kit`，以及 `Rare Earth + Rotor → Electric Thruster`
- 请求方：`Helios, Rotor Run`

罐体衬里 (Tank Lining) tank_lining

用于燃气罐和液体罐的耐腐蚀衬里。

- 产自: Iron + Titanium + Glass → Tank Lining
- 用于: Titanium + Lubricant + Rubber + Lining → Oil Tank (Medium) 以及 Titanium + Lubricant + Rubber + Lining → Oil Tank (Large)
- 请求方: Helios, Drone Chassis Build ; Helios, Reservoir Build ; Helios, Mid-Cargo Build

润滑剂 (Lubricant) lubricant

使驱动器和转子平稳运行的精炼石油衍生物。

- 产自: Iron + Oil → Lubricant + Tar
- 用于: Rare Earth + Control + Lubricant + Rubber → Heli Thruster、Titanium + Lubricant + Rubber → Oil Tank (Small)、Titanium + Lubricant + Rubber + Lining → Oil Tank (Medium), 以及 Titanium + Lubricant + Rubber + Lining → Oil Tank (Large)
- 请求方: Vestibule, Polymer Stockpile ; Vestibule, Bulk Polymer Run ; Vestibule, Polymer Run ; Vestibule, Pipe & Lubricant Order ; Vestibule, Refueling Hardware Build ; Vestibule, Polymer Bulk Order ; Vestibule, Polymer Megaorder ; Vestibule, Tether Megahaul ; Vestibule, Shielded Transport Trial

塑料 (Plastic) plastic

用于外壳和绝缘的石油衍生聚合物材料。

- 产自: Glass + Oil → Plastic + Tar
- 用于: Plastic + Forage + Water → Reinforced Biopolymer、Plastic + Rare Earth → Growth Accelerant, 以及 Plastic + Coolant + Valves → Sprinkler Pack Mk II
- 请求方: Spire, Polymer Optics Run ; Spire, Polymer Megastock ; Spire, Polymer Megaorder

橡胶 (Rubber) rubber

用于垫圈和密封件的柔性石油衍生材料。

- 产自 : Cobalt + Oil → Rubber + Tar
- 用于 : Rare Earth + Control + Lubricant + Rubber → Heli Thruster、Titanium + Lubricant + Rubber → Oil Tank (Small)、Titanium + Lubricant + Rubber + Lining → Oil Tank (Medium), 以及 Titanium + Lubricant + Rubber + Lining → Oil Tank (Large)
- 请求方 : Vestibule, Polymer Stockpile ; Vestibule, Bulk Polymer Run ; Vestibule, Polymer Run ; Vestibule, Refueling Hardware Build ; Vestibule, Polymer Bulk Order ; Vestibule, Polymer Megaorder ; Vestibule, Flexible Habitat Liners

焦油 (Tar) tar

用于肥料、高级部件和精炼机的重油残渣。石油产品配方会将其作为副产品生成；重油裂解会直接生成它。

- 产自 : Heavy Oil Cracking: Oil → Tar
- 副产品 : Iron + Oil → Lubricant + Tar、Glass + Oil → Plastic + Tar, 以及 Cobalt + Oil → Rubber + Tar
- 用于 : Pipes + Valves + Rare Earth + Titanium + Tar → Coolant Loop、Neutronium + Cells + Controls + Rare Earth + Tar → Neutron Capacitor、Tar + Glass → Fertilizer、Tar + Glass + Panel → Fertilizer Mk II、Tar + Glass + Capacitor + Rare Earth → Fertilizer Mk III、Raw Sulfur Gas + Tar → Sulfur Gas、Raw Cryofluid + Tar → Cryofluid、Raw Chlorine + Tar → Chlorine, 以及 Raw Quicksilver + Tar → Quicksilver

强化生物聚合物 (Reinforced Biopolymer) reinforced_biopolymer

由塑料、植物饲料和水制成的结构性生物复合材料。用于后期地球出口和高级生物化学。

- 产自 : Plastic + Forage + Water → Reinforced Biopolymer
- 用于 : Biopolymer + Forage + Water → Enrichment Compound
- 请求方 : Spire, Biopolymer Thermal Stock ; Vestibule, Flexible Habitat Liners ; Helios, Enriched Habitat Trial ; Spire, Biosphere Export Program

富集化合物 (Enrichment Compound) `enrichment_compound`

由强化生物聚合物、植物饲料和水制成的浓缩生物材料，用于后期地球出口。

- 产自：`Biopolymer + Forage + Water → Enrichment Compound`
- 请求方：Helios, Enriched Habitat Trial；Spire, Biosphere Export Program

冷却回路 (Coolant Loop) `coolant_loop`

用于反应堆级热负荷的闭式冷却组件。

- 产自：`Pipes + Valves + Rare Earth + Titanium + Tar → Coolant Loop`
- 用于：`Capacitor + Controls + Rare Earth + Coolant → Yield Amplifier`、`Plastic + Coolant + Valves → Sprinkler Pack Mk II`、`Controls + Panels + Coolant → Sprinkler Pack Mk III`，以及 `Capacitors + Rare Earth + Controls + Coolant → Habitat Pack Mk II`
- 请求方：Spire, Polymer Megastock；Spire, T4 Reactor Build；Spire, Biopolymer Thermal Stock

中子电容器 (Neutron Capacitor) `neutron_capacitor`

用于极端能量应用的中子矿储能设备。

- 产自：`Neutronium + Cells + Controls + Rare Earth + Tar → Neutron Capacitor`
- 用于：`Tar + Glass + Capacitor + Rare Earth → Fertilizer Mk III`、`Capacitor + Controls + Rare Earth + Coolant → Yield Amplifier`、`Capacitor + Rare Earth + Controls → Grow Lamp Pack Mk III`，以及 `Capacitors + Rare Earth + Controls + Coolant → Habitat Pack Mk II`
- 请求方：Spire, Neutron Capacitor Order；Spire, Capacitor Bulk Order；Spire, T4 Reactor Build

肥料 (Fertilizer) `fertilizer`

基础土壤营养混合物。一次田地施用可使生长中的作物单独获得基础 Forage 产量的 2 倍 (+100%)，持续 8 小时。同等级施用会延长持续时间；当前施用失效后再切换等级。其产量加成会叠加到提供者和 Yield Amplifier 加成上。Plant Terraformer 每消耗一个物品，就为其配方提供 10 点效力。未使用的效力会留在机器中，供后续批次使用。

• 产自: `Tar + Glass → Fertilizer`

肥料 Mk II (Fertilizer Mk II) `fertilizer_mk2`

浓缩营养混合物。一次田地施用可使生长中的作物单独获得基础 Forage 产量的 3 倍 (+200%)，持续 8 小时。同等级施用会延长持续时间；当前施用失效后再切换等级。其产量加成会叠加到提供者和 Yield Amplifier 加成上。Plant Terraformer 每消耗一个物品，就为其配方提供 30 点效力。未使用的效力会留在机器中，供后续批次使用。

• 产自: `Tar + Glass + Panel → Fertilizer Mk II`

肥料 Mk III (Fertilizer Mk III) `fertilizer_mk3`

顶级营养混合物。一次田地施用可使生长中的作物单独获得基础 Forage 产量的 5 倍 (+400%)，持续 8 小时。同等级施用会延长持续时间；当前施用失效后再切换等级。其产量加成会叠加到提供者和 Yield Amplifier 加成上。Plant Terraformer 每消耗一个物品，就为其配方提供 50 点效力。未使用的效力会留在机器中，供后续批次使用。

• 产自: `Tar + Glass + Capacitor + Rare Earth → Fertilizer Mk III`

生长加速剂 (Growth Accelerant) `growth_accelerant`

一次田地施用可使生长中的作物生长速度翻倍，持续 8 小时；在需求得到满足时，作物会更快达到相同产量。额外施用会延长持续时间。它也会作为 Plant Terraformer 最终转化配方的完整物品输入被消耗；不存在效力等级。

• 产自: `Plastic + Rare Earth → Growth Accelerant`

产量增幅器 (Yield Amplifier) `yield_amplifier`

一次施用可在整块田地范围内，为生长中的作物增加 +200% 基础 Forage 产量（单独为 3 倍），持续 24 小时。其加成会叠加到提供者和 Fertilizer 加成上。额外施用会延长持续时间。成熟作物会保留已经存入的 Forage。

- 产自：Capacitor + Controls + Rare Earth + Coolant → Yield Amplifier

生长灯 Mk II 升级套件 (Grow Lamp Mk II Upgrade Pack) `grow_lamp_upgrade_pack_mk2`

将一个 Grow Lamp 升级为 Mk II：单独提供 2 倍基础 Forage 产量 (+100%)，启用时功耗为 25 W。灯具通电并启用时，该加成会在生长期间作用于四个正交相邻格中需要光照的作物。覆盖范围和生长速度不变。只有覆盖范围内等级最高的灯具计入；其加成会叠加到其他产量加成上。

- 产自：Panel + Glass + Rare Earth → Grow Lamp Pack Mk II
- 部件文档：Grow Lamp

生长灯 Mk III 升级套件 (Grow Lamp Mk III Upgrade Pack) `grow_lamp_upgrade_pack_mk3`

将一个 Grow Lamp 升级为 Mk III：单独提供 4 倍基础 Forage 产量 (+300%)，启用时功耗为 100 W。灯具通电并启用时，该加成会在生长期间作用于四个正交相邻格中需要光照的作物。覆盖范围和生长速度不变。只有覆盖范围内等级最高的灯具计入；其加成会叠加到其他产量加成上。

- 产自：Capacitor + Rare Earth + Controls → Grow Lamp Pack Mk III
- 部件文档：Grow Lamp

喷水器 Mk II 升级套件 (Sprinkler Mk II Upgrade Pack) `sprinkler_upgrade_pack_mk2`

将一个 Sprinkler 升级为 Mk II：单独提供 2 倍基础 Forage 产量 (+100%)，启用时功耗为 25 W，获得供给时耗水量为 10 t/h。喷水器通电、启用并获得供给时，该加成会在生长期间作用于四个正交相邻格中需要水分的作物。覆盖范围和生长速度不变。只有覆盖范围内等级最高的喷水器计入；其加成会叠加到其他产量加成上。

- 产自：Plastic + Coolant + Valves → Sprinkler Pack Mk II
- 部件文档：Sprinkler

喷水器 Mk III 升级套件 (Sprinkler Mk III Upgrade Pack) `sprinkler_upgrade_pack_mk3`

将一个 Sprinkler 升级为 Mk III：单独提供 4 倍基础 Forage 产量 (+300%)，启用时功耗为 100 W，获得供给时耗水量为 200 t/h。喷水器通电、启用并获得供给时，该加成会在生长期间作用于四个正交相邻格中需要水分的作物。覆盖范围和生长速度不变。只有覆盖范围内等级最高的喷水器计入；其加成会叠加到其他产量加成上。

- 产自：Controls + Panels + Coolant → Sprinkler Pack Mk III
- 部件文档：Sprinkler

铅板 (Lead Plate) `lead_plate`

轧制铅屏蔽板，用于容器、燃料棒和反应堆内部结构。

- 产自：Lead Plate
- 用于：Oxygen Upgrade Pack Mk IV、Heat Upgrade Pack Mk IV、Pressure Upgrade Pack Mk IV、Lead Cask、Shield Plating、Fuel Rod，以及 Nuclear Battery
- 请求方：Helios, Plate Order；Vestibule, Shielded Transport Trial；Helios, Reactor Vessel Chain

数据库 / 物品 (DATABASE / ITEMS)

农业与饲料 (Agriculture & Feeds)

植物和野生动物链的物资：**salt**、制造的种子、**forage**、生物饲料和田地设备套件。

盐 (Salt) **salt**

抽水产生的矿物副产品。水井会在产出淡水的同时产出它。

制种机套件 (Seed Maker Kit) **seed_maker_kit**

用于 **seed_maker** 的制造部署包。从 Inventory 将它部署到运行中的前哨站，然后在 Biosphere, Plants 下管理建筑。

- 产自：**Frame + Control + Panel → Seed Maker Kit**
- 部署：Seed Maker

生长灯套件 (Grow Lamp Kit) **grow_lamp_kit**

用于 **grow_lamp** 的田地套件。将其携带在 Inventory 中，并从移动式 Harvester 部署；部署后的照明提供者会服务 Harvester 网格中周围的格子。

- 产自：**Frame + Panel + Glass → Grow Lamp Kit**
- 部署：Grow Lamp

喷水器套件 (Sprinkler Kit) **sprinkler_kit**

用于 **sprinkler** 的田地套件。将其携带在 Inventory 中，并从移动式 Harvester 部署；部署后的供水者会服务 Harvester 网格中周围的格子。

- 产自：**Frame + Pipes + Valve → Sprinkler Kit**
- 部署：Sprinkler

分配器套件 (Dispenser Kit) `dispenser_kit`

用于 dispenser 的田地套件。将其携带在 Inventory 中，并从移动式 Harvester 部署；部署后的供盐者会服务 Harvester 网格中周围的格子。

- 产自：`Frame + Control + Panel` → `Dispenser Kit`
- 部署：`Dispenser`

废物处理器套件 (Waste Processor Kit) `garbage_disposal_kit`

用于 `garbage_disposal` 的制造部署包，显示为 Waste Processor。从 Inventory 将其部署到运行中的前哨站。其脚本可以永久销毁物品、液体或气体，每次选择一个流。

- 产自：`Frame + Iron + Panel` → `Waste Processor Kit`
- 部署：`Waste Processor`

Sunpetal 种子 (Sunpetal Seed) `seed_sunpetal`

来自 `seed_maker` 的储存 Sunpetal 种系。需要 Light，在满足条件 12 h 后成熟，基础产量为 10 Forage。

Shadeleaf 种子 (Shadeleaf Seed) `seed_shadeleaf`

来自 `seed_maker` 的储存 Shadeleaf 种系。需要 Shade，在满足条件 12 h 后成熟，基础产量为 10 Forage。

Dewmoss 种子 (Dewmoss Seed) `seed_dewmoss`

来自 `seed_maker` 的储存 Dewmoss 种系。需要 Water，在满足条件 12 h 后成熟，基础产量为 10 Forage。

Lonethorn 种子 (Lonethorn Seed) **seed_lonethorn**

来自 **seed_maker** 的储存 Lonethorn 种系。要求所有正交相邻格为空 (正上、正下、正左和正右)，在满足条件 24 h 后成熟，基础产量为 25 Forage。

Packfern 种子 (Packfern Seed) **seed_packfern**

来自 **seed_maker** 的储存 Packfern 种系。要求至少有两个正交相邻的 Packfern (正上、正下、正左或正右)，在满足条件 24 h 后成熟，基础产量为 25 Forage。

Twinvine 种子 (Twinvine Seed) **seed_twinvine**

来自 **seed_maker** 的储存 Twinvine 种系。要求有一个正交相邻的 Dewmoss (正上、正下、正左或正右)，在满足条件 24 h 后成熟，基础产量为 25 Forage。

Spitebud 种子 (Spitebud Seed) **seed_spitebud**

来自 **seed_maker** 的储存 Spitebud 种系。要求没有正交相邻的 Packfern (正上、正下、正左或正右)，在满足条件 24 h 后成熟，基础产量为 25 Forage。

Sunspur 种子 (Sunspur Seed) **seed_sunspur**

来自 **seed_maker** 的储存 Sunspur 种系。需要 Light，且所有正交相邻格为空 (正上、正下、正左和正右)，在满足条件 24 h 后成熟，基础产量为 25 Forage。

Glowvine 种子 (Glowvine Seed) **seed_glowvine**

来自 **seed_maker** 的储存 Glowvine 种系。需要 Light 和 Water，在满足条件 48 h 后成熟，基础产量为 60 Forage。

Crowncap 种子 (Crowncap Seed) **seed_crowncap**

来自 **seed_maker** 的储存 Crowncap 种系。需要 Shade, 且至少有两个正交相邻的 Crowncap (正上、正下、正左或正右), 在满足条件 48 h 后成熟, 基础产量为 60 Forage。

Pondmoss 种子 (Pondmoss Seed) **seed_pondmoss**

来自 **seed_maker** 的储存 Pondmoss 种系。需要 Water, 且至少有两株正交相邻的 Pondmoss (正上、正下、正左或正右), 在满足条件 48 h 后成熟, 基础产量为 60 Forage。

Saltbloom 种子 (Saltbloom Seed) **seed_saltbloom**

来自 **seed_maker** 的储存 Saltbloom 种系。需要 Salt, 在满足条件 72 h 后成熟, 基础产量为 120 Forage。

Brinethorn 种子 (Brinethorn Seed) **seed_brinethorn**

来自 **seed_maker** 的储存 Brinethorn 种系。需要 Salt, 且所有正交相邻格为空 (正上、正下、正左和正右), 在满足条件 72 h 后成熟, 基础产量为 120 Forage。

Saltmate 种子 (Saltmate Seed) **seed_saltmate**

来自 **seed_maker** 的储存 Saltmate 种系。需要 Salt 和一个正交相邻的 Saltbloom (正上、正下、正左或正右), 在满足条件 72 h 后成熟, 基础产量为 120 Forage。

Grandbloom 种子 (Grandbloom Seed) **seed_grandbloom**

来自 **seed_maker** 的储存 Grandbloom 种系。需要 Light、Water, 且所有正交相邻格为空 (正上、正下、正左和正右), 在满足条件 72 h 后成熟, 基础产量为 150 Forage。

植物饲料 (Plant Forage) **forage**

收获并移除成熟植物时获得的实体作物产量。它是所有生物饲料的卡路里基础，也是 Plant Terraformer 批次的主要输入。

• 用于: Plastic + Forage + Water → Reinforced Biopolymer、Biopolymer + Forage + Water → Enrichment Compound、Forage + Sea Algae + Snow Moss → Salt Tortoise Feed、Forage + Lava Algae + Cave Moss → Magmatic Annelid Feed、Forage + Cave/Frost/Coral Fungus → Mycelial Husk Feed、Forage + Magma Crust + Stone Mat → Mantle Strider Feed、Forage + Heat Lichen + Frost Lichen → Glasswing Mantis Feed、Forage + Cold Spores + Hot Spores → Veil Mantle Feed、Forage + Stone Lichen + Salt Crust → Vault Crab Feed、Forage + Brine Plankton + Deep Algae + Tide Moss → Tidal Cephalopod Feed、Forage + Ice Crust + Ash Spores → Bone Walker Feed、Forage + Vent Algae + Steam Moss + Sea Algae → Vent Drifter Feed、Forage + Hot Spores + Crystal Spores → Hive Sentinel Feed、Forage + Cave Moss + Heat Crust → Hollow Choir Feed、Forage + Shore Lichen + Crystal Spores + Vent Fungus → Ferric Sea-Lily Feed、Forage + Stone Mat + Cinder Lichen → Crustal Echo Feed、Forage + Ice Algae + Frost Lichen + Vent Algae + Black Fungus → Glacial Wurm Feed, 以及 Forage + Sulfur Moss + Cinder Lichen + Cold Spores + Stone Lichen → Spire Drake Feed

盐沼龟饲料 (Salt Tortoise Feed) **feed_salt_tortoise**

为沿海物种 Salt Tortoise 压制的饲料。Habitat 殖民地只接受其专属配方。

• 产自: Forage + Sea Algae + Snow Moss → Salt Tortoise Feed

岩浆环节动物饲料 (Magmatic Annelid Feed) **feed_magmatic_annelid**

为火山物种 Magmatic Annelid 压制的饲料。Habitat 殖民地只接受其专属配方。

• 产自: Forage + Lava Algae + Cave Moss → Magmatic Annelid Feed

菌丝壳饲料 (Mycelial Husk Feed) **feed_mycelial_husk**

为深洞穴物种 Mycelial Husk 压制的饲料。Habitat 殖民地只接受其专属配方。

• 产自: Forage + Cave/Frost/Coral Fungus → Mycelial Husk Feed

曼特尔行者饲料 (Mantle Strider Feed) **feed_mantle_strider**

为火山物种曼特尔行者压制的饲料。栖息地殖民地只接受其专用配方。

产出：采集物 + 岩浆地壳 + 石质垫 → 曼特尔行者饲料

玻璃翼螳螂饲料 (Glasswing Mantis Feed) **feed_glasswing_mantis**

为地热物种玻璃翼螳螂压制的饲料。栖息地殖民地只接受其专用配方。

产出：采集物 + 热地衣 + 冰霜地衣 → 玻璃翼螳螂饲料

面纱披风饲料 (Veil Mantle Feed) **feed_veil_mantle**

为冰原物种面纱披风压制的饲料。栖息地殖民地只接受其专用配方。

产出：采集物 + 寒冷孢子 + 炽热孢子 → 面纱披风饲料

宝库蟹饲料 (Vault Crab Feed) **feed_vault_crab**

为深层洞穴物种宝库蟹压制的饲料。栖息地殖民地只接受其专用配方。

产出：采集物 + 石质地衣 + 盐壳 → 宝库蟹饲料

潮汐头足类饲料 (Tidal Cephalopod Feed) **feed_tidal_cephalopod**

为沿海物种潮汐头足类压制的饲料。栖息地殖民地只接受其专用配方。

产出：采集物 + 盐水浮游生物 + 深水藻类 + 潮汐苔藓 → 潮汐头足类饲料

骨行者饲料 (Bone Walker Feed) **feed_bone_walker**

为冰原物种骨行者压制的饲料。栖息地殖民地只接受其专用配方。

产出：采集物 + 冰壳 + 灰烬孢子 → 骨行者饲料

喷口漂流者饲料 (Vent Drifter Feed) **feed_vent_drifter**

为地热物种喷口漂流者压制的饲料。栖息地殖民地只接受其专用配方。

产出：采集物 + 喷口藻类 + 蒸汽苔藓 + 海藻 → 喷口漂流者饲料

蜂巢哨兵饲料 (Hive Sentinel Feed) **feed_hive_sentinel**

为地热物种蜂巢哨兵压制的饲料。栖息地殖民地只接受其专用配方。

产出：采集物 + 炽热孢子 + 晶体孢子 → 蜂巢哨兵饲料

空洞合唱饲料 (Hollow Choir Feed) **feed_hollow_choir**

为深层洞穴物种空洞合唱压制的饲料。栖息地殖民地只接受其专用配方。

产出：采集物 + 洞穴苔藓 + 热地壳 → 空洞合唱饲料

铁质海百合饲料 (Ferric Sea-Lily Feed) **feed_ferric_sea_lily**

为沿海物种铁质海百合压制的饲料。栖息地殖民地只接受其专用配方。

产出：采集物 + 岸边地衣 + 晶体孢子 + 喷口真菌 → 铁质海百合饲料

地壳回声饲料 (Crustal Echo Feed) **feed_crustal_echo**

为深层洞穴物种地壳回声压制的饲料。栖息地殖民地只接受其专用配方。

产出：采集物 + 石质垫 + 灰烬地衣 → 地壳回声饲料

冰川巨龙饲料 (Glacial Wyrn Feed) **feed_glacial_wyrn**

为冰原物种冰川巨龙压制的饲料。栖息地殖民地只接受其专用配方。

产出：采集物 + 冰藻 + 冰霜地衣 + 喷口藻类 + 黑色真菌 → 冰川巨龙饲料

尖塔龙饲料 (Spire Drake Feed) **feed_spire_drake**

为火山物种尖塔龙压制的饲料。栖息地殖民地只接受其专用配方。

产出：采集物 + 硫磺苔藓 + 灰烬地衣 + 寒冷孢子 + 石质地衣 → 尖塔龙饲料

数据库 / 物品

生命形态

在五种生物群系中采集的本土生命形态。它们会在精华液化器（essence_liquifier）中液化为生物群系精华，并用于生物饲料配方。

冰藻（Ice Algae） ice_algae

冻结在融水薄膜中的苍白藻带，在冰层下泛着微光。是冰原上最常见的收获物。

生物群系	冰原（Frozen）
稀有度	常见（Common）
用途	采集物 + 冰藻 + 冰霜地衣 + 喷口藻类 + 黑色真菌 → 冰川巨龙饲料

雪苔藓（Snow Moss） snow_moss

覆盖在漂移雪迎风面的低矮苔藓垫，随每阵风起伏。凡是积雪并保持不融的地方，它都会茂密生长。

生物群系	冰原（Frozen）
稀有度	常见（Common）
用途	采集物 + 海藻 + 雪苔藓 → 盐龟饲料

冰霜地衣（Frost Lichen） frost_lichen

结壳地衣在日光下变淡，在漫长夜晚中变暗。它会附着在所有裸露的冰冻岩石上。

生物群系	冰原（Frozen）
稀有度	常见（Common）
用途	采集物 + 热地衣 + 冰霜地衣 → 玻璃翼螳螂饲料；采集物 + 冰藻 + 冰霜地衣 + 喷口藻类 + 黑色真菌 → 冰川巨龙饲料

寒冷孢子（Cold Spores）cold_spores

休眠的孢子簇通过完全停止活动来抵御严寒。其精华密度远高于地表苔垫。

生物群系	冰原（Frozen）
稀有度	不常见（Uncommon）
用途	采集物 + 寒冷孢子 + 炽热孢子 → 面纱披风饲料；采集物 + 硫磺苔藓 + 灰烬地衣 + 寒冷孢子 + 石质地衣 → 尖塔龙饲料

冰壳（Ice Crust）ice_crust

与冰本身熔合的活体地壳，可剥离成易碎薄片。生长缓慢，但也因此更加丰厚。

生物群系	冰原（Frozen）
稀有度	不常见（Uncommon）
用途	采集物 + 冰壳 + 灰烬孢子 → 骨行者饲料

冰霜真菌（Frost Fungus）frost_fungus

只有在最深的严寒中，稀有子实体才会从冰层下顶出。是冰原生物群系最密集的精华来源。

生物群系	冰原（Frozen）
稀有度	稀有（Rare）
用途	采集物 + 洞穴/冰霜/珊瑚真菌 → 菌丝外壳饲料

海藻（Sea Algae） `sea_algae`

在浅滩中漂流的长条藻带。是沿海地区的主要收获物，沿每条潮线都很丰富。

生物群系	沿海（Coastal）
稀有度	常见（Common）
用途	采集物 + 海藻 + 雪苔藓 → 盐龟饲料；采集物 + 喷口藻类 + 蒸汽苔藓 + 海藻 → 喷口漂流者饲料

潮汐苔藓（Tide Moss） `tide_moss`

铺满潮滩的弹性苔藓，在两次潮汐之间被风梳平。

生物群系	沿海（Coastal）
稀有度	常见（Common）
用途	采集物 + 盐水浮游生物 + 深水藻类 + 潮汐苔藓 → 潮汐头足类饲料

岸边地衣（Shore Lichen） `shore_lichen`

耐盐地衣在飞溅带形成条带，在强光下会漂白得更加明亮。

生物群系	沿海（Coastal）
稀有度	常见（Common）
用途	采集物 + 岸边地衣 + 晶体孢子 + 喷口真菌 → 铁质海百合饲料

盐水浮游生物（Brine Plankton） `brine_plankton`

在浓盐水中大量繁殖的发光浮游生物。繁殖群漂过时，水面会泛起绿色光芒。

生物群系	沿海（Coastal）
稀有度	不常见（Uncommon）
用途	采集物 + 盐水浮游生物 + 深水藻类 + 潮汐苔藓 → 潮汐头足类饲料

盐壳（Salt Crust） salt_crust

在蒸发盐滩中生长的活体地壳，可剥离成苍白板片。

生物群系	沿海（Coastal）
稀有度	不常见（Uncommon）
用途	采集物 + 石质地衣 + 盐壳 → 宝库蟹饲料

珊瑚真菌（Coral Fungus） coral_fungus

分枝状真菌生长物，沿避风海岸模拟珊瑚形态。是沿海生物群系最丰厚的收获物。

生物群系	沿海（Coastal）
稀有度	稀有（Rare）
用途	采集物 + 洞穴/冰霜/珊瑚真菌 → 菌丝外壳饲料

喷口藻类（Vent Algae） vent_algae

喜热藻类在喷口周围的温暖涌流中延展。凡是地面灼热之处都能繁茂生长。

生物群系	地热（Geothermal）
稀有度	常见（Common）
用途	采集物 + 喷口藻类 + 蒸汽苔藓 + 海藻 → 喷口漂流者饲料；采集物 + 冰藻 + 冰霜地衣 + 喷口藻类 + 黑色真菌 → 冰川巨龙饲料

蒸汽苔藓（Steam Moss） steam_moss

直接生长在喷口羽流中的苔藓，随上升气流摇摆。喷口喷吐越强烈的地方，它越湿润。

生物群系	地热（Geothermal）
稀有度	常见（Common）
用途	采集物 + 喷口藻类 + 蒸汽苔藓 + 海藻 → 喷口漂流者饲料

热地衣 (Heat Lichen) `heat_lichen`

覆盖喷口地带温暖岩石的地衣，在直射光下会变亮。

生物群系	地热 (Geothermal)
稀有度	常见 (Common)
用途	采集物 + 热地衣 + 冰霜地衣 → 玻璃翼螳螂饲料

炽热孢子 (Hot Spores) `hot_spores`

需要喷口热量才能开启的孢子簇。精华高度浓缩，封存着等待下一次热脉冲。

生物群系	地热 (Geothermal)
稀有度	不常见 (Uncommon)
用途	采集物 + 寒冷孢子 + 炽热孢子 → 面纱披风饲料；采集物 + 炽热孢子 + 晶体孢子 → 蜂巢哨兵饲料

热地壳 (Heat Crust) `heat_crust`

生长在喷口岩石上的含矿地壳，可敲下温暖板片。

生物群系	地热 (Geothermal)
稀有度	不常见 (Uncommon)
用途	采集物 + 洞穴苔藓 + 热地壳 → 空洞合唱饲料

喷口真菌 (Vent Fungus) `vent_fungus`

扎根于喷口喉部的稀有子实体。是地热生物群系最密集的精华来源。

生物群系	地热 (Geothermal)
稀有度	稀有 (Rare)
用途	采集物 + 岸边地衣 + 晶体孢子 + 喷口真菌 → 铁质海百合饲料

硫磺苔藓（Sulfur Moss） `sulfur_moss`

染成黄色的苔藓铺在冷却的流体上，在灰烬风中不断变换形状。

生物群系	火山（Volcanic）
稀有度	常见（Common）
用途	采集物 + 硫磺苔藓 + 灰烬地衣 + 寒冷孢子 + 石质地衣 → 尖塔龙饲料

灰烬地衣（Cinder Lichen） `cinder_lichen`

顽强的地衣在新鲜煤渣地带扎根，光线最强处的颜色最浅。

生物群系	火山（Volcanic）
稀有度	常见（Common）
用途	采集物 + 石质垫 + 灰烬地衣 → 地壳回声饲料；采集物 + 硫磺苔藓 + 灰烬地衣 + 寒冷孢子 + 石质地衣 → 尖塔龙饲料

灰烬孢子（Ash Spores） `ash_spores`

孢子簇随落灰飘落，在所到之处沉积。这是火山地区的主要收获物。

生物群系	火山（Volcanic）
稀有度	常见（Common）
用途	采集物 + 冰壳 + 灰烬孢子 → 骨行者饲料

熔岩藻（Lava Algae） `lava_algae`

生活在冷却流边缘的藻带，在黑色岩石映衬下泛着微光。

生物群系	火山（Volcanic）
稀有度	不常见（Uncommon）
用途	采集物 + 熔岩藻 + 洞穴苔藓 → 岩浆环节动物饲料

岩浆地壳（Magma Crust） `magma_crust`

与冷却岩浆黏合的活体地壳，可掰下厚重板片。

生物群系	火山（Volcanic）
稀有度	不常见（Uncommon）
用途	采集物 + 岩浆地壳 + 石质垫 → 曼特尔行者饲料

黑色真菌（Black Fungus） `black_fungus`

扎根于火山玻璃中的稀有黑色子实体。是火山生物群系最丰厚的精华来源。

生物群系	火山（Volcanic）
稀有度	稀有（Rare）
用途	采集物 + 冰藻 + 冰霜地衣 + 喷口藻类 + 黑色真菌 → 冰川巨龙饲料

洞穴苔藓（Cave Moss） `cave_moss`

苍白苔藓铺满深层洞穴地面，被通道中的缓慢气流轻轻搅动。

生物群系	深层（Deep）
稀有度	常见（Common）
用途	采集物 + 熔岩藻 + 洞穴苔藓 → 岩浆环节动物饲料；采集物 + 洞穴苔藓 + 热地壳 → 空洞合唱饲料

石质地衣（Stone Lichen） `stone_lichen`

地衣覆盖着裸露的地下岩石，会对仅有的一点光线产生反应。

生物群系	深层（Deep）
稀有度	常见（Common）
用途	采集物 + 石质地衣 + 盐壳 → 宝库蟹饲料；采集物 + 硫磺苔藓 + 灰烬地衣 + 寒冷孢子 + 石质地衣 → 尖塔龙饲料

晶体孢子（Crystal Spores）`crystal_spores`

闪烁的孢子簇会在黑暗中捕捉并重新发出光线。这是深层生物群系的常见收获物。

生物群系	深层（Deep）
稀有度	常见（Common）
用途	采集物 + 炽热孢子 + 晶体孢子 → 蜂巢哨兵饲料；采集物 + 岸边地衣 + 晶体孢子 + 喷口真菌 → 铁质海百合饲料

深水藻类（Deep Algae）`deep_algae`

发光藻带生长在静止的地下水中，照亮它们生长的水池。

生物群系	深层（Deep）
稀有度	不常见（Uncommon）
用途	采集物 + 盐水浮游生物 + 深水藻类 + 潮汐苔藓 → 潮汐头足类饲料

石质垫（Stone Mat）`stone_mat`

致密的活体垫层与深层岩石熔合，可切割成板块。

生物群系	深层（Deep）
稀有度	不常见（Uncommon）
用途	采集物 + 岩浆地壳 + 石质垫 → 曼特尔行者饲料；采集物 + 石质垫 + 灰烬地衣 → 地壳回声饲料

洞穴真菌（Cave Fungus）`cave_fungus`

稀有发光真菌在最深处的洞室中结出子实体。是深层生物群系最密集的精华来源。

生物群系	深层（Deep）
稀有度	稀有（Rare）
用途	采集物 + 洞穴/冰霜/珊瑚真菌 → 菌丝外壳饲料

数据库 / 物品

野外资源

采集者在野外收集的散落资源。距离基地越远，价值越高。

土壤样本 (Soil Sample) **soil_sample**

一勺本土风化层，是采集者最常捡到的东西。

售价：50 cr

基础植物 (Basic Plant) **basic_plant**

耐寒的本土植物标本，在地表各处都很常见。

售价：125 cr

有机物 (Organic Matter) **organic_matter**

腐烂的生物材料；更丰厚的发现物位于稍远处。

售价：250 cr

矿物碎片 (Mineral Fragment) **mineral_fragment**

结晶化的矿物碎块，具有可观的销售价值。

售价：425 cr

稀有真菌 (Rare Fungi) **rare_fungi**

在中环区域发现的不常见真菌生长物。

售价：850 cr

晶体碎片 (Crystal Shard) **crystal_shard**

来自外环区域的珍贵晶体构造。

售价：1,700 cr

外星化石 (Alien Fossil) **alien_fossil**

远古行星生命留下的保存痕迹，回到地球后是收藏家的珍品。

售价：2,500 cr

奇异化合物 (Exotic Compound) **exotic_compound**

最稀有的野外发现物：一种尚未分类的化合物，地球实验室愿意为它支付高价。

售价：5,000 cr

数据库 / 物品

生物学样本

从野外回收并在生物实验室（Bio Lab）中分析的生物碎片。生物订单（Bio Orders）会按名称请求这些样本。

颅板（Cranial Plate） **gw_cranial_plate**

在低温静滞中保存的头骨额骨板。两个眼眶位于中央额嵴两侧；眉冠尖刺完好。

稀有度：os frontale glaciatum, 传奇（Legendary）

生物群系：冰原（Frozen）

请求方：顶级标本档案、诺克图娜冰原汇编、冰川巨龙饲养档案

分析地点：生物实验室（Bio Lab）

尾鳍（Caudal Fin） **gw_caudal_fin**

从咸淡水冲刷带回收的尾鳍膜。附着节点被盐分侵蚀，藤壶群落紧附其上。

稀有度：pinna caudalis pelagica, 传奇（Legendary）

生物群系：沿海（Coastal）

请求方：红树林组织托运单、大陆架顶点项目、沿海大汇编

分析地点：生物实验室（Bio Lab）

心脏节点 (Cardiac Node) **gw_cardiac_node**

浸入矿物盐的心肌。冠状血管中流淌着钙化金，中央节点仍保留残余热荷。

稀有度：cor mineralizatum，传奇 (Legendary)

生物群系：地热 (Geothermal)

请求方：烧结组织托运单、地热场顶点项目、地热大汇编

分析地点：生物实验室 (Bio Lab)

颌牙 (Jaw Fang) **gw_jaw_fang**

从冷却熔岩流中回收的捕食者獠牙。一侧覆有黑曜石；内部裂缝仍在喷出余烬热量。

稀有度：dens praedatorius，传奇 (Legendary)

生物群系：火山 (Volcanic)

请求方：火成组织托运单、喷发区顶点项目、火山大汇编

分析地点：生物实验室 (Bio Lab)

脊椎 (Spinal Vertebra) **gw_spinal_vertebra**

来自深海平原、因压力熔合的椎骨。生物发光残留物仍在神经管中脉动，DNA 活性尚未测试。

稀有度：vertebra abyssalis，传奇 (Legendary)

生物群系：深层 (Deep)

请求方：无光组织托运单、深沟顶点项目、深渊大汇编

分析地点：生物实验室 (Bio Lab)

背甲 (Dorsal Carapace) `vc_dorsal_carapace`

保存在冰川中的分节背壳。中央脊和侧向棘刺完好；节间膜已矿化。

稀有度：scutum dorsale glaciatum, 稀有 (Rare)

生物群系：冰原 (Frozen)

请求方：冰冻礁体普查、冰川化指数

分析地点：生物实验室 (Bio Lab)

下颌爪 (Mandible Claw) `vc_mandible_claw`

从咸淡水冲刷带回收的钳状螯。锯齿状咬合边缘完好；藤壶群落与腕部关节熔合。

稀有度：chela cancrida, 稀有 (Rare)

生物群系：沿海 (Coastal)

请求方：河口测定、破浪带生物调查

分析地点：生物实验室 (Bio Lab)

触角簇 (Antenna Cluster) `vc_antenna_cluster`

来自单个甲壳安装点的感觉触角束。五根鞭毛在关节处被矿物熔合；化学感受器已经钙化。

稀有度：flagella sensoria, 稀有 (Rare)

生物群系：地热 (Geothermal)

请求方：矿物地壳测定、喷气孔生物调查

分析地点：生物实验室 (Bio Lab)

步行足关节 (Walking Leg Joint) **vc_walking_leg**

从熔岩流中回收的关节式步行足。基节完好；胫节-趾节连接处覆有黑曜石。余热热量仍沿裂缝渗出。

稀有度：pereiopodium articulatum, 稀有 (Rare)

生物群系：火山 (Volcanic)

请求方：熔岩管测定、火山泥流生物调查

分析地点：生物实验室 (Bio Lab)

眼柄 (Eye-Stalk) **vc_eye_stalk**

分节眼柄上的生物发光复眼。回收自深海平原，晶面保存完好，神经核心仍发出微弱脉冲。

稀有度：ommatophorus profundus, 稀有 (Rare)

生物群系：深层 (Deep)

请求方：冷泉测定、远洋生物调查

分析地点：生物实验室 (Bio Lab)

头盖 (Cranium) **oc_cranium**

从冰川中回收的球状外套膜头骨。软骨骨架保存完好；中央窦腔仍留有残余墨汁。

稀有度：cranium cephalopodi, 稀有 (Rare)

生物群系：冰原 (Frozen)

请求方：头足类组织请求

分析地点：生物实验室 (Bio Lab)

触手臂 (Tentacle Arm) **oc_tentacle_arm**

关节式触手，沿长度方向排列着两排吸盘。近端边缘可见沿海盐蚀痕迹。

稀有度：brachium tentaculi, 稀有 (Rare)

生物群系：沿海 (Coastal)

请求方：泻湖标本研究、红树林组织托运单

分析地点：生物实验室 (Bio Lab)

几丁质喙 (Chitin Beak) **oc_chitin_beak**

两片鹦鹉式几丁质下颌在单个关节处相对连接。切割边缘完好；边缘覆有矿物地壳。

稀有度：rostrum chitinosum, 稀有 (Rare)

生物群系：地热 (Geothermal)

请求方：蒸汽场标本研究、烧结组织托运单

分析地点：生物实验室 (Bio Lab)

墨囊 (Ink Sac) **oc_ink_sac**

保留喷射管的外套膜墨汁储囊。一侧因火山暴露而留下灼烧痕迹。

稀有度：saccus atramenti, 稀有 (Rare)

生物群系：火山 (Volcanic)

请求方：煤渣标本研究、火成组织托运单

分析地点：生物实验室 (Bio Lab)

相机镜头眼 (Camera Lens Eye) **oc_lens_eye**

来自深层种群的半球形镜头眼。视网膜层仍会发出微弱的生物发光脉冲。

稀有度：oculus camera, 稀有 (Rare)

生物群系：深层 (Deep)

请求方：盐水池标本研究、无光组织托运单

分析地点：生物实验室 (Bio Lab)

头骨 (Skull) **bw_skull**

细长的双足头骨，具有成对眼眶和向后弯曲的颌部。颅顶遍布冰霜侵蚀痕迹。

稀有度：cranium ossarium, 稀有 (Rare)

生物群系：冰原 (Frozen)

请求方：捕食者对比套件、顶级标本档案、骨行者觅食索引、低温流体精炼协议

分析地点：生物实验室 (Bio Lab)

前爪 (Foreclaw) **bw_foreclaw**

来自小型前肢的钩状抓握爪。内弯处被盐侵蚀；几丁质鞘部分完好。

稀有度：manus rapax, 稀有 (Rare)

生物群系：沿海 (Coastal)

请求方：冲浪带目录、藤壶碎片清单

分析地点：生物实验室 (Bio Lab)

肋笼段 (Ribcage Segment) **bw_ribcage**

三对相互连接的肋骨与一段椎骨相连。矿物钙化物填满肋间隙。

稀有度: thorax costarum, 稀有 (Rare)

生物群系: 地热 (Geothermal)

请求方: 火山口边缘目录、热液碎片清单

分析地点: 生物实验室 (Bio Lab)

后肢骨 (Hindlimb Bone) **bw_hindlimb**

来自主要步行足的强健趾行动物股骨和胫骨。余烬灼烧疤痕沿承重轴延伸。

稀有度: femur digitigradi, 稀有 (Rare)

生物群系: 火山 (Volcanic)

请求方: 火山碎屑目录、火山碎片清单

分析地点: 生物实验室 (Bio Lab)

尾刺 (Tail Spike) **bw_tail_spike**

带侧向倒刺的分节锥形尾刺。骨髓通道中仍带有生物发光残留物。

稀有度: spiculum caudale, 稀有 (Rare)

生物群系: 深层 (Deep)

请求方: 化能合成目录、超深海碎片清单

分析地点: 生物实验室 (Bio Lab)

钟形膜 (Bell Membrane) **vd_bell**

保留径向管道图案的半透明钟形膜。背侧顶端覆有霜状结晶。

- 稀有度: **calyptra bellaria**, 稀有 (Rare)
- 生境: 冰冻 (Frozen)
- 请求方: 冰冻礁脉普查 (Frozen Reef Census) 与冷冻流体提纯协议 (Cryofluid Refinement Protocol)
- 分析地点: 生物实验室 (Bio Lab)

刺丝囊环 (Nematocyst Ring) **vd_nematocyst**

来自钟缘的刺细胞簇。虽已被盐分侵蚀，发射囊仍保留毒液载荷。

- 稀有度：**cnidocytus ringens**，稀有 (Rare)
- 生境：海岸 (Coastal)
- 请求方：冲浪带目录 (Surf-Zone Catalog) 与大陆架顶点项目 (Continental-Shelf Capstone)
- 分析地点：生物实验室 (Bio Lab)

神经网络 (Neural Mesh) **vd_neural_mesh**

来自钟体内部的分支状神经网络。加热时神经节节点仍会发光；神经干上沉积着矿物外壳。

- 稀有度：**rete nervosum**，稀有 (Rare)
- 生境：地热 (Geothermal)
- 请求方：火山口边缘目录 (Caldera-Rim Catalog) 与地热田顶点项目 (Geothermal-Field Capstone)
- 分析地点：生物实验室 (Bio Lab)

光器官 (Photophore) **vd_photophore**

来自钟壁的发光器官。暴露在空气中时，带有余烬灼痕的透镜仍会发出微光。

- 稀有度：**photophorum luminosum**, 稀有 (Rare)
- 生境：火山 (Volcanic)
- 请求方：火山灰目录 (Tephra Catalog) 与喷发带顶点项目 (Eruption-Zone Capstone)
- 分析地点：生物实验室 (Bio Lab)

拖曳触须 (Trailing Tendril) **vd_tendrill**

末端尖端带有光器官珠的长条拖曳触须。近端节段沿长度方向发生了压力熔合。

- 稀有度：**tendrillum nematocum**, 稀有 (Rare)
- 生境：深海 (Deep)
- 请求方：化能合成目录 (Chemosynthetic Catalog) 与深海沟顶点项目 (Deep-Trench Capstone)
- 分析地点：生物实验室 (Bio Lab)

子实体 (Fruiting Body) **mh_fruiting_body**

菌盖的菌幕组织完好，冠部覆有霜晶。孢子印在低温保存条件下仍具活性。

- 稀有度：**basidiocarpus glaciatus**, 罕见 (Uncommon)
- 生境：冰冻 (Frozen)
- 请求方：冷生植物孢子组 (Cryophyte Spore Panel)、诺克图尔纳冰冻纲要 (Nocturna Frozen Compendium) 与骨行者觅食索引 (Bone Walker Forage Index)
- 分析地点：生物实验室 (Bio Lab)

孢子荚 (Spore Pod) mh_spore_pod

悬挂的孢子荚，附着着菌丝锚定丝。底面有被盐分侵蚀的释放孔。

- 稀有度：sporangium dependens, 罕见 (Uncommon)
- 生境：海岸 (Coastal)
- 请求方：盐水标本组 (Brine Specimen Panel)、泻湖标本研究 (Lagoon Specimen Study) 与红树林组织托运单 (Mangrove Tissue Consignment)
- 分析地点：生物实验室 (Bio Lab)

菌丝根 (Mycelium Root) mh_mycelium_root

分支状菌丝根节段，隔膜分隔仍然完整。矿物外壳覆盖着根干连接处。

- 稀有度：hyphae radicales, 罕见 (Uncommon)
- 生境：地热 (Geothermal)
- 请求方：硫标本组 (Sulfur Specimen Panel)、蒸汽场标本研究 (Steam-Field Specimen Study) 与烧结组织托运单 (Sinter Tissue Consignment)
- 分析地点：生物实验室 (Bio Lab)

几丁质节点 (Chitin Node) mh_chitin_node

来自菌丝网络的硬化隔膜节点。余烬灼烧形成的陨坑状伤痕遍布几丁质加固带。

- 稀有度：nodus septalis, 罕见 (Uncommon)
- 生境：火山 (Volcanic)
- 请求方：降灰标本组 (Ashfall Specimen Panel)、火山渣标本研究 (Cinder Specimen Study) 与火成组织托运单 (Igneous Tissue Consignment)
- 分析地点：生物实验室 (Bio Lab)

柱头盘 (Stigmatic Disc) mh_stigmatic_disc

来自菌盖顶端的感觉盘，是该物种具有诊断意义的“眼”结构。生物发光核心仍在微弱脉动。

- 稀有度: stigma sapientiae, 罕见 (Uncommon)
- 生境: 深海 (Deep)
- 请求方: 海沟标本组 (Trench Specimen Panel)、盐水池标本研究 (Brine-Pool Specimen Study) 与无光组织托运单 (Aphotic Tissue Consignment)
- 分析地点: 生物实验室 (Bio Lab)

下颚 (Mandible) hs_mandible

钩状捕食者下颚，切缘排列着五枚锯齿状牙。几丁质凹坑沿线附着霜晶。

- 稀有度: mandibula chitinsa, 稀有 (Rare)
- 生境: 冰冻 (Frozen)
- 请求方: 捕食者对照集 (Predator Comparison Set)、顶级标本档案 (Apex Specimen Dossier) 与冰川巨龙饲养档案 (Glacial Wyrms Husbandry Dossier)
- 分析地点: 生物实验室 (Bio Lab)

前翼膜 (Forewing Membrane) hs_wing_membrane

前翼膜保留完整的前缘脉与横脉细胞图案。盐蚀已在数处磨出针孔。

- 稀有度: membrana alaria, 稀有 (Rare)
- 生境: 海岸 (Coastal)
- 请求方: 海带森林碎片批次 (Kelp-Forest Fragment Batch) 与海岸大纲要 (Coastal Grand Compendium)
- 分析地点: 生物实验室 (Bio Lab)

胸部甲片 (Thorax Plate) **hs_thorax_plate**

宽大的装甲胸部节段，带有三对腿部关节窝和两对翼基关节窝。甲片上分布着矿物金色斑块。

- 稀有度：**scutum thoracicum**, 稀有 (Rare)
- 生境：地热 (Geothermal)
- 请求方：泥锅碎片批次 (Mud-Pot Fragment Batch) 与地热大纲要 (Geothermal Grand Compendium)
- 分析地点：生物实验室 (Bio Lab)

腹部节段 (Abdomen Segment) **hs_abdomen_segment**

带环纹的腹部节段，具有诊断性的黄黑图案。一侧留有余烬灼痕。

- 稀有度：**segmentum abdominis**, 稀有 (Rare)
- 生境：火山 (Volcanic)
- 请求方：火山渣碎片批次 (Scoria Fragment Batch) 与火山大纲要 (Volcanic Grand Compendium)
- 分析地点：生物实验室 (Bio Lab)

复眼 (Compound Eye) **hs_compound_eye**

半球形复眼，完整保留六边形小眼面网格。透镜仍存有残余生物发光。

- 稀有度：**oculus compositus**, 稀有 (Rare)
- 生境：深海 (Deep)
- 请求方：鲸落碎片批次 (Whale-Fall Fragment Batch) 与深渊大纲要 (Abyssal Grand Compendium)
- 分析地点：生物实验室 (Bio Lab)

螯肢 (Chelicera) ms_chelicera

可活动的颚状口器，带有弯曲的皮下注射式毒牙。毒液通道贯穿毒牙全长。

- 稀有度：chelicera venenifera，罕见 (Uncommon)
- 生境：冰冻 (Frozen)
- 请求方：节肢动物配对研究 (Arthropod Pair Study) 与冷冻流体提纯协议 (Cryofluid Refinement Protocol)
- 分析地点：生物实验室 (Bio Lab)

足跗节 (Leg Tarsus) ms_leg_tarsus

三段远端腿节依次为股节、胫节和跗节，末端是相对的钩爪。感觉刚毛完好。

- 稀有度：tarsus chelifer，罕见 (Uncommon)
- 生境：海岸 (Coastal)
- 请求方：滨岸组织批次 (Littoral Tissue Lot)、海带森林碎片批次 (Kelp-Forest Fragment Batch) 与藤壶碎片清单 (Barnacle Fragment Manifest)
- 分析地点：生物实验室 (Bio Lab)

脚须 (Pedipalp) ms_pedipalp

头部前方的感觉附肢，末端是球状脚须器官。基部节段覆有矿物钙化板。

- 稀有度：pedipalpus sensorius，罕见 (Uncommon)
- 生境：地热 (Geothermal)
- 请求方：喷气孔组织批次 (Fumarole Tissue Lot)、泥锅碎片批次 (Mud-Pot Fragment Batch) 与热液碎片清单 (Hydrothermal Fragment Manifest)
- 分析地点：生物实验室 (Bio Lab)

腹部硬片 (Abdomen Sclerite) `ms_abdomen_sclerite`

来自后体的背侧装甲板，带有三个 V 形节段。中线沿线布满余烬灼烧形成的陨坑。

- 稀有度：`sclerites dorsalis`，罕见 (Uncommon)
- 生境：火山 (Volcanic)
- 请求方：黑曜石组织批次 (Obsidian Tissue Lot)、火山渣碎片批次 (Scoria Fragment Batch) 与火山碎片清单 (Volcanic Fragment Manifest)
- 分析地点：生物实验室 (Bio Lab)

眼簇 (Eye Cluster) `ms_eye_cluster`

头胸部背侧甲片，保留全部八只单眼：两只大型主眼和六只次眼。

- 稀有度：`ocelli octuplae`，罕见 (Uncommon)
- 生境：深海 (Deep)
- 请求方：底栖组织批次 (Benthic Tissue Lot)、鲸落碎片批次 (Whale-Fall Fragment Batch) 与超深渊碎片清单 (Hadopelagic Fragment Manifest)
- 分析地点：生物实验室 (Bio Lab)

孔缘唇 (Aperture Lip) `hc_aperture_lip`

体腔的月牙形外缘，软组织曾从此处伸出。珍珠层内壁完好；虹吸管缺口保存完整。

- 稀有度：`labium nacre`，稀有 (Rare)
- 生境：冰冻 (Frozen)
- 请求方：冰川作用索引 (Glaciation Index)
- 分析地点：生物实验室 (Bio Lab)

壳旋 (Shell Whorl) `hc_shell_whorl`

部分螺旋碎片，包含一个完整壳旋以及下一个壳旋的起始部分。具有诊断意义的波状缝合线清晰可见。

- 稀有度：`verticillus conchae`，稀有 (Rare)
- 生境：海岸 (Coastal)
- 请求方：浅滩标本库 (Shoal Specimen Cache) 与海岸大纲要 (Coastal Grand Compendium)
- 分析地点：生物实验室 (Bio Lab)

隔板 (Septum Plate) `hc_septum_plate`

内部腔室壁，呈凹形碟状，具有诊断性的褶边缝合缘和中央虹吸管孔。

- 稀有度：`septum chambrae`，稀有 (Rare)
- 生境：地热 (Geothermal)
- 请求方：石灰华标本库 (Travertine Specimen Cache) 与地热大纲要 (Geothermal Grand Compendium)
- 分析地点：生物实验室 (Bio Lab)

钩喙 (Hooked Beak) `hc_beak`

中央口部的一对几丁质颚，呈鸚鵡式钩状，在单个铰链处连接。

- 稀有度：`rhynchos sireni`，稀有 (Rare)
- 生境：火山 (Volcanic)
- 请求方：浮石标本库 (Pumice Specimen Cache) 与火山大纲要 (Volcanic Grand Compendium)
- 分析地点：生物实验室 (Bio Lab)

触手冠 (Tentacle Crown) `hc_tentacle_crown`

六条可活动触手汇聚于中央枢纽，每条触手末端都有一个生物发光器官。

- 稀有度：`corona brachialis`，稀有 (Rare)
- 生境：深海 (Deep)
- 请求方：俯冲带标本库 (Subduction Specimen Cache) 与深渊大纲要 (Abyssal Grand Compendium)
- 分析地点：生物实验室 (Bio Lab)

头部乳突 (Cranial Papilla) `ma_cranial_papilla`

头部前段，五个感觉乳突环绕末端口部排列。节段内的生物发光环处于活跃状态。

- 稀有度：`prostomium sensorium`，普通 (Common)
- 生境：冰冻 (Frozen)
- 请求方：霜冻组织样本 (Frost Tissue Sample)、土壤富集测定 (Soil Enrichment Assay) 与冰冻组织托运单 (Frozen Tissue Consignment)
- 分析地点：生物实验室 (Bio Lab)

角质层蜕皮 (Cuticle Molt) `ma_cuticle_molt`

玻璃般外层角质层脱落的节段，带有五道环形纹。背面遍布盐蚀穿孔。

- 稀有度：`exuvium vitreum`，普通 (Common)
- 生境：海岸 (Coastal)
- 请求方：潮汐组织样本 (Tidal Tissue Sample)、珊瑚生物纲要 (Coral Bio Compendium)、河口测定 (Estuary Assay) 与海带森林碎片批次 (Kelp-Forest Fragment Batch)
- 分析地点：生物实验室 (Bio Lab)

神经节节点 (Ganglion Node) `ma_ganglion_node`

双侧神经节，两个叶由横向连合连接，四条连接支完好。核心仍保留生物发光。

- 稀有度：`ganglion paramedianum`，普通 (Common)
- 生境：地热 (Geothermal)
- 请求方：喷口组织样本 (Vent Tissue Sample)、热生物纲要 (Thermal Bio Compendium)、矿物外壳测定 (Mineral-Crust Assay) 与泥锅碎片批次 (Mud-Pot Fragment Batch)
- 分析地点：生物实验室 (Bio Lab)

几丁质刚毛 (Chitinous Seta) `ma_chitinous_seta`

疣足刚毛束，五根几丁质针从肉质基叶向外辐射。带倒钩的末端完好。

- 稀有度：`setae parapodialis`，普通 (Common)
- 生境：火山 (Volcanic)
- 请求方：玄武岩组织样本 (Basalt Tissue Sample)、火山口生物纲要 (Caldera Bio Compendium)、熔岩管测定 (Lava-Tube Assay) 与火山渣碎片批次 (Scoria Fragment Batch)
- 分析地点：生物实验室 (Bio Lab)

发光环 (Luminous Ring) `ma_luminous_ring`

身体节段横截面，保留具有诊断意义的节段中部生物发光环和侧向疣足。

- 稀有度：`annulus luminosus`，普通 (Common)
- 生境：深海 (Deep)
- 请求方：深渊组织样本 (Abyssal Tissue Sample)、深水带生物纲要 (Bathic Bio Compendium)、冷泉测定 (Cold-Seep Assay) 与鲸落碎片批次 (Whale-Fall Fragment Batch)
- 分析地点：生物实验室 (Bio Lab)

棱晶眼 (Prismatic Eye) gm_compound_eye

半球形复眼，完整保留六边形小眼面网格。透镜折射仍会产生棱晶般的光泽。

- 稀有度: **oculus crystallinus**, 罕见 (Uncommon)
- 生境: 冰冻 (Frozen)
- 请求方: 节肢动物配对研究 (Arthropod Pair Study)、捕食者对照集 (Predator Comparison Set) 与冰川巨龙饲养档案 (Glacial Wyrms Husbandry Dossier)
- 分析地点: 生物实验室 (Bio Lab)

折叠翼 (Folded Wing) gm_folded_wing

沿中央折痕折叠一次的膜质前翼，即静止构型。前缘脉保存完好。

- 稀有度: **ala plicata**, 罕见 (Uncommon)
- 生境: 海岸 (Coastal)
- 请求方: 珊瑚生物纲要 (Coral Bio Compendium) 与浅滩标本库 (Shoal Specimen Cache)
- 分析地点: 生物实验室 (Bio Lab)

捕捉爪 (Raptorial Claw) gm_raptorial_claw

折叠的前肢，股节与胫节在膝部铰链处闭合，内缘棘刺相互咬合。末端跗爪完好。

- 稀有度: **brachium raptorium**, 罕见 (Uncommon)
- 生境: 地热 (Geothermal)
- 请求方: 热生物纲要 (Thermal Bio Compendium) 与石灰华标本库 (Travertine Specimen Cache)
- 分析地点: 生物实验室 (Bio Lab)

腹部鞘 (Abdominal Sheath) **gm_abdominal_sheath**

半透明的锥形腹部节段，带有五道环形纹和成对末端尾须。折射性角质层保存完好。

- 稀有度：**abdomen vitreum**, 罕见 (Uncommon)
- 生境：火山 (Volcanic)
- 请求方：火山口生物纲要 (Caldera Bio Compendium) 与浮石标本库 (Pumice Specimen Cache)
- 分析地点：生物实验室 (Bio Lab)

触角鞭 (Antennal Whip) **gm_antennal_whip**

细长丝状触角，包括柄节、梗节关节以及带有感觉微毛的完整珠状鞭节。

- 稀有度：**flagellum sensorium**, 罕见 (Uncommon)
- 生境：深海 (Deep)
- 请求方：深水带生物纲要 (Bathic Bio Compendium) 与俯冲带标本库 (Subduction Specimen Cache)
- 分析地点：生物实验室 (Bio Lab)

花萼板 (Calyx Plate) **fs_calyx_plate**

五边形杯体节段，中央有腕部关节孔。铁氧化在三处留下锈橙色斑块。

- 稀有度：**tabula calicis**, 稀有 (Rare)
- 生境：冰冻 (Frozen)
- 请求方：冰川作用索引 (Glaciation Index)
- 分析地点：生物实验室 (Bio Lab)

腕节段 (Arm Segment) `fs_arm_segment`

腕部中段，带有十个交替排列的小羽枝和八处小骨片关节。羽枝刚毛已被盐分侵蚀。

- 稀有度：`brachium pinnatum`，稀有 (Rare)
- 生境：海岸 (Coastal)
- 请求方：浅滩标本库 (Shoal Specimen Cache)
- 分析地点：生物实验室 (Bio Lab)

柄柱节 (Stalk Columnal) `fs_stalk_columnal`

支撑柄的一段，由六个堆叠的圆盘柱节组成，并由中央轴管穿过。表面有矿物外壳。

- 稀有度：`columna stipitis`，稀有 (Rare)
- 生境：地热 (Geothermal)
- 请求方：石灰华标本库 (Travertine Specimen Cache)
- 分析地点：生物实验室 (Bio Lab)

口面盖 (Oral Tegmen) `fs_oral_tegmen`

花萼的圆盘形覆盖物，中央是五角星形口部，带有五条步带食物沟和偏心肛锥。

- 稀有度：`tegumentum orale`，稀有 (Rare)
- 生境：火山 (Volcanic)
- 请求方：浮石标本库 (Pumice Specimen Cache)
- 分析地点：生物实验室 (Bio Lab)

固着根须 (Holdfast Rootlet) `fs_holdfast_rootlet`

柄部基座带有七条向外辐射的根状触须，是将动物固定在基质上的锚。尖端珠状结构仍保留生物发光。

- 稀有度：`cirri ancorae`，稀有 (Rare)
- 生境：深海 (Deep)
- 请求方：俯冲带标本库 (Subduction Specimen Cache)
- 分析地点：生物实验室 (Bio Lab)

头骨冠 (Cranial Crest) `sd_cranial_crest`

三根黑曜玻璃冠刺安装在头骨背侧甲板上。每根冠刺的前缘都保留着完整晶面。

- 稀有度：`crista petrosa`，传奇 (Legendary)
- 生境：冰冻 (Frozen)
- 请求方：诺克图尔纳冰冻纲要 (Nocturna Frozen Compendium)
- 分析地点：生物实验室 (Bio Lab)

翼膜 (Wing Membrane) `sd_wing_membrane`

皮革状翼膜，保留翼指骨和三个关节连接。纤维条纹从骨骼向外辐射。

- 稀有度：`patagium alarum`，传奇 (Legendary)
- 生境：海岸 (Coastal)
- 请求方：藤壶碎片清单 (Barnacle Fragment Manifest)、大陆架顶点项目 (Continental-Shelf Capstone) 与海岸大纲要 (Coastal Grand Compendium)
- 分析地点：生物实验室 (Bio Lab)

黑曜鳞 (Obsidian Scale) `sd_obsidian_scale`

大型菱形背鳞，四个晶面在中央脊处汇合。表面具有锐利的镜面光泽。

- 稀有度：`squama obsidiana`, 传奇 (Legendary)
- 生境：地热 (Geothermal)
- 请求方：热液碎片清单 (Hydrothermal Fragment Manifest)、地热田顶点项目 (Geothermal-Field Capstone) 与地热大纲要 (Geothermal Grand Compendium)
- 分析地点：生物实验室 (Bio Lab)

尾棍 (Tail Club) `sd_tail_barb`

末端尾棍，菱形尾刺带有两根侧向切割棘。余烬灼烧形成的陨坑使尾刺头部和刺柄都留下伤痕。

- 稀有度：`spiculum caudae`, 传奇 (Legendary)
- 生境：火山 (Volcanic)
- 请求方：火山碎片清单 (Volcanic Fragment Manifest)、喷发带顶点项目 (Eruption-Zone Capstone) 与火山大纲要 (Volcanic Grand Compendium)
- 分析地点：生物实验室 (Bio Lab)

爪 (Talon) `sd_talon`

向后弯曲的后足爪，附着趾节和可活动的足垫碎片。爪尖仍残留生物发光物质。

- 稀有度：`unguis raptoris`, 传奇 (Legendary)
- 生境：深海 (Deep)
- 请求方：超深渊碎片清单 (Hadopelagic Fragment Manifest)、深海沟顶点项目 (Deep-Trench Capstone) 与深渊大纲要 (Abyssal Grand Compendium)
- 分析地点：生物实验室 (Bio Lab)

柄眼 (Stalked Eye) `ce_stalked_eye`

长眼柄末端连接半球形复眼透镜。小眼面网格仍能产生微弱折射。

- 稀有度：`oculus pedunculatus`, 稀有 (Rare)
- 生境：冰冻 (Frozen)
- 请求方：头足类组织请求 (Cephalopod Tissue Request)
- 分析地点：生物实验室 (Bio Lab)

游泳足叶 (Swimmeret Lobe) `ce_swimmeret_lobe`

来自身体中段的侧向桨状结构，四条径向肋骨从根部关节延伸到前缘。表面遍布盐蚀痕迹。

- 稀有度：`lobus natatorius`, 稀有 (Rare)
- 生境：海岸 (Coastal)
- 请求方：碎浪生物调查 (Breaker Bio Survey)
- 分析地点：生物实验室 (Bio Lab)

口盘 (Mouth Disc) `ce_mouth_disc`

正面的口板装置，同心外环由牙板组成，包围着一圈向内指向的毒牙。

- 稀有度：`discus oralis`, 稀有 (Rare)
- 生境：地热 (Geothermal)
- 请求方：硫化喷气孔生物调查 (Solfatara Bio Survey)
- 分析地点：生物实验室 (Bio Lab)

巨附肢 (Great Appendage) `ce_great_appendage`

正面的抓握臂，由五个可活动节段组成，每段都有向内弯曲的钩状棘，末端是反曲尾刺。

- 稀有度：`brachium magnum`，稀有 (Rare)
- 生境：火山 (Volcanic)
- 请求方：火山碎屑流生物调查 (Lahar Bio Survey)
- 分析地点：生物实验室 (Bio Lab)

头部光器官 (Cephalic Photophore) `ce_cephalic_photophore`

纤细的生物发光臂，带有四个可活动节段，末端连接明亮的光器官透镜。

- 稀有度：`photophorum cephalicum`，稀有 (Rare)
- 生境：深海 (Deep)
- 请求方：远洋生物调查 (Pelagic Bio Survey)
- 分析地点：生物实验室 (Bio Lab)

盾片 (Scute Plate) `st_scute_plate`

六边形甲壳板，带有同心生长环，可见四道，约代表四十年的生长。接缝处有盐晶增生。

- 稀有度：`scutum vertebrale`，普通 (Common)
- 生境：冰冻 (Frozen)
- 请求方：壳片调查 (Shell Fragment Survey)、诺克图尔纳冰冻纲要 (Nocturna Frozen Compendium)、土壤富集测定 (Soil Enrichment Assay) 与面纱外套饮食调查 (Veil Mantle Diet Survey)
- 分析地点：生物实验室 (Bio Lab)

腹甲碎片 (Plastron Shard) `st_plastron_shard`

腹部甲壳碎片，显示中央腹甲缝合线以及连接背甲的横桥。底面聚集着藤壶簇。

- 稀有度：`scutum plastrale`，普通 (Common)
- 生境：海岸 (Coastal)
- 请求方：礁石碎片调查 (Reef Fragment Survey)、盐沼普查 (Saltmarsh Census) 与冲浪带目录 (Surf-Zone Catalog)
- 分析地点：生物实验室 (Bio Lab)

肢爪 (Limb Claw) `st_limb_claw`

前肢趾端是一枚粗重的弯曲角蛋白爪。鳞片状甲板皮肤纹理保存完好。

- 稀有度：`unguis pedis`，普通 (Common)
- 生境：地热 (Geothermal)
- 请求方：间歇泉碎片调查 (Geyser Fragment Survey)、温泉普查 (Hot-Spring Census) 与火山口边缘目录 (Caldera-Rim Catalog)
- 分析地点：生物实验室 (Bio Lab)

喙 (Beak) `st_beak`

角蛋白喙鞘，上、下喙鞘均完好，保留切缘和同心生长脊。

- 稀有度：`rhamphotheca`，普通 (Common)
- 生境：火山 (Volcanic)
- 请求方：岩浆碎片调查 (Magma Fragment Survey)、火山碎屑普查 (Pyroclast Census) 与火山灰目录 (Tephra Catalog)
- 分析地点：生物实验室 (Bio Lab)

背甲神经板 (Carapace Neural Plate) **st_carapace_neural**

来自背甲背柱的中央神经板，脊管保存完好，连同全部六个椎骨突起。

- 稀有度：**tabula neuralis**, 普通 (Common)
- 生境：深海 (Deep)
- 请求方：深渊碎片调查 (Hadal Fragment Survey)、午夜带普查 (Midnight-Zone Census) 与化能合成目录 (Chemosynthetic Catalog)
- 分析地点：生物实验室 (Bio Lab)

头部角 (Cephalic Horn) **vm_cephalic_horn**

头部前方成对的可抓握叶，用于将小型猎物引向口部。感觉乳突完好。

- 稀有度：**cornu cephalicum**, 罕见 (Uncommon)
- 生境：冰冻 (Frozen)
- 请求方：面纱外套饮食调查 (Veil Mantle Diet Survey)
- 分析地点：生物实验室 (Bio Lab)

翼片 (Wing Sheet) **vm_wing_sheet**

中翼段，保留五根软骨辐条和背侧斑驳图案。前缘脊完好。

- 稀有度：**tegmen pectorale**, 罕见 (Uncommon)
- 生境：海岸 (Coastal)
- 请求方：盐沼普查 (Saltmarsh Census) 与碎浪生物调查 (Breaker Bio Survey)
- 分析地点：生物实验室 (Bio Lab)

鳃丝 (Gill Filament) **vm_gill_filament**

来自鳃裂的分支状呼吸组织，八条初级鳃丝及其沿线的次级鳃小片均完好。

- 稀有度：**filamenta branchialia**, 罕见 (Uncommon)
- 生境：地热 (Geothermal)
- 请求方：温泉普查 (Hot-Spring Census) 与硫化喷气孔生物调查 (Solfatara Bio Survey)
- 分析地点：生物实验室 (Bio Lab)

尾刺 (Tail Barb) **vm_tail_barb**

长而带锯齿的尾部尖刺，带有五枚向后指的倒钩齿。刺柄上有余烬灼烧形成的陨坑伤痕。

- 稀有度：**aculeus caudae**, 罕见 (Uncommon)
- 生境：火山 (Volcanic)
- 请求方：火山碎屑普查 (Pyroclast Census) 与火山碎屑流生物调查 (Lahar Bio Survey)
- 分析地点：生物实验室 (Bio Lab)

腹侧光器官 (Ventral Photophore) `vm_ventral_photophore`

腹侧皮肤斑块，嵌有七个光器官，排列成具有诊断意义的星座图案。

- 稀有度：`photophora ventralis`，罕见 (Uncommon)
- 生境：深海 (Deep)
- 请求方：午夜带普查 (Midnight-Zone Census) 与远洋生物调查 (Pelagic Bio Survey)
- 分析地点：生物实验室 (Bio Lab)

数据库 / 物品 (DATABASE / ITEMS)

实验室试剂 (Lab Reagents)

生物实验室在分析碎片时消耗的实验室用品。从商店购买。

碱性缓冲液 (Alkaline Buffer) `alkaline_buffer`

用于多种样本分析的实验室试剂。

- 售价 (Sells for) : 1 cr
- 商店价格 (Shop price) : 1 cr
- 组件文档 (Component docs) : 生物实验室 (Bio Lab)

冷冻溶剂 (Cryo-Solvent) `cryo_solvent`

用于深度冷冻样本的实验室试剂。

- 售价 (Sells for) : 2 cr
- 商店价格 (Shop price) : 2 cr
- 组件文档 (Component docs) : 生物实验室 (Bio Lab)

蛋白质标记物 (Protein Marker) `protein_marker`

用于识别稀有物种的实验室试剂。

- 售价 (Sells for) : 40 cr
- 商店价格 (Shop price) : 40 cr
- 组件文档 (Component docs) : 生物实验室 (Bio Lab)

螯合剂 (Chelating Agent) `chelating_agent`

用于处理富含金属样本的实验室试剂。

- 售价 (Sells for) : 200 cr
- 商店价格 (Shop price) : 200 cr
- 组件文档 (Component docs) : 生物实验室 (Bio Lab)

酶溶液 (Enzyme Solution) `enzyme_solution`

用于分析活体组织样本的实验室试剂。

- 售价 (Sells for) : 1,000 cr
- 商店价格 (Shop price) : 1,000 cr
- 组件文档 (Component docs) : 生物实验室 (Bio Lab)

数据库 / 流体 (DATABASE / FLUIDS)

流体 (Fluids)

所有在管道中流动的物质，每种流体各有一个条目。气体管道 (Gas Pipes) 输送气体，液体管道 (Liquid Pipes) 输送液体。完整的机器 `connect()` 关系会为每个物理组件确定一种精确物质；同一组件上的不同精确物质会发生冲突。完整路由模型请参阅“[流动网络 / 流体 \(Flow Networks, Fluids\)](#)”。

工业流体 (INDUSTRIAL FLUIDS)

动力与生产经济中的三种工作流体。

蒸汽 (Steam) `steam`

来自行星热力喷口的过热蒸气。由 `thermal_cap` 捕获，然后由 `steam_turbine` 消耗以产生电网电力，或由 `steam_condenser` 消耗以制取清水。

- 类型 (Type) : 气体 (Gas)
- 生产者 (Produced by) : 热力捕获器 (Thermal Cap)
- 消耗者 (Consumed by) : 制造机 (Fabricator)、蒸汽涡轮机 (Steam Turbine)、蒸汽冷凝器 (Steam Condenser) 和生物施法器 (Bio Caster) ; 以及钛 + 钴 → 涡轮转子 (Turbine Rotor)、稀土 + 转子 → 电推进器 (Electric Thruster)、钛 + 玻璃 → 货舱舱体 (小型) (Cargo Pod (Small))、钛 + 玻璃 + 面板 → 货舱舱体 (中型) (Cargo Pod (Medium))、钛 + 玻璃 + 面板 → 货舱舱体 (大型) (Cargo Pod (Large)) , 以及中子材料 + 电池单元 + 控制器 + 稀土 + 焦油 → 中子电容器 (Neutron Capacitor)
- Tier 输入 (Tier input) : 热力发生器 Mk III (Heat Generator Mk III) , 12 t/h

水 (Water) **water**

由 **water_pump** 从地下井抽取的新鲜液态水，或由 **steam_condenser** 从蒸汽回收的水。抽取地下水会留下盐作为副产品。为耗水型机器等级与工业提供供给。

- 类型 (Type) : 液体 (Liquid)
- 生产者 (Produced by) : 蒸汽冷凝器 (Steam Condenser) 和水泵 (Water Pump)
- 消耗者 (Consumed by) : 制造机 (Fabricator)、生物施法器 (Bio Caster)、行星地形改造器 (Plant Terraformer)、喷淋器 (Sprinkler) 和反应堆 (Reactor) ; 以及铁 + 钛 → 机架 (Machine Frame)、铁 + 玻璃 → 电路面板 (Circuit Panel)、面板 + 钛 + 玻璃 → 控制单元 (Control Unit)、钴 + 铁 + 玻璃 → 电池单元 (Battery Cell)、铁 + 钛 + 玻璃 → 储罐内衬 (Tank Lining)、塑料 + 饲料 + 水 → 加固生物聚合物 (Reinforced Biopolymer)、生物聚合物 + 饲料 + 水 → 富集化合物 (Enrichment Compound)、机架 + 控制器 + 气体管道 + 液体管道 → 无人机基地套件 (Drone Depot Kit)、机架 + 控制器 + 面板 → 无人机基地套件 (中型) (Drone Depot Kit (Medium))、机架 + 控制器 + 面板 → 无人机基地套件 (大型) (Drone Depot Kit (Large))、机架 + 控制器 + 面板 → 采矿钻机套件 (Mining Drill Kit)、机架 + 控制器 + 面板 + 转子 → 工业采矿钻机套件 (Industrial Mining Drill Kit)、机架 + 控制器 + 面板 + 转子 (复数) → 重型采矿钻机套件 (Heavy Mining Drill Kit)、稀土 + 钛 + 控制器 → 无人机 (小型) (Drone (Small))、稀土 + 钛 + 控制器 → 无人机 (中型) (Drone (Medium))、稀土 + 钛 + 控制器 → 无人机 (大型) (Drone (Large))、稀土 + 控制器 + 润滑剂 + 橡胶 → 直升机推进器 (Heli Thruster)、钛 + 润滑剂 + 橡胶 → 油罐 (小型) (Oil Tank (Small))、钛 + 润滑剂 + 橡胶 + 内衬 → 油罐 (中型) (Oil Tank (Medium))、钛 + 润滑剂 + 橡胶 + 内衬 → 油罐 (大型) (Oil Tank (Large))、管道 + 阀门 + 稀土 + 钛 + 焦油 → 冷却剂回路 (Coolant Loop)、中子材料 + 电池单元 + 控制器 + 稀土 + 焦油 → 中子电容器 (Neutron Capacitor)、机架 + 控制器 + 面板 → 种子制造器套件 (Seed Maker Kit)、机架 (复数) + 控制器 (复数) + 面板 (复数) + 液体管道 → 行星地形改造器套件 (Plant Terraformer Kit)、机架 + 管道 + 阀门 → 喷淋器套件 (Sprinkler Kit)、焦油 + 玻璃 → 肥料 (Fertilizer)、焦油 + 玻璃 + 面板 → 肥料 Mk II (Fertilizer Mk II)、焦油 + 玻璃 + 电容器 + 稀土 → 肥料 Mk III (Fertilizer Mk III)、塑料 + 稀土 → 生长加速剂 (Growth Accelerant)、电容器 + 控制器 + 稀土 + 冷却剂 → 产量放大器 (Yield Amplifier)、控制器 + 面板 + 稀土 + 阀门 → 行星地形改造器包 Mk II (Plant Terraformer Pack Mk II)、电容器 + 稀土 + 控制器 → 生长灯包 Mk III (Grow Lamp Pack Mk III)、控制器 + 面板 + 冷却剂 → 喷淋器包 Mk III (Sprinkler Pack Mk III)、以及电容器 (复数) + 稀土 + 控制器 + 冷却剂 → 栖息地包 Mk II (Habitat Pack Mk II)
- Tier 输入 (Tier input) : 氧气发生器 Mk III (Oxygen Generator Mk III) , 8 t/h ; 压力发生器 Mk III (Pressure Generator Mk III) , 5 t/h

油 (Oil) **oil**

由 **oil_pump** 从油井抽取的重质原油。由 **oil_generator** 燃烧发电，并加工成聚合物、润滑剂 (**lubricant**) 和焦油 (**tar**) 。

- 类型 (Type) : 液体 (Liquid)
- 生产者 (Produced by) : 油泵 (Oil Pump)
- 消耗者 (Consumed by) : 制造机 (Fabricator)、油发电机 (Oil Generator) 和无人机服务站 (Drone Service Station) ; 以及铁 + 油 → 润滑剂 + 焦油、玻璃 + 油 → 塑料 + 焦油、钴 + 油 → 橡胶 + 焦油，以及重油裂解 (Heavy Oil Cracking) : 油 → 焦油

生物群落精华 (BIOME ESSENCES)

从本土生命体中压榨出的生物浓缩物。每种生物群落都会产出自己的精华。

冻原精华 (Frozen Essence) `frozen_essence`

在 `essence_liquifier` 中将冻原生物群落的生命体液化后得到的浅色晶体浓缩物。

- 类型 (Type) : 液体 (Liquid)
- 生产者 (Produced by) : 精华液化器 (Essence Liquifier)
- 消耗者 (Consumed by) : 生物质混合器 (Biomass Mixer)

海岸精华 (Coastal Essence) `coastal_essence`

在 `essence_liquifier` 中将海岸生物群落的生命体液化后得到的咸绿色浓缩物。

- 类型 (Type) : 液体 (Liquid)
- 生产者 (Produced by) : 精华液化器 (Essence Liquifier)
- 消耗者 (Consumed by) : 生物质混合器 (Biomass Mixer)

地热精华 (Geothermal Essence) `geothermal_essence`

在 `essence_liquifier` 中将地热生物群落的生命体液化后得到的温暖矿物浓缩物。

- 类型 (Type) : 液体 (Liquid)
- 生产者 (Produced by) : 精华液化器 (Essence Liquifier)
- 消耗者 (Consumed by) : 生物质混合器 (Biomass Mixer)

火山精华 (Volcanic Essence) **volcanic_essence**

在 **essence_liquifier** 中将火山生物群落的生命体液化后得到的深红色含硫浓缩物。

- 类型 (Type) : 液体 (Liquid)
- 生产者 (Produced by) : 精华液化器 (Essence Liquifier)
- 消耗者 (Consumed by) : 生物质混合器 (Biomass Mixer)

深渊精华 (Deep Essence) **deep_essence**

在 **essence_liquifier** 中将深渊生物群落的生命体液化后得到的紫色地下浓缩物。

- 类型 (Type) : 液体 (Liquid)
- 生产者 (Produced by) : 精华液化器 (Essence Liquifier)
- 消耗者 (Consumed by) : 生物质混合器 (Biomass Mixer)

异种气体 (EXOTIC GASES)

使用 **exotic_gas_cap** 从循环喷口矿床中抽取的气体。常见气体可以直接流向用途；稀有气体以原始形态到达，需要精炼。

氨 (Ammonia) **ammonia**

从喷口矿床直接抽取的常见碱性异种气体。可直接使用，无需精炼。

- 类型 (Type) : 气体 (Gas)

沼气 (Swamp Gas) **swamp_gas**

从低地喷口矿床抽取的常见沼泽气体。可直接使用，无需精炼。

- 类型 (Type) : 气体 (Gas)

原始硫气 (Raw Sulfur Gas) `raw_sulfur_gas`

来自火山气体喷口的未精炼硫原料。精炼器会将其纯化为硫气 (`sulfur_gas`)，过程中燃烧焦油。

- 类型 (Type)：气体 (Gas)
- 精炼为 (Refines into)：硫气 (Sulfur Gas)
- 消耗配方 (Consumed by)：原始硫气 + 焦油 → 硫气

硫气 (Sulfur Gas) `sulfur_gas`

在精炼器 (`refiner`) 中从 `raw_sulfur_gas` 纯化得到的精炼硫气。

- 类型 (Type)：气体 (Gas)
- 精炼来源 (Refined from)：原始硫气 (Raw Sulfur Gas)
- 配方输出 (Recipe output)：原始硫气 + 焦油 → 硫气

原始氯气 (Raw Chlorine) `raw_chlorine`

来自气体喷口的未精炼氯原料。以高昂的焦油成本精炼为氯气。

- 类型 (Type)：气体 (Gas)
- 精炼为 (Refines into)：氯气 (Chlorine)
- 消耗配方 (Consumed by)：原始氯气 + 焦油 → 氯气

氯气 (Chlorine) `chlorine`

经过精炼的腐蚀性氯气，是最稀有的异种气体。

- 类型 (Type)：气体 (Gas)
- 精炼来源 (Refined from)：原始氯气 (Raw Chlorine)
- 配方输出 (Recipe output)：原始氯气 + 焦油 → 氯气

异种液体 (EXOTIC LIQUIDS)

使用 `exotic_spring_tap` 从泉水矿床中抽取的液体。常见液体可以直接流向用途；稀有液体以原始形态到达，需要精炼。

盐水 (Brine) `brine`

从泉水矿床抽取的常见富含矿物质液体。可直接使用，无需精炼。

- 类型 (Type) : 液体 (Liquid)

原始低温流体 (Raw Cryofluid) `raw_cryofluid`

来自泉水抽取的未精炼过冷原料。精炼器会将其纯化为低温流体。

- 类型 (Type) : 液体 (Liquid)
- 精炼为 (Refines into) : 低温流体 (Cryofluid)
- 消耗配方 (Consumed by) : 原始低温流体 + 焦油 → 低温流体

低温流体 (Cryofluid) `cryofluid`

在精炼器 (`refiner`) 中从 `raw_cryofluid` 纯化得到的精炼过冷流体。

- 类型 (Type) : 液体 (Liquid)
- 精炼来源 (Refined from) : 原始低温流体 (Raw Cryofluid)
- 配方输出 (Recipe output) : 原始低温流体 + 焦油 → 低温流体

原始水银 (Raw Quicksilver) `raw_quicksilver`

来自泉水抽取的未精炼液态金属汞合金。以高昂的焦油成本精炼为水银。

- 类型 (Type) : 液体 (Liquid)
- 精炼为 (Refines into) : 水银 (Quicksilver)
- 消耗配方 (Consumed by) : 原始水银 + 焦油 → 水银

水银 (Quicksilver) **quicksilver**

经过精炼的液态金属，是最稀有的异种液体。

- 类型 (Type) : 液体 (Liquid)
- 精炼来源 (Refined from) : 原始水银 (Raw Quicksilver)
- 配方输出 (Recipe output) : 原始水银 + 焦油 → 水银

研究 (Research)

本版本的完整科技树。研究项目在解锁前仍会记录，因此你可以搜索未来系统、了解其门槛并制定发展计划。

地形改造指数 (TERRAFORM INDEX)

摄像机画面 (Camera Feed) **research_camera_feed** - 尚未解锁 (NOT YET UNLOCKED)

实时观察诺克图娜 (Nocturna) 的变化。启动外部摄像机，通过实时画面观察外界。风暴横穿盆地，冰层消退，奇异生命开始扎根，而你建造的基地在黑暗中发光。

- 门槛 (Threshold) : 地形改造指数 (Terraform Index) 5,000
- 科技 ID (Tech id) : **camera_feed_unlock**

船载计算机 (Ship Computer) **research_computer** - 尚未解锁 (NOT YET UNLOCKED)

解锁计算机仪表盘访问权限。解锁“飞船 → 计算机 (Ship → Computer)”页面：用于系统状态、脚本管理、游乐场、成就和通知的仪表盘。进一步研究会增加更多计算机标签页，首先是资料库 (Library)。

- 门槛 (Threshold) : 地形改造指数 (Terraform Index) 10,000
- 科技 ID (Tech id) : **ship_computer**

共享资料库 (Shared Library) **research_shared_library** - 尚未解锁 (NOT YET UNLOCKED)

在脚本之间复用代码。为船载计算机增加资料库 (Library) 标签页。在资料库中编写一次可复用的辅助函数，然后从任意机器脚本导入，让共享逻辑集中在一个地方，而不是到处复制。

- 门槛 (Threshold) : 地形改造指数 (Terraform Index) 20,000
- 科技 ID (Tech id) : **shared_library**

信号总线（Signal Bus） **research_signal_bus** - 尚未解锁（NOT YET UNLOCKED）

实现脚本到脚本的消息传递。为船载计算机增加信号总线（Signal Bus）标签页。脚本通过在命名频道上发送排队消息或广播共享值来彼此协调：这是明确且可安全保存的共享状态，而不是隐藏的全局变量。

- 门槛（Threshold）：地形改造指数（Terraform Index） 35,000
- 科技 ID（Tech id）：**signal_bus_unlock**

数据归档（Data Archive） **research_data_archive** - 尚未解锁（NOT YET UNLOCKED）

持久化已学习的表格。为船载计算机增加数据归档（Data Archive）标签页。脚本可以存储查找表、校准样本、已发现地图及其他学习成果；这些知识会跨越重启以及存档/加载持续存在。使用信号总线处理实时协调，使用数据归档保存长期数据。

- 门槛（Threshold）：地形改造指数（Terraform Index） 70,000
- 科技 ID（Tech id）：**data_archive_unlock**

先锋号底盘（Pioneer Chassis） **research_pioneer** - 尚未解锁（NOT YET UNLOCKED）

解锁远征车辆。解锁商店中的八槽先锋号底盘（Pioneer chassis）。它没有内置电池、货舱或导航（Nav），必须安装另行购买的硬件后才能移动。

- 门槛（Threshold）：地形改造指数（Terraform Index） 100,000
- 科技 ID（Tech id）：**pioneer_unlock**

供应物流（Supply Logistics） **research_orders_system** - 尚未解锁（NOT YET UNLOCKED）

解锁订单和供应码头采购。这是一套面向地球需求的远程物流协议。商店中会出现供应码头（Supply Docks），在吞吐量研究之前每小时运输 25 件物品；建造更多码头即可并行服务订单。

- 门槛（Threshold）：地形改造指数（Terraform Index） 110,000
- 科技 ID（Tech id）：**orders_system_unlock**

前哨站建设 (Outpost Construction) **research_outpost_hub** - 尚未解锁 (NOT YET UNLOCKED)

发现遍布行星的新地点。解锁商店中的前哨站套件 (Outpost Kit)。建立前哨站仍需先研究建造器模块 (Constructor Module)，并使用装备建造器的先锋号：在计划模式 (Plan Mode) 中放置前哨站蓝图，或调用 `construction_blueprint.plan_structure("outpost", x, y)`，将套件装入先锋号货舱，然后建造。每个前哨站初始有 20 栋建筑的软容量。之后的扩展前哨站基础设施 (Expanded Outpost Infrastructure) 会使每个前哨站的容量增加

5。边际套件价格根据已建立的扩展前哨站数量，加上实体存储中持有的未建造前哨站套件计算：15,000 cr、30,000 cr、50,000 cr、75,000 cr、110,000 cr、200,000 cr、350,000 cr、550,000 cr、800,000 cr，之后每个套件均为 1,000,000 cr。

- 门槛 (Threshold)：地形改造指数 (Terraform Index) 120,000
- 科技 ID (Tech id)：`outpost_unlock`

制造 (Fabrication) **research_fabricator** - 尚未解锁 (NOT YET UNLOCKED)

解锁重型制造。商店中可购买制造机 (Fabricator)。这台工业装配压机将精炼材料组合为成品设备、模块和车辆框架。

- 门槛 (Threshold)：地形改造指数 (Terraform Index) 130,000
- 科技 ID (Tech id)：`fabricator_unlock`

制图学 (Cartography) **research_cartography** - 尚未解锁 (NOT YET UNLOCKED)

标注行星地图。增加地图标记：你可以在计划模式中放置带名称和颜色的注释，也可以从任意脚本通过 `get_component("markers")` 放置。标记是笔记，不是建造物。它们不消耗资源，不需要先锋号，放置后立即出现，因此勘测脚本可以记录“在哪里发现了什么”。

- 门槛 (Threshold)：地形改造指数 (Terraform Index) 140,000
- 科技 ID (Tech id)：`cartography_unlock`

控制室 (Control Room) **research_custom_panels** - 尚未解锁 (NOT YET UNLOCKED)

创建由脚本驱动的控制面板。打开控制室 (Control Room)，这是一个用于脚本驱动仪表板卡片的顶层页面。使用绘制和输入 API 构建时钟、舰队面板、喷口周期时间线和恢复控制。

- 门槛 (Threshold) : 地形改造指数 (Terraform Index) 150,000
- 科技 ID (Tech id) : **custom_panels_unlock**

基础无人机操作 (Basic Drone Operations) **research_drones** - 尚未解锁 (NOT YET UNLOCKED)

在生物圈时代之前开启空中物流。解锁无人机系统及其舰队仪表板，即行星地图下的“车辆 / 无人机 / 支援 (Vehicles / Drones / Support)”列表。电动无人机最高以 300 m/h 飞行，全油门时消耗 5 Wh/h；能耗随油门的平方增加。无人机硬件仍需要通过地球订单获得蓝图配方。制造出的设施与底盘从库存部署到前哨站；野外采矿钻机使用在计划模式中放置、或由脚本放置的行星地图蓝图。

- 门槛 (Threshold) : 地形改造指数 (Terraform Index) 180,000
- 科技 ID (Tech id) : **drones_unlock**

生物圈 (Biosphere) **research_biosphere** - 尚未解锁 (NOT YET UNLOCKED)

解锁物质供应链。解锁物质等级供应链，并使便携式生物扫描仪、便携式生物提取器、精华液化器和物质混合器在商店中可用。扫描生物群落地块，提取生命体样本，再通过这条链路生产物质。

- 门槛 (Threshold) : 地形改造指数 (Terraform Index) 210,000
- 科技 ID (Tech id) : **biosphere_unlock**

更大堆叠 (Bigger Stacks) `research_high_density_storage` - 尚未解锁 (NOT YET UNLOCKED)

每个库存槽容纳双倍物品。重建诺克图娜基地的货架，使每个槽位能够放置更高的堆叠。现在可堆叠的物品每槽最多 20 个，而不是 10 个。设备、模块及其他单件物品仍各占一个槽位。这只会改变基地库存，不影响车辆货舱、存储箱或仓库。

- 门槛 (Threshold) : 地形改造指数 (Terraform Index) 250,000
- 科技 ID (Tech id) : `high_density_storage_unlock`

天气计划 (Weather Program) `research_weather_program` - 尚未解锁 (NOT YET UNLOCKED)

追踪风暴和宝藏信号。解锁天气网络 (Weather Network)，在诺克图娜基地免费安装一座已完成的气象站及其建筑槽位，并使额外气象站在商店中可购买。观察当地风暴，处理实时传输，并将可选结果发布到信号板 (Signal Board)。

- 门槛 (Threshold) : 地形改造指数 (Terraform Index) 330,000
- 科技 ID (Tech id) : `weather_program_unlock`

快速送料器 (Fast Feeders) `research_fast_feeders` - 尚未解锁 (NOT YET UNLOCKED)

物品传输吞吐量翻倍。将每次定时离散物品传输的持续时间减少 50%。机器端点在更快的处理周期完成前仍保持占用。流体和手动世界操作不受影响。

- 门槛 (Threshold) : 地形改造指数 (Terraform Index) 400,000
- 科技 ID (Tech id) : `fast_feeders_unlock`

货舱扩展 (Cargo Expansion) `research_cargo_expansion` - 尚未解锁 (NOT YET UNLOCKED)

购买额外库存槽位。解锁库存页面上的 EXPAND 按钮。每个槽位都比上一个更贵：第 37 个槽位从 500 cr 起，逐步增加到最后一个槽位约 146,000 cr，总上限为 60 个。完全扩容的总成本约为 666,000 cr。

- 门槛 (Threshold) : 地形改造指数 (Terraform Index) 520,000
- 科技 ID (Tech id) : `cargo_expansion_unlock`

天气预报 (Weather Forecasting) `research_weather_forecasting` - 尚未解锁 (NOT YET UNLOCKED)

提前三倍远距离看到正在接近的风暴。将新气象站报告和 Incoming 覆盖范围从 8 个世界时钟小时延长至 24 个。预报仍仅针对预计进入通电气象站覆盖范围的风暴。

- 门槛 (Threshold) : 地形改造指数 (Terraform Index) 550,000
- 科技 ID (Tech id) : `weather_forecasting_unlock`

高压流体运输 (High-Pressure Fluid Transport) `research_high_pressure_fluid_transport` - 尚未解锁 (NOT YET UNLOCKED)

远程气体和液体吞吐量变为三倍。改造所有已完成和未来的气体管道、液体管道组件。每条匹配的源端和汇端连接链路现在最多承载 6,000 t/h, 而不是 2,000 t/h。管道长度和内部支路仍不会增加容量; 独立的连接链路和独立组件保留各自的预算。

- 门槛 (Threshold) : 地形改造指数 (Terraform Index) 600,000
- 科技 ID (Tech id) : `high_pressure_fluid_transport_unlock`

核计划 (Nuclear Program) `research_nuclear_program` - 尚未解锁 (NOT YET UNLOCKED)

反应堆 (Reactor)。解锁商店中的反应堆: 装入燃料棒, 让冷却水流过, 昼夜不停、风暴晴天均输出 5,000 W。一根燃料棒在热度 1.0 时可持续 72 小时, 燃料消耗遵循指令设定的热度。让堆芯保持在绿色区间; 推得过热就会过热并安全关闭。

- 门槛 (Threshold) : 地形改造指数 (Terraform Index) 650,000
- 科技 ID (Tech id) : `nuclear_program_unlock`

存储箱 (Storage Bins) `research_storage_bin` - 尚未解锁 (NOT YET UNLOCKED)

单一材料库存箱。解锁商店中的存储箱 (Storage Bin)。这是可部署在前哨站的单一材料容器，用于储存原材料和精炼材料，每个箱子只能存放一种材料。留下采掘所得，分阶段准备所需物资。（稍后研究的仓库 (Warehouse) 是更大的多材料仓储。）

- 门槛 (Threshold) : 温度 (Temperature) 5
- 科技 ID (Tech id) : `storage_bin_unlock`

货舱架 (Cargo Rack) `research_cargo_rack_small` - 尚未解锁 (NOT YET UNLOCKED)

解锁模块化车辆货舱存储。解锁商店中的小型货舱架 (Small Cargo Rack) 和便携式储物箱 (Portable Bin)。这是一个带 1 个储物箱的货舱架，让模块化车辆可以将采掘的矿石运回基地。

- 门槛 (Threshold) : 温度 (Temperature) 8
- 科技 ID (Tech id) : `cargo_rack_small_unlock`

建造器模块 (Constructor Module) `research_constructor_module` - 尚未解锁 (NOT YET UNLOCKED)

建造行星地图蓝图。解锁商店中先锋号的建造器模块 (Constructor Module)。在行星地图的计划模式中规划建造，或使用 `get_component("construction_blueprint")`，将所需套件、管段或桥梁装入先锋号货舱，然后驾车前往现场建造前哨站、管道、电力线和机器。

- 门槛 (Threshold) : 温度 (Temperature) 10
- 科技 ID (Tech id) : `constructor_module_unlock`

电池座 (Battery Holder) **research_battery_holder_small** - 尚未解锁 (NOT YET UNLOCKED)

解锁便携式电池舱。解锁商店中的小型电池座 (Small Battery Holder)。为模块化车辆增加一个电池舱的便携式电池安装位；任何离开车辆充电站的远征都离不开它。

- 门槛 (Threshold) : 温度 (Temperature) 12
- 科技 ID (Tech id) : **battery_holder_small_unlock**

热力发生器 Mk II (Heat Generator Mk II) **research_heat_mk2_pack** - 尚未解锁 (NOT YET UNLOCKED)

更高的热输出。解锁商店中的热力发生器 Mk II 升级包 (Heat Generator Mk II Upgrade Pack)。将 Mk I 加热器改造为 Mk II, 以获得更高输出。

- 门槛 (Threshold) : 温度 (Temperature) 80
- 科技 ID (Tech id) : **heat_mk2_pack_unlock**

生物发光灌注 (Bioluminescent Infusion) **research_luminizer** - 尚未解锁 (NOT YET UNLOCKED)

解锁生物发光器 (Bio Luminizer)。在商店中解锁生物发光器：这台海岸机器会将生物发光碎片的光辉调谐为订单要求的精确目标颜色。海岸生物订单需要灌注后的碎片。

- 门槛 (Threshold) : 温度 (Temperature) 90
- 科技 ID (Tech id) : **luminizer_unlock**

中型电池座 (Medium Battery Holder) **research_battery_holder_medium** - 尚未解锁 (NOT YET UNLOCKED)

解锁双舱电池座。解锁商店中的中型电池座 (Medium Battery Holder)。可安装两块便携式电池, 为模块化车辆提供紧凑的续航升级。

- 门槛 (Threshold) : 温度 (Temperature) 280
- 科技 ID (Tech id) : **battery_holder_medium_unlock**

前哨站扩展 (Outpost Expansion) `research_outpost_expansion` - 尚未解锁 (NOT YET UNLOCKED)

使每个前哨站的建筑容量增加 5。当前和未来的每个前哨站软建筑容量均增加 5：诺克图娜基地从 25 栋增加到 30 栋，已建立的前哨站从 20 栋增加到 25 栋。超过新上限的建筑仍会产生正常的过度拥挤惩罚。

- 门槛 (Threshold)：温度 (Temperature) 400
- 科技 ID (Tech id)：`outpost_expansion_unlock`

大型电池 (Large Battery) `research_large_battery` - 尚未解锁 (NOT YET UNLOCKED)

一个单元格容纳 5 倍电网存储。解锁商店中的大型电池 (Large Battery)：可提供 2,500 Wh 的基地存储容量，是基础电池单元的五倍。一件大型电池可替代一组小型电池。

- 门槛 (Threshold)：温度 (Temperature) 700
- 科技 ID (Tech id)：`large_battery_unlock`

深海调理 (Deep-Sea Conditioning) `research_bio_conditioner` - 尚未解锁 (NOT YET UNLOCKED)

解锁生物调理器 (Bio Conditioner)。在商店中解锁生物调理器：这台深海机器依据公开规则书，通过 5 阶段接受/拒绝检查流程进行碎片质量控制。深海生物订单需要经过调理的碎片。仅需电力，不需要流体或材料。

- 门槛 (Threshold)：温度 (Temperature) 1,200
- 科技 ID (Tech id)：`bio_conditioner_unlock`

油发电机 (Oil Generator) `research_oil_generator` - 尚未解锁 (NOT YET UNLOCKED)

高密度燃油发电。解锁商店中的油发电机 (Oil Generator)。一台机器燃烧油料最多产生 700 W (是蒸汽轮机峰值的数倍)，并可通过脚本对油门进行全范围控制。满输出时消耗 8 t/h。

- 门槛 (Threshold)：温度 (Temperature) 1,500
- 科技 ID (Tech id)：`oil_generator_unlock`

直升机无人机 (Heli-Drones) `research_heli_drones` - 尚未解锁 (NOT YET UNLOCKED)

远程燃油空运。允许在无人机上安装直升机推进器和油罐。它们的制造机配方需另行通过前厅订单 (Vestibule orders) 获得。直升机无人机以 900 m/h 飞行, 满油门时消耗 5 t/h 的油; 能耗随油门平方增加。三种油罐尺寸决定其航程, 无人机服务站从油料输入为它们补充燃油。

- 门槛 (Threshold) : 温度 (Temperature) 6,000
- 科技 ID (Tech id) : `heli_drones_unlock`

避雷针 (Lightning Rods) `research_lightning_rod` - 尚未解锁 (NOT YET UNLOCKED)

捕获雷击, 储存涌流。解锁避雷针套件 (Lightning Rod Kit) 配方。这种由风暴充能的储能装置可储存 4,000 Wh, 捕获 600 m 范围内的雷击, 并位于电池之后向电网供电。耐久度每天下降 0.05; 除非脚本使用 1 个风暴玻璃 (Storm Glass) 修复, 否则捕获能力会降至零。

- 门槛 (Threshold) : 温度 (Temperature) 8,000
- 科技 ID (Tech id) : `lightning_rod_unlock`

燃料组装器 (Fuel Assembler) `research_fuel_assembler` - 尚未解锁 (NOT YET UNLOCKED)

原始铀 + 铅 → 燃料棒。解锁商店中的燃料组装器 (Fuel Assembler)。这台核工坊将原始铀和铅外壳压制成燃料棒, 运行时功耗约为 1,800 W。

- 门槛 (Threshold) : 温度 (Temperature) 10,000
- 科技 ID (Tech id) : `fuel_assembler_unlock`

热力发生器 Mk IV (Heat Generator Mk IV) `research_heat_mk4_pack` - 尚未解锁 (NOT YET UNLOCKED)

核时代热力等级, 由燃料棒供能。在制造机处解锁热力升级包 Mk IV (Heat Upgrade Pack Mk IV) 配方。这个最终等级完全取代蒸汽输入, 改为使用由连接的铅罐 (Lead Cask) 供料的燃料棒弹匣, 并消耗约为 Mk I 100 倍的电力。没有燃料棒的加热器会停止产热, 而不会逐渐损坏。

- 门槛 (Threshold) : 温度 (Temperature) 42,000
- 科技 ID (Tech id) : `heat_mk4_pack_unlock`

自动送料器 (Auto Feeders) **research_auto_feeders** - 尚未解锁 (NOT YET UNLOCKED)

解锁材料流动。允许物品通过机器 I/O 端口进行保留属性的流动。在诺克图娜基地，库存只能作为来源或目的地；其他端点必须位于机器所在的前哨站，或位于车辆实际所在位置。具有相同物品 ID 但属性不同的物品仍属于不同堆叠。

- 门槛 (Threshold) : 氧气 (Oxygen) 1
- 科技 ID (Tech id) : **feeder_unlock**

地球许可 (Earth Clearance) **research_earth_clearance** - 尚未解锁 (NOT YET UNLOCKED)

高级合约访问权限。随着大气氧气开始上升，解锁三个高级合约。

- 门槛 (Threshold) : 氧气 (Oxygen) 3
- 科技 ID (Tech id) : **earth_clearance**

矿石精炼 (Ore Refinement) **research_smelter** - 尚未解锁 (NOT YET UNLOCKED)

解锁精炼功能。解锁商店中的熔炼炉 (Smelter)。这台工业熔炉将原始矿石还原为适用于高级机械的精炼金属和组件。

- 门槛 (Threshold) : 氧气 (Oxygen) 5
- 科技 ID (Tech id) : **smelter_unlock**

车辆充电站 (Vehicle Charging Station) **research_charging_station** - 尚未解锁 (NOT YET UNLOCKED)

解锁车辆充电功能。解锁商店中的车辆充电站 (Vehicle Charging Station)。Mk I 有 1 个充电位和 30 W 充电预算；后续升级包会提高舰队容量和共享单车充电速度。

- 门槛 (Threshold) : 氧气 (Oxygen) 9
- 科技 ID (Tech id) : **charging_station_unlock**

Sport Nav（运动导航）

research_nav_sport - 尚未解锁

高性能传动系统接入。解锁商店中的 Sport Nav 模块。在同一载具中，单个 Sport Nav 相比 Basic Nav 可将最高速度提高至 2 倍，但移动功耗为 2.6 倍；全油门时，每米电池消耗约为 Basic Nav 的 1.3 倍。继续安装 Sport Nav 会以递增成本叠加效果，尤其适用于满载的长途运输路线。

阈值	Oxygen（氧气）50
Tech id	nav_sport_unlock

Warehouse（仓库）

research_warehouse - 尚未解锁

批量存储建筑。解锁商店中的 Warehouse：一个可存放 10,000 单位、多材料的仓库，包含 5 个每个容量为 2,000 的材料锁定槽位。可将其用作无人机配送和大型载具运输的缓冲区。

阈值	Oxygen（氧气）150
Tech id	warehouse_unlock

Verified Contractor（认证承包商）

research_verified_contractor - 尚未解锁

解锁新一批合约。你的地球化改造产出已通过地球的审核阈值。第二批合约（谜题更密集、奖励更高）会发送到你的终端，并为 Contractor Reputation（承包商声望）阶梯增加新的等级。

阈值	Oxygen（氧气）300
Tech id	verified_contractor_unlock

Volcanic Forge-Casting（火山锻铸）

research_bio_caster - 尚未解锁

解锁 Bio Caster。Bio Caster 是一种火山机器，会使用制造材料对碎片进行热铸；它也会在商店中解锁。火山 Bio Orders（生物订单）需要锻铸碎片。

阈值	Oxygen（氧气）350
Tech id	bio_caster_unlock

Liquid Tank（液体罐）

research_liquid_tank - 尚未解锁

缓冲任意一种液体。解锁商店中的 Liquid Tank：一个被动式 100 吨缓冲罐，会锁定它接收的第一种液体（水、油或任意生物群系精华），在排空前只储存该液体。

阈值	Oxygen（氧气）400
Tech id	liquid_tank_unlock

Waste Processing（废物处理）

research_garbage_disposal - 尚未解锁

可编程地销毁不需要的物品和流体。解锁 Waste Processor Kit 配方。该机器会永久销毁一条选定的流：任何物品都可通过 feeder 输入端，任何液体都可通过 **liquid_in**，任何气体都可通过 **gas_in**。这为焦油、多余的水和不需要的气体提供明确的溢流路径，但不会回收电力或材料。在 Fabricator（制造机）中制造套件；其脚本会选择并启用活动模式。

阈值	Oxygen（氧气）1,400
Tech id	garbage_disposal_unlock

Petroleum Survey（石油勘测）

research_petroleum_survey - 尚未解锁

Deep Sonar（深层声纳）可发现油井。解锁地下油藏勘测和 Oil Pump 配方。需要 Deep Sonar：油井属于后期声纳才能发现的隐藏层。在 Fabricator 中制造 Oil Pump，将其部署在已勘测的油井上，然后把石油输送回基地，用于发电和精炼。

阈值	Oxygen（氧气）1,500
Tech id	petroleum_survey_unlock

Large Battery Holder（大型电池支架）

research_battery_holder_large - 尚未解锁

三槽电池安装支架。解锁商店中的 Large Battery Holder。三个电池位足以在漫长的外环远征中维持 Heavy Drill（重型钻机）的功耗。

阈值	Oxygen（氧气）1,800
Tech id	battery_holder_large_unlock

Large Cargo Rack（大型货架）

research_cargo_rack_large - 尚未解锁

三槽货物安装架。解锁商店中的 Large Cargo Rack。三个货箱可在大型采矿行动中提供最大运输容量。

阈值	Oxygen（氧气）2,000
Tech id	cargo_rack_large_unlock

Heavy Drill（重型钻机）

research_drill_heavy - 尚未解锁

硬度 4 的开采能力。解锁商店中的 Heavy Drill 模块。它是唯一能够开采硬度 4 的 Neutronium（中子矿）的钻机，而 Neutronium 是制造 Mk III 所必需的材料。

阈值	Oxygen（氧气）2,500
Tech id	drill_heavy_unlock

Shielded Logistics（屏蔽物流）

research_shielded_logistics - 尚未解锁

Lead Casks（铅桶）和热货物处理。热货物需要屏蔽通道：Lead Casks 是 Raw Uranium（原铀）和 Fuel Rods（燃料棒）唯一的固定存放处；Shield Plating（屏蔽镀层）让无人机能够提取 Raw Uranium 而不会获得暴露值。解锁这两种物品的可用性；制造配方通过承包商订单获得。

阈值	Oxygen（氧气）3,000
Tech id	shielded_logistics_unlock

Shielded Depot Operations（屏蔽仓库运行）

research_shielded_depot_ops - 尚未解锁

通过 Drone Depots（无人机仓库）运输核货物。为每个 Drone Depot 加装 containment（隔离设施），让热货物能够通过物流网络。在研究此技术前，仓库会直接拒收 Raw Uranium 和 Fuel Rods，而加装镀层的无人机也无处卸载它们。该效果适用于你拥有的每个仓库以及之后建造的每个仓库。

阈值	Oxygen（氧气）8,000
Tech id	shielded_depot_ops_unlock

Oxygen Generator Mk IV（氧气发生器 Mk IV）

research_oxygen_mk4_pack - 尚未解锁

核时代的氧气等级，由燃料棒供能。解锁 Fabricator 中的 Oxygen Upgrade Pack Mk IV 配方。这个最终等级完全以 Fuel Rod magazine（燃料棒弹匣）取代进水口；弹匣由连接的 Lead Cask 供料，功耗约为 Mk I 的 100 倍。没有燃料棒的发生器会停止生产，而不会逐渐退化。

阈值	Oxygen（氧气）40,000
Tech id	oxygen_mk4_pack_unlock

PRESSURE（压力）

Mining Operations（采矿作业）

research_mining - 尚未解锁

解锁采矿仪表盘、Mining 商店分类和行星地图。在派出载具前先勘察地表。

阈值	Pressure（压力）0.1
Tech id	mining_unlock

Rover Chassis（漫游车底盘）

research_rover - 尚未解锁

解锁第一种远征载具 Rover。它集成了 100 Wh 电池和容量为 10 单位的货舱，二者均不可更换；Nav、Sonar 和 Drill 使用三个专用模块槽位。

阈值	Pressure（压力）0.11
Tech id	rover_unlock

Nav Module（导航模块）

research_nav_module - 尚未解锁

解锁驾驶硬件和商店中的 Nav 模块。让载具能够导航，并从自身脚本中驾驶。停止、完成或出错都会取消由脚本控制的移动。

阈值	Pressure（压力）0.12
Tech id	nav_module_unlock

Sonar Module（声纳模块）

research_sonar_module - 尚未解锁

解锁勘测硬件和商店中的 Basic Sonar 模块。Sonar 会从载具当前位置执行点扫描，发现附近的地图地点并将其记录到 Journal（日志）中。

阈值	Pressure（压力）0.15
Tech id	sonar_module_unlock

Drill Module（钻机模块）

`research_deep_extraction` - 尚未解锁

解锁钻探硬件和商店中的 Drill 模块。从已勘测的采矿地点提取原矿。

阈值	Pressure（压力）0.2
Tech id	<code>deep_extraction_unlock</code>

Pressure Generator Mk II（压力发生器 Mk II）

`research_pressure_mk2_pack` - 尚未解锁

更高的压力产量。解锁商店中的 Pressure Generator Mk II Upgrade Pack。可将 Mk I 压力发生器改装为 Mk II，以提高产量。

阈值	Pressure（压力）1.2
Tech id	<code>pressure_mk2_pack_unlock</code>

Geological Survey（地质勘测）

`research_geological_survey` - 尚未解锁

让声纳探测热泉。扩展 Sonar Module，使其除采矿地点外还能探测热泉。已勘测的热泉会带有各自的读数并添加到 Journal 中，从而可以找到蒸汽的采集位置。

阈值	Pressure（压力）2
Tech id	<code>geological_survey_unlock</code>

Thermal Cap（热能罩）

`research_thermal_cap` - 尚未解锁

从热泉捕获蒸汽。解锁 Thermal Cap 及其 Thermal Cap Kit 配方。在 Fabricator 中制造套件，然后让装有 Constructor（建造器）的 Pioneer 根据 Planet Map 蓝图，在已勘测的热泉上建造热能罩。热能罩会在热泉活跃阶段捕获 Steam（蒸汽），并将其释放到明确连接的目的地；远程目的地还需要兼容且已完成的 Gas Pipe（气体管道）路线。腔室达到 100% 时会释放全部储存的 Steam，因此其脚本必须释放、输送或缓解压力。将蓝图放在 Plan Mode（规划模式）中，或使用 `construction_blueprint.plan_structure("thermal_cap", x, y)`。

阈值	Pressure（压力）2.5
Tech id	<code>thermal_cap_unlock</code>

Gas Tank（气体罐）

research_steam_tank - 尚未解锁

在机器之间缓冲一种气体。解锁商店中的 Gas Tank：一个被动式 5000 吨缓冲罐，会锁定它接收的第一种气体（蒸汽、氨、沼气或任意气体），在排空前只储存该气体。

阈值	Pressure（压力）2.75
Tech id	steam_tank_unlock

Steam Turbine（蒸汽涡轮机）

research_steam_turbine - 尚未解锁

将蒸汽转化为电网电力。解锁商店中的 Steam Turbine。全油门时，它消耗 90 t/h 的 Steam 并产生 108 W；油门由脚本控制。

阈值	Pressure（压力）3
Tech id	steam_turbine_unlock

Medium Cargo Rack（中型货架）

research_cargo_rack_medium - 尚未解锁

双货箱安装架。解锁商店中的 Medium Cargo Rack 和 Heavy Portable Bin。每次运行可携带更多货物的两个货箱，并且能为第二种矿物留出空间。

阈值	Pressure（压力）4.5
Tech id	cargo_rack_medium_unlock

Wide Sonar（广域声纳）

research_sonar_wide - 尚未解锁

解锁中环勘测硬件和商店中的 Wide Sonar 模块。将扫描范围扩展至 180 m，并可探测硬度最高为 3 的矿物，包括中环和外环中的 Titanium（钛）、Cobalt（钴）与 Rare Earth（稀土）。

阈值	Pressure（压力）6
Tech id	sonar_wide_unlock

Hydrology Survey（水文勘测）

research_hydrology_survey - 尚未解锁

让声纳发现水井。解锁地下水藏勘测和 Water Pump 配方。声纳扫描现在会揭示地图各处的水井。水井会按照产量等级（1× / 2× / 3×）持续产水。在 Fabricator 中制造 Water Pump，将其部署在已勘测的水井上，然后把产物输送回基地。

阈值	Pressure（压力）30
Tech id	hydrology_survey_unlock

Deep Sonar（深层声纳）

research_sonar_deep - 尚未解锁

解锁外环勘测硬件和商店中的 Deep Sonar 模块。将扫描范围扩展至 280 m，并可在外环探测硬度 4 的矿物 Neutronium。它还会加深热能和奇异资源的勘测细节，并且在完成 Petroleum Survey 后才能用于揭示油井。

阈值	Pressure（压力）60
Tech id	sonar_deep_unlock

Gene Sequencing（基因测序）

research_dna_sequencer - 尚未解锁

解锁 DNA Sequencer。DNA Sequencer 会在商店中解锁；这是一种地热机器，可以读取并重写碎片 DNA。Geothermal Bio Orders（地热生物订单）需要经过工程处理、携带特定基因的碎片。

阈值	Pressure（压力）120
Tech id	dna_sequencer_unlock

Prime Contractor（首席承包商）

research_prime_contractor - 尚未解锁

解锁顶级合约批次。在大气时代计划中解锁三个困难合约。完成这些合约会增加 Contractor Reputation；最后一批合约会随 Smart Contractor（智能承包商）到来。

阈值	Pressure（压力）130
Tech id	prime_contractor_unlock

Bulk Logistics II（批量物流 II）

research_dispatch_mk2 - 尚未解锁

四倍 Supply Dock（补给码头）吞吐量。将每个 Supply Dock 每次脉冲的发出数量从 1 单位提高到 4 单位，使有效速率从 25 units/h 跃升至 100 units/h。脉冲间隔不变，只有数据包大小按比例增加。

阈值	Pressure（压力）150
Tech id	dispatch_mk2_unlock

Pressure Generator Mk IV（压力发生器 Mk IV）

research_pressure_mk4_pack - 尚未解锁

核时代的压力等级，由燃料棒供能。解锁 Fabricator 中的 Pressure Upgrade Pack Mk IV 配方。这个最终等级完全以 Fuel Rod magazine 取代进水口；弹匣由连接的 Lead Cask 供料，功耗约为 Mk I 的 100 倍。没有燃料棒的发生器会停止生产，而不会逐渐退化。

阈值	Pressure（压力）2,000
Tech id	pressure_mk4_pack_unlock

BIOMASS（生物量）

Seed Maker（种子制造机）

research_seed_maker - 尚未解锁

将生命体组合成种子。开启 Plants（植物）等级，并解锁 Seed Maker Kit 配方。该机器消耗三种不同的已收获生命体各 1 t，以测试可行的种子配方。大多数 `combine()` 结果的 `.status == "sludge"`，不会产生物品；可行配方在这颗星球上是独一无二的。在 Fabricator 中制造套件，从 Inventory（库存）将其部署到前哨站，然后把找到的种子种入 Harvester（收割机）的田地。

阈值	Biomass（生物量）500
Tech id	seed_maker_unlock

Plant Terraformer（植物地球化改造器）

research_plant_terraformer - 尚未解锁

将收获的 Forage（饲料）转化为 Plants 进度。在 2,000 t Biomass 时解锁 Plant Terraformer Kit 配方。它是唯一能够创造永久 Plants km² 的机器。向其中投入已收获的 Forage，然后使用 Water（水）、Salt（盐）、Fertilizer（肥料）和 Growth Accelerant（生长促进剂）满足累计阶段阶梯。制造套件，将其部署到前哨站，连接供应源，并从其脚本中启用。根据收获量能够供应的程度，建造任意数量的设备。

阈值	Biomass（生物量）2,000
Tech id	plant_terraformer_unlock

Heavy Portable Battery（重型便携电池）

research_heavy_portable_battery - 尚未解锁

解锁高容量载具电池。解锁商店中的 Heavy Portable Battery。这是一块 100 Wh 的便携电池，容量为标准电池的两倍，可装入任意载具支架槽位，使远离 Vehicle Charging Station 的长途车队获得持续供能。

阈值	Biomass（生物量）3,000
Tech id	heavy_portable_battery_unlock

Smart Contractor（智能承包商）

research_smart_contractor - 尚未解锁

解锁最终合约批次。你的 Biomass 恢复成果打开了地球的最终承包商频道。解锁 Beat the System、Core Sample 和 Lattice 这三个最困难的编程合约，并完成 Contractor Reputation 阶梯。

阈值	Biomass（生物量）20,000
Tech id	smart_contractor_unlock

Large Warehouse（大型仓库）

research_high_bay_warehousing - 尚未解锁

一座建筑中可存放 15 种材料。解锁商店中的 Large Warehouse。它可在 15 个容量为 2,000 的槽位中存放 30,000 单位材料，每个槽位会锁定一种材料。容量足以为广泛的生物和工业供应链分开存储所有材料，无需替代 Drone Depot 交接或脚本化转移。

阈值	Biomass（生物量）30,000
Tech id	high_bay_warehousing_unlock

Biomass Mixer Mk II（生物量混合器 Mk II）

research_biomass_mixer_mk2_pack - 尚未解锁

每吨精华产生的生物量提高 87.5%。解锁商店中的 Biomass Mixer Mk II Upgrade Pack。一个套件可将一台已部署的 Mk I 混合器改装为：仅消耗 2.4 倍精华即可产生 4.5 倍产出，但功耗为 5 倍。这样每吨精华的生物量提高 87.5%；由于 Liquifier（液化器）的进料量固定且稀有生命体再生缓慢，这一点很重要。不存在 Mk III。

阈值	Biomass（生物量）50,000
Tech id	biomass_mixer_mk2_pack_unlock

Bulk Logistics III（批量物流 III）

research_dispatch_mk3 - 尚未解锁

16 倍 Supply Dock 吞吐量。将每个 Supply Dock 每次脉冲的发出数量提高到 16 单位，使有效速率达到 400 units/h。脉冲间隔不变，只有数据包大小按比例增加。

阈值	Biomass（生物量）130,000
Tech id	dispatch_mk3_unlock

PLANTS（植物）

Sprinkler（喷灌器）

research_sprinkler - 尚未解锁

为网格提供水。解锁 Sprinkler Kit 配方。在 Fabricator 中制造套件，然后将其部署到 Harvester 田地中；在获得供应时，它会为四个正交相邻格（正上、正下、正左和正右）浇水。升级包会改善它所支持的作物。

阈值	Plants（植物）100,000
Tech id	sprinkler_unlock

Dispenser（分配器）

research_dispenser - 尚未解锁

为网格提供盐。解锁 Dispenser Kit 配方。在 Fabricator 中制造套件，然后将其部署到 Harvester 田地中；在获得供应时，它会为四个正交相邻格（正上、正下、正左和正右）加盐。盐是 Water Pump 的副产品。

阈值	Plants（植物）300,000
Tech id	dispenser_unlock

Grow Lamp（生长灯）

research_grow_lamp - 尚未解锁

为网格提供光照。解锁 Grow Lamp Kit 配方。在 Fabricator 中制造套件，然后将其部署到 Harvester 田地中，为四个正交相邻格提供光照。升级包会改善它所支持的作物。

阈值	Plants（植物）500,000
Tech id	grow_lamp_unlock

Field Automation（田地自动化）

`research_field_automation` - 尚未解锁

自动化收获、种植和处理。解锁商店中的 Crop Automator Kit。一个自动化器可以在以自身为中心的 5×5 区域内，为最多 24 个其他格子排队执行收获、种植和处理任务。默认情况下，它会按照 FIFO（先进先出，即最早的任务先运行）逐个执行；同时，其脚本可以重新排列尚未完成的工作。

阈值	Plants（植物）620,000
Tech id	<code>field_automation_unlock</code>

Large Liquid Tank（大型液体罐）

`research_reservoir_engineering` - 尚未解锁

容量是 Liquid Tank 的 10 倍。解锁商店中的 Large Liquid Tank。它的工作方式与 Liquid Tank 完全相同：接收最先抵达的液体，并且只储存该液体，直到耗尽；但容量为 1,000 t，而不是 100 t。在植物、精华或野生动物供应必须撑过长时间间隔的地方使用它。

阈值	Plants（植物）900,000
Tech id	<code>reservoir_engineering_unlock</code>

Steam Condensation（蒸汽冷凝）

`research_steam_condenser` - 尚未解锁

将热泉蒸汽转化为清水。在 Plants 达到 1,000,000 km² 时解锁商店中的 Steam Condenser。它最多消耗 250 t/h Steam，并产出等质量的清水，全油门时功耗为 150 W。使用 Gas Tanks 跨越热泉休眠阶段，并决定有多少蒸汽转化为水，而不是用于涡轮机发电。

阈值	Plants（植物）1,000,000
Tech id	<code>steam_condenser_unlock</code>

Plant Terraformer Mk II（植物地球化改造器 Mk II）

`research_plant_terraformer_mk2` - 尚未解锁

肥料和促进剂注入器。解锁 Fabricator 中的 Plant Terraformer Mk II Upgrade Pack 配方。Mk I 没有 Fertilizer 或 Growth Accelerant 注入器，因此只能转换至 Fields 阈值，然后停止。Mk II 增加两个注入器，将吞吐量从 400 提高到 2,200 Forage/h，将每步的输入送料处理量从 16 提高到 80 个物品，并将启用功耗从 180 W 提高到 900 W。每台机器都需要自己的套件。

阈值	Plants（植物）1,250,000
Tech id	<code>plant_terraformer_mk2_unlock</code>

Wildlife（野生动物）

`research_wildlife` - 尚未解锁

复苏并饲养第一批动物。开启 Wildlife 等级，并使 Habitat（栖息地）和 Feed Maker（饲料制造机）在商店中可用。Habitat 会复苏已编目的生物并容纳繁殖群体；Feed Maker 则使用收获的 Forage 加上从各生物群系收集的生命体来制作物种饲料。保持每个群体的饲料、气体和液体区间，群体就会繁殖到其上限。该技术以繁茂的 Plants 田地为前提，因为每个饲料配方都使用 Forage；已收获的生物群系生命体提供其余成分。

阈值	Plants（植物）2,250,000
Tech id	<code>wildlife_unlock</code>

Advanced Biopolymers（高级生物聚合物）

`research_advanced_biopolymers` - 尚未解锁

制造结构复合材料。解锁 Reinforced Biopolymer（增强型生物聚合物）的 Fabricator 配方。该复合材料把成熟的石油、植物和水生产体系结合起来，形成大型地球订单和高级生物化学所需的结构材料。

阈值	Plants（植物）4,000,000
Tech id	<code>advanced_biopolymers_unlock</code>

WILDLIFE（野生动物）

Exotic Husbandry（奇异生物饲养）

research_exotic_husbandry - 尚未解锁

获取奇异气体和液体。解锁奇异资源储藏勘测、Exotic Gas Cap Kit 与 Exotic Spring Tap Kit 配方，以及商店中的 Refiner（精炼器）。现在声纳可以发现普通和非普通奇异资源储藏；稀有储藏要等到 Deep Exotics（深层奇异资源）才会出现。捕获原始奇异气体或液体，然后将其精炼成 Habitat 区间所需的流体。在 Fabricator 中制造罩和抽取器。

阈值	Wildlife（野生动物）1,000
Tech id	exotic_husbandry_unlock

Enrichment Chemistry（富集化学）

research_habitat_development - 尚未解锁

合成一种后期生物出口产品。在 2,000 Wildlife 时解锁 Enrichment Compound（富集化合物）的 Fabricator 配方。这种高级生物材料用于后期地球出口；Habitat 适应性升级用 Insight（洞察力）购买，不会消耗该材料。

阈值	Wildlife（野生动物）2,000
Tech id	habitat_development_unlock

Deep Exotics（深层奇异资源）

research_deep_exotics - 尚未解锁

精炼最稀有的奇异资源。解锁稀有奇异气体和液体的声纳发现、提取与精炼能力；这些资源用于后期 Habitat 的需求。

阈值	Wildlife（野生动物）500,000
Tech id	deep_exotics_unlock

Habitat Engineering Mk II（栖息地工程 Mk II）

research_habitat_pack_mk2 - 尚未解锁

将一个围栏的承载容量翻倍。在 Fabricator 中解锁 Habitat Mk II Upgrade Pack 配方；需要 600,000 Wildlife，并且必须先研究 Deep Exotics。每个套件可将一座 Habitat 从 Mk I 升级为 Mk II，使承载容量翻倍，同时将成熟期功耗提高到 144-168 W。Mk II 只影响容量；Insight 适应性升级控制繁殖速度和生物成本。

阈值	Wildlife（野生动物）600,000
Tech id	habitat_pack_mk2

类型 / 核心数据 (TYPES / CORE DATA)

Bounds (边界)

由 `planet.get_bounds()` 返回。

方法 (METHODS)

`.min_x`

最小 X 坐标 (米)。

返回值 (RETURNS)

number

`.max_x`

最大 X 坐标 (米)。

返回值 (RETURNS)

number

`.min_y`

最小 Y 坐标 (米)。

返回值 (RETURNS)

number

`.max_y`

最大 Y 坐标 (米)。

返回值 (RETURNS)

number

类型 / 核心数据 (TYPES / CORE DATA)

Component (组件)

由 `get_component` / `self` 返回。

方法 (METHODS)

`.id`

该组件的程序化标识符。

返回值 (RETURNS)

`string`

类型 / 核心数据 (TYPES / CORE DATA)

PointOfInterest (兴趣点)

由 `nocturna.points_of_interest()` 返回。

方法 (METHODS)

`.x`

该接触点的整数 X 坐标（距基地的米数）。可直接传给 `Rover/Pioneer self.nav.set_target(p.x, p.y)` 或无人机的 `self.go_to(p.x, p.y)`。

返回值 (RETURNS)

number

`.y`

该接触点的整数 Y 坐标（距基地的米数）。

返回值 (RETURNS)

number

`.scanned`

一旦解析该接触点，此值即为 True：对于有产出的非生物量地点，是 Rover/Pioneer 的声纳勘测；对于生物量地点，是记录生命的无人机生物扫描；对于惰性接触点，则是发现它的声纳扫描。False 表示它仍在你的工作清单中：? 仍未识别。筛选 `not p.scanned`，即可找到尚未访问的地点。

返回值 (RETURNS)

boolean

类型 / 核心数据 (TYPES / CORE DATA)

Position

由 `self.nav.get_position()` 返回。

方法 (METHODS)

`.x`

此位置快照相对于基地的 X 坐标，单位为米。

返回值 (RETURNS)

number

`.y`

此位置快照相对于基地的 Y 坐标，单位为米。

返回值 (RETURNS)

number

`.__iter__()`

依次迭代 x、y，因此可以将此位置解包，或传递给 `list()`。

返回值 (RETURNS)

iterator<number>

ScriptCommand

由 `self.peek_command()` / `self.next_command()` 返回；当 `status == "ok"` 时，作为 `.command` 返回。

方法 (METHODS)

`.id`

命令进入此脚本队列时分配的唯一命令 ID。

返回值 (RETURNS)

string

`.name`

命令名称，例如 `"return_base"`。由你的脚本决定每个名称的含义。

返回值 (RETURNS)

string

`.args`

随命令发送的 JSON 安全参数字典。对于可选参数，使用 `.get(key, default)`。

返回值 (RETURNS)

dict

`.source`

命令来源：`"editor"`、`"script"`、`"signal"` 或 `"system"`。

返回值 (RETURNS) 可能的值 (POSSIBLE VALUES)

string `"editor"`、`"script"`、`"signal"`、`"system"`

`.created_at`

创建记录值：如果提供了现实世界时间戳，则为其毫秒数；否则为入队时的模拟刻，若两者都未提供则为 0。对于游戏时序，请使用 `.tick`。

返回值 (RETURNS)

`number`

`.tick`

命令排队时的游戏刻；如果不可用，则为 `None`。

返回值 (RETURNS)

`Optional[number]`

类型 / 世界与地点 (TYPES / WORLD & SITES)

Outpost

由 `get_component(outpost_id) / get_component_by_name(outpost_name)` 返回。

方法 (METHODS)

`.id`

不可变的实例 ID (例如 `"outpost_home"`、`"outpost_1"`)。在存档生命周期内保持稳定；需要跨重命名继续运行的脚本应使用此值。

返回值 (RETURNS)

`string`

`.name()`

显示名称。主基地默认为 `"Nocturna Base"`，新建前哨站则自动生成 `"Outpost N"` 名称；可在“计算机系统”选项卡中自由重命名。此值可变：涉及持久化时，优先使用 `id`。

返回值 (RETURNS)

`string`

`.coords()`

前哨站锚点的世界坐标，格式为 `[x, y]`。主基地位于 `[0, 0]`；新建前哨站携带在规划模式中选择的位置。

返回值 (RETURNS)

`[number, number]`

`.buildings_used()`

部署在此前哨站的建筑数量。传感器、移动单元、结构枢纽和 POI 提取器不计入其中。

返回值 (RETURNS)

number

`.buildings_capacity()`

此处的软建筑阈值。每超过阈值一座计数建筑，生产和服务吞吐量都会降低。Nocturna Base 额外拥有一些初始槽位；新建前哨站使用标准阈值。

返回值 (RETURNS)

number

`.is_full()`

当前哨站达到或超过软建筑阈值时返回 `True`。阈值本身不会阻止普通部署。

返回值 (RETURNS)

boolean

`.is_home()`

当前哨站为主基地（默认名称为 "Nocturna Base"）时返回 `True`。

返回值 (RETURNS)

boolean

`.buildings(type_id?)`

返回新的 `BuildingRef` 快照列表：每个已部署建筑对应一项。传入 `type_id`（例如 `"storage_bin"`）可进行筛选；省略时返回所有建筑。每个引用都包含 `.id` / `.name` / `.type_id` / `.outpost` / `.powered` / `.position`；要读取特定类型的实时信息（`count()`、`material()`、`recipe` 等），请调用 `get_component(ref.id)` 访问完整组件 API。

参数 (PARAMETERS)

名称	类型	描述
<code>type_id</code>	<code>string</code>	可选。用于筛选的机器类型 ID（例如 <code>"storage_bin"</code> ）。省略时返回此处的所有建筑。

返回值 (RETURNS)

`list<BuildingRef>`

`.harvesting_machines(type_id?)`

返回部署在此前哨站采收田中的固定机器的最新 `HarvestingMachineRef` 快照列表。目前只有主基地 `Nocturna Base` 拥有该田地，因此新建前哨站返回空列表。列表包括生长灯（`Grow Lamps`）、喷灌器（`Sprinklers`）、分配器（`Dispensers`）和作物自动器（`Crop Automators`）；这些设备占用田地格，但不会占用建筑容量，也不会出现在 `buildings()` 中。不包括移动收割机（`Harvester`）、作物、散落物品、水井或其他 POI 提取器。传入 `type_id` 可筛选，省略时返回全部四种固定机器。使用 `get_component(ref.id)` 读取特定类型的实时信息。

参数 (PARAMETERS)

名称	类型	描述
<code>type_id</code>	<code>string</code>	可选的固定采收田机器类型 ID： <code>"grow_lamp"</code> 、 <code>"sprinkler"</code> 、 <code>"dispenser"</code> 或 <code>"crop_automator"</code> 。省略时返回田地中部署的全部固定机器。

返回值 (RETURNS)

`list<HarvestingMachineRef>`

类型 / 世界与地点 (TYPES / WORLD & SITES)

OutpostNetwork

由 `get_component("outpost_network")` 返回。

相关对象类型 (RELATED OBJECT TYPES)

OutpostRef

方法 (METHODS)

`.outposts()`

所有拥有的前哨站的只读 `OutpostRef` 快照，包括主基地。将前哨站传递给其他 API 时使用 `.id`；需要最新数量和名称时，再次调用 `outposts()`。

返回值 (RETURNS)

`list<OutpostRef>`

`.home()`

主基地的 `OutpostRef` 快照。

返回值 (RETURNS)

OutpostRef

`.nearest(x, y)`

返回距离给定世界坐标最近的自有前哨站的 `OutpostRef` 快照。

参数 (PARAMETERS)

名称	类型	描述
<code>x</code>	<code>number</code>	世界 X 坐标
<code>y</code>	<code>number</code>	世界 Y 坐标

返回值 (RETURNS)

OutpostRef

类型 / 世界与地点 (TYPES / WORLD & SITES)

OutpostRef

由 `outpost_network.outposts()` / `outpost_network.home()` / `outpost_network.nearest()` 返回。

相关对象类型 (RELATED OBJECT TYPES)

BuildingRef

HarvestingMachineRef

方法 (METHODS)

.id

不可变的前哨站 ID，例如 "outpost_home" 或 "outpost_1"。将其与 `get_component(id)` 或需要稳定目标的 API 一起使用。

返回值 (RETURNS)

string

.name

返回此引用时的显示名称。

返回值 (RETURNS)

string

.x

前哨站占地左上锚点的世界 X 坐标，单位为距基地的米数。对于针对建筑的操作，应改为路由至该建筑的 **BuildingRef.position**。

返回值 (RETURNS)

number

`.y`

前哨站占地左上锚点的世界 Y 坐标，单位为距基地的米数。对于针对建筑的操作，应改为路由至该建筑的 `BuildingRef.position`。

返回值 (RETURNS)

`number`

`.position()`

前哨站占地的左上锚点，作为 `Position` 快照返回。它标识的是前哨站，而不是某座特定建筑的停靠点。

返回值 (RETURNS)

`Position`

`.coords()`

前哨站占地的左上锚点，格式为 `[x, y]`。要路由至某座特定的服务建筑，请使用 `buildings()` 和 `BuildingRef.position`。

返回值 (RETURNS)

`[number, number]`

`.is_home`

主基地时为 `True`。

返回值 (RETURNS)

`boolean`

`.biome`

此前哨站所在的生物群系："frozen"、"coastal"、"geothermal"、"volcanic"、"deep" 之一。由前哨站位置推导。建筑通过 `self.outpost.biome` 获取自身生物群系。

返回值 (RETURNS) 可能的值 (POSSIBLE VALUES)

`string "frozen"、"coastal"、"geothermal"、"volcanic"、"deep"`

`.buildings_used`

返回此引用时部署在此前哨站的建筑数量。

返回值 (RETURNS)

`number`

`.buildings_capacity`

返回此引用时此前哨站的软建筑阈值。超过阈值的建筑会降低生产和服务吞吐量。

返回值 (RETURNS)

`number`

`.is_full`

返回此引用时此前哨站已达到或超过软建筑阈值时为 `True`。阈值本身不会阻止普通部署。

返回值 (RETURNS)

`boolean`

`.buildings(type_id?)`

返回新的 `BuildingRef` 快照列表：每个已部署建筑对应一项。传入 `type_id`（例如 `"storage_bin"`）可进行筛选；省略时返回所有建筑。每个引用都包含 `.id / .name / .type_id / .outpost / .powered / .position`；要读取特定类型的实时信息（`count()`、`material()`、`recipe` 等），请调用 `get_component(ref.id)` 访问完整组件 API。

参数 (PARAMETERS)

名称	类型	描述
<code>type_id</code>	<code>string</code>	可选。用于筛选的机器类型 ID（例如 <code>"storage_bin"</code> ）。省略时返回此处的所有建筑。

返回值 (RETURNS)

`list<BuildingRef>`

`.harvesting_machines(type_id?)`

返回部署在此前哨站采收田中的固定机器的最新 `HarvestingMachineRef` 快照列表。目前只有主基地 `Nocturna Base` 拥有该田地，因此新建前哨站返回空列表。列表包括生长灯（`Grow Lamps`）、喷灌器（`Sprinklers`）、分配器（`Dispensers`）和作物自动器（`Crop Automators`）；这些设备占用田地格，但不会占用建筑容量，也不会出现在 `buildings()` 中。不包括移动收割机（`Harvester`）、作物、散落物品、水井或其他 POI 提取器。传入 `type_id` 可筛选，省略时返回全部四种固定机器。使用 `get_component(ref.id)` 读取特定类型的实时信息。

参数 (PARAMETERS)

名称	类型	描述
<code>type_id</code>	<code>string</code>	可选的固定采收田机器类型 ID： <code>"grow_lamp"</code> 、 <code>"sprinkler"</code> 、 <code>"dispenser"</code> 或 <code>"crop_automator"</code> 。省略时返回田地中部署的全部固定机器。

返回值 (RETURNS)

`list<HarvestingMachineRef>`

类型 / 世界与地点 (TYPES / WORLD & SITES)

BuildingRef

由 `outpost.buildings()` / `outpost_network.outposts()[i].buildings()` 返回。

方法 (METHODS)

`.id`

稳定的组件 ID。使用 `get_component(id)` 访问完整的机器 API（数量、配方、材料等）。

返回值 (RETURNS)

string

`.name`

返回此引用时的显示名称（例如 "Iron Ore Bin"）。要获取最新名称或机器实时状态，请调用 `get_component(id)`。

返回值 (RETURNS)

string

`.type_id`

机器类型 ID，例如 "storage_bin"、"smelter"、"solar_generator"。永久稳定；使用它进行分支逻辑。

返回值 (RETURNS)

string

`.outpost_id`

部署此建筑的前哨站 ID。

返回值 (RETURNS)

string

`.outpost`

所属前哨站的快照反向引用，类型为 `OutpostRef`；如果无法解析链接，则为 `None`。

返回值 (RETURNS)

`Optional[OutpostRef]`

`.powered`

返回此引用时建筑的电源开关处于开启状态时为 `True`。要读取实时供电或生产信息，请调用 `get_component(id)`。

返回值 (RETURNS)

`boolean`

`.position`

世界坐标，格式为 `[x, y]`。对大多数建筑而言，这是部署锚点，且不会移动。

返回值 (RETURNS)

`[number, number]`

类型 / 世界与地点 (TYPES / WORLD & SITES)

HarvestingMachineRef

由 `outpost.harvesting_machines()` / `outpost_network.home().harvesting_machines()` 返回。

方法 (METHODS)

`.id`

稳定的组件 ID。使用 `get_component(id)` 访问完整的生长灯 (Grow Lamp)、喷灌器 (Sprinkler)、分配器 (Dispenser) 或作物自动器 (Crop Automator) API。

返回值 (RETURNS)

string

`.name`

返回此引用时的显示名称。调用 `get_component(id)` 获取当前机器状态。

返回值 (RETURNS)

string

`.type_id`

固定采收田机器类型 ID: "grow_lamp"、"sprinkler"、"dispenser" 或 "crop_automator"。

返回值 (RETURNS) 可能的值 (POSSIBLE VALUES)

string "grow_lamp"、"sprinkler"、"dispenser"、"crop_automator"

`.powered`

返回此引用时机器的电源开关处于开启状态时为 `True`。调用 `get_component(id)` 获取实时状态和供应信息。

返回值 (RETURNS)

`boolean`

`.position`

固定机器占用的采收田区域，例如 `"B22"`。

返回值 (RETURNS)

`string`

类型 / 世界与地点 (TYPES / WORLD & SITES)

Planet

由 `transmitter.list_planets()` 返回。

方法 (METHODS)

`.id`

行星 ID：将此值传递给 `transmitter.connect()`。

返回值 (RETURNS)

`string`

`.name`

显示名称。

返回值 (RETURNS)

`string`

`.description`

行星简介。

返回值 (RETURNS)

`string`

类型 / 世界与地点 (TYPES / WORLD & SITES)

ScanResult

由 `scanner.scan()`、`harvester.collect()` 返回。

方法 (METHODS)

`.status`

稳定的结果代码。扫描器结果使用 "ok" / "empty"；收割机 (Harvester) 结果还可能报告 "holding"、"overheated"、"moving" (运输中)、"busy" (已被其他操作占用) 或 "collecting"。

返回值 (RETURNS) 可能的值 (POSSIBLE VALUES)

`string` "ok"、"empty"、"holding"、"overheated"、"moving"、"busy"、"collecting"

`.message`

面向玩家的扫描或收集结果说明。

返回值 (RETURNS)

`string`

`.id`

扫描区域中物品的 ID (没有物品时为空字符串)。

返回值 (RETURNS)

`string`

.name

物品显示名称。

返回值 (RETURNS)

string

.value

物品的信用点价值。

返回值 (RETURNS)

number

类型 / 世界与地点 (TYPES / WORLD & SITES)

Site

抽象类型 (ABSTRACT)

由 `SonarModule.scan().sites/SonarModule.survey().site` / 日志和地点绑定的机器查询返回。

具体子类型 (CONCRETE SUBTYPES)

`ExoticDeposit`

`GeologicalAnomaly`

`MiningSite`

`OilWell`

`ThermalVent`

`WaterWell`

方法 (METHODS)

`.id`

地点唯一标识符。

返回值 (RETURNS)

`string`

`.name`

便于人类阅读的显示名称。

返回值 (RETURNS)

`string`

`.x`

地点相对于基地的 X 坐标，单位为米。

返回值 (RETURNS)

number

`.y`

地点相对于基地的 Y 坐标，单位为米。

返回值 (RETURNS)

number

`.kind()`

返回 "mineral"/"thermal"/"water"/"oil"/"exotic"/"inert" 之一。使用它缩小到具体子类型：当 `site.kind() == "mineral"` 时，编辑器会在 `site` 上显示 MiningSite 专属字段。"inert" 表示声纳解析出了物理构造，但没有可提取信号。

返回值 (RETURNS) 可能的值 (POSSIBLE VALUES)

string "mineral"、"thermal"、"water"、"oil"、"exotic"、"inert"

`.position()`

带有 `.x` / `.y` 世界坐标的位置快照。

返回值 (RETURNS)

Position

`.surveyed`

当此对象返回时捕获的调查标志（Survey flag）；已解析的惰性接触点也会报告 **True**。该属性不会更新。采矿点字段是快照。井、喷口和奇异矿床方法会读取实时状态，但预调查声纳结果除外：它们会保留隐藏值，因此调查完成后应获取新对象。

返回值（RETURNS）

boolean

类型 / 世界与地点 (TYPES / WORLD & SITES)

ExoticDeposit

继承自 Site

返回来源：任何返回 Site 的 API，其中 `kind() == "exotic"`（例如 `exotic_gas_cap.deposit()`、`exotic_spring_tap.deposit()`、声纳 / 日志查询）。

方法 (METHODS)

`.id`

唯一的地点标识符。

返回值 (RETURNS)

string

`.name`

面向用户显示的可读名称。

返回值 (RETURNS)

string

`.x`

地点相对于基地的 X 坐标，单位为米。

返回值 (RETURNS)

number

`.y`

地点相对于基地的 Y 坐标，单位为米。

返回值 (RETURNS)

number

`.kind()`

取值为 "mineral" / "thermal" / "water" / "oil" / "exotic" / "inert" 之一。使用它来缩小到具体子类型：例如 `if site.kind() == "mineral":` 之后，编辑器会在 `site` 上显示 `MiningSite` 专用字段。`"inert"` 表示声纳解析出了一个物理构造，但没有可提取信号。

返回值 (RETURNS)	可能值 (POSSIBLE VALUES)
string	"mineral"、"thermal"、"water"、"oil"、"exotic"、"inert"

`.position()`

包含世界坐标 `.x` / `.y` 的位置快照。

返回值 (RETURNS)

`Position`

`.surveyed`

当此对象返回时捕获的调查标志；已解析的惰性接触点也会报告 `True`。该属性不会更新。采矿点字段是快照。井、喷口和奇异矿床方法会读取实时状态，但预调查声纳结果除外：它们会保留隐藏值，因此调查完成后应获取新对象。

返回值 (RETURNS)

`boolean`

.fluid()

该矿床释放的流体 ID，例如 "ammonia"（常见，可直接使用）或 "raw_chlorine"（稀有，需要 Refiner）。调查完成前为 None。

返回值 (RETURNS)	可能值 (POSSIBLE VALUES)
Optional[string]	"ammonia"、"swamp_gas"、"raw_sulfur_gas"、"raw_chlorine"、"brine"、"raw_cryofluid"、"raw_quicksilver"

.medium()

"gas"（使用 Exotic Gas Cap 进行抽取）或 "liquid"（使用 Exotic Spring Tap 进行抽取）。调查完成前为 None。

返回值 (RETURNS)	可能值 (POSSIBLE VALUES)
Optional[string]	"gas"、"liquid"

.rarity()

"common" 表示无需精炼即可释放可用流体；"uncommon" 和 "rare" 表示会产生由 Refiner 使用焦油转换的原料。越稀有的矿床越少见，且休眠时间越长。调查完成前为 None。

返回值 (RETURNS)	可能值 (POSSIBLE VALUES)
Optional[string]	"common"、"uncommon"、"rare"

.survey_level()

该矿床达到的最高调查层级："basic" / "wide" / "deep"，如果尚未调查则为 None。实时读取：更深层级的重新调查也会升级已持有的 Site 对象。更高层级会解锁下面的更多字段。预调查声纳结果仍保持未揭示；调查后应获取新对象。

返回值 (RETURNS)	可能值 (POSSIBLE VALUES)
Optional[string]	"basic"、"wide"、"deep"

`.current_phase()`

矿床当前的阶段："active"（活跃、正在释放）或 "dormant"（休眠、闲置）。实时读取：从已持有的 Site 对象轮询时会跟随循环变化。调查前返回 None。预调查声纳结果仍保持未揭示；调查后应获取新对象。

返回值 (RETURNS)	可能值 (POSSIBLE VALUES)
Optional[string]	"active"、"dormant"

`.cycle_active_minutes()`

活跃阶段的持续时间，单位为分钟。需要深度调查；否则返回 None。

返回值 (RETURNS)

Optional[number]

`.cycle_dormant_minutes()`

休眠阶段的持续时间，单位为分钟（稀有矿床的休眠时间最长）。需要深度调查；否则返回 None。

返回值 (RETURNS)

Optional[number]

`.next_phase_in()`

距离下一次阶段切换还剩的游戏分钟数。实时读取：可在控制循环中轮询，以便在进入休眠前采取行动。需要深度调查；否则返回 None。预调查声纳结果仍保持未揭示；调查后应获取新对象。

返回值 (RETURNS)

Optional[number]

`.base_rate()`

活跃阶段的峰值产出速率（吨/小时）。需要广域调查；基础调查时返回 **None**。

返回值 (RETURNS)

`Optional[number]`

`.current_rate()`

当前产出速率（吨/小时；休眠期间为 **0**）。实时读取。需要广域调查；基础调查时返回 **None**。预调查声纳结果仍保持未揭示；调查后应获取新对象。

返回值 (RETURNS)

`Optional[number]`

`.has_cap()`

布尔值：如果当前在此矿床上部署了 **Exotic Gas Cap** 或 **Exotic Spring Tap**，则为 **True**。实时读取。预调查声纳结果仍保持未揭示；调查后应获取新对象。

返回值 (RETURNS)

`boolean`

`.cap_id()`

当前部署的盖帽 / 抽取器的机器 ID；没有部署时为空字符串。实时读取。预调查声纳结果仍保持未揭示；调查后应获取新对象。

返回值 (RETURNS)

`string`

类型 / 世界与地点 (TYPES / WORLD & SITES)

GeologicalAnomaly

继承自 Site

返回来源：任何返回 Site 的 API, 其中 `kind() == "inert"`。

方法 (METHODS)

`.id`

唯一的地点标识符。

返回值 (RETURNS)

`string`

`.name`

面向用户显示的可读名称。

返回值 (RETURNS)

`string`

`.x`

地点相对于基地的 X 坐标, 单位为米。

返回值 (RETURNS)

`number`

`.y`

地点相对于基地的 Y 坐标, 单位为米。

返回值 (RETURNS)

`number`

.kind()

取值为 "mineral" / "thermal" / "water" / "oil" / "exotic" / "inert" 之一。使用它来缩小到具体子类型：例如 if site.kind() == "mineral": 之后，编辑器会在 site 上显示 MiningSite 专用字段。"inert" 表示声纳解析出了一个物理构造，但没有可提取信号。

返回值 (RETURNS)	可能值 (POSSIBLE VALUES)
string	"mineral"、"thermal"、"water"、"oil"、"exotic"、"inert"

.position()

包含世界坐标 .x / .y 的位置快照。

返回值 (RETURNS)

Position

.surveyed

当此对象返回时捕获的调查标志；已解析的惰性接触点也会报告 True。该属性不会更新。采矿点字段是快照。井、喷口和奇异矿床方法会读取实时状态，但预调查声纳结果除外：它们会保留隐藏值，因此调查完成后应获取新对象。

返回值 (RETURNS)

boolean

类型 / 世界与地点 (TYPES / WORLD & SITES)

MiningSite

继承自 Site

返回来源：任何返回 Site 的 API, 其中 `kind() == "mineral"`。

方法 (METHODS)

`.id`

唯一的地点标识符。

返回值 (RETURNS)

string

`.name`

面向用户显示的可读名称。

返回值 (RETURNS)

string

`.x`

地点相对于基地的 X 坐标, 单位为米。

返回值 (RETURNS)

number

`.y`

地点相对于基地的 Y 坐标, 单位为米。

返回值 (RETURNS)

number

.kind()

取值为 "mineral" / "thermal" / "water" / "oil" / "exotic" / "inert" 之一。使用它来缩小到具体子类型：例如 if site.kind() == "mineral": 之后，编辑器会在 site 上显示 MiningSite 专用字段。"inert" 表示声纳解析出了一个物理构造，但没有可提取信号。

返回值 (RETURNS)	可能值 (POSSIBLE VALUES)
string	"mineral", "thermal", "water", "oil", "exotic", "inert"

.position()

包含世界坐标 .x / .y 的位置快照。

返回值 (RETURNS)

Position

.surveyed

当此对象返回时捕获的调查标志；已解析的惰性接触点也会报告 True。该属性不会更新。采矿点字段是快照。井、喷口和奇异矿床方法会读取实时状态，但预调查声纳结果除外：它们会保留隐藏值，因此调查完成后应获取新对象。

返回值 (RETURNS)

boolean

.item_id

钻探该地点时产出的物品 ID（例如 "iron_ore"、"silicon"）。调查完成前为 None。

返回值 (RETURNS)	可能值 (POSSIBLE VALUES)
Optional[string]	"iron_ore", "silicon", "titanium", "cobalt", "rare_earth", "neutronium", "lead_ore"

`.hardness`

硬度等级（1-4）：决定钻机兼容性。调查完成前为 **None**。

返回值（RETURNS）

`Optional[number]`

`.purity`

产量倍率等级：`"standard"`（1×） / `"rich"`（2×） / `"pure"`（3×）。调查完成前为 **None**。

返回值（RETURNS）	可能值（POSSIBLE VALUES）
<code>Optional[string]</code>	<code>"standard"</code> 、 <code>"rich"</code> 、 <code>"pure"</code>

类型 / 世界与地点 (TYPES / WORLD & SITES)

OilWell

继承自 Site

返回来源：任何返回 Site 的 API，其中 `kind() == "oil"`（例如 `oil_pump.well()`、声纳 / 日志查询）。

方法 (METHODS)

`.id`

唯一的地点标识符。

返回值 (RETURNS)

string

`.name`

面向用户显示的可读名称。

返回值 (RETURNS)

string

`.x`

地点相对于基地的 X 坐标，单位为米。

返回值 (RETURNS)

number

`.y`

地点相对于基地的 Y 坐标，单位为米。

返回值 (RETURNS)

number

.kind()

取值为 "mineral" / "thermal" / "water" / "oil" / "exotic" / "inert" 之一。使用它来缩小到具体子类型：例如 if site.kind() == "mineral": 之后，编辑器会在 site 上显示 MiningSite 专用字段。"inert" 表示声纳解析出了一个物理构造，但没有可提取信号。

返回值 (RETURNS)	可能值 (POSSIBLE VALUES)
string	"mineral"、"thermal"、"water"、"oil"、"exotic"、"inert"

.position()

包含世界坐标 .x / .y 的位置快照。

返回值 (RETURNS)

Position

.surveyed

当此对象返回时捕获的调查标志；已解析的惰性接触点也会报告 True。该属性不会更新。采矿点字段是快照。井、喷口和奇异矿床方法会读取实时状态，但预调查声纳结果除外：它们会保留隐藏值，因此调查完成后应获取新对象。

返回值 (RETURNS)

boolean

.yield_tier()

取值为 "standard" (1×) / "rich" (2×) / "pure" (3×) 之一。调查完成前为 None。检查当前调查进度，但预调查声纳结果会持续返回 None；调查后应获取新对象。

返回值 (RETURNS)	可能值 (POSSIBLE VALUES)
Optional[string]	"standard"、"rich"、"pure"

`.flow_rate()`

峰值石油流量，单位为吨/小时。对于 `standard/rich/pure`，分别为 8/16/24。油井会在活跃和休眠阶段之间脉冲运行：休眠油井在任何节流设置下都不会输送石油（读取泵的 `well_active()`）。调查完成前为 `None`。实时读取，但预调查声纳结果会持续返回 `None`；调查后应获取新对象。

返回值 (RETURNS)

`Optional[number]`

`.has_pump()`

布尔值：如果当前在此油井上部署了 `Oil Pump`，则为 `True`。预调查声纳结果始终返回 `False`；调查后应获取新对象以读取实时状态。

返回值 (RETURNS)

`boolean`

`.pump_id()`

部署在此油井上的 `Oil Pump` 的当前机器 ID；没有部署时为空字符串。预调查声纳结果始终返回空字符串；调查后应获取新对象以读取实时状态。

返回值 (RETURNS)

`string`

类型 / 世界与地点 (TYPES / WORLD & SITES)

ThermalVent

继承自 Site

返回来源：任何返回 Site 的 API，其中 `kind() == "thermal"`（例如 `thermal_cap.vent()`、声纳 / 日志查询）。

方法 (METHODS)

`.id`

唯一的地点标识符。

返回值 (RETURNS)

string

`.name`

面向用户显示的可读名称。

返回值 (RETURNS)

string

`.x`

地点相对于基地的 X 坐标，单位为米。

返回值 (RETURNS)

number

`.y`

地点相对于基地的 Y 坐标，单位为米。

返回值 (RETURNS)

number

.kind()

取值为 "mineral" / "thermal" / "water" / "oil" / "exotic" / "inert" 之一。使用它来缩小到具体子类型：例如 if site.kind() == "mineral": 之后，编辑器会在 site 上显示 MiningSite 专用字段。"inert" 表示声纳解析出了一个物理构造，但没有可提取信号。

返回值 (RETURNS)	可能值 (POSSIBLE VALUES)
string	"mineral"、"thermal"、"water"、"oil"、"exotic"、"inert"

.position()

包含世界坐标 .x / .y 的位置快照。

返回值 (RETURNS)

Position

.surveyed

当此对象返回时捕获的调查标志；已解析的惰性接触点也会报告 True。该属性不会更新。采矿点字段是快照。井、喷口和奇异矿床方法会读取实时状态，但预调查声纳结果除外：它们会保留隐藏值，因此调查完成后应获取新对象。

返回值 (RETURNS)

boolean

.survey_level()

该喷口达到的最高调查层级："basic" / "wide" / "deep"，如果尚未调查则为 None。实时读取：更深层级的重新调查也会升级已持有的 Site 对象。更高层级会解锁下面的更多字段。预调查声纳结果仍保持未揭示；调查后应获取新对象。

返回值 (RETURNS)	可能值 (POSSIBLE VALUES)
Optional[string]	"basic"、"wide"、"deep"

`.cycle_active_minutes()`

活跃阶段的持续时间，单位为分钟。需要深度调查；否则返回 `None`。

返回值 (RETURNS)

`Optional[number]`

`.cycle_dormant_minutes()`

休眠阶段的持续时间，单位为分钟。需要深度调查；否则返回 `None`。

返回值 (RETURNS)

`Optional[number]`

`.current_phase()`

喷口当前的阶段：`"active"` 或 `"dormant"`。实时读取：从已持有的 `Site` 对象轮询时会跟随循环变化。调查前返回 `None`。预调查声纳结果仍保持未揭示；调查后应获取新对象。

返回值 (RETURNS)	可能值 (POSSIBLE VALUES)
<code>Optional[string]</code>	<code>"active"</code> 、 <code>"dormant"</code>

`.next_phase_in()`

距离下一次阶段切换还剩的游戏分钟数。实时读取：可在控制循环中轮询，以便在进入休眠前采取行动。需要深度调查；否则返回 `None`。预调查声纳结果仍保持未揭示；调查后应获取新对象。

返回值 (RETURNS)

`Optional[number]`

`.base_steam_rate()`

活跃阶段的峰值蒸汽速率（吨/小时）。需要广域调查；基础调查时返回 `None`。

返回值 (RETURNS)

`Optional[number]`

`.current_steam_rate()`

当前蒸汽速率（吨/小时；休眠阶段为 `0`）。实时读取。需要广域调查；基础调查时返回 `None`。预调查声纳结果仍保持未揭示；调查后应获取新对象。

返回值 (RETURNS)

`Optional[number]`

`.has_cap()`

布尔值：如果当前在此喷口上部署了 `Thermal Cap`，则为 `True`。实时读取。预调查声纳结果仍保持未揭示；调查后应获取新对象。

返回值 (RETURNS)

`boolean`

`.cap_id()`

当前部署的 `Thermal Cap` 的机器 ID；没有部署盖帽时为空字符串。实时读取。预调查声纳结果仍保持未揭示；调查后应获取新对象。

返回值 (RETURNS)

`string`

类型 / 世界与地点 (TYPES / WORLD & SITES)

WaterWell

继承自 Site

返回来源：任何返回 Site 的 API，其中 `kind() == "water"`（例如 `water_pump.well()`、声纳 / 日志查询）。

方法 (METHODS)

`.id`

唯一的地点标识符。

返回值 (RETURNS)

string

`.name`

面向用户显示的可读名称。

返回值 (RETURNS)

string

`.x`

地点相对于基地的 X 坐标，单位为米。

返回值 (RETURNS)

number

`.y`

地点相对于基地的 Y 坐标，单位为米。

返回值 (RETURNS)

number

.kind()

取值为 "mineral" / "thermal" / "water" / "oil" / "exotic" / "inert" 之一。使用它来缩小到具体子类型：例如在 `if site.kind() == "mineral":` 之后，编辑器会在 `site` 上显示 `MiningSite` 专用字段。"inert" 表示声纳解析出了一个物理构造，但没有可提取信号。

返回值 (RETURNS)	可能值 (POSSIBLE VALUES)
string	"mineral"、"thermal"、"water"、"oil"、"exotic"、"inert"

.position()

包含世界坐标 `.x` / `.y` 的位置快照。

返回值 (RETURNS)

Position

.surveyed

当此对象返回时捕获的调查标志 (Survey flag) ；已解析的惰性接触点也会报告 `True`。该属性不会更新。采矿点字段是快照。井、喷口和奇异矿床方法会读取实时状态，但预调查声纳结果除外：它们会保留隐藏值，因此调查完成后应获取新对象。

返回值 (RETURNS)

boolean

.yield_tier()

取值为 "standard" (1×) / "rich" (2×) / "pure" (3×) 之一。调查完成前为 `None`。该方法会检查当前调查进度；但对于预调查声纳结果，它会持续返回 `None`，因此调查完成后应获取新对象。并在基础调查时就会完整揭示。

返回值 (RETURNS)	可能值 (POSSIBLE VALUES)
Optional[string]	"standard"、"rich"、"pure"

`.flow_rate()`

该井每小时产生的水量（吨）。对于 `standard/rich/pure`，分别为 10 / 20 / 30。调查完成前为 `None`。该方法读取实时状态，但预调查声纳结果会持续返回 `None`；调查完成后应获取新对象。

返回值 (RETURNS)

`Optional[number]`

`.has_pump()`

布尔值：如果当前在该井上部署了 Water Pump，则为 `True`。预调查声纳结果始终返回 `False`；调查完成后应获取新对象以读取实时状态。

返回值 (RETURNS)

`boolean`

`.pump_id()`

当前部署在该井上的 Water Pump 机器 ID；没有泵时为空字符串。预调查声纳结果始终返回空字符串；调查完成后应获取新对象以读取实时状态。

返回值 (RETURNS)

`string`

类型 / 存储与库存 (TYPES / STORAGE & INVENTORY)

Battery

返回来源：`self.battery` (车辆)

相关对象类型 (RELATED OBJECT TYPES)

- `Holder`
- `PortableBattery`

方法 (METHODS)

`.level()`

以 0-1 之间的分数表示的充电电量。

返回值 (RETURNS)

`number`

`.wh()`

当前电量，单位为 Wh (所有电池合计)。

返回值 (RETURNS)

`number`

`.capacity()`

最大容量，单位为 Wh。

返回值 (RETURNS)

`number`

`.holders()`

车辆上当前安装的所有 Battery Holder 列表。Rover 使用密封电池，因此该列表为空。

返回值 (RETURNS)

`list<Holder>`

类型 / 存储与库存 (TYPES / STORAGE & INVENTORY)

Holder

返回来源：`self.battery.holders()`

方法 (METHODS)

`.id`

Holder 模块 ID，例如 `"battery_holder_medium"`。

返回值 (RETURNS)

`string`

`.size`

Holder 尺寸：`"small" / "medium" / "large"`。

返回值 (RETURNS)	可能值 (POSSIBLE VALUES)
<code>string</code>	<code>"small"、"medium"、"large"</code>

`.capacity`

该 Holder 中所有电池的额定 Wh 总量。

返回值 (RETURNS)

`number`

`.wh`

该 Holder 中当前的 Wh 数（电量池水平 × 额定容量）。

返回值 (RETURNS)

`number`

.batteries

按内部插槽索引的列表：每个条目是 `PortableBattery` 对象，或者表示空插槽的 `None`。

返回值 (RETURNS)

```
list<Optional[PortableBattery]>
```

类型 / 存储与库存 (TYPES / STORAGE & INVENTORY)

PortableBattery

返回来源：`self.battery.holders()[...].batteries[...]`

方法 (METHODS)

`.id`

便携式电池物品 ID：`"portable_battery"` 或 `"heavy_portable_battery"`。

返回值 (RETURNS)	可能值 (POSSIBLE VALUES)
<code>string</code>	<code>"portable_battery"</code> 、 <code>"heavy_portable_battery"</code>

`.level()`

以 0-1 之间的分数表示的充电电量。

返回值 (RETURNS)

`number`

`.wh()`

当前电量，单位为 Wh。

返回值 (RETURNS)

`number`

`.capacity()`

额定容量，单位为 Wh。

返回值 (RETURNS)

`number`

类型 / 存储与库存 (TYPES / STORAGE & INVENTORY)

Cargo

返回来源：`self.cargo` (车辆)

相关对象类型 (RELATED OBJECT TYPES)

- Rack

方法 (METHODS)

`.count()`

车辆当前携带的总单位数。

返回值 (RETURNS)

number

`.capacity()`

车辆一体式货舱或已安装 Cargo Rack 能够携带的总单位数。

返回值 (RETURNS)

number

`.stacks()`

车辆货物中每个属性不同的 `ItemStack` 快照列表。使用它可以区分共享同一物品 ID 的不同变体。

返回值 (RETURNS)

`List<ItemStack>`

`.full()`

当一体式货舱或所有已安装货物箱都已装满时为 **True**。未安装货物箱的 Pioneer 也视为已满。

返回值 (RETURNS)

boolean

`.racks()`

当前安装的所有 Cargo Rack 列表。Rover 没有货架，因此该列表为空。

返回值 (RETURNS)

list<Rack>

`.compact()`

在 Pioneer 上，将相同物品 ID 集中到能够容纳这些物品的最少数量已安装 Portable Bin 中。属性不同的变体会保持独立；规划器会保留兼容货箱中装得最满的那些，以尽量减少实体搬运。需要 Auto Feeders；等待时间与重新定位的单位数成正比；在该周期内，Pioneer 会作为物料端点保持静止。此操作不会改变 `send()` 首先排空哪个货箱。

返回值 (RETURNS)	结果字段 (RESULT FIELDS)	成功载荷 (SUCCESS PAYLOAD)
TransferResult	.status、.message	.requested、.moved

结果 (OUTCOMES)

状态 (Status)	类型 (Kind)	含义 (Meaning)
"ok"	成功	将 .moved 个单位重新放入更少的 Pioneer Portable Bin。
"already_compact"	成功	该货物已经集中；无需移动 Portable Bin 中的货物。
"research_required"	拒绝	所需的物料转移研究尚未解锁。
"busy"	暂时性	此物料端点正在处理其他物料操作。
"source_changed"	暂时性	源在转移规划与提交之间发生了变化。
"target_full"	拒绝	目标没有可容纳匹配单位的容量。

`.discard(rack_index)`

永久抛弃指定 `rack_index` 中的全部物品；该索引必须是从零开始的整数。在没有货架的 Rover 上, `self.cargo.discard(0)` 指向一体式货舱。已经为空的货舱或货架会立即完成；销毁货物需要 1 小时, 并会暂停脚本。这是车辆货物销毁 API。

参数 (PARAMETERS)

名称 (Name)	类型 (Type)	描述 (Description)
<code>rack_index</code>	<code>number</code>	已安装 Cargo Rack 中从零开始的整数索引 (Rover 的一体式货舱为 0)。
返回值 (RETURNS)	结果字段 (RESULT FIELDS)	成功载荷 (SUCCESS PAYLOAD)
<code>DiscardResult</code>	<code>.status</code> 、 <code>.message</code>	<code>.requested</code> 、 <code>.discarded</code>

结果 (OUTCOMES)

状态 (Status)	类型 (Kind)	含义 (Meaning)
"ok"	成功	永久丢弃 <code>.discarded</code> 个单位。
"empty"	成功	所选货物区域原本就是空的。
"busy"	暂时性	车辆正忙于其他现场操作。
"invalid_rack"	拒绝	请求的货架不存在。

类型 / 存储与库存 (TYPES / STORAGE & INVENTORY)

Rack

返回来源：`self.cargo.racks()`

相关对象类型 (RELATED OBJECT TYPES)

- `Bin`

方法 (METHODS)

`.id`

货架模块 ID，例如 `"cargo_rack_medium"`。

返回值 (RETURNS)

`string`

`.size`

货架尺寸：`"small" / "medium" / "large"`。

返回值 (RETURNS)	可能值 (POSSIBLE VALUES)
<code>string</code>	<code>"small"、"medium"、"large"</code>

`.bins`

按内部插槽索引的列表：每个条目是 `Bin` 对象，或者表示空插槽的 `None`。

返回值 (RETURNS)

`list<Optional[Bin]>`

类型 / 存储与库存 (TYPES / STORAGE & INVENTORY)

Bin

返回来源：`Rack.bins`

方法 (METHODS)

`.id`

便携式货箱物品 ID：`"portable_bin"` 或 `"heavy_portable_bin"`。

返回值 (RETURNS)	可能值 (POSSIBLE VALUES)
<code>string</code>	<code>"portable_bin"</code> 、 <code>"heavy_portable_bin"</code>

`.capacity`

额定单位数：普通货箱为 25，重型货箱为 50。

返回值 (RETURNS)

`number`

`.count`

当前存储的单位数。

返回值 (RETURNS)

`number`

`.item_id`

已分配的物品 ID（矿物、锭、组件：即货箱所容纳的任何物品）；货箱为空或未分配时为 `None`。

返回值 (RETURNS)

`Optional[string]`

`.stacks`

该便携式货箱中属性不同的 `ItemStack` 快照。属性不同但 ID 相同的条目仍会分开列出。

返回值 (RETURNS)

`list<ItemStack>`

类型 / 存储与库存 (TYPES / STORAGE & INVENTORY)

DiscardResult

返回来源：Cargo.discard()、DroneCargo.discard()

方法 (METHODS)

.status

稳定的丢弃结果代码。

返回值 (RETURNS)		可能值 (POSSIBLE VALUES)
string	"ok"、"partial"、"empty"、"busy"、"invalid_rack"、"no_op"、"invalid_properties"、"invalid_property_match"、"source_changed"	

.message

面向玩家的丢弃结果说明。

返回值 (RETURNS)

string

.requested

请求销毁或实际观察到的单位数，必须是整数。

返回值 (RETURNS)

number

.discarded

永久销毁的整数单位数。

返回值 (RETURNS)

number

类型 / 存储与库存 (TYPES / STORAGE & INVENTORY)

InputSlot

返回来源：带有输入缓冲区的固定机器的 `self.input`

方法 (METHODS)

`.connect(name)`

通过稳定 ID 或显示名称设置兼容的物品源。两个固定端点必须共享同一前哨站。Rover 或 Pioneer 可从任何前哨站访问，但只有在停放于该机器服务区域内时才可访问。现场提取器的拾取输出只能由 Rover 或 Pioneer 拉取。无人机货物通过其 Drone Depot 移动。"`inventory`" 只有在端点位于 Nocturna Base 时才是货运源；远程固定端口使用本地 Storage Bin、Warehouse 或机器缓冲区。

参数 (PARAMETERS)

名称 (Name)	类型 (Type)	描述 (Description)
<code>name</code>	<code>string</code>	兼容物品源的稳定 ID 或显示名称，或 <code>inventory</code> 。
返回值 (RETURNS)	结果字段 (RESULT FIELDS)	成功载荷 (SUCCESS PAYLOAD)
<code>ActionResult</code>	<code>.status</code> 、 <code>.message</code>	<code>None</code>

结果 (OUTCOMES)

状态 (Status)	类型 (Kind)	含义 (Meaning)
<code>"ok"</code>	成功	操作成功完成。
<code>"not_found"</code>	拒绝	请求的对象、目标或记录不存在。
<code>"not_local"</code>	拒绝	请求的端点属于另一个前哨站。
<code>"same_endpoint"</code>	拒绝	请求的物品源与目标端点相同。
<code>"unsupported_source"</code>	拒绝	所选组件没有公开兼容的物品源。
<code>"source_is_vehicle"</code>	拒绝	请求的源是无人机。无人机货物通过其 Drone Depot 移动。

`.disconnect()`

清除当前源连接。不会移动或丢弃缓冲物品；使用 `eject(...)` 回收它们，或使用 `flush()` 销毁它们。

返回值 (RETURNS)	结果字段 (RESULT FIELDS)	成功载荷 (SUCCESS PAYLOAD)
<code>ActionResult</code>	<code>.status</code> 、 <code>.message</code>	<code>None</code>

结果 (OUTCOMES)

状态 (Status)	类型 (Kind)	含义 (Meaning)
<code>"ok"</code>	成功	操作成功完成。

`.connected_to()`

当前连接源的显示名称；未连接时为空字符串。

返回值 (RETURNS)

`string`

`.connected_id()`

当前连接源的稳定 ID；未连接时为空字符串。`connect()` 接受 ID 或显示名称，因此如果需要确认身份与传入值一致，应与此值比较：`connected_to()` 返回的是可重命名的名称。

返回值 (RETURNS)

`string`

`.take(item_id, count, properties=None, property_match=None)`

从已连接的源中取出最多 `count` 个 `item_id` 单位；默认情况下，属性字典会选择包含该子集的堆栈。第四个参数传入 `"exact"` 可选择完整身份；`None` 与 `"exact"` 一起使用时，只选择无属性物品。`"any"` 会忽略属性。源的精确属性始终会保留。转移会自动等待与移动单位数成正比的时间，并且需要 Auto Feeders 研究。

参数 (PARAMETERS)

名称 (Name)	类型 (Type)	描述 (Description)
<code>item_id</code>	<code>string</code>	要取出的物品 ID。
<code>count</code>	<code>number</code>	要取出的最大整数单位数。
<code>properties</code>	<code>any</code>	可选属性字典，默认按子集匹配。省略或为 <code>None</code> 时匹配任意属性；与 <code>property_match="exact"</code> 一起使用 <code>None</code> ，只选择无属性物品。
<code>property_match</code>	<code>string</code>	可选选择模式： <code>any</code> 、 <code>subset</code> 或 <code>exact</code> 。
返回值 (RETURNS)	结果字段 (RESULT FIELDS)	成功载荷 (SUCCESS PAYLOAD)
<code>TransferResult</code>	<code>.status</code> 、 <code>.message</code>	<code>.requested</code> 、 <code>.moved</code>

结果 (OUTCOMES)

状态 (Status)	类型 (Kind)	含义 (Meaning)
<code>"ok"</code>	成功	移动了全部 <code>.moved</code> 个请求单位。
<code>"partial"</code>	部分成功	请求了 <code>.requested</code> 个单位，但只移动了 <code>.moved</code> 个；源可用量或目标容量限制了转移。
<code>"no_op"</code>	成功	未请求任何单位，因此状态没有变化。
<code>"research_required"</code>	拒绝	所需的物料转移研究尚未解锁。
<code>"busy"</code>	暂时性	此物料端点正在处理其他物料操作。
<code>"invalid_properties"</code>	拒绝	物品属性选择器的形状或值无效。
<code>"invalid_property_match"</code>	拒绝	请求的属性匹配模式无效。
<code>"no_connection"</code>	拒绝	端口没有配置转移端点。
<code>"inventory_not_local"</code>	拒绝	此端点无法从当前前哨站访问主货运 Inventory。
<code>"same_endpoint"</code>	拒绝	配置的源与目标是同一物品端点。
<code>"source_missing"</code>	拒绝	配置的源不再存在。
<code>"source_under_construction"</code>	暂时性	配置的源仍在建造中。

状态（Status）		类型（Kind）		含义（Meaning）
"source_is_vehicle"	拒绝		移动源是无人机，或是未停靠在此端点服务区域内的地面车辆。	
"source_not_local"	拒绝			配置的源属于另一个前哨站。
"not_at_source"	拒绝			车辆位于源的服务区域之外。
"unsupported_source"	拒绝			配置的源没有公开兼容货物。
"source_wrong_material"	拒绝			源中包含的材料与请求的材料不同。
"source_empty"	拒绝			源中没有可用的匹配单位。
"source_reserved"	拒绝			源正在为活动操作预留其货物。
"source_changed"	暂时性			源在转移规划与提交之间发生了变化。
"buffer_full"	拒绝			接收方输入缓冲区已满。
"slots_full"	拒绝			目标有容量，但没有容纳该物品身份的插槽。
"target_wrong_material"	拒绝			目标已锁定为另一种材料，或只接受另一种材料。
"wrong_biome"	拒绝			由于生物群系不兼容，目标拒绝该生物材料。
"target_unconfigured"	拒绝			目标缺少接受该材料所需的世界状态配置。
"hot_cargo_requires_cask"	拒绝			该高温货物必须通过兼容的 Lead Cask 移动。
"order_item_not_required"	拒绝			当前订单不需要该物品。
"order_slots_full"	拒绝			每个 Supply Dock 材料插槽都已分配给其他所需物品。
"order_fulfilled"	拒绝			当前订单所需的该物品单位已经全部装载或发运。
"target_full"	拒绝			目标没有可容纳匹配单位的容量。
"target_changed"	暂时性			目标在转移规划与提交之间发生了变化。
"same_vehicle"	拒绝			车辆不能将货物转移给自己。
"source_moving"	暂时性			源车辆必须停下后才能交接货物。
"target_moving"	暂时性			目标车辆必须停下后才能接收货物。

状态 (Status)	类型 (Kind)	含义 (Meaning)
"out_of_range"	拒绝	车辆位于货物交接范围之外。

`.eject(destination, item_id, count, properties=None, property_match=None)`

从该缓冲区中回收指定 `item_id` 的最多 `count` 个整数量单位，不改变其源连接。请在 Nocturna Base 使用 `inventory`，或使用同一前哨站中兼容的同类本地仓储、机器输入端或停放的地面车辆。可选的 `properties` 遵循标准的 `any`、`subset` 或 `exact` 选择规则；精确属性会被保留。目标容量可能限制移动数量。转移会自动等待与移动单位数成比例的时间，并且需要完成 Auto Feeders 研究。

参数 (PARAMETERS)

名称	类型	描述
<code>destination</code>	<code>string</code>	本地兼容货运目标的稳定 ID 或显示名称，或 <code>inventory</code>
<code>item_id</code>	<code>string</code>	要回收的物品 ID
<code>count</code>	<code>number</code>	要回收的最大整数量单位
<code>properties</code>	<code>any</code>	可选属性字典，默认按子集匹配。省略或设为 <code>None</code> 时匹配任意属性；与 <code>property_match="exact"</code> 一起使用 <code>None</code> 时，仅选择无属性物品。
<code>property_match</code>	<code>string</code>	可选的选择模式： <code>any</code> 、 <code>subset</code> 或 <code>exact</code>
返回值 (RETURNS)	结果字段 (RESULT FIELDS)	成功负载 (SUCCESS PAYLOAD)
<code>TransferResult</code>	<code>.status</code> 、 <code>.message</code>	<code>.requested</code> 、 <code>.moved</code>

结果 (OUTCOMES)

状态 (Status)	类型 (Kind)	含义 (Meaning)
<code>"ok"</code>	成功	已移动全部 <code>.moved</code> 个请求单位。
<code>"partial"</code>	部分成功	已移动 <code>.requested</code> 中的 <code>.moved</code> 个单位；源可用量或目标容量限制了转移。
<code>"no_op"</code>	成功	未请求任何单位，因此状态没有改变。
<code>"research_required"</code>	拒绝	尚未解锁所需的材料转移研究。
<code>"busy"</code>	暂时性	此材料端点正在处理其他材料操作。
<code>"invalid_properties"</code>	拒绝	物品属性选择器的形状或值无效。
<code>"invalid_property_match"</code>	拒绝	请求的属性匹配模式无效。
<code>"inventory_not_local"</code>	拒绝	此端点从当前前哨站无法访问主基地货运 <code>Inventory</code> 。
<code>"same_endpoint"</code>	拒绝	配置的源和目标是同一个物品端点。

状态 (Status)	类型 (Kind)	含义 (Meaning)
"source_under_construction"	暂时性	配置的源仍在建造中。
"source_wrong_material"	拒绝	源中包含的材料与请求材料不同。
"source_empty"	拒绝	源中没有可用的匹配单位。
"source_reserved"	拒绝	源正为活动操作预留货物。
"source_changed"	暂时性	源在转移规划与提交之间发生了变化。
"target_missing"	拒绝	配置的目标已不存在。
"target_under_construction"	暂时性	配置的目标仍在建造中。
"target_is_vehicle"	拒绝	移动目标是无人机，或是未停靠在该端点服务区域内的地面车辆。
"target_not_local"	拒绝	配置的目标属于另一个前哨站。
"unsupported_target"	拒绝	配置的目标无法接收兼容货物。
"target_wrong_material"	拒绝	目标已锁定其他材料，或只接受其他材料。
"wrong_biome"	拒绝	目标因生物群系不兼容而拒绝该生物材料。
"target_unconfigured"	拒绝	目标缺少接受该材料所需的世界状态配置。
"hot_cargo_requires_cask"	拒绝	该高温货物必须通过兼容的 Lead Cask 转移。
"cask_accepts_hot_only"	拒绝	Lead Cask 只接受受支持的高温货物。
"order_item_not_required"	拒绝	活动订单不需要该物品。
"order_slots_full"	拒绝	每个 Supply Dock 材料槽都已分配给其他所需物品。
"order_fulfilled"	拒绝	活动订单已经装载或发运该物品所需的全部单位。
"slots_full"	拒绝	目标仍有容量，但没有用于该材料标识的槽位。
"target_full"	拒绝	目标没有可容纳匹配单位的空间。
"target_changed"	暂时性	目标在转移规划与提交之间发生了变化。

类型 / 存储与库存 (TYPES / STORAGE & INVENTORY)

ItemInfo

返回来源：`item_catalog.lookup(item_id)`

方法 (METHODS)

`.id`

每个物品的稳定物品 ID。

返回值 (RETURNS)

`string`

`.name`

本地化的物品显示名称。

返回值 (RETURNS)

`string`

`.category`

用于区分材料、生物物品和设备类型的稳定类别。

返回值 (RETURNS)	可能值 (POSSIBLE VALUES)
<code>string</code>	<code>"mineral"、"refined"、"crafted"、"agriculture"、"life_form"、"field_resource"、"biology_sample"、"reagent"、"equipment"、"module"、"portable"、"upgrade_pack"、"construction_kit"</code>

.stackable

多个单位共用一个库存堆叠时为 **True**；每个单位各自占用一个槽位时为 **False**。

返回值 (RETURNS)

boolean

.biome

生命体或生物样本的原生生物群系；不适用时为 **None**。

返回值 (RETURNS)	可能值 (POSSIBLE VALUES)
Optional[string]	"frozen"、"coastal"、"geothermal"、"volcanic"、"deep"

.rarity

生命体或生物样本的稀有度；不适用时为 **None**。

返回值 (RETURNS)	可能值 (POSSIBLE VALUES)
Optional[string]	"common"、"uncommon"、"rare"、"legendary"

.production_tier

创建该物品的最低生产等级；如果物品是原始、采集、购买或以其他方式获得的来源物品，则为 **None**。冶炼厂产品为 Tier 0。

返回值 (RETURNS)

Optional[number]

类型 / 存储与库存 (TYPES / STORAGE & INVENTORY)

ItemResult

返回来源: `harvester.store()` / `inventory.drop()`

方法 (METHODS)

`.status`

取值为 "ok"、"empty"、"inventory_full" 或 "invalid_slot"；具体含义由返回该结果的命令进一步限定。

返回值 (RETURNS)	可能值 (POSSIBLE VALUES)
string	"ok"、"empty"、"inventory_full"、"invalid_slot"

`.message`

面向玩家的物品操作说明。

返回值 (RETURNS)

string

`.item_id`

被移动或销毁物品的稳定 ID；没有物品发生变化时为 `None`。

返回值 (RETURNS)

Optional[string]

类型 / 存储与库存 (TYPES / STORAGE & INVENTORY)

ItemStack

返回来源：InputSlot.stacks()、VehicleInputSlot.stacks()、OutputSlot.stacks()、PickupOutputSlot.stacks()、Cargo.stacks()、Bin.stacks、storage_bin.stacks()、warehouse.stacks()

方法 (METHODS)

`.id`

该堆叠中每个单位共享的稳定物品 ID。

返回值 (RETURNS)

`string`

`.count`

具有完全相同属性标识的整数量单位数。

返回值 (RETURNS)

`number`

`.properties`

该物品的精确属性字典；普通商品为 **None**。属性不同的物品会形成独立堆叠，且永远不会合并。

返回值 (RETURNS)

`Optional[dict]`

类型 / 存储与库存 (TYPES / STORAGE & INVENTORY)

OutputSlot

返回来源：`self.output`（带输出端口的机器）

方法 (METHODS)

`.connect(name)`

按稳定 ID 或显示名称设置兼容的物品目标。两个固定端点必须位于同一前哨站。Rover 或 Pioneer 可从任何前哨站访问，但只有在停放于该机器服务区域内时才可访问。无人机货物通过其 Drone Depot 移动。

`inventory` 只有在端点位于 Nocturna Base 时才是货运目标；远程固定端口请使用本地 Storage Bin、Warehouse 或机器输入端。

参数名称 (Name)	类型 (Type)	描述 (Description)
<code>name</code>	<code>string</code>	兼容物品目标的稳定 ID 或显示名称，或 <code>inventory</code>
返回值 (RETURNS)	结果字段 (RESULT FIELDS)	成功负载 (SUCCESS PAYLOAD)
<code>ActionResult</code>	<code>.status</code> 、 <code>.message</code>	<code>None</code>

结果 (OUTCOMES)

状态 (Status)	类型 (Kind)	含义 (Meaning)
<code>"ok"</code>	成功	操作成功完成。
<code>"not_found"</code>	拒绝	请求的对象、目标或记录不存在。
<code>"not_local"</code>	拒绝	请求的端点属于另一个前哨站。
<code>"same_endpoint"</code>	拒绝	请求的物品源和目标是同一个端点。
<code>"unsupported_target"</code>	拒绝	所选组件没有公开兼容的物品目标。
<code>"target_is_vehicle"</code>	拒绝	请求的目标是无人机。无人机货物通过其 Drone Depot 移动。

`.disconnect()`

清除当前目标连接。不会移动或丢弃缓冲的输出。

返回值 (RETURNS)	结果字段 (RESULT FIELDS)	成功负载 (SUCCESS PAYLOAD)
ActionResult	.status、.message	None

结果 (OUTCOMES)

状态 (Status)	类型 (Kind)	含义 (Meaning)
"ok"	成功	操作成功完成。

`.connected_to()`

显示当前连接目标的名称；没有目标时为空字符串。

返回值 (RETURNS)

string

`.connected_id()`

显示当前连接目标的稳定 ID；没有目标时为空字符串。`connect()` 接受 ID 或显示名称，因此当你需要确认传入的身份是否匹配时，应与此值比较：`connected_to()` 返回的是可重命名的名称。

返回值 (RETURNS)

string

`.send(item_id, count, properties=None, property_match=None)`

向已连接目标发送最多 `count` 个 `item_id` 的整数量单位。属性字典默认选择包含该子集的堆叠。第四个参数传入 `"exact"` 可选择完整属性标识；`None`, `"exact"` 只选择无属性物品。`"any"` 会忽略属性。源物品的精确属性会被保留。转移会自动等待与移动单位数成比例的时间，并且需要完成 Auto Feeders 研究。

参数 (PARAMETERS)

名称	类型	描述
<code>item_id</code>	<code>string</code>	要发送的物品 ID
<code>count</code>	<code>number</code>	要发送的最大整数量单位
<code>properties</code>	<code>any</code>	可选属性字典，默认按子集匹配。省略或设为 <code>None</code> 时匹配任意属性；与 <code>property_match="exact"</code> 一起使用 <code>None</code> 时，仅选择无属性物品。
<code>property_match</code>	<code>string</code>	可选的选择模式： <code>any</code> 、 <code>subset</code> 或 <code>exact</code>
返回值 (RETURNS)	结果字段 (RESULT FIELDS)	成功负载 (SUCCESS PAYLOAD)
<code>TransferResult</code>	<code>.status</code> 、 <code>.message</code>	<code>.requested</code> 、 <code>.moved</code>

结果 (OUTCOMES)

状态 (Status)	类型 (Kind)	含义 (Meaning)
<code>"ok"</code>	成功	已移动全部 <code>.moved</code> 个请求单位。
<code>"partial"</code>	部分成功	已移动 <code>.requested</code> 中的 <code>.moved</code> 个单位；源可用量或目标容量限制了转移。
<code>"no_op"</code>	成功	未请求任何单位，因此状态没有改变。
<code>"research_required"</code>	拒绝	尚未解锁所需的材料转移研究。
<code>"busy"</code>	暂时性	此材料端点正在处理其他材料操作。
<code>"invalid_properties"</code>	拒绝	物品属性选择器的形状或值无效。
<code>"invalid_property_match"</code>	拒绝	请求的属性匹配模式无效。
<code>"no_connection"</code>	拒绝	端口没有配置转移端点。
<code>"inventory_not_local"</code>	拒绝	此端点从当前前哨站无法访问主基地货运 Inventory 。
<code>"same_endpoint"</code>	拒绝	配置的源和目标是同一个物品端点。
<code>"source_wrong_material"</code>	拒绝	源中包含的材料与请求材料不同。

状态（Status）		类型（Kind）	含义（Meaning）
"source_empty"		拒绝	源中没有可用的匹配单位。
"source_changed"		暂时性	源在转移规划与提交之间发生了变化。
"target_missing"		拒绝	配置的目标已不存在。
"target_under_construction"		暂时性	配置的目标仍在建造中。
"target_is_vehicle"	拒绝	移动目标是无人机，或是未停靠在该端点服务区域内的地面车辆。	
"target_not_local"		拒绝	配置的目标属于另一个前哨站。
"not_at_target"		拒绝	车辆位于目标服务区域之外。
"unsupported_target"		拒绝	配置的目标无法接收兼容货物。
"target_wrong_material"		拒绝	目标已锁定其他材料，或只接受其他材料。
"wrong_biome"		拒绝	目标因生物群系不兼容而拒绝该生物材料。
"target_unconfigured"		拒绝	目标缺少接受该材料所需的世界状态配置。
"mixed_materials"		拒绝	一次转移只能包含一个物品 ID。
"hot_cargo_requires_cask"		拒绝	该高温货物必须通过兼容的 Lead Cask 转移。
"cask_accepts_hot_only"		拒绝	Lead Cask 只接受受支持的高温货物。
"order_item_not_required"		拒绝	活动订单不需要该物品。
"order_slots_full"		拒绝	每个 Supply Dock 材料槽都已分配给其他所需物品。
"order_fulfilled"		拒绝	活动订单已经装载或发运该物品所需的全部单位。
"slots_full"		拒绝	目标仍有容量，但没有用于该材料标识的槽位。
"target_full"		拒绝	目标没有可容纳匹配单位的空间。
"target_changed"		暂时性	目标在转移规划与提交之间发生了变化。
"same_vehicle"		拒绝	车辆不能将货物转移给自身。
"source_moving"		暂时性	源车辆必须停止，才能交接货物。

状态 (Status)	类型 (Kind)	含义 (Meaning)
"target_moving"	暂时性	目标车辆必须停止，才能交接货物。
"out_of_range"	拒绝	车辆位于货物交接范围之外。

`.count()`

此端口缓冲区当前的总单位数。

返回值 (RETURNS)

`number`

`.capacity()`

此端口缓冲区可容纳的最大单位数。

返回值 (RETURNS)

`number`

`.stacks()`

当前缓冲的、按属性区分的 `ItemStack` 值快照列表。发送前可使用 `.properties` 检查并选择变体。

返回值 (RETURNS)

`List<ItemStack>`

类型 / 存储与库存 (TYPES / STORAGE & INVENTORY)

PickupOutputSlot

返回来源：田野 Water Pump 和 Mining Drill 上的 `self.output`

方法 (METHODS)

`.count()`

等待运输载具拾取的物品总单位数。

返回值 (RETURNS)

number

`.capacity()`

此拾取储料堆可容纳的最大物品单位数。

返回值 (RETURNS)

number

`.stacks()`

等待运输载具拾取的、按属性区分的 `ItemStack` 值快照列表。

返回值 (RETURNS)

`List<ItemStack>`

类型 / 存储与库存 (TYPES / STORAGE & INVENTORY)

Recipe

返回来源: Smelter、Fabricator、Feed Maker、Refiner 和 Fuel Assembler 上的 `list_recipes()` / `find_recipe()`

方法 (METHODS)

`.tier`

派生的生产等级。Smelter 基础设施为 Tier 0；其他配方的等级都比其最深层的制造输入高一级。

返回值 (RETURNS)

`number`

`.id`

配方 ID (例如 "smelt_iron_ingot")。在存档之间保持稳定；可用于比较配方, 或传回 `set_recipe(...)`。

返回值 (RETURNS)

`string`

`.name`

经过预翻译的配方显示名称。

返回值 (RETURNS)

`string`

.inputs

每次运行消耗材料的字典 {item_id: count}。可通过 .keys()、.values()、.items() 遍历，也可以索引访问：recipe.inputs["iron_ore"]。

返回值 (RETURNS)

dict<number>

.output_item

每次运行产生的产品 ID：离散配方使用物品 ID，流体输出配方（例如 Refiner）使用流体 ID。

返回值 (RETURNS)

string

.output_count

每次运行产生的 output_item 数量：物品以单位数计，流体以吨数计。

返回值 (RETURNS)

number

.duration_game_hours

在效率满载、输入充足、有电力且有输出空间时的基础循环时长，单位为小时。过度拥挤和输出受阻可能延长实际完成时间。

返回值 (RETURNS)

number

.power_draw

该配方运行时机器消耗的功率，单位为瓦特。

返回值 (RETURNS)

number

.fluid_inputs

每次运行完成时消耗的流体字典 {port_name: tons_per_run}（例如 "water_in"）。没有流体输入的配方返回空字典。

返回值 (RETURNS)

dict<number>

.input_fluid

通用输入端口所需的具体流体 ID；端口本身已经标识流体时为 None。Refiner 配方用此属性区分 gas_in / liquid_in 上的原料。

返回值 (RETURNS)	可能值 (POSSIBLE VALUES)
Optional[string]	"raw_sulfur_gas", "raw_cryofluid", "raw_chlorine", "raw_quicksilver"

.fluid_outputs

每次运行完成时沉积的流体字典 {port_name: tons_per_run}。物品输出配方返回空字典；Refiner 配方会在此列出 gas_out 或 liquid_out。

返回值 (RETURNS)

dict<number>

.output_fluid

通过通用输出端口产生的具体流体 ID；物品输出配方为 None。对于当前仅输出流体的 Refiner 配方，它与 output_item 相同。

返回值 (RETURNS)	可能值 (POSSIBLE VALUES)
Optional[string]	"sulfur_gas", "cryofluid", "chlorine", "quicksilver"

`.byproduct_item`

副产品的物品 ID；没有副产品的配方为 **None**。

返回值 (RETURNS)

`Optional[string]`

`.byproduct_count`

每次运行产生的 `byproduct_item` 单位数；没有副产品的配方为 **0**。

返回值 (RETURNS)

`number`

类型 / 存储与库存 (TYPES / STORAGE & INVENTORY)

SaleResult

返回来源：`shop.sell()` / `shop.sell_all()`

方法 (METHODS)

`.status`

取值为 "ok"、"not_sellable" 或 "no_stock"。

返回值 (RETURNS)	可能值 (POSSIBLE VALUES)
string	"ok"、"not_sellable"、"no_stock"

`.message`

面向玩家的出售结果说明。

返回值 (RETURNS)

string

`.item_id`

请求出售物品的稳定 ID。

返回值 (RETURNS)

string

`.units`

从 Inventory 中移除的整数量单位。

返回值 (RETURNS)

number

`.credits`

本次出售获得的整数量信用点。

返回值 (RETURNS)

`number`

类型 / 存储与库存 (TYPES / STORAGE & INVENTORY)

ShopItem

返回来源：`shop.get_catalogue()`

方法 (METHODS)

`.id`

物品 ID (用于购买)。

返回值 (RETURNS)

`string`

`.name`

物品显示名称。

返回值 (RETURNS)

`string`

`.cost`

以信用点计的价格。

返回值 (RETURNS)

`number`

类型 / 存储与库存 (TYPES / STORAGE & INVENTORY)

Slot

返回来源：`inventory.get_slots()`

方法 (METHODS)

`.slot`

槽位索引编号。

返回值 (RETURNS)

`number`

`.id`

物品 ID (槽位为空时空字符串)。

返回值 (RETURNS)

`string`

`.name`

物品显示名称。

返回值 (RETURNS)

`string`

`.value`

每件物品的信用点价值。

返回值 (RETURNS)

`number`

`.count`

此插槽中的堆叠数量。

返回值

`number`

`.properties`

此堆叠的精确属性字典；普通物品返回 `None`。

返回值

`Optional[dict]`

`.genes`

此持有的地热碎片携带的基因，例如 `["cold_tolerance", "pressure_tolerance"]`；无需加载 DNA Sequencer 即可直接从库存读取。拼接碎片恰好携带你设计的基因集合；原始碎片携带其天然基因。非地热物品返回空列表。

返回值

`list<string>`

`.glow`

持有的沿海碎片的实例级生物发光颜色 `[r,g,b]`（0-255）。其他物品返回 `None`。

返回值

`Optional[[number, number, number]]`

`.spliced`

如果这是已工程化（已经基因拼接）的地热碎片，则为 **True**：进一步用 DNA Sequencer 修改会摧毁它。原始物品则为 **False**。

返回值

boolean

类型 / 存储与库存

TransferResult

由 `InputSlot.take()`、`InputSlot.eject()`、`InputSlot.flush()`、`VehicleInputSlot.take()`、`OutputSlot.send()`、`Cargo.compact()`、存储箱传输方法和仓库压缩方法返回。

方法

`.status`

供脚本检查的稳定结果代码。`.message` 解释发生了什么。

返回值 | 可能的值

string | "ok"、"partial"、"no_op"、"already_compact"、"research_required"、"busy"、"invalid_properties"、"invalid_property_match"、"no_connection"、"inventory_not_local"、"source_missing"、"source_under_construction"、"source_is_vehicle"、"source_not_local"、"not_at_source"、"unsupported_source"、"source_wrong_material"、"source_empty"、"source_reserved"、"source_changed"、"buffer_full"、"slots_full"、"target_missing"、"target_under_construction"、"target_is_vehicle"、"target_not_local"、"not_at_target"、"unsupported_target"、"target_wrong_material"、"wrong_biome"、"needs_plating"、"cask_missing"、"target_unconfigured"、"mixed_materials"、"hot_cargo_requires_cask"、"cask_accepts_hot_only"、"order_item_not_required"、"order_slots_full"、"order_undefined"、"target_full"

可能的状态值还包括："target_changed"、"same_endpoint"、"same_storage"、"same_vehicle"、"source_moving"、"target_moving"、"out_of_range"。

`.message`

面向玩家、针对本次确切结果的上下文说明，适合用于日志和面向玩家的诊断。

返回值

string

`.requested`

调用请求的整数单位数。对于无参数的 `flush()` 或 `compact()`，这是操作观察到的缓冲数量或重新定位数量。

返回值

number

`.moved`

本次调用实际传输、重新定位或摧毁的整数单位数。对于被拒绝的结果始终为 0。

返回值

number

类型 / 存储与库存

VehicleInputSlot

由 Rover 和 Pioneer 上的 `self.input` 返回。

方法

`.connect(name)`

通过稳定 ID 或显示名称设置兼容的货物源。Rover 或 Pioneer 必须停放在静止源的服务区域内。Field Mining Drill 和 Water Pump 的库存堆支持载具提取。载具交接要求两辆载具都停止且彼此靠近。只有停在 Nocturna Base 时才提供 `"inventory"`；远程前哨站使用本地 Storage Bin、Warehouse 或机器缓冲区。

参数

名称 | 类型 | 描述

`name` | `string` | 兼容货物源的稳定 ID 或显示名称，或 `inventory`

返回值 | 结果字段 | 成功载荷

`ActionResult` | `.status`、`.message` | `None`

结果

状态 | 类型 | 含义

`"ok"` | 成功 | 操作成功完成。

`"not_found"` | 拒绝 | 请求的对象、目标或记录不存在。

`"same_endpoint"` | 拒绝 | 请求的物品源和目的地是同一个端点。

`"unsupported_source"` | 拒绝 | 所选组件未提供兼容的物品源。

`"source_is_vehicle"` | 拒绝 | 请求的源是无人机。无人机货物必须通过其 Drone Depot 移动。

`.disconnect()`

清除当前货物源连接。不会移动或丢弃货物。

返回值 | 结果字段 | 成功载荷

`ActionResult` | `.status`、`.message` | `None`

结果

状态 | 类型 | 含义

"ok" | 成功 | 操作成功完成。

`.connected_to()`

显示当前连接的货物源名称；未连接时返回空字符串。

返回值

`string`

`.connected_id()`

显示当前连接的货物源稳定 ID；未连接时返回空字符串。`.connect()` 接受 ID 或显示名称，因此当身份必须在重命名后仍保持不变时，应与此值比较。

返回值

`string`

`.take(item_id, count, properties=None, property_match=None)`

从已连接的源中将不超过整数 `count` 个 `item_id`

单位装入载具货舱。默认情况下，属性字典会选择包含该属性子集的堆叠。第四个参数传入 `"exact"`

可选择完整身份；传入 `None` 和 `"exact"` 时仅选择无属性物品。 `"any"`

会忽略属性。源物品的精确属性会被保留。传输会根据移动单位数自动等待相应时间，并要求解锁 Auto Feeders 研究。

参数

名称 | 类型 | 描述

`item_id` | `string` | 要装载的物品 ID

`count` | `number` | 要装载的最大整数单位数

`properties` | `any` | 可选属性字典，默认按子集匹配。省略或传入 `None` 匹配任意属性；与 `property_match="exact"` 一起使用 `None` 时，仅选择无属性物品。

`property_match` | `string` | 可选选择模式：`any`、`subset` 或 `exact`

返回值 | 结果字段 | 成功载荷

`TransferResult` | `.status`、`.message` | `.requested`、`.moved`

结果

状态 | 类型 | 含义

`"ok"` | 成功 | 已移动全部 `.moved` 个请求单位。

`"partial"` | 部分成功 | 已移动请求的 `.moved` / `.requested` 个单位；源的可用量或目的地容量限制了传输。

`"no_op"` | 成功 | 未请求任何单位，因此状态未改变。

`"research_required"` | 拒绝 | 所需的材料传输研究尚未解锁。

`"busy"` | 暂时性 | 此材料端点正在处理另一个材料操作。

`"invalid_properties"` | 拒绝 | 物品属性选择器的形状或值无效。

`"invalid_property_match"` | 拒绝 | 请求的属性匹配模式无效。

`"no_connection"` | 拒绝 | 端口没有配置传输端点。

`"inventory_not_local"` | 拒绝 | 此端点无法从当前前哨站访问主基地货运 Inventory。

`"same_endpoint"` | 拒绝 | 已配置的源和目的地是同一个物品端点。

`"source_missing"` | 拒绝 | 已配置的源不再存在。

`"source_under_construction"` | 暂时性 | 已配置的源仍在建造中。

状态 | 类型 | 含义

"source_is_vehicle" | 拒绝 | 移动源是无人机，或是未停靠在此端点服务区域内的地面载具。

"source_not_local" | 拒绝 | 已配置的源属于另一个前哨站。

"not_at_source" | 拒绝 | 载具位于源的服务区域之外。

"unsupported_source" | 拒绝 | 已配置的源未提供兼容货物。

"source_wrong_material" | 拒绝 | 源中包含的材料与请求的材料不同。

"source_empty" | 拒绝 | 源中没有可用的匹配单位。

"source_reserved" | 拒绝 | 源正在为活动操作预留其货物。

"source_changed" | 暂时性 | 源在传输规划和提交之间发生了变化。

"buffer_full" | 拒绝 | 接收端输入缓冲区已满。

"slots_full" | 拒绝 | 目的地有容量，但没有可容纳此材料身份的槽位。

"target_wrong_material" | 拒绝 | 目的地已锁定为或接受另一种材料。

"wrong_biome" | 拒绝 | 目的地因生物群系不兼容而拒绝此生物材料。

"target_unconfigured" | 拒绝 | 目的地缺少接受此材料所需的世界状态配置。

"hot_cargo_requires_cask" | 拒绝 | 此热货物必须通过兼容的 Lead Cask 移动。

"order_item_not_required" | 拒绝 | 当前订单不需要此物品。

"order_slots_full" | 拒绝 | 所有 Supply Dock 材料槽都已分配给其他必需物品。

"order_fulfilled" | 拒绝 | 当前订单所需的此物品单位已全部装载或发运。

"target_full" | 拒绝 | 目的地没有容纳匹配单位的容量。

"target_changed" | 暂时性 | 目的地在传输规划和提交之间发生了变化。

"same_vehicle" | 拒绝 | 载具不能向自身传输货物。

"source_moving" | 暂时性 | 源载具必须停止后才能交接货物。

"target_moving" | 暂时性 | 目的地载具必须停止后才能交接货物。

状态 | 类型 | 含义

"out_of_range" | 拒绝 | 载具位于货物交接范围之外。

`.count()`

载具货舱当前携带的总单位数。

返回值

number

`.capacity()`

载具货舱可容纳的最大单位数。

返回值

number

`.stacks()`

当前携带的、按属性区分的 `ItemStack` 值快照列表。属性不同的两个条目可能具有相同的 ID。

返回值

`List<ItemStack>`

类型 / 存储与库存

WarehouseSlot

由 `warehouse.slots()` 返回。

方法

`.index`

槽位索引（从 0 开始）。

返回值

number

`.item`

此槽位存放的物品 ID（槽位为空时为空字符串）。

返回值

string

`.count`

此槽位当前的单位数。

返回值

number

`.capacity`

此槽位的容量（单位数）。

返回值

number

`.properties`

此物理槽位中物品变体的精确属性字典；槽位为空或存放普通物品时为 **None**。

返回值

`Optional[dict]`

类型 / 机队与载具

CollectResult

由 `drone_small.collect()`、`drone_medium.collect()`、`drone_large.collect()` 返回。

方法

`.status`

稳定的收集结果："ok"、"moving"、"busy"、"nothing_here"、"no_cargo_space" 或 "scrambled"。

返回值 | 可能的值

string | "ok"、"moving"、"busy"、"nothing_here"、"no_cargo_space"、"scrambled"

`.message`

面向玩家、解释确切收集结果的说明。

返回值

string

`.item_id`

所收集材料的稳定 ID；未收集材料时为 `None`。

返回值

Optional[string]

`.collected`

本次调用提交至无人机货舱的整数单位数。

返回值

`number`

类型 / 机队与载具

ConstructorModule

由 Pioneer 的 `self.constructor` 返回。

方法

.execute(blueprint_id)

执行 Plan Mode 的建造或拆解蓝图。驾驶 Pioneer 进入 `blueprint.position` 的交互范围，然后传入来自 `get_component("construction_blueprint").pending_constructions()`、`.active_constructions()` 或 `.paused_constructions()` 的蓝图 ID。该操作会让出执行权。一个 Pioneer 一次只能执行一个现场操作。停止、断电、离开现场、救援或移除 Constructor Module 会暂停已付费的工作，但不会丢失进度或材料。所属 Pioneer 可以重新加入活动工作，包括在保存/加载之后；其他 Pioneer 无法夺取已归属的工作。

参数

名称 | 类型 | 描述

`blueprint_id` | `string` | 来自 `construction_blueprint` 队列的待处理、活动中或已暂停蓝图 ID。

返回值 | 结果字段 | 成功载荷

`ActionResult` | `.status`、`.message` | `None`

结果

状态 | 类型 | 含义

"ok" | 成功 | 操作成功完成。

"not_mounted" | 拒绝 | 必需模块未安装。

"not_found" | 拒绝 | 请求的对象、目标或记录不存在。

"locked" | 拒绝 | 必需的功能、配方或操作已锁定。

"already_active" | 暂时性 | 请求的操作已经处于活动状态。

"busy" | 暂时性 | 组件正在执行另一个操作。

"wrong_position" | 拒绝 | 工作人员未处于操作要求的位置。

"insufficient_materials" | 拒绝 | Pioneer 货舱中没有足够的所需建造材料。

"paused_no_power" | 暂时性 | 操作因没有电力而暂停。

"cargo_present" | 拒绝 | 现有货物阻止了请求的配置变更。

"blocked" | 拒绝 | 当前世界状态阻止了操作。

"paused" | 暂时性 | 操作已暂停。

"canceled" | 拒绝 | 操作已取消。

"not_ready" | 拒绝 | 操作的前置条件当前未满足。

类型 / 机队与载具

DrillModule

由载具的 `self.drill` 返回。

方法

`.mine()`

在静止状态下钻取一个单位。

返回值 | 结果字段 | 成功载荷

`ActionResult` | `.status`、`.message` | `None`

结果

状态 | 类型 | 含义

"ok" | 成功 | 操作成功完成。

"not_mounted" | 拒绝 | 必需模块未安装。

"not_at_site" | 拒绝 | 载具未处于兼容地点。

"not_surveyed" | 拒绝 | 矿物地点尚未勘测。

"too_hard" | 拒绝 | 目标超过已安装工具的硬度上限。

"no_cargo_space" | 拒绝 | 货舱没有可容纳结果的兼容空间。

"no_power" | 暂时性 | 组件没有可用电力。

"not_enough_power" | 拒绝 | 可用能量低于操作要求。

"busy" | 暂时性 | 组件正在执行另一个操作。

.hardness_limit()

此钻机可提取的最大矿物硬度（1 为 basic，3 为 Industrial，4 为 Heavy）。过时的模块引用会引发 **ReferenceError**。

返回值

number

引发异常

异常 | 条件

ReferenceError | 此 **DrillModule** 引用已过时，因为其模块不再安装。安装钻机后重新读取 **self.drill**。

.speed_multiplier()

钻取时间乘数（1.0 为 basic，0.75 为 Industrial，0.6 为 Heavy：数值越低速度越快）。过时的模块引用会引发 **ReferenceError**。

返回值

number

引发异常

异常 | 条件

ReferenceError | 此 **DrillModule** 引用已过时，因为其模块不再安装。安装钻机后重新读取 **self.drill**。

类型 / 机队与载具

DroneBattery

由电动无人机的 `self.battery` 返回。

方法

`.level()`

已安装 Battery Pack 的当前总电量（Wh）。无人机没有电动动力系统时会引发 `ReferenceError`。

返回值

`number`

引发异常

异常 | 条件

`ReferenceError` | 此无人机当前没有电动动力系统和已安装的 Battery Pack。

`.capacity()`

以 Wh 为单位的总电量容量。无人机没有电动动力系统时会引发 `ReferenceError`。

返回值

`number`

引发异常

异常 | 条件

`ReferenceError` | 此无人机当前没有电动动力系统和已安装的 Battery Pack。

`.percent()`

电量占比 0-1 (`level / capacity`)。无人机没有电动动力系统时会引发 `ReferenceError`。

返回值

`number`

引发异常

异常 | 条件

`ReferenceError` | 此无人机当前没有电动动力系统和已安装的 Battery Pack。

DroneCargo

返回值来源：`self.cargo`（无人机）

方法

`.count()`

所有货舱舱段加上生物提取器舱室中的单位总数。

返回值

number

`.capacity()`

所有容器的物理容量总和：每个已安装的货舱舱段（Small 100、Medium 250、Large 500），再加上生物提取器的 25 t 舱室。护盾装甲会使每个货舱舱段的容量减半。

返回值

number

`.contents()`

将 `item_id` 映射到单位数量的字典，包含货舱中当前存放的每种材料。可迭代调用 `.keys()` / `.items()`。

返回值

dict<number>

`.space_for(item_id)`

指定材料的可用空间。每个货舱舱段只能容纳一种材料，因此只有空舱段或已经存放 `item_id` 的舱段才会计入；生物提取器的 25t 舱室仅对生命体计入。如果没有空闲舱段或匹配的舱段，即使其他舱段仍有自身材料的空间，也会针对该材料返回 0。

参数

名称	类型	描述
<code>item_id</code>	<code>string</code>	要计算剩余空间的材料 ID

返回值

`number`

`.full()`

当 `count() >= capacity()` 时返回 True。如果所有舱段都被占用，无人机仍可能无法装入新的材料类型，即使 `full()` 为 False；对于特定材料，请使用 `space_for(item_id)`。

返回值

`boolean`

`.load(item_id, count, properties=None, property_match=None)`

将最多 `count` 个整数量单位装入该无人机的货舱，并保留精确属性。无人机完成前往停靠的无人机仓库的路线后，可以从仓库库存或现场采矿钻机装载。高温货物从本地铅罐装载，并且需要护盾装甲。

参数

名称	类型	描述
<code>item_id</code>	<code>string</code>	要装载的材料 ID
<code>count</code>	<code>number</code>	要装载的整数量单位上限
<code>properties</code>	<code>any</code>	可选属性字典，默认按子集匹配。省略或设为 <code>None</code> 时匹配任意属性；将 <code>None</code> 与 <code>property_match="exact"</code> 搭配，可仅选择无属性的物品。
<code>property_match</code>	<code>string</code>	可选选择模式： <code>any</code> 、 <code>subset</code> 或 <code>exact</code>

返回值

类型	结果字段	成功载荷
<code>TransferResult</code>	<code>.status</code> 、 <code>.message</code>	<code>.requested</code> 、 <code>.moved</code>

结果

状态	类别	含义
<code>"ok"</code>	成功	已移动全部请求数量，即 <code>.moved</code> 等于请求的单位数。
<code>"partial"</code>	部分成功	已移动请求数量中的 <code>.moved</code> 个；来源可用量或目标容量限制了传输。
<code>"no_op"</code>	成功	未请求任何单位，因此状态没有改变。
<code>"invalid_properties"</code>	拒绝	材料属性选择器的形状或值无效。
<code>"invalid_property_match"</code>	拒绝	请求的属性匹配模式无效。
<code>"research_required"</code>	拒绝	所需的材料传输研究尚未解锁。
<code>"target_moving"</code>	临时状态	目标车辆必须停下，才能交接货物。
<code>"not_at_source"</code>	拒绝	车辆位于来源的服务范围之外。
<code>"source_empty"</code>	拒绝	来源没有可用的匹配单位。
<code>"target_full"</code>	拒绝	目标没有可容纳匹配单位的空间。
<code>"needs_plating"</code>	拒绝	此高温货物要求无人机安装护盾装甲。
<code>"cask_missing"</code>	拒绝	该前哨站没有可用的兼容铅罐。
<code>"source_changed"</code>	临时状态	来源在传输规划与提交之间发生了变化。

状态	类别	含义
"target_changed"	临时状态	目标在传输规划与提交之间发生了变化。

`.unload(item_id, count, properties=None, property_match=None)`

将最多 `count` 个整数量单位从该无人机的货舱卸入其停靠的无人机仓库，并保留精确属性。高温货物会卸入兼容的铅罐。

参数

名称	类型	描述
<code>item_id</code>	<code>string</code>	要卸载的材料 ID
<code>count</code>	<code>number</code>	要卸载的整数量单位上限
<code>properties</code>	<code>any</code>	可选属性字典，默认按子集匹配。省略或设为 <code>None</code> 时匹配任意属性；将 <code>None</code> 与 <code>property_match="exact"</code> 搭配，可仅选择无属性的物品。
<code>property_match</code>	<code>string</code>	可选选择模式： <code>any</code> 、 <code>subset</code> 或 <code>exact</code>

返回值

类型	结果字段	成功载荷
<code>TransferResult</code>	<code>.status</code> 、 <code>.message</code>	<code>.requested</code> 、 <code>.moved</code>

结果

状态	类别	含义
<code>"ok"</code>	成功	已移动全部请求数量，即 <code>.moved</code> 等于请求的单位数。
<code>"partial"</code>	部分成功	已移动请求数量中的 <code>.moved</code> 个；来源可用量或目标容量限制了传输。
<code>"no_op"</code>	成功	未请求任何单位，因此状态没有改变。
<code>"invalid_properties"</code>	拒绝	材料属性选择器的形状或值无效。
<code>"invalid_property_match"</code>	拒绝	请求的属性匹配模式无效。
<code>"research_required"</code>	拒绝	所需的材料传输研究尚未解锁。
<code>"not_at_target"</code>	拒绝	车辆位于目标的服务范围之外。
<code>"source_empty"</code>	拒绝	来源没有可用的匹配单位。
<code>"slots_full"</code>	拒绝	目标仍有容量，但没有可用于该材料标识的舱位。
<code>"target_full"</code>	拒绝	目标没有可容纳匹配单位的空间。
<code>"cask_missing"</code>	拒绝	该前哨站没有可用的兼容铅罐。
<code>"source_changed"</code>	临时状态	来源在传输规划与提交之间发生了变化。
<code>"target_changed"</code>	临时状态	目标在传输规划与提交之间发生了变化。

`.discard(item_id, count, properties=None, property_match=None)`

从该无人机的货舱中永久销毁最多 `count` 个整数数量单位。此抛弃操作不需要空间站；排空后的舱段会解锁，以便装入新材料。

参数

名称	类型	描述
<code>item_id</code>	<code>string</code>	要销毁的材料 ID
<code>count</code>	<code>number</code>	要销毁的整数数量单位上限
<code>properties</code>	<code>any</code>	可选属性字典，默认按子集匹配。省略或设为 <code>None</code> 时匹配任意属性；将 <code>None</code> 与 <code>property_match="exact"</code> 搭配，可仅选择无属性的物品。
<code>property_match</code>	<code>string</code>	可选选择模式： <code>any</code> 、 <code>subset</code> 或 <code>exact</code>

返回值

类型	结果字段	成功载荷
<code>DiscardResult</code>	<code>.status</code> 、 <code>.message</code>	<code>.requested</code> 、 <code>.discarded</code>

结果

状态	类别	含义
<code>"ok"</code>	成功	永久抛弃 <code>.discarded</code> 个单位。
<code>"partial"</code>	部分成功	在请求的单位中永久抛弃 <code>.discarded</code> 个。
<code>"empty"</code>	成功	选定的货舱区域已经为空。
<code>"no_op"</code>	成功	未请求任何单位，因此没有单位被抛弃。
<code>"invalid_properties"</code>	拒绝	材料属性选择器的形状或值无效。
<code>"invalid_property_match"</code>	拒绝	请求的属性匹配模式无效。
<code>"source_changed"</code>	临时状态	传输规划与提交之间货物发生了变化。

类型 / 机队与车辆 (TYPES / FLEET & VEHICLES)

DroneOilTank

返回值来源：`self.oil_tank` (直升机无人机)

方法

`.level()`

已安装油箱中的当前油量，单位为吨。如果该无人机没有直升机动力系统，则抛出 `ReferenceError`。

返回值

`number`

抛出异常

异常	条件
<code>ReferenceError</code>	该无人机当前没有直升机动力系统。

`.capacity()`

油箱的总容量，单位为吨。如果该无人机没有直升机动力系统，则抛出 `ReferenceError`。

返回值

`number`

抛出异常

异常	条件
<code>ReferenceError</code>	该无人机当前没有直升机动力系统。

.percent()

油量占比， 范围为 0-1。如果该无人机没有直升机动力系统， 则抛出 **ReferenceError**。

返回值

number

抛出异常

异常	条件
ReferenceError	该无人机当前没有直升机动力系统。

类型 / 机队与车辆 (TYPES / FLEET & VEHICLES)

Fleet

返回值来源：`get_component("fleet")`

相关对象类型

- `DroneRef`
- `MobileUnitRef`
- `VehicleRef`

方法

`.vehicles()`

所有已拥有的地面车辆的只读 `VehicleRef` 快照。向空间站 API 传递车辆时使用 `.id`；再次调用 `vehicles()` 可获取最新的引用字段，或调用 `get_component(ref.id)` 获取实时车辆 API。

返回值

```
List<VehicleRef>
```

`.drones()`

所有已拥有的无人机的只读 `DroneRef` 快照。向空间站/救援 API 传递无人机时使用 `.id`；再次调用 `drones()` 可获取最新的引用字段，或调用 `get_component(ref.id)` 获取实时无人机 API。

返回值

```
List<DroneRef>
```

`.mobile_units()`

在一个快照列表中返回所有已拥有的车辆和无人机。使用 `.category` 在 "vehicle" 与 "drone" 之间分支；重新查询以获取最新位置/状态。

返回值

```
List<MobileUnitRef>
```

类型 / 机队与车辆 (TYPES / FLEET & VEHICLES)

DroneRef

返回值来源：`fleet.drones()`

方法

`.category`

始终为 "drone"。

返回值 | 可能的值

`string` | "drone"

`.id`

稳定的组件 ID。与 `get_component(id)` 搭配使用，或将其传递给无人机服务空间站和仓库 API。

返回值

`string`

`.name`

返回该引用时的显示名称。

返回值

`string`

`.kind`

无人机机体 ID。

返回值 | 可能的值

`string` | "drone_small"、"drone_medium"、"drone_large"

`.engine`

当未安装推进器时为 "electric"、"heli" 或空字符串。

返回值 | 可能的值

string | "", "electric", "heli"

`.status`

返回该引用时的活动/状态字符串。

返回值 | 可能的值

string | "idle", "traveling", "charging", "refueling", "waiting_service", "waiting_oil", "waiting_bay", "being_rescued", "holding_weather", "scrambled", "stalled_no_battery", "stalled_no_oil", "stalled_no_route"

`.x`

返回该引用时，相对于基地的 X 坐标，单位为米。

返回值

number

`.y`

返回该引用时，相对于基地的 Y 坐标，单位为米。

返回值

number

`.position()`

返回该引用生成时的位置快照。

返回值

`Position`

`.current_station`

返回该引用时所在的空间站 ID；如果不在空间站则为空字符串。

返回值

`string`

`.is_docked`

如果返回该引用时无人机停放在无人机空间站，则为 `True`。

返回值

`boolean`

`.battery_level`

返回该引用时电动无人机的充电比例（0-1）；直升机无人机返回 `None`。

返回值

`Optional[number]`

`.battery_wh`

返回该引用时电动无人机的电量，单位为 Wh；直升机无人机返回 `None`。

返回值

`Optional[number]`

`.battery_capacity`

电动无人机的电量容量，单位为 Wh；直升机无人机返回 **None**。

返回值

`Optional[number]`

`.oil_level`

返回该引用时直升机无人机的油量比例（0-1）；电动无人机返回 **None**。

返回值

`Optional[number]`

`.oil_tons`

返回该引用时直升机无人机的油量，单位为吨；电动无人机返回 **None**。

返回值

`Optional[number]`

`.oil_capacity`

直升机无人机的油量容量，单位为吨；电动无人机返回 **None**。

返回值

`Optional[number]`

`.is_being_rescued`

如果返回该引用时无人机正处于无人机服务空间站的救援任务中，则为 **True**。

返回值

`boolean`

`.rescue_status`

返回该引用时的救援状态："none"、"outbound"、"charging"、"carrying" 或 "returning"。

返回值 | 可能的值

string | "none"、"outbound"、"charging"、"carrying"、"returning"

类型 / 机队与车辆 (TYPES / FLEET & VEHICLES)

MobileUnitRef

返回值来源：`fleet.mobile_units()`

方法

`.category`

"vehicle" 或 "drone"。

返回值 | 可能的值

`string` | "vehicle"、"drone"

`.id`

稳定的组件 ID。与 `get_component(id)` 或空间站 API 搭配使用。

返回值

`string`

`.name`

返回该引用时的显示名称。

返回值

`string`

`.kind`

单位类型，例如 "rover"、"pioneer" 或无人机机体 ID。

返回值 | 可能的值

`string` | "rover"、"pioneer"、"drone_small"、"drone_medium"、"drone_large"

.status

返回该引用时的活动/状态字符串。

返回值 | 可能的值

string | "idle"、"moving"、"stranded"、"scanning"、"surveying"、"drilling"、"discarding"、"constructing"、"transferring"、"charging"、"queued"、"being_rescued"、"traveling"、"refueling"、"waiting_service"、"waiting_oil"、"waiting_bay"、"holding_weather"、"scrambled"、"stalled_no_battery"、"stalled_no_oil"、"stalled_no_route"

.x

返回该引用时，相对于基地的 X 坐标，单位为米。

返回值

number

.y

返回该引用时，相对于基地的 Y 坐标，单位为米。

返回值

number

`.position()`

返回该引用生成时的位置快照。

返回值

`Position`

`.is_docked`

如果返回该引用时单位停放在服务点/空间站，则为 `True`。

返回值

`boolean`

`.is_being_rescued`

如果返回该引用时该单位正被所属的救援/回收任务处理，则为 `True`。

返回值

`boolean`

`.rescue_status`

返回该引用时的救援状态：`"none"`、`"outbound"`、`"charging"`、`"carrying"` 或 `"returning"`。

返回值 | 可能的值

`string` | `"none"`、`"outbound"`、`"charging"`、`"carrying"`、`"returning"`

类型 / 机队与车辆 (TYPES / FLEET & VEHICLES)

VehicleRef

返回值来源：`fleet.vehicles()`

方法

`.category`

始终为 "vehicle"。

返回值 | 可能的值

`string` | "vehicle"

`.id`

稳定的组件 ID。与 `get_component(id)` 或 `charging_station.dispatch_rescue(id)` 搭配使用。

返回值

`string`

`.name`

返回该引用时的显示名称。

返回值

`string`

`.kind`

"rover" 或 "pioneer"。

返回值 | 可能的值

`string` | "rover"、"pioneer"

`.status`

返回该引用时车辆当前的活动状态。

返回值 | 可能的值

string | "idle"、"moving"、"stranded"、"scanning"、"surveying"、"drilling"、"discarding"、"constructing"、"transferring"、"charging"、"queued"、"being_rescued"

`.x`

返回该引用时，相对于基地的 X 坐标，单位为米。

返回值

number

`.y`

返回该引用时，相对于基地的 Y 坐标，单位为米。

返回值

number

`.position()`

返回该引用生成时的位置快照。

返回值

Position

`.battery_level`

返回该引用时的电量比例（0-1）。

返回值

number

`.battery_wh`

返回该引用时的电池电量，单位为 Wh。

返回值

number

`.battery_capacity`

最大电池容量，单位为 Wh。

返回值

number

`.is_docked`

如果返回该引用时车辆停在前哨站的 2×2 方格占地范围内（包括约 2 m 的服务边界），则为 True。处于移动中或仅仅停止但仍有活动路线的车辆不算已停靠。

返回值

boolean

.is_being_rescued

如果该车辆当时正在进行车辆充电站救援，则为 **True**。

返回值

boolean

.rescue_status

返回此引用时的救援状态："none"、"outbound"、"charging" 或 "returning"。

返回值

类型	可能值
string	"none"、"outbound"、"charging"、"returning"

TYPES / FLEET & VEHICLES（类型 / 车队与车辆）

MountSlot

由 rover / pioneer 上的 `self.modules()` 返回。

方法

`.index`

底盘上的插槽位置，从 0 开始计数。

返回值

number

`.type`

插槽类型，决定哪些模块可以装入（nav、sonar_basic、drill_basic、universal、thruster 或 drone_module）。

返回值

类型	可能值
string	"nav"、"sonar_basic"、"drill_basic"、"universal"、"thruster"、"drone_module"

`.module_id`

当前安装在此插槽中的模块 ID；如果为空则为 **None**。

返回值

Optional[string]

`.internal_count`

当前安装模块暴露的内部插槽数量。对于非容器模块为 0。

返回值

`number`

`.internal_items`

当前安装在该模块内部插槽中的物品 ID 列表。**None** 项表示对应内部插槽为空。

返回值

`list<Optional[string]>`

TYPES / FLEET & VEHICLES（类型 / 车队与车辆）

NavModule

由车辆的 `self.nav` 返回。

方法

`.set_target(x, y)`

设置要驶向的目标坐标。

参数

名称	类型	描述
x	number	目标 X 坐标
y	number	目标 Y 坐标

返回值

返回类型	结果字段	成功载荷
ActionResult	.status、.message	None

结果

状态	类别	含义
"ok"	success（成功）	操作已成功完成。
"not_mounted"	rejection（拒绝）	所需模块未安装。
"invalid"	rejection（拒绝）	一个或多个提供的参数超出接受范围。
"out_of_bounds"	rejection（拒绝）	请求的位置超出有效世界边界。
"busy"	transient（暂态）	该组件正在执行其他操作。

`.set_throttle(power)`

设置节流值（`0.0-1.0`；会被限制在该范围内）。

参数

名称	类型	描述
<code>power</code>	<code>number</code>	节流功率（ <code>0.0-1.0</code> ）

返回值

返回类型	结果字段	成功载荷
<code>ActionResult</code>	<code>.status</code> 、 <code>.message</code>	<code>None</code>

结果

状态	类别	含义
<code>"ok"</code>	<code>success</code> （成功）	操作已成功完成。
<code>"not_mounted"</code>	<code>rejection</code> （拒绝）	所需模块未安装。
<code>"busy"</code>	<code>transient</code> （暂态）	该组件正在执行其他操作。

`.throttle()`

当前节流设定值（`0.0-1.0`）。返回脚本最近通过 `set_throttle(...)` 写入的值。它不同于 `get_speed():throttle()` 表示意图（不会逐 tick 改变），而 `get_speed()` 表示车辆上一 tick 实际移动的速度。

返回值

`number`

`.brake()`

停止车辆（将节流值设为 0）。

返回值

返回类型	结果字段	成功载荷
<code>ActionResult</code>	<code>.status</code> 、 <code>.message</code>	<code>None</code>

结果

状态	类别	含义
<code>"ok"</code>	<code>success</code> （成功）	操作已成功完成。
<code>"not_mounted"</code>	<code>rejection</code> （拒绝）	所需模块未安装。

`.get_position()`

车辆位置，返回带有 `.x` 和 `.y` 的 `Position` 对象。

返回值

`Position`

`.get_speed()`

车辆速度，单位为 m/h。

返回值

`number`

`.get_distance_to(x, y)`

车辆到给定点的距离（米）。到达循环需要容差，通常应使用 `> 2`，而不是要求精确等于零。当下一步操作的目标是建筑物时，应驶向其 `BuildingRef.position`。

参数

名称	类型	描述
<code>x</code>	<code>number</code>	目标 X 坐标
<code>y</code>	<code>number</code>	目标 Y 坐标

返回值

`number`

`.speed_multiplier()`

最高速度倍率：基础值为 `1.0`；Pioneer 上每安装一个 Sport Nav，倍率增加 `1`。该倍率只影响速度；续航仍取决于电池、节流值、货物负载和移动耗能。

返回值

`number`

TYPES / FLEET & VEHICLES（类型 / 车队与车辆）

SonarModule

由车辆的 `self.sonar` 返回。

方法

.scan()

对附近地点进行点状扫描。一次完成的扫描可能找不到任何兼容接触点。矿物接触点受声纳范围和硬度限制；热能、水、石油和奇异接触点还要求匹配的研究。扫描会用新分类的地点更新 Journal（日志）。已失效的捕获模块引用会引发 `ReferenceError`。

返回值

返回类型	结果字段	成功载荷
<code>SonarScanResult</code>	<code>.status</code> 、 <code>.message</code>	<code>.sites</code>

结果

状态		类别	含义
"ok"	success（成功）		声纳扫描已完成。 <code>result.sites</code> 包含找到的所有兼容接触点；空列表表示范围内没有兼容接触点。
"too_hard"	partial（部分完成）	已完成的扫描探测到附近一个硬度超过所安装声纳上限、尚未解析的接触点。	
"tier_too_low"	partial（部分完成）	已完成的扫描探测到附近一个所安装声纳层级无法分类、尚未解析的接触点。	
"research_required"	partial（部分完成）	已完成的扫描探测到附近一个所需分类研究尚未解锁、尚未解析的接触点。	
"wrong_scanner"	partial（部分完成）	已完成的扫描探测到一个车辆声纳无法分类的附近接触点。	
"busy"	transient（暂态）		车辆当前正被另一个现场操作占用。
"no_power"	rejection（拒绝）		车辆电池没有足够能量执行声纳扫描。

引发异常

异常	条件
<code>ReferenceError</code>	此 <code>SonarModule</code> 引用已失效，因为其模块不再安装。安装声纳后重新读取 <code>self.sonar</code> 。

.survey(site)

显示一个可产出地点的可用详细信息。惰性的 `GeologicalAnomaly` 已经解析，因此勘测它是免费的。除非更高的声纳层级可以揭示更多内容，否则重复勘测同样免费。格式错误的地点值会引发 `ValueError`；已失效的捕获模块引用会引发 `ReferenceError`。

参数

名称	类型	描述
site	any	地点 ID 字符串，或来自 <code>scan()</code> 的 <code>Site</code>

返回值

返回类型	结果字段	成功载荷
<code>SurveyResult</code>	<code>.status</code> 、 <code>.message</code>	<code>.site</code>

结果

状态	类别	含义
"ok"	success（成功）	已成功勘测 <code>site_id</code> 。
"busy"	transient（暂态）	车辆当前正被另一个现场操作占用。
"not_discovered"	rejection（拒绝）	地点 <code>site_id</code> 不在该星球的已发现地点日志中。
"research_required"	rejection（拒绝）	地点 <code>site_id</code> 所需研究尚未解锁。
"tier_too_low"	rejection（拒绝）	所安装声纳层级无法勘测地点 <code>site_id</code> 。
"out_of_range"	rejection（拒绝）	地点 <code>site_id</code> 超出当前所安装声纳的范围。
"too_hard"	rejection（拒绝）	地点 <code>site_id</code> 超出所安装声纳的矿物硬度上限。
"no_power"	rejection（拒绝）	车辆电池没有足够能量勘测地点 <code>site_id</code> 。

引发异常

异常	条件
<code>ValueError</code>	<code>survey()</code> 要求非空的地点 ID，或一个 <code>Site</code> 对象。
<code>ReferenceError</code>	此 <code>SonarModule</code> 引用已失效，因为其模块不再安装。安装声纳后重新读取 <code>self.sonar</code> 。

`.range()`

声纳范围，单位为米（基础型 50、Wide 180、Deep 280）。已失效的捕获模块引用会引发 `ReferenceError`。

返回值

number

引发异常

异常	条件
ReferenceError	此 SonarModule 引用已失效，因为其模块不再安装。安装声纳后重新读取 <code>self.sonar</code> 。

`.hardness_limit()`

此声纳可识别的最大矿物硬度（基础型 1、Wide 3、Deep 4）。已失效的捕获模块引用会引发 `ReferenceError`。

返回值

number

引发异常

异常	条件
ReferenceError	此 SonarModule 引用已失效，因为其模块不再安装。安装声纳后重新读取 <code>self.sonar</code> 。

`.tier()`

所安装声纳提供的勘测深度层级：`"basic"` / `"wide"` / `"deep"`。它决定热能/奇异勘测的详细程度；解锁 Petroleum Survey（石油勘测）后，发现油井还需要 `"deep"`。已失效的捕获模块引用会引发 `ReferenceError`。

返回值

类型	可能值
string	<code>"basic"</code> 、 <code>"wide"</code> 、 <code>"deep"</code>

引发异常

异常	条件
ReferenceError	此 SonarModule 引用已失效，因为其模块不再安装。安装声纳后重新读取 <code>self.sonar</code> 。

TYPES / FLEET & VEHICLES (类型 / 车队与车辆)

SonarScanResult

由 `SonarModule.scan()` 返回。

方法

`.status`

稳定的扫描结果。`.sites` 列表为空且状态为 "ok"，表示扫描已完成，但没有需要玩家处理的接触点。

返回值

类型	可能值
string	"ok"、"too_hard"、"tier_too_low"、"research_required"、"wrong_scanner"、"busy"、"no_power"

`.message`

面向玩家的扫描结果说明。

返回值

string

`.sites`

本次已完成扫描解析出的地点；没有返回兼容地点时为空。

返回值

list<Site>

TYPES / FLEET & VEHICLES（类型 / 车队与车辆）

SurveyResult

由 SonarModule.survey() 返回。

方法

.status

状态可能为 "ok"、"busy"、"not_discovered"、"research_required"、"tier_too_low"、"out_of_range"、"too_hard" 或 "no_power"。

返回值

类型	可能值
string	"ok"、"busy"、"not_discovered"、"research_required"、"tier_too_low"、"out_of_range"、"too_hard"、"no_power"

.message

面向玩家的、关于勘测结果的准确说明。

返回值

string

.site

已勘测的具体 Site；如果勘测被拒绝则为 None。

返回值

Optional[Site]

TYPES / INFRASTRUCTURE & FLUIDS （类型 / 基础设施与流体）

Construction

由 `.pending_constructions()` / `.active_constructions()` / `.paused_constructions()` 返回。

方法

`.id`

唯一的蓝图 ID。将其传递给 `self.constructor.execute(id)`。

返回值

string

`.kind`

正在建造或拆除的内容。取值之一：`"pipe"` / `"power_line"` / `"gas_bridge"` / `"liquid_bridge"` / `"power_bridge"` / `"deconstruct"` / `"outpost"` / `"thermal_cap"` / `"water_pump"` / `"oil_pump"` / `"exotic_gas_cap"` / `"exotic_spring_tap"` / `"mining_drill"` / `"mining_drill_industrial"` / `"mining_drill_heavy"`。

返回值

类型	可能值
string	"pipe"、"power_line"、"gas_bridge"、"liquid_bridge"、"power_bridge"、"deconstruct"、"outpost"、"thermal_cap"、"water_pump"、"oil_pump"、"exotic_gas_cap"、"exotic_spring_tap"、"mining_drill"、"mining_drill_industrial"、"mining_drill_heavy"

.medium

物理公用设施层："gas"、"liquid" 或 "power"；点结构为 None。管道作业的 .kind == "pipe"，因此使用此字段区分 Gas Pipe（燃气管道）与 Liquid Pipe（液体管道）。拆除作业会报告目标的公用设施层。

返回值

类型	可能值
Optional[string]	"gas"、"liquid"、"power"

.position

以 Position 快照表示的、与瓦片对齐的世界坐标（.x、.y）。Pioneer 驶到这里即可开始/恢复建造。

返回值

Position

.progress

返回此 Construction 快照时的建造完成度，范围为 0-1。对于暂停的工作尤其有用：按完成度排序，优先恢复完成度最高的工作；要获取最新完成度，请重新查询建造列表。

返回值

number

.required_item

Pioneer 执行此作业前必须携带的物品 ID；对于拆除作业，以及已经开始的暂停作业，则为 None。

返回值

Optional[string]

`.required_count`

接受此作业所需的 `.required_item` 数量。大多数单件作业需要 1；拆除作业需要 0。

返回值

`number`

TYPES / INFRASTRUCTURE & FLUIDS（类型 / 基础设施与流体）

FluidPort

由流动网络机器上的任意 <fluid>_in / <fluid>_out 属性返回。

相关对象类型

FluidConnection

方法

.connect(target)

使用稳定机器 ID 或显示名称，记录或替换此端口唯一声明的目标。目标必须暴露兼容的反向端口。提供方或使用方都可以声明此关系；只需一方声明即可。局部机器会直接传输，而远程意图会等待一个已完成、无冲突且属于同一介质的组件连接到两个锚点。

参数

名称	类型	描述
target	string	提供方或使用方的稳定 ID 或显示名称

返回值

返回类型	结果字段	成功载荷
ActionResult	.status、.message	None

结果

状态	类别	含义
"ok"	success（成功）	操作已成功完成。
"not_found"	rejection（拒绝）	请求的对象、目标或记录不存在。
"incompatible"	rejection（拒绝）	所选端点或值不兼容。

`.disconnect()`

仅清除此端口声明的目标。缓冲的流体会保留。如果对等端独立声明了相同关系，则该反向声明仍然有效。

返回值

返回类型	结果字段	成功载荷
ActionResult	.status、.message	None

结果

状态	类别	含义
"ok"	success（成功）	操作已成功完成。

`.connected_to()`

返回此端口声明的目标显示名称；如果没有则返回空字符串。它不会列出由对等端口拥有的兼容反向声明。

返回值

string

`.connected_id()`

返回此端口声明的目标稳定 ID；如果没有则返回空字符串。`connect()` 接受 ID 或显示名称，因此当需要确认身份与所传值匹配时，应与此值比较：`connected_to()` 返回的是可重命名的名称。要获取每个有效对等端（包括由另一端拥有的声明），请使用 `connections()`。

返回值

string

`.connections()`

读取此端口上每个有效对等关系的只读快照，包括由对等端口编写的声明。每个 `FluidConnection` 都会报告对等端、确切流体（如果已知）、声明所有权和结构状态。管道 ID 会刻意不公开，也不会被选择。

返回值

list<FluidConnection>

`.level()`

此端口当前缓冲的流体量，单位为吨。像 Pump 输出端这样的直通源端口不存储任何内容，因此返回 0。

返回值

number

`.capacity()`

此端口缓冲区最多可容纳的吨数。

返回值

number

`.flow_rate()`

当前实时总流量，单位为 t/h。0

可能表示空闲、缺料、已满、不可达、存在冲突，或正处于模拟时序边界等待中；它不会删除连接或管道身份。

返回值

number

TYPES / INFRASTRUCTURE & FLUIDS（类型 / 基础设施与流体）

FluidConnection

由 `FluidPort.connections()` 返回。

方法

`.machine_id`

对等机器的稳定 ID。

返回值

`string`

`.machine_name`

对等机器当前的显示名称。

返回值

`string`

`.fluid`

由此关系建立的确切流体；当关系处于中性状态或不兼容时为 `None`。

返回值

类型	可能值
<code>Optional[str]</code>	<code>"steam"</code> 、 <code>"water"</code> 、 <code>"oil"</code> 、 <code>"frozen_essence"</code> 、 <code>"coastal_essence"</code> 、 <code>"geothermal_essence"</code> 、 <code>"volcanic_essence"</code> 、 <code>"deep_essence"</code> 、 <code>"ammonia"</code> 、 <code>"swamp_gas"</code> 、 <code>"raw_sulfur_gas"</code> 、 <code>"sulfur_gas"</code> 、 <code>"raw_chlorine"</code> 、 <code>"chlorine"</code> 、 <code>"brine"</code> 、 <code>"raw_cryofluid"</code> 、 <code>"cryofluid"</code> 、 <code>"raw_quicksilver"</code> 、 <code>"quicksilver"</code>

`.declared_by`

持久声明的拥有者：`"self"`、`"peer"` 或 `"both"`。一侧声明即可进行传输。

返回值

类型	可能值
<code>string</code>	<code>"self"</code> 、 <code>"peer"</code> 、 <code>"both"</code>

.state

连接状态：同一前哨站内的直接连接为 "local"；可用的远程管道网络为 "ready"；没有已完成组件可分配时为 "unreachable"；分配的组件携带不同流体连接时为 "conflict"；尚不知道流体时为 "neutral"；两种流体不一致时为 "incompatible"。

返回值

类型	可能值
string	"local"、"ready"、"unreachable"、"conflict"、"neutral"、"incompatible"

Pipe

返回自：`list_pipes()` / `get_pipe(pipe_id)`

方法

`.id`

此管道的唯一标识符。

返回值

string

`.start()`

此管道片段的几何起点坐标，类型为 `Position`。这是构造几何中的起点，并不表示流动方向。仅当格式错误的状态中不包含片段几何时才返回 `None`。

返回值

Optional[Position]

`.end()`

此管道片段的几何终点坐标，类型为 `Position`。这是构造几何中的终点，并不表示流动方向。仅当格式错误的状态中不包含片段几何时才返回 `None`。

返回值

Optional[Position]

`.type()`

管道硬件介质：`"gas"`（气体）或 `"liquid"`（液体）。要获取完整提供者-消费者连接所确定的确切物质，请使用 `contents()`。

返回值 | 可能值

string | `"gas"`, `"liquid"`

`.contents()`

由完整的玩家连接所确定、且其位置可到达此物理组件的唯一确切流体，例如 `"steam"`、`"water"` 或 `"oil"`。没有完整连接，或多个确切物质发生冲突时返回 `None`。活动状态、电力、节流、流量和余量都不会改变这一身份。

返回值 | 可能值

`Optional[string] | "steam", "water", "oil", "frozen_essence", "coastal_essence", "geothermal_essence", "volcanic_essence", "deep_essence", "ammonia", "swamp_gas", "raw_sulfur_gas", "sulfur_gases", "raw_chlorine", "chlorine", "brine", "raw_cryofluid", "cryofluid", "raw_quicksilver", "quicksilver"`

`.conflicting_contents()`

当多个连接使用此物理组件时，由完整的提供者-消费者连接确定的确切物质列表，按排序返回；如果没有冲突，则为空列表。非空结果表示流动已停止。

返回值

`list[string]`

`.connections()`

在此物理组件上建立的诊断机器端口声明。每个字典包含 `machine_id`、`port`、`direction`、`fluid` 以及该组件中的代表性 `pipe_id`；玩家不会直接连接到该管道 id。

返回值

```
list[dict[str, string]]
```

`.incompatible_sinks()`

直接连接但无法接受 `contents()` 的消费者 id。这些消费者不会收到任何内容，而兼容的分支会继续流动。实时读取。

返回值

```
list[string]
```

`.is_complete()`

当建造器（Constructor）完成铺设管道且流动可以运行时为 `True`。实时读取。

返回值

boolean

`.length()`

管道的总长度（米），为所有水平/垂直（H/V）片段长度之和。

返回值

number

`.laying_head()`

管道尚未完成时当前铺设头的 `[x, y]` 坐标；完成后为 `None`。实时读取。

返回值

`Optional[[number, number]]`

`.flow_rate()`

当前通过管道的流量，单位为每个世界小时的吨数（`t/h`）。管道未完成、停滞、源为空或发生冲突时为 `0`。实时读取。

返回值

`number`

`.state()`

管道当前状态：`"flowing"`（流动中）、`"stalled"`（停滞）、`"incomplete"`（未完成）、`"no_source"`（无源）或 `"conflict"`（冲突）。实时读取。

返回值 | 可能值

`string` | `"flowing"`, `"stalled"`, `"incomplete"`, `"no_source"`, `"conflict"`

类型 / 基础设施与流体

PowerGrid

返回自：`power_control.grids()/power_control.grid(target_id)`

相关对象类型

PowerGridMember

方法

`.anchor_id`

当前标识此连接电网的确定性实体 id，可传递给 `power.grid(...)`。重新布线后请重新发现电网，因为合并或拆分电网可能会改变其锚点。

返回值

string

`.outpost_ids`

连接到此电网的前哨 id，按确定性顺序排序。仅包含场地结构的电网会返回空列表。

返回值

list<string>

`.machine_ids`

分配给此电网的建筑和场地电力结构 id，按确定性顺序排序。移动单位和飞船设备不包括在内，因为它们的部署位置不是电气连接。

返回值

list<string>

.members

此电网上每个建筑和场地电力结构的电力详情，作为 **PowerGridMember** 快照返回。使用每个成员的 **.id** 配合 **get_component(...)**，以获取特定类型的配方、材料或控制方法。

返回值

```
list<PowerGridMember>
```

.generated

返回此快照时，该电网的活动发电机当前提供的电力，单位为 W。

返回值

number

.consumed

返回此快照时，该电网的活动消费者当前消耗的电力，单位为 W。

返回值

number

.net

当前电网发电量减去消耗量，单位为 W。负值表示当前负载需要使用储存的能量。

返回值

number

.stored

此电网上常规电池中储存的能量，单位为 Wh。

返回值

number

.capacity

此电网上常规电池的总容量，单位为 Wh。

返回值

number

.reserve_stored

此电网上闪电塔（Lightning Rod）当前储存的能量，单位为 Wh。只有在常规电池耗尽后才会使用此储备，并且它不会吸收普通发电盈余。

返回值

number

.reserve_capacity

此电网上闪电塔储备的总容量，单位为 Wh。

返回值

number

.has_generator

当此电网包含发电机时为 True，即使发电机当前输出为 0 W。

返回值

boolean

类型 / 基础设施与流体

PowerGridMember

返回自：`PowerGrid.members`

方法

`.id`

稳定的固定机器实例 id。将其传递给 `get_component(...)`、`power.grid(...)` 或断路器 API。

返回值

string

`.name`

返回此快照时的显示名称。

返回值

string

`.type_id`

稳定的机器类型 id，例如 `"solar_generator"`、`"battery"` 或 `"smelter"`。

返回值

string

`.outpost_id`

所属前哨 id；对于独立放置的场地结构，则为空字符串。

返回值

string

`.powered`

返回此快照时，机器的供电状态为开启时为 `True`。

返回值

boolean

`.roles`

此机器的电力能力：可以是 `"generator"`（发电）、`"consumer"`（耗电）、`"storage"`（储能）和 `"reserve"`（储备）的任意组合。空列表表示机器没有直接电气角色，但安装在已连接的前哨上。

返回值

`list<string>`

`.generated`

此机器当前提供的电力，单位为 W。发电机可能因为关闭、空闲、没有燃料或缺少阳光而报告 `0 W`。

返回值

number

`.consumed`

此机器当前消耗的电力，单位为 W。关闭或空闲的机器报告 `0 W`。

返回值

number

`.stored`

此常规电池中储存的能量，单位为 Wh；对于其他机器角色则为 `0`。

返回值

number

.capacity

常规电池容量，单位为 Wh；对于其他机器角色则为 0。

返回值

number

.reserve_stored

此机器中储存的闪电储备，单位为 Wh；对于其他机器角色则为 0。

返回值

number

.reserve_capacity

此机器的闪电储备容量，单位为 Wh；对于其他机器角色则为 0。

返回值

number

类型 / 基础设施与流体

PowerSummary

返回自：`power_control.total()`

方法

`.grid_count`

此快照中包含的、彼此独立且已完成的电网数量。

返回值

number

`.generated`

返回此快照时所有电网当前提供的电力，单位为 W。

返回值

number

`.consumed`

返回此快照时所有电网当前消耗的电力，单位为 W。

返回值

number

`.net`

所有电网的全星球发电量减去消耗量，单位为 W。

返回值

number

.stored

所有电网常规电池中储存的能量，单位为 Wh。

返回值

number

.capacity

所有电网常规电池的总容量，单位为 Wh。

返回值

number

.reserve_stored

所有电网闪电塔中储存的能量，单位为 Wh。

返回值

number

.reserve_capacity

所有电网闪电塔储备的总容量，单位为 Wh。

返回值

number

类型 / 订单与通信

BroadcastInfo

返回自：`comms.latest_info(channel)`；当状态为 "ok" 时，返回自 `comms.wait_broadcast(channel).broadcast`。

方法

`.value`

此广播值的副本；即使发布的值为 `None` 也会包含在内。对于 `latest_info()`，这是最新值；对于 `wait_broadcast()`，这是该等待操作捕获的首次发布值。

返回值

any

`.sender`

发布此广播的对象；如果保存的广播没有发送者信息，则为 `None`。

返回值

Optional[string]

`.age_seconds`

自此广播发布以来经过的模拟秒数，在读取 `latest_info()` 或 `wait_broadcast()` 恢复运行时采样。使用与 `sleep()` 和 `clock.elapsed_seconds()` 相同的时间基准；暂停模拟会冻结此时长。`None` 表示时间戳缺失或无效。返回的时长是一个快照。

返回值

Optional[number]

类型 / 订单与通信

CommsMessage

返回自：`comms.pending(channel)` 列表项；当状态为 "ok" 时，返回自 `comms.receive(channel).packet`、`comms.wait(channel).packet` 或 `comms.wait_any(channels).packet`。

方法

`.id`

由信号总线（Signal Bus）分配的单调递增消息 id。

返回值

number

`.sender`

发送此消息的脚本所有者 id。

返回值

string

`.tick`

发送消息时的游戏 tick。

返回值

number

`.value`

JSON 安全的消息值：`None`、布尔值、数字、字符串、列表，或键为字符串的字典。

返回值

any

类型 / 订单与通信

DockSlot

返回自：`supply_dock.slots()`

方法

`.index`

从零开始的槽位索引（0-4），在各次调用之间保持稳定。

返回值

number

`.item_id`

当前占用此槽位的物品 id；槽位为空时为 `None`。

返回值

`Optional[string]`

`.count`

此槽位当前储存的单位数量。

返回值

number

类型 / 订单与通信

Order

返回自：`orders.list_orders()` / `orders.list_weekly_orders()` / `orders.get_order()` / `orders.completed_orders()`（地球订单）

方法

`.id`

不可变的地球订单 id。适用于日志和仪表板；供应码头脚本使用它来识别当前分配给其码头的订单。

返回值

string

`.name`

地球订单经过预翻译的显示名称。

返回值

string

`.requires`

地球所需物品的字典 `{item_id: count}`。可以使用 `.keys()`、`.values()`、`.items()`，或直接索引读取：`order.requires["iron_ore"]`。

返回值

dict<number>

`.shipped`

已着陆的各物品数量字典 `{item_id: count}`。使用 `.get(item_id, 0)` 可安全读取。

返回值

`dict<number>`

`.reward_credits`

完成订单时发放的信用点数。

返回值

`number`

`.reward_kind`

可变奖励类型：`"recipe"`（配方）/`"tech"`（科技）；仅信用点奖励时为 `None`。

返回值 | 可能值

`Optional[string] | "recipe", "tech"`

`.reward_label`

可变奖励的预翻译说明；仅信用点奖励的地球订单为 `None`。

返回值

`Optional[string]`

`.status`

未履行时为 `"active"`，交付后为 `"completed"`。

返回值 | 可能值

`string | "active", "completed"`

`.kind`

订单来源："campaign"（战役）或 "weekly"（每周）。

返回值 | 可能值

`string` | "campaign", "weekly"

`.expires_day`

每周地球订单到期的日期；承包商战役订单为 **None**。

返回值

`Optional[number]`

`.contractor_id`

签发承包商的 id；每周地球订单为 **None**。该 id 在存档之间保持稳定，可用于针对特定战役合作伙伴进行分支处理。

返回值 | 可能值

`Optional[string]` | "helios_orbital", "spire_research", "vestibule_logistics"

`.contractor_name`

签发承包商的预翻译名称；每周地球订单为 **None**。

返回值

`Optional[string]`

类型 / 订单与通信

ReceiveResult

返回自：`comms.receive()`；`comms.wait()`。

方法

`.status`

当消费了排队消息时为 "ok"；当未指定 id 且队列为空时为 "empty"；当请求的 id 不存在时为 "not_found"；当频道 id 无效时为 "invalid_channel"。

返回值 | 可能值

string | "ok", "empty", "not_found", "invalid_channel"

`.message`

面向玩家的接收结果说明。

返回值

string

`.packet`

当状态为 "ok" 时，为已消费的 `CommsMessage`；否则为 `None`。请求消息 id 时，此数据包具有该确切 id。

返回值

`Optional[CommsMessage]`

类型 / 订单与通信

SendResult

返回自：`comms.send()`

方法

`.status`

发送结果。"ok" 表示消息已排队；任何其他结果都表示未发送消息。

返回值 | 可能值

string | "ok", "invalid_channel", "channel_limit", "queue_full", "id_exhausted", "invalid_value"

`.message`

面向玩家的发送结果说明。

返回值

string

`.message_id`

当状态为 "ok" 时，已排队消息的正整数 id；未发送消息时为 `None`。它与 `pending()` 中的 `.id` 以及 `receive().packet` 中的 `.id` 相匹配。使用它配合 `receive()`、`cancel()` 或 `update()`，以定位该确切请求。

返回值

Optional[number]

TransmitterInfo

返回值来源：`transmitter.get_info()`

方法

.connected

如果已连接到目标，则为 True。

返回值

boolean

.target

已连接的行星 ID；未连接时为 "none"。

返回值 | 可能的值

string | "none"

WaitAnyResult

返回值来源：`comms.wait_any()`

方法

`.status`

当某条排队消息已被消费时为 "ok"；当列出的任一频道 ID 无效时为 "invalid_channel"。有效的空队列会继续等待。

返回值 | 可能的值

string | "ok"、"invalid_channel"

`.message`

对等待结果的面向玩家说明。

返回值 string

`.channel`

当 `status == "ok"` 时，返回产生所消费消息的频道；否则为 `None`。

返回值 `Optional[string]`

`.packet`

当 `status == "ok"` 时，返回已消费的 `CommsMessage`；否则为 `None`。其中的 `id`、`sender`、`tick` 和复制后的 `value` 属于所选频道。

返回值 `Optional[CommsMessage]`

WaitBroadcastResult

返回值来源：`comms.wait_broadcast()`

方法

.status

捕获到新广播时为 "ok"；频道 ID 无效时为 "invalid_channel"。有效频道会持续等待，直到出现新广播。

返回值 | 可能的值

string | "ok"、"invalid_channel"

.message

对广播等待结果的面向玩家说明。

返回值 string

.broadcast

当 `status == "ok"` 时，返回捕获到的 `BroadcastInfo`；否则为 `None`。它包含发布内容的复制值、发送者以及脚本恢复运行时的存续时间。

返回值 `Optional[BroadcastInfo]`

类型 / 面板 (TYPES / PANELS)

Panel

返回值来源：`panel`（仅限面板脚本）。在控制室（Control Room）页面上创建一个面板；参见 `custom_panels`。

方法

`card(x, y, w, h, title?)`

带可选标题栏的有边框子区域。用于将相关内容分组，视觉效果与仪表板卡片风格一致。

[图示：原 PDF 中的卡片面板示意图，已保留于此处。]

参数

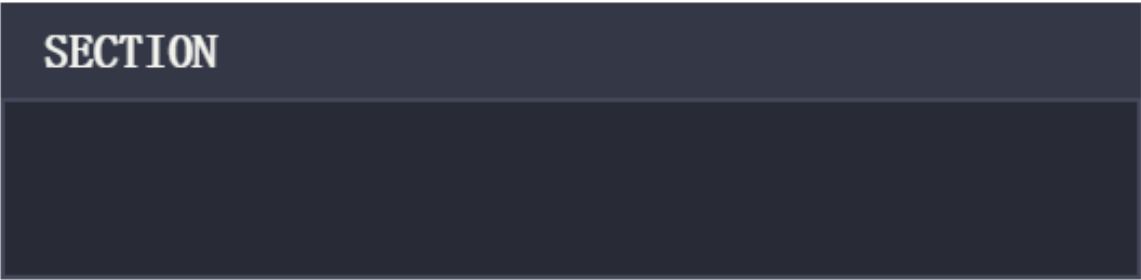
名称	类型	描述
<code>x</code>	<code>number</code>	X 坐标（像素）
<code>y</code>	<code>number</code>	Y 坐标（像素）
<code>w</code>	<code>number</code>	宽度（像素）
<code>h</code>	<code>number</code>	高度（像素）
<code>title</code>	<code>string</code>	可选的卡片标题

返回值 | 结果字段 | 成功载荷

`ActionResult` | `.status`、`.message` | `None`

结果

状态	类别	含义
<code>"ok"</code>	成功	操作已成功完成。



divider(x1, y1, x2, y2)

使用低调边框颜色绘制水平或垂直分隔线。用于将面板划分为不同的视觉区域。

[图示：原 PDF 中的分隔线示意图，已保留于此处。]

参数

名称	类型	描述
x1	number	起点 X 坐标（像素）
y1	number	起点 Y 坐标（像素）
x2	number	终点 X 坐标（像素）
y2	number	终点 Y 坐标（像素）

返回值 | 结果字段 | 成功载荷

`ActionResult` | `.status`、`.message` | `None`

结果："ok"（成功）- 操作已成功完成。



`label(x, y, text, style?)`

绘制具有语义样式的文本。`style` 可取 `"title"`（明亮、粗体）、`"caption"`（低调、全大写）、`"muted"`（次要文本）或 `"value"`（数值读数）。默认值为 `"title"`。当希望自动应用仪表板文本层级时，请使用它，而不是 `draw_text`。

[图示：原 PDF 中的标签文本示意图，已保留于此处。]

参数：`x` (`number`, X 坐标像素)、`y` (`number`, Y 坐标像素)、`text` (`string`, 标签文本)、`style` (`string`, 文本样式预设)。

返回值 | 结果字段 | 成功载荷

`ActionResult` | `.status`、`.message` | `None`

结果：`"ok"`（成功）- 操作已成功完成。

OXYGEN

`status_dot(x, y, r, status)`

根据状态字符串解析出彩色圆点。可识别的值包括："running"（绿色）、"paused"（警告色）、"error"（错误红色）和 "idle"（低调色）。绘制时会带有细微的外发光环。适用于机器列表中的逐行状态指示器。

[图示：原 PDF 中的状态圆点示意图，已保留于此处。]

参数：x (number, X 坐标像素)、y (number, Y 坐标像素)、r (number, 半径像素)、status (string, 状态预设)。

返回值 | 结果字段 | 成功载荷

ActionResult | .status、.message | None

结果："ok"（成功）- 操作已成功完成。



`toggle(x, y, on, label?)`

与仪表板机器开关控件一致的绿色/灰色电源切换胶囊。此控件只显示传入的 `on` 值；如需可点击控件，请使用 `panel.switch(...)`。

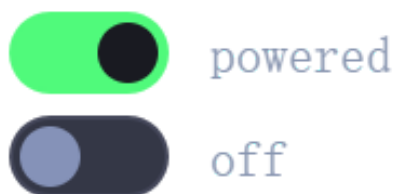
[图示：原 PDF 中的切换开关示意图，已保留于此处。]

参数：`x` (`number`, X 坐标像素)、`y` (`number`, Y 坐标像素)、`on` (`boolean`, 是否开启)、`label` (`string`, 可选标签)。

返回值 | 结果字段 | 成功载荷

`ActionResult` | `.status`、`.message` | `None`

结果："ok"（成功）- 操作已成功完成。



`pill(x, y, text, color?)`

带圆角的文本徽章，风格类似成就徽章。`color` 可接受主题标记 ("accent"、"success"、"warning"、"error"、"text-muted") 或十六进制颜色。可用于状态标签或分类标记。

[图示：原 PDF 中的圆角徽章示意图，已保留于此处。]

参数：`x` (number, X 坐标像素)、`y` (number, Y 坐标像素)、`text` (string, 徽章文本)、`color` (string, 主题标记或任意 CSS 颜色，如 `hex`、`rgb()`、`hsl()`、命名颜色)。

返回值 | 结果字段 | 成功载荷

`ActionResult` | `.status`、`.message` | `None`

结果："ok" (成功) - 操作已成功完成。



`counter(x, y, value, label?, size?)`

大型数字统计块：上方显示大号数值，下方显示小号全大写标签。用于标题式数字（信用点、天数、锭库存）。`size` 默认为 24。

[图示：原 PDF 中的计数器示意图，已保留于此处。]

参数：`x` (`number`, X 坐标像素)、`y` (`number`, Y 坐标像素)、`value` (`any`, 显示值)、`label` (`string`, 可选标签)、`size` (`number`, 可选数字字体大小像素，默认 24)。

返回值 | 结果字段 | 成功载荷

`ActionResult` | `.status`、`.message` | `None`

结果："ok" (成功) - 操作已成功完成。

87
SHIPPED

`progress_bar(x, y, w, h, fraction, color?)`

带轨道和填充强调色的水平填充条。`fraction` 会限制在 0 到 1 之间。`color` 默认为 "accent"；可使用 "success"、"warning"、"error" 表示交通灯式提示。

[图示：原 PDF 中的进度条示意图，已保留于此处。]

参数：`x` (number, X 坐标像素)、`y` (number, Y 坐标像素)、`w` (number, 宽度像素)、`h` (number, 高度像素)、`fraction` (number, 填充比例 0-1)、`color` (string, 主题标记或 CSS 颜色)。

返回值 | 结果字段 | 成功载荷

`ActionResult` | `.status`、`.message` | None

结果："ok" (成功) - 操作已成功完成。



`vertical_bar(x, y, w, h, fraction, color?)`

垂直填充条，从底部向上填充。`fraction` 和 `color` 的规则与 `progress_bar` 相同。当面板布局偏好纵向排列时使用（多罐堆叠、气氛堆叠）。

[图示：原 PDF 中的垂直填充条示意图，已保留于此处。]

参数：`x` (`number`, X 坐标像素)、`y` (`number`, Y 坐标像素)、`w` (`number`, 宽度像素)、`h` (`number`, 高度像素)、`fraction` (`number`, 填充比例 0-1)、`color` (`string`, 主题标记或 CSS 颜色)。

返回值 | 结果字段 | 成功载荷

`ActionResult` | `.status`、`.message` | `None`

结果："ok"（成功）- 操作已成功完成。



bar_chart(x, y, w, h, values, max?, labels?)

多柱比较图，使用主题交替颜色；可选在每根柱下方显示标签。**max** 为可选参数；省略时会自动缩放到最大值。

[图示：原 PDF 中的柱状图示意图，已保留于此处。]

参数：**x** (number, X 坐标像素)、**y** (number, Y 坐标像素)、**w** (number, 宽度像素)、**h** (number, 高度像素)、**values** (any, 要绘制的数值)、**max** (number, 可选最大值)、**labels** (any, 可选标签)。

返回值 | 结果字段 | 成功载荷

ActionResult | .status、.message | None

结果："ok"（成功）- 操作已成功完成。



`gauge(x, y, radius, fraction, label?)`

四分之三圆形的刻度盘，带弧形填充。`fraction`会限制在 0 到 1 之间，弧线从左下方开始，沿 270 ° 扫过至右下方。可选的中心标签位于中央（通常将数值作为文本显示）。

[图示：原 PDF 中的仪表盘示意图，已保留于此处。]

参数：`x` (`number`, X 坐标像素)、`y` (`number`, Y 坐标像素)、`radius` (`number`, 仪表盘半径像素)、`fraction` (`number`, 0-1 填充比例)、`label` (`string`, 可选标签)。

返回值 | 结果字段 | 成功载荷

`ActionResult` | `.status`、`.message` | `None`

结果："ok"（成功）- 操作已成功完成。



`spark_line(x, y, w, h, values)`

根据数值序列绘制紧凑的趋势线。玩家将数值累积到列表中，并在每个 tick 推入最新值；控件会将数值归一化到该序列的最小值/最大值范围，并在曲线下方填充一块低调区域。空序列或只有一个值的序列不执行任何操作。

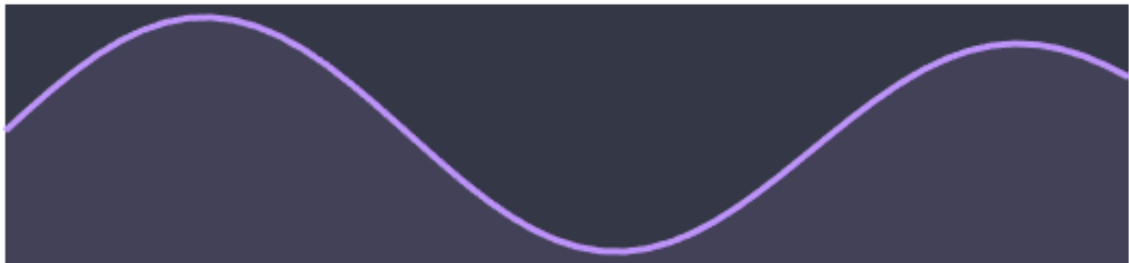
[图示：原 PDF 中的趋势线示意图，已保留于此处。]

参数：`x` (number, X 坐标像素)、`y` (number, Y 坐标像素)、`w` (number, 宽度像素)、`h` (number, 高度像素)、`values` (any, 要绘制的数值)。

返回值 | 结果字段 | 成功载荷

`ActionResult` | `.status`、`.message` | `None`

结果："ok" (成功) - 操作已成功完成。



`button(key, x, y, w?, h?, label?)`

可点击按钮。传入唯一的 `key`，使点击事件能够路由回该按钮。按钮只在被按下的那个 tick 返回 `True`（瞬时触发）；请据此分支：

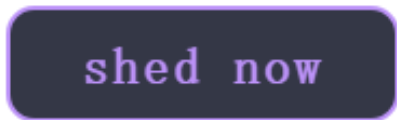
```
if panel.button("shed", 10, 12):  
    power.set_powered(...)
```

`w/h` 的默认值为 90×26 。这是第一个能让卡片响应玩家点击、而不只是进行绘制的输入控件。

[图示：原 PDF 中的按钮示意图，已保留于此处。]

参数：`key` (`string`，控件唯一 ID)、`x` (`number`，X 坐标像素)、`y` (`number`，Y 坐标像素)、`w` (`number`，宽度像素，默认 90)、`h` (`number`，高度像素，默认 26)、`label` (`string`，可选标签)。

返回值 `boolean`



`switch(key, x, y, default_on?, label?)`

交互式开关；玩家点击它即可翻转状态。传入唯一的 `key`；`default_on` 用于设置卡片首次运行时的初始状态。每个 tick 都返回当前布尔值，翻转状态会保存在卡片状态中，并在重新加载后保留。它不同于只显示传入状态的显示切换控件；这是一个可点击的控件：

```
auto = panel.switch("auto_recover", 165, 128, True)
```

[图示：原 PDF 中的交互式开关示意图，已保留于此处。]

参数：`key` (string, 控件唯一 ID)、`x` (number, X 坐标像素)、`y` (number, Y 坐标像素)、`default_on` (boolean, 初始开/关状态)、`label` (string, 可选标签)。

返回值 `boolean`



`slider(key, x, y, w, default?, label?)`

玩家点击即可设置数值的水平滑块。传入唯一的 `key`；`default`（0-1）设置初始值。每个 tick 都返回当前的 0-1 数值，并将其保存在卡片状态中。适用于玩家实时调节的阈值、购买触发等级或节流目标。

[图示：原 PDF 中的滑块示意图，已保留于此处。]

参数：`key` (`string`，控件唯一 ID)、`x` (`number`，X 坐标像素)、`y` (`number`，Y 坐标像素)、`w` (`number`，宽度像素)、`default` (`number`，初始值 0-1)、`label` (`string`，可选标签)。

返回值 `number`



`draw_text(x, y, text, size?, color?, wrap?)`

以指定大小使用等宽字体渲染文本。可选的

`wrap` (像素宽度) 会启用贪心式单词换行, 将文本排成多行块, 适用于日志流和订单简报。`color` 可接受主题标记或十六进制颜色。

[图示: 原 PDF 中的文本绘制示意图, 已保留于此处。]

参数: `x` (number, X 坐标像素)、`y` (number, Y 坐标像素)、`text` (string, 要绘制的文本)、`size` (number, 字体大小像素)、`color` (string, 主题标记或 CSS 颜色)、`wrap` (number, 可选换行宽度像素)。

返回值 | 结果字段 | 成功载荷

`ActionResult` | `.status`、`.message` | `None`

结果: "ok" (成功) - 操作已成功完成。

Hello, panel.

icon_ids()

返回 `draw_icon` 可以渲染的全部 ID 列表，并按排序顺序排列。列表涵盖的内容比玩家可持有的物品更多：流体、生物、精华和机器美术资源也都有图标。可用它构建选择器、在绘制前验证 ID，或在编写卡片时打印一次完整目录。

```
for icon in panel.icon_ids():
    print(icon)
```

返回值 `list<string>`

draw_icon(x, y, item_id, size?)

以请求的大小渲染游戏物品目录中的任意图标（默认 32 像素）。`item_id` 与传给 `inventory/storage_bin` API 的字符串相同（例如 `"iron_ore"`、`"iron_ingot"`、`"water"` 等），此外还包括玩家不会持有的内容，例如流体和生物。`panel.icon_ids()` 返回完整列表；每个物品自己的页面位于此面板下的 Database 中。未知 ID 会静默地不执行任何操作。

[图示：原 PDF 中的图标渲染示意图，已保留于此处。]

参数：`x` (`number`, X 坐标像素)、`y` (`number`, Y 坐标像素)、`item_id` (`string`, 要绘制图标的物品 ID)、`size` (`number`, 图标大小像素，默认 32)。

返回值 | 结果字段 | 成功载荷

`ActionResult` | `.status`、`.message` | `None`

结果：`"ok"`（成功）- 操作已成功完成。

draw_rect(x, y, w, h, color?)

使用指定颜色绘制空心矩形（默认使用 "border"）。可用于自定义小节边框或视觉框架。

图示：空心矩形绘制效果（原 PDF 图片，位于说明文字之后）。

参数

名称	类型	描述
x	number	X 坐标（像素）
y	number	Y 坐标（像素）
w	number	宽度（像素）
h	number	高度（像素）
color	string	主题令牌，或任意 CSS 颜色（hex、rgb()、hsl()、命名颜色）

返回值

返回类型	结果字段	成功载荷
ActionResult	.status、.message	None

结果

状态	类型	含义
"ok"	成功	操作已完成。



fill_rect(x, y, w, h, color?)

使用指定颜色绘制实心矩形（默认使用 "accent"）。可用于背景、进度填充和色块。

图示：实心矩形绘制效果（原 PDF 图片，位于说明文字之后）。

参数

名称	类型	描述
x	number	X 坐标（像素）
y	number	Y 坐标（像素）
w	number	宽度（像素）
h	number	高度（像素）
color	string	主题令牌，或任意 CSS 颜色（hex、rgb()、hsl()、命名颜色）

返回值

返回类型	结果字段	成功载荷
ActionResult	.status、.message	None

结果

状态	类型	含义
"ok"	成功	操作已完成。



draw_circle(x, y, r, color?)

使用指定颜色绘制空心圆。

图示：空心圆绘制效果（原 PDF 图片，位于说明文字之后）。

参数

名称	类型	描述
x	number	X 坐标（像素）
y	number	Y 坐标（像素）
r	number	半径
color	string	主题令牌，或任意 CSS 颜色（hex、rgb()、hsl()、命名颜色）

返回值

返回类型	结果字段	成功载荷
ActionResult	.status、.message	None

结果

状态	类型	含义
"ok"	成功	操作已完成。



fill_circle(x, y, r, color?)

使用指定颜色绘制实心圆盘。

图示：实心圆盘绘制效果（原 PDF 图片，位于说明文字之后）。

参数

名称	类型	描述
x	number	X 坐标（像素）
y	number	Y 坐标（像素）
r	number	半径
color	string	主题令牌，或任意 CSS 颜色（hex、rgb()、hsl()、命名颜色）

返回值

返回类型	结果字段	成功载荷
ActionResult	.status、.message	None

结果

状态	类型	含义
"ok"	成功	操作已完成。



draw_line(x1, y1, x2, y2, color?)

绘制一条直线。

图示：直线绘制效果（原 PDF 图片，位于说明文字之后）。

参数

名称	类型	描述
x1	number	起点 X 坐标（像素）
y1	number	起点 Y 坐标（像素）
x2	number	终点 X 坐标（像素）
y2	number	终点 Y 坐标（像素）
color	string	主题令牌，或任意 CSS 颜色（hex、rgb()、hsl()、命名颜色）

返回值

返回类型	结果字段	成功载荷
ActionResult	.status、.message	None

结果

状态	类型	含义
"ok"	成功	操作已完成。

clear()

清空面板画布。每次 while True: 循环迭代开始时都应调用它，以免旧图形残留在新绘制内容后面。

返回值

返回类型	结果字段	成功载荷
ActionResult	.status、.message	None

结果

状态	类型	含义
"ok"	成功	操作已完成。

`width()`

当前逻辑画布宽度（像素）：单列卡片为 **500**，双列卡片为 **1000**。可用于按比例定位。

返回值

`number`

`height()`

当前逻辑画布高度（像素）：单行卡片为 **200**，双行卡片为 **400**。可用于按比例定位。

返回值

`number`

类型 / 合约

Contract

返回自：`self.contract`

具体子类型

BeatTheSystemContract
BuriedFiveContract
ColdBootContract
CoreSampleContract
CorruptedArchiveContract
CrosstalkContract
DataTableContract
DriftingSignalContract
LatticeContract
RelayHackContract
SealedVaultContract
TerminalBreachContract
TheLoomContract
ThreeEchoesContract
XenogeneticsContract

方法

`.id`

用于传输答案的合约 ID。

返回值

返回类型	可能的值
string	"relay_hack"、"xenogenetics"、"corrupted_archive"、"sealed_vault"、"data_tablet"、"terminal_breach"、"drifting_signal"、"cold_boot"、"three_echoes"、"buried_five"、"the_loom"、"crosstalk"、"beat_the_system"、"cng_ore_sample"、"lattice"

`.name`

合约显示名称。

返回值

string

`.reward`

完成此合约可获得的信用点奖励。

返回值

number

`.status`

合约状态：`'available'`（可用）或 `'completed'`（已完成）。

返回值

返回类型	可能的值
<code>string</code>	<code>"available"</code> 、 <code>"completed"</code>

类型 / 合约

BeatTheSystemContract

扩展自 Contract。

返回自：self.contract (beat_the_system)

相关对象类型

Arbiter

方法

.id

用于传输答案的合约 ID。

返回值

返回类型	可能的值
string	"relay_hack", "xenogenetics", "corrupted_archive", "sealed_vault", "data_tablet", "terminal_breach", "drifting_signal", "cold_boot", "three_echoes", "buried_five", "the_loom", "crosstalk", "beat_the_system", "core_sample", "lattice"

.name

合约显示名称。

返回值

string

.reward

完成此合约可获得的信用点奖励。

返回值

number

.status

合约状态：'available'（可用）或 'completed'（已完成）。

返回值

返回类型	可能的值
string	"available"、"completed"

.arbiter

外星仲裁者（Arbiter）。它会优先采取能立即获胜的行动；否则阻止你的立即获胜行动；再否则优先选择中心、角落，最后选择边缘。若有多个选择并列，则随机决定。

返回值

Arbiter

类型 / 合约

Arbiter

返回自：`.arbiter`

方法

`.new_game()`

在空棋盘上开始一局新的 3×3 游戏；由你先行。游戏结束后，此调用会暂停约半秒，然后准备好下一块棋盘。

返回值

返回类型	结果字段	成功载荷
ActionResult	.status、.message	None

结果

状态	类型	含义
"ok"	成功	操作已完成。
"in_progress"	暂态	操作仍在进行中。

`.restart()`

放弃正在进行的比赛并重新开始；由你先行。在游戏中途放弃会被视为未获胜，并将当前连续胜局数重置为 0。与 `new_game()` 一样，两局之间会暂停约半秒。

返回值

返回类型	结果字段	成功载荷
ActionResult	.status、.message	None

结果

状态	类型	含义
"ok"	成功	操作已完成。

`.play(cell)`

在 0-8 范围内的整数格子中落子（按行优先顺序编号），然后由仲裁者回应。任意一行、一列或一条对角线上出现三个棋子即可获胜。非数字格子会引发 `TypeError`；非有限数、分数或超出范围的格子会在检查游戏状态之前引发 `ValueError`。

参数

名称	类型	描述
<code>cell</code>	<code>number</code>	要落子的棋盘格，取值为 0-8 的整数

返回值

返回类型	结果字段	成功载荷
<code>ActionResult</code>	<code>.status</code> 、 <code>.message</code>	<code>None</code>

结果

状态	类型	含义
<code>"ongoing"</code>	成功	游戏仍在进行。
<code>"win"</code>	成功	游戏以获胜结束。
<code>"loss"</code>	成功	游戏以失败结束。
<code>"draw"</code>	成功	游戏以平局结束。
<code>"occupied"</code>	拒绝	请求的格子、区域或 <code>Habitat</code> 已被占用。
<code>"no_game"</code>	拒绝	当前没有正在进行的比赛。

引发的异常

异常	条件
<code>TypeError</code>	<code>Arbiter.play()</code> 要求传入数值格子。
<code>ValueError</code>	<code>Arbiter.play()</code> 要求传入 0-8 范围内的有限整数格子。

`.board()`

以列表形式返回 9 个棋盘格，索引按行优先顺序取 0-8。每个格子的值为 ""（空）、"you" 或 "arbiter"。

返回值

`list<string>`

`.result()`

当前游戏结果："ongoing"、"win"、"loss"、"draw" 或 "no_game"（尚未开始游戏）。

返回值

返回类型	可能的值
string	"ongoing"、"win"、"loss"、"draw"、"no_game"

`.streak()`

当前脚本运行中的连续胜局数。若失败、平局、放弃游戏或重新运行脚本，则重置为 0。

返回值

number

`.target()`

完成合约所需的连续获胜次数。

返回值

number

`.token()`

要传输的通行码：当 `streak()` 达到 `target()` 时返回非空字符串，否则返回空字符串。

返回值

string

类型 / 合约

BuriedFiveContract

扩展自 Contract。

返回自：self.contract (buried_five)

相关对象类型

Analyzer

方法

.id

用于传输答案的合约 ID。

返回值

返回类型	可能的值
string	"relay_hack"、"xenogenetics"、"corrupted_archive"、"sealed_vault"、"data_tablet"、"terminal_breach"、"drifting_signal"、"cold_boot"、"three_echoes"、"buried_five"、"the_loom"、"crosstalk"、"beat_the_system"、"core_sample"、"lattice"

.name

合约显示名称。

返回值

string

.reward

完成此合约可获得的信用点奖励。

返回值

number

.status

合约状态：'available'（可用）或 'completed'（已完成）。

返回值

返回类型	可能的值
string	"available"、"completed"

.transmission

加密传输内容：由单字符令牌组成的列表。

返回值

list<string>

.analyzer

回收的分析器设备（Analyzer）：将五个令牌组成的分组还原为其展开前的单个令牌。

返回值

Analyzer

.layers

对传输内容应用的五重包装层数，取整数值。

返回值

number

类型 / 合约

Analyzer

返回自：`.analyzer`

方法

`.read(group)`

读取一个恰好包含五个字符串令牌的列表，并返回它们展开前对应的单个令牌。传入的参数不是列表，或列表中包含非字符串元素时，会引发 `TypeError`；长度错误或分组无法识别时，会引发 `ValueError`。

参数

名称	类型	描述
<code>group</code>	<code>list</code>	要折叠的五字符串令牌分组

返回值

`string`

引发的异常

异常	条件
<code>TypeError</code>	<code>Analyzer.read()</code> 要求传入只包含字符串令牌的列表。
<code>ValueError</code>	<code>Analyzer.read()</code> 要求传入恰好五个令牌，并且它们必须构成可识别的对齐分组。

类型 / 合约

ColdBootContract

扩展自 Contract。

返回自：`self.contract (cold_boot)`

方法

.id

用于传输答案的合约 ID。

返回值

返回类型	可能的值
string	"relay_hack"、"xenogenetics"、"corrupted_archive"、"sealed_vault"、"data_tablet"、"terminal_breach"、"drifting_signal"、"cold_boot"、"three_echoes"、"buried_five"、"the_loom"、"crosstalk"、"beat_the_system"、"cng_ore_sample"、"lattice"

.name

合约显示名称。

返回值

string

.reward

完成此合约可获得的信用点奖励。

返回值

number

.status

合约状态：'available'（可用）或 'completed'（已完成）。

返回值

返回类型	可能的值
string	"available"、"completed"

.program

该装置的字节码：由整数构成的列表。运行前请先复制：

```
memory = list(program)
```

返回值

list<number>

类型 / 合约

CoreSampleContract

扩展自 Contract。

返回自：self.contract (core_sample)

相关对象类型

CoreDevice

方法

.id

用于传输答案的合约 ID。

返回值

返回类型	可能的值
string	"relay_hack"、"xenogenetics"、"corrupted_archive"、"sealed_vault"、"data_tablet"、"terminal_breach"、"drifting_signal"、"cold_boot"、"three_echoes"、"buried_five"、"the_loom"、"crosstalk"、"beat_the_system"、"core_sample"、"lattice"

.name

合约显示名称。

返回值

string

`.reward`

完成此合约可获得的信用奖励。

返回值 (RETURNS)

`number`

`.status`

合约状态：'available' 或 'completed'。

返回值 (RETURNS) 可能值 (POSSIBLE VALUES)

`string "available", "completed"`

`.cores`

10 个受损核心，以字节列表的形式给出。传输途中损坏的字节显示为 **None**；请根据构造规则恢复它。

返回值 (RETURNS)

`list`

`.device`

重建设备：将重建后的核心提交给它。参见 **CoreDevice**。

返回值 (RETURNS)

CoreDevice

类型 / 合约 (TYPES / CONTRACTS)

CoreDevice

返回自：`.device`

方法 (METHODS)

`.submit(index, bytes)`

提交一个重建后的核心，用于整数槽位索引（0-9）。容器或元素类型错误会引发 `TypeError`；分数索引、超出范围的索引、错误的列表长度，或超出 0-255 范围的数值会引发 `ValueError`。被拒绝的提交不会锁定该槽位。

参数 (PARAMETERS)

名称 (Name)	类型 (Type)	描述 (Description)
<code>index</code>	<code>number</code>	整数核心槽位，范围 0-9
<code>bytes</code>	<code>list</code>	重建后的核心，以 0-255 范围内的整数型字节列表表示

返回值 (RETURNS) 结果字段 (RESULT FIELDS) 成功载荷 (SUCCESS PAYLOAD)

`ActionResult` `.status`、`.message` `None`

结果 (OUTCOMES)

状态 (Status)	类型 (Kind)	含义 (Meaning)
<code>"locked"</code>	成功 (success)	重建后的核心满足每一条构造规则，并保留所有幸存字节。
<code>"rejected"</code>	拒绝 (rejection)	提交的字节内容格式正确，但违反了一条或多条核心构造规则。

引发异常 (RAISES)

异常 (Exception)	条件 (Condition)
<code>TypeError</code>	<code>CoreDevice.submit()</code> 要求索引为数值型，并且列表只包含数值型字节。
<code>ValueError</code>	<code>CoreDevice.submit()</code> 要求槽位为 0-9 范围内的整数，并且列表长度正确，且所有字节均为 0-255 范围内的整数。

`.recovered()`

当前脚本运行中已锁定的 10 个核心数量。全新运行从 0 开始。

返回值 (RETURNS)

`number`

`.target()`

完成此合约必须恢复的核心数量：**10**。

返回值 (RETURNS)

`number`

`.token()`

要传输的通行码：当 `recovered()` 达到 `target()` 后为非空字符串，否则为空字符串。

返回值 (RETURNS)

`string`

类型 / 合约 (TYPES / CONTRACTS)

CorruptedArchiveContract

继承自 `Contract`

返回自: `self.contract (corrupted_archive)`

相关对象类型 (RELATED OBJECT TYPES)

`Archive`

方法 (METHODS)

`.id`

合约 ID (用于传输答案)。

返回值 (RETURNS) 可能值 (POSSIBLE VALUES)

```
string "relay_hack",  
"xenogenetics",  
"corrupted_archive",  
"sealed_vault",  
"data_tablet",  
"terminal_breach",  
"drifting_signal",  
"cold_boot",  
"three_echoes",  
"buried_five", "the_loom",  
"crosstalk",  
"beat_the_system",  
"core_sample", "lattice"
```

`.name`

合约显示名称。

返回值 (RETURNS)

`string`

`.reward`

完成此合约可获得的信用奖励。

返回值 (RETURNS)

`number`

`.status`

合约状态：'available' 或 'completed'。

返回值 (RETURNS) 可能值 (POSSIBLE VALUES)

`string "available", "completed"`

`.archive`

经过扰乱的数据存档。

返回值 (RETURNS)

`Archive`

类型 / 合约 (TYPES / CONTRACTS)

Archive

返回自：`.archive`

方法 (METHODS)

`.flip(row, col)`

揭示并返回整数网格单元格中的单词。参数类型错误会引发 `TypeError`；分数坐标、非有限坐标或越界坐标会引发 `ValueError`。

参数 (PARAMETERS)

名称 (Name)	类型 (Type)	描述 (Description)
<code>row</code>	<code>number</code>	整数网格行，范围 0 到 <code>rows - 1</code>
<code>col</code>	<code>number</code>	整数网格列，范围 0 到 <code>cols - 1</code>

返回值 (RETURNS)

`string`

引发异常 (RAISES)

异常 (Exception)	条件 (Condition)
<code>TypeError</code>	<code>Archive.flip()</code> 要求行和列坐标为数值型。
<code>ValueError</code>	<code>Archive.flip()</code> 要求坐标为有限的整数，并且位于存档网格内。

`.rows`

网格中的行数。

返回值 (RETURNS)

`number`

`.cols`

网格中的列数。

返回值 (RETURNS)

`number`

类型 / 合约 (TYPES / CONTRACTS)

CrosstalkContract

继承自 `Contract`

返回自：`self.contract (crosstalk)`

方法 (METHODS)

`.id`

合约 ID (用于传输答案)。

返回值 (RETURNS) 可能值 (POSSIBLE VALUES)

```
string "relay_hack",  
"xenogenetics",  
"corrupted_archive",  
"sealed_vault",  
"data_tablet",  
"terminal_breach",  
"drifting_signal",  
"cold_boot",  
"three_echoes",  
"buried_five", "the_loom",  
"crosstalk",  
"beat_the_system",  
"core_sample", "lattice"
```

`.name`

合约显示名称。

返回值 (RETURNS)

```
string
```

`.reward`

完成此合约可获得的信用奖励。

返回值 (RETURNS)

`number`

`.status`

合约状态：'available' 或 'completed'。

返回值 (RETURNS) 可能值 (POSSIBLE VALUES)

`string "available", "completed"`

`.input_x`

第一条截获的信号：由字母组成的字符串，其中散布着 0 和 1。其中一些位是诱饵。

返回值 (RETURNS)

`string`

`.input_y`

第二条截获的信号：形态与 `input_x` 相同。

返回值 (RETURNS)

`string`

`.min_length`

计入统计的位所需的最小回文长度：只有当字母以该位为中心镜像延伸到此跨度时，该位才符合条件。

返回值 (RETURNS)

`number`

类型 / 合约 (TYPES / CONTRACTS)

DataTabletContract

继承自 Contract

相关对象类型 (RELATED OBJECT TYPES)

DataTablet

方法 (METHODS)

.id

合约 ID (用于传输答案)。

返回值 (RETURNS) 可能值 (POSSIBLE VALUES)

```
string "relay_hack",  
"xenogenetics",  
"corrupted_archive",  
"sealed_vault",  
"data_tablet",  
"terminal_breach",  
"drifting_signal",  
"cold_boot",  
"three_echoes",  
"buried_five", "the_loom",  
"crosstalk",  
"beat_the_system",  
"core_sample", "lattice"
```

.name

合约显示名称。

返回值 (RETURNS)

```
string
```


`.reward`

完成此合约可获得的信用奖励。

返回值 (RETURNS)

`number`

`.status`

合约状态：'available' 或 'completed'。

返回值 (RETURNS) 可能值 (POSSIBLE VALUES)

`string "available", "completed"`

`.tablet`

数据平板扫描器。

返回值 (RETURNS)

`DataTable`

类型 / 合约 (TYPES / CONTRACTS)

DataTablet

返回自：`.tablet`

相关对象类型 (RELATED OBJECT TYPES)

`ProbeResult`

方法 (METHODS)

`.probe(row, col)`

探测一个整数单元格，并返回一个带有 `.char` 和整数 `.distance` 的 `ProbeResult`。参数类型错误会引发 `TypeError`；分数坐标或越界坐标会引发 `ValueError`。

参数 (PARAMETERS)

名称 (Name)	类型 (Type)	描述 (Description)
<code>row</code>	<code>number</code>	整数网格行，范围 <code>0</code> 到 <code>rows - 1</code>
<code>col</code>	<code>number</code>	整数网格列，范围 <code>0</code> 到 <code>cols - 1</code>

返回值 (RETURNS)

`ProbeResult`

引发异常 (RAISES)

异常 (Exception)	条件 (Condition)
<code>TypeError</code>	合约探测操作收到的参数或元素类型错误。
<code>ValueError</code>	合约探测操作收到的值、形状或坐标超出其文档规定的范围。

`.rows`

网格中的行数。

返回值 (RETURNS)

`number`

`.cols`

网格中的列数。

返回值 (RETURNS)

`number`

类型 / 合约 (TYPES / CONTRACTS)

ProbeResult

返回自：`tablet.probe()`

方法 (METHODS)

`.char`

此单元格中的字符。

返回值 (RETURNS)

`string`

`.distance`

到最近消息单元格的整数曼哈顿距离 (Manhattan distance, 按步数计)。0 表示此单元格就是消息单元格。

返回值 (RETURNS)

`number`

类型 / 合约 (TYPES / CONTRACTS)

DriftingSignalContract

继承自 Contract

返回自: `self.contract (drifting_signal)`

相关对象类型 (RELATED OBJECT TYPES)

SlabDevice

方法 (METHODS)

.id

合约 ID (用于传输答案)。

返回值 (RETURNS) 可能值 (POSSIBLE VALUES)

`string "relay_hack"、
"xenogenetics"、
"corrupted_archive"、
"sealed_vault"、
"data_tablet"、
"terminal_breach"、
"drifting_signal"、
"cold_boot"、
"three_echoes"、
"buried_five"、"the_loom"、
"crosstalk"、
"beat_the_system"、
"core_sample"、"lattice"`

.name

合约显示名称。

返回值 (RETURNS)

`string`

`.reward`

完成此合约可获得的信用奖励。

返回值 (RETURNS)

`number`

`.status`

合约状态：'available' 或 'completed'。

返回值 (RETURNS) 可能值 (POSSIBLE VALUES)

`string "available", "completed"`

`.device`

恢复出的石板装置。

返回值 (RETURNS)

`SlabDevice`

类型 / 合约 (TYPES / CONTRACTS)

SlabDevice

返回自：`.device`

方法 (METHODS)

`.slabs`

字母石板的当前状态。保留大写字母和空格。

返回值 (RETURNS)

`string`

类型 / 合约 (TYPES / CONTRACTS)

LatticeContract

继承自 Contract

返回自: `self.contract (lattice)`

相关对象类型 (RELATED OBJECT TYPES)

LatticeGrid

方法 (METHODS)

.id

合约 ID (用于传输答案)。

返回值 (RETURNS) 可能值 (POSSIBLE VALUES)

`string "relay_hack"、
"xenogenetics"、
"corrupted_archive"、
"sealed_vault"、
"data_tablet"、
"terminal_breach"、
"drifting_signal"、
"cold_boot"、
"three_echoes"、
"buried_five"、"the_loom"、
"crosstalk"、
"beat_the_system"、
"core_sample"、"lattice"`

.name

合约显示名称。

返回值 (RETURNS)

`string`

`.reward`

完成此合约可获得的信用奖励。

返回值 (RETURNS)

`number`

`.status`

合约状态：'available' 或 'completed'。

返回值 (RETURNS) 可能值 (POSSIBLE VALUES)

`string "available"、"completed"`

`.grid`

外星深度扫描晶格。只能探测已确认安全的单元格，以绘制易变节点的分布图；一旦触发节点，必须重置网格后才能继续探测。参见 `LatticeGrid`。

返回值 (RETURNS)

`LatticeGrid`

类型 / 合约 (TYPES / CONTRACTS)

LatticeGrid

返回自：`.grid`

方法 (METHODS)

`.width()`

以单元格计的网格宽度 (32)。

返回值 (RETURNS)

number

`.height()`

以单元格计的网格高度 (32)。

返回值 (RETURNS)

number

`.start()`

一个保证安全的立足单元格，以 `[x, y]` 形式返回。先探测它以获取读数并开始推导。

返回值 (RETURNS)

`list<number>`

`.reset()`

清除已触发的故障，使探测可以在同一脚本运行中继续。隐藏棋盘和保证安全的起始单元格不会改变。

返回值 (RETURNS) 结果字段 (RESULT FIELDS) 成功载荷 (SUCCESS PAYLOAD)

`ActionResult` `.status`、`.message` `None`

结果 (OUTCOMES)

状态 (Status)	类型 (Kind)	含义 (Meaning)
"ok"	成功 (success)	晶格故障已清除。隐藏棋盘和起始单元格保持不变。

`.probe(x, y)`

探测一个已确认安全的整数坐标单元格。读数是相邻易变节点的整数计数，范围为 0-8。触发节点会使晶格进入故障状态，直到调用 `reset()`。参数类型错误会引发 `TypeError`；分数坐标、非有限坐标或越界坐标会引发 `ValueError`，且不会触发完整晶格。

参数 (PARAMETERS)

名称 (Name)	类型 (Type)	描述 (Description)
x	number	整数列，范围 0-31
y	number	整数行，范围 0-31

返回值 (RETURNS) 结果字段 (RESULT FIELDS) 成功载荷 (SUCCESS PAYLOAD)

`LatticeProbeResult` `.status`、`.message` `.reading`

结果 (OUTCOMES)

状态 (Status)	类型 (Kind)	含义 (Meaning)
"ok"	成功 (success)	安全单元格中相邻的易变节点数量： <code>.reading</code> 。
"node_tripped"	拒绝 (rejection)	被探测的单元格包含易变节点，晶格已进入故障状态。
"lattice_tripped"	拒绝 (rejection)	先前对易变节点的探测已使晶格进入故障状态。

引发异常 (RAISES)

异常 (Exception)	条件 (Condition)
<code>TypeError</code>	<code>LatticeGrid.probe()</code> 要求 x 和 y 坐标为数值型。
<code>ValueError</code>	<code>LatticeGrid.probe()</code> 要求坐标为有限的整数，并且位于 32×32 网格内。

类型 / 合约 (TYPES / CONTRACTS)

RelayHackContract

继承自 Contract

返回自: `self.contract (relay_hack)`

相关对象类型 (RELATED OBJECT TYPES)

RelayLock

方法 (METHODS)

.id

合约 ID (用于传输答案)。

返回值 (RETURNS) 可能值 (POSSIBLE VALUES)

`string "relay_hack"、
"xenogenetics"、
"corrupted_archive"、
"sealed_vault"、
"data_tablet"、
"terminal_breach"、
"drifting_signal"、
"cold_boot"、
"three_echoes"、
"buried_five"、"the_loom"、
"crosstalk"、
"beat_the_system"、
"core_sample"、"lattice"`

.name

合约显示名称。

返回值 (RETURNS)

`string`

`.reward`

完成此合约可获得的信用奖励。

返回值 (RETURNS)

`number`

`.status`

合约状态：'available' 或 'completed'。

返回值 (RETURNS) 可能值 (POSSIBLE VALUES)

`string "available", "completed"`

`.lock`

要破解的中继锁。

返回值 (RETURNS)

`RelayLock`

类型 / 合约 (TYPES / CONTRACTS)

RelayLock

返回自：`.lock`

方法 (METHODS)

`.intercept(code)`

测试一个由恰好 6 个、范围为 0-99 的整数值组成的列表，并为每个转轮返回一个 `True/False` 值。参数类型错误会引发 `TypeError`；列表长度错误、值不是有限数或不是整数，以及值超出范围会引发 `ValueError`。

参数 (PARAMETERS)

名称 (Name)	类型 (Type)	描述 (Description)
<code>code</code>	<code>list</code>	候选列表，包含 6 个整数，每个值均为 0-99

返回值 (RETURNS)

`List[boolean]`

引发异常 (RAISES)

异常 (Exception)	条件 (Condition)
<code>TypeError</code>	合约探针收到的参数或元素类型错误。
<code>ValueError</code>	合约探针收到的值、形状或坐标超出文档规定的域。

`.tumblers`

转轮数量 (6)。

返回值 (RETURNS)

`number`

`.range`

每个转轮的范围 (`100 = 0-99`)。

返回值 (RETURNS)

`number`

类型 / 合约 (TYPES / CONTRACTS)

SealedVaultContract

继承自 `Contract`

返回自：`self.contract (sealed_vault)`

相关对象类型 (RELATED OBJECT TYPES)

`Vault`

方法 (METHODS)

`.id`

合约 ID (用于传输答案)。

返回值 (RETURNS) 可能值 (POSSIBLE VALUES)

`string "relay_hack"、`
`"xenogenetics"、`
`"corrupted_archive"、`
`"sealed_vault"、`
`"data_tablet"、`
`"terminal_breach"、`
`"drifting_signal"、`
`"cold_boot"、`
`"three_echoes"、`
`"buried_five"、 "the_loom"、`
`"crosstalk"、`
`"beat_the_system"、`
`"core_sample"、 "lattice"`

`.name`

合约显示名称。

返回值 (RETURNS)

`string`

`.reward`

完成此合约可获得的信用奖励。

返回值 (RETURNS)

`number`

`.status`

合约状态：'available' 或 'completed'。

返回值 (RETURNS) 可能值 (POSSIBLE VALUES)

`string "available", "completed"`

`.vault`

密封的金库通道系统。每次合约脚本运行开始时，其位置都会重置为 (0, 0)；迷宫布局保持不变。

返回值 (RETURNS)

`Vault`

类型 / 合约 (TYPES / CONTRACTS)

Vault

返回自：`.vault`

相关对象类型 (RELATED OBJECT TYPES)

`VaultPosition`

方法 (METHODS)

`.move(direction)`

沿指定方向移动一个单元格：`"north"`、`"south"`、`"east"` 或 `"west"`。非字符串方向会引发 `TypeError`；未知方向会在不移动的情况下引发 `ValueError`。

参数 (PARAMETERS)

名称 (Name)	类型 (Type)	描述 (Description)
<code>direction</code>	<code>string</code>	要移动的方向。

返回值 (RETURNS) 结果字段 (RESULT FIELDS) 成功载荷 (SUCCESS PAYLOAD)

`ActionResult` `.status`、`.message` `None`

结果 (OUTCOMES)

状态 (Status)	类型 (Kind)	含义 (Meaning)
<code>"path"</code>	成功 (success)	移动进入普通路径单元格。
<code>"wall"</code>	拒绝 (rejection)	墙壁阻挡了移动，位置未改变。
<code>"exit"</code>	成功 (success)	移动进入出口单元格。

引发异常 (RAISES)

异常 (Exception)	条件 (Condition)
<code>TypeError</code>	<code>Vault.move()</code> 要求方向为字符串。
<code>ValueError</code>	<code>Vault.move()</code> 只接受 <code>north</code> 、 <code>south</code> 、 <code>east</code> 或 <code>west</code> 。

`.position`

当前单元格的 `VaultPosition` 快照，包含 `.row` 和 `.col`。需要记住旧单元格时请将其保存；调用 `move()` 后再次读取 `vault.position` 可获得新单元格。

返回值 (RETURNS)

`VaultPosition`

`.escape()`

从出口单元格打开金库。

返回值 (RETURNS) 结果字段 (RESULT FIELDS) 成功载荷 (SUCCESS PAYLOAD)

`VaultEscapeResult.status`、`.message.key`

结果 (OUTCOMES)

状态 (Status)	类型 (Kind)	含义 (Meaning)
"ok"	成功 (success)	金库已打开并释放其密钥。
"not_at_exit"	拒绝 (rejection)	只有在出口单元格才能打开金库。

`.size`

方形迷宫网格的边长。迷宫是 `size × size` 个单元格；起点为 `(0, 0)`，出口为 `(size - 1, size - 1)`。

返回值 (RETURNS)

`number`

类型 / 合约 (TYPES / CONTRACTS)

VaultPosition

返回自: `vault.position`

方法 (METHODS)

`.row`

行坐标。

返回值 (RETURNS)

number

`.col`

列坐标。

返回值 (RETURNS)

number

`.__iter__()`

按先 `.row`、后 `.col` 的顺序迭代，因此可以解包此位置，或将其传递给 `list()`。

返回值 (RETURNS)

iterator<number>

类型 / 合约 (TYPES / CONTRACTS)

TerminalBreachContract

继承自 `Contract`

返回自：`self.contract (terminal_breach)`

相关对象类型 (RELATED OBJECT TYPES)

`AlienTerminal`

方法 (METHODS)

`.id`

合约 ID (用于传输答案)。

返回值 (RETURNS) 可能值 (POSSIBLE VALUES)

`string "relay_hack"、`
`"xenogenetics"、`
`"corrupted_archive"、`
`"sealed_vault"、`
`"data_tablet"、`
`"terminal_breach"、`
`"drifting_signal"、`
`"cold_boot"、`
`"three_echoes"、`
`"buried_five"、 "the_loom"、`
`"crosstalk"、`
`"beat_the_system"、`
`"core_sample"、 "lattice"`

`.name`

合约显示名称。

返回值 (RETURNS)

`string`

`.reward`

完成此合约可获得的信用奖励。

返回值 (RETURNS)

`number`

`.status`

合约状态：'available' 或 'completed'。

返回值 (RETURNS) 可能值 (POSSIBLE VALUES)

`string "available", "completed"`

`.terminal`

外星安全终端。

返回值 (RETURNS)

`AlienTerminal`

类型 / 合约 (TYPES / CONTRACTS)

AlienTerminal

返回自：`.terminal`

相关对象类型 (RELATED OBJECT TYPES)

`GuessResult`

方法 (METHODS)

`.guess(digits)`

测试一个由恰好 15 个、范围为 1-5 的整数数字组成的列表，并返回 `GuessResult`。先移除位置完全匹配的数字；随后，`.misplaced` 会统计剩余部分中共享的出现次数，且不会重复计数。参数或元素类型错误会引发 `TypeError`；长度错误、存在小数值，或数字超出范围会引发 `ValueError`。

参数 (PARAMETERS)

名称 (Name)	类型 (Type)	描述 (Description)
<code>digits</code>	<code>list</code>	候选列表，包含 15 个整数型数字，每个值均为 1-5

返回值 (RETURNS)

`GuessResult`

引发异常 (RAISES)

异常 (Exception)	条件 (Condition)
<code>TypeError</code>	合约探针收到的参数或元素类型错误。
<code>ValueError</code>	合约探针收到的值、形状或坐标超出文档规定的域。

`.length`

整数型代码长度 (15)。

返回值 (RETURNS)

`number`

类型 / 合约 (TYPES / CONTRACTS)

GuessResult

返回自 : `terminal.guess()`

方法 (METHODS)

`.correct`

处于正确位置的数字数量。

返回值 (RETURNS)

number

`.misplaced`

处于错误位置但数值正确的数字数量。

返回值 (RETURNS)

number

类型 / 合约 (TYPES / CONTRACTS)

TheLoomContract

继承自 `Contract`

返回自：`self.contract (the_loom)`

相关对象类型 (RELATED OBJECT TYPES)

`Loom`

方法 (METHODS)

`.id`

合约 ID (用于传输答案)。

返回值 (RETURNS) 可能值 (POSSIBLE VALUES)

`string "relay_hack"、`
`"xenogenetics"、`
`"corrupted_archive"、`
`"sealed_vault"、`
`"data_tablet"、`
`"terminal_breach"、`
`"drifting_signal"、`
`"cold_boot"、`
`"three_echoes"、`
`"buried_five"、 "the_loom"、`
`"crosstalk"、`
`"beat_the_system"、`
`"core_sample"、 "lattice"`

`.name`

合约显示名称。

返回值 (RETURNS)

`string`

`.reward`

完成此合约可获得的信用奖励。

返回值 (RETURNS)

`number`

`.status`

合约状态：'available' 或 'completed'。

返回值 (RETURNS) 可能值 (POSSIBLE VALUES)

`string "available", "completed"`

`.loom`

已回收的外星织机，即你的探针工具。调用 `loom.weave(a, b)` 可了解它如何将两个字符串编织成一个字符串。

返回值 (RETURNS)

`Loom`

`.record`

由两个长度相等、各含 21 个字符的线程生成的 42 字符编织记录。反转织机规则以解织；其中一条线程就是消息。

返回值 (RETURNS)

`string`

类型 / 合约 (TYPES / CONTRACTS)

Loom

返回自：`.loom`

方法 (METHODS)

`.weave(a, b)`

将两个字符串编织成一个字符串并返回。每个字符都是一个令牌。此操作是确定性的：相同输入始终以相同方式编织，因此可以自由探测。非字符串参数会引发 `TypeError`；任一输入超过 30 个字符会引发 `ValueError`。织机只负责正向编织；逆向过程需自行构建。

参数 (PARAMETERS)

名称 (Name)	类型 (Type)	描述 (Description)
<code>a</code>	<code>string</code>	第一个输入字符串
<code>b</code>	<code>string</code>	第二个输入字符串

返回值 (RETURNS)

`string`

引发异常 (RAISES)

异常 (Exception)	条件 (Condition)
<code>TypeError</code>	<code>Loom.weave()</code> 要求两个参数均为字符串。
<code>ValueError</code>	<code>Loom.weave()</code> 接受的每个输入最多为 30 个字符。

类型 / 合约 (TYPES / CONTRACTS)

ThreeEchoesContract

继承自 `Contract`

返回自：`self.contract (three_echoes)`

相关对象类型 (RELATED OBJECT TYPES)

`ThreeEchoesBroadcast`

方法 (METHODS)

`.id`

合约 ID (用于传输答案)。

返回值 (RETURNS) 可能值 (POSSIBLE VALUES)

```
string "relay_hack"、  
"xenogenetics"、  
"corrupted_archive"、  
"sealed_vault"、  
"data_tablet"、  
"terminal_breach"、  
"drifting_signal"、  
"cold_boot"、  
"three_echoes"、  
"buried_five"、"the_loom"、  
"crosstalk"、  
"beat_the_system"、  
"core_sample"、"lattice"
```

`.name`

合约显示名称。

返回值 (RETURNS)

```
string
```

`.reward`

完成此合约可获得的信用奖励。

返回值 (RETURNS)

`number`

`.status`

合约状态：'available' 或 'completed'。

返回值 (RETURNS) 可能值 (POSSIBLE VALUES)

`string "available", "completed"`

`.broadcast`

截获的广播：三个频率片段。

返回值 (RETURNS)

`ThreeEchoesBroadcast`

类型 / 合约 (TYPES / CONTRACTS)

ThreeEchoesBroadcast

返回自：`.broadcast`

方法 (METHODS)

`.freq_a`

有序片段，包含原始信号中的第 1、4、7 个字符，以及之后每隔三个位置出现的字符。

返回值 (RETURNS)

`string`

`.freq_b`

有序片段，包含原始信号中的第 2、5、8 个字符，以及之后每隔三个位置出现的字符。

返回值 (RETURNS)

`string`

`.freq_c`

有序片段，包含原始信号中的第 3、6、9 个字符，以及之后每隔三个位置出现的字符。

返回值 (RETURNS)

`string`

类型 / 合约 (TYPES / CONTRACTS)

XenogeneticsContract

继承自 `Contract`

返回自：`self.contract (xenogenetics)`

方法 (METHODS)

.id

合约 ID (用于传输答案)。

返回值 (RETURNS) 可能值 (POSSIBLE VALUES)

```
string "relay_hack",  
"xenogenetics",  
"corrupted_archive",  
"sealed_vault",  
"data_tablet",  
"terminal_breach",  
"drifting_signal",  
"cold_boot",  
"three_echoes",  
"buried_five", "the_loom",  
"crosstalk",  
"beat_the_system",  
"core_sample", "lattice"
```

.name

合约显示名称。

返回值 (RETURNS)

```
string
```

`.reward`

完成此合约可获得的信用奖励。

返回值 (RETURNS)

`number`

`.status`

合约状态：'available' 或 'completed'。

返回值 (RETURNS) 可能值 (POSSIBLE VALUES)

`string "available", "completed"`

`.earth_ref`

50 条已知的地球基因序列（字符串列表）。

返回值 (RETURNS)

`list<string>`

`.samples`

1000 个采集的 DNA 样本（字符串列表）。

返回值 (RETURNS)

`list<string>`

ContractScript

返回自：`self`（在合约脚本中）

方法（METHODS）

`.name`

脚本名称。

返回值（RETURNS）

`string`

`.contract`

包含 ID、名称、奖励以及合约专属 API 的合约对象。

返回值（RETURNS）

`Contract`

LatticeProbeResult

返回自：LatticeGrid.probe()

方法 (METHODS)

.status

"ok"、"node_tripped" 或 "lattice_tripped"。

返回值 (RETURNS) | 可能值 (POSSIBLE VALUES)

string | "ok"、"node_tripped"、"lattice_tripped"

.message

面向玩家的探测结果说明。

返回值 (RETURNS)

string

.reading

当 `.status == "ok"` 时，为 0-8 范围内的相邻节点整数数量；否则为 `None`。

返回值 (RETURNS)

Optional[number]

VaultEscapeResult

返回自：`Vault.escape()`

方法 (METHODS)

.status

"ok" 或 "not_at_exit"。

返回值 (RETURNS) | 可能值 (POSSIBLE VALUES)

string | "ok"、"not_at_exit"

.message

面向玩家的逃脱结果说明。

返回值 (RETURNS)

string

.key

当 `.status == "ok"` 时为保险库密钥字符串；否则为 `None`。

返回值 (RETURNS)

Optional[string]

类型 / 内置类型 (TYPES / BUILT-IN TYPES)

`dict`

返回自：字典字面量 `{k: v}` · `dict()` · 返回字典的方法

方法 (METHODS)

`.length`

键值对数量。与 `len(d)` 相同。属性：不带括号。

返回值 (RETURNS)

`number`

`.keys()`

按插入顺序返回所有键的列表。

返回值 (RETURNS)

`list`

`.values()`

按插入顺序返回所有值的列表。

返回值 (RETURNS)

`list`

`.items()`

按插入顺序返回所有 `(key, value)` 元组的列表。常见模式：`for k, v in d.items(): ...`。

返回值 (RETURNS)

`list`

`.has(key, /)`

如果字典中存在 `key`，则为 `True`。等价于 Python 的 `key in d`。在下标访问前使用此方法进行安全的成员资格测试。

参数 (PARAMETERS)

名称 | 类型 | 描述

`key` | `any` | 要检查的键

返回值 (RETURNS)

`boolean`

`.get(key, default=None, /)`

返回 `key` 对应的值；如果键缺失，则返回 `default`（省略时为 `None`）。对于缺失的可哈希键，绝不会抛出异常；不可哈希的键仍会抛出 `TypeError`，与 Python 一致。

参数 (PARAMETERS)

名称 | 类型 | 描述

`key` | `any` | 要查找的键

`default` | `any` | 缺失时要返回的值

返回值 (RETURNS)

`any`

`.pop(key, default?, /)`

移除 `key` 并返回其值。除非提供 `default`，否则在键不存在时抛出异常；提供 `default` 时，缺失键路径会返回 `default`，且字典保持不变。与 Python 的 `dict.pop(k, default)` 一致。

参数 (PARAMETERS)

名称 | 类型 | 描述

`key` | `any` | 要移除的键

`default` | `any` | 键缺失时返回的值

返回值 (RETURNS)

`any`

`.popitem()`

移除并返回最后插入的 (`key`, `value`)

对。字典为空时抛出异常。插入顺序是确定的，但只有在确实打算把字典当作栈使用时才应使用此方法。

返回值 (RETURNS)

tuple

`.setdefault(key, default=None, /)`

如果 `d[key]` 存在，则返回它；否则设置 `d[key] = default` 并返回 `default`。适合构建分组集合。

参数 (PARAMETERS)

名称 | 类型 | 描述

`key` | any | 键

`default` | any | 缺失时要设置的值

返回值 (RETURNS)

any

`.update(other?, /, **kwargs)`

将条目合并到此字典中，并覆盖匹配的键。接受另一个字典、(`key`, `value`)

对可迭代对象、关键字参数，或三者组合：`d.update(other, a=1, b=2)`。就地修改。

参数 (PARAMETERS)

名称 | 类型 | 描述

`other` | any | 字典或由 (`key`, `value`) 对组成的可迭代对象

返回值 (RETURNS)

None

`.copy()`

返回字典的浅拷贝。顶层键和值会复制到一个新字典中；嵌套的可变值（列表、字典）与原对象共享。

返回值 (RETURNS)

`dict`

`.clear()`

移除所有条目。

返回值 (RETURNS)

`None`

类型 / 内置类型 (TYPES / BUILT-IN TYPES)

generator

返回自：调用包含 `yield` 的函数 · 生成器表达式

方法 (METHODS)

`.send(value)`

恢复生成器，并使 `value` 成为暂停状态下 `yield` 表达式的结果。返回下一项产出值。在第一次 `yield` 之前发送非 `None` 值会抛出 `TypeError`；生成完毕时抛出 `StopIteration`。

参数 (PARAMETERS)

名称 | 类型 | 描述

`value` | `any` | 传递给暂停状态下 `yield` 的值

返回值 (RETURNS)

`any`

`.throw(exception)`

在生成器暂停的 `yield` 处抛出异常。如果生成器捕获异常并再次产出，则返回下一项；否则异常继续传播。

参数 (PARAMETERS)

名称 | 类型 | 描述

`exception` | `any` | 异常实例或异常类

返回值 (RETURNS)

`any`

`.close()`

在生成器暂停的 `yield` 处抛出 `GeneratorExit`，从而停止生成器。返回前会执行 `finally` 清理操作。关闭过程中如果生成器再次产出，则抛出 `RuntimeError`。

返回值 (RETURNS)

`None`

`.__iter__()`

返回此生成器。生成器是一次性迭代器。

返回值 (RETURNS)

`generator`

`.__next__()`

以 `None` 恢复生成器，并返回其下一项产出值。生成完毕时抛出 `StopIteration`。

返回值 (RETURNS)

`any`

`.__name__`

创建此生成器的生成器函数名称。

返回值 (RETURNS)

`string`

`.__qualname__`

创建此生成器的生成器函数的限定名称。

返回值 (RETURNS)

`string`

类型 / 内置类型 (TYPES / BUILT-IN TYPES)

list

返回自：列表字面量 `[1, 2, 3]` · `list(iterable)` · 返回列表的方法

方法 (METHODS)

`.length`

列表中的项目数量。与 `len(lst)` 相同。属性：不带括号。

返回值 (RETURNS)

number

`.append(item, /)`

将 `item` 添加到列表末尾。返回 `None`。就地修改。如果列表会超过解释器的最大长度上限，则抛出异常。

参数 (PARAMETERS)

名称 | 类型 | 描述

`item` | `any` | 要追加的值

返回值 (RETURNS)

`None`

`.pop(index=-1, /)`

移除并返回一个元素。默认移除最后一个元素 (`pop()`)。传入整数索引可移除指定项目：`lst.pop(0)`

移除第一个，`lst.pop(-1)`

移除最后一个。负索引从末尾计数。空列表或超出范围的索引会抛出异常。非整数索引也会抛出异常。

参数 (PARAMETERS)

名称 | 类型 | 描述

`index` | `number` | 要移除的位置（默认值：`-1`，即最后一项）

返回值 (RETURNS)

`any`

`.remove(item, /)`

按值移除 `item` 的第一次出现。如果未找到则抛出异常。如果需要先检查是否存在，请先使用 `.index(item)`。

参数 (PARAMETERS)

名称 | 类型 | 描述

`item` | `any` | 要移除的值

返回值 (RETURNS)

`None`

`.insert(index, item, /)`

在位置 `index` 插入 `item`，并将后续项目向右移动。`insert(0, x)` 会将 `x` 放在最前面。超出范围的索引会被限制到两端（不会报错）。如果列表会超过最大长度上限，则抛出异常。

参数 (PARAMETERS)

名称 | 类型 | 描述

`index` | `number` | 位置

`item` | `any` | 值

返回值 (RETURNS)

`None`

`.index(item, start=0, end=None, /)`

返回 `item` 第一次出现的位置。可选的 `start` 和 `end` 将搜索限制在一个切片内（遵循 Python 风格：负索引从末尾计数，并限制在边界内）。如果范围内未找到，则抛出异常。

参数 (PARAMETERS)

名称 | 类型 | 描述

`item` | `any` | 要查找的值

`start` | `number` | 起始索引（默认值 0）

`end` | `number` | 结束索引（不包含，默认值为长度）

返回值 (RETURNS)

`number`

`.count(item, /)`

统计列表中 `item` 的出现次数。相等性按值判断（数字、字符串、布尔值会进行深度比较）。

参数 (PARAMETERS)

名称 | 类型 | 描述

`item` | `any` | 要计数的值

返回值 (RETURNS)

`number`

`.sort(*, key=None, reverse=False)`

使用 `<` 运算符在相互可比较的键之间就地排序列表。`key=None`（默认值）时，每个值就是自身的键。数字键和布尔键可在同一族内比较，字符串按字典序比较；用户对象可以定义 `<` 所需的精确富比较槽，不支持的组合会抛出异常。每个项目只调用一次 `key=`，且该函数必须是纯函数：不能暂停脚本或修改游戏状态。`reverse` 会按真值测试，有限输入会在解释器的集合限制内急切处理。

返回值 (RETURNS)

`None`

`.reverse()`

就地反转列表。返回 `None`。

返回值 (RETURNS)

`None`

`.copy()`

返回列表的浅拷贝。修改拷贝不会影响原列表；嵌套的可变项目仍与原列表共享。

返回值 (RETURNS)

`list`

`.extend(iterable, /)`

将另一个有限可迭代对象中的每一项追加到此列表末尾。就地修改。如果结果会超过最大长度上限，则抛出异常。

参数 (PARAMETERS)

名称 | 类型 | 描述

`iterable` | `any` | 要追加项目的有限可迭代对象

返回值 (RETURNS)

`None`

`.clear()`

移除所有项目。返回 `None`。等价于 `lst[:] = []`。

返回值 (RETURNS)

`None`

类型 / 内置类型 (TYPES / BUILT-IN TYPES)

set

返回自：集合字面量 `{1, 2}` · `set(iterable)` · 集合代数运算符

方法 (METHODS)

`.length`

唯一成员的数量。与 `len(s)` 相同。属性：不带括号。

返回值 (RETURNS)

number

`.add(item, /)`

将 `item` 添加到集合中。如果已经存在则不产生影响。`item` 可以是任何可哈希值，包括由可哈希值组成的元组以及正确实现哈希的用户类实例。

参数 (PARAMETERS)

名称 | 类型 | 描述

`item` | any | 要添加的值

返回值 (RETURNS)

None

`.remove(item, /)`

移除 `item`。如果不存在则抛出异常。安全形式请使用 `.discard()`。

参数 (PARAMETERS)

名称 | 类型 | 描述

`item` | any | 要移除的值

返回值 (RETURNS)

None

.discard(item, /)

如果 **item** 存在则移除；不存在时不报错。

参数 (PARAMETERS)

名称 | 类型 | 描述

item | **any** | 要移除的值

返回值 (RETURNS)

None

.pop()

移除并返回任意一个元素。顺序是确定的（按插入顺序），但脚本不应依赖该顺序。集合为空时抛出异常。

返回值 (RETURNS)

any

.clear()

移除所有成员。

返回值 (RETURNS)

None

.copy()

返回浅拷贝。

返回值 (RETURNS)

set

`.has(item, /)`

如果集合中存在 `item`, 则为 `True`。等价于 Python 的 `item in s`。

参数 (PARAMETERS)

名称 | 类型 | 描述

`item` | `any` | 要检查的值

返回值 (RETURNS)

`boolean`

`.union(*others)`

返回一个新集合, 其中包含此集合以及 `others` 中每个有限可迭代对象的成员。不带参数时返回副本。

返回值 (RETURNS)

`set`

`.intersection(*others)`

返回一个新集合, 其中包含此集合与 `others` 中每个有限可迭代对象共有的成员。不带参数时返回副本。

返回值 (RETURNS)

`set`

`.difference(*others)`

返回一个新集合, 移除 `others` 中任何有限可迭代对象所包含的成员。不带参数时返回副本。

返回值 (RETURNS)

`set`

`.symmetric_difference(other, /)`

返回一个新集合，其中包含恰好属于此集合或有限可迭代对象 `other` 之一的成员。等价于 $a \mathop{\Delta} b$ 。

参数 (PARAMETERS)

名称 | 类型 | 描述

`other` | `any` | 有限可迭代对象

返回值 (RETURNS)

`set`

`.update(*others)`

添加 `others` 中每个有限可迭代对象的所有成员。就地修改；不带参数时不执行任何操作。

返回值 (RETURNS)

`None`

`.intersection_update(*others)`

仅保留与 `others` 中每个有限可迭代对象共有的成员。就地修改；不带参数时不执行任何操作。

返回值 (RETURNS)

`None`

`.difference_update(*others)`

移除 `others` 中任何有限可迭代对象所包含的成员。就地修改；不带参数时不执行任何操作。

返回值 (RETURNS)

`None`

`.symmetric_difference_update(other, /)`

将此集合替换为恰好属于此集合或有限可迭代对象 `other` 之一的成员。

参数 (PARAMETERS)

名称 | 类型 | 描述

`other` | `any` | 有限可迭代对象

返回值 (RETURNS)

`None`

`.issubset(other, /)`

如果此集合的每个成员也存在于有限可迭代对象 `other` 中，则为 `True`。对于集合而言等价于 `a <= b`。

参数 (PARAMETERS)

名称 | 类型 | 描述

`other` | `any` | 有限可迭代对象

返回值 (RETURNS)

`boolean`

`.issuperset(other, /)`

如果此集合包含有限可迭代对象 `other` 的每个成员，则为 `True`。对于集合而言等价于 `a >= b`。

参数 (PARAMETERS)

名称 | 类型 | 描述

`other` | `any` | 有限可迭代对象

返回值 (RETURNS)

`boolean`

`.isdisjoint(other, /)`

如果此集合与有限可迭代对象 `other` 没有共同成员，则为 `True`。

参数 (PARAMETERS)

名称 | 类型 | 描述

`other` | `any` | 有限可迭代对象

返回值 (RETURNS)

`boolean`

类型 / 内置类型 (TYPES / BUILT-IN TYPES)

`slice`

返回自：`slice(stop) · slice(start, stop, step)`

方法 (METHODS)

`.start`

起始边界；省略时为 `None`。对于 `slice(stop)`，`.start` 为 `None`，而 `.stop` 为传入的参数。

返回值 (RETURNS)

any

`.stop`

结束边界；省略时为 `None`。

返回值 (RETURNS)

any

`.step`

步长边界；省略时为 `None`。序列索引不接受为 `0` 的步长。

返回值 (RETURNS)

any

类型 / 内置类型 (TYPES / BUILT-IN TYPES)

`str`

返回方式：字符串字面量 · `str(value)` · 返回字符串的方法

方法 (METHODS)

`.length`

字符串中的字符数。与 `len(s)` 相同。属性：无需括号。

返回值

`number`

`.upper()`

返回所有字符均为大写形式的副本。

返回值

`string`

`.lower()`

返回所有字符均为小写形式的副本。

返回值

`string`

`.strip(chars=None, /)`

返回去除两端空白（或指定 `chars` 中字符）的副本。显式传入 `None` 与省略 `chars` 相同。`chars` 会被视为要剥离的字符集合，而不是子字符串：`"...hello...".strip(".") → "hello"`。

参数 (PARAMETERS)

名称	类型	描述
<code>chars</code>	<code>string</code>	要剥离的字符，或表示空白的 <code>None</code>

返回值

`string`

`.lstrip(chars=None, /)`

与 `strip()` 类似，但只从左侧去除字符。显式传入 `None` 表示去除空白。

参数 (PARAMETERS)

名称	类型	描述
<code>chars</code>	<code>string</code>	要剥离的字符，或表示空白的 <code>None</code>

返回值

`string`

`.rstrip(chars=None, /)`

与 `strip()` 类似，但只从右侧去除字符。显式传入 `None` 表示去除空白。

参数 (PARAMETERS)

名称	类型	描述
<code>chars</code>	<code>string</code>	要剥离的字符，或表示空白的 <code>None</code>

返回值

`string`

`.split(sep=None, maxsplit=-1)`

将字符串拆分为子字符串。`sep=None` 时，每个 Python 空白字符都会分隔单词，连续空白会合并，并忽略开头和结尾的空白；但如果有限的 `maxsplit` 使未拆分的剩余部分保留空白，则不忽略该部分。分隔符为非空字符串时，会按字面匹配并保留空字段。两个参数都支持关键字形式。

参数 (PARAMETERS)

名称	类型	描述
<code>sep</code>	<code>any</code>	非空字符串分隔符，或表示按空白分隔的 <code>None</code>
<code>maxsplit</code>	<code>number</code>	最大拆分次数 (默认 -1)

返回值

`list<string>`

`.splitlines(keepends=False)`

按 Python 的行边界拆分，包括 `\n`、`\r\n`、`\r`、垂直制表符、换页符、Unicode NEL，以及 Unicode 行/段落分隔符。末尾的边界不会增加额外的空项。传入 `keepends=True` 可保留每个边界。

参数 (PARAMETERS)

名称	类型	描述
<code>keepends</code>	<code>boolean</code>	是否保留行边界字符

返回值

`list<string>`

`.join(iterable, /)`

连接有限可迭代对象中的所有字符串，并在字符串之间插入当前字符串作为分隔符。`",".join(["a", "b", "c"]) → "a,b,c"`。所有元素都必须是字符串。

参数 (PARAMETERS)

名称	类型	描述
<code>iterable</code>	<code>any</code>	由字符串组成的有限可迭代对象

返回值

`list<string>`

`.find(substring, start=0, end=None, /)`

返回 `substring` 出现位置的最小索引；如果未找到则返回 `-1`。可选的 `start` 和 `end` 参数将搜索限制在一个切片内。负索引从末尾开始计数。

参数 (PARAMETERS)

名称	类型	描述
<code>substring</code>	<code>string</code>	要查找的子字符串
<code>start</code>	<code>number</code>	起始索引（默认 0）
<code>end</code>	<code>number</code>	结束索引，不包含该位置（默认长度）

返回值

`number`

`.index(substring, start=0, end=None, /)`

与 `find()` 类似，但当子字符串不存在时会引发错误。如果希望未找到时返回 `-1`，请使用 `find()`。

参数 (PARAMETERS)

名称	类型	描述
<code>substring</code>	<code>string</code>	要查找的子字符串
<code>start</code>	<code>number</code>	起始索引（默认 0）
<code>end</code>	<code>number</code>	结束索引，不包含该位置（默认长度）

返回值

`number`

`.replace(old, new, count=-1, /)`

返回将每个 `old` 替换为 `new` 的副本。可选的 `count` 参数限制替换次数：`s.replace("a", "b", 1)` 只替换第一次出现的位置。

参数 (PARAMETERS)

名称	类型	描述
<code>old</code>	<code>string</code>	要查找的子字符串
<code>new</code>	<code>string</code>	替换字符串
<code>count</code>	<code>number</code>	最大替换次数（默认：全部）

返回值

`string`

`.startswith(prefix, start=0, end=None, /)`

如果字符串以 `prefix` 开头，则返回 `True`。`prefix` 可以是单个字符串，也可以是只包含字符串的元组：`"abc".startswith(("ab", "xy")) → True`。可选的 `start/end` 参数将检查限制在切片内（负索引从末尾开始计数）。

参数 (PARAMETERS)

名称	类型	描述
<code>prefix</code>	<code>string</code>	前缀字符串或字符串元组
<code>start</code>	<code>number</code>	起始索引（默认 0）
<code>end</code>	<code>number</code>	结束索引，不包含该位置（默认长度）

返回值

`boolean`

`.endswith(suffix, start=0, end=None, /)`

如果字符串以 `suffix` 结尾，则返回 `True`。接受单个字符串或只包含字符串的元组。可选的 `start / end` 参数将检查限制在切片内，适合在较大字符串中检查后缀而无需重新构造字符串。

参数 (PARAMETERS)

名称	类型	描述
<code>suffix</code>	<code>string</code>	后缀字符串或字符串元组
<code>start</code>	<code>number</code>	起始索引（默认 0）
<code>end</code>	<code>number</code>	结束索引，不包含该位置（默认长度）

返回值

`boolean`

`.count(substring, start=0, end=None, /)`

统计 `substring` 的不重叠出现次数。可选的 `start` 和 `end` 参数将搜索限制在切片内。空子字符串返回长度加 1。

参数 (PARAMETERS)

名称	类型	描述
<code>substring</code>	<code>string</code>	要统计的子字符串
<code>start</code>	<code>number</code>	起始索引
<code>end</code>	<code>number</code>	结束索引，不包含该位置

返回值

`number`

```
.zfill(width, /)
```

在开头填充零，使字符串至少达到width个字符。开头的+或-符号会被保留。

参数 (PARAMETERS)

名称	类型	描述
width	number	目标宽度

返回值

string

```
.center(width, fill=" ", /)
```

将字符串居中放置在width个字符的范围内，并在两侧填充fill（默认为空格）。fill必须是单个字符。

参数 (PARAMETERS)

名称	类型	描述
width	number	目标宽度
fill	string	填充字符（默认为空格）

返回值

string

```
.ljust(width, fill=" ", /)
```

在width个字符的范围内左对齐，并在右侧用fill填充。

参数 (PARAMETERS)

名称	类型	描述
width	number	目标宽度
fill	string	填充字符（默认为空格）

返回值

string

```
.rjust(width, fill=" ", /)
```

在 `width` 个字符的范围内右对齐，并在左侧用 `fill` 填充。

参数 (PARAMETERS)

名称	类型	描述
<code>width</code>	<code>number</code>	目标宽度
<code>fill</code>	<code>string</code>	填充字符（默认为空格）

返回值

`string`

```
.title()
```

返回标题格式的副本：每一段有大小写的字符都会以其 Unicode 标题大小写映射开头，后续字符则转为小写。

返回值

`string`

```
.capitalize()
```

返回副本：第一个字符转为标题大小写，其余字符转为小写。

返回值

`string`

```
.casefold()
```

应用与 Python 兼容的 Unicode 大小写折叠，以便进行不区分大小写的比较。对 ASCII 而言其功能类似 `.lower()`，但对许多 Unicode 字符的处理更强。

返回值

`string`

`.swapcase()`

交换每个字符的大小写（小写 ↔ 大写）。

返回值

string

`.isdigit()`

当字符串非空且只包含 Unicode 数字字符时返回 `True`，其中包括十进制数字和兼容性数字。

返回值

boolean

`.isalpha()`

当字符串非空且只包含 Unicode 字母时返回 `True`。

返回值

boolean

`.isalnum()`

当字符串非空且只包含 Unicode 字母或数字时返回 `True`。

返回值

boolean

`.isspace()`

当字符串非空且只包含空白字符时返回 `True`。

返回值

boolean

`.islower()`

当字符串至少包含一个有大小写之分的字符，且每个有大小写之分的字符都是小写时返回 `True`。

返回值

`boolean`

`.isupper()`

当字符串至少包含一个有大小写之分的字符，且每个有大小写之分的字符都是大写时返回 `True`。

返回值

`boolean`

`.istitle()`

当字符串至少包含一个有大小写之分的字符，且每段有大小写之分的字符都以大写或 Unicode 标题大小写字符开头、后续仅包含小写字符时返回 `True`。无大小写之分的字符会分隔这些字符段。

返回值

`boolean`

`.rfind(substring, start=0, end=None, /)`

返回 `substring` 出现位置的最大索引；如果未找到则返回 `-1`。这是从右侧执行的 `find()` 对应操作，适合解析最后一个分隔符的位置。

参数 (PARAMETERS)

名称	类型	描述
<code>substring</code>	<code>string</code>	要查找的子字符串
<code>start</code>	<code>number</code>	起始索引（默认 0）
<code>end</code>	<code>number</code>	结束索引，不包含该位置（默认长度）

返回值

`number`

`.rindex(substring, start=0, end=None, /)`

与 `rfind()` 类似，但当子字符串不存在时会引发错误。

参数 (PARAMETERS)

名称	类型	描述
<code>substring</code>	<code>string</code>	要查找的子字符串
<code>start</code>	<code>number</code>	起始索引（默认 0）
<code>end</code>	<code>number</code>	结束索引，不包含该位置（默认长度）

返回值

`number`

`.format(*args, **kwargs)`

用参数替换占位符。自动编号的 `{}` 会消耗下一个位置参数，显式的 `{0}` / `{1}` 引用指定的位置参数，而 `{name}` 会查找关键字参数：`"hi {name}".format(name="world")` → `"hi world"`。对于格式说明符 / 转换 (`{x:>10}`、`{x!r}`)，建议使用 f-string。

返回值

`string`

`.partition(sep, /)`

在非空 `sep` 的第一次出现位置拆分为三部分：（前半部分、`sep`、后半部分）。如果找不到 `sep`，则返回（原字符串、`""`、`""`）。

参数 (PARAMETERS)

名称	类型	描述
<code>sep</code>	<code>string</code>	非空分隔符

返回值

`[string, string, string]`

`.rpartition(sep, /)`

与 `partition()` 类似，但在非空 `sep` 的最后一次出现位置拆分。如果找不到，则返回（`""`、`""`、原字符串）。

参数 (PARAMETERS)

名称	类型	描述
<code>sep</code>	<code>string</code>	非空分隔符

返回值

`[string, string, string]`

`.removeprefix(prefix, /)`

返回去除开头 `prefix` 的副本（仅在前缀存在时去除）。当不确定前缀是否存在时，这比切片更安全。

参数 (PARAMETERS)

名称	类型	描述
<code>prefix</code>	<code>string</code>	要移除的前缀

返回值

`string`

`.removesuffix(suffix, /)`

返回去除末尾 `suffix` 的副本（仅在后缀存在时去除）。

参数 (PARAMETERS)

名称	类型	描述
<code>suffix</code>	<code>string</code>	要移除的后缀

返回值

`string`

类型 / 内置类型 (TYPES / BUILT-IN TYPES)

tuple

返回方式：元组字面量 (1, 2) · enumerate() · 返回元组的方法

方法 (METHODS)

.length

元组中的元素数。与 len(t) 相同。属性：无需括号。

返回值

number

.index(item, start=0, end=None, /)

返回 item 第一次出现的索引。可选的 start 和 end 参数将搜索限制在切片内。如果在该范围内找不到，则引发错误。

参数 (PARAMETERS)

名称	类型	描述
item	any	要查找的值
start	number	起始索引（默认 0）
end	number	结束索引，不包含该位置（默认长度）

返回值

number

.count(item, /)

统计元组中 item 的出现次数。

参数 (PARAMETERS)

名称	类型	描述
item	any	要统计的值

返回值

number

类型 / 生物圈 (TYPES / BIOSPHERE)

AnalyzeResult

返回方式: `bio_lab.analyze()`

方法 (METHODS)

`.status`

状态为 "ok"、"busy"、"input_empty" 或 "invalid_specimen"。

返回值 | 可能的值 (POSSIBLE VALUES)

string | "ok"、"busy"、"input_empty"、"invalid_specimen"

`.message`

面向玩家的分析结果说明。

返回值

string

`.info`

已完成的 `AnalyzeInfo`；如果分析被拒绝，则为 `None`。

返回值

`Optional[AnalyzeInfo]`

类型 / 生物圈 (TYPES / BIOSPHERE)

BioExtractionResult

返回方式：`PortableBioExtractor.extract()`

方法 (METHODS)

.status

状态为 "ok"、"not_mounted"、"scrambled"、"busy"、"not_at_location"、"not_scanned"、"cooling" 或 "no_cargo_space"。

返回值 | 可能的值 (POSSIBLE VALUES)

string | "ok"、"not_mounted"、"scrambled"、"busy"、"not_at_location"、"not_scanned"、"cooling"、"no_cargo_space"

.message

面向玩家的生物提取结果说明。

返回值

string

.extracted

本次提取提交到无人机货舱的吨数；每种拒绝情况均为 0。

返回值

number

类型 / 生物圈 (TYPES / BIOSPHERE)

BioScanResult

返回方式：`PortableBioScanner.scan()`

方法 (METHODS)

.status

状态为 "ok"、"not_mounted"、"scrambled"、"busy" 或 "not_at_location"。

返回值 | 可能的值 (POSSIBLE VALUES)

string | "ok"、"not_mounted"、"scrambled"、"busy"、"not_at_location"

.message

面向玩家的生物扫描结果说明。

返回值

string

.scan

已完成的 `LifeFormScanResult`；如果扫描被拒绝，则为 `None`。

返回值

`Optional[LifeFormScanResult]`

类型 / 生物圈 (TYPES / BIOSPHERE)

Cell

返回方式：`self.cell(sector)/self.cells()`

方法 (METHODS)

.id

区域 ID，例如 "E13"（行字母 A-H + 列号 1-24）。请自行解析它来计算四个正交邻居：网格不会提供 `neighbors()` 辅助方法。

返回值

string

.plant

所种植物的物种 ID，例如 "sunpetal"；如果单元格没有植物，则为 `None`。这是物种 ID，而不是 "seed_sunpetal" 物品 ID。空单元格仍可能包含提供器或未收集的物品，因此也要检查 `.status`。它与 `SeedRecipe.species` 以及 "seed:<species>" 中的物种部分相匹配。

返回值 | 可能的值 (POSSIBLE VALUES)

`Optional[string] | "sunpetal"、"shadeleaf"、"dewmoss"、"lonethorn"、"packfern"、"twinvine"、"spitebud"、"sunspur"、"glowvine"、"crowncape"、"pondmoss"、"saltbloom"、"brinethorn"、"saltmate"、"gerandbloom"`

.growth

作物生长进度为 **0-1**。 **1.0**

表示已可收获。只有在所有要求都满足时才会推进，否则会暂停（绝不会回退）。清除作物会将其重置为 **0**；部分收割器（Harvester）收获会使作物保持成熟状态 **1.0**，并保留剩余的可收获物。

返回值

number

.status

表示区域中有什么，或作物正在进行什么：自然地面扫描前为 **"unknown"**；**"empty"**（可种植的裸地）、**"item"**（未收集的地表物品）、**"base"**（收割器基地）、**"provider"**（田间设备）、**"growing"**（生长中）、**"stalled"**（停滞）或 **"mature"**（成熟）。玩家创建的植物和提供器仍会保持可见。只有 **"empty"** 接受种植或部署。

返回值 | 可能的值（POSSIBLE VALUES）

string | **"unknown"**、**"empty"**、**"item"**、**"base"**、**"growing"**、**"stalled"**、**"mature"**、**"provider"**

.lit

当单元格处于有效的 24 小时收割器照明处理期间，或被通电的生长灯覆盖时为 **True**。这满足光照要求；遮阴植物需要该值为 **False**。

返回值

boolean

.watered

当单元格处于有效的 24 小时收割器浇水处理期间，或被已通电且有供给的喷洒器覆盖时为 **True**。这满足水分要求。当两种来源都不活跃时，植物会暂停。

返回值

boolean

`.salted`

当单元格处于有效的 24 小时收割器盐处理期间，或被已通电且有供给的分配器覆盖时为 `True`。这满足盐分要求。当两种来源都不活跃时，植物会暂停。

返回值

`boolean`

`.manual_light_remaining`

收割器施加的照明处理剩余小时数，范围为 0-24。不包括生长灯覆盖。每次成功调用 `light()` 都会将其重置为 24 小时，并替换任何剩余时间。

返回值

`number`

`.manual_water_remaining`

收割器施加的浇水处理剩余小时数，范围为 0-24。不包括喷洒器覆盖。每次成功调用 `water()` 都会将其重置为 24 小时，并替换任何剩余时间。

返回值

`number`

`.manual_salt_remaining`

收割器施加的盐处理剩余小时数，范围为 0-24。不包括分配器覆盖。每次成功调用 `dispense_salt()` 都会将其重置为 24 小时，并替换任何剩余时间。

返回值

`number`

`.fertilized`

当该单元格仍有任何肥料剂量时为 **True**。未施加剂量或完全耗尽时为 **False**。

返回值

boolean

`.fertilizer_remaining`

肥料效果剩余小时数。每施加一单位会增加 8 小时；没有剩余效果时读数为 0。

返回值

number

`.fertilizer_tier`

当前生效的肥料等级：1、2 或 3。没有剩余肥料时读数为 0。

返回值

number

`.accelerated`

当该单元格仍有任何生长加速剂量时为 **True**。未施加剂量或完全耗尽时为 **False**。

返回值

boolean

`.accelerant_remaining`

生长加速剂效果剩余小时数。每施加一单位会增加 8 小时；没有剩余效果时读数为 0。

返回值

number

`.forage`

该作物目前积累的全部饲料量。只要满足要求，它会随着可见的生长而增加，并包含多样性与田地加成；当 `growth == 1.0` 时冻结。收割机（Harvester）会移动 Inventory 能容纳的最大数量，并将剩余部分留在作物上。作物自动化器（Crop Automator）会移动其输出端能容纳的数量，丢弃剩余部分并清空单元格；如果输出端完全充满，则作业会被阻塞，但不会改变作物。饲料用于野生动物（Wildlife）生产和植物地形改造器（Plant Terraformer）批处理。

返回值

number

类型 / 生物圈 (TYPES / BIOSPHERE)

LifeFormScanResult

返回自：`PortableBioScanner.scan().scan` (状态为 "ok") / 日志中的生物地点查询

相关对象类型 (RELATED OBJECT TYPES)

LifeFormSample

方法 (METHODS)

`.coord`

扫描位置的整数坐标 [x, y]。

返回值

[number, number]

`.life_forms`

LifeFormSample 条目的列表：每个图块包含 0-3 个条目。空列表表示“已扫描，但此处没有任何内容”。

返回值

`List<LifeFormSample>`

`.is_empty`

如果此图块没有发现任何生命形态，则为 `True`。

返回值

`boolean`

类型 / 生物圈 (TYPES / BIOSPHERE)

LifeFormSample

返回自：`PortableBioScanner.scan().scan.life_forms[i]` (状态为 "ok")

方法 (METHODS)

`.type`

生命形态项目 ID (例如 "ice_algae"、"snow_moss")。

返回值：`string`

可能的值

"ice_algae"、"snow_moss"、"frost_lichen"、"cold_spores"、"ice_crust"、"frost_fungus"、"sea_algae"、"tide_moss"、"shore_lichen"、"brine_plankton"、"salt_crust"、"coral_fungus"、"vent_algae"、"steam_moss"、"heat_lichen"、"hot_spores"、"heat_crust"、"vent_fungus"、"sulfur_moss"、"cinder_lichen"、"ash_spores"、"lava_algae"、"magma_crust"、"black_fungus"、"cave_moss"、"stone_lichen"、"crystal_spores"、"deep_algae"、"stone_mat"、"cave_fungus"

`.tons`

扫描时该生命形态在图块上可用的吨数（峰值，不是当前状态）。

返回值

number

`.remaining_tons`

当前可用的吨数。无人机的部分采集会减少此值；经过坐标冷却时间后，它会恢复为 `.tons`。

返回值

number

`.rarity`

稀有度层级：`"common"`（普通）、`"uncommon"`（非普通）或 `"rare"`（稀有）。它同时决定精华液化器（Essence Liquifier）每吨产生的精华量，以及提取后的冷却时间。

返回值：`string`

可能的值：`"common"`、`"uncommon"`、`"rare"`

`.biome`

该物种所属的原生生物群系：`"frozen"`、`"coastal"`、`"geothermal"`、`"volcanic"` 或 `"deep"`。该值按物种固定（从不表示其生长所在的地形），并且正是其精华液化器所消耗的生物群系。与 `nocturna.life_form_biome(sample.type)` 的值相同。

返回值：`string`

可能的值：`"frozen"`、`"coastal"`、`"geothermal"`、`"volcanic"`、`"deep"`

类型 / 生物圈 (TYPES / BIOSPHERE)

PlantRequirement

返回自：`SeedRecipe.requirements`

方法 (METHODS)

`.kind`

栽培条件：`"light"`、`"shade"`、`"water"`、`"salt"`、`"spacer"`、`"cluster"`、`"companion"` 或 `"antagonist"`。`SeedRecipe.requirements` 中的每个条目都是必需条件；组合物种会公开多个条目。

返回值：`string`

可能的值：`"light"`、`"shade"`、`"water"`、`"salt"`、`"spacer"`、`"cluster"`、`"companion"`、`"antagonist"`

`.species`

用于关系条件的相关物种键：对于 `"companion"`，表示所需的邻居；对于 `"antagonist"`，表示禁止的邻居。对于其他所有类型均为 `None`。

返回值：`Optional[string]`

可能的值：`"sunpetal"`、`"shadeleaf"`、`"dewmoss"`、`"lonethorn"`、`"packfern"`、`"twinvine"`、`"spitebud"`、`"sunspur"`、`"glowvine"`、`"crowncap"`、`"pondmoss"`、`"saltbloom"`、`"brinethorn"`、`"saltmate"`、`"grandbloom"`

类型 / 生物圈 (TYPES / BIOSPHERE)

PortableBioExtractor

返回自：`self.bio_extractor` (无人机)

方法 (METHODS)

`.extract()`

在悬停无人机下方发现的永久生物地点进行采集。采集完成前无人机会一直处于占用状态，包括脚本停止并重新启动期间。模块具有一个 25 t 的舱室，可容纳一种生命形态；货舱吊舱 (Cargo Pods) 会增加容量。部分耗尽状态会持续保留，只有地点被清空后才开始冷却。

返回值：`BioExtractionResult`

结果字段：`.status`、`.message`

成功载荷：`.extracted`

结果 (OUTCOMES)

状态	类型	含义
"ok"	成功	生物材料已提取到无人机货舱： <code>.extracted</code> t。
"not_mounted"	拒绝	无人机未安装 Portable Bio Extractor。
"scrambled"	暂态	无人机电子设备受到辐射扰乱，无法进行提取。
"busy"	暂态	此无人机或该地点正在进行生物提取。

"not_at_location"		拒绝	无人机必须完成路线并悬停在生物地点。
"not_scanned"		拒绝	无人机下方没有已扫描并记录的生物地点。
"cooling"		暂态	已耗尽的生物地点仍在补充中。
"no_cargo_space"	拒绝	无法提取剩余生命形态：每个兼容容器都已装满，或已分配给另一种类型。每个提取器舱室和 Cargo Pod 同一时间只能容纳一种生命形态。	

类型 / 生物圈（TYPES / BIOSPHERE）

PortableBioScanner

返回自：self.bio_scanner（无人机）

方法（METHODS）

.scan()

在无人机悬停处的当前整数坐标执行扫描。有效的非地点坐标会以空生物扫描结果完成。重复扫描免费。

返回值：BioScanResult

结果字段：.status、.message

成功载荷：.scan

结果（OUTCOMES）

状态	类型	含义
"ok"	成功	已完成无人机当前坐标处的生物扫描。
"not_mounted"	拒绝	无人机未安装 Portable Bio Scanner。
"scrambled"	暂态	无人机电子设备受到辐射扰乱，无法扫描。
"busy"	暂态	无人机正被另一项生物操作占用。
"not_at_location"	拒绝	无人机必须完成路线并悬停在整数生物调查坐标处。

类型 / 生物圈 (TYPES / BIOSPHERE)

SeedRecipe

返回自：`self.recipes()` (Seed Maker)

方法 (METHODS)

`.tier`

该已发现 Seed Maker 配方的生产层级。种子配方属于 Tier 1, 因为其输入物是采集到的源材料。

返回值：`number`

`.species`

该配方生成的裸植物物种 ID, 例如 "sunpetal"。它与 `Cell.plant` 匹配, 用于标识生物物种, 而不是种植命令使用的实体种子项目。

返回值：`string`

可能的值："sunpetal"、"shadeleaf"、"dewmoss"、"lonethorn"、"packfern"、"twinvine"、"spitebud"、"sunspur"、"glowvine"、"crowncap"、"pondmoss"、"saltbloom"、"brinethorn"、"saltmate"、"grandbloom"

`.seed_id`

该配方生成的实体种子项目 ID，例如 `"seed_sunpetal"`。将此值传给 Harvester 的 `load_seed()` 和 `plant()`，或者在向 Crop Automator 的输入端装入相同项目后传给其 `plant()`。

返回值：`string`

可能的值：`"seed_sunpetal"`、`"seed_shadeleaf"`、`"seed_dewmoss"`、`"seed_lonethorn"`、`"seed_packfern"`、`"seed_twinvine"`、`"seed_spitebud"`、`"seed_sunspur"`、`"seed_glowvine"`、`"seed_crowncap"`、`"seed_pondmoss"`、`"seed_saltbloom"`、`"seed_brinethorn"`、`"seed_saltmate"`、`"seed_grandbloom"`

`.blend`

能生成该种子的 3 个生命形态 ID 的确切列表（例如 `["ice_algae", "sea_algae", "vent_moss"]`）。顺序无关紧要：再次组合这三个 ID 即可重新生成该种子。

返回值：`list<string>`

`.requirements`

列出每项栽培条件，类型为 `PlantRequirement`。每个条目都有 `.kind` 和可选的 `.species`，用于标识所需的共生物种或禁止的拮抗物种。列出的每一项要求都必须满足，生长才会推进。

返回值：`list<PlantRequirement>`

`.requirement`

要求类型的紧凑摘要，例如 "light"、"light+water" 或 "salt+companion"。使用 `.requirements` 来做可编程布局决策，因为此摘要会有意省略共生物种/拮抗物种的身份。

返回值：`string`

可能的值："light"、"shade"、"water"、"spacer"、"cluster"、"companion"、"antagonist"、"light+space
r"、"light+water"、"shade+cluster"、"water+cluster"、"salt"、"salt+spacer"、"salt+companion"、"
light+water+spacer"

`.growth_time`

达到成熟 (`growth ==`

1.0) 所需的满足要求的小时数。生长时间越长的物种越难培育、价值越高；只有所有要求都满足时，生长才会推进。

返回值：`number`

`.base_yield`

该物种在多样性、处理和田地加成之前产出的整数量饲料。选择作物时，应将此值与 `.growth_time` 和 `.requirements` 一起比较。

返回值：`number`

类型 / 生物圈 (TYPES / BIOSPHERE)

SeedResult

返回自：`seed_maker.combine()`

方法 (METHODS)

`.status`

值为 "seed_found"、"sludge"、"locked"、"busy"、"missing_life_forms" 或 "output_full"。

返回值：`string`

可能的值："seed_found"、"sludge"、"locked"、"busy"、"missing_life_forms"、"output_full"

`.message`

面向玩家的组合试验结果说明。

返回值：`string`

`.seed_id`

存入输出端的稳定种子项目 ID；如果未生成种子，则为 None。

返回值：Optional[string]

可能的值："seed_sunpetal"、"seed_shadeleaf"、"seed_dewmoss"、"seed_lonethorn"、"seed_packfern"、"seed_twinvine"、"seed_spitebud"、"seed_sunspur"、"seed_glowvine"、"seed_crowncap"、"seed_pond moss"、"seed_saltbloom"、"seed_brinethorn"、"seed_saltmate"、"seed_grandbloom"

`.species`

稳定的已发现植物物种 ID；如果未生成种子，则为 None。

返回值：Optional[string]

可能的值："sunpetal"、"shadeleaf"、"dewmoss"、"lonethorn"、"packfern"、"twinvine"、"spitebud"、"sunspur"、"glowvine"、"crowncap"、"pondmoss"、"saltbloom"、"brinethorn"、"saltmate"、"grandbloom"

类型 / 内置模块 (TYPES / BUILT-IN MODULES)

Match

返回自：`re.search()` · `re.match()` · `re.fullmatch()`

方法 (METHODS)

`.pattern`

生成此匹配结果的正则表达式模式字符串。

返回值：`string`

`.string`

被搜索的字符串。

返回值：`string`

`.group(index=0)`

返回索引 `index` 对应的匹配文本。组 0 是整个匹配结果。未匹配的可选组返回 `None`；超出范围的组会引发异常。

参数

名称	类型	描述
<code>index</code>	<code>number</code>	捕获组索引；默认为 0

返回值：`Optional[string]`

`.groups(default=None)`

返回一个捕获组元组，不包含组 0。未匹配的组使用 `default`；如果省略，则 `default` 为 `None`。

参数

名称	类型	描述
<code>default</code>	<code>any</code>	未匹配可选组所使用的值

返回值：`tuple`

`.start(index=0)`

返回该组的起始字符索引。未匹配的可选组返回 -1。

参数：`index`（捕获组索引；默认值为 0）

返回值：`number`

`.end(index=0)`

返回该组的结束字符索引。未匹配的可选组返回 -1。

参数：index（捕获组索引；默认值为 0）

返回值：number

`.span(index=0)`

返回该组的 (start, end)。未匹配的可选组返回 (-1, -1)。

参数：index（捕获组索引；默认值为 0）

返回值：tuple<number>

类型 / 探索 (TYPES / EXPLORATION)

CatalogedCreature

返回自: `journal.cataloged_creatures(planet_id)`

方法 (METHODS)

`.creature_id`

`habitat.set_revival_target(creature_id)` 接受的稳定生物 ID。

返回值: `string`

可能的值: "salt_tortoise"、"magmatic_annelid"、"mycelial_husk"、"mantle_strider"、"glasswing_mantis"、"veil_mantle"、"vault_crab"、"tidal_cephalopod"、"bone_walker"、"vent_drifter"、"hive_sentinel"、"hollow_choir"、"ferric_sea_lily"、"crustal_echo"、"glacial_wyrm"、"spire_drake"

`.name`

面向玩家显示的生物名称。用于可读输出；自动化时使用 `.creature_id`。

返回值: `string`

`.rarity`

稀有度层级：`"common" | "uncommon" | "rare" | "legendary"`。稀有度决定复苏试剂的数量。

返回值：`string`

可能的值：`"common"、"uncommon"、"rare"、"legendary"`

`.fragment_ids`

完成该生物图鉴条目的五个已分析碎片 ID，按生物群系顺序排列。

返回值：`list<string>`

`.feed_item_id`

该生物接受的确切饲料项目。在复苏前，至少要在 Habitat 中准备 `.revive_feed_required` 个单位。

返回值：`string`

可能的值：`"feed_salt_tortoise"、"feed_magmatic_annelid"、"feed_mycelial_husk"、"feed_mantle_strider"、"feed_glasswing_mantis"、"feed_veil_mantle"、"feed_vault_crab"、"feed_tidal_cephalopod"、"feed_bone_walker"、"feed_vent_drifter"、"feed_hive_sentinel"、"feed_hollow_choir"、"feed_ferrocenic_sea_lily"、"feed_crustal_echo"、"feed_glacial_wyrm"、"feed_spire_drake"`

`.feed_recipe_id`

生成 `.feed_item_id` 的 Feed Maker 配方 ID。匹配的配方仍必须通过 Bio Orders 解锁；在 `feed_maker.list_recipes()` 中查找其当前 Recipe，以获取成分数量。

返回值：`string`

可能的值：`"craft_feed_salt_tortoise"`、`"craft_feed_magmatic_annelid"`、`"craft_feed_mycelial_husk"`、`"craft_feed_mantle_strider"`、`"craft_feed_glasswing_mantis"`、`"craft_feed_veil_mantle"`、`"craft_feed_vault_crab"`、`"craft_feed_tidal_cephalopod"`、`"craft_feed_bone_walker"`、`"craft_feed_vent_drifter"`、`"craft_feed_hive_sentinel"`、`"craft_feed_hollow_choir"`、`"craft_feed_ferric_sea_lily"`、`"craft_feed_crustal_echo"`、`"craft_feed_glacial_wyrm"`、`"craft_feed_spire_drake"`

`.revive_feed_required`

选择该生物后、`revive()` 可以开始前，必须在 Habitat 中准备的最少整数量饲料单位。

返回值：`number`

`.revive_reagents`

该生物对应稀有度的确切复苏采购清单，形式为 `{reagent_id: quantity}`。通过 Habitat 的试剂输入端准备列表中的全部数量。

返回值：`dict<number>`

类型 / 探索 (TYPES / EXPLORATION)

CatalogedFragment

返回自 : `journal.cataloged_fragments(planet_id)`

方法 (Methods)

`.fragment_id`

稳定的片段 ID (stable fragment ID) (例如 `"gw_cranial_plate"`)。它与 `BioOrder.requires` 中的键匹配, 因此脚本会用它来规划订单履行。

返回值 (Returns) : `string`

可选值 (Possible values) :

`"gw_cranial_plate"`、`"gw_caudal_fin"`、`"gw_cardiac_node"`、`"gw_jaw_fang"`、`"gw_spinal_vertebra"`、
`"vc_dorsal_carapace"`、`"vc_mandible_claw"`、`"vc_antenna_cluster"`、`"vc_walking_leg"`、`"vc_eye_stalk"`、
`"oc_cranium"`、`"oc_tentacle_arm"`、`"oc_chitin_beak"`、`"oc_ink_sac"`、`"oc_lens_eye"`、`"bw_skull"`、
`"bw_foreclaw"`、`"bw_ribcage"`、`"bw_hindlimb"`、`"bw_tail_spike"`、`"vd_bell"`、`"vd_nematocyst"`、
`"vd_neural_mesh"`、`"vd_photophore"`、`"vd_tendrill"`、`"mh_fruiting_body"`、`"mh_spore_pod"`、
`"mh_mycelium_root"`、`"mh_chitin_node"`、`"mh_stigmatic_disc"`、`"hs_mandible"`、`"hs_wing_membrane"`、
`"hs_thorax_plate"`、`"hs_abdomen_segment"`、`"hs_compound_eye"`、`"ms_chelicera"`、`"ms_leg_tarsus"`、
`"ms_pedipalp"`

"ms_abdomen_sclerite", "ms_eye_cluster", "hc_aperture_lip", "hc_shell_whorl", "hc_septum_plate",
",
"hc_beak", "hc_tentacle_crown", "ma_cranial_papilla", "ma_cuticle_molt", "ma_ganglion_node",
"ma_chitinous_seta", "ma_luminous_ring", "gm_compound_eye", "gm_folded_wing", "gm_raptorial_claw",
",
"gm_abdominal_sheath", "gm_antennal_whip", "fs_calyx_plate", "fs_arm_segment", "fs_stalk_columnal",
",
"fs_oral_tegmen", "fs_holdfast_rootlet", "sd_cranial_crest", "sd_wing_membrane", "sd_obsidian_scale",
",
"sd_tail_barb", "sd_talon", "ce_stalked_eye", "ce_swimmeret_lobe", "ce_mouth_disc",
"ce_great_appendage", "ce_cephalic_photophore", "st_scute_plate", "st_plastron_shard", "st_lim_b_claw",
",
"st_beak", "st_carapace_neural", "vm_cephalic_horn", "vm_wing_sheet", "vm_gill_filament",
"vm_tail_barb", "vm_ventral_photophore"

`.name`

面向玩家显示的片段名称（player-facing fragment name）（例如 **"Beak"**）。用于生成易读的输出；在匹配 `BioOrder.requires` 或移动物品时，请使用 `.fragment_id`。

返回值：`string`

`.biome`

发现该片段的生物群系（biome）：**"frozen"** 表示 Nocturna 的初始生物群系。按 `biome` 过滤结果列表，可将范围限定到特定前哨站的生物群系。

返回值：`string`

可选值：`"frozen"`、`"coastal"`、`"geothermal"`、`"volcanic"`、`"deep"`

`.coords`

可收集片段的世界坐标 `[x, y]`。将其传递给匹配前哨站的生物采集器（Bio Collector）中的 `bio_collector.collect(coords)`。

返回值：`[number, number]`

`.rarity`

稀有度等级（rarity tier）：`"common"` | `"uncommon"` | `"rare"` | `"legendary"`。继承自所属生物。

返回值：`string`

可选值：`"common"`、`"uncommon"`、`"rare"`、`"legendary"`

类型 / 探索 (TYPES / EXPLORATION)

Marker

由 `markers.get()` / `markers.list()` 返回。

方法 (Methods)

`.id`

放置标记时传入的稳定身份标识。将其传回 `markers.get(...)` 或 `markers.remove(...)`；还可以使用它的前缀对标记族进行分组。

返回值：`string`

`.x`

以米为单位的世界 X 坐标。可直接传给 `self.nav.set_target(m.x, m.y)`、无人机的 `self.go_to(m.x, m.y)`，或 `construction_blueprint.plan_structure(kind, m.x, m.y)`。

返回值：`number`

`.y`

以米为单位的世界 Y 坐标。

返回值：`number`

`.label`

显示在地图和标记列表中的文本。空字符串表示放置标记时未提供标签，此时地图会显示其 ID。

返回值：`string`

`.note`

将鼠标悬停在标记上时显示的较长备注。空字符串表示未填写备注。

返回值：`string`

`.icon`

绘制在标记图钉内部的字形，例如 `"x"`、`"check"` 或 `"hammer"`。

返回值：`string`

可选值：`"pin"`、`"x"`、`"check"`、`"circle"`、`"flag"`、`"crosshair"`、`"warning"`、`"hammer"`、`"resource"`、`"power"`、`"fluid"`、`"star"`

`.color`

标记颜色，例如 `"accent"` 或 `"error"`。

返回值：`string`

可选值：`"neutral"`、`"accent"`、`"success"`、`"warning"`、`"error"`、`"violet"`

类型 / 基础设施 (TYPES / INFRASTRUCTURE)

BlueprintPlanResult

由 construction_blueprint 规划命令返回。

方法 (Methods)

.status

稳定的规划结果。读取 `.blueprint_ids` 之前，应先根据此字段进行分支处理。

返回值：`string`

可选值："ok"、"locked"、"invalid_kind"、"invalid_rotation"、"invalid_medium"、"invalid_axis"、"invalid_layer"、"out_of_bounds"、"wrong_target"、"unsurveyed_target"、"too_hard"、"target_claimed"、"occupied"、"clearance"、"invalid_route"、"blocked"、"already_exists"、"already_queued"、"ambiguous_target"、"nothing_here"

.message

面向玩家的文字说明，解释确切的规划结果。

返回值：`string`

`.blueprint_ids`

新创建的蓝图 ID。除 "ok" 之外的所有结果均为空。

返回值：`list<string>`

类型 / 系统 (TYPES / SYSTEM)

ActionResult

由不带额外结果字段的游戏玩法命令返回。

方法 (Methods)

`.status`

脚本用于检查的稳定结果代码；`.message` 会解释发生了什么。

返回值：`string`

可选值：`"ok"`、`"not_mounted"`、`"invalid"`、`"out_of_bounds"`、`"busy"`、`"not_at_site"`、`"not_surveyed"`、`"too_hard"`、`"no_cargo_space"`、`"no_power"`、`"not_enough_power"`、`"not_found"`、`"locked"`、`"already_active"`、`"wrong_position"`、`"insufficient_materials"`、`"paused_no_power"`、`"cargo_present"`、`"blocked"`、`"paused"`、`"canceled"`、`"not_ready"`、`"not_local"`、`"same_endpoint"`、`"unsupported_source"`、`"source_is_vehicle"`、`"unsupported_target"`、`"target_is_vehicle"`、`"incompatible"`、`"duplicate"`、`"invalid_type"`、`"path"`、`"wall"`、`"exit"`、`"in_progress"`、`"ongoing"`、`"win"`、`"loss"`、`"draw"`、`"occupied"`、`"no_game"`、`"rejected"`、`"booting"`、`"already_booted"`、`"activating"`、`"already_online"`、`"boot_required"`、`"initializing"`、`"power_required"`、`"insufficient_credits"`

可选值 (续)：`"inventory_full"`、`"not_toggleable"`、`"under_construction"`、`"not_connected"`、`"no_script"`、`"already_running"`、`"not_powered"`、`"started"`、`"already_repaired"`、`"no_source"`、`"already_testing"`、`"too_far"`、`"already_here"`、`"overheated"`、`"moving"`、`"dropped"`、`"empty"`、`"invalid_seed"`、`"no_seed"`、`"holding"`、`"not_empty"`、`"base_sector"`、`"no_kit"`、`"invalid_kit"`、`"not_plantable"`、`"insufficient_water"`、`"tank_empty"`、`"already_full"`、`"no_salt"`、`"invalid_input"`、`"no_plant"`、`"already_mature"`、`"tier_conflict"`、`"no_dose"`、`"nothing"`、`"script_present"`、`"partial"`、`"not_mature"`、`"no_forage"`、`"correct"`、`"incorrect"`、`"already_completed_correct"`、`"already_completed_incorrect"`、`"accepted"`、`"wrong_planet"`、`"wrong_contract"`、`"unknown_contract"`、`"key_is_planet"`、`"unknown_key"`、`"not_at_service_point"`、`"item_not_in_inventory"`、`"unknown_module"`、`"slot_not_compatible"`、`"slot_occupied"`、`"invalid_slot"`

可选值（续）："capability_already_mounted"、"slot_empty"、"holder_not_empty"、"invalid_internal_slot"、"not_container"、"item_not_accepted"、"internal_slot_occupied"、"internal_slot_empty"、"container_not_empty"、"charging"、"queued"、"target_reached"、"not_docked"、"station_offline"、"already_dispatched"、"unknown_recipe"、"offline"、"recipe_locked"、"material_mismatch"、"material_present"、"invalid_channel"、"invalid_value"、"channel_limit"、"invalid_key"、"invalid_coords"、"invalid_icon"、"invalid_color"、"invalid_text"、"limit_reached"、"entry_limit"、"unknown_order"、"completed"、"station_not_found"、"out_of_range"、"scrambled"、"drill_not_found"、"invalid_target"、"not_at_station"、"wrong_engine_for_module"、"cargo_capacity_exceeded"、"module_not_mounted"、"hot_cargo_requires_plating"、"refueling"、"no_oil"、"not_stranded"、"no_rescue"、"worker_not_present"

可选值（续）："construction_dependency"、"complete"、"no_input"、"output_full"、"no_active"、"cargo_occupied"、"no_fragment"、"input_occupied"、"source_empty"、"source_busy"、"wrong_outpost"、"invalid_source"、"invalid_reagent"、"invalid_qty"、"invalid_properties"、"invalid_property_match"、"insufficient_input"、"recipe_mismatch"、"input_empty"、"not_analyzed"、"invalid_specimen"、"chamber_occupied"、"not_in_input"、"invalid_fragment"、"destroyed"、"unknown_gene"、"invalid_recipe"、"wrong_materials"、"wrong_fragment"、"no_recipe"、"conditioned"、"burned"、"no_run"、"species_exists"、"unknown_creature"、"not_cataloged"、"no_target"、"wrong_feed"、"insufficient_feed"

可选值（续）："insufficient_reagents"、"no_colony"、"not_established"、"insufficient_capacity"、
"unknown_node"、"already_purchased"、"population_locked"、"insufficient_insight"

可选值（续）："output_busy"、"no_op"、"no_material"

.message

面向玩家的文字说明，解释命令的确切结果。适用于日志和面向玩家的诊断信息。

返回值：**string**

类型 / 系统 (TYPES / SYSTEM)

CommandResult

由 `Component.next_command()` 返回。

方法 (Methods)

`.status`

命令被取出时为 "ok", 否则为 "empty"。

返回值: `string`

可选值: "ok"、"empty"

`.message`

面向玩家的文字说明, 解释命令出队的结果。

返回值: `string`

`.command`

取出的 `ScriptCommand`; 队列为空时为 `None`。

返回值: `Optional[ScriptCommand]`

类型 / 系统 (TYPES / SYSTEM)

CountResult

由队列、丢弃、清空和批量计数命令返回。

方法 (Methods)

.status

有一个或多个条目发生变化时为 "ok", 没有变化时为 "no_op", 也可能是特定于命令的拒绝结果, 例如 "invalid_channel" 或 "invalid_key"。

返回值: `string`

可选值: "ok"、"no_op"、"invalid_channel"、"invalid_key"

.message

面向玩家的文字说明, 解释计数操作的结果。

返回值: `string`

.count

受该命令影响的条目或单位数量, 为整数。

返回值: `number`

类型 / 系统 (TYPES / SYSTEM)

CropJob

由作物自动机 (Crop Automator) 的 `current_job()` 和 `get_queue()` 返回。

方法 (Methods)

`.id`

稳定的整数作业 ID。

返回值：number

`.action`

请求执行的工作："harvest"、"plant" 或 "apply"。

返回值：string

可选值："harvest"、"plant"、"apply"

`.sector`

目标田地区域。

返回值：string

`.item_id`

请求使用的种子或处理物品；收获作业为 `None`。

返回值：Optional[string]

`.state`

作业状态为 "working"、"blocked" 或 "queued"。

返回值：string

可选值："working"、"blocked"、"queued"

`.progress`

已完成比例，范围为 0-1。待处理作业的值为 0。

返回值：number

`.blocker`

确切的物理阻塞原因；作业处于工作中或排队中时为 **None**。

返回值：Optional[string]

可选值："not_placed"、"no_power"、"no_seed"、"no_material"、"output_full"、"results_full"

类型 / 系统 (TYPES / SYSTEM)

CropJobResult

由 Crop Automator 的 `next_result()` 返回。

方法 (Methods)

`.status`

作业的终止结果；没有已完成结果等待时为 "empty"。

返回值：string

可选值："ok"、"partial"、"empty"、"out_of_range"、"no_plant"、"not_mature"、"no_forage"、"not_empty"、"base_sector"、"already_mature"、"tier_conflict"、"invalid_seed"、"invalid_material"

`.message`

面向玩家的文字说明，解释终止结果。

返回值：string

`.job_id`

已完成的作业 ID；没有结果时为 `None`。

返回值：Optional[number]

.action

已完成的操作；没有结果时为 **None**。

返回值：Optional[string]

可选值："harvest"、"plant"、"apply"

.sector

已完成的目标区域；没有结果时为 **None**。

返回值：Optional[string]

.item_id

请求使用的种子或处理物品；对于收获和空结果为 **None**。

返回值：Optional[string]

.collected

收获时提交到输出箱的觅食资源（Forage）单位整数。其他情况下始终为 0。

返回值：number

.discarded

收获时因空间不足而丢弃的觅食资源（Forage）单位整数。仅当结果为 "partial" 时非零；无论哪种情况，该格都会被清空。

返回值：number

类型 / 系统 (TYPES / SYSTEM)

JobReceipt

由作物自动机 (Crop Automator) 的作业提交操作返回。

方法 (Methods)

.status

作业进入先进先出 (FIFO) 队列时为 **"queued"**；否则为作业未被接受的确切原因。

返回值：**string**

可选值：**"queued"**、**"queue_full"**、**"not_placed"**、**"out_of_range"**、**"invalid_seed"**、**"invalid_material"**

.message

面向玩家的文字说明，解释提交结果。

返回值：**string**

.job_id

稳定的整数作业 ID；未创建作业时为 **None**。

返回值：**Optional[number]**

`queue_position`

提交时的执行位置（从 1 开始计数），包括当前活动作业；如果请求被拒绝，则为 **None**。

返回值

`Optional[number]`

类型 / 地形改造 (TYPES / TERRAFORMING)

AnalyzeInfo

返回自：`bio_lab.analyze().info`，且状态为 "ok" 之后

方法 (METHODS)

.fragment_id

已发现的片段 ID (例如 "gw_cranial_plate")。将其用作所得样本的库存键：生物订单 (Bio Orders) 会按此 ID 请求片段。

返回值 可能的值

```
string "gw_cranial_plate",  
"gw_caudal_fin",  
"gw_cardiac_node",  
"gw_jaw_fang",  
"gw_spinal_vertebra",  
"vc_dorsal_carapace",  
"vc_mandible_claw",  
"vc_antenna_cluster",  
"vc_walking_leg",  
"vc_eye_stalk",  
"oc_cranium",  
"oc_tentacle_arm",  
"oc_chitin_beak",  
"oc_ink_sac",  
"oc_lens_eye", "bw_skull",  
"bw_foreclaw",  
"bw_ribcage",  
"bw_hindlimb",  
"bw_tail_spike",  
"vd_bell",  
"vd_nematocyst",  
"vd_neural_mesh",  
"vd_photophore",  
"vd_tendrill",  
"mh_fruiting_body",  
"mh_spore_pod",  
"mh_mycelium_root",  
"mh_chitin_node",  
"mh_stigmatic_disc",  
"hs_mandible",
```

"hs_wing_membrane",
"hs_thorax_plate",
"hs_abdomen_segment",
"hs_compound_eye",
"ms_chelicera",
"ms_leg_tarsus",
"ms_pedipalp",

"ms_abdomen_sclerite",
"ms_eye_cluster",
"hc_aperture_lip",
"hc_shell_whorl",
"hc_septum_plate",
"hc_beak",
"hc_tentacle_crown",
"ma_cranial_papilla",
"ma_cuticle_molt",
"ma_ganglion_node",
"ma_chitinous_seta",
"ma_luminous_ring",
"gm_compound_eye",
"gm_folded_wing",
"gm_raptorial_claw",
"gm_abdominal_sheath",
"gm_antennal_whip",
"fs_calyx_plate",
"fs_arm_segment",
"fs_stalk_columnal",
"fs_oral_tegmen",
"fs_holdfast_rootlet",
"sd_cranial_crest",
"sd_wing_membrane",
"sd_obsidian_scale",
"sd_tail_barb",
"sd_talon",
"ce_stalked_eye",
"ce_swimmeret_lobe",
"ce_mouth_disc",
"ce_great_appendage",
"ce_cephalic_photophore",
"st_scute_plate",
"st_plastron_shard",
"st_limb_claw", "st_beak",
"st_carapace_neural",

"vm_cephalic_horn",
"vm_wing_sheet",
"vm_gill_filament",
"vm_tail_barb",
"vm_ventral_photophore"

.name

已分析片段的显示名称。

返回值

string

.rarity

稀有度等级："common"（普通）|"uncommon"（罕见）|"rare"（稀有）|"legendary"（传奇）。

返回值 可能的值

string "common"、"uncommon"、
"rare"、"legendary"

.required_recipe

提取样本所需的试剂：dict {reagent_id: qty}。遍历 .items()，并为每个键值对调用 lab.load(rid, qty)，然后调用 lab.extract()。

返回值

dict

.coords

采集该标本的世界坐标 [x, y]。

返回值

[number, number]

.distance

从该前哨站到标本的直线距离，单位为米（在采集时设置）。

返回值

number

类型 / 地形改造 (TYPES / TERRAFORMING)

BioCasterRecipe

返回自：`bio_caster.list_recipes()` / `bio_caster.find_recipe(fragment_id)`

方法 (METHODS)

`.fragment_id`

该配方要锻造的火山片段 ID。在 Caster 自己的脚本中，片段准备好处理后，将其传给 `self.set_recipe(recipe.fragment_id)`。

返回值 可能的值

```
string "gw_jaw_fang"、  
"vc_walking_leg"、  
"oc_ink_sac"、  
"bw_hindlimb"、  
"vd_photophore"、  
"mh_chitin_node"、  
"hs_abdomen_segment"、  
"ms_abdomen_sclerite"、  
"hc_beak"、  
"ma_chitinous_seta"、  
"gm_abdominal_sheath"、  
"fs_oral_tegmen"、  
"sd_tail_barb"、  
"ce_great_appendage"、  
"st_beak"、"vm_tail_barb"
```

`.tier`

该锻造配方的派生生产等级。

返回值

number

`.materials`

精确的制造材料采购清单，格式为 `{item_id: count}`。供应脚本无需选择配方或持有其片段即可遍历 `.items()`。

返回值

`dict<number>`

`.temperature_range`

该配方的目标温度闭区间 `[low, high]`，单位为 $^{\circ}\text{C}$ 。

返回值

`[number, number]`

类型 / 地形改造 (TYPES / TERRAFORMING)

BioOrder

返回自：`bio_exchange.orders()` / `bio_exchange.active_order()` /
`get_component("bio_exchange_1").orders()`

方法 (METHODS)

`.id`

稳定的生物订单 ID（例如 "bio_order_03"）。传给 `bio_exchange.set_order(id)`，将该订单设为当前生物订单。

返回值

string

`.name`

生物订单的显示名称（例如 "Cryophyte Spore Panel"）。

返回值

string

`.biome`

所需片段所属的生物群系（例如 "frozen"）。如果你没有对应生物群系的前哨站，则暂时无法完成该生物群系的订单。

返回值 可能的值

string "frozen"、"coastal"、
"geothermal"、"volcanic"、
"deep"

`.requires`

片段采购清单：`dict {fragment_id: count}`。遍历 `.items()` 可查看需要交付哪些片段以及每种片段的数量。

返回值

`dict`

`.reward`

满足所有要求时支付的信用点。订单只支付一次，因为订单是一次性的。

返回值

`number`

`.status`

相对于该生物交换站的状态：`"available"`（当前未选中）、`"active"`（当前已选中）或 `"complete"`（已在全局完成）。全局完成优先于选中状态；某个可用订单可能已经存在交付进度。

返回值 可能的值

`string "available"、"active"、
"complete"`

`.delivered`

每种片段的交付进度：`dict {fragment_id: count}`，在服务该订单的所有生物交换站之间共享。与 `.requires` 比较即可查看还需交付的内容。

返回值

`dict`

`.in_transit`

已提交交付的样本，格式为 `{fragment_id: count}`，在服务该订单的所有交换站之间共享。其中包括已暂存于交换站输入区的合格样本，以及正在进行的定时交付。样本在 `deliver()` 将其移入交付流程期间会持续计入；交付完成或暂存到 `self.output` 以退款时才会离开计数。重新查询订单以获取当前值。`.percent` 只统计已完成的交付。

返回值

`dict<number>`

`.percent`

总体完成度：0-100，表示已交付样本数除以所有片段的总需求量。

返回值

`number`

`.target_glow`

仅适用于沿海注入订单：所需片段必须通过 Bio Luminizer（生物发光器）注入的精确光辉 `[r,g,b]`（0-255）。非沿海订单为 `None`。`deliver()` 只会匹配光辉与此值相等的发光片段。

返回值

`Optional[[number, number, number]]`

`.required_genes`

地热订单所需的基因集合，格式为 `{fragment_id: [attribute_id, ...]}`；其他订单为 `None`。每个片段必须恰好携带列出的 1-3 个基因。使用 DNA Sequencer（DNA 测序仪）进行工程改造；原始片段以及带有额外基因的片段均不被接受。

返回值

`Optional[dict[str, list[string]]]`

类型 / 地形改造 (TYPES / TERRAFORMING)

ChamberFragment

返回自 : dna_sequencer.chamber

方法 (METHODS)

.fragment_id

装入腔室的地热片段 ID (例如 "gw_cardiac_node")。

返回值 可能的值

```
string "gw_cardiac_node"、  
"vc_antenna_cluster"、  
"oc_chitin_beak"、  
"bw_ribcage"、  
"vd_neural_mesh"、  
"mh_mycelium_root"、  
"hs_thorax_plate"、  
"ms_pedipalp"、  
"hc_septum_plate"、  
"ma_ganglion_node"、  
"gm_raptorial_claw"、  
"fs_stalk_columnal"、  
"sd_obsidian_scale"、  
"ce_mouth_disc"、  
"st_limb_claw"、  
"vm_gill_filament"
```

.name

装入腔室的地热片段面向玩家显示的名称。

返回值

string

`.genes`

腔室中的片段当前携带的基因（例如 `["cold_tolerance", "pressure_tolerance"]`）。与调用 `genes()` 的结果相同。

返回值

`list<string>`

`.spliced`

该片段使用过一次安全拼接后为 `True`。再次调用 `splice()` 会摧毁该片段。为 `False` 表示仍可进行一次工程改造。

返回值

`boolean`

类型 / 地形改造 (TYPES / TERRAFORMING)

ChamberSample

返回自 : `bio_luminizer.chamber`

方法 (METHODS)

`.fragment_id`

装入腔室的沿海片段 ID (例如 `"gw_caudal_fin"`)。

返回值 可能的值

```
string "gw_caudal_fin"、  
"vc_mandible_claw"、  
"oc_tentacle_arm"、  
"bw_foreclaw"、  
"vd_nematocyst"、  
"mh_spore_pod"、  
"hs_wing_membrane"、  
"ms_leg_tarsus"、  
"hc_shell_whorl"、  
"ma_cuticle_molt"、  
"gm_folded_wing"、  
"fs_arm_segment"、  
"sd_wing_membrane"、  
"ce_swimmeret_lobe"、  
"st_plastron_shard"、  
"vm_wing_sheet"
```

`.name`

装入腔室的沿海片段面向玩家显示的名称。

返回值

```
string
```

`.glow`

样本当前的光辉 `[r,g,b]` (0-255)：调谐前的暗淡起始颜色。读取它、订单的 `target_glow` 以及 `lamp_signature(...)`，即可求出灯具设置。

返回值

`[number, number, number]`

类型 / 地形改造 (TYPES / TERRAFORMING)

FragmentLocation

返回自：`bio_collector.scan()`

方法 (METHODS)

`.coords`

该片段的世界坐标 `[x, y]`。传给 `bio_collector.collect(coords)` 即可取回片段。

返回值

`[number, number]`

`.distance`

从该前哨站到片段的直线距离，单位为米。扫描结果按升序排列：`result[0]` 是最近的片段。

返回值

`number`

`.cataloged`

如果该片段已经分析并加入日志，则为 `True`；对于仍未知的点则为 `False`。它决定是否提供可选的 `.fragment_id`、`.name` 和 `.rarity` 字段。

返回值

`boolean`

`.fragment_id`

当 `.cataloged` 为 `True` 时返回已知的片段 ID；对于仍未知的点则为 `None`。它与 `BioOrder.requires` 中的键相匹配。

返回值 可能的值

```
Optional[string] "gw_cranial_plate",  
"gw_caudal_fin",  
"gw_cardiac_node",  
"gw_jaw_fang",  
"gw_spinal_vertebra",  
"vc_dorsal_carapace",  
"vc_mandible_claw",  
"vc_antenna_cluster",  
"vc_walking_leg",  
"vc_eye_stalk",  
"oc_cranium",  
"oc_tentacle_arm",  
"oc_chitin_beak",  
"oc_ink_sac",  
"oc_lens_eye", "bw_skull",  
"bw_foreclaw",  
"bw_ribcage",  
"bw_hindlimb",  
"bw_tail_spike",  
"vd_bell",  
"vd_nematocyst",  
"vd_neural_mesh",  
"vd_photophore",  
"vd_tendrill",  
"mh_fruiting_body",  
"mh_spore_pod",  
"mh_mycelium_root",  
"mh_chitin_node",  
"mh_stigmatic_disc",  
"hs_mandible",  
"hs_wing_membrane",  
"hs_thorax_plate",
```

"hs_abdomen_segment",
"hs_compound_eye",
"ms_chelicera",
"ms_leg_tarsus",
"ms_pedipalp",
"ms_abdomen_sclerite",
"ms_eye_cluster",

"hc_aperture_lip",
"hc_shell_whorl",
"hc_septum_plate",
"hc_beak",
"hc_tentacle_crown",
"ma_cranial_papilla",
"ma_cuticle_molt",
"ma_ganglion_node",
"ma_chitinous_seta",
"ma_luminous_ring",
"gm_compound_eye",
"gm_folded_wing",
"gm_raptorial_claw",
"gm_abdominal_sheath",
"gm_antennal_whip",
"fs_calyx_plate",
"fs_arm_segment",
"fs_stalk_columnal",
"fs_oral_tegmen",
"fs_holdfast_rootlet",
"sd_cranial_crest",
"sd_wing_membrane",
"sd_obsidian_scale",
"sd_tail_barb",
"sd_talon",
"ce_stalked_eye",
"ce_swimmeret_lobe",
"ce_mouth_disc",
"ce_great_appendage",
"ce_cephalic_photophore",
"st_scute_plate",
"st_plastron_shard",
"st_limb_claw", "st_beak",
"st_carapace_neural",
"vm_cephalic_horn",
"vm_wing_sheet",

"vm_gill_filament",
"vm_tail_barb",
"vm_ventral_photophore"

`.name`

当 `.cataloged` 为 `True` 时返回面向玩家显示的片段名称；对于仍未知的点则为 `None`。用于可读日志和面向 UI 的决策，同时保留 `.fragment_id` 作为稳定的自动化键。

返回值

`Optional[string]`

`.rarity`

当 `.cataloged` 为 `True` 时返回稀有度等级；对于仍未知的点则为 `None`。扫描结果不会暴露配方或生物身份。

返回值 可能的值

`Optional[string]` "common"、"uncommon"、
"rare"、"legendary"

类型 / 地形改造 (TYPES / TERRAFORMING)

HabitatBonusNode

返回自：`Habitat.get_bonus_tree().nodes` 和 `Habitat.get_active_bonuses()`

方法 (METHODS)

`.id`

可传给 `unlock_bonus(...)` 的稳定节点 ID。

返回值

string

`.source_species`

拥有该节点的生物 ID。全局突破仍根据一个来源物种来获得。

返回值

string

`.slot`

技能树位置："adaptation"（适应）或 "breakthrough"（突破）。

返回值 可能的值

string "adaptation"、
"breakthrough"

`.scope`

效果范围："species"（物种）只影响 `source_species`；"global"（全局）影响当前及未来的所有 Habitat 殖民地。

返回值 可能的值

string "species"、"global"

.depth

技能树深度 1-2。

返回值

number

.name

面向玩家显示的作者定义节点名称。

返回值

string

.description

作者定义的精确效果摘要。

返回值

string

.state

当前购买状态。

返回值 可能的值

string "purchased"（已购买）、"available"（可用）、
"unaffordable"（负担不起）、
"population_locked"（人口锁定）

.purchased

该节点是否已永久购买。

返回值

boolean

`.active`

该已购买节点当前是否至少有一个效果正在应用于返回它的 Habitat。未购买节点始终为 `False`。

返回值

`boolean`

`.insight_cost`

完整的共享 Insight 成本。

返回值

`number`

`.local_population`

最低本地殖民地人口：适应项为 0，突破项为 10,000。

返回值

`number`

`.unmet`

当前所有未满足的要求：最多为 `"population"` 和 `"insight"`。

返回值

`list<string>`

类型 / 地形改造 (TYPES / TERRAFORMING)

HabitatBonusTree

返回自: `Habitat.get_bonus_tree()`

方法 (METHODS)

`.species`

显示的已编写树所属的生物 ID；选择目标前为空字符串。

返回值

string

`.shared_insight`

当前可用于购买的精确共享 Insight，包括保留的小数部分。

返回值

number

`.purchased_count`

从该物种树中购买的永久节点数量。

返回值

number

`.nodes`

按稳定顺序排列的仅限该物种的适应项和全物种突破项，包括锁定节点和已购买节点。

返回值

`List<HabitatBonusNode>`

类型 / 地形改造 (TYPES / TERRAFORMING)

HabitatInsight

返回自: `Habitat.get_insight()`

方法 (METHODS)

`.shared`

任何 Habitat 当前可支出的精确共享 Insight, 包括保留的小数部分。

返回值

number

`.shared_exact`

`shared` 的别名, 为已读取这个明确精确余额的脚本保留。

返回值

number

`.rate_per_hour`

按照当前增长和补给情况, 该殖民地预计每小时产生的 Insight。

返回值

number

`.lifetime_produced`

该殖民地在其生命周期内产生的 Insight, 包括小数部分。

返回值

number

`.producing`

该殖民地是否按当前增长率产生正值 Insight。

返回值

`boolean`

类型 / 地形改造 (TYPES / TERRAFORMING)

Specimen

返回自：`bio_collector.cargo/bio_lab.specimen`

方法 (METHODS)

`.coords`

采集该样本的世界坐标 [x, y]。在保存和加载之间保持稳定。

返回值

`[number, number]`

`.distance`

从采集前哨站到采集地点的直线距离，单位为米（在采集时设定）。

返回值

`number`

`.stage`

workflow阶段：分析前为 "collected"，实验室通过 `lab.analyze()` 揭示片段和配方后为 "analyzed"。

返回值 可能的值

`string "collected"、"analyzed"`

`.fragment_id`

片段 ID (例如 `"gw_cranial_plate"`)。生物采集器货舱在片段编目后会显示该 ID；生物实验室输入只有在该样本达到 `"analyzed"` 状态后才会显示它，即使该片段已经编目。否则为 `None`。

返回值 可能的值

```
Optional[string] "gw_cranial_plate",  
"gw_caudal_fin",  
"gw_cardiac_node",  
"gw_jaw_fang",  
"gw_spinal_vertebra",  
"vc_dorsal_carapace",  
"vc_mandible_claw",  
"vc_antenna_cluster",  
"vc_walking_leg",  
"vc_eye_stalk",  
"oc_cranium",  
"oc_tentacle_arm",  
"oc_chitin_beak",  
"oc_ink_sac",  
"oc_lens_eye", "bw_skull",  
"bw_foreclaw",  
"bw_ribcage",  
"bw_hindlimb",  
"bw_tail_spike",  
"vd_bell",  
"vd_nematocyst",  
"vd_neural_mesh",  
"vd_photophore",  
"vd_tendrill",  
"mh_fruiting_body",  
"mh_spore_pod",  
"mh_mycelium_root",  
"mh_chitin_node",  
"mh_stigmatic_disc",  
"hs_mandible",  
"hs_wing_membrane",
```

"hs_thorax_plate",
"hs_abdomen_segment",
"hs_compound_eye",
"ms_chelicera",
"ms_leg_tarsus",
"ms_pedipalp",
"ms_abdomen_sclerite",

"ms_eye_cluster",
"hc_aperture_lip",
"hc_shell_whorl",
"hc_septum_plate",
"hc_beak",
"hc_tentacle_crown",
"ma_cranial_papilla",
"ma_cuticle_molt",
"ma_ganglion_node",
"ma_chitinous_seta",
"ma_luminous_ring",
"gm_compound_eye",
"gm_folded_wing",
"gm_raptorial_claw",
"gm_abdominal_sheath",
"gm_antennal_whip",
"fs_calyx_plate",
"fs_arm_segment",
"fs_stalk_columnal",
"fs_oral_tegmen",
"fs_holdfast_rootlet",
"sd_cranial_crest",
"sd_wing_membrane",
"sd_obsidian_scale",
"sd_tail_barb",
"sd_talon",
"ce_stalked_eye",
"ce_swimmeret_lobe",
"ce_mouth_disc",
"ce_great_appendage",
"ce_cephalic_photophore",
"st_scute_plate",
"st_plastron_shard",
"st_limb_claw", "st_beak",
"st_carapace_neural",
"vm_cephalic_horn",

"vm_wing_sheet",
"vm_gill_filament",
"vm_tail_barb",
"vm_ventral_photophore"

`.name`

面向玩家显示的片段名称（例如 "Beak"）。片段身份未知时为 `None`。

返回值

`Optional[string]`

`.rarity`

稀有度等级：`"common" | "uncommon" | "rare" | "legendary"`。分析前为 `None`。

返回值 可能的值

`Optional[string]` `"common"`、`"uncommon"`、
`"rare"`、`"legendary"`

`.recipe`

所需试剂配方：`dict {reagent_id: qty}`。分析前为 `None`。

返回值

`Optional[dict]`

`.production_tier`

该样本生物实验室提取配方的派生等级；其身份和试剂配方未知时为 `None`。

返回值

`Optional[number]`

`.glow`

沿海样本的生物发光颜色 `[r,g,b]` (0-255) , 即生物发光调谐器 (Bio Luminizer) 调节到某个订单目标的暗淡起始颜色。其他所有生物群系均为 **None**。

返回值

`Optional[[number, number, number]]`

`.genes`

该地热样本携带的基因 (例如 `["heat_resistance", "acid_resistance"]`) 。DNA 链本身属于内部实现：你在基因层级上操作。非地热样本为空。

返回值

`list<string>`

类型 / 地形改造 (TYPES / TERRAFORMING)

WasteDumpResult

返回自：`oxygen_generator.dump_waste()`

方法 (METHODS)

`.status`

废物舱倾倒后始终为 "ok"。

返回值 可能的值

`string "ok"`

`.message`

面向玩家的说明，解释倾倒操作及其导致的效率惩罚。

返回值

`string`

`.penalty`

本次倾倒施加的效率惩罚，范围为 0 到 1。

返回值

`number`

类型 / 天气与天空 (TYPES / WEATHER & SKY)

SignalReceiver

返回自 : `weather_station.signal_receiver`

方法 (METHODS)

`.transmissions()`

该供电台站当前可听到的原始传输。可能同时存在多个事件。排序是稳定的，但共享同一个数据包编号的副本不会按有效性排序。接收器不保存历史记录。

返回值

`list<SignalTransmission>`

类型 / 天气与天空 (TYPES / WEATHER & SKY)

SignalTransmission

返回自 : `SignalReceiver.transmissions()`

方法 (METHODS)

`.event_id`

每个数据包和噪声副本共享的事件 ID。

返回值

string

`.number`

声明的数据包编号。不同副本可能声明相同的编号。

返回值

number

`.total`

该事件声明的数据包总数。

返回值

number

`.channel`

接收到该副本的频道。

返回值 可能的值

string "broadcast"、"frozen"、
"coastal"、"geothermal"、
"volcanic"、"deep"

`.data`

编码后的移动记录：`event_id|number|total|dx|dy`。编号记录组成一个从 $(0, 0)$ 开始的移动序列；`dx` 和 `dy` 是该记录带符号的移动偏移量，序列端点就是事件坐标。

返回值

string

`.checksum`

传输随附的校验和：`.data` 中每个字符 ASCII 码的完整总和，不执行模运算约简。

返回值

number

`.emitted_at_gh`

该事件中每个有效副本和含噪副本共享的风暴开始世界时钟时间戳。

返回值

number

`.expires_at_gh`

该事件实时监听窗口关闭时的世界时钟时间戳。

返回值

number

`.source_station_id`

接收到该副本的实体气象站。

返回值

string

类型 / 天气与天空 (TYPES / WEATHER & SKY)

Storm

返回自 : `WeatherReport.active()`

方法 (METHODS)

`.id`

稳定的风暴 ID。

返回值

`string`

`.x`

本报告观测时的单元格中心 x 坐标，单位为米。

返回值

`number`

`.y`

本报告观测时的单元格中心 y 坐标，单位为米。

返回值

`number`

`.kind()`

"dust" 或 "thunder"。沙尘事件会传输一条原始铀后效信息。雷暴会为符合条件的避雷针充电，并可能产生风暴玻璃信息。

返回值 可能的值

`string "dust"、"thunder"`

`.radius_m()`

单元格半径，单位为米。直线路线上的氢无人机一旦当前位置进入该单元格就会停留；它不会自动绕行。电动无人机会直接穿过。

返回值

number

`.speed()`

移动速度，单位为米/小时。

返回值

number

`.heading()`

单位移动方向，表示为 `[dx, dy]`。

返回值

`[number, number]`

`.intensity()`

报告时观测到的强度，范围为 0-1。

返回值

number

`.expires_in()`

该观测单元格消散前剩余的实时小时数。

返回值

number

`.eta_to(x, y)`

单元格边缘抵达该点所需的小时数。如果该点已在单元格内，则为 0；如果轨迹在消散前永远无法抵达，则为 **None**。

参数

名称	类型	描述
x	number	世界 x 坐标，单位为米。
y	number	世界 y 坐标，单位为米。

返回值

`Optional[number]`

类型 / 天气与天空 (TYPES / WEATHER & SKY)

WeatherEventForecast

返回自：`WeatherReport.forecast()`

方法 (METHODS)

`.id`

与活动中的 **Storm** 以及之后的信号传输共享的稳定事件 ID。

返回值

`string`

`.kind()`

广义事件类别：`"dust"` 或 `"thunder"`。

返回值 可能的值

`string "dust"、"thunder"`

`.arrival_window()`

在本报告观测时间之后，单元格预计进入覆盖范围的 `[earliest, latest]` 小时窗口。

返回值

`[number, number]`

`.corridor()`

粗略预测的移动走廊，类型为 **Zone**；它是风暴信息，而不是隐藏的后效路径。

返回值

Zone

`.intensity_range()`

观测到的预测范围 `[low, high]`, 每个值均为 0-1。

返回值

`[number, number]`

类型 / 天气与天空 (TYPES / WEATHER & SKY)

WeatherReport

返回自: `weather_station.observe()` / `weather_station.last_report()`

方法 (METHODS)

`.id`

用于溯源和日志记录的稳定报告 ID。

返回值

string

`.source_id`

生成该报告的实体气象站 ID。

返回值

string

`.observed_at_gh`

测量该报告时的世界时钟时间戳。

返回值

number

`.age_gh()`

该不可变报告按世界时钟计算的实时年龄，单位为小时。

返回值

number

`.coverage()`

观测时冻结的该台站本地覆盖范围，类型为 `Zone`。

返回值

`Zone`

`.active()`

在观测时刻位于本地覆盖范围内的活动 `Storm` 快照列表。

返回值

`List<Storm>`

`.forecast()`

预计在未来 8 个世界时钟小时内进入覆盖范围的本地 `WeatherEventForecast` 条目；研究“天气预报”（Weather Forecasting）后，该窗口扩展为 24 小时。

返回值

`List<WeatherEventForecast>`

类型 / 天气与天空 (TYPES / WEATHER & SKY)

WeatherSignalBoard

返回类型 : `weather_station.signal_board`

方法 (METHODS)

`.reveal(transmission)`

将一条传输发布到其声明的编号槽位中。不同事件会替换此站点当前的面板内容。对于同一事件，已发布的槽位保持不变。面板会检查结构和边界，但不会验证含义或校验和的有效性。

参数 (PARAMETERS)

名称 (Name)		类型 (Type)	描述 (Description)
<code>transmission</code>	<code>any</code>		一个 <code>SignalTransmission</code> 、类似 <code>vars(signal)</code> 的已保存字典，或具有相同字段的类实例。要求包含字符串类型的 <code>event_id</code> 、 <code>channel</code> 和 <code>data</code> ，以及位于信号槽位限制内的整数 <code>number</code> 和 <code>total</code> 。可选的字符串 <code>source_station_id</code> 在省略时默认为此站点。
返回值 (RETURNS)		结果字段 (RESULT FIELDS)	成功载荷 (SUCCESS PAYLOAD)
<code>ActionResult</code>		<code>.status</code> 、 <code>.message</code>	<code>None</code>

结果 (OUTCOMES)

状态 (Status)	类型 (Kind)	含义 (Meaning)
<code>"ok"</code>	成功 (success)	操作已完成。
<code>"no_power"</code>	暂时性 (transient)	组件没有可用电力。
<code>"duplicate"</code>	拒绝 (rejection)	此条目已经存在，未添加或更改任何内容。
<code>"invalid_type"</code>	拒绝 (rejection)	提供的值不具备所需的 API 类型和来源信息。

.reject(transmission)

将所提供事件的拒绝计数加一，但不打开槽位。不同事件会替换此站点当前的面板内容。

参数 (PARAMETERS)

名称 (Name)	类型 (Type)	描述 (Description)
transmission	any	一个 <code>SignalTransmission</code> 、类似 <code>vars(signal)</code> 的已保存字典，或具有与 <code>reveal()</code> 相同字段和验证规则的类实例。
返回值 (RETURNS)	结果字段 (RESULT FIELDS)	成功载荷 (SUCCESS PAYLOAD)
<code>ActionResult</code>	<code>.status</code> 、 <code>.message</code>	<code>None</code>

结果 (OUTCOMES)

状态 (Status)	类型 (Kind)	含义 (Meaning)
"ok"	成功 (success)	操作已完成。
"no_power"	暂时性 (transient)	组件没有可用电力。
"invalid_type"	拒绝 (rejection)	提供的值不具备所需的 API 类型和来源信息。

.resolve(event_id, info)

为一个事件发布最多 6 行带标签的数据。不同事件会替换此站点当前的面板内容。面板会显示所提供的值，但不会对其进行验证。

参数 (PARAMETERS)

名称 (Name)	类型 (Type)	描述 (Description)
event_id	string	这些行所描述的事件 ID。
info	any	由字符串键组成的标签与显示值字典。
返回值 (RETURNS)	结果字段 (RESULT FIELDS)	成功载荷 (SUCCESS PAYLOAD)
<code>ActionResult</code>	<code>.status</code> 、 <code>.message</code>	<code>None</code>

结果 (OUTCOMES)

状态 (Status)	类型 (Kind)	含义 (Meaning)
"ok"	成功 (success)	操作已完成。
"no_power"	暂时性 (transient)	组件没有可用电力。
"invalid_type"	拒绝 (rejection)	提供的值不具备所需的 API 类型和来源信息。

`.clear()`

清除面板、拒绝计数以及任何已发布的结论。

返回值 (RETURNS)	结果字段 (RESULT FIELDS)	成功载荷 (SUCCESS PAYLOAD)
ActionResult	.status、.message	None

结果 (OUTCOMES)

状态 (Status)	类型 (Kind)	含义 (Meaning)
"ok"	成功 (success)	操作已完成。
"no_power"	暂时性 (transient)	组件没有可用电力。

`.status()`

此站点当前面板发布的元数据。

返回值 (RETURNS)

WeatherSignalBoardStatus

类型 / 天气与天空 (TYPES / WEATHER & SKY)

WeatherSignalBoardStatus

返回类型 : `weather_station.signal_board.status()`

方法 (METHODS)

`.has_input`

此站点是否已发布 Signal Board 值。

返回值 (RETURNS)

`boolean`

`.published_at_gh`

发布时间戳；没有输入时为 `None`。

返回值 (RETURNS)

`Optional[number]`

`.freshness`

`"no_input"`、`"fresh"` 或 `"stale"`。发布内容在世界时钟经过 2 小时后会变为过期 (`stale`)。

返回值 (RETURNS)	可能的值 (POSSIBLE VALUES)
<code>string</code>	<code>"no_input"</code> 、 <code>"fresh"</code> 、 <code>"stale"</code>

`.event_id`

面板上显示的事件 ID；没有输入时为字符串。

返回值 (RETURNS)

`string`

类型 / 天气与天空 (TYPES / WEATHER & SKY)

WeatherStrike

返回类型：`weather_station.strikes()`

方法 (METHODS)

`.id`

稳定的观测雷击 ID。

返回值 (RETURNS)

`string`

`.event_id`

雷暴事件 ID。

返回值 (RETURNS)

`string`

`.observed_at_gh`

雷击发生时的世界时钟小时数。

返回值 (RETURNS)

`number`

`.energy_wh`

雷击携带的电能，单位为 Wh。

返回值 (RETURNS)

`number`

.caught

一个符合条件的 Lightning Rod 是否已将此次雷击存入电能库。

返回值 (RETURNS)

boolean

类型 / 天气与天空 (TYPES / WEATHER & SKY)

Zone

返回类型：`WeatherReport.coverage()` 和 `WeatherEventForecast.corridor()`

方法 (METHODS)

`.intersect(other)`

仅保留两个区域共有的几何范围。

参数 (PARAMETERS)

名称 (Name)	类型 (Type)	描述 (Description)
<code>other</code>	<code>any</code>	另一个 <code>Zone</code> 。

返回值 (RETURNS)

Zone

`.union(other)`

保留任一方覆盖的全部范围。

参数 (PARAMETERS)

名称 (Name)	类型 (Type)	描述 (Description)
<code>other</code>	<code>any</code>	另一个 <code>Zone</code> 。

返回值 (RETURNS)

Zone

`.subtract(other)`

从此区域中移除另一个区域的几何范围。

参数 (PARAMETERS)

名称 (Name)	类型 (Type)	描述 (Description)
<code>other</code>	<code>any</code>	另一个 <code>Zone</code> 。

返回值 (RETURNS)

Zone

.diff(other)

保留两个区域中仅存在于其中一方的几何范围。

参数 (PARAMETERS)

名称 (Name)	类型 (Type)	描述 (Description)
other	any	另一个 Zone。

返回值 (RETURNS)

Zone

.center()

以 [x, y] (单位:m) 返回质心; 区域为空时返回 **None**。它描述的是可见的风暴几何范围, 而不是其隐藏的后果。

返回值 (RETURNS)

Optional[[number, number]]

.area()

覆盖面积, 单位为 m²。

返回值 (RETURNS)

number

.contains(x, y)

当该点位于区域内部时返回 True。

参数 (PARAMETERS)

名称 (Name)	类型 (Type)	描述 (Description)
x	number	世界坐标 x, 单位为 m。
y	number	世界坐标 y, 单位为 m。

返回值 (RETURNS)

boolean

`.is_empty()`

当该区域不覆盖任何范围时返回 True。

返回值 (RETURNS)

`boolean`